

PyTorch 面试通关手册

从 Tensor 与 autograd , 到 torch.compile、分布式 , 与从零手
写模型

duo.an

2026-09-04

目录

1	前言	10
1.1	PyTorch 面试到底在考什么	10
1.2	全书结构	10
1.3	配套代码	11
1.4	三种读法	11
1.5	前置知识，与不讲什么	12
1.6	与仓库里另外三本书的关系	12
1.7	被问到不会的题，怎么答	12
2	Tensor : storage、stride、dtype、device	13
2.1	Tensor = Storage + 元数据	13
2.2	is_contiguous 的判定规则	14
2.3	view vs reshape vs permute vs transpose	14
2.4	expand vs repeat : stride=0 的广播视图	15
2.5	共享内存 vs 复制：一张清单	16
2.6	clone / detach / .data : 谁保留 autograd 连接	17
2.7	dtype : 位宽、动态范围与训练偏好	17
2.8	device 与 .to() : 什么时候真的异步	18
2.9	in-place 操作与 autograd	19
2.10	内存格式 : channels_last	20
2.11	面试考点	20
3	索引、广播与形状变换	22
3.1	广播规则	22
3.2	basic indexing vs advanced indexing	22
3.3	gather / scatter / index 系列	23
3.4	einsum 速成	24
3.5	形状变换 API 的选择	25
3.6	cat / stack / split / chunk	26
3.7	matmul 的广播规则	26
3.8	reduce : dim 与 keepdim	27
3.9	数值稳定 : softmax、logsumexp、log_softmax	28
3.10	面试考点	28
4	nn.Module 的一切	30
4.1	四张注册表与 <code>__setattr__</code>	30
4.2	register_buffer 与 persistent=False	31
4.3	容器 : ModuleList / ModuleDict / Sequential	31
4.4	state_dict 与 load_state_dict	32
4.5	train() 与 eval() : 只影响两类层	33
4.6	参数初始化	33
4.7	权重共享 (tied embedding)	34
4.8	hooks 全表	35
4.9	遍历参数 : named_parameters 与 weight decay 分组	36
4.10	冻结参数 : <code>requires_grad_(False)</code> vs 从 optimizer 排除	36

4.11	<code>to()</code> / <code>cuda()</code> 的不一致：module 原地改，tensor 返回新对象	37
4.12	面试考点	37
5	数据管线：Dataset / DataLoader / Sampler	39
5.1	两种 Dataset：map-style 与 iterable-style	39
5.2	DataLoader 的参数，逐个讲清	40
5.3	<code>num_workers</code> 的工作原理	41
5.4	<code>pin_memory</code> 为什么能加速	42
5.5	worker 的四类坑	43
5.6	随机性：多 worker 下的 RNG	43
5.7	<code>collate_fn</code> ：从样本列表到 batch	44
5.8	Sampler：决定看哪些样本、什么顺序	45
5.9	判断 dataloader 是不是瓶颈	47
5.10	padding 浪费：bucketing 与 packing	48
5.11	面试考点	49
6	训练循环的每一行	51
6.1	骨架：七行代码，每行都有考点	51
6.2	<code>zero_grad(set_to_none=True)</code> 为什么更好	52
6.3	梯度累积	52
6.4	混合精度 AMP	53
6.5	梯度裁剪	55
6.6	优化器	56
6.7	学习率调度	57
6.8	loss 的几个坑	59
6.9	checkpoint：存什么、怎么恢复	59
6.10	验证循环	60
6.11	常见训练 bug 清单	61
6.12	面试考点	62
7	Autograd：动态反向图的完整机制	64
7.1	反向图长什么样	64
7.2	<code>leaf</code> 、 <code>requires_grad</code> 、 <code>.grad</code> 三者的关系	65
7.3	<code>requires_grad</code> 的传播规则	66
7.4	backward 的执行：拓扑序与“累加”	66
7.5	<code>backward()</code> 的参数与 <code>torch.autograd.grad()</code>	67
7.6	<code>retain_graph</code> ：什么时候必须开，为什么默认关	68
7.7	<code>create_graph=True</code> 与二阶导	68
7.8	数学视角：链式法则与 VJP	70
7.9	<code>detach</code> / <code>no_grad</code> / <code>inference_mode</code> / <code>enable_grad</code>	70
7.10	<code>torch.autograd.Function</code> ：自定义前反向	71
7.11	用 <code>gradcheck</code> 验证自定义 backward	73
7.12	in-place 操作与 version counter	73
7.13	“梯度是 None” 排查清单	75
7.14	activation checkpointing 的 autograd 视角	75
7.15	hook：在反向路径上插手	76
7.16	面试考点	77
8	Dispatcher：一次 <code>a + b</code> 到底走了什么路	80

8.1	分层总览	80
8.2	Dispatcher 的核心： $op \times DispatchKey \rightarrow kernel$	80
8.3	TensorImpl / Storage / TensorOptions	82
8.4	TensorIterator：elementwise op 的通用引擎	83
8.5	类型提升（type promotion）	83
8.6	注册自定义 op：torch.ops 与 torch.library	84
8.7	CPU 与 CUDA kernel 的差异	85
8.8	后端开关：cuDNN benchmark 与 TF32	86
8.9	面试怎么讲：60 秒的架构答案	87
8.10	面试考点	87
9	显存：allocator、峰值与 OOM	89
9.1	显存都花在哪	89
9.2	激活显存为什么是大头	90
9.3	caching allocator：为什么 empty_cache() 通常没用	90
9.4	碎片化	91
9.5	排查工具	91
9.6	省显存的手段，按性价比排序	92
9.7	activation checkpointing 的取舍	93
9.8	OOM 排查流程	94
9.9	那些不是泄漏的“泄漏”	94
9.10	面试考点	95
10	CUDA 执行模型：异步、stream、同步点	97
10.1	kernel launch 是异步的	97
10.2	隐式同步点	98
10.3	stream：并行的执行队列	99
10.4	stream 与 allocator 的坑：record_stream	101
10.5	pinned memory 与 non_blocking=True	101
10.6	CUDA graph：把一串 launch 固化成一次	102
10.7	CPU-bound 还是 GPU-bound	103
10.8	多进程与 CUDA	103
10.9	面试考点	104
11	随机性、确定性与可复现	106
11.1	随机数从哪来	106
11.2	完整的复现清单	106
11.3	为什么设了 seed 结果还不一样	107
11.4	浮点不结合律：所有问题的根	108
11.5	use_deterministic_algorithms 与它的代价	109
11.6	TF32：同样的代码在 V100 和 A100 上结果不同	109
11.7	什么时候需要 bit-wise 确定性	110
11.8	数值调试：怎么比较两个结果	110
11.9	NaN / Inf 排查	111
11.10	面试考点	112
12	性能分析与优化清单	114
12.1	先测再改	114
12.2	正确计时	115

12.3	<code>torch.profiler</code>	115
12.4	怎么读 timeline	117
12.5	roofline 直觉：这个 op 值不值得 fuse	117
12.6	MFU：一个数说清训练效率	118
12.7	优化清单，按性价比排序	118
12.8	“优化了但没变快”	119
12.9	面试考点	120
13	TorchDynamo：从 Python 字节码到 FX Graph	122
13.1	四层流水线	122
13.2	Dynamo 怎么接管 Python 执行	122
13.3	guard：编译产物是特化的	123
13.4	graph break：图被切开	124
13.5	手动控制编译边界	127
13.6	为什么 Dynamo 成了，TorchScript 没成	127
13.7	面试考点	128
14	AOTAutograd：提前把反向图也编译出来	130
14.1	先算清楚“只编前向”值多少	130
14.2	怎么在 dispatcher 层 trace 出反向	130
14.3	functionalization：把副作用消掉	131
14.4	decomposition 与 PrimTorch	132
14.5	min-cut partitioner：显存和重算的取舍	132
14.6	与 activation checkpointing 的关系	134
14.7	反向不总能提前编出来：compiled autograd	134
14.8	怎么看 AOTAutograd 产出的图	134
14.9	面试考点	135
15	TorchInductor：codegen 与 fusion	137
15.1	流水线	137
15.2	为什么 fusion 是主要收益来源	137
15.3	fusion 的种类与阻碍	138
15.4	看真实生成的 Triton kernel	139
15.5	matmul 不靠 fusion	140
15.6	编译缓存	141
15.7	Inductor 和手写 Triton 的边界	142
15.8	面试考点	142
16	生产里用 torch.compile	145
16.1	参数逐个说清	145
16.2	reduce-overhead 就是 CUDA graph	146
16.3	动态 shape	146
16.4	编译时间	148
16.5	调试流程	148
16.6	数值差异不是 bug	149
16.7	常见坑清单	149
16.8	与其他特性组合	150
16.9	什么时候不该用	151
16.10	面试考点	152

17	导出与部署	154
17.1	三代方案	154
17.2	jit.trace vs jit.script	154
17.3	torch.export	155
17.4	AOTInductor	157
17.5	ONNX	157
17.6	推理侧的常规动作	158
17.7	state_dict vs 整模型保存	159
17.8	面试考点	159
18	torch.distributed 基础：通信原语与进程组	162
18.1	为什么要分布式：两道墙	162
18.2	集合通信原语的精确语义	163
18.3	Ring AllReduce：为什么与 N 无关	164
18.4	init_process_group 到底做了什么	165
18.5	torchrun：谁来设这些环境变量	166
18.6	ProcessGroup 与 sub-group	167
18.7	同步、异步，以及 collective 的调用契约	168
18.8	硬件拓扑决定并行策略怎么摆	169
18.9	NCCL 环境变量速查	170
18.10	通信量估算：一套通用方法	170
18.11	面试考点	171
19	DDP：原理、实现与调优	173
19.1	为什么 nn.DataParallel 被废弃	173
19.2	DDP 的数学基础	173
19.3	bucket + backward hook：本章核心	174
19.4	find_unused_parameters 的代价	176
19.5	no_sync()：梯度累积时别通信	177
19.6	DDP 与 BatchNorm	177
19.7	DDP + torch.compile	178
19.8	手写一个极简 DDP	178
19.9	DDP 常见坑清单	179
19.10	扩展性与大 batch	180
19.11	面试考点	181
20	ZeRO 与 FSDP：把训练状态切开	183
20.1	动机：DDP 冗余了什么	183
20.2	ZeRO 三级：切什么、省多少、多付多少	183
20.3	FSDP 的执行流程	184
20.4	FSDP 的关键配置	185
20.5	HYBRID_SHARD：把通信压回机内	187
20.6	FSDP1 vs FSDP2	187
20.7	checkpoint：分片状态怎么存	189
20.8	ZeRO-Offload / ZeRO-Infinity	189
20.9	FSDP 常见坑	189
20.10	DDP 还是 FSDP：决策清单	190
20.11	面试考点	190

21	张量并行与流水并行	193
21.1	什么时候必须超越 FSDP	193
21.2	Tensor Parallel : MLP 的 column-then-row	193
21.3	Sequence Parallel : 把 TP 没切到的那部分切掉	197
21.4	Pipeline Parallel : 按层切 stage	198
21.5	组合并行 : 3D / 4D 的维度排布	201
21.6	面试考点	202
22	DeviceMesh 与 DTensor	204
22.1	手写并行的痛点	204
22.2	DeviceMesh : 给 world 的每一维起名字	204
22.3	DTensor : local tensor + mesh + placements	205
22.4	placement 转换与 collective 的对应关系	205
22.5	用 DTensor 表达 TP	206
22.6	FSDP2 就是 mesh 上的 <code>Shard(0)</code>	208
22.7	手工构造与转换 DTensor	208
22.8	收益与代价	209
22.9	面试考点	210
23	分布式 debug、checkpoint 与容错	212
23.1	hang 的排查 : 最高频的分布式面试题	212
23.2	其他典型故障	214
23.3	环境变量生产模板	214
23.4	数值不一致的排查	215
23.5	Checkpoint	216
23.6	容错与弹性	218
23.7	面试考点	219
24	手写基础层 : 从零实现 torch.nn	221
24.1	Linear	221
24.2	Softmax	222
24.3	LayerNorm	223
24.4	RMSNorm	224
24.5	BatchNorm1d	225
24.6	Dropout	227
24.7	Embedding	228
24.8	Conv2d : im2col / unfold	229
24.9	MaxPool2d	231
24.10	汇总 : 九个层的对照表	232
24.11	面试考点	232
25	手写 autograd Function : backward 全靠自己推	234
25.1	标准模板与五条规则	234
25.2	MyReLU : 最简模板	235
25.3	MyGELU : 手推导数	236
25.4	Softmax 的 backward : 最高频的手推题	237
25.5	LayerNorm 的 backward : 难度最高的手推题	238
25.6	gradcheck : 怎么证明你推对了	240
25.7	Straight-Through Estimator : backward 可以不是真导数	242

25.8	GradientReversal : DANN 的核心	242
25.9	什么时候真的需要自定义 Function	243
25.10	面试考点	244
26	经典模型 : MLP / CNN / RNN / Seq2Seq	246
26.1	MLP : 送分题里的失分点	246
26.2	ResNet BasicBlock : ReLU 加在哪	247
26.3	LSTMCell : 4 个门与两套 bias	248
26.4	LSTM : 沿时间步展开, 以及 <code>h_n</code> 的形状陷阱	250
26.5	GRUCell : 重置门乘在哪	251
26.6	Bahdanau vs Luong : Transformer 之前的 attention	252
26.7	Seq2Seq : encoder + decoder + attention	254
26.8	RNN 家族对比	256
26.9	面试考点	256
27	Transformer 全家桶 : 从 attention 到 generate	258
27.1	Scaled dot-product attention	258
27.2	MultiHeadAttention : 那个静默出错的 reshape	260
27.3	MHA / MQA / GQA : 一条连续的谱	262
27.4	RoPE : 为什么它是相对位置编码	264
27.5	mask : causal、padding, 以及它们的组合	265
27.6	KV cache : 为什么需要, 怎么实现, 怎么验	266
27.7	TransformerBlock : pre-norm、RMSNorm、SwiGLU	268
27.8	MiniGPT 与参数量估算	270
27.9	generate : greedy / temperature / top-k / top-p	271
27.10	复杂度 : 什么时候 attention 才是瓶颈	272
27.11	面试考点	273
28	进阶模型 : ViT / CLIP / MoE / Diffusion / LoRA	276
28.1	ViT : 把图像变成 token 序列	276
28.2	CLIP : 对称对比损失	278
28.3	MoE : top-k gating 与负载均衡	280
28.4	Diffusion : DDPM 的前向闭式解与反向采样	283
28.5	LoRA : 低秩旁路	285
28.6	面试考点	288
29	训练技巧类手写题	290
29.1	Label Smoothing Cross Entropy	290
29.2	Focal Loss	291
29.3	Mixup 与 CutMix	292
29.4	ModelEMA	293
29.5	Warmup + Cosine Scheduler	294
29.6	梯度累积的等价性证明	296
29.7	手写优化器	297
29.8	手写 <code>clip_grad_norm_</code>	299
29.9	面试考点	300
30	快问快答题库	303
30.1	基础与 Tensor	303
30.2	autograd 与训练	305

30.3	显存与性能	306
30.4	torch.compile	308
30.5	分布式	309
30.6	手写题速答	311
30.7	开放题与故障排查	313
31	附录 A：常见报错速查	317
31.1	view size is not compatible with input tensor's size and stride	317
31.2	Expected all tensors to be on the same device	317
31.3	dtype 不匹配：must have the same dtype / should be the same	317
31.4	CUDA out of memory. Tried to allocate	318
31.5	a leaf Variable that requires grad is being used in an in-place operation	318
31.6	one of the variables needed for gradient computation has been modified by an inplace operation	319
31.7	Trying to backward through the graph a second time	319
31.8	element 0 of tensors does not require grad and does not have a grad_fn	320
31.9	.grad 是 None（不报错，但更难查）	320
31.10	CUDA error: device-side assert triggered	320
31.11	Expected more than 1 value per channel when training	321
31.12	Sizes of tensors must match except in dimension / stack expects each tensor to be equal size	321
31.13	Error(s) in loading state_dict: Missing key(s) / Unexpected key(s) ..	321
31.14	DataLoader worker (pid xxx) is killed by signal	322
31.15	Cannot re-initialize CUDA in forked subprocess	322
31.16	Address already in use	322
31.17	NCCL: unhandled system error 与 Watchdog caught collective operation timeout	323
31.18	torch._dynamo.exc.Unsupported	323
31.19	hit config.recompile_limit	324
31.20	loss 变成 NaN	324
31.21	报错关键字 → 去看哪一章	324
32	附录 B：自测清单与复习路线	326
32.1	自测清单	326
32.2	按岗位定制复习路线	328
32.3	时间预算	329
32.4	手写题清单	329
32.5	面试前一晚：一页速览	330
33	附录 C：PyTorch 2.x 版本差异	332
33.1	1.x → 2.0：唯一的分水岭	332
33.2	逐版本要点	332
33.3	那些“默认值变了”的坑	333
33.4	已废弃 / 不推荐清单	335
33.5	生态：各个 torch* 包是干什么的	335
33.6	面试怎么答版本问题	336

前言

PyTorch 面试到底在考什么

一场 PyTorch 面试基本按三层递进往下挖，每一层都可能是终点：

1. **API 层**：`view` 和 `reshape` 有什么区别？`zero_grad` 为什么要调？——答对了只能证明你写过代码。
2. **原理层**：`view` 为什么会失败？失败条件用 `stride` 怎么表达？`set_to_none=True` 为什么更省显存？——答对了说明你知道底下发生了什么。
3. **手写层**：白板上二十行写个 `LayerNorm`，写完解释 `eps` 为什么在 `sqrt` 里面。——答对了说明你不是背的。

市面上的材料要么是 API 教程（只有第一层），要么是八股题库（第二层的结论，没有推导过程，追问一句就穿帮）。这本书的写法是把三层焊在一起：每个知识点从“面试怎么问”出发，讲清机制，然后给一段真能跑的代码。

定位是高频题，不是冷门八股。每一章的题都是真会被问到的——`view` vs `reshape`、`autograd` 的 `None` 梯度、DDP 为什么分 bucket、`graph break` 是什么、手写 MHA。凡是“面试官几乎不会问”的边角 API，一律没写。

环境是 `torch 2.10.0+cu128 / CUDA 12.8 / A100-SXM4-80GB`。版本敏感的地方都标了版本，PyTorch 2.x 的版本差异单独放在附录 C。

全书结构

部分	章节	读完能答上什么
Part 1 基础	1-5	张量占多少内存、哪个 op 偷偷 copy 了；广播和 <code>einsum</code> ； <code>nn.Module</code> 的四张注册表和 <code>state_dict</code> ； <code>DataLoader</code> 的 <code>worker/pin_memory/collate_fn</code> ；训练循环七行里每一行的考点（AMP、梯度累积、裁剪、调度器）
Part 2 原理	6-11	反向图怎么建、 <code>.grad</code> 为什么是 <code>None</code> 、 <code>retain_graph</code> 什么时候真的需要；一次 <code>a + b</code> 走过 <code>dispatcher</code> 的哪几层；显存花在哪、 <code>caching allocator</code> 和碎片； <code>kernel launch</code> 的异步语义与隐式同步点；为什么设了 <code>seed</code> 结果还不一样；怎么用 <code>profiler</code> 定位瓶颈
Part 3 compile	12-16	<code>Dynamo</code> 怎么改 Python 字节码、 <code>guard</code> 和 <code>graph break</code> ； <code>AOTAutograd</code> 怎么把反向也编出来、 <code>min-cut partitioner</code> 的取舍； <code>Inductor</code> 的 <code>fusion</code> 收益从哪来；生产里 <code>mode</code> / 动态 <code>shape</code> / 编译时间怎么调； <code>torch.export</code> 与 <code>AOTInductor</code> 的部署路线
Part 4 分布式	17-22	集合通信原语的精确语义与通信量估算；DDP 的 <code>bucket + backward hook</code> ； <code>ZeRO</code> 三级省多少、 <code>FSDP</code> 的 <code>gather/reshard</code> 流程；TP 的 <code>column-then-row</code> 与 PP 的 <code>bubble</code> ； <code>DeviceMesh/DTensor</code> ； <code>hang</code> 怎么排查、分片 <code>checkpoint</code> 怎么存
Part 5 手写题	23-29	白板写 <code>Linear</code> / <code>LayerNorm</code> / <code>BatchNorm</code> / <code>conv2d</code> ；自定义 <code>autograd.Function</code> 与 <code>gradcheck</code> ； <code>LSTM</code> / <code>GRU</code> / <code>Bahdanau attention</code> ； <code>MHA</code> / <code>GQA</code> / <code>RoPE</code> / <code>KV cache</code> / <code>MiniGPT</code> ； <code>ViT</code> / <code>CLIP loss</code> / <code>MoE</code> /

		DDPM / LoRA ; label smoothing / mixup / EMA / warmup-cosine / 手写 AdamW ; 最后一章是限时题库
附录	A-C	A : 拿着报错原文查成因和修法 ; B : 五百多条自测项 + 按岗位的复习路线 + 面试前一晚速览 ; C : PyTorch 2.x 版本差异 , 答“你用的什么版本”这类题

Part 1-4 有依赖 (Part 3 假设你读过第 6 章的 autograd , Part 4 假设你读过第 5 章的训练循环); Part 5 是独立的 , 任何时候都能翻开练。

配套代码

Part 5 的每道题都有一份可运行的参考实现 + pytest 对齐测试 , 在 `python/pytorch/interview/` 。总共 6 个文件、约 6100 行、186 个测试。

文件	对应章	测试数	内容
<code>test_layers.py</code>	23	31	Linear / LayerNorm / RMSNorm / BatchNorm1d / Dropout / Embedding / Softmax / conv2d / MaxPool2d
<code>test_custom_autograd.py</code>	24	33	<code>autograd.Function</code> 三件套、STE、 <code>GradientReversal</code> 、 <code>gradcheck</code> / <code>gradgradcheck</code>
<code>test_classic_models.py</code>	25	31	MLP / ResNet BasicBlock / LSTMCell / GRUCell / Bahdanau / Luong / Seq2Seq
<code>test_transformer.py</code>	26	32	SDPA / MHA / GQA / RoPE / mask / KVCache / SwiGLU / MiniGPT / generate
<code>test_advanced_models.py</code>	27	26	ViT / CLIP loss / MoE 路由与 aux loss / DDPM / LoRA
<code>test_training_tricks.py</code>	28	33	label smoothing / focal / mixup / cutmix / EMA / warmup-cosine / 手写 SGD 与 AdamW / clip-grad-norm

跑法 (全部是 CPU 测试 , 秒级):

```
cd /path/to/playground-cuda
pytest python/pytorch/interview/ -q # 全跑, 186 passed
pytest python/pytorch/interview/test_transformer.py -q # 单文件
pytest python/pytorch/interview/ -q -k "rope or kv_cache" # 按关键字挑
```

这些测试的价值不在“能跑”，在验收标准：几乎每道题都把官方实现的权重拷进手写版本，用 `torch.testing.assert_close` 做逐元素比对。面试里写完代码接一句“我会把 `nn.LSTMCell` 的权重拷进来做数值对齐”，比任何解释都有说服力。

三种读法

(a) 一周突击。时间只够读一遍的话：第 1、5、6 章 (张量 + 训练循环 + autograd , 这三章的题占了基础面的一半), 第 8、18、19 章 (显存 + DDP + FSDP), 第 12、15 章 (Dynamo + compile 实践), Part 5 的第 23、26 章。然后直接做第 29 章的限时题库自测，错哪题回去补哪章。

(b) 系统学习。 顺序读，每章的代码自己敲一遍——特别是 Part 5，看懂和写出来是两回事。手写题写完就跑对应的 `pytest` 验证，不要停留在“我觉得对”。

(c) 查漏。 直接翻附录 B 的自测清单，逐条问自己“不看书能不能答出来”，勾不上的条目按后面标的章号回去读。这是复习到后期最有效率的用法。

前置知识，与不讲什么

假设你会 Python、写过 PyTorch 训练脚本、知道什么是反向传播和 Adam。

不讲：Python 语法基础；深度学习入门理论（梯度下降怎么来的、CNN 为什么有效）；具体业务模型的调参经验（数据配比、超参搜索）；也不讲 CUDA kernel 怎么写。

与仓库里另外三本书的关系

这本是横向的入口书，另外三本是纵向的深挖：

- `books/distributed-training/`：分布式训练实战。本书 Part 4 讲到“能在面试里把 DDP/FSDP/TP/PP 说清楚”为止；ZeRO 的完整推导、Ring Attention、DualPipe、FP8、MegaScale 级容错在那本。
- `books/cuda/`：CUDA kernel 优化。本书第 9、14 章讲到“知道 kernel launch 和 fusion 在干什么”为止；怎么手写一个 kernel、怎么用 `ncu` 调 occupancy 在那本。
- `books/moe/`：Sparse MoE 专题。本书第 27 章只写到面试常考的 top-k gating + load-balancing loss；grouped GEMM、all-to-all、capacity 设计在那本。

面到分布式或推理岗、且这本书的 Part 4 已经能顺畅讲下来的话，去读那两本。

被问到不会的题，怎么答

这是最被低估的一项能力。面试官问到你不会的东西，硬编是最差的选项——编错一个细节，后面所有回答的可信度一起打折。

正确的答法有三步：

1. 说清你知道的边界。“FSDP2 我只用过 `fully_shard` 的默认配置，`reshard_after_forward` 的取舍我没实测过。”明确划线，面试官立刻知道该往哪边追问，反而显得你对自己的知识有把握。
2. 把它归约到你会的东西。“但它本质上是 ZeRO-3，参数在 forward 前 AllGather、用完 reshard，通信量是 DDP 的 1.5 倍——按这个模型推，`reshard_after_forward=False` 应该是拿显存换掉一次 AllGather。”这一步展示的是推理能力，比背对答案更值钱。
3. 说你会怎么验证。“我会跑一个 profile 看 AllGather 有没有消失，再用 `torch.cuda.max_memory_allocated` 对比峰值。”能说出验证手段，说明你平时是靠测量而不是靠猜。

三步说完，一道不会的题也能拿到分。整本书的面试考点框都是按这个思路写的：给结论，也给结论怎么来的，以及怎么自己验证。

翻页开始。

Tensor : storage、stride、dtype、device

面试问 Tensor ,问的不是 `torch.zeros` 怎么调用 ,而是“这个张量占多少内存”、“哪个操作偷偷 copy 了”、“为什么改了一个视图另一个也变了”。这一章把 Tensor 拆成两半：一块一维连续的 Storage , 加一组描述“怎么走这块内存”的元数据。看懂这个模型 , `view` / `reshape` / `expand` / `contiguous` 这一串题就变成同一道题的不同问法。autograd 的图结构见第 6 章 , `caching allocator` 与显存峰值见第 8 章。

Tensor = Storage + 元数据

一个 `torch.Tensor` 只有两部分：

- **Storage** : 一块一维、连续、无类型的裸内存 (CPU 上的 RAM 或 GPU 上的 HBM) , 多个 Tensor 可以指向同一个 Storage。
- 元数据 : `shape`、`stride`、`storage_offset`、`dtype`、`device`、`layout`、`requires_grad`。这些都是 CPU 侧的小整数 , 改它们不碰 GPU。

逻辑下标 $(i_0, i_1, \dots, i_{n-1})$ 映射到 storage 线性下标的公式只有一条：

$$\text{idx} = \text{offset} + \sum_{k=0}^{n-1} i_k \cdot \text{stride}_k$$

`stride[k]` 的含义是“第 k 维下标 +1 , storage 下标前进多少个元素” (单位是元素 , 不是字节) 。

```
x = torch.arange(12).reshape(3, 4)
x.shape                # torch.Size([3, 4])
x.stride()             # (4, 1)
x.storage_offset()     # 0
x.untyped_storage().nbytes() # 96 = 12 个 int64 × 8 字节 (注意返回的是字节数)
x.data_ptr()           # storage 基址 + offset*itemsz
```

x = torch.arange(12).reshape(3, 4)

logical view	0	1	2	3
	4	5	6	7
	8	9	10	11

storage (1-D, row-major)

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

shape=(3, 4) stride=(4, 1) offset=0

图 1 contiguous 张量 : `stride=(4,1)` , 逐行扫 storage , 逻辑顺序 = 物理顺序。

`reshape(3,4)` 之所以零成本 , 是因为它只写了一组新的 `shape` / `stride` ——storage 一个字节都没动。

x.T — 同一 **storage** , **stride** 交换

logical view	0	4	8
	1	5	9
	2	6	10
	3	7	11

storage (1-D, row-major)

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

shape=(4, 3) stride=(1, 4) offset=0

图 2 `transpose` 只交换 `stride`。`stride=(1,4)` 表示“往右走一格跳 4 个元素”，逻辑上的一行在 `storage` 里是跳跃的 → non-contiguous。

Key insight. `view`、`transpose`、`permute`、`slice`、`expand`、`squeeze`、`unsqueeze` 全都是纯元数据操作，时间复杂度与张量大小无关。真正花钱的只有“必须重排内存”的那几个：`contiguous`、`clone`、`repeat`、以及 fallback 到 `copy` 的 `reshape`。

is_contiguous 的判定规则

“contiguous”（准确说是 `torch.contiguous_format`）的定义：从最后一维往前推，`stride` 必须等于其后所有维度 `shape` 的乘积。

$$\text{stride}_k = \prod_{j>k} \text{shape}_j$$

外加两条实现细节，面试爱追问：

- **size** 为 1 的维度被跳过：它的 `stride` 取任何值都不影响遍历，所以不参与判定。
`torch.randn(4, 1, 5).transpose(1, 2)` 依然 `is_contiguous() == True`。
- 空张量恒为 **contiguous**：`numel() == 0` 直接返回 `True`。

```
x = torch.arange(12).reshape(3, 4)
x.is_contiguous()           # True    stride=(4,1) == (4,1)
x.T.is_contiguous()         # False   stride=(1,4) != (3,1)
x[:, 1:3].is_contiguous()   # False   stride=(4,1)，但行内只取 2 个
x[1:3, :].is_contiguous()   # True    切最外维不破坏连续性
torch.randn(4, 1, 5).transpose(1, 2).is_contiguous() # True (size-1 维被忽略)
```

规律记这一句：切最外层维度、或转置 **size** 为 1 的维度，不破连续性；其它 **stride** 重排基本都破。

view vs reshape vs permute vs transpose

操作	做什么	可能 copy	要求
<code>view(...)</code>	改 <code>shape/stride</code>	否	新 <code>shape</code> 必须能用纯 <code>stride</code> 表达，否则报错
<code>reshape(...)</code>	先试 <code>view</code> ，失败则 <code>contiguous().view()</code>	是	无

<code>transpose(d0, d1)</code>	交换两维的 shape 与 stride	否	无
<code>permute(*dims)</code>	任意重排维度	否	必须给出全部维度
<code>movedim(src, dst)</code>	把某维搬到某位置	否	比 <code>permute</code> 好读
<code>contiguous()</code>	按当前逻辑顺序重排内存	是（已连续则 no-op）	无
<code>flatten(s, e)</code>	合并连续维度	是（必要时）	无

`x.T.view(...)` 报错的原因不是“PyTorch 保守”，而是 stride 系统表达不出来。`x.T` 的 shape 是 (4,3)、stride 是 (1,4)；要把它看成 (12,)，需要一个走位序列 0,4,8,1,5,9,...，这不是任何单一 stride 能生成的等差数列。所以 `view` 只能拒绝：

```
x = torch.arange(12).reshape(3, 4)
x.T.view(12)      # RuntimeError: view size is not compatible with input tensor's
                  # size and stride ... Use .reshape(...) instead.
x.T.reshape(12)   # OK — 内部做了 contiguous(), 发生了一次真实 copy
x.T.contiguous().view(12)  # 等价, 但 copy 是你写出来的, 读代码的人看得见
```

Warning. `reshape` 会不会 copy 取决于输入的 **stride**，也就是取决于运行时数据。这意味着显存峰值不确定：一个 12 GB 的 activation 走到 `reshape` 那行，可能白涨 12 GB。在显存吃紧的路径上（比如 attention 里 merge head 之后）请显式写 `.contiguous().view(...)`，让 copy 出现在代码里而不是藏在语义里。

一个高频追问：`permute` 之后想 **flatten** 怎么办？用 `reshape`，或者显式 `contiguous().view(...)`。分布式代码里（NCCL 要求连续 buffer）一律显式，见第 17 章。

expand vs repeat : stride=0 的广播视图

`expand` 把 size 为 1 的维度“拉长”，实现方式是该维 stride 设成 0——下标怎么变，storage 位置都不动。所以它是零 copy 的视图。`repeat` 是真的把数据复制 n 份。

`torch.arange(4).expand(3, 4) — stride[0] = 0`

logical view

0	1	2	3
0	1	2	3
0	1	2	3

storage (1-D, row-major)

0	1	2	3
---	---	---	---

shape=(3, 4) stride=(0, 1) offset=0

图 3 `expand` 出来的 3 行都映射到 storage 的同一 4 个格子。storage 只有 4 个元素，逻辑上却有 12 个位置。

```
r = torch.arange(4)          # shape (4,)
e = r.expand(3, 4)          # shape (3,4) stride (0,1)
e.untyped_storage().nbytes() # 32 — 还是那 4 个 int64, storage 没变大
p = r.repeat(3, 1)          # shape (3,4), storage 变成 12 个元素, 真 copy
```

```
e.expand(3, 5)          # RuntimeError: 只能 expand size 为 1 的维度
r.unsqueeze(0).expand(3, 4) # 想 expand 先 unsqueeze 造出 size-1 维
```

Warning. 往 `expand` 的结果里写数据是错的。多个逻辑位置指向同一块内存,写入语义未定义:

```
e = torch.zeros(1, 4).expand(3, 4)
e.add_(1)      # RuntimeError: unsupported operation: more than one element of
the
               # written-to tensor refers to a single memory location.
```

PyTorch 的 `TensorIterator` 会做 `internal-overlap` 检查并报错,但不是所有 **op** 都走这条检查(尤其是自定义 CUDA kernel 和某些 `index_put_` 路径)。规则: `expand` 的结果只读;要写就先 `.clone()` 或改用 `repeat`。

选择原则:只是要参与广播运算(`a.expand(...)+b`)就用 `expand`,省一次 HBM 往返;需要一块独立可写的、或者要交给 NCCL / 外部 kernel 的连续内存,才用 `repeat`。实际上大多数时候连 `expand` 都不用写——算子的广播会自动做等价的事。

共享内存 vs 复制:一张清单

共享 storage (视图)	新分配 storage (copy)
<code>view</code> / <code>reshape</code> (能走 <code>view</code> 时)	<code>clone()</code>
<code>transpose</code> / <code>permute</code> / <code>movedim</code> / <code>.T</code>	<code>contiguous()</code> (原本不连续时)
basic indexing: <code>x[1]</code> 、 <code>x[:, 2:5]</code> 、 <code>x[..., ::2]</code>	advanced indexing: <code>x[[0,2]]</code> 、 <code>x[mask]</code>
<code>squeeze</code> / <code>unsqueeze</code> / <code>expand</code> / <code>broadcast_to</code>	<code>repeat</code> / <code>repeat_interleave</code> / <code>tile</code>
<code>detach()</code> / <code>requires_grad_()</code>	<code>to(dtype)</code> 、 <code>float()</code> 、 <code>half()</code> (<code>dtype</code> 变了必 copy)
<code>split</code> / <code>chunk</code> / <code>unbind</code> / <code>narrow</code>	<code>cat</code> / <code>stack</code> / <code>pad</code>
<code>t.numpy()</code> (CPU) <code>torch.from_numpy(a)</code>	<code>.cpu()</code> / <code>.cuda()</code> (跨 device 必 copy)
<code>as_strided</code> (危险,可越界)	<code>torch.tensor(data)</code> (永远 copy)

`.to()` 的语义是“需要才 **copy**”: `x.to(torch.float32)` 在 `x` 已是 `fp32` 时返回 `x` 本身(同一对象), `dtype` 或 `device` 不同时才分配新内存。想强制拿到副本用 `x.to(dtype, copy=True)`。

NumPy 侧记三条: `torch.from_numpy(a)` 共享内存(改一个另一个变); `t.numpy()` 也共享,但 `t` 必须在 CPU 且 `requires_grad=False`; `torch.tensor(a)` 永远 **copy**, `torch.as_tensor(a)` 尽量共享。

Note. 验证是否共享, `data_ptr()` 不够——切片会有 `offset`。可靠做法是比较 `x.untyped_storage().data_ptr() == y.untyped_storage().data_ptr()`。

clone / detach / .data : 谁保留 autograd 连接

这四个写法看着差不多，autograd 行为完全不同：

写法	新内存	连着 autograd	用途
<code>x.clone()</code>	是	是 (CloneBackward0 , 梯度回传给 <code>x</code>)	要副本又要可微
<code>x.detach()</code>	否 (共享 storage)	否	只想切断梯度，不想花内存
<code>x.detach().clone()</code>	是	否	存快照 / 传给日志或 EMA , 最安全
<code>x.data</code>	否	否，且不更新 version counter	不要用

```
x = torch.randn(3, requires_grad=True)
y = (x * 2)

y.clone().sum().backward()    # x.grad 有值 — clone 是可微的
y.detach().sum().backward()    # RuntimeError: does not require grad
```

Warning. 不要用 `.data`。它绕过了 autograd 的 version counter，导致 in-place 修改不被察觉，backward 拿着已被改坏的值算梯度，不报错、结果错：

```
x = torch.randn(4, requires_grad=True)
y = x.sigmoid()                # backward 需要 y 本身
y.data.mul_(0)                 # 静默改坏 y, version counter 不变
y.sum().backward()             # 不报错, x.grad 全是 0, 纯错
```

换成 `y.detach().mul_(0)` 就会正常抛 “one of the variables needed for gradient computation has been modified”。推荐写法：切梯度用 `detach()`，要存副本用 `detach().clone()`，改参数用 `with torch.no_grad():`。

顺序上 `detach().clone()` 比 `clone().detach()` 好一点点：前者 clone 的是一个不需要梯度的张量，不会在图上挂一个立刻被丢弃的 CloneBackward0 节点。功能上两者等价。

dtype : 位宽、动态范围与训练偏好

dtype	位宽	指数位	尾数位	max	eps
float32	32	8	23	3.4e38	1.19e-07
TF32 (计算模式)	32 存 / 19 算	8	10	3.4e38	9.77e-04
float16	16	5	10	65504	9.77e-04
bfloat16	16	8	7	3.39e38	7.81e-03

float8_e4m3fn	8	4	3	448	0.125
float8_e5m2	8	5	2	57344	0.25

关键对比就一句：**bf16** 和 **fp32** 的指数位都是 8 位，动态范围一样；**fp16** 只有 5 位指数。所以 fp16 的上限是 65504、最小正规数约 $6.1e-5$ ，训练中很容易出现两类问题：大 logit / 大梯度上溢成 inf，小梯度下溢成 0。这就是 fp16 训练必须配 GradScaler（把 loss 放大再缩回来）的原因。bf16 的范围与 fp32 相同，通常不需要 scaler；代价是尾数只剩 7 位，eps 差了约 8 倍，所以累加类操作（reduction、optimizer state、norm 统计量）仍然要在 fp32 里做。混精细节见第 5 章。

TF32 不是存储 dtype，而是 Ampere+ 上 tensor core 的一种 fp32 matmul 计算模式：输入按 10 位尾数截断，累加仍是 fp32。它只影响 matmul / conv 的精度，不改变张量的 dtype 和显存占用。

```
torch.set_float32_matmul_precision("high")    # 允许 matmul 走 TF32 (推荐写法)
torch.backends.cuda.matmul.allow_tf32 = True  # 旧开关，等价；默认 False
torch.backends.cudnn.allow_tf32 = True        # conv 的开关，默认已是 True
```

查一个 dtype 的边界不要背，用 torch.finfo（浮点）/ torch.iinfo（整数）：

```
fi = torch.finfo(torch.bfloat16)
fi.max, fi.min, fi.eps, fi.tiny, fi.bits    # tiny = 最小正规数
torch.finfo(torch.float16).max              # 65504.0
torch.iinfo(torch.int8).max                 # 127

# 实用场景：给 attention mask 填一个"安全的负无穷"
scores.masked_fill_(mask, torch.finfo(scores.dtype).min)
```

Note. 上面用 finfo(dtype).min 而不是 float("-inf")：-inf 在整行全被 mask 时会让 softmax 产出 nan（分子分母都是 0），用有限的最小值则得到均匀分布，训练不会炸。这是个常考的小陷阱。

类型提升遵循 NumPy 风格的规则：fp16 + fp32 → fp32，int + float → float，但 Python 标量不参与提升——torch.ones(3, dtype=torch.float16) * 2.0 仍是 fp16。

device 与 .to()：什么时候真的异步

.to(device) 跨 device 一定 copy。真正的考点是它同步不同步。

```
x = torch.randn(1024, 1024, pin_memory=True)    # pinned (page-locked) host 内存
g = x.to("cuda", non_blocking=True)              # 真异步：立刻返回，DMA 在后台跑

y = torch.randn(1024, 1024)                     # 普通 pageable 内存
g2 = y.to("cuda", non_blocking=True)             # 参数被忽略，实际是同步 copy
```

原因：DMA 引擎只能安全地读物理地址固定的内存。pageable 内存可能被 OS 换页，所以 CUDA 必须先拷到一块内部 staging buffer 再传，这一步是同步的。DataLoader(pin_memory=True) 就是为了让 batch 落在 pinned 内存里，配上 non_blocking=True 才能把 H2D 藏进上一个 step 的计算里。注意 pin_memory() 自身的分配是昂贵且同步的，不要在训练循环里现场 pin。

Warning. D2H 方向的 `non_blocking=True` 是个陷阱。
`gpu_t.to("cpu", non_blocking=True)` 会立刻返回，但数据还没到；此时读 CPU 张量拿到的是垃圾。必须自己插同步点：

```
cpu_t = gpu_t.to("cpu", non_blocking=True)
torch.cuda.current_stream().synchronize() # 或 event.synchronize()
print(cpu_t)                               # 现在才是对的
```

H2D 方向不用担心：后续 kernel 排在同一个 stream 上，顺序天然被保证。

会隐式同步（阻塞 CPU 直到 GPU 追上）的常见操作，写训练循环时要盯：`.item()`、`.tolist()`、`float(t)` / `bool(t)`、`print(t)`、`.cpu()` / `.numpy()`、`torch.nonzero`、以及所有依赖 `nonzero` 的布尔掩码索引 `x[mask]`。所以 `loss.item()` 每 step 都调是在拖 pipeline——攒到 log 间隔再调，或者把 loss 累加在 GPU 上。

```
torch.cuda.set_sync_debug_mode("warn") # 或 "error"：任何隐式同步都告警
```

in-place 操作与 autograd

in-place op（带下划线后缀：`add_`、`relu_`、`clamp_`，以及 `x[0] = 1`、`x += 1`）省一次内存分配，但和 autograd 有两处硬冲突：

1. 叶子张量不能原地改：`w.add_(1)` 在 `w.requires_grad=True` 且是 leaf 时报 “a leaf Variable that requires grad is being used in an in-place operation”。这正是 optimizer 必须在 `torch.no_grad()` 下更新参数的原因。
2. **backward** 需要的中间值不能被改：每个张量有 version counter，in-place 会 +1。backward 时发现存下来的版本号变了就报 “one of the variables needed for gradient computation has been modified by an inplace operation”。

```
x = torch.randn(4, requires_grad=True)
y = x.sigmoid()
y.mul_(2)           # sigmoid 的 backward 需要 y 本身 → 这里就埋雷
y.sum().backward()  # RuntimeError: ... modified by an inplace operation

# 而 relu 的 backward 只需要 mask，所以 inplace 是安全的：
h = torch.relu_(x * 2) # OK，nn.ReLU(inplace=True) 同理
```

规则：**backward** 用到 **output** 的 **op** (`sigmoid`、`tanh`、`exp`、`softmax`) 后面不能 **in-place**；只用到 **input** 或 **mask** 的 **op** (`relu`、`conv`、`matmul`) 通常安全。不确定就别 in-place，省的那点内存不值得调这个 bug。

`torch.no_grad()` 下 in-place 一律放行，因为不建图、不存中间值。手写 optimizer 的模板：

```
with torch.no_grad():
    for p in model.parameters():
        p.add_(p.grad, alpha=-lr) # 原地更新，不产生新张量
```

内存格式：channels_last

memory_format 是 stride 的另一种排布约定，不改 shape。NCHW 张量转 channels_last 后 shape 仍是 (N,C,H,W)，但 stride 变成 (H*W*C, 1, W*C, C)——物理上按 NHWC 存。

```
x = torch.randn(8, 64, 56, 56)
xl = x.to(memory_format=torch.channels_last)
xl.shape                    # 不变 (8, 64, 56, 56)
xl.stride()                 # (200704, 1, 3584, 64)
xl.is_contiguous()          # False (按 NCHW 判定)
xl.is_contiguous(memory_format=torch.channels_last) # True
model = model.to(memory_format=torch.channels_last) # 权重也要转
```

好处是 cuDNN / cutlass 的卷积 tensor core kernel 原生吃 NHWC，用 NCHW 时框架要插隐式转置。CNN + AMP 场景值得试。Transformer 用不到——那边全是 matmul，没有 channel 维。

面试考点

面试考点. Q1：view 和 reshape 的区别？什么时候必须用 reshape？

A：view 只改元数据、绝不 copy，要求目标 shape 能被单一 stride 表达，否则报错。reshape 先试 view，失败就 contiguous().view()，可能悄悄 copy。permute / transpose / 跨维切片之后想合并维度，就只能用 reshape（或显式 contiguous().view()）。显存敏感的路径推荐显式写 contiguous()，让 copy 可见。

面试考点. Q2：is_contiguous() 到底怎么判定？为什么 randn(4,1,5).transpose(1,2) 还是 contiguous？

A：判定是“从最后一维往前，stride[k] 等于其后所有 shape 的乘积”。实现里 size 为 1 的维度被跳过——它的 stride 取什么值都不影响遍历顺序，所以转置一个 size-1 维度不破坏连续性。空张量也恒为 contiguous。

面试考点. Q3：expand 和 repeat 的区别？为什么不能往 expand 的结果里写？

A：expand 把 size-1 维的 stride 设成 0，零 copy 的视图；repeat 真复制数据、显存翻 n 倍。stride=0 意味着多个逻辑位置映射到同一块内存，写入语义未定义，TensorIterator 的 internal-overlap 检查会报错。要写先 clone()，或直接用 repeat。

面试考点. Q4：clone() / detach() / .data 分别做什么？推荐怎么写？

A：clone() 新内存且可微，梯度经 CloneBackward0 回传。detach() 共享内存但切断图。.data 也切断图，但不更新 version counter，会让 autograd 察觉不到 in-place 修改，产生静默的错误梯度。推荐：切梯度 detach()，存快照 detach().clone()，改参数 with torch.no_grad():。

面试考点. Q5：为什么大模型训练偏好 bf16 而不是 fp16？

A：bf16 的指数位是 8 位，动态范围和 fp32 一致；fp16 只有 5 位指数，max 是 65504、最小正规数约 $6.1e-5$ ，大 logit 会上溢成 inf、小梯度会下溢成 0，必须配 GradScaler。bf16 通

常不需要 scaler，代价是尾数只有 7 位、`eps` 约 $7.8e-3$ ，所以 reduction、optimizer state、norm 统计量仍要放在 fp32。

面试考点. **Q6**：`non_blocking=True` 什么时候真的异步？

A：H2D 方向必须源张量在 `pinned` (page-locked) 内存里；`pageable` 内存会退化成本地 copy，因为 DMA 需要物理地址固定。所以要配 `DataLoader(pin_memory=True)`。D2H 方向 `non_blocking=True` 会立刻返回但数据未就绪，读之前必须自己 `synchronize()`，否则拿到垃圾数据。

面试考点. **Q7**：训练循环里哪些操作会隐式同步 GPU？怎么查？

A：`.item()`、`.cpu()`、`.numpy()`、`.tolist()`、`print(tensor)`、`bool(tensor)`，以及 `torch.nonzero` 和依赖它的布尔掩码索引 `x[mask]`。每 step 调 `loss.item()` 就是在打断 CPU 的 launch pipeline。用 `torch.cuda.set_sync_debug_mode("warn")` 可以把所有隐式同步点打出来。

面试考点. **Q8**：什么样的 in-place 操作会破 autograd？为什么 `nn.ReLU(inplace=True)` 却没问题？

A：backward 需要 **output** 的 op 不能被 in-place 覆盖 (sigmoid、tanh、exp、softmax)，version counter 一变 backward 就报错。relu 的 backward 只需要“哪些位置大于 0”这个 mask，可以从改写后的 output 直接推出来，所以 in-place 安全。另外叶子参数在 `requires_grad=True` 时不允许 in-place，必须包在 `torch.no_grad()` 里——这就是 optimizer 的写法。

索引、广播与形状变换

这一章是“shape 对了但值不对”这类 bug 的集中营。广播规则、fancy indexing 会不会 copy、gather 的 index 该是什么形状、chunk 为什么少给了一块——这些题面试官爱问，因为它们直接对应线上事故。第 1 章讲了 stride 决定“哪些操作是视图”，这一章讲的是“这些操作的语义到底是什么”。

广播规则

只有三条，从最右边的维度开始对齐往左比：

1. 两边长度相等 → 合法；
2. 有一边是 1 → 合法，把它 expand 到另一边（stride 设 0，见第 1 章）；
3. 一边维度不存在（更短）→ 合法，在左侧补 1。

其它情况报“The size of tensor a (X) must match the size of tensor b (Y) at non-singleton dimension”。

```
(8, 1, 6, 1) 与 (7, 1, 5) -> (8, 7, 6, 5) # OK
( 256, 256, 3) 与 (3,) -> (256, 256, 3) # OK
( 256, 256, 3) 与 (256, 256) -> 报错 (3 vs 256)
```

危险的地方在于广播成功也可能是错的。三个真实踩坑：

```
# 坑 1: 最经典的 loss bug
pred = torch.randn(64, 1) # 模型忘了 squeeze
target = torch.randn(64) # label 是 1-D
(pred - target).shape # (64, 64) — 不报错！
((pred - target) ** 2).mean() # 算的是所有 pair 的距离，loss 假性偏大且几乎不下降

# 坑 2: 忘了 keepdim
x = torch.randn(32, 32) # B 恰好等于 S，最阴的情况
x / x.sum(-1) # (32,32)/(32,): 第 j *列* 被第 j 行的和除，错
x / x.sum(-1, keepdim=True) # 正确

# 坑 3: mask 维度对齐
scores = torch.randn(4, 8, 16, 16) # (B, H, S, S)
mask = torch.zeros(4, 16).bool() # (B, S) padding mask
scores.masked_fill(mask, -1e4) # 报错: 4 vs 16
scores.masked_fill(mask[:, None, None, :], -1e4) # 正确: (B,1,1,S)
```

Key insight. 防广播 bug 的三个习惯：reduce 一律带 `keepdim=True`；构造 mask 时显式写出所有 `None` 让 rank 一眼可数；关键位置 `assert pred.shape == target.shape`。
`torch.broadcast_shapes` 可以不分配内存先算 shape。

basic indexing vs advanced indexing

basic indexing（整数、slice、None、Ellipsis）返回视图，因为结果能用一组新的（shape, stride, offset）表达。**advanced indexing**（index 是 list / 整型 tensor / bool tensor）返回副本，因为取出来的位置一般不是等差的。

```
x = torch.arange(20).reshape(4, 5)
x[1:3, 2:4]      # view: shape (2,2) stride (5,1) offset 7
x[..., None, 2]   # view
x[[0, 2]]         # copy: advanced
x[x > 10]         # copy: bool mask (内部走 nonzero, 在 GPU 上还会隐式同步)
x[[0, 2]] = 0     # 有效! 赋值走 index_put_, 直接写原张量
```

Warning. advanced indexing 返回副本, 所以 `x[[0,2]] += 1` 生效 (Python 会翻译成 `__setitem__`), 但 `x[[0,2]].add_(1)` 不生效——改的是临时副本, 且不会报错。bool mask 同理: `x[mask].zero_()` 是无效操作。要就地改用 `x[mask] = 0` 或 `x.masked_fill_(mask, 0)`。

多个 index tensor 的语义 (高频题)

多个 index tensor 之间是先互相广播、再逐元素配对取值, 不是“先取行再取列”。

```
x = torch.arange(35).reshape(5, 7)
i = torch.tensor([0, 1, 2])
j = torch.tensor([3, 4, 5])

x[i, j]          # shape (3,): 取 x[0,3], x[1,4], x[2,5] — 逐元素配对
x[i][:, j]       # shape (3,3): 先选 3 行, 再从这 3 行里选 3 列 — 笛卡尔积
x[i[:, None], j] # shape (3,3): 等价于上面, i 变成 (3,1) 与 j 的 (3,) 广播成 (3,3)
```

要“对角线取值”用 `x[i, j]`, 要“子矩阵”用 `x[i[:, None], j]`。这是 gather 与 index_select 语义差别的根源, 也是最容易在白板上写错的一行。

Note. 混用 basic 与 advanced 时, 结果维度的位置有个规则: 如果所有 advanced index 是相邻的, 广播出来的维度就留在原位; 如果被 slice 隔开了, 广播维度会被搬到最前面。例: `x` shape (2,3,4,5), `x[:, idx, idx2, :]` 得到 (2, B, 5); 而 `x[idx, :, idx2]` 得到 (B, 3) —— B 跑到最前。不确定就 print shape, 不要靠记忆。

gather / scatter / index 系列

op	语义	典型用途
<code>gather(dim, index)</code>	<code>index</code> 与 <code>input</code> 同 ndim, 输出 <code>shape = index.shape</code> 。逐位置沿 <code>dim</code> 取	取每个 token 的 label logit、beam search 收集
<code>scatter_(dim, index, src)</code>	<code>gather</code> 的逆: <code>self[..., index[...], ...] = src[...]</code>	one-hot、 label smoothing
<code>scatter_add_(dim, index, src)</code>	同上但累加, 重复 index 全部相加	稀疏梯度归约、MoE 的 combine
<code>index_select(dim, index)</code>	<code>index</code> 是 1-D, 沿 <code>dim</code> 整片选, 返回 copy	取子词表、按 rank 选 shard

<code>index_add_(dim, index, src)</code>	沿 <code>dim</code> 整片累加	embedding 反向、MoE unpermute
<code>masked_fill_(mask, v)</code>	<code>mask</code> 为 True 的位置填标量 <code>v</code> (<code>mask</code> 可广播)	attention causal / padding mask
<code>where(cond, a, b)</code>	三方广播后逐元素选	数值保护、条件分支向量化

`gather` 的 `index` 形状是最容易记错的：`index` 的 `ndim` 必须等于 `input` 的 `ndim`，除了 `dim` 那一维，其它维度的 `size` 必须能与 `input` 对上；输出 `shape` 就等于 `index.shape`。

```
logits = torch.randn(4, 10)          # (B, C)
labels = torch.tensor([3, 1, 9, 0])   # (B,)

# 取每个样本正确类别的 logit
picked = logits.gather(1, labels[:, None]).squeeze(1)  # (4,)
# 等价的 advanced indexing 写法, 更好读
picked = logits[torch.arange(4), labels]              # (4,)

# one-hot: scatter_ 在零矩阵上打点
onehot = torch.zeros(4, 10).scatter_(1, labels[:, None], 1.0)
# 更简单的等价写法
onehot = torch.nn.functional.one_hot(labels, num_classes=10).float()
```

Warning. `torch.where` 会同时求值两个分支，所以它不能用来避开 `nan`：

```
x = torch.tensor([0.0, 1.0], requires_grad=True)
y = torch.where(x > 0, torch.log(x), torch.zeros_like(x))
y.sum().backward()
x.grad      # tensor([nan, 1.]) — log(0) 的 -inf 在反向里污染了梯度
```

正确做法是先把输入夹到安全区间：`torch.log(x.clamp_min(1e-12))`，或者用 `masked_fill` 把非法位置换成合法值再算。

`scatter_` 遇到重复 `index` 时“哪个写赢”是未定义的（GPU 上不确定）；需要确定性就用 `scatter_add_` 或 `index_add_`。注意 `index_add_` / `scatter_add_` 的浮点累加顺序在 GPU 上也是不确定的，逐 bit 复现要看第 10 章。

einsum 速成

`einsum` 用下标字符串描述“哪些维度相乘、哪些维度求和”：出现在输入但不出现在输出的下标被 **sum** 掉，其它下标逐元素配对。

```
A = torch.randn(3, 4); B = torch.randn(4, 5)
torch.einsum("ik,kj->ij", A, B)          # matmul: k 被消掉

Q = torch.randn(2, 8, 128, 64)           # (B, H, S, D)
K = torch.randn(2, 8, 128, 64)
V = torch.randn(2, 8, 128, 64)
S = torch.einsum("bhqd,bhkd->bhqk", Q, K) # attention score, 省掉 transpose
O = torch.einsum("bhqk,bhkd->bhqk", S.softmax(-1), V)
```



```

torch.einsum("i,j->ij", torch.randn(3), torch.randn(4)) # outer product (3,4)
torch.einsum("bij,bjk->bik", X, Y) # bmm
torch.einsum("ii->i", M) # 取对角线
torch.einsum("bhd->bh", T) # 沿最后一维 sum
torch.einsum("...ij->...ji", T) # 转置, 支持 ellipsis

```

什么时候用: 下标一多 (4-5 维带 batch 和 head), `einsum` 比一串 `permute` + `reshape` + `bmm` 好读得多, 而且不会写错维度顺序。什么时候不用: 热点路径。`einsum` 内部会把式子降解成 `permute` + `reshape` + `bmm`, 其中 `reshape` 可能触发 `copy`; 它也拿不到 flash attention 这类融合 kernel。attention 请直接调 `F.scaled_dot_product_attention`, 别自己 `einsum`。

Note. 三个以上操作数时收缩顺序会显著影响 FLOPs, PyTorch 通过 `opt_einsum` 自动挑 (`torch.backends.opt_einsum.enabled`, 默认开)。两个操作数时没有选择空间。

形状变换 API 的选择



图 4 多头 attention 的形状流水线。只有最后一步真的搬内存, 前面全是 stride 改写。把 `reshape` 写成 `contiguous().view(...)` 能让这次 copy 显式可见。

API	什么时候用
<code>view(shape)</code>	确定输入连续、想让不连续时报错而不是静默 copy
<code>reshape(shape)</code>	不确定连续性、可以接受 copy
<code>flatten(start, end)</code>	合并一段相邻维度, 比手算乘积安全
<code>unflatten(dim, sizes)</code>	拆一个维度, (B,S,H) -> (B,S,N,D) 的首选
<code>squeeze(dim)</code>	去掉指定的 size-1 维。一定带 dim

<code>unsqueeze(dim)</code>	插一个 size-1 维；等价于索引里的 <code>None</code>
<code>movedim(src, dst)</code>	只想搬一两个维度，比 <code>permute</code> 少写一堆下标
<code>permute(*dims)</code>	完全重排，必须列出所有维度

-1 的推断规则：最多一个 -1，其余维度的乘积必须整除 `numel()`，否则报 “shape is invalid for input of size N”。`view(-1)` 是“拉平成 1-D”。注意 `numel() == 0` 时 -1 无法推断，会直接报错。

Warning. 不带参数的 `squeeze()` 是 **batch size = 1** 时的经典事故源。

```
x = torch.randn(1, 1, 768)      # batch=1 的推理请求
x.squeeze().shape                # (768,) — batch 维也被吃了，后面全错
x.squeeze(1).shape              # (1, 768) — 正确
```

训练时 `batch=32` 一切正常，上线遇到单条请求就炸。永远写 `squeeze(dim)`。torch 2.0+ 支持一次给多个：`squeeze((1, 2))`。

cat / stack / split / chunk

`cat` 沿已有维度拼接，`ndim` 不变，除 `dim` 外其它维度必须一致。`stack` 造一个新维度，所有输入 `shape` 必须完全相同。

```
a, b = torch.randn(2, 3), torch.randn(2, 3)
torch.cat([a, b], dim=0).shape    # (4, 3)
torch.stack([a, b], dim=0).shape  # (2, 2, 3)
torch.stack(xs, 0)                # == torch.cat([x[None] for x in xs], 0)
```

反向的两个 API 差别更值得记：

```
t = torch.arange(10)
[c.shape[0] for c in t.split(3)]  # [3, 3, 3, 1] — 每块固定 3，最后一块剩多少给多少
[c.shape[0] for c in t.chunk(4)]  # [3, 3, 3, 1] — 要 4 块，ceil(10/4)=3
t.split([2, 5, 3])               # 显式给每块大小，最精确的写法
```

Warning. `chunk(n)` 可能返回少于 `n` 块。它先算 `ceil(numel/n)` 作为块大小，再按这个大小切：

```
len(torch.arange(9).chunk(4))      # 3 — ceil(9/4)=3，切成 3+3+3，第 4 块不存在
len(torch.arange(9).tensor_split(4)) # 4 — 3+2+2+2，保证块数
```

在按 `rank` 切数据、按 GPU 数切 `batch` 的代码里，`chunk` 会让某些 `rank` 拿不到数据然后 `collective` 挂死。要“保证 `n` 块”用 `tensor_split(n)`；要“精确指定大小”用 `split([...])`。

`split` / `chunk` / `unbind` 返回的都是视图。但沿 `stride` 较大的维度切出来的片不连续，交给 NCCL 之前要 `.contiguous()`（第 17 章）。

matmul 的广播规则

`torch.matmul`（也就是 `@`）按输入的 `ndim` 分情况处理：

输入	输出	行为
<code>(n) @ (n)</code>	<code>()</code>	点积，返回 0-D 标量
<code>(m,n) @ (n,p)</code>	<code>(m,p)</code>	普通矩阵乘 = <code>mm</code>
<code>(m,n) @ (n)</code>	<code>(m)</code>	矩阵-向量 = <code>mv</code>
<code>(n) @ (n,p)</code>	<code>(p)</code>	左边补 1 变 <code>(1,n)</code> ，算完再去掉
<code>(B,m,n) @ (B,n,p)</code>	<code>(B,m,p)</code>	batch 矩阵乘 = <code>bmm</code>
<code>(B,H,m,n) @ (n,p)</code>	<code>(B,H,m,p)</code>	最后两维做矩阵乘，前面的 batch 维广播
<code>(1,H,m,n) @ (B,1,n,p)</code>	<code>(B,H,m,p)</code>	batch 维按标准广播规则对齐

核心一句：最后两维做矩阵乘，前面所有维度按标准广播规则对齐。

`mm` / `bmm` / `mv` 是不带广播、不带 `ndim` 推断的严格版本：`mm` 只吃 2-D，`bmm` 只吃 3-D 且两边 `batch` 维必须相等。写库代码时用它们更好——形状写错会立刻报错，而不是被广播规则“救活”成一个错的 `shape`。

Note. `matmul` 广播 `batch` 维时可能需要物化被广播的一侧（把 `stride=0` 展开成真实内存），`batch` 维很大时这是隐藏的显存开销，`(1,H,m,n) @ (B,1,n,p)` 这种双向广播尤其要留意。另外 `nn.Linear` 内部走 `addmm`，把 `bias + A @ B` 融成一个 `kernel`。

reduce : dim 与 keepdim

`sum` / `mean` / `max` / `norm` / `argmax` 这类 op 的三个约定：

- `dim` 不给 → 对全部元素 `reduce`，返回 0-D 标量。
- `dim` 给了 → 该维消失（`keepdim=False`，默认），或变成 1（`keepdim=True`）。
- `dim` 可以是 `tuple`：`x.sum(dim=(1, 2))`。但 `max` / `min` 只接受单个 `dim`，且返回 `(values, indices)` 命名元组——直接当张量用会报错。

```
x = torch.randn(4, 8, 16)
x.sum().shape           # ()
x.sum(dim=1).shape      # (4, 16)
x.sum(dim=1, keepdim=True).shape  # (4, 1, 16)
x.sum(dim=(1, 2)).shape # (4,)
v, i = x.max(dim=-1)    # 两个返回值
x.amax(dim=-1).shape    # 只要值、支持多 dim 时用 amax/amin
x.mean()                # 整型张量会报错，先 .float()
```

`keepdim=True` 能救广播 bug，因为它让结果的 `rank` 与输入一致，后续广播必然按你期望的维度对齐：

```
x = torch.randn(32, 32)
x - x.mean(-1, keepdim=True)  # 逐行去均值：正确
x - x.mean(-1)                # (32,32) - (32,) 按最后一维对齐：逐列去均值，错
```

所以凡是 **reduce** 的结果还要参与与原张量的运算，就带 `keepdim=True`。`norm` / `softmax` / `attention` 里到处都是这个模式。

数值稳定：softmax、logsumexp、log_softmax

朴素 softmax 会溢出：x_i = 1000 时 e^{x_i} 在 fp32 里就是 inf，inf/inf 得 nan。标准做法是先减去最大值——因为分子分母同乘一个常数不改变结果：

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} = \frac{e^{x_i-m}}{\sum_j e^{x_j-m}}, \quad m = \max_k x_k$$

减完之后所有指数的自变量都 ≤ 0 ， $e^{x_i-m} \in (0, 1]$ ，永不上溢；分母至少有一项等于 1，永不为 0。下溢只会让某些项变成 0，不影响结果的正确性。

logsumexp 用同一个技巧，把这个模式单独封成一个算子：

$$\text{LSE}(x) = \log \sum_i e^{x_i} = m + \log \sum_i e^{x_i-m}$$

于是 log-softmax 只是一次减法：

$$\log \text{softmax}(x)_i = x_i - \text{LSE}(x)$$

为什么 **log_softmax(x)** 比 **log(softmax(x))** 稳定：后者要先把概率算出来，当某个 x_i 远小于最大值时 softmax 的结果下溢成 0，log(0) 得 $-\infty$ ，梯度变 nan。前者直接算 $x_i - \text{LSE}(x)$ ，中间从不出现“接近 0 的概率”，再小的 logit 也只得到一个很负的有限值，而且少一次舍入。

```
x = torch.tensor([0.0, -1000.0])
torch.log(torch.softmax(x, -1))      # tensor([0., -inf])
torch.log_softmax(x, -1)             # tensor([ 0., -1000.]) 正确
torch.logsumexp(x, -1)               # tensor(0.)
```

所以损失函数一律用 logits 版本：F.cross_entropy(logits, labels) 内部是 log_softmax + nll_loss，F.binary_cross_entropy_with_logits 用了等价的 $\log(1+\exp(-|x|))$ 变形。不要自己先 softmax / sigmoid 再喂给 nll_loss / bce。

Key insight. 混精下这条更要紧。bf16 只有 7 位尾数，手写 exp / log 组合会迅速丢精度。PyTorch 的 softmax / log_softmax / cross_entropy 在 half 输入上会内部升到 fp32 累加再降回来，这是你自己写 exp(x)/exp(x).sum() 拿不到的。

面试考点

面试考点. Q1：广播规则是什么？举一个“广播成功但结果是错的”的例子。

A：从最右维度对齐往左比，每个维度要么相等、要么其中一个为 1、要么不存在（补 1）。经典错例：pred 是 (N,1)、target 是 (N,)，pred - target 广播成 (N,N)，算出的是所有 pair 的差，loss 不报错但根本不收敛。防御手段是 reduce 带 keepdim=True 加上关键位置 assert pred.shape == target.shape。

面试考点. Q2：x[i, j] 和 x[i][:, j] 有什么区别？

A: `x[i, j]` 把 `i` 和 `j` 先互相广播再逐元素配对取值, `i`、`j` 都是长 3 的 1-D 时结果是 (3,) , 取的是 `x[0,3]`, `x[1,4]`, `x[2,5]`。 `x[i][:, j]` 是先选 3 行再选 3 列, 结果 (3,3) 的笛卡尔积。想用一次索引拿到子矩阵, 写 `x[i[:, None], j]`。

面试考点. Q3: 哪些索引返回视图、哪些返回副本? 有什么实际影响?

A: basic indexing (整数、slice、None、Ellipsis) 返回视图, 因为结果能用新的 (shape, stride, offset) 表达。advanced indexing (整型/bool tensor、list) 返回副本。影响是 `x[mask].zero_()` 或 `x[[0,2]].add_(1)` 改的是临时副本、静默无效, 必须写成 `x[mask] = 0` (走 `index_put_`) 或 `x.masked_fill_(mask, 0)`。另外 bool mask 索引在 GPU 上内部走 `nonzero`, 会隐式同步。

面试考点. Q4: `gather` 的 `index` 形状有什么要求? 怎么取每个样本正确类别的 `logit`?

A: `index` 的 `ndim` 必须等于 `input` 的 `ndim`, 输出 `shape` 就等于 `index.shape`。 `logits` 是 (B,C)、`labels` 是 (B,) 时要写 `logits.gather(1, labels[:, None]).squeeze(1)`; 等价且更好读的写法是 `logits[torch.arange(B), labels]`。

面试考点. Q5: `scatter_` 和 `scatter_add_` 的区别?

A: `scatter_` 是赋值, 重复 `index` 时哪个写赢在 GPU 上是未定义的; `scatter_add_` 是累加, 重复 `index` 全部相加, 语义确定 (但浮点累加顺序仍不保证逐 bit 复现)。one-hot 用 `scatter_`, 把稀疏更新聚合回稠密张量 (embedding 反向、MoE combine) 用 `scatter_add_` 或 `index_add_`。

面试考点. Q6: `chunk(4)` 一定返回 4 块吗?

A: 不一定。`chunk(n)` 先算 `ceil(numel/n)` 当块大小再切, `arange(9).chunk(4)` 只返回 3 块 (3+3+3)。按 `rank` 切数据时这会让某些 `rank` 空手然后 `collective` 挂死。要保证块数用 `tensor_split(n)`, 要精确控制大小用 `split([...])`。另外 `chunk` / `split` 沿 `stride` 大的维度切出来的片是非连续的, 交给 NCCL 前要 `contiguous()`。

面试考点. Q7: `matmul`、`@`、`bmm`、`mm` 该用哪个?

A: `@` 就是 `matmul`: 最后两维做矩阵乘, 前面的 `batch` 维按广播规则对齐, 还会根据 `ndim` 自动退化成点积 / `mv`。`mm` 只吃 2-D, `bmm` 只吃 3-D 且 `batch` 维必须严格相等。写库代码优先用 `mm` / `bmm`, 因为形状写错会立刻报错, 不会被广播“救活”成一个错的 `shape`。

面试考点. Q8: 为什么 `log_softmax` 比 `log(softmax(x))` 稳定? `cross_entropy` 该喂什么?

A: `softmax` 先算出概率, `logit` 很小的位置会下溢成 0, `log(0)` 得 $-\infty$, 梯度变 `nan`。`log_softmax` 直接算 $x_i - \text{LSE}(x)$, 其中 $\text{LSE}(x) = m + \log \sum e^{x_i - m}$, 中间不出现接近 0 的概率, 且少一次舍入。所以 `F.cross_entropy` 必须喂 **logits**, 它内部就是 `log_softmax` + `nll_loss`; 自己先 `softmax` 再取 `log` 是错的写法。

nn.Module 的一切

`nn.Module` 是 PyTorch 里唯一有“状态”的抽象，也是面试最容易问出深浅的地方。表面看它就是 `__init__` 加 `forward`，实际上是四张注册表加一套 `__setattr__` 魔法，所有诡异行为（参数没进 optimizer、`load_state_dict` 报 missing key、`eval()` 后指标突变）都能从这四张表解释。训练循环与 optimizer 的配合见第 5 章，hook 在分布式里的用法见第 18 章。

四张注册表与 `__setattr__`

`nn.Module.__init__` 建立的核心状态是四个有序字典：

注册表	装什么	进 <code>state_dict</code>
<code>_parameters</code>	<code>nn.Parameter</code> ，要被 optimizer 更新的张量	是
<code>_buffers</code>	<code>register_buffer</code> 注册的张量，不需要梯度	<code>persistent=True</code> 才进
<code>_modules</code>	子 <code>nn.Module</code>	递归展开
hook 字典（ <code>_forward_hooks</code> 等）	各类回调	否

`__setattr__` 被重写过：`self.x = v` 时先看 `v` 的类型，`nn.Parameter` 进 `_parameters`，`nn.Module` 进 `_modules`，普通 tensor 落到 `__dict__`（什么都不进）。四张表决定三件事：谁被 `parameters()` 遍历到（进 optimizer）、谁被 `state_dict()` 存下来、谁被 `.to(device)` 搬走。

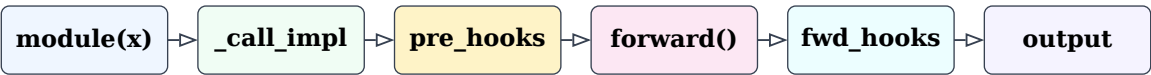


图 5 `module(x)` 而不是 `module.forward(x)` 的原因：`__call__` 走 `_call_impl`，`forward_pre_hooks` 与 `forward_hooks` 只在这条路上触发。直接调 `forward()` 会绕过所有 hook，也绕过 DDP / FSDP 的 unshard 逻辑。

`nn.Parameter` 为什么要包一层

`nn.Parameter` 是 `Tensor` 子类，只加两件事：默认 `requires_grad=True`，以及作为类型标记让 `__setattr__` 认出它。没有它就得靠“需不需要梯度”来猜，而这个判断在冻结参数时立刻失效——被冻结的参数依然要进 checkpoint、依然要被 `.cuda()` 搬走。类型标记与梯度开关必须解耦。

```
class M(nn.Module):
    def __init__(self):
        super().__init__()
        self.w = nn.Parameter(torch.randn(4, 4)) # 必须先调，否则四张表不存在 # -> _parameters
        self.register_buffer("mu", torch.zeros(4)) # -> _buffers
        self.fc = nn.Linear(4, 4) # -> _modules
        self.scale = torch.ones(4) # -> __dict__，孤儿！

m = M().cuda()
m.w.device, m.mu.device # cuda:0, cuda:0
m.scale.device # cpu — .cuda() 没搬它，forward 里一用就 device mismatch
```

register_buffer 与 persistent=False

buffer 是“属于模型状态、但不参与梯度更新”的张量。它跟着 `.to(device)` 走、跟着 `state_dict` 存，但不出现在 `parameters()` 里，所以 optimizer 看不见它。

```
self.register_buffer("running_mean", torch.zeros(dim))      # 存进 ckpt
self.register_buffer("causal_mask", mask, persistent=False) # 不存进 ckpt
self.register_buffer("cos_cached", cos, persistent=False)   # RoPE 缓存
```

`persistent=False` 用于能从超参重算出来的东西：causal mask、RoPE 的 sin/cos 表、位置编码常量。好处是省 checkpoint 体积，且改 `max_seq_len` 时不会因形状不匹配而 `load_state_dict` 失败。

为什么 BN 的 `running_mean` / `running_var` 是 **buffer** 而不是 **parameter**：它们是对训练数据分布的统计量、靠 EMA 累积，不是梯度学出来的（BN 里靠梯度学的是 `weight` / `bias`，那两个才是 `Parameter`）；但推理完全依赖它们，必须存进 checkpoint。“要持久化 + 不要梯度”正好是 `buffer` 的定义。BN 还有第三个 `buffer` `num_batches_tracked`，用于 `momentum=None` 时的累积平均。

容器：ModuleList / ModuleDict / Sequential

容器	语义	有 forward
<code>nn.Sequential</code>	按顺序调用，自带 <code>forward</code> ，只能单输入单输出	有
<code>nn.ModuleList</code>	像 <code>list</code> 一样索引 / 遍历， <code>forward</code> 要自己写	无
<code>nn.ModuleDict</code>	像 <code>dict</code> 一样按 <code>key</code> 取，顺序 = 插入顺序	无
<code>nn.ParameterList</code> / <code>ParameterDict</code>	同上，但装 <code>nn.Parameter</code>	无

```
# 有分支 / 残差 / 需要 layer index -> ModuleList, forward 自己写
self.layers = nn.ModuleList([Block(dim) for _ in range(n_layers)])
# 纯串行无分支 -> Sequential, 省一段 forward
self.mlp = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(), nn.Linear(4 * d, d))
# 按名字取 -> ModuleDict
self.heads = nn.ModuleDict({"cls": nn.Linear(d, 10), "reg": nn.Linear(d, 1)})
```

Warning. 用 `python list / dict` 装 `module`，参数不会被注册。这是最经典的静默 bug：

```
class Bad(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = [nn.Linear(4, 4) for _ in range(3)]  # 普通 list!

m = Bad()
len(list(m.parameters()))  # 0 — optimizer 拿到空参数列表
m.state_dict().keys()      # 空 — checkpoint 存不下这些层
m.cuda()                   # 这些 Linear 还在 CPU 上
```


症状是“loss 不动”或 device mismatch，而不是报错。换成 `nn.ModuleList([...])` 即可，同理 `self.w = [p1, p2]` 要换 `nn.ParameterList`。自查一句：
`assert sum(p.numel() for p in m.parameters()) > 0。`

state_dict 与 load_state_dict

`state_dict()` 返回一个 `OrderedDict`，key 是点分路径，value 是 `detach()` 后的张量，与模型参数共享同一块 **storage**。所以它不是快照：

```
sd = model.state_dict()
optimizer.step()           # 参数原地更新
sd["fc.weight"]            # 也跟着变了！
```

要真快照（EMA、best-model 缓存）必须显式 clone：
`{k: v.detach().clone() for k, v in model.state_dict().items()}`。想拿到还挂在 autograd 图上的原始 Parameter 用 `state_dict(keep_vars=True)`。

`load_state_dict()` 反过来是原地 **copy**：内部在 `torch.no_grad()` 下做 `param.copy_(input_param)`，不重新绑定对象。所以 optimizer 里持有的 Parameter 引用在 load 之后依然有效，不需要重建 optimizer。代价是 shape 必须对上，否则报“size mismatch for ...: copying a param with shape ...”。

```
missing, unexpected = model.load_state_dict(sd, strict=False)
# missing: 模型里有、checkpoint 里没有的 key (新加的层、persistent=False 的 buffer)
# unexpected: checkpoint 里有、模型里没有的 key (删掉的层、多余的 module. 前缀)
assert not unexpected, f"checkpoint 里有模型不认识的 key: {unexpected}"
print(f"未加载 (应当只有新 head): {missing}")
```

`strict=False` 的正当用途只有两类：把预训练 backbone 加载到带新 head 的模型；跳过 `persistent=False` 的 buffer。用完必须检查 **missing / unexpected**——否则一个拼错的前缀会让整个 backbone 静默保持随机初始化，训练“能跑但收敛很差”。

DDP 的 module. 前缀

`DistributedDataParallel` 把原模型挂在 `.module` 上，所以它 `state_dict()` 的每个 key 都多一层 `module.` 前缀。存的时候剥掉，加载时就不用管：

```
torch.save(model.module.state_dict(), path)           # 推荐：存裸模型，与并行方式解耦

# 已经存成带前缀的了，加载时剥掉：
from torch.nn.modules.utils import consume_prefix_in_state_dict_if_present
sd = torch.load(path, map_location="cpu", weights_only=True)
consume_prefix_in_state_dict_if_present(sd, "module.")
model.load_state_dict(sd)
```

Note. torch 2.4 起 `torch.load` 的 `weights_only` 默认是 `True`（只反序列化张量，不执行 pickle 里的任意代码）。加载旧 checkpoint 遇到 `UnpicklingError` 就是这个原因。分布式 checkpoint 见第 22 章。

train() 与 eval() : 只影响两类层

`model.train()` / `model.eval()` 只做一件事：递归地把每个子 module 的 `self.training` 置成 `True` / `False`。行为改变发生在那些读了这个标志的层里：

层	train() vs eval() 的差别
Dropout / Dropout1d/2d/3d / AlphaDropout	train 时按 <code>p</code> 置零并放大 $\frac{1}{1-p}$ ；eval 时恒等映射
BatchNorm1d/2d/3d	train 用当前 batch 统计量并更新 running stats；eval 用 running stats
InstanceNorm (<code>track_running_stats=True</code>)	同 BN
LayerNorm / GroupNorm / RMSNorm	没有区别，永远用当前样本统计量

所以 `eval()` 之后指标突变，八成是 BN 的 running stats 没喂够（batch 太小、momentum 太大），或者数据分布与训练时不一致。

`eval()` 与 `torch.no_grad()` 是两件不同的事

- `eval()` 改的是层的计算行为，不影响建图，梯度照算、显存照吃。
- `no_grad()` 改的是是否建 **autograd** 图，完全不影响 Dropout / BN 的行为。

推理两个都要（`model.eval()` 加 `with torch.inference_mode():`）。也有只要一个的场景：BN 的 warmup 校准需要 `train()` + `no_grad()`——要更新 running stats，但不要梯度。

Warning. 只写 `torch.no_grad()` 不写 `model.eval()`，是验证集指标偏低最常见的原因：Dropout 仍在随机置零，BN 仍在用 batch 统计量并且污染 **running stats**。反过来只写 `eval()` 不写 `no_grad()`，会白建一整张 autograd 图、显存翻倍甚至 OOM，但结果是对的。

`inference_mode()` 比 `no_grad()` 更快（还免掉 version counter 和 view 追踪），代价是里面产生的张量不能在外面参与 autograd。RL rollout 这种要把结果拿回来继续训练的场景只能用 `no_grad()`。

参数初始化

`nn.Linear` 的默认初始化是 `kaiming_uniform_(weight, a=math.sqrt(5))`。代入 kaiming uniform 的公式，`a=sqrt(5)` 让 $\text{gain} = \sqrt{\frac{2}{1+5}} = \sqrt{\frac{1}{3}}$ 、边界 $= \text{gain} \cdot \sqrt{\frac{3}{\text{fan_in}}} = \sqrt{\frac{1}{\text{fan_in}}}$ ，也就是 $U\left(-\frac{1}{\sqrt{\text{fan_in}}}, \frac{1}{\sqrt{\text{fan_in}}}\right)$ 。bias 用同样的边界。

Note. `a=sqrt(5)` 是对齐旧版 Torch 的历史遗留，不是“为 ReLU 调好的 Kaiming”。要按 ReLU 推荐值初始化得自己写 `kaiming_normal_(w, nonlinearity="relu")`。

Xavier (Glorot) 与 Kaiming (He) 的选择标准是激活函数是否关于 0 对称：

方法	方差	适用激活
Xavier / Glorot	$\frac{2}{\text{fan_in} + \text{fan_out}}$	tanh、sigmoid、恒等——正负对称，前后向方差都想守住
Kaiming / He	$\frac{2}{\text{fan_in}}$ (<code>fan_in</code> 模式)	ReLU、LeakyReLU——砍掉一半输出，方差要补 2 倍回来

Kaiming 的 2 倍就是“ReLU 让一半神经元输出 0、输出方差减半”的补偿。transformer 常用的 GELU / SiLU 更接近 ReLU 族，一般按 Kaiming 或直接用固定小 std (GPT-2 系用 `normal_(std=0.02)`)

常用 `nn.init`: `xavier_uniform_` / `xavier_normal_` / `kaiming_uniform_` / `kaiming_normal_` / `normal_` / `zeros_` / `ones_` / `trunc_normal_` / `orthogonal_`，全部 `in-place`。`apply()` 会后序递归遍历所有子 module (先子后父) 并逐个调用给定函数，是自定义初始化的标准姿势：

```
def _init_weights(self, module):
    if isinstance(module, (nn.Linear, nn.Embedding)):
        nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if getattr(module, "bias", None) is not None:
            nn.init.zeros_(module.bias)
    elif isinstance(module, nn.LayerNorm):
        nn.init.ones_(module.weight)
        nn.init.zeros_(module.bias)

# 在 __init__ 末尾:
self.apply(self._init_weights)
# 残差分支的输出投影额外缩小，防止深层激活方差随层数线性增长 (GPT-2 的做法)
for name, p in self.named_parameters():
    if name.endswith("proj.weight"):
        nn.init.normal_(p, std=0.02 / math.sqrt(2 * cfg.n_layer))
```

权重共享 (tied embedding)

把输入 embedding 与输出投影绑成同一个 `Parameter`，省一份 `vocab * dim` 的参数，同时是一种正则。写法就是直接赋值：

```
class LM(nn.Module):
    def __init__(self, vocab, dim):
        super().__init__()
        self.emb = nn.Embedding(vocab, dim)
        self.lm_head = nn.Linear(dim, vocab, bias=False)
        self.lm_head.weight = self.emb.weight  # 同一个 Parameter 对象

m = LM(100, 8)
m.lm_head.weight is m.emb.weight  # True
len(list(m.parameters()))  # 1 — named_parameters 去重
sorted(m.state_dict().keys())  # ['emb.weight', 'lm_head.weight']
— 不去重
```

关键的不对称：`named_parameters()` / `parameters()` 默认去重 (`remove_duplicate=True`)，`state_dict()` 不去重。前者保证 optimizer 不会对同一个参数更新两次；后者保证 checkpoint 自描述、能被没有 tying 的模型加载 (两个 key 指向同一块内存，`torch.save` 正确处理引用共享，磁盘上不存两份)。反向时两处用法的梯度会自动累加到同一个 `.grad`，这正是 tying 想要的语义。

Warning. 绑定必须在构造时用赋值完成。写成 `self.lm_head.weight.data = self.emb.weight.data` 只是让两个 `Parameter` 对象共享

storage: `parameters()` 会返回两个, `optimizer` 更新两次, `momentum` 各存一份, 行为与 `True` 不同且难查。

hooks 全表

hook	触发时机与签名	真实用途
<code>register_forward_pre_hook</code>	<code>forward</code> 之前, <code>fn(mod, args)</code> ; 返回值替换输入	FSDP 在这里 all-gather 出完整参数
<code>register_forward_hook</code>	<code>forward</code> 之后, <code>fn(mod, args, out)</code> ; 返回值替换输出	抓中间激活做特征提取、量化时统计 activation 分布
<code>register_full_backward_pre_hook</code>	该 module 的 <code>grad_output</code> 到手时, <code>fn(mod, grad_out)</code>	反向前把参数从 CPU 换回 GPU (activation offload)
<code>register_full_backward_hook</code>	<code>grad_input</code> 算完后, <code>fn(mod, grad_in, grad_out)</code>	逐层梯度范数监控、定位哪一层先出 nan
<code>Tensor.register_hook</code>	该张量的梯度算好时, <code>fn(grad)</code> ; 返回值替换梯度	DDP 绑在参数上触发 bucket ready、梯度裁剪 / 反向 mask
<code>register_state_dict_pre_hook</code>	<code>state_dict()</code> 组装之前	FSDP 在存之前把分片参数聚合成完整张量
<code>register_load_state_dict_post_hook</code>	<code>load_state_dict()</code> 完成之后	加载后重新切分参数、校验 <code>missing_keys</code>

```
grad_norms = {}
def watch(name):
    def hook(mod, grad_in, grad_out):
        grad_norms[name] = grad_out[0].norm().item()
    return hook

handles = [m.register_full_backward_hook(watch(n))
            for n, m in model.named_modules() if isinstance(m, nn.Linear)]
for h in handles:
    h.remove()  # 用完必须 remove, 否则闭包持有张量引用 -> 显存泄漏
```

Warning. 三个坑: (1) hook 必须 `remove()`, 闭包里捕获的张量会阻止显存回收; (2) `register_backward_hook` (无 `full_`) 是废弃接口, 在多输入 / in-place 时给错的 `grad_input`, 一律用 `full_backward` 版; (3) 直接调 `model.forward(x)` 绕过 `__call__`, 所有 hook 都不触发——这也是 DDP / FSDP 包过的模型必须用 `model(x)` 调用的原因。

`state_dict` 系列 hook 的公开接口在 `torch 2.3-2.5` 之间调整过 (早期只有带下划线的私有版本), 用之前先确认版本上的签名。全局 hook 在 `torch.nn.modules.module.register_module_forward_hook`。

遍历参数：named_parameters 与 weight decay 分组

```
for name, p in model.named_parameters(): ... # 'layers.0.attn.qkv.weight'
for name, m in model.named_modules(): ... # '' 是模型自己，然后逐层展开
for name, b in model.named_buffers(): ...
model.parameters(recurse=False) # 只要本层直属参数，不含子 module
model.named_parameters(prefix="student") # 给 key 加前缀，多模型合并时有用
```

最常考的实用代码是 **weight decay** 分组：decay 只应作用在矩阵型权重上。bias 和 norm 的 weight / bias 都是逐通道的一维参数，对它们做 L2 会把 scale 往 0 拉，损害表达能力且没有正则收益。判据用 `p.ndim` 最稳，不依赖命名习惯：

```
def param_groups(model, weight_decay=0.1):
    decay, no_decay = [], []
    for name, p in model.named_parameters():
        if not p.requires_grad:
            continue # 冻结的参数别放进 optimizer
        if p.ndim <= 1 or name.endswith(".bias"):
            no_decay.append(p) # bias、LayerNorm/RMSNorm 的 weight、可学
scale
        else:
            decay.append(p) # Linear / Embedding / Conv 的权重矩阵
    return [{"params": decay, "weight_decay": weight_decay},
            {"params": no_decay, "weight_decay": 0.0}]

opt = torch.optim.AdamW(param_groups(model), lr=3e-4, betas=(0.9, 0.95))
```

两个细节：`named_parameters()` 默认去重，所以 tied embedding 不会被放进两个组；`nn.Embedding` 是二维的、会落进 decay 组——是否给 embedding 加 decay 各家做法不一。

冻结参数：requires_grad_(False) vs 从 optimizer 排除

	<code>p.requires_grad_(False)</code>	不放进 optimizer
算梯度	不算	算
<code>.grad</code> 显存	不占	占（与参数同大小）
反向计算量	该子图被剪掉，省时间	照跑
weight decay / momentum	不适用	不更新，但可能残留 state
被 <code>state_dict</code> 保存	是	是

结论：要冻结就用 `requires_grad_(False)`，它同时省显存和算力；“从 optimizer 排除”只在你想手动接管更新（自定义 EMA、手写 LoRA 合并）时才用。实践中两个一起做：

```
model.backbone.requires_grad_(False) # Module 上也有这个方法，递归生效
opt = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad],
lr=1e-4)
```

Warning. 两个坑。（1）只做 `requires_grad_(False)` 但仍把参数留在 **optimizer** 里，且它们之前已经有过 `.grad`（冻结发生在若干 step 之后），Adam 会继续用残留的 `.grad`

和 momentum 更新它们——参数在“冻结”后还在动。冻结时顺手 `p.grad = None` 并重建 optimizer。(2) 冻了参数没冻 BN: BN 在 `train()` 下仍会更新 running stats, backbone 的行为还在变, 要配 `backbone.eval()`。DDP 下参数完全不需要梯度时还要注意 `find_unused_parameters`, 见第 18 章。

`to()` / `cuda()` 的不一致: **module** 原地改, **tensor** 返回新对象

```
t = torch.randn(4)
t.to("cuda")           # 返回新张量, t 本身还在 CPU! 必须写 t = t.to("cuda")

m = nn.Linear(4, 4)
m.to("cuda")           # 原地改 module (也返回 self, 所以 m = m.to(...)) 也对
next(m.parameters()).device  # cuda:0
```

这个不一致是有原因的: `Tensor` 语义上不可变, 改 dtype / device 必须换内存; 而 `Module` 是容器, `_apply()` 遍历所有 parameter / buffer, 把 `param.data` 换成搬过去的新张量, `Parameter` 对象本身的身份保持不变。身份不变很关键——optimizer 按对象引用持有参数, 如果 `.cuda()` 换掉了 `Parameter` 对象, 已建好的 optimizer 就全指向 CPU 上的旧对象。

Warning. 先 `.cuda()`, 再建 optimizer。optimizer 的 state (Adam 的 `exp_avg` / `exp_avg_sq`) 是第一次 `step()` 时按当时的 device 惰性创建的。更实际的问题是恢复训练: `model.cuda()` 之后再 `optimizer.load_state_dict(...)` 才能让 state 落在正确 device; 反过来先加载 CPU state 再 `model.cuda()`, optimizer state 留在 CPU, `step()` 时 device mismatch。标准顺序是 **build model**, **.cuda()**, **build optimizer load** 两个 **state_dict**。

还有一条: `module.to()` 只搬 `_parameters` 和 `_buffers`, 放在 `self.__dict__` 里的普通张量搬不走——这是 `register_buffer` 存在的第二个理由。`.to(dtype)` 在 module 上也只影响浮点参数与 buffer, 整型 buffer (比如 `num_batches_tracked`) 会被跳过。

面试考点

面试考点. Q1: `nn.Module` 有哪几张注册表? `nn.Parameter` 为什么不能直接用 `Tensor` 代替?

A: `_parameters`、`_buffers`、`_modules`, 加一组 hook 字典, `__setattr__` 按类型分流。`nn.Parameter` 的作用是类型标记, 让 `__setattr__` 认出“这是要被 optimizer 更新的参数”。不能靠 `requires_grad` 判断, 因为冻结的参数 `requires_grad=False` 但依然要进 `state_dict`、依然要被 `.cuda()` 搬走——类型标记和梯度开关必须解耦。

面试考点. Q2: 为什么 BN 的 `running_mean` 是 buffer 不是 parameter? `persistent=False` 什么时候用?

A: 它是靠 EMA 累积的数据统计量、不是梯度学出来的, 所以不该进 optimizer; 但推理完全依赖它, 必须存进 checkpoint。“要持久化 + 不要梯度”正好是 buffer 的定义 (BN 里靠梯度学的是 `weight` / `bias`) `persistent=False` 给能从超参重算的常量用: causal mask、RoPE 的 `sin/cos` 表——checkpoint 更小, 改 `max_seq_len` 时也不会 shape 不匹配。

面试考点. Q3 : 用 python list 装 module 会怎样？

A : `__setattr__` 只识别 `nn.Parameter` 和 `nn.Module` , 普通 list 落到 `__dict__` , 里面的 module 一个都不注册。后果是 `parameters()` 返回空 (optimizer 什么都不更新)、`state_dict()` 存不下、`.cuda()` 搬不走。症状是 loss 不动或 device mismatch , 不报错。换 `nn.ModuleList` / `nn.ParameterList` 。

面试考点. Q4 : `state_dict()` 返回的是引用还是拷贝？`load_state_dict` 是 copy 还是重绑定？

A : 返回 `detach()` 后的张量, 与参数共享 **storage** , `optimizer.step()` 之后值会跟着变, 不能当快照用——要快照得 `{k: v.detach().clone() for ...}` 。`load_state_dict` 是在 `no_grad()` 下做 `param.copy_(...)` 的原地拷贝、不重绑对象, 所以 optimizer 持有的引用 load 后依然有效, 不用重建。

面试考点. Q5 : `model.eval()` 和 `torch.no_grad()` 的区别？推理要写哪个？

A : 两件不同的事。`eval()` 只是递归置 `self.training = False` , 改变 Dropout (变恒等映射) 和 BatchNorm (改用 running stats 且不再更新) 的计算行为, 但照样建图、照样吃显存。`no_grad()` 只是不建图, 完全不影响 Dropout / BN。推理两个都要：只写 `no_grad` 会让 Dropout 还在丢、BN 还在污染 running stats (验证指标偏低)；只写 `eval` 结果对但显存翻倍。纯推理用 `inference_mode()` 。

面试考点. Q6 : Xavier 和 Kaiming 怎么选？`nn.Linear` 默认用的是哪个？

A : 看激活是否关于 0 对称。tanh / sigmoid / 恒等用 Xavier , 方差 $\frac{2}{f_{an_in} + f_{an_out}}$; ReLU 族用 Kaiming , 方差 $\frac{2}{f_{an_in}}$, 那个 2 是“ReLU 砍掉一半输出、方差减半”的补偿。`nn.Linear` 默认 `kaiming_uniform(a=sqrt(5))` , 代入公式恰好等于 $U\left(-\frac{1}{\sqrt{f_{an_in}}}, \frac{1}{\sqrt{f_{an_in}}}\right)$, 这是对旧版 Torch 的历史遗留、不是为 ReLU 调好的值。

面试考点. Q7 : tied embedding 怎么写？`state_dict` 和 `parameters()` 里各是什么表现？

A : `self.lm_head.weight = self.emb.weight` , 赋的是同一个 `Parameter` 对象。`named_parameters()` 默认 `remove_duplicate=True` 只返回一个, 所以 optimizer 不会更新两次；`state_dict()` 不去重, 两个 key 都在但指向同一块内存, 保证 checkpoint 自描述。反向时梯度自动累加。不能写 `lm_head.weight.data = emb.weight.data` ——那是两个 `Parameter` 共享 storage , 会被更新两次。

面试考点. Q8 : 为什么 `t.to("cuda")` 要接返回值, `model.to("cuda")` 不用？

A : Tensor 语义不可变, 换 device 必须换内存, 所以 `.to()` 返回新对象、原张量不动。Module 是容器, `_apply()` 遍历 parameter / buffer 把 `param.data` 替换成新张量, **Parameter** 对象的身份保持不变——这是必须的, 因为 optimizer 按对象引用持有参数。两个推论：`module.to()` 只搬 `_parameters` 和 `_buffers` , 塞在 `self.__dict__` 里的张量搬不走；标准顺序是 model 先 `.cuda()` 再建 optimizer / 加载 optimizer state , 否则 `step()` 会 device mismatch。

数据管线：Dataset / DataLoader / Sampler

数据管线是面试里最容易问出真实经验的地方：`nn.Module` 写错了 `loss` 会炸，数据管线写错了训练照样收敛，只是慢一半、或者悄悄把同一批增强样本喂了 4 遍。面试官问 `num_workers`、`pin_memory`、`set_epoch`，考的不是 API 名字，而是你有没有真的看过 GPU 利用率的锯齿波。

这一章讲三件事：`Dataset` 怎么定义“一条样本”，`DataLoader` 怎么把它变成 `batch` 并与训练重叠，`Sampler` 怎么决定“看哪些样本、以什么顺序”。GPU 侧的拷贝机制见第 9 章，`profiler` 用法见第 11 章，`DistributedSampler` 之外的分布式数据切分见第 18 章。

两种 Dataset：map-style 与 iterable-style

PyTorch 只有两种数据集协议，选哪种取决于你能不能随机寻址。

map-style：实现 `__getitem__(idx)` 和 `__len__()`。`DataLoader` 先用 `Sampler` 生成一串 `index`，再让 `worker` 去取。

```
from torch.utils.data import Dataset

class TokenDataset(Dataset):
    def __init__(self, path):
        self.offsets = load_index(path) # 只存偏移表，不 load 全量数据
        self.path, self.fh = path, None # 句柄延迟到 worker 里再开

    def __len__(self):
        return len(self.offsets)

    def __getitem__(self, idx):
        if self.fh is None: # 每个 worker 各开一份
            self.fh = open(self.path, "rb")
        self.fh.seek(self.offsets[idx])
        return decode(self.fh.read(...)) # 返回 tensor / dict / tuple
```

iterable-style：继承 `IterableDataset`，实现 `__iter__()`。适合无法预知长度、或者只能顺序读的源：Kafka / 消息队列、无限流、S3 上的 `shard` 文件、需要边读边解码的 `webdataset`。

```
from torch.utils.data import IterableDataset, get_worker_info

class ShardStream(IterableDataset):
    def __init__(self, shards):
        self.shards = shards

    def __iter__(self):
        info = get_worker_info()
        shards = self.shards
        if info is not None: # 多 worker：必须自己分片
            shards = shards[info.id :: info.num_workers]
```

```

for s in shards:
    for rec in read_records(s):
        yield rec

```

	map-style	iterable-style
必须实现	<code>__getitem__</code> + <code>__len__</code>	<code>__iter__</code>
<code>shuffle=True</code>	可用 (走 <code>RandomSampler</code>)	不可用, <code>DataLoader</code> 直接报错
<code>sampler / batch_sampler</code>	可用	不可用
多 worker 分片	<code>DataLoader</code> 按 index 自动分	你自己分
<code>len(data_loader)</code>	准确	取决于你是否实现 <code>__len__</code> (可能不准)
典型场景	本地文件、内存数据集、图像目录	流式、无限数据、超大 shard

Warning. `IterableDataset` + `num_workers>0` 且没有按 `get_worker_info()` 分片, 是最高频的静默数据 bug: 每个 worker 都会把 `__iter__` 从头跑一遍, 于是每条样本被喂 `num_workers` 遍, epoch 长度也变成 `num_workers` 倍。loss 曲线照样下降, 只是你在做 4 倍重复数据的训练。

```

# 错: 4 个 worker 各自遍历全部 shards → 数据 ×4
def __iter__(self):
    for s in self.shards:
        yield from read_records(s)

```

修法有两种: **shard 级切分** (上面的 `shards[id::num_workers]`, 要求 shard 数 $\geq$ worker 数, 否则有 worker 空转); 或**样本级切分** (`if i % num_workers == info.id`, 负载均衡但每个 worker 仍要解码全部记录, 白做 IO)。生产上优先 shard 级, 并让 shard 数是 worker 数的整数倍。

分布式下还要做两级切分: 先按 `dist.get_rank() / world_size` 分给各进程, 再按 `worker_info` 分给进程内的 worker, 总切分数 `world_size * num_workers`。`DistributedSampler` 对 `iterable-style` 无效。

DataLoader 的参数, 逐个讲清

```

loader = DataLoader(dataset, batch_size=64, shuffle=True,
                    num_workers=8, pin_memory=True, drop_last=True,
                    persistent_workers=True, prefetch_factor=4,
                    collate_fn=my_collate, worker_init_fn=my_init)

```

batch_size 每批样本数。设成 `None` 表示“不自动 batch”, 此时 `DataLoader` 直接把单条样本交给 `collate_fn` (流式场景里 dataset 自己已经 yield 好 batch 时用)。

shuffle 只是 `sampler=RandomSampler(dataset)` 的语法糖。与 `sampler` 互斥, 二者同时给会报错。分布式下不要用 `shuffle=True`, 把打乱交给 `DistributedSampler`。

num_workers 加载子进程数。0 表示在主进程里同步加载。

pin_memory 把 batch 拷进 page-locked (pinned) 内存, 为异步 H2D 拷贝铺路, 见下面 `pin_memory` 一节。

drop_last 丢掉最后一个不足 `batch_size` 的残批。开它的三个理由：BatchNorm 遇到 `batch=1` 会报错；`torch.compile` 下变化的 batch size 会触发重编译；DDP 各 rank 残批大小不同会让梯度权重失衡。

persistent_workers `True` 时 epoch 结束不杀 worker。默认 `False`，意味着每个 **epoch** 都要重新 **fork + 重建 dataset** 状态，小数据集上这个开销能占 epoch 时间的可观比例。

prefetch_factor 每个 worker 最多预备几个 batch (`num_workers>0` 时默认 2)。在途 batch 总量 = `num_workers * prefetch_factor` 个 batch，这直接决定了 DataLoader 的常驻内存。

collate_fn 把 `list[样本]` 拼成 batch，跑在 worker 进程里，见下面 `collate` 一节。

worker_init_fn 在每个 worker 进程里、开始取数据之前调用一次，签名 `fn(worker_id)`。用来设 RNG、绑核、打开句柄。

timeout 从 worker 队列取 batch 的超时秒数，0 为不超时。设个正值能把“worker 卡死”从 hang 变成异常。

Note. 较新版本加了 `in_order` 参数，默认 `True`。设 `False` 允许 worker 乱序返回 batch，慢 worker 不再拖住整条流水线，代价是 batch 顺序不可复现。样本解码耗时方差大（比如混了超长视频）时值得试。

num_workers 的工作原理

`num_workers>0` 时 DataLoader 的结构是一条多进程流水线：

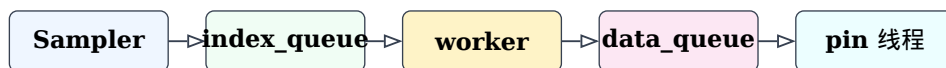


图 6 `num_workers>0` 时的数据流。Sampler 和队列在主进程，`__getitem__` + `collate_fn` 在子进程，pin 线程回到主进程，最后交给训练循环。

1. 主进程的 Sampler 生成 index (或 index 列表)，round-robin 塞进每个 worker 的 `index_queue`。
2. 每个 worker 循环：取 index → 调 `dataset[idx]` → 调 `collate_fn` 拼 batch → 把结果写进共享内存、把句柄放进公共的 `data_queue`。
3. 主进程（若 `pin_memory=True` 则由专门的 pin 线程）从 `data_queue` 拿到结果，按原始顺序重排后交给训练循环。
4. 主进程每消费掉一个 batch，就补发一个新 index，维持“在途 `num_workers * prefetch_factor` 个 batch”的水位。

关键点是预取：worker 在你还在做 `loss.backward()` 的时候就已经在准备后面的 batch 了。所以理想情况下 `next(iter(loader))` 是一次队列 pop，几乎不耗时——数据加载被完全隐藏在 GPU 计算后面。

`num_workers=0` 就没有这条流水线：`for batch in loader` 会在主进程里同步跑 `__getitem__` 和 `collate_fn`。CPU 解码和 GPU 计算严格串行，`step time` = 加载时间 + 计算时间。

num_workers=0 串行 vs num_workers=4 + pin_memory 重叠

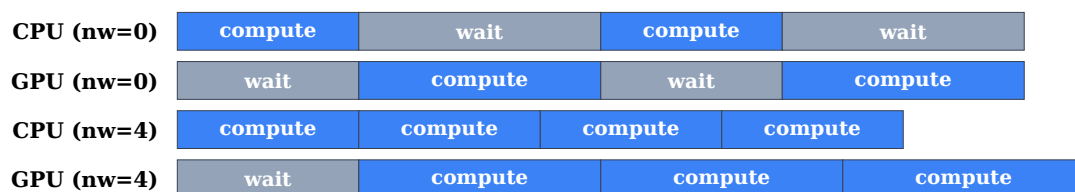


图 7 上两行 num_workers=0 : CPU 解码 (compute) 与 GPU 计算交替, 谁在干活另一边就在 wait。下两行 4 个 worker 预取: 只有第一个 batch 有冷启动的 wait, 之后 GPU 背靠背跑, 解码被隐藏。

Key insight. DataLoader 的目标不是“加载得快”, 而是“加载得不比 GPU 慢”。只要解码一个 batch 的时间 / num_workers < GPU 算一个 batch 的时间, 再多加 worker 也不会让 step 变快。判断有没有到这个点, 见后面“判断 dataloader 是不是瓶颈”一节。

pin_memory 为什么能加速

普通 torch.Tensor 在 pageable (可换页) 内存里。CUDA 的 DMA 引擎不能直接读 pageable 内存——操作系统随时可能把那些页换出去。所以 cudaMemcpyAsync 从 pageable 内存出发时, driver 只能先把数据同步拷到一块内部的 pinned 中转缓冲区, 再从中转区发 DMA。结果: 多一次 CPU 侧 memcpy, 而且这个“异步”拷贝实际上会阻塞调用线程。

pinned (page-locked) 内存被锁定在物理页上, DMA 引擎可以直接读, cudaMemcpyAsync 才真正是异步的: 立即返回, 拷贝在 copy engine 上与 compute kernel 并行。

```
loader = DataLoader(ds, batch_size=64, num_workers=8, pin_memory=True)

for x, y in loader:
    x = x.to("cuda", non_blocking=True)      # 必须配 non_blocking
    y = y.to("cuda", non_blocking=True)
    out = model(x)                            # 排在同一 stream, 自动等拷贝完
```

Warning. pin_memory=True 和 non_blocking=True 必须成对出现, 单独用哪一个都没有意义:

- 只开 pin_memory, .to(device) 默认 non_blocking=False → 仍然同步等待, 你只是白花了 pin 的开销。
- 只开 non_blocking, 源是 pageable 内存 → driver 走中转缓冲, 实际仍然同步。这是最容易自欺欺人的一种“优化”。

另外: pinned 内存是稀缺的物理资源。num_workers * prefetch_factor * batch 大小 全都常驻 pinned, 几十 GB 的 pin 会拖慢整个系统的内存分配, 甚至触发 OOM killer。数据量大时先降 prefetch_factor。

pin 操作由主进程里一个专门的线程做, 不在 worker 里做 (worker 里 pin 的内存没法跨进程传)。所以这一步是单线程的, 超大 batch 下 pin 本身也可能成为瓶颈——此时让 dataset 返回 uint8 而不是 float32、把归一化搬到 GPU 上做, 能直接把要 pin 的字节数降到 1/4。

worker 的四类坑

进程启动开销。默认 `persistent_workers=False`，每个 epoch 的第一个 `iter()` 都要 fork 出 `num_workers` 个进程、重跑 `worker_init_fn`、重建 dataset 内部状态（重开文件、重连数据库）。数据集小、epoch 短时这个固定开销很显眼。开 `persistent_workers=True` 即可，代价是 worker 常驻内存。

shared memory 不足。worker 通过 `/dev/shm` 把 tensor 传回主进程。容器里 `/dev/shm` 默认常常只有 64 MB，一旦装不下就报：

```
RuntimeError: DataLoader worker (pid 12345) is killed by signal: Bus error.
It is possible that dataloader's workers are out of shared memory.
```

修法：`docker run --shm-size=16g`（或 `--ipc=host`）；或降 `num_workers` / `prefetch_factor` / `batch_size`。用 `df -h /dev/shm` 确认实际大小。

num_workers 过大。worker 数超过物理核数后开始 CPU 争抢：context switch 变多、每个 worker 变慢、总吞吐反而下降。而且每个 worker 都是完整的 Python 进程，dataset 里如果 hold 了大对象（比如整个 `list` 的 Python str），内存是 `num_workers` 倍。经验起点：`num_workers = min(物理核数 / 每卡进程数, 8)`，然后实测调整。

Warning. fork 与 CUDA 不兼容。Linux 上 DataLoader 默认用 `fork` 起 worker。CUDA context 无法跨 fork 继承，所以 worker 里一旦碰 CUDA 就报：

```
RuntimeError: Cannot re-initialize CUDA in forked subprocess.
To use CUDA with multiprocessing, you must use the 'spawn' start method
```

实践规则：`__getitem__` 里只返回 **CPU tensor**，`.cuda()` 一律在训练循环里做。dataset 也不要再 `__init__` 里 hold CUDA tensor（fork 后子进程拿到的是无效句柄）。真要在 worker 里用 GPU（比如 GPU 解码），得传 `multiprocessing_context="spawn"`，代价是启动慢很多、dataset 必须可 pickle。

Note. `torch.set_num_threads(1)` 已经在 worker 里自动设好了。但如果 dataset 用了 OpenCV / numpy 的多线程后端，还要手动 `cv2.setNumThreads(0)`、设 `OMP_NUM_THREADS=1`，否则 8 个 worker 各开 8 个线程会把机器打满。

随机性：多 worker 下的 RNG

DataLoader 每次建迭代器时，从主进程的 `generator` 抽一个 `base_seed`，然后在 worker *i* 里执行：

```
seed = base_seed + worker_id
random.seed(seed)                    # Python 内置 random
torch.manual_seed(seed)              # torch CPU RNG
np.random.seed(_generate_state(base_seed, worker_id)) # numpy 全局 RNG
```

所以 `random`、`torch`、`np.random` 这三个全局 RNG 在各 worker 里是不同的。`torch.initial_seed()` 在 worker 里返回的就是上面那个 `seed`，`get_worker_info().seed` 也是它。

Warning. “多 worker 下 numpy 增强重复” 这个经典 bug，在现代 PyTorch 里不是由 `np.random` 全局 RNG 引起的（它已经被逐 worker 播种了）。真正还会踩的是 **fork** 前就创建好的 **RNG** 对象：

```
class BadAug(Dataset):
    def __init__(self):
        self.rng = np.random.default_rng(0)    # ← fork 前建好

    def __getitem__(self, i):
        # 每个 worker 继承同一份 rng 状态 → 各 worker 产出完全相同的抖动序列
        return self.rng.random(3)
```

同理还有 `__init__` 里建的 `torch.Generator()`、`random.Random(0)` 实例，以及某些第三方增强库在 import 时快照的 RNG。判断标准很简单：**RNG** 状态是不是在 **fork** 之前就固定了。

修法一：在 `worker_init_fn` 里按 worker 重新播种。

```
def worker_init_fn(worker_id):
    info = torch.utils.data.get_worker_info()
    info.dataset.rng = np.random.default_rng(info.seed)    # info.dataset 是本
    worker 的副本
```

修法二：不持有 RNG 对象，`__getitem__` 里直接用 `np.random` / `torch.rand` 全局接口。

修法三（最干净）：让随机性只依赖 index，`np.random.default_rng((epoch, idx))`，增强结果可复现、与 worker 数无关。

要让整条管线可复现，需要固定三处：`DataLoader` 的 `generator`（决定 shuffle 顺序和 `base_seed`）、`worker_init_fn`（决定 worker 内 RNG）以及全局的 `torch.manual_seed`。完整的确定性讨论见第 10 章。

```
g = torch.Generator()
g.manual_seed(1234)
loader = DataLoader(ds, batch_size=32, shuffle=True, num_workers=4,
                    generator=g, worker_init_fn=worker_init_fn)
```

`collate_fn`：从样本列表到 **batch**

默认的 `default_collate` 按类型递归处理一个 `list[样本]`：

- `Tensor` → `torch.stack(batch, 0)`，要求所有样本 **shape** 完全一致
- `numpy array` → 先转 tensor 再 stack；`int` / `float` → 转成 1-D tensor
- `str` / `bytes` → 原样保留成 list（不转 tensor）
- `Mapping`（dict）→ 逐 key 递归，输出同样 key 的 dict
- `Sequence`（tuple / list）→ 先转置再逐位置递归：`[(x1,y1),(x2,y2)]` 变成 `(stack([x1,x2]), stack([y1,y2]))`
- `namedtuple` → 保留类型，逐字段递归

`__getitem__` 返回 dict 时特别顺手，因为 `default_collate` 会自动把 `{"input": t, "label": i}` 拼成 `{"input": (B,...), "label": (B,)}`。

需要自定义 collate 的最常见理由是变长序列——shape 不一致，stack 直接报错。

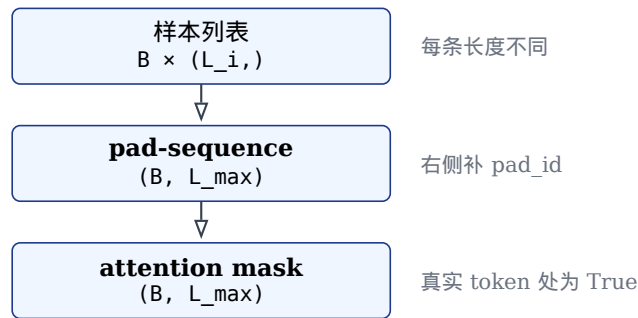


图 8 变长序列 collate 的 shape 变化。L_{max} 是这个 batch 内的最大长度，不是全局最大长度。

```
from torch.nn.utils.rnn import pad_sequence

def pad_collate(batch, pad_id: int = 0):
    """batch: list of dict(input_ids=LongTensor(L_i,), label=int)"""
    seqs = [b["input_ids"] for b in batch]
    lengths = torch.tensor([s.numel() for s in seqs])

    # batch_first=True → (B, L_max); 只 pad 到本 batch 的最大长度
    input_ids = pad_sequence(seqs, batch_first=True, padding_value=pad_id)
    attn_mask = torch.arange(input_ids.size(1))[None, :] < lengths[:, None]

    return {
        "input_ids": input_ids,          # (B, L_max) long
        "attention_mask": attn_mask,     # (B, L_max) bool
        "lengths": lengths,              # (B,) long, 给 pack_padded_sequence 用
        "labels": torch.tensor([b["label"] for b in batch]),
    }

loader = DataLoader(ds, batch_size=32, collate_fn=pad_collate, num_workers=4)
```

三个细节值得在面试里主动说：

1. 只 **pad** 到本 **batch** 最大长度，不是全局最大长度。配合下面的 bucketing，padding 浪费能降一个数量级。
2. collate_fn 是在 **worker** 进程里执行的（num_workers>0 时），所以把 padding、tokenize 这类 CPU 活放这里是免费的并行；反过来说，collate 里绝对不能碰 CUDA。
3. mask 的 dtype 用 bool，别用 float——scaled_dot_product_attention 和 masked_fill_ 都吃 bool mask，float mask 语义还容易和 additive mask 搞混。

想在 collate 里丢掉坏样本（解码失败时 __getitem__ 返回 None）：
batch = [b for b in batch if b is not None]，空 batch 返回 None 并在训练循环里 continue。代价是 batch size 会变，与 torch.compile 的静态 shape 假设冲突。

Sampler：决定看哪些样本、什么顺序

Sampler 产出的是 **index**，不是数据。DataLoader 里的层次是：Sampler 出单个 index → BatchSampler 攒成 index 列表 → worker 按列表取样本 → collate_fn 拼 batch。

Sampler	语义与用途
---------	-------

SequentialSampler	<code>range(len(ds))</code> 。 <code>shuffle=False</code> 时的默认值，验证集用它
RandomSampler	<code>shuffle=True</code> 时的默认值。 <code>replacement=False</code> 是无放回全排列；开 <code>replacement=True</code> 可配 <code>num_samples</code> 做有放回过采样
WeightedRandomSampler	按逐样本权重有放回采样，类别不均衡的标准解法
SubsetRandomSampler	在给定 <code>index</code> 子集里随机采，做 <code>train/val split</code> 很方便
BatchSampler	把上面任一个包一层，一次吐一个 <code>index list</code> 。传给 <code>batch_sampler=</code> 参数
DistributedSampler	按 <code>rank</code> 取交错分片，DDP 必用

类别不均衡： `WeightedRandomSampler` 的 `weights` 是长度等于数据集大小的逐样本权重，不是逐类别权重。最常见的写法是“权重 = 1 / 该样本所属类别的样本数”：

```
from torch.utils.data import WeightedRandomSampler

labels = torch.tensor([ds[i]["label"] for i in range(len(ds))]) # 或从元信息直接读
counts = torch.bincount(labels)                               # (num_classes,)
weights = 1.0 / counts[labels].float()                         # (N,) 逐样本权重

sampler = WeightedRandomSampler(weights, num_samples=len(ds), replacement=True)
loader = DataLoader(ds, batch_size=64, sampler=sampler) # 注意：不能再传
shuffle=True
```

`replacement=True` 意味着一个 `epoch` 内少数类样本会被重复看到、多数类样本可能一次都没看到。它和“给 `loss` 加 `class weight`”是两种思路：前者改数据分布（每个 `batch` 内类别均衡，`BatchNorm` 统计更稳），后者改梯度权重（不重复数据，但 `batch` 内仍不均衡）。面试里能说出这个对比就够了。

`DistributedSampler` 的机制，源码只有几行但每一行都是考点：

```
# rank r, world_size W, epoch e
g = torch.Generator(); g.manual_seed(self.seed + self.epoch)
indices = torch.randperm(len(dataset), generator=g).tolist() # 各 rank 结果相同

if not drop_last:
    indices += indices[: total_size - len(indices)] # 重复开头几条，padding 到整除
else:
    indices = indices[:total_size] # 截掉尾巴

indices = indices[rank : total_size : W] # 交错取，步长 W
```

两个要点：

1. 各 **rank** 用同一个 **seed** 做同一个 **randperm**，再按 `[rank::W]` 交错切片。这保证了不重不漏，且不需要任何通信。
2. 各 `rank` 的分片必须等长，否则先跑完的 `rank` 会让其他 `rank` 在 `AllReduce` 上永久 hang。所以 `drop_last=False`（默认）会重复开头的若干条样本补到 `total_size`，`drop_last=True` 则丢掉尾部不足整除的部分。

Warning. 忘记 `sampler.set_epoch(epoch)` 是最高频的 **DDP** 数据 **bug**。seed 是 `self.seed + self.epoch`，而 `self.epoch` 初始是 0 且不会自己变。不调 `set_epoch`，每个 epoch 的 `randperm` 完全一样 → 你的模型在按固定顺序反复看同一串数据，shuffle 形同虚设。

```
sampler = DistributedSampler(ds, shuffle=True)
loader = DataLoader(ds, batch_size=64, sampler=sampler, num_workers=8)

for epoch in range(epochs):
    sampler.set_epoch(epoch)      # ← 必须在建迭代器之前调
    for batch in loader:
        ...
```

它没有任何报错或警告，只表现为“收敛比单卡差一点”。面试里被问到“DDP 训练效果不如单卡，你怎么排查”，`set_epoch` 应该是你前三个怀疑对象之一。

Warning. 验证集上 `drop_last=False` 会让 **metric** 偏。padding 出来的重复样本会被算进 `accuracy / loss` 的分母，各 rank AllReduce 求平均后结果比真值略偏。严谨做法：验证时每个 rank 记录 (sum, count) 而不是均值，AllReduce 后再相除；或者手动截掉 rank 分片里超出 `len(dataset)` 的那几条。

判断 dataloader 是不是瓶颈

按代价从低到高排查：

1. 看 **GPU** 利用率波形。 `nvidia-smi dmon` 或 `nvitop`，如果 util 是“100% → 0% → 100%”的锯齿而不是稳定平台，GPU 在等数据。
2. 把 **dataset** 换成常量。最直接的判据：用一个 `__getitem__` 返回预生成 tensor 的假 dataset 跑同样的循环。step time 明显变快 → 瓶颈在数据；几乎不变 → 瓶颈在模型。这个实验 5 行代码，比读 profiler 快得多。
3. **torch.profiler** 看 **gap**。trace 里两个 step 之间有一大段 GPU 空白，且 CPU 侧停在 `DataLoader.__next__` / `_get_data` 上，就是在等 worker 出货。
4. 拉 **prefetch_factor** 看变化。如果只有前几个 step 慢、后面就跟上了，说明只是冷启动，不是持续瓶颈。

确认是数据瓶颈后，按收益 / 改动成本排序去修：

版本	核心思路	相对成本 / 收益
加 <code>num_workers</code>	多进程并行解码	改一个数字，先做这个
<code>persistent_workers</code> + <code>prefetch_factor</code>	消除 epoch 冷启动、加深队列	改参数，代价是内存
<code>pin_memory</code> + <code>non_blocking</code>	H2D 真异步，与 compute 重叠	两行代码
轻量化返回值	返回 <code>uint8</code> ，归一化搬到 GPU	改 dataset + 训练循环

预处理成二进制格式	webdataset / tfrecord / numpy memmap , 去掉逐文件 IO 和解码	要跑一次离线转换
GPU 解码	nvJPEG / DALI / torchcodec , 解码搬上 GPU	引入依赖, 吃 GPU 算力

Key insight. 绝大多数“dataloader 慢”最后都归结到两件事之一：小文件随机 IO（几百万个 jpg，每次 open + seek）或 **Python** 层解码（PIL、json.loads）。前者的正解是打包成大 shard 顺序读，后者的正解是离线预处理成能 memmap 直接读的二进制。加 worker 只是把这两个问题并行化，治不了根。

padding 浪费：bucketing 与 packing

变长序列 batch 里，pad_sequence 把所有样本补到本 batch 最长。如果一个 batch 里混了长度 8 和长度 512 的样本，绝大部分算力花在 pad token 上——它们的输出会被 mask 掉，纯属白算。

bucketing（长度分桶）：让长度相近的样本进同一个 batch。实现方式是自定义 BatchSampler：先按长度粗排序（加一点随机扰动以免每个 epoch batch 组成完全固定），再切成 batch，最后把 batch 的顺序打乱。

```
class LengthBucketSampler(torch.utils.data.Sampler):
    def __init__(self, lengths, batch_size, pool=100):
        self.lengths, self.bs = lengths, batch_size
        self.pool = pool * batch_size  # 局部排序窗口

    def __iter__(self):
        idx = torch.randperm(len(self.lengths)).tolist()  # 先全局打乱
        batches = []
        for i in range(0, len(idx), self.pool):
            chunk = sorted(idx[i : i + self.pool], key=lambda j: self.lengths[j])
            batches += [chunk[k : k + self.bs] for k in range(0, len(chunk),
self.bs)]
        yield from (batches[i] for i in torch.randperm(len(batches)).tolist())

    def __len__(self):
        return (len(self.lengths) + self.bs - 1) // self.bs

loader = DataLoader(ds, batch_sampler=LengthBucketSampler(lens, 32),
                    collate_fn=pad_collate, num_workers=4)
```

packing（序列拼接）：更彻底——把多条短序列首尾拼进一个固定长度 L 的序列，padding 几乎为零，每个 batch 的 token 数恒定（对 torch.compile 的静态 shape 也友好）。代价是需要防止 attention 跨越样本边界：要么传 block-diagonal mask，要么用 FlashAttention 的 varlen 接口（cu_seqlens 累积长度）。LLM 预训练几乎都用 packing，SFT 阶段则要格外小心 loss mask 别把上一条样本的 label 算进来。

Note. bucketing 让每个 batch 的 token 数不固定，配固定 `batch_size` 会导致显存忽高忽低（长 batch OOM）。生产上常按 **token** 预算组 batch：`batch_size = max_tokens // 本 batch 最长长度`。这需要 `batch_sampler` 而不是 `batch_size`。

面试考点

面试考点. Q1：Dataset 和 IterableDataset 的区别？各自什么时候用？

A：map-style 实现 `__getitem__` + `__len__`，支持随机寻址，DataLoader 用 Sampler 生成 index 并自动在 worker 间分配；iterable-style 实现 `__iter__`，只能顺序读，不支持 `shuffle / sampler`。本地文件、内存数据集用前者；Kafka、无限流、超大 shard 用后者。后者在多 worker 下必须自己按 `get_worker_info()` 分片。

面试考点. Q2：IterableDataset 配 `num_workers=4` 会发生什么？

A：如果不分片，4 个 worker 各自把 `__iter__` 从头跑一遍，每条样本被喂 4 遍，epoch 变成 4 倍长，而且完全不报错。正确写法是在 `__iter__` 里读 `get_worker_info()`，按 `shards[info.id::info.num_workers]` 切分。分布式下还要再叠一层按 rank 的切分，总切分数是 `world_size * num_workers`。

面试考点. Q3：`num_workers` 设多少合适？设太大有什么问题？

A：起点是 `min(物理核数 / 每卡进程数, 8)`，然后实测。太大有三个问题：worker 数超过核数后 CPU 争抢导致每个 worker 都变慢；每个 worker 是完整 Python 进程，内存是 `num_workers` 倍；在途 batch 数 `num_workers * prefetch_factor` 变大，`/dev/shm` 和 pinned 内存吃紧。判断“够不够”的标准是 GPU util 是否稳定，不是 worker 越多越好。

面试考点. Q4：`pin_memory=True` 为什么能加速？单独开它有用吗？

A：DMA 引擎不能直接读 pageable 内存，driver 得先同步拷到内部 pinned 中转区，所谓的异步拷贝实际会阻塞。pinned 内存锁定物理页，`cudaMemcpyAsync` 才真异步，能与 compute overlap。必须配 `.to(device, non_blocking=True)` 才生效——只开 `pin_memory` 而 `.to()` 是同步的，等于白花 pin 的开销；只开 `non_blocking` 而源是 pageable，driver 仍走同步路径。

面试考点. Q5：报 `DataLoader worker is killed by signal: Bus error` 怎么排查？

A：几乎总是 `/dev/shm` 不够——worker 通过共享内存把 tensor 传回主进程，容器里 `/dev/shm` 默认可能只有 64 MB。修法：`docker run --shm-size=16g` 或 `--ipc=host`；或降 `num_workers / prefetch_factor / batch_size`。另一类可能是 worker 里真的 OOM 或 `segfault`，用 `num_workers=0` 复现就能看到真实 traceback。

面试考点. Q6：多 worker 下数据增强重复是怎么回事？

A: 现代 PyTorch 已经给每个 worker 分别播种了 `random`、`torch` 和 `np.random` 三个全局 RNG (`seed = base_seed + worker_id`), 所以用全局接口不会重复。还会踩坑的是 **fork** 之前就创建好的 **RNG** 对象——比如在 `Dataset.__init__` 里 `self.rng = np.random.default_rng(0)`, fork 后每个 worker 继承同一份状态, 产出完全相同的随机序列。修法是在 `worker_init_fn` 里按 `get_worker_info().seed` 重新播种, 或者让随机性只依赖 (epoch, idx)。

面试考点. **Q7**: 写一个变长序列的 `collate_fn`。

A: 用 `pad_sequence(seqs, batch_first=True, padding_value=pad_id)` 补到本 **batch** 最大长度, 同时用 `arange(L_max)[None,:] < lengths[:,None]` 造 bool attention mask, 并把 `lengths` 一起返回。关键点: 只 pad 到 batch 内最大长度而不是全局最大; `collate_fn` 跑在 worker 进程里所以 CPU 开销是并行的; mask 用 bool dtype。

面试考点. **Q8**: `DistributedSampler` 是怎么保证各 rank 不重不漏的? 为什么必须 `set_epoch` ?

A: 所有 rank 用同一个 `seed + epoch` 做同一个 `randperm`, 然后各自取 `indices[rank::world_size]` 交错分片, 不需要通信就能保证不重不漏。`set_epoch(epoch)` 是唯一改变 seed 的途径, 不调它 `self.epoch` 永远是 0, 每个 epoch 的顺序完全相同, shuffle 失效。这个 bug 不报错, 只表现为收敛变差。

面试考点. **Q9**: `DistributedSampler` 的 `drop_last` 有什么讲究?

A: `drop_last=False` (默认) 会重复开头的若干样本, 把总数 padding 到 `world_size` 的整数倍, 因为 DDP 要求各 rank 步数完全相同——否则先跑完的 rank 会让其他 rank 在 AllReduce 上 hang。训练时这几条重复样本无所谓; 验证时会污染 **metric**, 应该改成各 rank 上报 (sum, count) 再 AllReduce 相除, 或手动截掉多出来的样本。

面试考点. **Q10**: 怎么确认瓶颈在 dataloader, 确认后按什么顺序优化?

A: 最快的判据是把 dataset 换成返回常量 tensor 的假 dataset, step time 明显变快就是数据瓶颈; 辅助证据是 GPU util 呈锯齿、profiler trace 里 step 之间有 GPU 空白且 CPU 卡在 `DataLoader.__next__`。优化顺序: 加 `num_workers` → `persistent_workers` + `prefetch_factor` → `pin_memory` + `non_blocking` → 返回 uint8 把归一化搬 GPU → 离线预处理成二进制 shard → GPU 解码 (DALI / nvJPEG)。前三步改参数, 后三步改架构。

训练循环的每一行

训练循环是面试里最经典的白板题：面试官让你“写一个训练循环”，然后从你写的每一行往下挖。`zero_grad` 为什么放这儿、`set_to_none` 是什么、AMP 和 `grad clip` 谁先谁后、`scheduler` 一个 epoch 调一次还是一个 step 调一次——这些问题的答案都藏在这十几行代码里。

这一章把训练循环拆开：梯度的生命周期、混合精度的数值机制、优化器的更新公式、checkpoint 要存什么。autograd 的图机制见第 6 章，DDP 的通信细节见第 18 章，显存分析见第 8 章。

骨架：七行代码，每行都有考点

```
model.train() # 1
for epoch in range(num_epochs):
    for x, y in loader:
        x, y = x.to(dev, non_blocking=True), y.to(dev, non_blocking=True)

        optimizer.zero_grad(set_to_none=True) # 2
        logits = model(x) # 3
        loss = criterion(logits, y) # 4
        loss.backward() # 5
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # 6
        optimizer.step() # 7
        scheduler.step() # 8
```

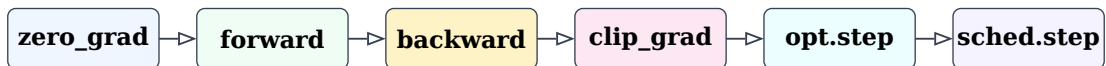


图 9 一个 step 的固定顺序。clip_grad 必须夹在 backward 和 step 之间——梯度已算好、参数还没更新的那个窗口。

- 1 `model.train()` 只把所有 module 的 `self.training` 置 `True`。它不影响 autograd，只影响 Dropout（是否丢）和 BatchNorm（用 batch 统计还是 running 统计）。
- 2 `zero_grad` 清掉上一个 step 的梯度。必须在 `backward()` 之前，因为 autograd 是累加到 `.grad` 上的。
- 3-4 `forward + loss` 建 autograd 图，中间激活被挂住（训练显存的主要来源）。loss 必须是标量，否则 `backward()` 要你显式给 `gradient=`。
- 5 `backward()` 反向遍历图，把梯度累加进各叶子的 `.grad`，同时释放图（除非 `retain_graph=True`）。DDP 的 AllReduce 就挂在这一步的 hook 上。
- 6 `clip_grad_norm` 原地缩放 `.grad`。位置唯一：backward() 之后、step() 之前。
- 7 `optimizer.step()` 读 `.grad`，按更新公式改 `.data`。它不会清梯度，这是 `zero_grad` 存在的原因。
- 8 `scheduler.step()` 改 `optimizer.param_groups[i]["lr"]`，供下一个 `step()` 使用。必须在 `optimizer.step()` 之后。

Warning. `scheduler.step()` 写在 `optimizer.step()` 前面，会让整条 lr 曲线前移一格，torch 会打 `UserWarning: Detected call of lr_scheduler.step() before optimizer.step()` ——

别忽略它。warmup 阶段尤其致命：第一个 step 本该用接近 0 的 lr，颠倒后直接用上第二步的值，配合大 batch 常表现为“训练前 100 步 loss 尖刺”。

zero_grad(set_to_none=True) 为什么更好

set_to_none=True 是 torch 2.0+ 的默认值。两种做法的区别：

	set_to_none=False	set_to_none=True
做了什么	对每个 .grad 跑一次 zero_()	直接 p.grad = None
kernel 数	每个参数一个 memset	零个 kernel
显存	grad buffer 全程常驻	buffer 被释放，forward 峰值期少占一份
.grad 累加	autograd 原地 add_ 进旧 buffer	autograd 直接接管新 tensor
没收到梯度的参数	.grad 是 0，仍会被 step() 更新	.grad 是 None，被 step() 跳过

最后一行是真正的语义差别，也是面试能答出深度的点：Adam / SGD-with-momentum 在梯度为 0 时更新量并不为 0——momentum buffer 还有残值，weight decay 还在作用。所以 set_to_none=False 下，本 step 没参与计算的参数（MoE 里没被路由到的 expert、多任务里没用到的 head）依然会被 momentum 推着走、被 weight decay 缩小。set_to_none=True 让它们真正不动。

Warning. 代价是 .grad 可能是 None，`sum(p.grad.norm() ** 2 for p in model.parameters())` 这类梯度监控代码会 `AttributeError`。自己写的统计要先 `if p.grad is not None` 过滤；`clip_grad_norm` 内部已经过滤了，不用操心。

梯度累积

显存装不下大 batch 时，把它切成 `accum_steps` 个 micro-batch，各自 `backward()` 累积梯度，攒满后 `step()` 一次。数学上等价于一个大 batch——前提是 `loss` 除以 `accum_steps`。完整代码见下面的 AMP 组合写法，这里先说四个要点：

1. **zero_grad** 只在累积窗口边界调，不能每个 micro-step 都调，否则累积失效。
2. **loss** 要除以 `accum_steps`。CrossEntropyLoss 默认 `reduction="mean"`，每个 micro-batch 的 loss 已经是自己那份的均值；直接累加得到 4 倍的均值梯度，等于偷偷把 lr 放大了 4 倍。
3. **scheduler.step()** 跟着 **optimizer.step()** 走，不跟 micro-step。所以“总步数”要按 `len(loader) // accum_steps` 算，算错会让 cosine 曲线走不完或提前触底。
4. **clip** 在累积完成后做，clip 的是完整的累积梯度。

Warning. DDP 下每次 `backward()` 都会触发一轮梯度 AllReduce，而前 `accum_steps - 1` 次的通信纯属浪费——反正还要继续累加。用 `ddp_model.no_sync()` 包住非边界的 micro-step（完整写法见下面的组合代码），通信量直接降到 `1/accum_steps`。

注意最后一个 micro-step 必须在 `no_sync()` 之外，否则梯度永远不同步、各 rank 参数会 drift。FSDP 也有 `no_sync()`，但它还要控制 unshard/reshard 时机，代价是多存一份完整梯度。

Note. 除以 `accum_steps` 严格等价的前提是各 `micro-batch` 大小相同。变长序列 + token 级 loss 下有效 token 数不同，严格做法是 `reduction="sum"` 累加、最后除以窗口内总 token 数。这个偏差在长短句混排的数据上是实打实的，但很多代码库直接忽略了。

混合精度 AMP

AMP 的核心思路：参数和优化器状态保持 **fp32**，只把 **matmul / conv** 这类计算密集的 **op** 的输入降到 **fp16** 或 **bf16**。`torch.autocast` 是一个 `dispatch` 层的开关，它按一张内置的 `op` 列表决定每个 `op` 用什么精度。

```
with torch.autocast("cuda", dtype=torch.bfloat16):
    logits = model(x)          # matmul / conv / linear → bf16
    loss = criterion(logits, y) # cross_entropy → 仍走 fp32
loss.backward()               # autocast 区间外！
```

哪些 op 在 autocast 下仍走 fp32

不是所有 `op` 都降精度。低精度不安全的两类 `op` 被 `autocast` 强制留在 `fp32`：

- 数值范围敏感的：`exp`、`pow`、`log`、`softmax`、`log_softmax`、`cross_entropy`、`nll_loss`、`binary_cross_entropy_with_logits`
- 归约类：`sum`、`norm`、`layer_norm`、`batch_norm`、`group_norm`——归约会把误差累积放大，`fp16` 累加几千个数很快就失精度

所以你不需手动给 `LayerNorm` 或 `loss` 加 `.float()`，`autocast` 已经处理了。反过来，不要把 `backward()` 放进 `autocast` 块：反向的 `dtype` 是前向记录下来的，`autocast` 只需要包住 `forward` 和 `loss`。

Warning. `nn.BCELoss` 在 `autocast` 下会直接报错（“unsafe to autocast”），因为 `sigmoid` 输出在 `fp16` 下会饱和到精确的 0 或 1，`log(0)` 得到 `-inf`。改用 `BCEWithLogitsLoss`。

bf16 vs fp16：为什么只有 fp16 需要 GradScaler

	指数位	尾数位	后果
fp32	8	23	基准
fp16	5	10	动态范围只到约 6×10^{-5} ，小梯度直接 underflow 成 0
bf16	8	7	动态范围与 fp32 相同，精度低但不会 underflow

激活的梯度天然很小（尤其深网络的浅层）。`fp16` 下大量梯度落到 `6e-5` 以下就变成 0，梯度信息直接丢掉——这不是精度问题，是完全丢失。`bf16` 牺牲尾数换来和 `fp32` 一样的指数范围，所以不会有这个问题，也就不需要 `GradScaler`。

选择规则：Ampere 及更新（A100 / H100）一律用 `bf16`，代码更简单、不会有 `scale` 抖动；只有 V100 / T4 这类不支持 `bf16` 的卡才用 `fp16 + GradScaler`。

GradScaler 的工作原理

思路是“把梯度整体搬到 `fp16` 能表示的区间里再搬回来”：

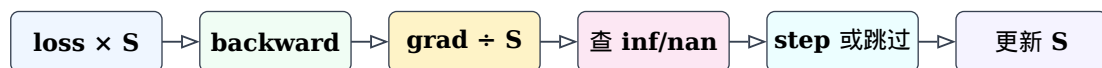


图 10 GradScaler 的一个 step。查到 inf/nan 就整步跳过并把 S 减半，连续 2000 步正常则把 S 翻倍。

1. `scaler.scale(loss)` 把 loss 乘一个缩放因子 S (默认初值 65536)。链式法则下所有梯度都被同倍放大，小梯度被抬进 fp16 的可表示区间。
2. `scaler.step(optimizer)` 先把梯度除回 S (`unscale_`)，再检查有没有 `inf / nan`。
3. 有 $\rightarrow$ 说明 S 太大导致溢出，整个 **step** 被跳过 (不调 `optimizer.step()`)，并在 `update()` 里把 S 乘 `backoff_factor` (默认 0.5)。
4. 没有 $\rightarrow$ 正常 `optimizer.step()`；连续 `growth_interval` (默认 2000) 步都没溢出，就把 S 乘 `growth_factor` (默认 2.0)，试探更大的范围。

所以训练刚开始会看到前几十步被跳过、`scaler.get_scale()` 从 65536 一路降下来，这是正常的自适应过程，不是 bug。但如果一直在跳步 (`get_scale()` 掉到个位数还在降)，说明梯度里有真的 `inf / nan`，去查数据和 loss，不是 scaler 的问题。

`unscale_` 与 `grad clipping` 的顺序

Warning. 必须先 `unscale_` 再 `clip`。这是 AMP 里最高频的面试题，也是最常见的静默错误。

梯度在 `backward()` 之后是被放大 S 倍的。直接对放大后的梯度做 `clip_grad_norm_(params, 1.0)`，实际上是把梯度裁到了“真实范数 $1/S$ ”—— $S = 65536$ 时相当于 clip 到 $1.5e-5$ 。每一步的更新量都被压到几乎为零，loss 曲线看起来“在收敛但极慢”，没有任何报错。

```

scaler.scale(loss).backward()
scaler.unscale_(optimizer)                # grad ÷= S
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # 现在是真实尺度
scaler.step(optimizer)                    # 不会重复 unscale
scaler.update()

```

`scaler.step()` 内部会检查该 `optimizer` 是否已经 `unscale` 过，所以不会除两次。但 `unscale_(optimizer)` 在一个 `step` 里对同一个 `optimizer` 只能调一次，调两次抛 `RuntimeError`。

完整组合：AMP + 梯度累积 + `clip` + `scheduler`

白板高频题，这段值得背下来：

```

import contextlib, torch

USE_FP16 = False # A100/H100 上用 bf16
amp_dtype = torch.float16 if USE_FP16 else torch.bfloat16
scaler = torch.amp.GradScaler("cuda", enabled=USE_FP16) # bf16 时 enabled=False
accum_steps = 4

model.train()
optimizer.zero_grad(set_to_none=True)

for step, (x, y) in enumerate(loader):

```



```

x, y = x.to(dev, non_blocking=True), y.to(dev, non_blocking=True)
is_last = (step + 1) % accum_steps == 0

# DDP: 非边界 micro-step 关掉梯度同步
sync_ctx = contextlib.nullcontext() if is_last else model.no_sync()

with sync_ctx:
    with torch.autocast("cuda", dtype=amp_dtype):
        loss = criterion(model(x), y) / accum_steps
        scaler.scale(loss).backward() # bf16 时 scale 是 no-op

if is_last:
    scaler.unscale_(optimizer) # 必须在 clip 之前
    grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer) # 有 inf/nan 则跳过
    scaler.update()
    optimizer.zero_grad(set_to_none=True)
    scheduler.step() # 与 optimizer.step() 同频

```

`enabled=False` 的 `GradScaler` 所有方法都退化成 `no-op`，所以这段代码切 `bf16` / `fp16` 只改一个 `flag`，不用写两条分支。这是面试里可以主动提的工程细节。要监控就打 `grad_norm`、`scheduler.get_last_lr()[0]`、`scaler.get_scale()` 这三个值。

Note. `scaler.step()` 跳过某步时 `scheduler.step()` 照样会走，等于“消耗了一步 `lr` 预算但没更新参数”。几万步的量级下可以忽略。

梯度裁剪

$$g \leftarrow g \cdot \min\left(1, \frac{\text{max_norm}}{\|g\|_2 + \varepsilon}\right), \quad \|g\|_2 = \sqrt{\sum_p \|g_p\|_2^2}$$

`clip_grad_norm_` 的语义是全局范数：把所有参数的梯度当成一个拼起来的长向量，算它的 `L2` 范数，超过 `max_norm` 就整体等比缩放。方向不变，只改长度。

`clip_grad_value_(params, v)` 则是逐元素 `clamp(-v, v)`，会改变梯度方向（大分量被砍、小分量不动），几乎只在 `RNN` 里还偶尔见到。现代训练一律用 `clip_grad_norm_`。

它的返回值是裁剪前的总范数（一个 GPU 上的标量 `tensor`，别在 `hot loop` 里 `.item()`），是免费的监控信号：稳定贴在 `max_norm` 上说明一直在被裁、`clip` 值设小了；突然尖刺到几百通常是脏数据或 `lr` 过大；掉到接近 0 说明梯度消失，或者你在 `clip` 之前忘了 `unscale_`。

Note. `FSDP` 下不要用 `torch.nn.utils.clip_grad_norm_` ——每个 `rank` 只持有梯度的一个 `shard`，各自算局部范数得到的全局范数是错的。要用 `FSDP` 实例自己的 `model.clip_grad_norm_(max_norm)`，它内部会 `AllReduce` 汇总范数。`DDP` 不受影响，因为 `backward()` 结束时每个 `rank` 的梯度已经完整且同步过了。

优化器

更新公式

SGD + momentum (PyTorch 的形式, `dampening=0`) :

$$\begin{aligned}b_t &= \mu b_{t-1} + g_t \\p_t &= p_{t-1} - \eta b_t\end{aligned}$$

注意 PyTorch 不对 g_t 乘 $(1 - \mu)$ (那是另一种常见写法)。所以换 μ 时有效步长也跟着变, `momentum=0.9` 的有效 lr 大约是 `momentum=0` 的 10 倍。

Adam :

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t && \text{(一阶动量)} \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 && \text{(二阶动量)} \\\hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} && \text{(偏差校正)} \\p_t &= p_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}\end{aligned}$$

偏差校正是必要的: $m_0 = v_0 = 0$, 前几步被严重低估 ($t = 1$ 时 $m_1 = 0.1g_1$), 不除 $1 - \beta_1^t$ 就等于自带一个隐式 warmup, 且 $\beta_2 = 0.999$ 下要几千步才恢复正常。

AdamW : 只改最后一行, weight decay 不进梯度, 直接作用在参数上。

$$p_t = p_{t-1} - \eta \lambda p_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

AdamW 与 Adam + L2 的区别

这是必问题。Adam(`weight_decay`= λ) 做的是 **L2** 正则: 把 λp 加进梯度,

$$g_t \leftarrow g_t + \lambda p_{t-1}$$

然后这一项和真实梯度一起被 $\sqrt{\hat{v}_t}$ 除。后果是衰减强度被自适应缩放污染了: 梯度历史大的参数 ($\hat{v}$ 大) 被除得多, 实际 decay 变弱; 梯度小的参数反而被 decay 得狠。这与“weight decay 应该均匀地把所有权重往 0 拉”的初衷相反。

AdamW 的 **decoupled weight decay** 把这一项挪到自适应缩放之外, 每个参数每步都被乘上固定的 $(1 - \eta\lambda)$, 与它的梯度历史无关。

实践结论: Transformer 训练一律用 AdamW。注意两者的 `weight_decay` 默认值不同 (Adam 是 0, AdamW 是 0.01), 而且同一个 λ 在两者下强度完全不同——从 Adam 迁到 AdamW 时不能照搬这个超参。

param_groups : 谁不该被 weight decay

bias 和归一化层的权重不该被 weight decay : 它们不是“连接强度”。把 LayerNorm 的 weight 往 0 拉等于削弱这一层的表达能力, 而 bias 只有一个自由度、正则它没有防过拟合的意义。判据很简单——所有 1-D 参数都不 decay。

```
decay = [p for p in model.parameters() if p.requires_grad and p.ndim > 1]
no_decay = [p for p in model.parameters() if p.requires_grad and p.ndim <= 1]

optimizer = torch.optim.AdamW(
    [{"params": decay, "weight_decay": 0.1},
     {"params": no_decay, "weight_decay": 0.0}],
    lr=3e-4, betas=(0.9, 0.95), eps=1e-8, fused=True,
)
```

param_groups 里每个 dict 可以覆盖任何超参(lr / betas / weight_decay), 分层 lr(backbone 小 lr、 head 大 lr) 是同一个机制。scheduler 会按组分别调 lr , 所以 get_last_lr() 返回的是一个 list。

foreach 与 fused

一个 7B 模型有几百个参数张量。朴素实现下 optimizer.step() 要为每个张量跑好几个 kernel(乘、加、除.....), 几千次 kernel launch 的 CPU 开销可能比实际计算还长。

版本	核心思路	相对成本 / 收益
for-loop	逐参数逐 op 起 kernel	kernel 数 = O(参数数 × op 数)
foreach (默认)	multi-tensor apply , 一个 kernel 处理一批张量	kernel 数降到 O(op 数) , 峰值显存略高
fused	整条更新公式融进一个 kernel , 读写各一次	kernel 数最少 , 且省 HBM 往返

foreach=True 是 CUDA 上的默认行为(参数 dtype/device 一致时)。优化器 step 是纯 memory-bound 的, 所以 fused=True 省掉的中间量 HBM 往返是实打实的收益, 但要显式打开, 且限制更多: 只支持 CUDA 上的 float 参数, 不支持 sparse 梯度, 不能配 differentiable=True。不确定的话先用默认的 foreach , 再试 fused 并对比 step time。

学习率调度

warmup 为什么必要

三个独立的原因, 能说清任意两个就够:

1. **Adam** 的二阶动量在开头不可靠。 $\hat{v}_t$ 只由前几个 batch 估出, 方差极大; 分母不准则更新量不准, 此时 lr 若已是峰值, 一步就能破坏初始化。这也是 RAdam 的出发点。
2. 大 **batch** 要配大 **lr**, 而大 **lr** 在初期最危险。随机初始化的网络梯度方向也接近随机, 沿随机方向走一大步没有任何好处。
3. **Transformer** 的结构性原因。残差 + LayerNorm 在初期对 lr 极敏感, Post-LN 不加 warmup 基本训不起来(Pre-LN 好一些但也建议加)。

典型配置: 总步数的 1%-5% 做线性 warmup , 之后 cosine 衰减到峰值的 10%。

```

from torch.optim.lr_scheduler import LinearLR, CosineAnnealingLR, SequentialLR

warmup, total = 2000, 100_000
sched = SequentialLR(optimizer, milestones=[warmup], schedulers=[
    LinearLR(optimizer, start_factor=1e-3, total_iters=warmup),
    CosineAnnealingLR(optimizer, T_max=total - warmup, eta_min=3e-5),
])

```

per-step 还是 per-epoch

Warning. 这是最容易写错、又最难发现的 **scheduler bug** : **scheduler** 的“单位”是 **scheduler.step()** 的调用次数，不是 **epoch** 也不是 **batch**。

```

# 错: T_max 按 step 数给, 却在 epoch 循环里调 step()
sched = CosineAnnealingLR(optimizer, T_max=100_000)      # 100k steps
for epoch in range(50):
    for batch in loader:
        ...; optimizer.step()
    sched.step()                                          # 只调了 50 次 → cosine 才走了 0.05%

```

结果是 lr 基本没衰减，训练末期还在用峰值 lr，loss 平台期怎么也下不去。反过来把 epoch 级的 `T_max=50` 配 per-step 调用，则 lr 在头 50 步就跌到底，之后全程接近 0。

规则：`T_max / total_iters` 的单位必须和 `step()` 的调用频率一致。`OneCycleLR` 和所有 warmup 方案都必须 per-step (`OneCycleLR` 在超出 `total_steps` 时会直接抛异常，算比较友好)。`ReduceLR0nPlateau` 是唯一的例外，它 per-epoch 调用且要传指标：`sched.step(val_loss)`。

自检方法：把 `scheduler.get_last_lr()[0]` 打进日志，看第一步和最后一步的值是否符合预期。这一行日志能挡住 90% 的 scheduler bug。

常用调度的选择：

调度	特点与场景
linear decay	从峰值线性降到 0 或某个下限。LLM 预训练常用，简单、可预测
cosine	先慢后快再慢，末期 lr 很小有利于收敛。视觉和 LLM 都是默认选择
<code>OneCycleLR</code>	warmup 到峰值再退火到极低，配合 momentum 反向摆动。小数据集快速收敛 (super-convergence)，必须 per-step
constant + warmup	不衰减。continual pretrain、或者你打算随时从中间 resume 时用
<code>ReduceLR0nPlateau</code>	指标不降就砍 lr。适合不知道总步数的场景，但对大规模训练太被动

Key insight. cosine 的一个实际约束：曲线依赖总步数。如果你训到一半想延长训练，原来的 cosine 已经衰减到很低，续训效果会很差；而且从 checkpoint resume 时必须把 `scheduler.state_dict()` 一起恢复，否则 lr 会跳回峰值。想保留“随时延长”的自由度，用 constant + warmup 或 WSD (warmup-stable-decay) 这类可以事后决定衰减点的方案。

loss 的几个坑

CrossEntropyLoss 吃 logits，不吃 probs

`nn.CrossEntropyLoss` 内部就是 `log_softmax` + `nll_loss`，所以它要的是原始 **logits**：`criterion(model(x), y)`，其中 `model(x)` 是 (N, C) 的原始分数、`y` 是 $(N,)$ 的 `int64` 类别下标。

Warning. 最经典的初学者 bug：模型末尾加了 `nn.Softmax(dim=-1)`，又用 `CrossEntropyLoss`，等于做了两次 softmax。它不报错，只是把已经在 $[0, 1]$ 的概率再压一次，logits 差距被极度压缩、梯度变平——训练能跑，但收敛慢得离谱、准确率上不去。同理 `nn.LogSoftmax` + `CrossEntropyLoss` 也是错的（该配 `NLLLoss`）。判断方法：分类模型的 `forward` 末尾不该有任何激活函数。

为什么要融在一起？`log_softmax` 用的是 `log-sum-exp` 稳定形式（先减 `max`），而单独 `log(softmax(x))` 在 logits 大时溢出、小时 `log(0)` 得 `-inf`。融合还省掉一次中间张量的 HBM 往返——词表 128k 时那个中间张量是 $(B, T, 128k)$ ，非常可观。

其他几个参数：

ignore_index 默认 `-100`。target 等于这个值的位置既不算 **loss** 也不进 `reduction="mean"` 的分母。padding 位的 label 设成 `-100` 就自动被跳过，这是变长序列训练的标准做法（见第 4 章的 `collate`）。

reduction `"mean"`（默认，按有效元素数平均）/ `"sum"` / `"none"`（逐元素 loss，做 per-sample 加权或 token 级分析时用）。

label_smoothing 把 one-hot target 软化成 $1 - \alpha$ 与 $\alpha/(C - 1)$ ，抑制过度自信，分类任务常设 0.1。
shape 约定 logits 要是 $(N, C, d_1, \dots)$ ，类别维在第 1 维。LM 训练里通常直接 flatten：`criterion(logits.reshape(-1, V), labels.reshape(-1))`。

BCEWithLogitsLoss 比 sigmoid + BCELoss 稳定

同样的融合逻辑。`BCEWithLogitsLoss` 用的是数值稳定形式：

$$\ell = \max(x, 0) - xy + \log(1 + e^{-|x|})$$

这个式子对任意的 $|x|$ 都不溢出。而 fp32 下 `sigmoid(x)` 在 x 超过 17 左右就返回精确的 1.0，接着 `BCELoss` 算 `log(1 - 1.0) = log(0) = -inf`，loss 变 `nan`、梯度全丢；fp16 下这个阈值更低。它还额外带 `pos_weight` 参数处理正负样本不均衡，这是相比手写组合的另一个好处。

checkpoint：存什么、怎么恢复

Warning. 只存 `model.state_dict()` 的 checkpoint 是不能续训的。恢复训练至少需要下面这些，缺任何一项都会让 `resume` 后的曲线出现可见的跳变。

```
torch.save({
    "model":      model.state_dict(),          # 不是 model 本身！
    "optimizer":  optimizer.state_dict(),      # Adam 的 m/v，缺了等于重新 warmup
    "scheduler":  scheduler.state_dict(),      # 缺了 lr 会跳回峰值
    "scaler":     scaler.state_dict(),         # fp16 的 scale 值
```

```

"epoch": epoch, "step": global_step,      # DistributedSampler.set_epoch 要用
"rng": {"torch": torch.get_rng_state(),    # dropout / shuffle 可复现
        "cuda": torch.cuda.get_rng_state_all()},
"config": vars(args),                     # 没有 config 的 ckpt 半年后就是废铁
}, path)

```

为什么只存 `state_dict()` 而不是整个 `model` : `torch.save(model)` 会 pickle 模型的类引用, 加载时必须能 import 到完全同名同路径的类——文件挪个目录、类改个名、重构一下模块结构, ckpt 就废了。而且 pickle 一个对象等于允许执行任意代码, `weights_only=True` 加载不了它。 `state_dict()` 只是纯粹的 `dict[str, Tensor]`, 与代码解耦。

```

ckpt = torch.load(path, map_location="cpu", weights_only=True)
model.load_state_dict(ckpt["model"])
optimizer.load_state_dict(ckpt["optimizer"])      # 先建好 optimizer 再 load
scheduler.load_state_dict(ckpt["scheduler"])
start_step = ckpt["step"] + 1

```

`map_location="cpu"` 不写的话 tensor 会被恢复到保存时所在的设备。8 卡训练里每个 rank 都 `torch.load` 一份, 全部落到 `cuda:0` → 第一张卡直接 OOM。习惯性写 `map_location="cpu"`, 再靠 `load_state_dict` 搬到各自的卡上。

`weights_only=True` torch 2.6+ 的默认值, 用受限 unpickler 只允许 tensor 和基础类型, 杜绝“加载别人的 ckpt 等于执行别人的代码”。副作用是自定义类 (包括某些 `argparse.Namespace`) 会加载失败, 此时把 config 单独存成 json, 别为图省事关掉这个开关。

Note. 两个 key 前缀问题: DDP 包装后 key 带 `module.` 前缀, `torch.compile` 后带 `_orig_mod.` 前缀, 跨形态加载会报一堆 missing/unexpected key。干净的做法是保存时就取原始 module。分布式场景更推荐 `torch.distributed.checkpoint`, 见第 22 章。

验证循环

```

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()                                # Dropout 关闭, BN 用 running
    stats
    total_loss, total_n = 0.0, 0
    for x, y in loader:
        x, y = x.to(dev, non_blocking=True), y.to(dev, non_blocking=True)
        with torch.autocast("cuda", dtype=torch.bfloat16):
            loss = criterion(model(x), y)
            total_loss += loss.item() * y.size(0)    # 按样本数加权, 别直接平均 batch 均值
            total_n += y.size(0)
    model.train()                                  # ← 切回来!
    return total_loss / total_n

```

`model.eval()` 和 `torch.no_grad()` 是两件独立的事, 必须都做:

- `model.eval()` 改的是 **module** 行为: Dropout 变恒等, BatchNorm 用 running mean/var 而不是 batch 统计。少了它, 验证结果会随 batch 组成波动, 而且 BN 的 running stats 会被验证数据污染。

- `torch.no_grad()` 改的是 **autograd** 行为：不建图、不存激活。少了它，结果数值上完全正确，但会白存一整轮激活，验证集稍大就 OOM。

`inference_mode` 与 `no_grad`

`torch.inference_mode()` 比 `no_grad()` 更激进：除了不建图，它还关掉了 **version counter** 和 **view tracking**——这两个机制是 autograd 用来检测“你原地改了一个 backward 需要的张量”的。关掉它们省下每个 op 的一点簿记开销，小 kernel 密集的推理里有可测量的收益。

代价是它产出的张量被标记为 **inference tensor**，一旦进入需要 autograd 的计算就报 `RuntimeError: Inference tensors cannot be saved for backward`。

```
with torch.inference_mode():                # 纯推理 / 独立验证循环，更快
    preds = model(x)

with torch.no_grad():                       # 蒸馏 teacher：输出要回到训练图
    teacher_logits = teacher(x)
loss = kd_loss(student(x), teacher_logits)  # 换成 inference_mode 这行就报错
```

规则：张量会不会流回训练图？会（伪标签、蒸馏 teacher、RL rollout）→ `no_grad`；不会 → `inference_mode`。

常见训练 bug 清单

Warning. 忘记 `zero_grad()`。梯度会跨 step 累加，等价于一个不断变大的 batch + 不断放大的 lr，loss 先降后炸。症状是“前几十步正常，然后 loss 变 nan”。

Warning. 在循环里累加 **loss** 张量。 `total_loss += loss` 累加的是带 **autograd** 图的张量，整个 epoch 的计算图都被 hold 住不释放，显存线性增长直到 OOM。

正确写法是 `total_loss += loss.item()`（取出 python float，会触发一次 GPU 同步）或 `total_loss += loss.detach()`（留在 GPU 上不同步，最后再 `.item()`）。注意 `.item()` 强制同步 CPU 和 GPU，hot loop 里每步都调会打断 CPU 的提前 launch、让 GPU 出气泡；高频日志用 `detach()` 攒着，隔 N 步再 `.item()`。`clip_grad_norm_` 的返回值同理。

Warning. `eval()` 之后忘记切回 `train()`。验证完直接继续训练，Dropout 全程关闭、BN 一直用 running stats。训练 loss 会看起来更低（Dropout 关了嘛），但正则化没了，泛化变差。这个 bug 极难发现，因为没有任何报错，指标也只是“稍微不对”。养成习惯：`evaluate()` 函数自己在返回前 `model.train()`，或者用 try/finally 保证。

Warning. `scheduler.step()` 与 `optimizer.step()` 顺序颠倒，以及 **scheduler** 的单位和调用频率不一致（见前面 scheduler 一节）。两个都不报错，只让 lr 曲线悄悄地错。

把 `retain_graph=True` 当创可贴用。遇到“Trying to backward through the graph a second time”就加 `retain_graph=True`，等于把显存泄漏当成修复。真正的原因通常是张量跨 step 复用了（RNN 的 hidden state 忘了 `detach()`），或者对同一个 loss 调了两次 `backward()`。先找根因。

面试考点

面试考点. Q1 : `optimizer.zero_grad(set_to_none=True)` 和 `=False` 有什么区别? 为什么前者是默认?

A: `False` 是给每个 `.grad` 跑一次 `zero_()`; `True` 直接置 `None`。三个好处: 省掉每个参数一个 `memset` kernel; `grad` buffer 被释放, `forward` 峰值期少占一份显存; 语义更对——梯度为 0 时 Adam / momentum 的更新量不是 0 (momentum 有残值、weight decay 仍在作用), 所以 `=False` 会让本 step 没参与计算的参数被继续推着走, `=True` 则让 `step()` 直接跳过它们。代价是 `.grad` 可能是 `None`, 自己写的梯度监控要过滤。

面试考点. Q2 : 梯度累积怎么写才对? 为什么 `loss` 要除以 `accum_steps`?

A: `zero_grad` 移到累积窗口边界, 每个 `micro-step` 只做 `(loss / accum_steps).backward()`, 攒满后 `clip` → `optimizer.step()` → `zero_grad` → `scheduler.step()`。除以 `accum_steps` 是因为 `reduction="mean"` 下每个 `micro-batch` 的 `loss` 已经是自己的均值, 直接累加会得到 `accum_steps` 倍的梯度, 等于偷偷放大了 `lr`。严格来说这要求各 `micro-batch` 大小相同; `token` 级 `loss` 要改用 `reduction="sum"` 除总 `token` 数。DDP 下还要加一条: 每次 `backward()` 都触发一轮梯度 AllReduce, 前 `accum_steps - 1` 次纯属浪费。用 `ddp_model.no_sync()` 包住非边界的 `micro-step`, 通信量降到 `1/accum_steps`; 最后一个 `micro-step` 必须在 `no_sync()` 之外, 否则梯度永远不同步、各 rank 参数 drift。

面试考点. Q3 : `bf16` 和 `fp16` 训练有什么区别? 为什么 `bf16` 不需要 `GradScaler`?

A: `fp16` 是 5 位指数 + 10 位尾数, 动态范围只到约 $6e-5$; `bf16` 是 8 位指数 + 7 位尾数, 动态范围和 `fp32` 一样。激活梯度天然很小, `fp16` 下大量梯度直接 underflow 成 0, 必须靠 `GradScaler` 把 `loss` 放大后再反向。 `bf16` 不会 underflow, 所以不需要 `scaler`; 代价是尾数只有 7 位, 精度更差, 但训练对精度的容忍度远高于对范围的容忍度。A100 / H100 上一律用 `bf16`。

面试考点. Q4 : `GradScaler` 是怎么工作的?

A: `scale(loss)` 把 `loss` 乘一个因子 S (初值 65536), 链式法则下所有梯度同倍放大, 小梯度被抬进 `fp16` 可表示区间。 `step(optimizer)` 先 `unscale_` 把梯度除回来, 再检查 `inf/nan`: 有就整步跳过并把 S 乘 0.5; 没有就正常 `step`, 连续 2000 步无溢出则把 S 翻倍。所以训练开头看到几十步被跳过、`scale` 一路下降是正常的自适应; 如果 `scale` 掉到个位数还在降, 是数据或 `loss` 里真有 `nan`。

面试考点. Q5 : AMP 下做梯度裁剪, `unscale_` 和 `clip` 的顺序是什么? 搞错会怎样?

A: 必须先 `scaler.unscale_(optimizer)` 再 `clip_grad_norm_`。 `backward()` 之后梯度是被放大 S 倍的, 直接 `clip` 到 `max_norm=1.0` 相当于把真实梯度裁到 $1/S$ ($S = 65536$ 时是 $1.5e-5$), 每一步的更新量都被压到接近 0。它不报任何错, 只表现为“`loss` 在降但慢得离谱”。 `scaler.step()` 会检测到已经 `unscale` 过, 不会重复除; 但 `unscale_` 对同一 `optimizer` 一个 `step` 只能调一次。

面试考点. Q6 : AdamW 和 Adam + `weight_decay` 的区别?

A : `Adam(weight_decay= $\lambda$ )` 是 L2 正则——把 λp 加进梯度，于是这一项也被 $\sqrt{\hat{v}}$ 除，衰减强度被自适应缩放污染：梯度历史大的参数实际 decay 反而弱。AdamW 是 **decoupled weight decay**，把 $-\eta\lambda p$ 直接作用在参数上，与梯度历史无关，每个参数每步都乘固定的 $(1 - \eta\lambda)$ 。Transformer 训练一律用 AdamW，且从 Adam 迁过来时不能照搬 λ （两者默认值也不同：0 vs 0.01）。

面试考点. Q7 : warmup 为什么必要？

A : 三个原因。一是 Adam 的 $\hat{v}_t$ 在前几步只由极少样本估出，方差极大，分母不平时大 lr 一步就能破坏初始化（RAdam 就是针对这点）。二是大 batch 要配大 lr，而初期梯度方向接近随机，沿随机方向走大步没有收益只有风险。三是 Transformer 的残差 + LayerNorm 结构在初期对 lr 极敏感，Post-LN 不加 warmup 基本训不起来。典型配置是总步数的 1%-5%。

面试考点. Q8 : `CrossEntropyLoss` 前面能不能加 softmax？

A : 不能。它内部就是 `log_softmax + nll_loss`，要的是原始 logits。加了 softmax 等于做两次，logits 差距被极度压缩、梯度变平，不报错但收敛极慢。融合的另外两个理由：`log_softmax` 用 `log-sum-exp` 稳定形式，避免 `log(0)` 和溢出；省掉 (B, T, V) 这个巨大中间张量的 HBM 往返。同理 `BCEWithLogitsLoss` 优于 `sigmoid + BCELoss`——后者在 `logit` 大于 17 左右时 `sigmoid` 饱和成精确的 1.0，`log(1-1)` 得 `-inf`。

面试考点. Q9 : checkpoint 要存哪些东西？为什么不直接 `torch.save(model)`？

A : 至少要 `model / optimizer / scheduler / scaler` 的 `state_dict()`，加上 `epoch` 和 `global step`，再加 RNG state 和 `config`。少了 `optimizer` 就丢了 Adam 的 `m/v`，`resume` 后相当于重新 warmup；少了 `scheduler` 则 lr 跳回峰值。不存整个 `model` 是因为 `torch.save(model)` `pickle` 的是类引用，改个类名或挪个文件 `ckpt` 就废了，而且 `pickle` 等于允许执行任意代码，`weights_only=True` 加载不了。加载时一定要写 `map_location="cpu"`，否则 8 个 rank 的 tensor 全落到 `cuda:0` 上 OOM。

面试考点. Q10 : `no_grad` 和 `inference_mode` 有什么区别？

A : `no_grad` 只是不建图。`inference_mode` 还额外关掉 version counter 和 view tracking（`autograd` 用来检测非法原地修改的簿记），所以更快，但产出的张量是 `inference tensor`，一旦进入需要 `autograd` 的计算就报 `Inference tensors cannot be saved for backward`。判断标准：张量会不会流回训练图？蒸馏 teacher 输出、伪标签、RL rollout 都会 → 用 `no_grad`；纯推理和独立的验证循环 → 用 `inference_mode`。另外别忘了 `model.eval()` 是另一件独立的事，管的是 Dropout 和 BN 的行为，两者都得做。

Autograd：动态反向图的完整机制

面试问 PyTorch，绕不开 autograd。而且问法是分层的：初级问“zero_grad 为什么要调”，中级问“retain_graph 什么时候要开”，高级问“你自己写过 autograd.Function 吗，ctx.needs_input_grad 干什么用”。这一章把这条链从数学(VJP)一直讲到实现(version counter、grad_fn.next_functions)，目标是：面试官往任何一层深挖，你都有话说。

本章只讲 autograd 引擎本身。它在 dispatcher 里是怎么一层 key 的，见第 7 章；activation 显存怎么算，见第 8 章；torch.compile 下的 AOTAutograd 怎么把这张图提前 trace 出来，见第 13 章。

反向图长什么样

autograd 图是 **forward** 时动态构建的。每执行一个 op，如果它的输入里有任何一个 requires_grad=True，PyTorch 就在输出 tensor 上挂一个 grad_fn 节点，并让这个节点的 next_functions 指回输入的 grad_fn。forward 跑完，反向图就已经建好了——backward() 只是沿着这些指针走一遍。

```
import torch

x = torch.randn(4, 8, requires_grad=True)
W = torch.randn(8, 16, requires_grad=True)

h = x @ W          # h.grad_fn = <MmBackward0>
r = h.relu()       # r.grad_fn = <ReluBackward0>
loss = r.sum()      # loss.grad_fn = <SumBackward0>

loss.grad_fn.next_functions # ((<ReluBackward0>, 0),)
r.grad_fn.next_functions   # ((<MmBackward0>, 0),)
h.grad_fn.next_functions   # ((<AccumulateGrad>, 0), (<AccumulateGrad>, 0))
```

最后一行是关键：MmBackward0 的两个输入 x 和 W 都是叶子，所以它们对应的节点是 AccumulateGrad —— 一个“把算出来的梯度加到 .grad 上”的特殊节点。整张图的叶子节点全是 AccumulateGrad。

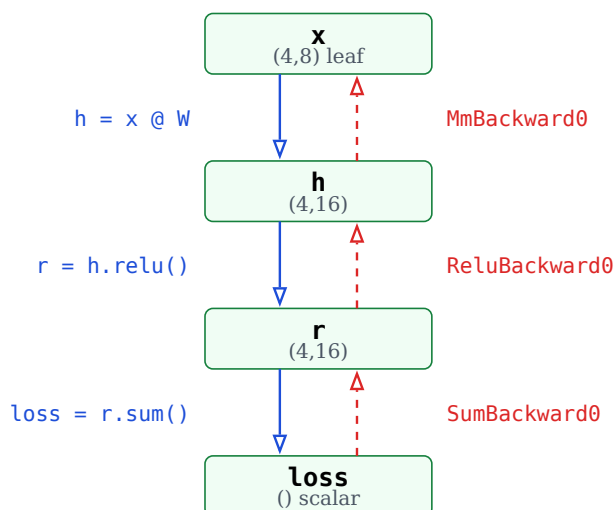


图 11 `loss = (x @ W).relu().sum()` 的 autograd 图。蓝色实线是 forward 建图方向, 红色虚线是 `backward()` 的执行方向。`W` 那一支没画出来, 它挂在 `MmBackward0` 的第二个 `next_functions` 上。

Key insight. “动态图”的准确含义不是“每次 forward 重新 trace”, 而是反向图是 **forward** 的副产品: 没有单独的建图阶段, `grad_fn` 是在每个 op 返回时就地挂上去的。所以 Python 里的 `if / for / 变长循环` 天然被支持 —— 走到哪条分支, 图就长成什么样。代价是每步都要重建, 这也是 `torch.compile` 要用 `AOTAutograd` 把图提前抓出来的原因。

leaf、requires_grad、.grad 三者的关系

leaf tensor (叶子) 的定义: `grad_fn is None` 的 tensor。两类: 用户直接创建的 (`torch.randn(...)`、`nn.Parameter`), 以及 `requires_grad=False` 的任何 tensor (它压根不参与建图)。

```
x = torch.randn(3, requires_grad=True)
x.is_leaf          # True
y = x * 2
y.is_leaf          # False — y 是运算产生的
z = torch.randn(3)
z.is_leaf          # True — requires_grad=False 也算叶子
```

三者的关系一句话说清: `requires_grad=True` 决定这个 **tensor** 要不要参与建图; `is_leaf` 决定 **backward** 的梯度要不要写进 `.grad`。只有同时满足 `is_leaf` and `requires_grad` 的 tensor, `backward` 才会往它的 `.grad` 里累加。

为什么这么设计? 因为中间结果的梯度在训练里没人要。一个 70B 模型 forward 会产生几万个中间 tensor, 如果每个都存 `.grad`, 显存直接翻倍。优化器只需要参数 (叶子) 的梯度。

要看中间结果的梯度 (调试梯度爆炸/消失时常做), 用 `retain_grad()`:

```
x = torch.randn(3, requires_grad=True)
h = x * 2
h.retain_grad()      # 必须在 backward 之前调
(h ** 2).sum().backward()
```

```
print(h.grad)          # tensor([...]) — 拿到了
print(x.grad)          # 照常
```

Note. `retain_grad()` 只是让引擎在这个节点上额外挂一个 hook 把梯度存下来，不改变图结构，也不影响其他节点的梯度值。它的显存代价 = 这个 tensor 大小。调试完记得去掉。

requires_grad 的传播规则

规则只有一条：一个 **op** 的输出 `requires_grad = any(输入.requires_grad)`。没有例外，没有隐式衰减。

```
a = torch.randn(3, requires_grad=True)
b = torch.randn(3)                # requires_grad=False
(a + b).requires_grad              # True
(b + b).requires_grad              # False
```

这条规则的直接后果：冻结一部分网络时，只 `requires_grad_(False)` 是不够省显存的。假设你冻了前 10 层、训后 10 层，前 10 层的参数确实不建图了，但只要输入本身携带梯度（比如你在做 LoRA、prompt tuning），或者第 11 层开始要梯度，那么第 11 层往后的 activation 全都要保存。真正省显存的是“冻结的前缀整段放进 `no_grad()`”：

```
with torch.no_grad():
    feats = frozen_backbone(x)    # 这一段不建图，activation 用完即释放
out = head(feats)                 # 从这里开始建图
out.sum().backward()
```

Warning. `nn.Module.requires_grad_(False)` 只作用于参数和 **buffer**，不作用于输入。冻 backbone 却发现显存没降，八成是没用 `no_grad()` 把这一段包起来，或者 backbone 后面还有 `requires_grad=True` 的 adapter 让整条链继续建图。

backward 的执行：拓扑序与“累加”

`loss.backward()` 做的事：

1. 从 `loss.grad_fn` 出发，按 `next_functions` 做一次反向拓扑排序。一个节点只有当它所有下游（**forward** 意义上的消费者）的梯度都到齐了才能执行——这就是为什么残差连接、多头共享输入这种“一个 tensor 被用了两次”的情况能正确求和。
2. 逐个执行 `grad_fn`。每个 `grad_fn` 的输入是“输出的梯度”，返回值是“输入的梯度”，个数严格对应 forward 的输入个数。
3. 走到 `AccumulateGrad`（叶子）时，把梯度累加到 `.grad`：`p.grad = p.grad + g`，而不是 `p.grad = g`。
4. 默认释放中间的 buffer（`save_for_backward` 存的东西），图结构本身还在，但再 backward 一次会报错。

第 3 点是全书最高频的面试题来源：

```
for batch in loader:
    optimizer.zero_grad(set_to_none=True)  # ← 不写这行，梯度会跨 step 累积
    loss = model(batch).mean()
    loss.backward()
    optimizer.step()
```

面试考点。面试官几乎必问：“PyTorch 为什么不自动清梯度？”标准答案：因为累加是特性不是 **bug**。梯度累积 (gradient accumulation) 实现大 batch、多个 loss 分别 backward 合并、RNN 里 BPTT 分段回传，全都依赖这个语义。如果引擎自动清零，这些场景就得自己发明一套缓存。把清零交给 optimizer，是把策略留给用户。

`zero_grad(set_to_none=True)` 是 torch 2.0+ 的默认值：它把 `.grad` 置成 `None` 而不是填 0。好处有两个：省一次 `memset` (大模型上不是小数目)，以及让“没参与 forward 的参数”的 `.grad` 保持 `None`，optimizer 会跳过它们 (而 0 会被当成真梯度，带 weight decay 或 momentum 的 optimizer 还是会更新参数)。

梯度累积就是直接用这个语义：连续 backward N 次再 `step` 一次，每次的 loss 除以 N (否则等效学习率涨了 N 倍)。完整训练循环写法见第 5 章。

backward() 的参数与 torch.autograd.grad()

为什么非标量必须传 **gradient**

`backward()` 计算的不是 Jacobian，而是 **vector-Jacobian product (VJP)**。`loss.backward()` 之所以能省略参数，是因为 `loss` 是标量，上游向量默认取 `torch.ones()`。张量 `y` 没有天然的“上游向量”，所以必须显式给：

```
y = model(x)                # (B, C)，不是标量
y.backward(torch.ones_like(y)) # 等价于 y.sum().backward()
v = torch.randn_like(y)
y.backward(v)                # 计算  $v^T \cdot (dy/dx)$ 
```

不 传
就是这个报错：`RuntimeError: grad can be implicitly created only for scalar outputs`。修法要么 `.sum()` / `.mean()` 归约成标量，要么想清楚你的上游向量是什么。

torch.autograd.grad vs backward

```
g, = torch.autograd.grad(loss, [x], create_graph=False) # 返回 tuple，不写 x.grad
loss.backward()                                         # 写 x.grad，返回 None
```

区别有四点，面试里说全能加分：

	<code>backward()</code>	<code>torch.autograd.grad()</code>
返回值	<code>None</code>	梯度 tuple
是否写 <code>.grad</code>	写 (累加到叶子)	不写，不污染已有梯度
作用范围	整张图的所有叶子	只算你指定的 <code>inputs</code> ，其余分支被剪掉

典型用途	训练主循环	二阶导、gradient penalty、meta-learning、影响函数
------	-------	---

第三点是性能上的实际差别：`torch.autograd.grad(loss, [x])` 只会执行“从 `loss` 到 `x` 的路径上”的节点，其他分支不跑。做 gradient penalty 时用 `grad` 而不是 `backward` 能省掉一次对全部参数求导。

`allow_unused=True` 用于 `inputs` 里可能有没参与 forward 的 tensor：不加会报 “One of the differentiated Tensors appears to not have been used in the graph”，加了会返回 `None`。torch 2.x 里还有 `materialize_grads=True`，把那些 `None` 换成全零 tensor，写循环时省一堆判断。

`retain_graph`：什么时候必须开，为什么默认关

`backward` 走完一个节点，就把它 `save_for_backward` 的中间 buffer 释放掉。所以对同一张图 `backward` 第二次会报：

```
RuntimeError: Trying to backward through the graph a second time
(or directly access saved tensors after they have already been freed).
```

需要 `retain_graph=True` 的真实场景只有一类：同一张 **forward** 图要被回传多次。

```
h = shared_encoder(x)                # 共享的 forward
loss_a = head_a(h).mean()
loss_b = head_b(h).mean()

loss_a.backward(retain_graph=True)    # 第一次，保留 buffer
loss_b.backward()                    # 第二次，用完释放
```

但上面这个例子其实有更好的写法：`(loss_a + loss_b).backward()` —— 一次 `backward`，梯度天然求和，不用留 buffer。绝大多数 `retain_graph=True` 都是误用，是把“我不知道为什么报错”用一个开关压住了。

Warning. 滥用 `retain_graph=True` 的典型翻车：在训练循环里对每个 batch 都开，结果 activation 永远不释放，几十个 step 后 OOM。更隐蔽的一种是把上一步的 `loss` 或 hidden state 跨 step 带进下一步（RNN 里常见），整条历史图都被保活。RNN 的正确做法是 truncated BPTT：`hidden = hidden.detach()` 切断历史。

自检方法：如果你写了 `retain_graph=True`，问自己“我到底 `backward` 了几次”。答案是 1 次，就该删掉它。

`create_graph=True` 与二阶导

`create_graph=True` 让 **backward** 过程本身也建图，于是梯度 `g` 变成一个有 `grad_fn` 的普通 tensor，可以对它再求导。它隐含 `retain_graph=True`。

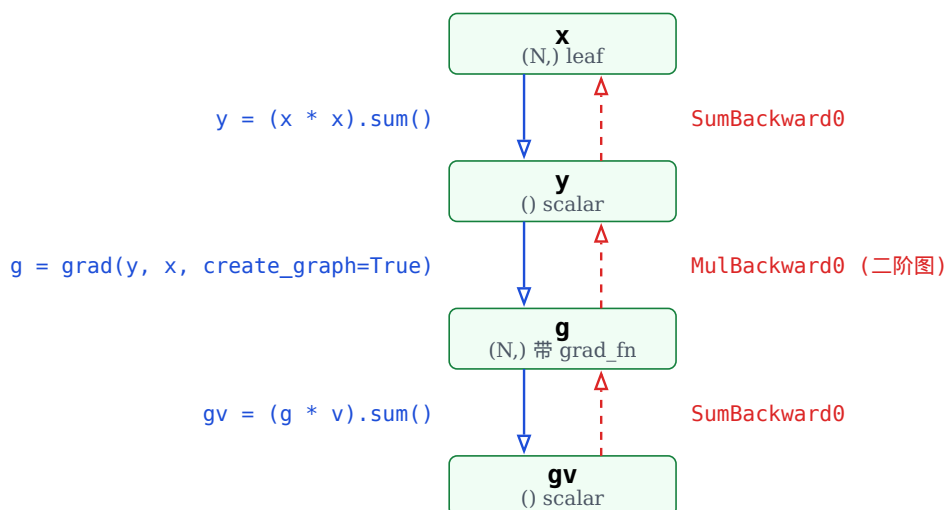


图 12 `create_graph=True` 时，一阶梯度 `g` 自己也挂上了 `grad_fn`，于是对 `gv` 再 backward 就得到 Hessian-vector product。这是所有二阶方法的公共骨架。

Hessian-vector product (HVP) 完整例子。直接算 Hessian 是 $O(N^2)$ 存储，实际永远算 HVP：

```
import torch

def hvp(f, x, v):
    """返回 H @ v，其中 H = d^2 f / dx^2。只需两次反向，不实体化 Hessian。"""
    x = x.detach().requires_grad_(True)
    y = f(x)
    g, = torch.autograd.grad(y, x, create_graph=True)  # 一阶梯度，带图
    hv, = torch.autograd.grad((g * v).sum(), x)        # 对 g·v 再求导
    return hv

x = torch.randn(5)
v = torch.randn(5)
f = lambda t: (t ** 3).sum()  # f'' = 6t, 所以 H = diag(6x)
print(torch.allclose(hvp(f, x, v), 6 * x * v))  # True
```

原理： $\nabla_x(g^T v) = \nabla_x((\nabla f)^T v) = H v$ (v 与 x 无关，所以能提出来)。

Gradient penalty (WGAN-GP 的核心)：

```
def gradient_penalty(D, real, fake):
    eps = torch.rand(real.size(0), 1, 1, 1, device=real.device)
    x_hat = (eps * real + (1 - eps) * fake).requires_grad_(True)
    d_hat = D(x_hat)
    g, = torch.autograd.grad(
        outputs=d_hat, inputs=x_hat,
        grad_outputs=torch.ones_like(d_hat),  # 非标量输出，必须给上游向量
        create_graph=True,                    # penalty 要参与主 loss 的 backward
        retain_graph=True,
    )
    return ((g.flatten(1).norm(2, dim=1) - 1) ** 2).mean()

loss = d_fake.mean() - d_real.mean() + 10.0 * gradient_penalty(D, real, fake)
loss.backward()  # 这里会走二阶路径
```

Warning. `create_graph=True` 的两个坑。第一，显存：反向图被完整保留，峰值大致翻倍，而且很容易忘记它隐含 `retain_graph=True`，导致图不释放。第二，不是所有 **op** 都二阶可导：很多 fused kernel 在二阶时会报 “derivative for ... is not implemented”。自己写的 `autograd.Function` 也一样——只有当它的 `backward` 本身完全由可微 op 组成时才支持二阶；标了 `@torch.autograd.function.once_differentiable` 的则明确声明“不支持”。遇到这类报错，通常只能把那一段换成纯 op 组合。

数学视角：链式法则与 VJP

设 forward 是一串函数复合 $L = f_n \circ \dots \circ f_1(x)$ ，中间量 $u_k = f_k(u_{k-1})$ 。链式法则给出

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial u_n} \cdot \frac{\partial u_n}{\partial u_{n-1}} \cdot \frac{\partial u_{n-1}}{\partial x}$$

这串 Jacobian 连乘从左往右算，还是从右往左算，结果一样，代价天差地别。

- 反向模式 (**reverse mode**)：从左往右，每一步都是“行向量 $\times$ 矩阵”，即 VJP。始终只操作向量，代价 $\approx$ 一次 forward 的常数倍。
- 前向模式 (**forward mode**)：从右往左，每一步是“矩阵 $\times$ 列向量”，即 JVP。一次只能得到“输出对某一个输入方向”的导数。

设输入维度 n 、输出维度 m 。要拿到完整 Jacobian，反向模式要跑 m 次，前向模式要跑 n 次。深度学习里 $n \approx 10^9$ （参数量）， $m = 1$ （标量 loss），所以反向模式赢了九个数量级。这就是 autograd 叫 “backpropagation” 的全部理由。

$$\bar{u}_{k-1} = J_k^T \bar{u}_k, \quad J_k = \frac{\partial u_k}{\partial u_{k-1}}$$

关键实现细节：每个 `grad_fn` 从来不显式构造 J_k ，它只实现“给我上游向量 $\bar{u}_k$ ，我返回 $J_k^T \bar{u}_k$ ”。比如 `MmBackward0` 对 $y = xW$ 的 VJP 是 $\bar{x} = \bar{y}W^T$ 、 $\bar{W} = x^T \bar{y}$ ——两次 matmul，从没出现过那个 $(BH) \times (BO)$ 的巨大 Jacobian。

Note. 前向模式在 PyTorch 里也有：`torch.func.jvp` / `torch.autograd.forward_ad`。它在“输入维度小、输出维度大”时才划算，实际用途主要是算 JVP 做数值验证，以及某些 ODE/物理仿真场景。面试里提一句“PyTorch 也有 forward-mode AD，只是深度学习的维度形状不适合它”就够了。

detach / no_grad / inference_mode / enable_grad

这四个东西经常被混着用，但语义差别很实在：

机制	是否建图	in-place 限制	version counter	性能
<code>detach()</code>	新 tensor 不带梯度，与原图断开	与原 tensor 共享 storage，改它会改到原 tensor	与原 tensor 共享同一个，污染仍会被检测到	零开销，不 copy

<code>no_grad()</code>	块内所有 op 都不建图	允许,手写参数更新常用	照常维护	省掉建图与 activation 保存
<code>inference_mode()</code>	不建图,新建的是 inference tensor	出了块就禁止 in-place	不维护,也不走 view 记账	比 <code>no_grad</code> 更快
<code>enable_grad()</code>	强制建图,可嵌在 <code>no_grad</code> 内反转	无额外限制	照常	无

```
# no_grad 的经典用途: 手写参数更新
with torch.no_grad():
    for p in model.parameters():
        p -= lr * p.grad          # 直接改 leaf, 不建图

# enable_grad 嵌在 no_grad 里反转 (eval 时要做 gradient-based 分析会用)
with torch.no_grad():
    feats = backbone(x)
    with torch.enable_grad():
        feats = feats.detach().requires_grad_(True)
        score = probe(feats).sum()
        g, = torch.autograd.grad(score, feats)
```

Warning. `inference_mode()` 里创建的 tensor 带一个特殊标记,不能被 **autograd** 保存。经典翻车: dataloader 或预处理函数被包进 `inference_mode()`, 产出的 tensor 喂进训练,报错

```
RuntimeError: Inference tensors cannot be saved for backward.
```

修法: 要么把预处理换成 `torch.no_grad()`, 要么在出块前 `.clone()` 成普通 tensor。规则是简单的: 纯推理服务用 **`inference_mode()`**, 训练过程中的任何局部禁梯度用 **`no_grad()`**。

`detach()` 与 `detach_()` 的差别: 前者返回新 tensor (原 tensor 不变), 后者原地把当前 tensor 变成叶子 (切断它的 `grad_fn`)。RNN 里的 `hidden = hidden.detach()` 用前者。`.data` 是历史遗留的第五种写法, 它绕过 **version counter**, 所以能悄悄制造错误梯度而不报错 —— 新代码一律不要用 `.data`。

`torch.autograd.Function` : 自定义前反向

什么时候需要自己写: 数学上有闭式导数比 `autograd` 逐 op 展开更快更稳 (比如 softmax-cross-entropy 融合); forward 里调了不可微的外部 kernel (自定义 CUDA、Triton); 想在反向里塞通信 (DDP/TP 里的 all-reduce) 或者改梯度 (梯度反转)。

```
class ScaledTanh(torch.autograd.Function):

    @staticmethod
    def forward(ctx, x, scale, clamp=False):
        y = scale * torch.tanh(x)
        # save_for_backward 只能存 tensor; 非 tensor 挂 ctx 属性。
        # 存输出而不是输入: tanh 的导数能用输出反推, 省一份显存。
        ctx.save_for_backward(y)
```

```

        ctx.scale = scale
    return y

    @staticmethod
    def backward(ctx, grad_out):
        (y,) = ctx.saved_tensors
        grad_x = None
        # 只在真的需要时才算，省掉无用计算
        if ctx.needs_input_grad[0]:
            # d/dx [s*tanh(x)] = s*(1 - tanh(x)^2) = s - y^2/s
            grad_x = grad_out * (ctx.scale - y * y / ctx.scale)
        # 返回值个数必须 == forward 的输入个数 (不含 ctx)
        # scale 是 float、clamp 是 bool，都不需要梯度 → None
        return grad_x, None, None

y = ScaledTanh.apply(x, 2.0)

```

必须记住的四条规则：

1. `forward` / `backward` 都是 `@staticmethod`，用 `Fn.apply(...)` 调用，不要直接 `Fn.forward(...)`（那样不建图）。
2. `backward` 的返回值个数必须等于 `forward` 的输入个数（不含 `ctx`）。不需要梯度的位置返回 `None`。这是最高频的错误：“expected N gradients but got M”。
3. 需要在 `backward` 里用到的 tensor 一律走 `ctx.save_for_backward(...)` + `ctx.saved_tensors`，不要直接 `ctx.x = x`。前者会走 version counter 检查，能抓到“存下来的东西被 in-place 改了”；后者绕过检查，静默给你错梯度。
4. `ctx.needs_input_grad` 是一个 bool tuple，长度等于 `forward` 输入个数。用它跳过不需要的分支。

新版的 `setup_context` 写法

torch 2.0+ 推荐把“存 context”从 `forward` 里拆出来，这样 `forward` 是一个纯函数，能被 `torch.func (vmap / functorch)` 和 `torch.compile` 正确处理：

```

class MyOp(torch.autograd.Function):

    @staticmethod
    def forward(x, scale):                # 注意：没有 ctx 参数
        return scale * x.sigmoid()

    @staticmethod
    def setup_context(ctx, inputs, output):
        x, scale = inputs
        ctx.save_for_backward(output)      # 存输出比存输入省显存
        ctx.scale = scale

    @staticmethod
    def backward(ctx, grad_out):
        (y,) = ctx.saved_tensors
        # d/dx [s*sigmoid(x)] = y * (1 - y/s)，用输出反推
        return grad_out * y * (1 - y / ctx.scale), None

```

两种写法都还支持。旧写法 (`forward(ctx, ...)`) 在纯训练场景没问题；写库、或者要跟 `vmap` / `torch.compile` 配合，就用 `setup_context`。

`mark_dirty` 与 `mark_non_differentiable`

- `ctx.mark_dirty(x)` : `forward` 里原地修改了输入时必须调。它告诉 `autograd` 去 bump 这个 tensor 的 version counter，否则后续会用到旧值算出错误梯度。
- `ctx.mark_non_differentiable(idx)` : `forward` 返回了不可微的输出 (比如 `argmax` 出来的整数索引、`mask`) 时调，`autograd` 就不会为它准备梯度、也不会在你没返回对应梯度时报错。

```
class InplaceRelu(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x):
        ctx.mark_dirty(x)          # 声明 x 被原地改了
        x.relu_()
        ctx.save_for_backward(x)
        return x
    @staticmethod
    def backward(ctx, g):
        (y,) = ctx.saved_tensors
        return g * (y > 0)
```

更多可直接背的模板 (梯度反转层、straight-through estimator、fused 实现) 在第 24 章。

用 `gradcheck` 验证自定义 `backward`

自己写的 `backward` 有没有算错，靠肉眼看不出来。 `torch.autograd.gradcheck` 用有限差分逼近数值梯度，跟你的解析梯度对比：

```
from torch.autograd import gradcheck

x = torch.randn(4, 5, dtype=torch.double, requires_grad=True)  # ← 必须 double
assert gradcheck(lambda t: ScaledTanh.apply(t, 2.0), (x,), eps=1e-6, atol=1e-4)

# 二阶 (只有当你的 backward 本身由可微 op 组成时才会通过)
from torch.autograd import gradgradcheck
assert gradgradcheck(lambda t: ScaledTanh.apply(t, 2.0), (x,))
```

Warning. `gradcheck` 必须用 `float64`。这是面试高频追问。原因：有限差分 $\frac{f(x+\varepsilon)-f(x-\varepsilon)}{2\varepsilon}$ 的误差里有一项正比于 $\frac{\text{机器精度}}{\varepsilon}$ 。float32 的机器精度是 $\approx 10^{-7}$ ，取 $\varepsilon = 10^{-6}$ 时这一项就有 10^{-1} 量级，噪声比信号还大，必然 fail。float64 的机器精度 $\approx 10^{-16}$ ，才有足够余量。

其他常见 fail 原因：函数在测试点不可导 (ReLU 在 0 附近、`abs`、`clamp` 边界 —— 换个输入点或加个偏移)；op 有随机性 (dropout，要么固定 `seed` 要么设 `nondet_tol`)；输入没设 `requires_grad=True`。

in-place 操作与 version counter

每个 tensor 的 storage 上带一个 **version counter**。任何 in-place op (后缀 `_` 的方法、`+=`、切片赋值 `x[0] = ...`) 都会把它 +1。 `save_for_backward` 存 tensor 时会顺手记下当时的版本号，

backward 取出来时比对 —— 不一致就报错。这是 PyTorch 能“动态图 + in-place 还不算错梯度”的机制保障。

两个报错长得像，成因完全不同：

Warning. 报 错
1: a leaf Variable that requires grad is being used in an in-place operation.

最小复现：

```
w = torch.randn(3, requires_grad=True)
w.add_(1)          # RuntimeError
```

成因：对叶子做 in-place，会让这个叶子的历史无法表达（它既是叶子又有了 `grad_fn`，矛盾）。真实场景是手写参数更新时忘了包 `no_grad`，或者做参数初始化/裁剪时直接改了 `nn.Parameter`。

修法：包进 `no_grad()`，或者对 `.detach()` 出来的视图操作。

```
with torch.no_grad():
    w.add_(1)          # OK
# 或
w.detach().add_(1)    # OK, 共享 storage
```

Warning. 报 错 2:
one of the variables needed for gradient computation has been modified by an inplace operation: [torch.FloatTensor [3]], which is output 0 of AddBackward0, is at version 1; expected version 0 instead.

最小复现：

```
x = torch.randn(3, requires_grad=True)
y = x * 2
z = y.pow(2)          # PowBackward0 存了 y (版本 0)
y.add_(1)             # y 的版本变成 1
z.sum().backward()    # RuntimeError
```

成因：某个 `grad_fn` 保存了 `y` 用于反向，`y` 却在 backward 之前被改了。注意不是所有 **in-place** 都会触发 —— 比如 `y.exp()` 的反向只需要输出，不需要 `y`，所以改 `y` 没事。这也是为什么这个 bug 常常“换个激活函数就好了”，很迷惑。

修法：把那次 in-place 换成 out-of-place (`y = y + 1`)，或在 in-place 之前 `y = y.clone()`。
定位手段：`torch.autograd.set_detect_anomaly(True)` 会打印出“是哪个 forward op 产生了这个坏梯度”的栈，代价是慢好几倍，只在 debug 时开。

典型来源：`nn.ReLU(inplace=True)` 接在需要保存输出的 op 后面、residual 里写了 `out += identity`、以及在 forward 里手动改 activation 做归一化。`out += identity` 换成 `out = out + identity` 就好。

Key insight. version counter 只在 **autograd** 记账 的路径上生效。`x.data` 和 `inference_mode()` 都绕过它 —— 前者是历史遗留 API，后者是设计上就不做记账（换性能）。

所以“用了 `.data` 之后梯度悄悄变错”是最难查的一类 bug：没有任何报错。新代码只用 `detach()`。

“梯度是 `None`” 排查清单

这是调试时最常遇到的现象，按这个顺序查基本一次命中：

原因	判定	修法
不是叶子	<code>t.is_leaf == False</code>	是中间量就用 <code>retain_grad()</code> ；本该是参数就检查是不是被重新赋值了
<code>requires_grad=False</code>	<code>t.requires_grad == False</code>	<code>t.requires_grad_(True)</code> ，参数检查有没有被 <code>requires_grad_(False)</code> 冻住
在 <code>no_grad</code> / <code>inference_mode</code> 里建的	<code>out.grad_fn is None</code>	把建图的那段挪出上下文
被 <code>detach()</code> / <code>.data</code> 断开	沿 <code>grad_fn</code> 往回走断在哪	删掉那次 <code>detach</code>
参数没参与 forward	该参数的 <code>grad</code> 恒为 <code>None</code>	检查分支逻辑；DDP 场景要 <code>find_unused_parameters=True</code> （见第 18 章）
optimizer 没管这个参数	参数不在 <code>param_groups</code> 里	构造 optimizer 前先把模块搬到设备、加完所有子模块
被 <code>set_to_none=True</code> 清了	<code>.grad is None</code> 且在 <code>step</code> 之后	正常行为，不是 bug

```
# 单个 tensor：把状态一次打全
print(t.is_leaf, t.requires_grad, t.grad_fn)

# 模块级批量检查：这些参数这一步根本没被更新
for n, p in model.named_parameters():
    if p.requires_grad and p.grad is None:
        print("no grad:", n)
```

activation checkpointing 的 autograd 视角

`torch.utils.checkpoint.checkpoint(fn, *args)` 做的事，用 autograd 的语言说：在 **forward** 时不保存 `fn` 内部的中间 **activation**，只保存输入；**backward** 走到这里时，先用保存的输入重跑一遍 `fn` 的 **forward**（这次建图），再正常反向。用一次额外的 forward 换掉整段的 activation 显存。

```
from torch.utils.checkpoint import checkpoint

class Block(torch.nn.Module):
    def forward(self, x):
        return self.mlp(self.attn(self.ln(x)) + x)

# 每个 block 只保存输入，内部 activation 全部丢弃
```

```
for blk in self.blocks:
    x = checkpoint(blk, x, use_reentrant=False)
```

`use_reentrant=False` 是新推荐实现 (torch 2.1+ 起不显式传会告警, 未来会变成默认)。它与旧的 `reentrant` 版本的差别:

	<code>use_reentrant=True</code> (旧)	<code>use_reentrant=False</code> (新)
实现方式	在 <code>backward</code> 里再调一次 <code>autograd.Function</code> , 嵌套进入引擎	基于 saved-tensor hooks, 不嵌套
输入没有 <code>requires_grad</code>	直接静默不算梯度	正常工作
<code>torch.autograd.grad()</code>	不支持 (只支持 <code>backward()</code>)	支持
<code>backward</code> hook	触发时机不对, 容易漏	正常
非 tensor 输出 / 变长输出	不支持	支持
early stop	无, 整段都要重算	有 (<code>early_stop=True</code> , 梯度算够就停)

RNG 一致性。 dropout 在 forward 和 recompute 时如果抽到不同的 mask, 梯度就是错的 (而且不会报错, 只表现为收敛变差)。`checkpoint` 默认 `preserve_rng_state=True`: forward 时把 CPU 和当前 CUDA device 的 RNG state 存下来, recompute 之前恢复回去, 跑完再切回来。代价是每个 checkpoint 段有一次 RNG state 的存取 (很便宜)。

Warning. 几个 checkpoint 的实际坑:

- `preserve_rng_state=True` 只保存“当前 device”的 RNG state。如果 `fn` 内部切了 device 或用了多个 device, 重算的随机数不保证一致。
- 被 checkpoint 的函数应当是确定性的。除了 RNG, 还包括: 不要在 `fn` 里读写会被外部改动的全局状态, 不要依赖上一次调用的副作用。`determinism_check="default"` 会检查重算的输出 shape/dtype 是否一致, 但不检查数值。
- 与 `torch.compile` 组合时行为在演进 (编译器可能自己决定重算哪些 op, 见第 13 章的 min-cut partitioner), 版本敏感, 升级 torch 后要重新 profile。

Key insight. 面试问“checkpointing 省了多少显存、花了多少时间”, 标准答法是给公式不给假数字: L 层均匀分段成 $\sqrt{L}$ 段时, activation 显存从 $O(L)$ 降到 $O(\sqrt{L})$, 额外计算约等于多一次 forward, 即总计算量从 $2F$ (forward + backward $\approx 2 \times$) 变成 $3F$ 量级, 因此吞吐损失通常在 20%–30%。具体数字必须自己 profile, 跟模型形状、是否 memory-bound 强相关。

hook: 在反向路径上插手

Tensor.register_hook

签名 `t.register_hook(fn)`, `fn(grad) -> Tensor or None`。返回非 `None` 就替换这个梯度, 返回 `None` 就只是旁观。触发时机: 这个 tensor 的梯度算好、往下游传之前。

```
# 用途 1: 监控 (不改梯度)
handle = h.register_hook(lambda g: print("grad norm:", g.norm().item()))
```

```

...
handle.remove()          # 记得摘，否则一直挂着

# 用途 2：梯度反转层（domain-adversarial training 的核心，一行搞定）
lambda_ = 0.5
feat.register_hook(lambda g: -lambda_ * g)

# 用途 3：per-tensor 梯度裁剪（比全局 clip_grad_norm_ 更早介入）
for p in model.parameters():
    p.register_hook(lambda g: g.clamp(-1.0, 1.0))

```

Note. 梯度反转层也可以写成 `autograd.Function`（forward 恒等、backward 取负），两种写法等价。hook 版本更短，`Function` 版本能作为一个可复用的 module 插进 `nn.Sequential`，也更容易被 `torch.compile` 处理。第 24 章给 `Function` 版本。

hook 的一个重要工程用途：**DDP** 的梯度同步就是挂在参数上的 **hook**。参数 `p` 的梯度一算出来，hook 立刻触发，把它标记进 bucket；bucket 满就发一个异步 all-reduce。这就是 DDP 能让通信与 backward 重叠的机制（见第 18 章）。

`Module.register_full_backward_hook`

签名 `fn(module, grad_input, grad_output)`，在这个 module 的输入梯度全部算好后触发。返回一个新的 `grad_input` tuple 就能替换。

```

def watch(mod, grad_in, grad_out):
    gi = grad_in[0]
    if gi is not None and not torch.isfinite(gi).all():
        print(f"NaN/Inf in grad_input of {mod.__class__.__name__}")

for m in model.modules():
    m.register_full_backward_hook(watch)  # 定位 NaN 从哪一层开始

```

`register_full_backward_hook` 与已废弃的 `register_backward_hook` 的区别：后者在 module 有多个输入/输出或内部有多个 op 时触发时机不正确（它实际挂在最后一个 op 上），新代码一律用 `full` 版本。

Warning. hook 有两个共同的坑。第一，不摘会泄漏：`register_hook` 返回的 `handle` 如果不 `remove()`，闭包捕获的对象（常常是整个 module 或一个大 tensor）就一直活着。第二，**hook** 里做 `.item()` / `print` 会强制同步 GPU，在训练主循环里挂满监控 hook 能把吞吐打掉一半。监控类 hook 建议按步数采样，比如每 100 步才真正算一次 norm。

面试考点

面试考点. **Q1**：为什么每个 step 都要 `optimizer.zero_grad()`？

A：因为 backward 是把梯度累加到 `.grad`，不是覆盖。累加是有意设计的：梯度累积做大 batch、多个 loss 分别回传、RNN 分段 BPTT 都依赖它。所以清零的时机交给用户。推荐 `set_to_none=True`，省一次 `memset`，而且能让没参与 forward 的参数保持 `None` 从而被 `optimizer` 跳过。

面试考点. Q2：什么是 leaf tensor？为什么只有它有 `.grad`？

A：grad_fn is None 的 tensor 是叶子，通常是用户创建的 tensor 和 nn.Parameter。backward 只把梯度写进“叶子且 requires_grad”的 `.grad`，因为中间 activation 的梯度训练里没人要，全存下来显存会翻倍。要看中间梯度调 `retain_grad()`。

面试考点. Q3：非标量 tensor 调 `backward()` 为什么报错？

A：autograd 算的是 VJP 不是 Jacobian，必须有一个上游向量。标量 loss 的上游向量默认取 1，所以能省略；非标量没有天然默认值，得自己传 `y.backward(v)`，语义是 $v^T \frac{\partial y}{\partial x}$ 。传 `ones_like(y)` 就等价于 `y.sum().backward()`。

面试考点. Q4：`retain_graph=True` 什么时候要开？为什么默认关？

A：只有“同一张 forward 图要 backward 多次”时才要开，比如共享 encoder 后两个 head 分别回传。默认关是因为 backward 走完后会释放 `save_for_backward` 的中间 buffer，这是省显存的大头。滥用会导致 activation 不释放、几十步后 OOM。多数情况的正确解法是 `(loss_a + loss_b).backward()` 一次搞定。

面试考点. Q5：`torch.autograd.grad()` 和 `backward()` 有什么区别？

A：四点。`grad()` 返回梯度 tuple 且 不写 `.grad`（不污染训练主循环累积的梯度）；只计算到你指定的 `inputs`，其他分支被剪掉所以更快；天然支持 `create_graph` 做高阶导；`backward()` 则是走遍整张图、写所有叶子的 `.grad`，是训练主循环的写法。

面试考点. Q6：怎么算 Hessian-vector product？为什么不直接算 Hessian？

A：两次反向。先 `g = grad(loss, x, create_graph=True)` 拿到带图的一阶梯度，再 `grad((g * v).sum(), x)` 得到 Hv 。因为 $\nabla_x (g^T v) = Hv$ 。不直接算 Hessian 是因为它是 $N \times N$ ， N 是参数量，存不下也算不动；而 HVP 的代价只是常数倍的 forward。

面试考点. Q7：`detach()`、`no_grad()`、`inference_mode()` 分别什么时候用？

A：`detach()` 断开单个 tensor 的历史，共享 storage、零开销，RNN 截断 BPTT 和“用中间量当常数”时用。`no_grad()` 让一整块代码不建图，训练中做验证、手写参数更新时用。`inference_mode()` 比 `no_grad()` 更激进，连 version counter 和 view 记账都不做，因此更快，但产出的 tensor 不能进 autograd——纯推理服务用它，训练过程里不要用。

面试考点. Q8：自己写 `autograd.Function` 有哪些必须遵守的规则？

A：`forward/backward` 都是 `@staticmethod`，用 `.apply()` 调；`backward` 返回值个数必须等于 `forward` 输入个数，不需要梯度的位置返回 `None`；要在反向用到的 tensor 用 `ctx.save_for_backward` 存（走 version counter 检查），非 tensor 挂 `ctx` 属性；用 `ctx.needs_input_grad` 跳过不需要的分支；forward 里改了输入要 `ctx.mark_dirty()`。写完用 `gradcheck` 在 float64 上验一遍。

面试考点. Q9：`gradcheck` 为什么必须用 double？

A：它用有限差分做数值梯度，误差里有一项是“机器精度 / eps”。float32 机器精度约 10^{-7} ，配上 10^{-6} 的 eps，噪声就到 10^{-1} 量级，直接淹没信号。float64 机器精度约 10^{-16} ，才有足够余量。此外还要保证测试点上函数可导（避开 ReLU 的 0 点）op 是确定性的。

面试考点. **Q10**：报错 “one of the variables needed for gradient computation has been modified by an inplace operation” 怎么查？

A：说明某个 `grad_fn` 保存的 tensor 在 backward 前被 in-place 改了，autograd 靠 version counter 检测到。常见来源是 `nn.ReLU(inplace=True)`、residual 里的 `out += identity`、手动改 activation。修法是换成 out-of-place 或先 `clone()`。定位用 `torch.autograd.set_detect_anomaly(True)`，它会打印产生这个坏梯度的 forward 栈，代价是慢好几倍，只在 debug 时开。注意不是所有 in-place 都会触发 —— 反向只需要输出的 op（如 `exp`、`sigmoid`）改输入没事，所以这类 bug 常常表现得很随机。

Dispatcher：一次 `a + b` 到底走了什么路

“你了解 PyTorch 的内部实现吗”是中高级岗位的标准问题。绝大多数候选人的回答停在“Python 调 C++，C++ 调 CUDA”，这个答案换不来任何加分。真正的答案是：PyTorch 的核心是一个 多分派 (**multiple dispatch**) 表，op 是行、DispatchKey 是列，autograd、AMP、量化、torch.compile 的 tracing 全都是这张表上的一层，而不是散落在各处的 if。把这个讲清楚，面试官会立刻知道你读过源码。

这一章跟着 `a + b` 走完全程。CUDA 层面的 async launch 细节见第 9 章，Inductor 怎么把这些 op 融成一个 kernel 见第 14 章。

分层总览



图 13 `a + b` 的调用栈。前两层是薄的胶水，Dispatcher 是决策中心，ATen kernel 是真正干活的 C++/CUDA 代码，重型线性代数再转手给厂商库。

一层层说：

1. **Python 前端**。`a + b` 触发 `Tensor.__add__`，它是 `torch._C.TensorBase` 上的一个 C 函数槽位，不是 Python 代码。`torch.add(a, b)` 走的是同一条路。这一层的开销主要是 Python 解释器本身和参数解析（`PythonArgParser` 把 Python 对象翻译成 C++ 类型），单次约几微秒——小 kernel 场景下这就是“Python overhead”的来源，也是 CUDA Graph 和 `torch.compile` 要消灭的东西。
2. **torch._C (pybind + 代码生成)**。`torch/_C/_VariableFunctions.pyi` 里那几千个函数签名是从 `native_functions.yaml` 自动生成的。这一层负责把 Python 参数转成 `at::Tensor`，然后调用 `at::add(a, b)`。
3. **Dispatcher**。查表：根据这个 op 的 schema 和参数的 DispatchKey，决定调哪个 kernel。这是本章的主题。
4. **ATen kernel**。真正的实现。`aten/src/ATen/native/` 下按后端分目录：`native/cpu/`、`native/cuda/`、`native/mkldnn/`。
5. 厂商库。matmul 转给 cuBLAS/cuBLASLt，卷积转给 cuDNN，CPU 上转给 oneDNN/MKL。PyTorch 自己不写 GEMM。

Key insight. 这五层里，只有 **Dispatcher** 是 **PyTorch** 的独特设计，其他四层任何框架都有。所以面试问“PyTorch 架构”，答案的重心必须放在 dispatcher 上。

Dispatcher 的核心： $\text{op} \times \text{DispatchKey} \rightarrow \text{kernel}$

op 与 schema

每个 op 由 `native_functions.yaml` 里的一条 **schema** 定义。schema 是这个 op 的类型签名，包括参数名、类型、默认值、哪些参数会被原地修改（`(a!)` 标记）返回值。运行时可以直接查：

```
import torch
print(torch.ops.aten.add.Tensor._schema)
# aten::add.Tensor(Tensor self, Tensor other, *, Scalar alpha=1) -> Tensor
```

同一个“逻辑 op”可以有多个 **overload** : `add.Tensor`、`add.Scalar`、`add.out`、`add_(in-place)` , 它们是 dispatcher 里独立的条目。

DispatchKey 与 key set

每个 tensor 携带一个 **DispatchKeySet** (一个位集合), 描述“我需要哪些层来处理”。可以直接打出来：

```
>>> torch._C._dispatch_key_set(torch.randn(3, device="cuda"))
DispatchKeySet(CUDA, ADInplaceOrView, AutogradCUDA, AutocastCUDA)
```

常见的 key 及其职责：

DispatchKey	这一层干什么
Autocast{CUDA,CPU}	AMP：按 op 的白/黑名单把输入 cast 成 <code>bfloat16</code> / <code>float16</code> , 然后 <code>redispatch</code>
Autograd{CPU,CUDA}	建反向图：挂 <code>grad_fn</code> 、记录 <code>next_functions</code> 、 <code>save_for_backward</code> , 然后 <code>redispatch</code>
ADInplaceOrView	维护 in-place 与 view 的记账 :bump version counter、记录 view 关系
Functionalize	把 in-place / view 改写成纯函数形式, <code>torch.compile</code> 与 <code>export</code> 的前置
Python / PythonTLSSnapshot	把分派交回 Python (<code>__torch_dispatch__</code> , <code>FakeTensor</code> / 各种 tensor subclass 靠它)
BackendSelect	factory 函数 (<code>torch.empty</code> 等) 没有输入 tensor , 靠 <code>device</code> 参数决定后端
Meta	只推 <code>shape/dtype</code> 不算数值 , <code>FakeTensor</code> 与 <code>torch.compile</code> 的 <code>shape</code> 推断
Sparse{CPU,CUDA} / NestedTensor	稀疏、变长布局的独立实现
CPU / CUDA / MPS / XPU	真正算数的后端 kernel , 分派链的终点

关键机制：按优先级逐层剥离

`DispatchKey` 有一个 全局固定的优先级 (在 `c10/core/DispatchKey.h` 里按枚举顺序排)。dispatcher 每次只取 key set 里 优先级最高的那个 **key** , 找到对应 kernel 执行；那个 kernel 干完自己的活以后，把这个 key 从 set 里“剥掉” (`c10::impl::ExcludeDispatchKeyGuard`) , 再 **redispatch** —— 于是控制权落到下一个 key。

所以一次 `a + b` (`a`、`b` 在 CUDA 上、`requires_grad=True`、处在 `autocast` 上下文里) 的完整路径是：

```
at::add(a, b)
└─ Autocast: 查 add 的策略 → 按 promote 规则统一 dtype , cast 输入
    └─ redispatch (Autocast 已排除)
```

```

└─ Autograd: 记录 AddBackward0, 把输入的 grad_fn 连进 next_functions
    └─ redispatch (Autograd 已排除)
        └─ ADInplaceOrView: add 不是 in-place, 这层基本 fallthrough
            └─ redispatch
                └─ CUDA: aten::native::add → TensorIterator → 启动
                    elementwise kernel

```

Key insight. 这段路径就是本章最值钱的一句话：**autograd** 不是 **add** 实现里的一个 **if (requires_grad)**，而是分派链上的一整层。同理，AMP 不是在 `nn.Linear` 里写了个 `cast`，量化不是重写了一遍 `op`，`FakeTensor` 不是 `mock`。它们都是 在同一张分派表上注册的一层 **kernel**，彼此正交、可任意组合。这就是为什么 PyTorch 能把 **autograd + AMP + 量化 + compile** 叠在一起而不互相污染，而这也是面试里能拉开差距的点。

Note. 想亲眼看这张表，用 `torch._C._dispatch_dump("aten::add.Tensor")`，它会把这个 `op` 上注册的所有 `key` 和对应 `kernel` 的源文件位置全部打出来。跑一遍比读十页文档管用。另外 `TORCH_SHOW_DISPATCH_TRACE=1` 环境变量（`debug build` 才有）能打印运行时的每次 `redispatch`。

fallthrough 与 boxed fallback

不是每一层都要为每个 `op` 写 `kernel`。两个机制补足：

- **fallthrough**：这一层对这个 `op` 没事可做，直接透传给下一个 `key`。`ADInplaceOrView` 对绝大多数非 `in-place` `op` 就是 `fallthrough`，零开销。
- **boxed fallback**：给一整个 `key` 注册一个“通用兜底 `kernel`”，它把所有参数打包成 `IValue` 列表统一处理。`autograd` 的“not implemented”报错、`Functionalize` 的通用改写、`Python` `key` 的回调都用这个。代价是 `boxing/unboxing` 的开销，好处是几千个 `op` 只写一份代码。

TensorImpl / Storage / TensorOptions

分派看的是 `tensor` 上的元数据，所以要知道 `tensor` 到底存了什么。

- **Storage**：一块连续的字节 + 一个 `Allocator`。它不知道 `shape`、不知道 `dtype`，就是一段内存。多个 `tensor` 可以共享同一个 `Storage`（这就是 `view`）。显存实际是 `caching allocator` 管的，见第 8 章。
- **TensorImpl**：`sizes`、`strides`、`storage_offset`、`dtype`、`device`、`DispatchKeySet`、`version_counter`、`autograd_meta`（`grad_fn` / `grad` / `requires_grad` 挂在这里）。`Tensor` 本身只是一个指向 `TensorImpl` 的智能指针，所以 `Tensor` 传参是廉价的。
- **TensorOptions**：`(dtype, device, layout, requires_grad, memory_format)` 的打包。`torch.empty(..., dtype=, device=)` 传下去的就是它，`BackendSelect` `key` 靠它决定分派到哪个后端。

DeviceGuard 是一个 `RAII` 对象：构造时把当前 `CUDA device` 切到目标 `device`，析构时切回去。每个 `CUDA kernel` 的入口都有一个，这就是为什么你可以在 `device 0` 的上下文里直接对一个 `device 1` 的 `tensor` 做运算而不用手动 `set_device`。相应的 `OptionalDeviceGuard` 用于“可能不需要切”的场合，省掉一次 `cudaSetDevice`。

Warning. DeviceGuard 只管当前 **device**，不管当前 **stream**。自己起 side stream 做 overlap 时（见第 9 章），跨 stream 用到的 tensor 必须自己 `record_stream()` 或用 event 同步，caching allocator 不会替你推断依赖。这是手写 overlap 最常见的一类内存复用 bug：tensor 被提前回收给了别的 kernel，数据被覆盖，而且不报错。

TensorIterator : elementwise op 的通用引擎

`add`、`mul`、`relu`、`where`、`copy_`、各种归约 —— 这几百个 op 的 CUDA kernel 不是一个个手写的，它们共用 `TensorIterator`。它负责：

1. 广播 (**broadcasting**)：把所有输入的 shape 对齐到公共 shape，对被广播的维度把 stride 设成 0。
2. 类型提升：按下一节的规则算出 common dtype，必要时插入 `cast`。
3. 维度合并 (**coalescing**)：如果相邻维度在内存里是连续的，就把它们合并成一维。(1024, 1024) 的 contiguous tensor 会被压成 (1048576,)，于是能用最简单的一维 grid-stride loop。
4. 维度重排：按 stride 大小重排循环顺序，让最内层循环走内存里最连续的方向。
5. 并行化与向量化：CPU 上切分给 `at::parallel_for`；CUDA 上决定 grid/block、以及能不能用 `float4` 这类向量化访存（要求指针对齐且 stride 为 1）。

Key insight. 理解 `TensorIterator` 就能直接回答“为什么 elementwise op 是 memory-bound”：一次 $y = a + b$ 要从 HBM 读 2 份数据、写 1 份，中间只做一次浮点加法。算术强度 $\approx 1 \frac{\text{FLOP}}{12} \text{ byte}$ ，而 A100 的 FP32 算力与 HBM 带宽之比在 10^2 量级 —— 差了两个数量级，所以时间 100% 花在搬数据上。

推论：把 n 个连续的 **elementwise op** 融成一个 **kernel**，就能把 **HBM** 往返从 n 次降到 1 次，理论加速接近 n 倍。这正是 `torch.compile` / `Inductor` 的第一大收益来源（第 14 章），也是为什么“给 elementwise op 换更快的算法”毫无意义 —— 它根本不在算。

类型提升 (type promotion)

规则简单但有一个高频坑。PyTorch 把参与运算的操作数分成三档 **participating category**：张量 ($\text{dim} \geq 1$)、0 维张量 (scalar tensor)、Python 标量。提升按类别的优先级来：有 $\text{dim} \geq 1$ 的张量参与时，0 维张量和 Python 标量都不参与 **dtype** 决策，只跟着走。

```
import torch
h = torch.ones(3, dtype=torch.float16)

torch.result_type(h, 2.0)                # float16 - Python 标量
不提升
torch.result_type(h, torch.tensor(2.0))  # float16 - 0 维张量也不
提升
torch.result_type(h, torch.tensor([2.0])) # float32 - 1 维张量参与，
提升了
```

第三行就是坑：混合精度训练里，你以为在做一个 fp16 的逐元素运算，因为某个常量被写成 `torch.tensor([eps])`（多了一对方括号）就悄悄提升成了 fp32，多出一份显存、还可能在 `torch.compile` 里造成额外的 kernel。

同一类别内部按“能不能无损容纳”提升：`bool < 整数 < 浮点 < 复数`，同类内按位宽。跨类别时结果取更宽的那类。

```
torch.result_type(torch.ones(3, dtype=torch.int64),
                  torch.ones(3, dtype=torch.float16))    # float16 — 浮点类别赢，
不看位宽
```

Warning. 整数除法与 `in-place` 是两个额外的坑。

- `a / b` 对整数 `tensor` 返回浮点（true division）。想要整除用 `torch.div(a, b, rounding_mode="floor")`，// 在旧版本有过 deprecation 反复，现在等价于 `floor` 模式。
- **in-place op** 不做类型提升：输出 `dtype` 永远是左操作数的 `dtype`。同类别内会静默降精度——`x_fp16.add_(y_fp32)` 不报错，但结果被截回 `fp16`；跨类别则直接报错——`x_int64.add_(y_fp32)` 得到 `result type Float can't be cast to the desired output type Long`。所以 AMP 代码里的 `+=` 既可能悄悄丢精度也可能突然报错，两种都要留意。写成 `out-of-place` 就按正常提升规则走。

注册自定义 **op**：`torch.ops` 与 `torch.library`

`torch.ops.<namespace>.<op>.<overload>` 是访问 dispatcher 里任何 `op` 的统一入口，包括 PyTorch 自带的（`torch.ops.aten.add.Tensor`）和你自己注册的。

现代做法是 `torch.library.custom_op`（torch 2.4+ 稳定）。它比早年的 `torch.library.Library("myns", "DEF")` + 手写 schema 字符串好用得多：schema 从 Python 类型注解自动推断，也不需要写 C++。

```
import torch
from torch import Tensor

@torch.library.custom_op("mylib::rms_norm", mutates_args=())
def rms_norm(x: Tensor, weight: Tensor, eps: float = 1e-6) -> Tensor:
    # 这里可以调你自己的 CUDA / Triton kernel
    var = x.float().pow(2).mean(-1, keepdim=True)
    return (x.float() * torch.rsqrt(var + eps)).to(x.dtype) * weight

@rms_norm.register_fake
def _(x, weight, eps=1e-6):
    # 只描述 shape / dtype / device，不碰数值
    return torch.empty_like(x)

def _backward(ctx, grad):
    x, weight = ctx.saved_tensors
    ... # 返回与 forward 输入一一对应的梯度
    return grad_x, grad_w, None

def _setup_context(ctx, inputs, output):
    x, weight, eps = inputs
    ctx.save_for_backward(x, weight)
```



```
torch.library.register_autograd(
    "mylib::rms_norm", _backward, setup_context=_setup_context)

y = torch.ops.mylib.rms_norm(x, w)          # 走 dispatcher, 和内置 op 平权
```

三个要点：

- **mutates_args** 必须如实填。写了哪些参数会被原地改，**Functionalize** 层才能正确改写，**torch.compile** 才敢重排。填错会得到静默的错误结果。
- **register_fake**（旧名 **impl_abstract**）是给编译器用的。**torch.compile** 在 **trace** 时用 **FakeTensor** 跑一遍图来推断每个中间量的 **shape/dtype**，它不能真的执行你的 **CUDA kernel**（没有真实数据、也不该花那个时间）。不注册 **fake**，遇到你的 **op** 就 **graph break**，编译收益直接归零。
- **register_autograd** 单独注册反向，语义上等价于 **autograd.Function**，但它注册在 **dispatcher** 的 **Autograd key** 上，因此对 **torch.compile**、**vmap**、**export** 都可见。库代码用它，训练脚本里的一次性需求用 **autograd.Function**（第 6 章）就够。

用 **torch.library.opcheck(op, args)** 可以一次性检查 **schema** 一致性、**fake kernel** 与真实 **kernel** 的 **shape** 是否吻合、**autograd** 是否正确——写完自定义 **op** 应当跑一遍。

Note. **torch.library** 这套 API 从 **torch 2.1** 到 **2.4** 经历过多次改名（**impl_abstract** → **register_fake**、**define** → **custom_op**），且仍在演进。这里给的是 **2.4+** 的稳定形态，具体签名以你使用版本的文档为准。

C++ 扩展 vs torch.compile / Triton

写自定义 **kernel** 有三条路，面试里问“你会怎么优化这个算子”时要能说清取舍：

方案	适合	代价
torch.compile	连续的 elementwise/归约链 ，只是想省 HBM 往返	几乎零成本，先试这个
Triton kernel + custom_op	有特殊访存模式（ flash-attention 类）需要手控 tile 和 shared memory	要懂 GPU 内存层级，调 num_warps / BLOCK ；可与 compile 共存
C++/CUDA 扩展	要用 CUDA 特有功能（ warp intrinsic 、 TMA 、自定义 PTX ）或要接第三方库	编译工具链、跨平台构建、要自己写 fake kernel 和 autograd

顺序是明确的：先 **torch.compile**，不够再 **Triton**，最后才 **C++**。绝大多数“我要写个 **CUDA kernel**”的需求，**torch.compile** 已经能拿到 **80%** 的收益且零维护成本。

CPU 与 CUDA kernel 的差异

同一个 **op** 落到 **CPU key** 和 **CUDA key** 上是两份完全不同的代码：

- **CUDA** 是异步 **launch**。**at::native::add** 里的 **cudaLaunchKernel** 立刻返回，**kernel** 排进当前 **stream** 排队执行。所以 **Python** 侧测到的“**op** 耗时”常常只是 **launch** 的几微秒，真正的执行时间要靠 **torch.cuda.synchronize()** 或 **CUDA event** 才能测准。这也意味着 **CPU** 端的 **dispatch** 开销可以被 **GPU** 执行掩盖——只要 **kernel** 够大。**kernel** 太小时 **CPU** 反而成了瓶颈（**launch bound**），这是 **CUDA Graph** 和 **torch.compile** 的用武之地。详见第 9 章。

- **CPU** 是同步的, 并且用 `at::parallel_for` 做线程级并行。底层后端由构建时决定 (OpenMP 或 TBB), 线程数用 `torch.set_num_threads(n)` 控制。DataLoader 的 worker 进程里通常要把它设成 1, 否则每个 worker 都开满线程会互相抢核, 反而更慢。

```
torch.set_num_threads(8)          # op 内部并行 (intra-op)
torch.set_num_interop_threads(2)  # op 之间并行 (inter-op), 必须在任何 op 之前调
```

后端开关: cuDNN benchmark 与 TF32

这几个全局开关面试常问, 因为它们直接影响“你的模型为什么这次快下次慢/为什么精度对不上”。

```
torch.backends.cudnn.benchmark = True      # 卷积算法自动选优
torch.backends.cuda.matmul.allow_tf32 = True # matmul 走 TF32
torch.backends.cudnn.allow_tf32 = True     # 卷积走 TF32 (历史上默认就是 True)
torch.set_float32_matmul_precision("high") # 上面那个 matmul 开关的高层封装
```

cudnn.benchmark 的语义: 第一次遇到某个 (input shape, dtype, conv 参数) 组合时, 实测所有候选算法各跑一遍, 选最快的缓存起来; 之后同样的 shape 直接用。收益在固定 shape 的 CNN 上很明显, 代价是每个新 shape 都要付一次 benchmark 的时间。所以 **shape** 频繁变化的场景 (变长序列、动态 **batch**) 要关掉它, 否则会不停地重新 benchmark, 反而更慢。它还是不确定性的来源之一 (不同次运行可能选到不同算法), 见第 10 章。

TF32 是什么: Ampere (A100) 引入的 tensor core 数据格式, 19 位 —— 1 符号 + 8 指数 + 10 尾数。指数位跟 FP32 一样 (所以动态范围相同, 不会溢出), 尾数只有 10 位 (所以精度约等于 FP16 的尾数)。用法是: 输入输出仍是 FP32, 只在 tensor core 内部把尾数截断到 10 位做乘法, 累加仍用 FP32。

格式	指数位	尾数位	说明
FP32	8	23	基准
TF32	8	10	动态范围同 FP32, 精度约同 FP16; 只用于 tensor core 的乘法
FP16	5	10	动态范围小, 训练需要 loss scaling
BF16	8	7	动态范围同 FP32, 精度更低, 训练不需要 loss scaling

`set_float32_matmul_precision` 的三档与 `allow_tf32` 的关系: "highest" = 纯 FP32 (等价 `allow_tf32 = False`); "high" 和 "medium" 都允许用 TF32 (等价 `allow_tf32 = True`), 区别在于 "medium" 还允许用 bf16 做更激进的近似。torch 2.x 里 **matmul** 的 **TF32** 默认是关的, 而 **cuDNN** 卷积的 **TF32** 历史上默认是开的 —— 这个不对称经常让人困惑。

Warning. 典型翻车: 训练脚本里开了 `set_float32_matmul_precision("high")`, 然后拿这个模型的输出跟一份 FP32 参考结果对比数值, 发现相对误差到了 10^{-3} 量级, 怀疑代码写错了。其实是 TF32 的尾数只有 10 位, 这个误差完全正常。

规则: 训练开 **TF32** (收益大、对收敛无影响); 写单元测试和做数值对拍时关掉它。

```
torch.backends.cuda.matmul.allow_tf32 = False # 对拍前先关
```

`torch.__config__.show()` 打印这个 build 的全部信息: 编译器版本、MKL/MKL-DNN 版本、CUDA / cuDNN / NCCL 版本、是否开了各种后端。排查“为什么这台机器上慢/为什么这

个 op 不可用”时第一件事就是跑它。 `torch.backends.cudnn.version()`、`torch.version.cuda`、`torch.cuda.get_device_capability()` 是更精确的单点查询。

面试怎么讲：60 秒的架构答案

被问“讲讲 PyTorch 的架构”，按这个结构说，不要展开成源码导读：

PyTorch 分五层：Python 前端、`torch._C` 的 pybind 胶水、dispatcher、ATen kernel、厂商库。真正有设计含量的是中间的 dispatcher。

它本质是一张二维表：行是 op（由 `native_functions.yaml` 的 schema 定义），列是 DispatchKey。每个 tensor 带一个 DispatchKeySet，dispatcher 按固定优先级取出最高的那个 key、调对应 kernel，那个 kernel 干完自己的事就把这个 key 排除掉再 redispatch，控制权落到下一层。

所以一次 CUDA 上的 `a + b`，如果在 autocast 里而且要梯度，会依次经过 Autocast（统一 dtype）、Autograd（挂 `AddBackward0`）、`ADInplaceOrView`（记账，这里是 fallthrough），最后落到 CUDA kernel，由 TensorIterator 处理广播、类型提升、维度合并，再启动一个 elementwise kernel。

这个设计的价值在于：autograd、AMP、量化、FakeTensor 的 shape 推断、`torch.compile` 的 functionalization，全都是这张表上正交的一层，可以任意组合而不互相污染。换成“在每个 op 里写 if”的实现，这些功能就没法叠加了。

想再深一层，就接一句“最下面的 elementwise op 全是 memory-bound，所以 Inductor 的主要收益是把它们融成一个 kernel 省 HBM 往返”——顺势把话题引到你更熟的第 14 章。

面试考点

面试考点. Q1：一次 `a + b` 在 PyTorch 内部发生了什么？

A：Python 的 `__add__` 是 C 函数槽位，经参数解析进 `at::add`，交给 dispatcher。dispatcher 按 DispatchKeySet 的优先级逐层走：Autocast 统一 dtype、Autograd 挂 `AddBackward0` 并连 `next_functions`、`ADInplaceOrView` 记账（非 in-place 时 fallthrough），每层做完自己的事就排除该 key 再 redispatch，最后落到 CUDA kernel。CUDA kernel 里由 TensorIterator 处理广播、类型提升、维度合并，然后异步 launch 一个 elementwise kernel。

面试考点. Q2：DispatchKey 是什么？为什么说 autograd 是“一层”而不是一个 if？

A：DispatchKey 标识“这个 tensor 需要哪些层参与处理”，每个 tensor 带一个 key set。dispatcher 按全局固定的优先级取最高的 key 找 kernel，kernel 执行完把该 key 排除后 redispatch，形成一条剥洋葱式的链。autograd 就是链上优先级高于后端的一层 kernel：它建完图就把 Autograd key 排除掉再往下调真正的计算。这样 AMP、量化、FakeTensor、functionalization 也各占一层，彼此正交、可任意组合。

面试考点. Q3：TensorIterator 干什么？为什么 elementwise op 是 memory-bound？

A：它是所有 elementwise 和归约 op 共用的引擎，负责广播（把被广播维的 stride 设 0）、类型提升、连续维度合并、循环重排、并行化与向量化。elementwise op 的算术强度极低——

`y = a + b` 读 2 份写 1 份只做 1 次加法, 约 1 FLOP / 12 byte, 而 GPU 的算力带宽比差两个数量级, 所以时间全花在 HBM 往返上。推论是把 n 个连续 elementwise op 融成一个 kernel 能接近 n 倍加速, 这就是 Inductor 的主要收益来源。

面试考点. **Q4**: `torch.tensor(2.0)` 和 `torch.tensor([2.0])` 跟一个 fp16 张量相乘, 结果 dtype 一样吗?

A: 不一样。类型提升按“参与类别”分档: 只要有 $\text{dim} \geq 1$ 的张量参与, 0 维张量和 Python 标量就不参与 dtype 决策。所以 `fp16_tensor * torch.tensor(2.0)` 是 fp16, `fp16_tensor * torch.tensor([2.0])` 是 fp32。混合精度代码里多写一对方括号就会引入意外的 fp32 提升。用 `torch.result_type(a, b)` 可以提前查。

面试考点. **Q5**: 注册自定义 op 时为什么必须写 `register_fake`?

A: `torch.compile` 在 trace 时用 `FakeTensor` 走一遍图来推断每个中间量的 shape/dtype, 它没有真实数据、也不该真的执行你的 kernel。`register_fake` 就是告诉编译器“给我这样的输入, 输出是什么 shape/dtype/device”。不注册的话, 编译器碰到你的 op 只能 graph break, 编译收益直接没了。同理 `mutates_args` 要如实填, 否则 functionalization 会改写错、编译器会做不安全重排。

面试考点. **Q6**: TF32 是什么? `allow_tf32` 和 `set_float32_matmul_precision` 什么关系?

A: TF32 是 Ampere 起 tensor core 支持的格式, 8 位指数 (动态范围同 FP32) + 10 位尾数 (精度约同 FP16), 累加仍用 FP32。输入输出都还是 FP32, 只是乘法时截断尾数。`set_float32_matmul_precision("high"/"medium")` 是 `torch.backends.cuda.matmul.allow_tf32 = True` 的高层封装, "highest" 对应关闭。torch 2.x 里 matmul 的 TF32 默认关、cuDNN 卷积的默认开。训练建议开 (收益明显、不影响收敛), 做数值对拍和写单测时必须关。

面试考点. **Q7**: `torch.backends.cudnn.benchmark = True` 什么时候该开, 什么时候是坑?

A: 它让 cuDNN 在第一次遇到某组 shape 时实测所有候选卷积算法、选最快的缓存下来。固定 shape 的 CNN 训练该开, 收益明显。变长输入 / 动态 batch 场景是坑: 每个新 shape 都要重新 benchmark 一遍, 总开销可能超过收益。它还引入不确定性 —— 不同次运行可能选到不同算法导致数值不完全复现, 要严格复现就得关掉。

面试考点. **Q8**: 想加速一个自定义算子, `torch.compile`、Triton、C++ 扩展怎么选?

A: 按成本从低到高试。先 `torch.compile` —— 如果瓶颈是一串 elementwise/归约的 HBM 往返, Inductor 融合就能拿到大部分收益, 零维护成本。不够再写 Triton kernel 并用 `torch.library.custom_op` 注册进 dispatcher, 适合需要手控 tile 和 shared memory 的访存模式 (flash-attention 那一类)。只有当你要用 CUDA 特有功能 (warp intrinsic、自定义 PTX) 或接第三方库时才上 C++/CUDA 扩展, 代价是构建工具链和手写 fake kernel、autograd 的维护成本。

显存：allocator、峰值与 OOM

显存题是训练框架岗位的必考题，而且是最容易分辨“真练过”和“只看过博客”的一道。面试官问的不是 `torch.cuda.empty_cache()` 怎么调，而是：你的模型为什么占这么多、峰值在哪一步、`reserved` 比 `allocated` 大 20 GB 说明什么、为什么明明还剩 30 GB 却报 OOM。这一章把显存拆成“算得出来的静态部分”和“必须测出来的动态部分”，然后讲 caching allocator 的机制、碎片化的成因，最后给一套 OOM 排查流程。省显存的分布式手段 (ZeRO / FSDP) 只在这里列出定位，细节见第 19 章。

显存都花在哪

一张卡上的显存分成六块，性质完全不同：

成分	大小	随 batch 变化
参数	由参数量 P 与 dtype 决定	否
梯度	与参数同 shape	否
优化器状态	Adam 是 2 份 moment (+ fp32 master)	否
激活	$\propto \text{batch} \times \text{seq} \times \text{hidden} \times \text{layers}$	是
临时 buffer	kernel workspace、AllReduce 的 flat buffer、 <code>cat</code> 的输出	部分
CUDA context + NCCL	每进程约 0.5-1 GB，加每个 communicator 的 buffer	否

前三块可以纯手算，这是面试白板上要能当场推的。用 P 表示参数个数，单位是 byte：

$$M_{\text{static}} = \underbrace{2P}_{\text{bf16 参数}} + \underbrace{2P}_{\text{bf16 梯度}} + \underbrace{4P}_{\text{fp32 master}} + \underbrace{8P}_{\text{Adam } m,v} = 16P$$

有意思的是，纯 fp32 训练 + Adam 也是 $4P + 4P + 8P = 16P$ ——AMP 省的不是这块，省的是激活和算力。真正把 $16P$ 压下去要动优化器：纯 bf16 训练 (不留 master weight、moment 也用 bf16) 是 $8P$ ，8-bit optimizer 是 $2P + 2P + 4P + 2P \approx 10P$ 。

代进 7B： $16 \times 7 \times 10^9 = 112 \text{ GB}$ 。

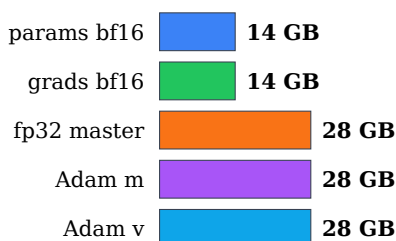


图 14 7B 模型 bf16 混合精度 + AdamW 的静态显存分解，合计 112 GB。一张 **80 GB** 的 **A100** 装不下一个 **7B** 的训练态，激活还没算。这就是 ZeRO / FSDP 存在的理由。

Key insight. 面试标准答案的骨架：静态显存 $\approx 16P$ byte (bf16 + AdamW)，激活显存另算且与 batch 线性相关。7B 在单卡 80 GB 上放不下训练态，但推理只要 $2P = 14 \text{ GB}$ ——这个对比能一句话说明“训练比推理贵在哪”。

激活显存为什么是大头

激活是 forward 里被 autograd 存下来、等 backward 用的中间张量（机制见第 6 章）。它随 batch 线性涨，所以是唯一一个“你能调”的大头。

一个 Transformer layer 大约要存这些：LayerNorm 的输入与 `rstd`、QKV 投影的输入、attention 的输出、FFN 的输入与中间的 $4h$ 张量、GELU 的输入、dropout mask。粗算成“若干份 $b \times s \times h$ ”：

$$A_{\text{layer}} \approx c \cdot b \cdot s \cdot h \cdot \text{sizeof}(\text{dtype}), \quad c \approx 10 \sim 20$$

c 取决于实现细节（有没有 fuse、存不存 GELU 输入），量级上取 16 就做估算。代进 $b = 4$ 、 $s = 4096$ 、 $h = 4096$ 、bf16、32 层：

$$16 \times 4 \times 4096 \times 4096 \times 2 \text{ B} \approx 2.1 \text{ GB} \quad \text{每层} \rightarrow \times 32 \approx 69 \text{ GB}$$

激活比全部静态显存还大。而且注意这里没有 $b \cdot a \cdot s^2$ 那一项——那是手写 attention 把 $s \times s$ 的 score 矩阵 materialize 出来时才有的， $s = 4096$ 、 $a = 32$ 、 $b = 4$ 时它单独就是 $4 \times 32 \times 4096^2 \times 2 \text{ B} \approx 4.3 \text{ GB}$ 每层。用 `F.scaled_dot_product_attention`（FlashAttention 后端）就没有这一项，这是 SDPA 最大的显存收益，比它的速度收益更值得在面试里提。

Note. 这些系数是量级估算，不是测量值。真要知道自己模型的激活占多少，跑一个 step 然后 `max_memory_allocated()` - 静态部分，或者直接看下面的 memory snapshot。

caching allocator：为什么 `empty_cache()` 通常没用

`cudaMalloc` / `cudaFree` 是同步调用，一次几十到几百微秒，而训练一个 step 要分配几千次张量。所以 PyTorch 自己实现了一层 caching allocator：向驱动一次性申请大块 segment，之后在这些 segment 里切 block 给你，释放时不还给驱动，只放回自己的空闲链表。

于是有了两个数，这是最高频的显存题：

<code>memory_allocated()</code>	你的张量真正占用的字节数，allocator 记账得到
<code>memory_reserved()</code>	allocator 从 CUDA 驱动手里拿到的总字节数

`reserved >= allocated` 恒成立。`nvidia-smi` 看到的是 `reserved` 再加 CUDA context，所以它总比 `memory_allocated()` 大一截——这不是泄漏。

allocator 的三个关键设计：

- 按 **size** 分池。小于 1 MB 的请求走 small pool（从 2 MB 的 segment 里切），大于 1 MB 走 large pool（20 MB 或按需的 segment）。block 大小向上圆整到 512 B 的倍数。分池是为了让大量小张量不去污染大块连续空间。
- stream-aware**。每个 block 记着自己是在哪个 stream 上分配的。只有同 stream 的后续请求才能直接复用它，跨 stream 复用必须先同步——这条在第 9 章的 `record_stream()` 那节会咬人。
- 不还给驱动。所以第二个 step 起几乎不再调 `cudaMalloc`，稳态下分配是纯用户态操作。

`torch.cuda.empty_cache()` 做的事就是把空闲 segment 还给驱动。它不会减少 `memory_allocated`，只会减少 `memory_reserved`。什么时候有用：你要在同一进程里把显存让给别的库（比如 vLLM、TensorRT、另一个 CUDA context）。什么时候没用（也就是绝大多数情况）：训练 loop 里为了“防 OOM”而周期性调用——它只是把缓存丢掉，下个 step 再重新 `cudaMalloc` 拿回来，纯亏时间。

Warning. 在训练 loop 里每 step 调 `torch.cuda.empty_cache()` 是典型的 cargo cult。它引入同步、把 allocator 的缓存清空，让每个 step 都要重新走 `cudaMalloc`，step time 明显变差，而且并不能解决真正的峰值超限。唯一合理的用法是“阶段切换”处：训练结束、开始 eval 之前，或者要交显存给别的进程/库时调一次。

碎片化

判据只有一条：`reserved` 明显大于 `allocated`（比如差 20% 以上）就是碎片。

典型报错长这样：

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 2.00 GiB.
GPU 0 has a total capacity of 79.14 GiB of which 5.62 GiB is free.
Process ... has 73.52 GiB memory in use. Of the allocated memory
68.10 GiB is allocated by PyTorch, and 1.83 GiB is reserved by PyTorch
but unallocated.
```

关键在最后一句：`reserved but unallocated` 有 1.83 GB，`free` 还有 5.62 GB，却申请不到 2 GB 连续空间。这就是碎片——空闲字节总量够，但没有一块连续的够大。

成因几乎总是“分配大小在变”：

- 变长 seq：s 每个 batch 都不同，激活张量的大小跟着变，旧 block 的尺寸和新请求对不上
- 动态 batch / bucketing dataloader
- eval 用的 batch 和 train 不一样大，两套尺寸交替
- 中途插入的大临时分配（一次 `torch.cat` 出一个巨大张量）把 segment 切碎

对策一是 `expandable_segments`：

```
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
```

它把固定大小的 segment 换成“可以往后扩展的虚拟地址区间”（用 CUDA 的 virtual memory management API）。这样一个 segment 能随需求长大，而不是“要 2.5 GB 但手里只有一堆 2 GB 的 segment”。变长 shape 的场景下这个开关经常是零成本的净收益，值得默认打开试一次。

其他可调项（同一个环境变量，逗号分隔）：`max_split_size_mb:N` 禁止把大于 N MB 的 block 切开给小请求；`garbage_collection_threshold:0.8` 在 `reserved` 超过 80% 上限时主动回收空闲 block 而不是直接 OOM。

Note. `expandable_segments` 在较新版本里已经相当稳定，但它改变的是底层分配方式，遇到过与某些自定义 CUDA 扩展（自己拿 pointer 做 IPC 的）不兼容。上线前测一遍功能正确性。

排查工具

第一层是计数器，插在训练 loop 里几乎零成本：

```
import torch

torch.cuda.reset_peak_memory_stats()
train_one_step()
print(f"cur alloc {torch.cuda.memory_allocated() / 2**30:.2f} GiB")
```



```
print(f"peak alloc {torch.cuda.max_memory_allocated() / 2**30:.2f} GiB")
print(f"cur resv {torch.cuda.memory_reserved() / 2**30:.2f} GiB")
print(f"peak resv {torch.cuda.max_memory_reserved() / 2**30:.2f} GiB")
```

`reset_peak_memory_stats()` 每 step 清零，才能定位“峰值出现在哪一步”而不是“整个 job 的峰值”。`torch.cuda.memory_summary()` 打一张按 size bin 分的详细表，`torch.cuda.memory_stats()` 返回同样内容的 dict，里面 `num_alloc_retries` 这个 key 特别有用：它大于 0 就说明 allocator 曾经因为拿不到空间而被迫 `empty_cache()` 重试，这是碎片的直接证据。

第二层是 memory snapshot，能看到“每一块显存是哪行 Python 代码分配的”：

```
import torch

torch.cuda.memory._record_memory_history(max_entries=200_000)

for step in range(3):          # 跑几个 step 到稳态就够了
    train_one_step()

torch.cuda.memory._dump_snapshot("mem.pickle")
torch.cuda.memory._record_memory_history(enabled=None)  # 关掉
```

把 `mem.pickle` 拖到 pytorch.org/memory_viz（纯前端，文件不上传）。这张图的横轴是时间、纵轴是显存，每个色块是一次分配，点进去有完整的 Python 调用栈。你要在图里找三样东西：最高的那根柱子（峰值时刻）、峰值时最厚的那几层（谁占的）、还有从头贯穿到尾的横条（这就是泄漏：一块显存从分配到 dump 都没释放）。

Warning. `_record_memory_history` 是带下划线的私有 API，签名在版本间变过（`enabled` 参数取 `"all"` / `"state"` / `None`）。它开着的时候会记录每次分配的调用栈，有明显开销，只在排查时开，不要留在生产训练脚本里。

省显存的手段，按性价比排序

版本	核心思路	相对成本 / 收益
<code>set_to_none=True</code>	<code>zero_grad(set_to_none=True)</code> 让 grad 张量真正释放而不是填 0	省 $2P-4P$ ，零代价
AMP / bf16	参数与激活都减半，激活是大头所以收益大	省激活约一半，还更快
SDPA / FlashAttention	不 materialize $s \times s$ 的 score 矩阵	省掉 bas^2 那一项
micro-batch + 梯度累积	激活与 batch 线性相关，切小就线性下降	几乎零代价
activation checkpointing	只存 layer 边界，backward 时重算	激活降一个数量级，算力 +30%
8-bit optimizer	moment 用 int8 + 分块量化存	$16P \rightarrow 10P$

参数 / 优化器 offload	把 fp32 master 与 moment 放 CPU	省 12P , 受 PCIe 限制
ZeRO / FSDP	参数、梯度、优化器状态按 DP 维度切开	静态显存除以 DP 度 , 见第 19 章

前四条应该无脑上,第五条要算一下值不值,后三条是“实在装不下才动”的手段。面试里被问“你怎么省显存”,按这个顺序讲就是满分答案——重点是先说哪些是零成本的,再说哪些是拿算力/带宽换显存的。

activation checkpointing 的取舍

原理: forward 时对被包住的段落不存中间激活,只存输入; backward 需要时重跑一遍 forward 把激活算回来。

```
from torch.utils.checkpoint import checkpoint

class Block(nn.Module):
    def forward(self, x):
        x = x + self.attn(self.ln1(x))
        x = x + self.mlp(self.ln2(x))
        return x

# 在模型的 forward 里逐层包
for blk in self.blocks:
    x = checkpoint(blk, x, use_reentrant=False)
```

`use_reentrant=False` 必须显式写。老的 `reentrant` 实现不支持在 `checkpoint` 段里用 `.grad / hook`, 也不支持部分输出不需要梯度, 且要求输入必须有 `requires_grad`; 新实现基于 `TorchDispatchMode`, 限制少得多。torch 2.x 里不传这个参数会 warning, 未来默认值会翻转。

代价: 多一遍 forward。整层 `checkpoint` 的话, backward 原本约等于 2 倍 forward 的计算量, 现在变成 3 倍 forward, 所以总计算量涨约 $\frac{1}{3}$, 实测通常落在 +25% 到 +35% (取决于被重算的部分是 memory-bound 还是 compute-bound)。收益: 每层激活从 $c \cdot bsh$ 降到约 $1 \cdot bsh$, 上面那个 69 GB 的例子降到约 4.3 GB。

selective checkpointing 是更好的折中: 只重算便宜的 op, 把贵的 op (matmul、attention) 的输出留下来。norm、GELU、dropout 这些 elementwise/memory-bound op 重算几乎不花时间, 但它们的激活占了相当比例; 反过来 matmul 重算最贵。

```
import torch
from torch.utils.checkpoint import (
    checkpoint, create_selective_checkpoint_contexts, CheckpointPolicy,
)

_save_ops = {torch.ops.aten.mm.default,
torch.ops.aten._scaled_dot_product_flash_attention.default}

def policy_fn(ctx, op, *args, **kwargs):
    if op in _save_ops:
        return CheckpointPolicy.MUST_SAVE # 贵的: 存下来
    return CheckpointPolicy.PREFER_RECOMPUTE # 便宜的: 重算
```

```
def ctx_fn():
    return create_selective_checkpoint_contexts(policy_fn)

x = checkpoint(blk, x, use_reentrant=False, context_fn=ctx_fn)
```

工程上更常见的粗粒度版本是“每 k 层 checkpoint 一层”(`if i % 2 == 0`), 实现两行就够, 效果是显存和算力都取中间值。面试里能说出“selective 而不是全 checkpoint”就已经比大多数人细了。

Key insight. checkpointing 是用算力买显存。买它的唯一正当理由是：省下来的显存能让你把 batch 开大, 而更大的 batch 带来的 GPU 利用率提升 > 重算多花的 30% 算力。如果开了 checkpoint 但 batch 没变大, 你就是纯亏。

OOM 排查流程

按顺序做, 不要跳步：

1. 看报错里的三个数。Tried to allocate X、free Y、reserved but unallocated Z。如果 Y 很小 → 真的不够, 走第 2 步；如果 Y 不小但 $X >$ 最大连续块 → 碎片, 走第 5 步。
2. 确认是稳态峰值还是某一步的尖峰。每 step 打 `max_memory_allocated()` 并 `reset_peak_memory_stats()`。如果第 1 个 step 就 OOM, 是静态部分算错了；如果跑了几百步才 OOM, 是某种累积。
3. 看 **snapshot** 找最大分配和贯穿到底的横条。峰值通常出现在 backward 刚开始（激活最多的时刻）或者 optimizer step 里（要临时开与参数同 shape 的 buffer）。
4. 检查有没有累积带图的张量。这是“跑几百步才 OOM”的头号原因, 见下一节。
5. 碎片处理：先开 `expandable_segments=True`, 再考虑把变长 seq pad 到几个固定档位 (bucketing), 让分配尺寸收敛到有限几种。
6. 检查 **eval** / 推理路径。忘了 `torch.no_grad()` 或 `model.eval()`, eval 就会像训练一样存激活, 而 eval 的 batch 往往更大。
7. 检查 **dataloader**。 `pin_memory=True` 占的是 CPU 内存不是显存；但如果你在 `Dataset.__getitem__` 里 `.cuda()`, 或者把整个数据集 `.to('cuda')` 预加载, 那就是显存。
8. 还是不够, 才动第 19 章的分布式手段。

那些不是泄漏的“泄漏”

Python 有 GC, CUDA 显存也由 allocator 管, 真正的显存泄漏在 **PyTorch** 里非常罕见。你遇到的 99% 是“你自己拿着引用不放”。三种典型写法：

Warning. 1. 把 `loss` 而不是 `loss.item()` 存进 `list`。 `loss` 带着 `grad_fn`, `grad_fn` 引用着整张 autograd 图, 图引用着这个 step 的全部激活。存 1000 个 step 就是 1000 份激活。

```
losses.append(loss)           # 泄漏：整张图被保活
losses.append(loss.item())    # 对：取标量（有一次同步，见第 9 章）
losses.append(loss.detach())  # 也对：想留在 GPU 上做平均时用这个
```

2. 把带 `grad` 的 `tensor` 存进外部容器 / `buffer`。

```
self.cache = hidden          # 泄漏: hidden.grad_fn 保活整张图
self.cache = hidden.detach()  # 对
```

同理, accumulate 指标要写 `total += loss.detach() * n`, 写成 `total += loss * n` 会把每个 step 的图串成一条越来越长的链。

3. hook 里持有引用。

```
acts = {}
def hook(mod, inp, out):
    acts[mod] = out          # 泄漏: out 一直被 dict 拿着
    handle = m.register_forward_hook(hook)
    # 排查完必须 handle.remove(), 且存 out.detach().cpu()
```

hook 的 handle 不 `remove()` 也是常见的隐性泄漏源——尤其是在一个循环里反复注册。

还有两个容易忽略的: `torch.cuda.max_memory_allocated()` 记的是历史峰值, 看着不降不代表当前占着; 异常处理里 `except` 块捕获的 `traceback` 会持有栈帧, 栈帧里的局部变量 (包括那个巨大的 activation) 跟着被保活, 这就是“OOM 之后重试还是 OOM”的原因, 处理办法是在 `except` 里先 `del` 掉相关变量。

面试考点

面试考点. Q1: `memory_allocated` 和 `memory_reserved` 有什么区别? 为什么 `nvidia-smi` 的数比 `memory_allocated` 大?

A: `allocated` 是活着的张量占的字节; `reserved` 是 PyTorch caching allocator 从驱动那里申请到的总量, 包含它缓存起来没还的空闲 block。 `nvidia-smi` 看到的约等于 `reserved` 加 CUDA context (每进程约 0.5-1 GB) 加 NCCL buffer, 所以必然更大。两者差得多说明缓存里有大量空闲 block, 即碎片。

面试考点. Q2: `torch.cuda.empty_cache()` 什么时候有用?

A: 几乎只有一种场景有用——要把显存让给同进程里的其他库或其他 CUDA context。它把空闲 segment 还给驱动, 只降 `reserved`, 不降 `allocated`, 也不能降低你的峰值需求。放在训练 loop 里每 step 调是负优化: 清掉缓存后每步都要重新走同步的 `cudaMalloc`。

面试考点. Q3: 报 “tried to allocate 2 GiB” 但 `nvidia-smi` 显示还剩 6 GB, 怎么解释?

A: 碎片化。空闲字节的总量够, 但没有一块连续的 2 GiB。看报错里的 `reserved but unallocated` 和 `memory_stats()["num_alloc_retries"]` 确认。成因是分配尺寸在变 (变长 seq、动态 batch)。对策: `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`, 或把 seq 长度 bucketing 到有限档位。

面试考点. Q4: 估算一个 7B 模型 bf16 混合精度 + AdamW 训练要多少显存。

A: 静态部分 $16P$ byte: bf16 参数 $2P$ + bf16 梯度 $2P$ + fp32 master $4P$ + Adam 的 m 和 v 各 $4P$, 代进 7B 是 112 GB。加上激活 (与 $\text{batch} \times \text{seq}$ 线性相关) 和约 1 GB 的 context。
结论: 单张 80 GB A100 装不下, 必须 ZeRO/FSDP 切分或 offload。推理只要 $2P = 14$ GB。

面试考点. Q5: 训练跑了几百个 step 才 OOM, 第一个怀疑对象是什么?

A: 累积了带 `grad_fn` 的张量。最经典的是 `losses.append(loss)` 而不是 `loss.item()` —— `loss` 引用整张 autograd 图, 图引用整个 step 的激活。同类还有 `total_loss += loss` (应该 `+= loss.detach()`)、forward hook 把 `out` 存进 dict 不 `detach`、module 上挂 `self.cache = hidden`。用 memory snapshot 找“从头贯穿到尾的横条”能一眼定位。

面试考点. Q6: activation checkpointing 省多少、贵多少? 什么时候不该用?

A: 每层激活从“十几份 *bsh*”降到约“一份 *bsh*”, 量级上是一个数量级; 代价是 backward 时多跑一遍 forward, 总计算量涨约三分之一 (实测常见 +25%–35%)。不该用的情况: 显存本来够用, 或者省下的显存没被用来加大 batch——那就是纯亏算力。更优的做法是 selective checkpointing, 只重算 norm/GELU 这类 memory-bound op, 保留 matmul 和 attention 的输出。

面试考点. Q7: caching allocator 为什么是 stream-aware 的?

A: 每个 block 记录分配它的 stream。释放后只能被同 stream 的请求直接复用, 因为 allocator 只能保证同 stream 内的顺序执行。跨 stream 复用需要同步。这带来一个实际的坑: 在非默认 stream 上创建、在默认 stream 上使用的张量, 必须调 `tensor.record_stream(stream)`, 否则它可能在消费 kernel 还没跑完时就被回收复用, 产生静默的数据损坏。详见第 9 章。

面试考点. Q8: `zero_grad(set_to_none=True)` 和 `set_to_none=False` 差在哪?

A: `False` 是把 `.grad` 张量原地填 0, 张量还占着显存; `True` 是把 `.grad` 置为 `None`, 张量被释放, 下次 backward 时重新分配。前者省一次分配但常占 $2P-4P$ 显存, 后者省显存。torch 2.0 起默认值就是 `True`。副作用要知道: `grad` 变成 `None` 后, 任何直接读 `p.grad` 做统计的代码要处理 `None`; 带 momentum 的 SGD 在“某参数这步没有梯度”时, 两种模式的更新行为会不一样 (`None` 时跳过, 填 0 时仍然按 momentum 更新)。

CUDA 执行模型：异步、stream、同步点

这一章解释一件事：你的 **Python** 代码和 **GPU** 上真正发生的事情，在时间上是错开的。 `y = a @ b` 返回时，那个 matmul 很可能一个 SM 都还没碰。理解这一点之后，一连串面试题就都通了——为什么你用 `time.time()` 测的 kernel 时间是假的、为什么 `loss.item()` 会让训练变慢、为什么通信能和计算重叠、CUDA graph 到底省了什么。这些是 AI Infra 面试的白板常客，尤其“写一段正确的 GPU 计时代码”几乎必考。

kernel launch 是异步的

CPU 调一次 `torch.mm`，走完 dispatcher (第 7 章) 之后最终是一次 `cudaLaunchKernel`。这个调用把 kernel 塞进 stream 的队列就立刻返回，不等 GPU 执行。

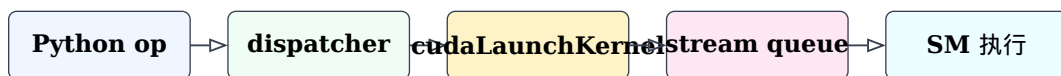


图 15 一次 op 的路径。前三步在 CPU 上，返回后 CPU 就继续往下跑；后两步在 GPU 上异步发生。稳态下 CPU 通常领先 GPU 几百次 launch。

这个设计是为性能服务的：单次 launch 的 CPU 开销大约几微秒，而一个 kernel 在 GPU 上可能跑几十到几百微秒。异步让 CPU 得以“跑在前面”，把队列填满，GPU 就不会因为等 CPU 而空转。

直接后果是你不能用 **CPU** 时钟测 **GPU** 时间：

```
import time, torch

x = torch.randn(4096, 4096, device="cuda")

t0 = time.perf_counter()
y = x @ x
t1 = time.perf_counter()
print(f"{{t1 - t0}} * 1e3: .3f} ms") # 测到的是 launch 开销，不是 matmul 时间
```

这段代码在 A100 上会打出几十微秒级的数字，而一个 4096^3 的 fp32 matmul 显然不可能这么快。它测的是“把 kernel 塞进队列”的时间。

正确的计时模板（这是白板题的标准答案，背下来）：

```
import torch

def bench(fn, warmup: int = 20, iters: int = 100) -> float:
    """返回 fn 的中位数耗时（毫秒）。fn 不应该包含 CPU 同步。"""
    for _ in range(warmup): # 1) warmup: JIT、cudnn 选算法、allocator 稳态
        fn()
    torch.cuda.synchronize()

    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    samples = []
    for _ in range(iters):
        start.record() # 2) event 打在 stream 里，测的是 GPU 时间
```



```

    fn()
    end.record()
    end.synchronize()
    samples.append(start.elapsed_time(end))    # 毫秒, float

    samples.sort()                            # 3) 取中位数, 不取平均: 平均被离群值污染
    return samples[len(samples) // 2]

print(bench(lambda: x @ x))

```

三个要点缺一不可: **warmup** (第一次调用包含 cudnn/cublas 的算法选择、Triton 编译、allocator 首次 `cudaMalloc`)、用 **CUDA event** 或 **synchronize()**、多次取中位数。第 11 章会给一个更省事的封装 `torch.utils.benchmark.Timer`, 它自动处理这三件事, 日常测量优先用它。

Key insight. CUDA event 记录的是“GPU 走到 stream 里这个位置的时刻”, 所以 `start.elapsed_time(end)` 是纯 GPU 时间, 不含 CPU launch 开销。如果你想测的恰恰是“CPU 会不会成为瓶颈”, 那就要用 wall-clock 测整个 loop 并对比 GPU 时间——两者的差就是 CPU 侧的开销。

隐式同步点

异步是默认行为, 但很多操作会强制 CPU 等 GPU。这张表是面试直接问的:

操作	为什么同步
<code>t.item()</code>	要把标量搬回 CPU, 必须等产生它的 kernel 跑完
<code>t.cpu()</code> / <code>t.to("cpu")</code>	D2H 拷贝, 且目标是非 pinned 内存时必然同步
<code>t.numpy()</code> / <code>t.tolist()</code>	同上, 本质是 D2H
<code>print(t)</code>	格式化要读取数值 → D2H
<code>if t:</code> / <code>bool(t)</code> / <code>float(t)</code>	Python 需要真值 → D2H
<code>t.nonzero()</code> / <code>t[mask]</code>	输出 shape 依赖数据, CPU 要知道 shape 才能分配
<code>torch.cuda.synchronize()</code>	显式等整个 device
<code>Event.synchronize()</code> / <code>Work.wait()</code>	显式等一个点
allocator 拿不到空间	触发 <code>cudaFree</code> 重试, <code>cudaFree</code> 是同步的
<code>assert</code> / <code>torch.isnan(t).any().item()</code>	<code>.item()</code> 的伪装形式, 最容易漏

`t.nonzero()` 这类“输出 shape 依赖数据”的 op 是最不容易想到的一类。同理还有 `torch.unique`、`torch.masked_select`、以及任何用 tensor 值做 Python 控制流的地方。

为什么 **loss.item()** 会拖慢训练: 稳态下 CPU 领先 GPU 几百次 launch, 队列是满的。`loss.item()` 一来, CPU 必须停下来等 GPU 把这个 step 的所有 kernel 排干; 等它拿到值继续往下跑时, 队列已经空了, GPU 只能一边等 CPU 重新填队列一边零星地干活。损失的时间约等于“CPU 发完一个 step 的 launch 所需的时间”。小模型、小 batch 时这个数很可观。

```

# 差: 每 step 一次同步
for x, y in loader:
    loss = train_step(x, y)
    logger.log(loss.item())

```



```
# 好：在 GPU 上累积，按 log 间隔（比如 50 step）才同步一次
loss_acc = torch.zeros((), device="cuda")
for i, (x, y) in enumerate(loader):
    loss = train_step(x, y)
    loss_acc += loss.detach()
    if (i + 1) % 50 == 0:
        logger.log(loss_acc.item() / 50) # 50 步只同步 1 次
        loss_acc.zero_()
```

注意用 `loss.detach()` 而不是 `loss`，否则会把整张 autograd 图串起来（第 8 章）。怎么抓出自己代码里的隐式同步：

```
import torch

torch.cuda.set_sync_debug_mode("warn") # 或 "error" 直接抛异常
# 底层等价写法：torch._C._cuda_setSyncDebugMode(1)

train_one_step() # 任何隐式同步都会报出调用位置

torch.cuda.set_sync_debug_mode("default")
```

"error" 模式在排查“这个 step 里到底谁在同步”时最好用——它直接给你一个带完整 Python 栈的异常。注意 `torch.cuda.synchronize()` 这类显式同步不会被它报出来，它只管隐式的。

Warning. 别把 `CUDA_LAUNCH_BLOCKING=1` 当性能工具。它让每次 launch 都同步，唯一用途是让 CUDA 报错的栈落在真正出错的那一行（否则异步执行下，报错会出现在之后某个不相关的 op 上）。它会训练慢好几倍，调完必须去掉。

stream：并行的执行队列

Stream 是一个 FIFO 的 kernel 队列。规则只有两条：同一个 stream 内的 kernel 严格按提交顺序执行；不同 stream 之间没有顺序保证，硬件资源够就并行跑。

PyTorch 默认把所有 op 提交到当前 device 的默认 stream，所以你写的代码天然顺序语义的。要并行就得自己开 stream，并用 event 手动建立依赖关系：

API	语义
<code>s = torch.cuda.Stream()</code>	新建一个 stream
<code>with torch.cuda.stream(s):</code>	块内的 op 提交到 s
<code>torch.cuda.current_stream()</code>	当前 stream
<code>s2.wait_stream(s1)</code>	s2 后续的 kernel 等 s1 当前已提交的全部完成
<code>e = torch.cuda.Event(); e.record(s1)</code>	在 s1 里打一个点
<code>s2.wait_event(e)</code>	s2 等这个点

`wait_stream` 和 `wait_event` 都是在 GPU 上建立依赖，CPU 不阻塞——这是它们和 `synchronize()` 的根本区别。

下面是一个真能跑的 overlap 例子：把“搬下一个 batch 上 GPU”和“算当前 batch”重叠起来。这是 dataloader 之后最常见的一处 overlap 机会。

```
import torch

copy_stream = torch.cuda.Stream()

def prefetch(batch_cpu: torch.Tensor) -> torch.Tensor:
    """在 copy_stream 上异步搬运。batch_cpu 必须是 pinned memory。"""
    with torch.cuda.stream(copy_stream):
        return batch_cpu.to("cuda", non_blocking=True)

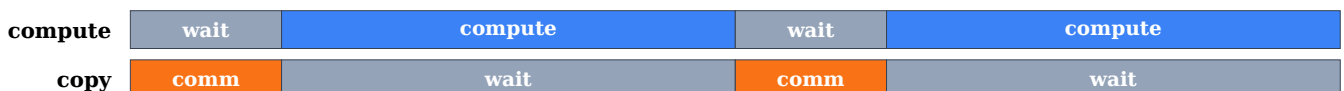
it = iter(loader)
nxt = prefetch(next(it))

for _ in range(n_steps):
    cur = torch.cuda.current_stream()
    x = nxt
    cur.wait_stream(copy_stream)      # 主 stream 等这次拷贝完成
    x.record_stream(cur)              # 关键：告诉 allocator 主 stream 也在用 x

    try:                              # 先发起下一次拷贝，再开始计算
        nxt = prefetch(next(it))
    except StopIteration:
        nxt = None

    loss = model(x).sum()              # 与 nxt 的 H2D 拷贝并行
    loss.backward()
    opt.step()
    opt.zero_grad(set_to_none=True)
```

无 prefetch：每 step 先等拷贝，再算



有 prefetch：拷贝 k+1 与 compute k 并行



图 16 两个 stream 的时间线（示意，比例不代表实测）。overlap 之后拷贝时间被藏进计算时间里，前提是拷贝时间 < 计算时间。

Note. `dist.all_reduce(..., async_op=True)` 不需要你自己开 stream——`ProcessGroupNCCL` 内部就有专属 stream，返回的 `Work` 对象调 `work.wait()` 是让当前 stream 去等它，而不是阻塞 CPU。所以 DDP 的 overlap 是自动的（第 18 章）。自己开 stream 主要用于拷贝、以及把两段互不依赖的计算并行。

stream 与 allocator 的坑：record_stream

caching allocator 是 stream-aware 的（第 8 章）：每个 block 记着自己在哪个 stream 上分配。当这个 block 被释放时，allocator 认为“只要那个 **stream** 上的后续操作能保证顺序，就可以立刻复用它”。

Warning. 如果一个张量在 stream A 上分配、在 stream B 上使用，而 Python 侧的引用在 B 的 kernel 跑完之前就消失了，allocator 会以为它自由了，把这块显存分给 A 上的下一个请求。于是 B 的 kernel 读到的是别人写进去的数据——静默的数值错误，不报错、不崩，只是 **loss** 变成 **NaN** 或者悄悄训坏。

```
# 错：x 在 copy_stream 上分配，在默认 stream 上使用
with torch.cuda.stream(copy_stream):
    x = cpu_batch.to("cuda", non_blocking=True)
torch.cuda.current_stream().wait_stream(copy_stream)
y = model(x)          # 用了，但 allocator 不知道默认 stream 在用它

# 对：显式登记
with torch.cuda.stream(copy_stream):
    x = cpu_batch.to("cuda", non_blocking=True)
torch.cuda.current_stream().wait_stream(copy_stream)
x.record_stream(torch.cuda.current_stream())
y = model(x)
```

`record_stream(s)` 的语义是“这块显存要等 `s` 上当前已提交的工作全部完成之后才可以被复用”。规则很简单：任何跨 **stream** 使用的张量都要 `record_stream`。这类 bug 极难调试，因为它依赖时序，小规模测试往往复现不出来。

pinned memory 与 non_blocking=True

主机内存默认是 pageable 的，操作系统可以随时把它换出去。DMA 引擎不能直接读这种内存，所以一次 pageable H2D 拷贝实际是“驱动先拷到内部的 staging buffer，再 DMA”——中间那步是同步的 CPU 拷贝。

所以 `non_blocking=True` 只在 **pinned (page-locked)** 内存上才真的异步。在 pageable 内存上传 `non_blocking=True` 不报错也不生效，这是最常见的“我设了却没效果”。

```
x = torch.randn(1024, 1024)          # pageable
x.to("cuda", non_blocking=True)       # 实际同步

x = torch.randn(1024, 1024).pin_memory() # pinned
x.to("cuda", non_blocking=True)       # 真异步
```

`DataLoader(pin_memory=True)` 就是让 worker 把 batch 放进 pinned 内存，这样训练循环里的 `.to("cuda", non_blocking=True)` 才有意义。两者要配对使用，只开一个没用。

代价：pinned 内存不能被换出，占的是“操作系统不能动”的物理内存；`pin_memory()` 本身是一次同步的分配 + 拷贝，不便宜。所以不要在热路径里反复 `pin_memory()`，要么让 `DataLoader` 做，要么自己预分配一组固定的 pinned buffer 循环用。

D2H 方向同理：`t.to("cpu", non_blocking=True)` 要目标是 pinned 才异步，而且返回后你不能立刻读它——必须先同步。忘了这一点就会读到未写完的数据。

CUDA graph：把一串 launch 固化成一次

每个 kernel 都要一次 `cudaLaunchKernel`，CPU 开销几微秒。一个 Transformer 的 step 有几千个 kernel，CPU 侧就是几毫秒到十几毫秒的固定开销。当 kernel 本身很小（小 batch、LLM 的 decode 阶段一次只算一个 token）时，这个开销会成为瓶颈，GPU 大部分时间在等 CPU 发指令。

CUDA graph 的做法是：录一遍，把整个 launch 序列（含它们之间的依赖）存成一张图；之后 replay 只有一次提交开销。

```
import torch

model = MyModel().cuda().eval()

# 静态输入输出 buffer —— 地址在 capture 后就不能变了
static_x = torch.randn(8, 512, device="cuda")

# 1) warmup: 必须在 side stream 上跑几步，让 cudnn/cublas 选好算法、
#    让 allocator 进入稳态，否则 capture 会把这些一次性行为录进去
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for _ in range(3):
        static_y = model(static_x)
torch.cuda.current_stream().wait_stream(s)

# 2) capture
g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g):
    static_y = model(static_x)

# 3) replay: 只改 buffer 的内容，不能换 buffer
for batch in batches:
    static_x.copy_(batch)          # 原地写入，地址不变
    g.replay()
    consume(static_y)              # static_y 每次 replay 后被覆盖，要用就先 copy 走
```

限制清单（面试会挨个问）：

- **shape** 必须静态。换 batch size 或 seq len 就要重新 capture 一张图。变长输入通常 pad 到几个固定档位，每档一张图。
- 地址必须固定。输入输出都得是预分配的 buffer，用 `copy_` 灌数据。这也意味着 replay 之后 `static_y` 会被下一次 replay 覆盖。
- 不能有 **CPU** 同步。被 capture 的代码里出现 `.item()`、`print(tensor)`、`torch.cuda.synchronize()` 会直接失败。
- 不能有数据依赖的控制流。`if t.max() > 0`：这种在 capture 时走的哪个分支就永远是哪个分支。
- 不能有随机性副作用之外的 **CPU** 逻辑。RNG 是特殊处理过的（graph 会捕获 generator 状态），但普通 Python 逻辑在 replay 时根本不执行。

如果只是给一个 `nn.Module` 上 `graph`, `torch.cuda.make_graphed_callables` 会帮你把 `forward` 和 `backward` 都 `capture` 好, 可以直接参与 `autograd` :

```
model = torch.cuda.make_graphed_callables(model, (static_x,), num_warmup_iters=3)
loss = model(static_x).sum()
loss.backward() # backward 也是 replay
```

实践中你几乎不会手写这些——`torch.compile(mode="reduce-overhead")` 底层就是 `CUDA graph`, 它顺带帮你处理了 `buffer` 管理和 `shape` 分档。细节见第 15 章。

Key insight. `CUDA graph` 消除的是 **CPU launch** 开销, 一点也不加速 `kernel` 本身。所以判据很清晰: 如果你的 `GPU` 已经吃满 (util 接近 100%、`timeline` 上 `kernel` 之间没有空隙), `graph` 带来的收益是零。它只在 `CPU-bound` 的场景有意义。

CPU-bound 还是 GPU-bound

这是优化的第一个分叉口, 判错方向就白干。三个由粗到细的方法:

1. `nvidia-smi dmon` 或 `nvidia-smi -l 1` 看 **SM** 利用率。持续 90%+ 说明 `GPU` 忙; 在 40%-70% 之间跳说明有空隙。注意 `util` 的定义是“这一采样周期内有没有 `kernel` 在跑”, 一个只用 1 个 `SM` 的 `kernel` 也算 100%, 所以它只能排除 `CPU-bound`, 不能证明 `GPU` 用得好。
2. 加倍 **batch size** 看 **step time**。step time 涨得远小于 2 倍 → 之前 `GPU` 没吃满, 大概率 `CPU-bound` 或访存瓶颈。step time 接近线性涨 → `GPU-bound`。这个实验两分钟就能做, 性价比最高。
3. 看 **profiler** 或 `nsys` 的 **timeline** 找 **gap**。`kernel` 之间有肉眼可见的空白, 且空白期 `CPU` 那一行在忙 → `CPU launch` 跟不上。空白期 `CPU` 也闲 → 在等 `data` 或等通信。第 11 章讲怎么读这张图。

`CPU-bound` 的典型对策: `torch.compile` (减少 `kernel` 数量) `CUDA graph` (消除 `launch` 开销) 去掉 `loop` 里的 `.item()` 和 `print`、把 `Python` 侧的 `per-parameter` 循环换成 `torch._foreach_*` 或 `fused optimizer`。

多进程与 CUDA

CUDA context 不能跨 `fork` 使用。一旦父进程初始化过 `CUDA` (哪怕只是 `torch.cuda.is_available()` 之后建了 `context`), `fork` 出来的子进程碰 `CUDA` 就会报 `Cannot re-initialize CUDA in forked subprocess`。原因是 `CUDA driver` 的状态里有大量文件描述符和 `mmap` 区域, `fork` 只复制内存不复制这些内核态资源。

```
import torch.multiprocessing as mp

def worker(rank):
    torch.cuda.set_device(rank)
    ...

if __name__ == "__main__":
    mp.spawn(worker, nprocs=2) # spawn, 不是 fork
    # 或者显式: mp.set_start_method("spawn", force=True)
```

`DataLoader(num_workers=4)` 在 `Linux` 上默认用 `fork`, 这没问题——只要 **worker** 里不碰 **CUDA**。这也是“不要在 `Dataset.__getitem__` 里 `.cuda()`”的硬性理由之一。

设备可见性有两层，容易混：

- `CUDA_VISIBLE_DEVICES=2,3` 是进程级的过滤，进程内看到的 `cuda:0` 就是物理的 GPU 2。它必须在 CUDA 初始化之前设好，在 Python 里 `os.environ[...] = ...` 得写在 `import torch` 触发初始化之前才有效——最稳的是在 shell 里设。
- `torch.cuda.set_device(local_rank)` 是设进程的默认 **device**，之后 `torch.randn(..., device="cuda")` 落在这张卡上。分布式训练每个进程启动后第一件事就该是这一句，否则所有进程默认都用 `cuda:0`，NCCL 会直接挂或者显存爆掉。

`torchrun` 会帮你设好 `LOCAL_RANK`，标准开头是：

```
import os, torch, torch.distributed as dist

local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)                # 必须在 init_process_group 之前
dist.init_process_group(backend="nccl", device_id=torch.device(f"cuda:
{local_rank}"))
```

面试考点

面试考点. **Q1**：写一段正确测量一个 CUDA op 耗时的代码，说明每一步为什么必要。

A: `warmup` 若干次(排除 cudnn/cublas 算法选择、Triton 编译、allocator 首次 `cudaMalloc`) → `torch.cuda.synchronize()` → 用 `torch.cuda.Event(enable_timing=True)` 在 stream 里打 start/end，`end.synchronize()` 后读 `start.elapsed_time(end)` → 跑几十次取中位数。直接用 `time.perf_counter()` 包住一个 op 测到的是 launch 开销(几微秒)，不是 kernel 时间。日常测量用 `torch.utils.benchmark.Timer`，它把这些都封装好了。

面试考点. **Q2**：列举 PyTorch 里的隐式同步点。

A: `.item()`、`.cpu()` / `.numpy()` / `.tolist()`、`print(tensor)`、`bool(tensor)` 或用 tensor 值做 if、输出 shape 依赖数据的 op (`nonzero`、`masked_select`、`unique`、布尔索引)、`Work.wait()`、以及 allocator 空间不足时触发的 `cudaFree` 重试。用 `torch.cuda.set_sync_debug_mode("error")` 可以把它们全抓出来。

面试考点. **Q3**：为什么训练 loop 里每 step 打 `loss.item()` 会变慢？

A: 稳态下 CPU 领先 GPU 几百次 launch，队列是满的。`.item()` 强制 CPU 等 GPU 排干队列；等它拿到值时队列已空，GPU 得一边等 CPU 重新填一边零星工作。损失约等于 CPU 发完一个 step 的 launch 所需时间，小模型上占比可观。做法是在 GPU 上用 `loss.detach()` 累积，每 N 步同步一次。

面试考点. **Q4**：`torch.cuda.synchronize()`、`Event.synchronize()`、`stream.wait_stream()` 三者区别？

A: `torch.cuda.synchronize()` 阻塞 CPU 直到整个 device 上所有 stream 排空；`Event.synchronize()` 阻塞 CPU 直到某一个点；`stream.wait_stream()` / `wait_event()` 不阻塞 CPU，只是在 GPU 上给这个 stream 插一条依赖边。做 overlap 只能用第三种，用前两种就把并行性同步掉了。

面试考点. Q5：跨 stream 用同一块 tensor 要注意什么？

A：必须 `tensor.record_stream(consuming_stream)`。caching allocator 按分配 stream 记账，只要 Python 引用消失它就认为可复用；如果消费 kernel 在另一个 stream 上还没跑完，这块显存就会被别的分配覆盖，产生静默的数值错误。`record_stream` 告诉 allocator 必须等那个 stream 上已提交的工作完成才能复用。这类 bug 依赖时序，小规模复现不出来。

面试考点. Q6：`non_blocking=True` 什么时候真的异步？

A：只有源（H2D）或目标（D2H）是 pinned / page-locked 内存时。pageable 内存的拷贝需要驱动先做一次同步的 staging 拷贝，`non_blocking=True` 会被静默忽略。所以 `DataLoader(pin_memory=True)` 和 `.to("cuda", non_blocking=True)` 必须配对。代价是 pinned 内存不可换出，且 `pin_memory()` 本身是一次同步分配。

面试考点. Q7：CUDA graph 解决什么问题？有什么限制？什么时候没用？

A：解决 CPU 侧的 kernel launch 开销——把成千上万次 `cudaLaunchKernel` 压成一次 replay。限制：shape 必须静态、输入输出地址必须固定（用 `copy_` 灌数据）被 capture 的段里不能有 CPU 同步和数据依赖的控制流。GPU 已经吃满时收益为零，它只在 CPU-bound 场景（小 batch、LLM decode）有意义。`torch.compile(mode="reduce-overhead")` 底层就是它。

面试考点. Q8：为什么多进程用 CUDA 必须 spawn 不能 fork？DataLoader 的 worker 为什么可以 fork？

A：CUDA context 里有大量文件描述符和 mmap 映射，`fork` 只复制内存不复制这些内核态资源，子进程碰 CUDA 会报 `Cannot re-initialize CUDA in forked subprocess`。DataLoader 的 worker 用 `fork` 没问题是因为它们只做 CPU 侧的数据处理，从不触碰 CUDA——这也是“不要在 `Dataset.__getitem__` 里 `.cuda()`”的硬理由。

随机性、确定性与可复现

“我设了 seed，为什么两次跑出来的 loss 不一样？”——这是最能区分“背过八股”和“真训过模型”的一道题。标准答案不是“再多设几个 seed”，而是：浮点加法不满足结合律，而 GPU 上的规约顺序不是固定的。这一章先把随机数的来源理清楚，给一份完整的复现清单，然后解释为什么清单做全了结果仍可能有差异，最后给数值调试和 NaN 排查的方法。核心结论提前说：工程上追求的是“可复现的实验结论”，不是“逐 bit 相同的数值”。

随机数从哪来

PyTorch 里不止一个随机数生成器 (generator)，这是第一个坑：

Generator	说明
CPU generator	<code>torch.default_generator</code> ，管所有 CPU 上的 <code>randn</code> / <code>dropout</code> 等
每个 CUDA device 一个	device 0 和 device 1 的 generator 是独立的，状态互不影响
<code>DataLoader</code> worker	每个 worker 进程的 seed 由主进程派生，见下
用户显式创建的	<code>g = torch.Generator(device="cuda"); g.manual_seed(0)</code> ，传给需要的 op
Python <code>random</code>	标准库， <code>random.shuffle</code> 之类用它
NumPy	<code>np.random</code> ，很多 <code>Dataset</code> 的数据增强用它

`torch.manual_seed(n)` 会同时设置 CPU generator 和所有 CUDA device 的 generator(等价于额外调了 `torch.cuda.manual_seed_all(n)`)。所以日常不需要单独写 `torch.cuda.manual_seed`——这也是一道爱问的细节题。但它不会管 Python 的 `random` 和 NumPy。

`DataLoader` 的 worker seed 是派生的：主进程从 `DataLoader` 的 `generator` 里抽一个 `base_seed`，worker *i* 拿到 `base_seed + i`，然后在 worker 里用它设置 `torch.manual_seed`、`random.seed` 和 `np.random.seed`。

Note. “必须写 `worker_init_fn` 去 seed NumPy”是一条过时的建议——较新版本的 PyTorch 已经在 worker 启动时帮你 seeded 了 `random` 和 `numpy`。仍然需要 `worker_init_fn` 的场合是：你用了其他带全局 RNG 状态的库 (OpenCV、`albumentations`、某些 tokenizer)，或者想按自己的规则派生 seed。

但要注意 `base_seed` 是每次创建 **iterator** 时重新抽的。所以第 2 个 epoch 的数据增强随机性和第 1 个不同(这是我们想要的)，而如果你想让整个 job 从头到尾可复现，必须给 `DataLoader` 传一个显式的 `generator`。

完整的复现清单

```
import os, random
import numpy as np
import torch

def seed_everything(seed: int = 42) -> torch.Generator:
    os.environ["PYTHONHASHSEED"] = str(seed) # 只对新起的子进程生效
```

```

    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)                # 含所有 CUDA device
    g = torch.Generator()
    g.manual_seed(seed)
    return g

g = seed_everything(42)

def worker_init_fn(worker_id: int) -> None:
    # 只在用了 torch 不负责 seed 的第三方库时才需要
    s = torch.initial_seed() % 2**32
    np.random.seed(s)
    random.seed(s)

loader = torch.utils.data.DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=4,
    generator=g,                          # 决定 shuffle 顺序与 worker 的 base_seed
    worker_init_fn=worker_init_fn,
    persistent_workers=True,
)

```

`PYTHONHASHSEED` 影响的是 `str/bytes` 的哈希值,从而影响 `set` 和 `dict` 的迭代顺序。在 Python 里设 `os.environ` 对当前进程无效(哈希种子在解释器启动时就定了),要真正生效必须在 `shell` 里 `export PYTHONHASHSEED=42` 或者用 `torchrun` 传环境变量。大多数训练代码不依赖 `set` 顺序,所以这条经常可以跳过——但如果你的词表构建、数据分片用了 `set`,它就是必要的。

为什么设了 **seed** 结果还不一样

`seed` 只控制随机数序列。结果的差异还有另外一整类来源,它们和 `seed` 无关:

- **非确定性 kernel**。任何用 `atomicAdd` 做规约的 kernel,多个线程往同一个地址累加的到达顺序由硬件调度决定,每次都不同。浮点加法不满足结合律(下一节),所以顺序不同结果就不同。典型 op: `scatter_add_`、`index_add_`、`index_select` 的反向、`bincount`、`embedding` 的反向(稠密梯度累加)、`nll_loss` 的反向、各种上采样(`interpolate`)的反向、`histc`。
- **cuda benchmark**。`torch.backends.cudnn.benchmark = True` 会在第一次遇到某个 shape 时实测几种卷积算法挑最快的。机器负载不同 → 挑中不同算法 → 数值不同。
- **TF32 / reduced precision**。见后面单独一节。
- 多线程规约顺序。CPU 上的 `sum` 用 OpenMP 分块并行,块数取决于 `torch.set_num_threads()` 和实际线程数,块数变了归约树就变了。
- **NCCL** 的规约顺序。AllReduce 在不同拓扑(ring vs tree)不同 message size 下走不同算法,规约顺序不同。同一个 job 换机器、换 `NCCL_ALGO` 就会得到不同的 bit。这是“分布式训练两次跑 loss 逐 bit 不一致”的直接原因。
- **torch.compile / autotune**。Inductor 的 kernel 选择、tile 大小 autotune 结果与机器负载相关,而不同的 tile 划分意味着不同的规约顺序。

- 未初始化内存。极少见但存在：某些 op 会读到 `torch.empty` 的未初始化内容（通常是 bug），`torch.utils.deterministic.fill_uninitialized_memory = True`（默认已开）可以让这种情况暴露出来。

浮点不结合律：所有问题的根

这是根因，也是面试里最该讲清楚的一点。fp32 只有 24 bit 有效位（尾数 23 bit + 隐含的 1），每次加法都要把结果舍入到最近的可表示值。舍入发生在哪、发生几次，取决于加法的顺序。

$$(a + b) + c \neq a + (b + c) \quad \text{在浮点下}$$

一个能直接跑的最小例子（fp32， $\text{eps} = 2^{-23} \approx 1.19 \times 10^{-7}$ ）：

```
import torch

a = torch.tensor(1.0, dtype=torch.float32)
b = torch.tensor(6e-8, dtype=torch.float32) # 比 eps 小一点

print(repr(((a + b) + b).item())) # 1.000000238418579 → 1 + 2*eps
print(repr((a + (b + b)).item())) # 1.0000001192092896 → 1 + 1*eps
```

左边先算 $1 + 6e-8$ ， $6e-8$ 大于 $\text{eps}/2$ 所以向上进一位得到 $1 + \text{eps}$ ，再加一次又进一位得到 $1 + 2*\text{eps}$ 。右边先算 $6e-8 + 6e-8 = 1.2e-7$ ，这个数刚好略大于 eps ，加到 1 上只进一位。同样三个数、同样两次加法，差了一个 eps 。

放大到真实规约的规模（A100，fp32， 10^6 个 `randn(seed=0)` 元素，只是换个求和顺序）：

```
x = torch.randn(1_000_000) # torch.manual_seed(0)
x.sum() # -1561.4530029296875
x.flip(0).sum() # -1561.4528808593750 ← 同样的数，反着加
x.double().sum() # -1561.4529594577189 ← fp64 参考值
```

相对误差量级 10^{-7} ，正好是 fp32 的精度极限。GPU 上的规约是分块的树形归约，块的划分和 kernel 的 grid 配置有关，所以“顺序”在你眼里是不可控的。

再看一个 GPU 上真实的非确定性 op（A100，`index_add_` 把 2^{20} 个 fp32 累加进 64 个 bin，同一个进程内连续跑 5 次）：

```
idx = torch.randint(0, 64, (1 << 20,), device="cuda")
src = torch.randn(1 << 20, device="cuda")
outs = []
for _ in range(5):
    o = torch.zeros(64, device="cuda")
    o.index_add_(0, idx, src)
    outs.append(o.clone())
# 5 次结果互不相同，两两最大绝对差约  $1e-3$ 
```

注意这里 seed 完全固定、输入完全一样，差异纯粹来自 `atomicAdd` 的到达顺序。 $\frac{2^{20}}{64} = 16384$ 次累加进一个 bin，误差就累积到 10^{-3} 这个量级了。

Key insight. 面试标准答案：**seed** 保证的是“随机数序列相同”，不是“数值结果相同”。GPU kernel 的并行规约顺序不固定，而浮点加法不满足结合律，所以逐 bit 相同需要额外强制 (`use_deterministic_algorithms`)，且要付性能代价。分布式场景下 NCCL 的规约顺序还会随拓扑和 message size 变，更难保证。

`use_deterministic_algorithms` 与它的代价

```
import torch

torch.use_deterministic_algorithms(True)           # 核心开关
torch.backends.cudnn.deterministic = True          # 卷积走确定性算法
torch.backends.cudnn.benchmark = False             # 关掉算法自动选择
```

`use_deterministic_algorithms(True)` 的行为分三种：有确定性实现的 op 切换到确定性版本（通常更慢，因为要放弃 `atomicAdd` 改用排序或分段规约）；没有确定性实现的 op 直接抛 `RuntimeError`，错误信息里会告诉你是哪个 op；少数 op 需要额外的环境变量配合。

最常见的那个额外要求是 cuBLAS：

```
export CUBLAS_WORKSPACE_CONFIG=:4096:8    # 或 :16:8
```

cuBLAS 在多个 stream 上复用 workspace 时，split-k 的分块数会随可用 workspace 变化，导致规约顺序不确定。固定 workspace 配置就固定了分块。不设这个变量而开了确定性模式，涉及 matmul 的地方会直接报错。

`torch.use_deterministic_algorithms(True, warn_only=True)` 是一个实用的中间档：不支持的 op 只 warning 不报错，让你先跑起来、同时知道哪些地方仍然不确定。

三个档位的取舍：

版本	核心思路	相对成本 / 收益
不做任何事	只设 seed	零代价；loss 曲线形状一致，逐点数值不同
<code>warn_only=True</code>	1. <code>cudnn.deterministic</code> 关 <code>benchmark</code>	轻微变慢；知道哪些 op 还不确定
完全确定性	1. <code>use_deterministic_algorithms(True)</code> + <code>CUBLAS_WORKSPACE_CONFIG</code>	部分 op 报错需改代码；卷积和规约类 op 明显变慢

TF32：同样的代码在 V100 和 A100 上结果不同

A100 起的 Tensor Core 支持 TF32：和 fp32 一样的 8 bit 指数范围，但尾数只有 10 bit。用它做 matmul 快得多，代价是精度从约 7 位有效十进制数掉到约 3 位。

默认值经常被记错，这是个好的考点。在当前的 PyTorch 里：

```
torch.backends.cuda.matmul.allow_tf32    # False — matmul 默认不用 TF32
torch.backends.cudnn.allow_tf32          # True  — 卷积默认用 TF32
torch.get_float32_matmul_precision()     # "highest"
```

也就是说 **fp32 matmul** 默认是不走 **TF32** 的 (1.12 版本改过一次默认值, 从 True 改成 False, 因为太多人被静默的精度损失坑到); 而 **cuda** 卷积默认是走 **TF32** 的。要打开 **matmul** 的 **TF32**:

```
torch.set_float32_matmul_precision("high") # 允许 TF32
# "highest" = 严格 fp32; "high" = TF32; "medium" = 允许 bf16 级别
# 等价的底层开关: torch.backends.cuda.matmul.allow_tf32 = True
```

较新版本还提供了更细的 `fp32_precision` 接口 (`torch.backends.cuda.matmul.fp32_precision`, 取值 `"none"` / `"tf32"` / `"ieee"`), 语义更明确, 逐步替代 `allow_tf32` 这个布尔量。

Warning. **TF32** 是“同样的代码在不同 GPU 上结果不同”的头号原因: V100 没有 **TF32**, A100 有; 同一份代码在两台机器上跑, 卷积部分的精度就不一样。做跨机器的数值对比时先把 `torch.backends.cudnn.allow_tf32 = False`、`torch.set_float32_matmul_precision("highest")` 关掉, 再比。

反过来, 正常训练时你应该开 **TF32**——**bf16/TF32** 级别的精度对 **SGD** 类优化过程完全够用, 收益是实打实的速度。

什么时候需要 **bit-wise** 确定性

需要的场景其实很少:

- 调试。定位“哪一步开始出 NaN”、二分查找引入 bug 的 commit, 必须先消除噪声, 否则你分不清是自己改坏了还是浮点抖动。
- 回归测试。CI 里断言“这个 kernel 的输出和黄金值一致”。
- 复现别人的 **bug**, 尤其是那种“跑到第 3721 步崩”的。
- 某些合规/审计要求。

不需要的场景是大多数: 正常训练只需要统计等价。两次跑出来 loss 曲线逐点不同但形状一致、最终指标在同一个方差范围内, 这就是合格的可复现性。强行追求逐 bit 相同要付性能代价, 而且在多机场景下 (NCCL 拓扑、机器数变化) 根本做不到。

Key insight. 面试怎么答: “我追求的是可复现的实验, 不是逐 **bit** 相同的数值。”具体做法是固定 seed 和数据顺序, 把超参和 commit hash 记进 log, 然后用多个 **seed** 跑同一配置来估计指标的方差——只有当两个配置的差异大于种子间方差时, 才说这个改动有效。这个回答同时展示了工程能力和实验素养, 比背 API 强得多。

数值调试: 怎么比较两个结果

比较不能用 `==`。 `torch.allclose` 的判据是 $|a - b| \leq atol + rtol \cdot |b|$:

场景	rtol / atol	理由
fp32 单个 op	1e-5 / 1e-8	接近 fp32 的 eps 量级
fp32 长规约 (大 matmul)	1e-4 / 1e-6	误差随规约长度累积
bf16	1.6e-2 / 1e-5	bf16 只有 8 bit 尾数
fp16	1e-3 / 1e-5	10 bit 尾数

优先用 `torch.testing.assert_close`，它比 `allclose` 好在三点：按 dtype 自动选默认容差、失败时打印最大差值出现在哪个位置、还会检查 dtype 和 shape 是否一致。

```
import torch

torch.testing.assert_close(mine, reference) # 用 dtype 默认容差
torch.testing.assert_close(mine, reference, rtol=1e-4, atol=1e-6)
```

两个实用原则：

1. 低精度实现要和高精度参考比，不是和另一个低精度实现比。验证 bf16 kernel 时，参考值用 fp64 算，然后看 bf16 结果的相对误差是否在 bf16 的理论精度内。两个 bf16 实现互相比，误差会互相掩盖。
2. 比 **loss** 曲线，不比单点 **loss**。单个 step 的 loss 差 10^{-4} 什么都说明不了。要看的是几百步的曲线是否重合、以及最终指标是否落在多种子的方差范围内。

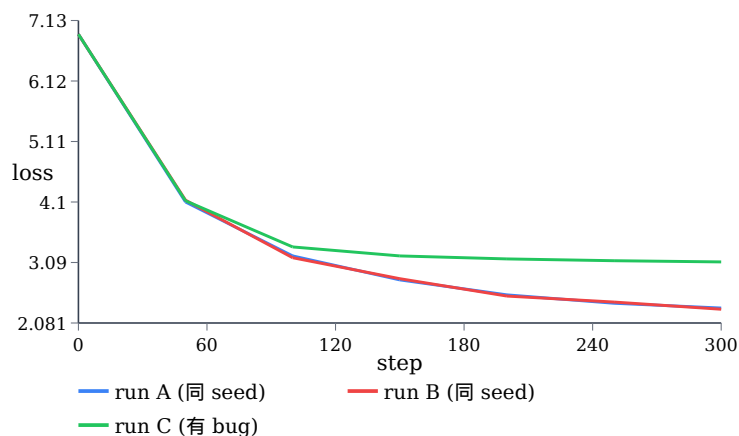


图 17 示意图，非实测数据。A 和 B 是同一配置的两次运行：逐点数值不同（浮点噪声），但曲线重合，这就是合格的可复现性。C 从第 100 步开始系统性偏离——这才是真 bug 的信号。判据是“偏离是否超出多种子的方差范围”，不是“数值是否相同”。

NaN / Inf 排查

NaN 一出现就会通过梯度污染整个模型，所以要尽早发现。

最便宜的检查点，开销可以忽略（每 step 一次同步，本来 `clip_grad_norm_` 之后往往就有）：

```
loss.backward()
total_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

if not torch.isfinite(total_norm):
    # 梯度炸了：跳过这一步，别让 NaN 进参数
    opt.zero_grad(set_to_none=True)
    n_skipped += 1
    continue
opt.step()
```

`clip_grad_norm_` 反正要算全局 norm，顺手检查它是否有限，是零额外成本的守卫。注意判断要放在 `opt.step()` 之前——NaN 一旦进了参数就再也回不来了。

NaN 最常来自这几个地方：

- `log(x)` 当 $x = 0$, `log_softmax` 之外手写的 softmax 忘了减 max 导致 exp 溢出
- 除零 : `normalize` 时 `x / x.norm()` 而 `x` 全零 ; `var` 为 0 时的标准化忘了 `eps`
- `sqrt(0)` 的反向 : $\frac{d}{dx}\sqrt{x} = \frac{1}{2\sqrt{x}}$ 在 0 点是 `inf` 。 `x.norm()` 、 `x.pow(0.5)` 在 0 点反向都会炸。写成 `torch.sqrt(x + 1e-12)` 或 `torch.clamp_min(x, 1e-12).sqrt()`
- attention mask 用 `-inf` 填充 , 而某一行全部被 mask 掉 $\rightarrow$ softmax 得到 $\frac{0}{0} \rightarrow \text{NaN}$ 。用 `-1e9` (`fp32`) 或 `torch.finfo(dtype).min` 代替 `-inf` , 或者保证每行至少一个位置可见
- `fp16` 溢出 : `fp16` 最大值 65504 , 激活或梯度超过就变 `inf` 。这是 `GradScaler` 存在的理由 ; `bf16` 指数范围和 `fp32` 一样 , 基本免疫这个问题——这也是现在训练一律用 `bf16` 而不是 `fp16` 的主因
- 学习率过大导致的正常发散 , 先炸 `grad norm` 再炸 `loss`

定位到具体 **op** 用 `anomaly detection` :

```
with torch.autograd.detect_anomaly():           # 只在调试时用
    loss = model(x).sum()
    loss.backward()
# 一旦某个 backward 产生 NaN , 抛异常并打印该 op 的 forward 调用栈
```

它的价值是把“backward 里出的 NaN”映射回 **forward** 里那一行代码 , 这个信息用别的办法很难拿到。代价是每个 op 都要存 forward 的 Python 栈并检查输出 , 慢好几倍 , 绝对不能留在生产脚本里。

Warning. 别在训练 loop 里逐层插 `if torch.isnan(t).any()` :。每一次都是一个 `.item()` 级别的隐式同步 (第 9 章) , 几十个检查点能让 step time 翻倍。正确做法是 : 平时只在 `loss` 和 `grad norm` 上各检查一次 (用得上现成的同步点) , 确认有问题之后再开 `detect_anomaly` 或用 `forward hook` 逐层抓一次。

面试考点

面试考点. **Q1** : `torch.manual_seed` 会设置 CUDA 的 generator 吗 ?

A : 会 , 它等价于同时设置 CPU generator 和所有 CUDA device 的 generator , 不需要额外调 `torch.cuda.manual_seed_all` 。但它管不到 Python 的 `random` 和 NumPy 的 `np.random` , 这两个要单独设。注意每个 CUDA device 有独立的 generator。

面试考点. **Q2** : 为什么固定了所有 seed , 两次训练的 loss 还是不逐 bit 相同 ?

A : seed 只固定随机数序列。差异来自 : 用 `atomicAdd` 做规约的非确定性 kernel (`index_add_` 、 `scatter_add_` 、 `embedding` 反向等) , 累加顺序由硬件调度决定 ; `cuda.benchmark` 可能选到不同算法 ; `TF32` 等低精度路径 ; CPU 多线程的分块规约 ; 分布式下 `NCCL` 的规约顺序随拓扑和 message size 变。根因是浮点加法不满足结合律 , 所以顺序不同则结果不同。

面试考点. **Q3** : 用一个例子说明浮点加法不满足结合律 , 以及它为什么和分布式训练有关。

A : `fp32` 下取 `a=1.0` 、 `b=6e-8` (略小于 `eps=1.19e-7`) : `(a+b)+b` 得 `1+2*eps` , `a+(b+b)` 得 `1+eps` , 因为舍入发生的次数和位置不同。AllReduce 要把 N 张卡的梯度加起来 , `ring` 和 `tree` 算法的加法顺序不同、message 分块大小不同 , 得到的和就差在最后几个 bit。梯度差几个 bit 经过几千步优化会放大成可见的 loss 差异——但这属于正常的数值噪声 , 不是 bug。

面试考点. **Q4** : `torch.use_deterministic_algorithms(True)` 做什么? 代价是什么?

A: 让有确定性实现的 op 切到确定性版本 (放弃 `atomicAdd` , 改用排序或分段规约 , 通常更慢) , 没有确定性实现的 op 直接抛 `RuntimeError` 。还需要配合 `torch.backends.cudnn.deterministic = True` 、 `benchmark = False` , 以及给 cuBLAS 设 `CUBLAS_WORKSPACE_CONFIG=:4096:8` (否则 `matmul` 会报错) 。 `warn_only=True` 是实用的中间档 : 不支持的 op 只警告 , 让你先知道哪些地方还不确定。

面试考点. **Q5** : TF32 是什么? 默认开不开?

A: A100 起 Tensor Core 支持的格式, 指数范围和 fp32 一样, 尾数只有 10 bit。当前 PyTorch 里 `torch.backends.cuda.matmul.allow_tf32` 默认是 **False** (1.12 改过默认值) , 而 `torch.backends.cudnn.allow_tf32` 默认是 **True**——`matmul` 不用、卷积用。要开 `matmul` 的 TF32 写 `torch.set_float32_matmul_precision("high")` 。它也是“同样代码在 V100 和 A100 上结果不同”的主要原因。

面试考点. **Q6** : 你怎么保证实验可复现?

A: 追求的是可复现的实验结论而不是逐 bit 相同。做法 : 固定 `torch.manual_seed` / `random.seed` / `np.random.seed` , 给 `DataLoader` 显式传 `generator` , 记录 commit hash、完整超参、依赖版本和硬件型号 ; 然后用多个 **seed** 跑同一配置来估计指标方差 , 只有当配置间差异大于种子间方差时才认为改动有效。只有调试和回归测试才需要打开完全确定性模式。

面试考点. **Q7** : 怎么排查 NaN?

A: 分两层。廉价守卫 : `clip_grad_norm` 的返回值本来就要算 , 顺手 `torch.isfinite(total_norm)` 检查 , 不通过就跳过这一步、别让 NaN 进参数 ; 再在 loss 上查一次。定位阶段 : `torch.autograd.detect_anomaly()` 能把 backward 里的 NaN 映射回 forward 的那行代码 , 但慢好几倍 , 只调试用。高发位置 : `log(0)` 、除零、`sqrt(0)` 的反向 (导数是 $1/(2*\sqrt{x})$) 、attention 某行被完全 mask 后 softmax 的 $\frac{0}{0}$ 、以及 fp16 溢出 (这就是要用 bf16 的原因) 。

面试考点. **Q8** : 怎么验证你手写的 kernel 和 `torch.nn` 的实现等价?

A: 用 `torch.testing.assert_close` , 它按 dtype 给默认容差、失败时报出最大差异的位置。关键是参考值要用更高精度算 : 验证 bf16 实现就拿 fp64 结果做参考 , 看相对误差是否在 bf16 的理论精度 (尾数 8 bit , 约 10^{-2} 相对) 之内 ; 两个低精度实现互比会互相掩盖误差。绝不能用 `==` , 也不要再在长规约上用 fp32 的默认容差——误差随规约长度累积。

性能分析与优化清单

面试里问“你会怎么优化训练速度”，最差的回答是背一串 API（“上 AMP、上 compile、上 flash attention”），最好的回答是描述一个定位流程：先测出瓶颈在哪一层，再针对那一层动手。这一章讲怎么正确测量（计时是白板高频题）、怎么读 profiler 的 timeline、怎么用 roofline 和 MFU 两个数值直觉判断“这个优化值不值得做”，最后给一份按性价比排序的清单。CUDA kernel 级别的细节（ncu 的各项指标、occupancy 调优）见仓库里的 CUDA 书 [books/cuda/](#)。

先测再改

优化的第一原则：不测量就是猜。而且要按层次测，每一层的结论决定下一层看哪里：

1. 端到端 **step time**。最重要的一个数，因为它是你实际关心的东西。同时记 tokens/s 或 samples/s，这才是跨配置可比的指标（batch 一变，step time 就没法比了）。
2. 拆成 **data / compute / comm** 三块。三个问题：dataloader 跟得上吗？GPU 在算的时候在干什么？分布式通信藏住了吗？这一层决定了你该去优化哪个子系统，绝大多数人跳过这一层直接去调 **kernel**，然后发现瓶颈根本不在那。
3. 单 **kernel**。到这一层才谈 fusion、occupancy、访存 pattern。

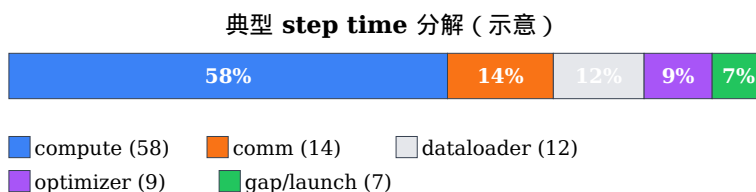


图 18 一个 step 的时间去哪了。这张图的数字是示意，你必须自己测——但形状是有代表性的：compute 是主体，而剩下 40% 里往前每一块都可能是你的瓶颈。先知道自己这张图长什么样，再决定优化谁。

怎么测出这张图？最省事的办法是三次消融，每次去掉一块，看 step time 掉多少：

```
# 1) 只跑 dataloader，不算：得到 data 的独立耗时
for x, y in loader:
    pass

# 2) 用一个固定的常量 batch 反复训练，绕开 dataloader：
#     step time 明显下降 → data 是瓶颈
x, y = next(iter(loader))
x, y = x.cuda(), y.cuda()
for _ in range(n):
    train_step(x, y)

# 3) 单卡跑同样的 micro-batch（不起 DDP）：
#     与多卡的 per-step 时间差就是没藏住的通信
```

消融比读 trace 快得多，也不容易看错。确认了瓶颈在哪一块之后，再用 profiler 进那一块看细节。

正确计时

第 9 章讲了为什么 `time.perf_counter()` 包一个 op 测出来的是假的 (launch 是异步的), 也给了手写的 CUDA event 模板。日常用不着手写——`torch.utils.benchmark.Timer` 自动处理 warmup、同步、自适应重复次数和离群值统计：

```
import torch
import torch.utils.benchmark as benchmark

x = torch.randn(4096, 4096, device="cuda", dtype=torch.bfloat16)

t = benchmark.Timer(
    stmt="x @ x",
    globals={"x": x},
    label="matmul",
    sub_label="4096^3 bf16",
    num_threads=1,
)
m = t.blocked_autorange(min_run_time=1.0)
print(m)           # 打印 median / IQR / 测量次数, IQR 太大会自动警告
print(m.median)    # 秒
```

它的输出里最该看的是 **IQR** (四分位距) 占 **median** 的比例。超过 10% 说明系统有波动 (别的进程在抢卡、时钟在降频、你自己代码里有 CPU 抖动), 这时候任何“优化了 5%”的结论都不可信, 先把环境弄干净。

测一整个训练 step (含 backward 和 optimizer) 时用同样的方法, 但要注意：

```
def step():
    opt.zero_grad(set_to_none=True)
    loss = model(x).sum()
    loss.backward()
    opt.step()

m = benchmark.Timer(stmt="step()", globals={"step":
step}).blocked_autorange(min_run_time=5.0)
```

Warning. 测 `torch.compile` 时 warmup 必须够长。第一次调用要跑 Dynamo trace + Inductor 编译 + Triton 编译, 可能是几十秒; 而且每种新的输入 **shape** 都会触发一次重编译。如果你的 warmup 只有 3 次, 测出来的“compile 更慢”全是编译时间。做法: warmup 至少跑到 step time 稳定 (打印每次的耗时看它什么时候收敛), 并且 warmup 和测量用同样的 **shape**。

`torch.profiler`

Timer 告诉你“多久”, profiler 告诉你“时间花在哪个 op、哪个 kernel 上”。完整用法：

```
import torch
from torch.profiler import profile, ProfilerActivity, schedule,
tensorboard_trace_handler

prof_schedule = schedule(
```

```

skip_first=3,      # 前 3 步完全不管 (还在 warmup)
wait=1,            # 空转 1 步
warmup=1,          # profiler 自己预热 1 步 (丢弃这步的数据)
active=3,          # 真正采集 3 步
repeat=1,
)

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    schedule=prof_schedule,
    on_trace_ready=tensorboard_trace_handler("./tb_log"),
    record_shapes=True,      # 记录输入 shape, 能看出是哪个 shape 慢
    profile_memory=True,     # 记录每个 op 的显存增减
    with_stack=True,        # 记录 Python 调用栈 (开销不小)
    with_flops=True,        # 对 matmul/conv 估算 FLOPs
) as prof:
    for step in range(12):
        train_one_step()
        prof.step()         # 必须调, 否则 schedule 不会推进

```

`schedule` 那几个参数是必须理解的: profiler 本身有开销, 全程开着会把 timeline 拖变形, 所以只在稳态的几个 step 上采集。 `prof.step()` 忘了写是最常见的错误——schedule 不推进, 最后什么都没采到。

看结果有两条路。快速看排行榜:

```

print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=20))

# 按 shape 分组, 定位"是哪个 shape 慢"
print(prof.key_averages(group_by_input_shape=True).table(
    sort_by="self_cuda_time_total", row_limit=20))

```

`cuda_time_total` 含子调用, `self_cuda_time_total` 只算这个 op 自己。找热点用 `self_*` (否则最上面永远是 `nn.Module.__call__` 这种包壳), 看“某个模块整体占多少”用非 `self` 的。

Note. 较新版本里这些列名从 `cuda_*` 逐步改成了 `device_*` (为了兼容非 NVIDIA 后端), `sort_by="self_device_time_total"` 和 `"self_cuda_time_total"` 目前都能用。看到 deprecation warning 就换成 `device_*` 的写法。

另一条路是看 timeline, 信息量大得多:

```

prof.export_chrome_trace("trace.json")

```

用 `ui.perfetto.dev` 打开 (比老的 `chrome://tracing` 好用很多, 支持 SQL 查询)。如果用了 `tensorboard_trace_handler`, 直接 `tensorboard --logdir ./tb_log` 看 PyTorch Profiler 插件。

给自己的代码段打标签, 让 timeline 可读:

```

from torch.profiler import record_function

```

```

with record_function("## data ##"):
    x, y = next(loader_it)
with record_function("## forward ##"):
    out = model(x)
with record_function("## backward ##"):
    loss.backward()

```

怎么读 timeline

打开 trace 之后按这个顺序看：

1. 先看 **GPU** 那几行有没有空隙 (**gap**)。这是最重要的一眼。GPU 行密不透风 → GPU-bound，去看单 kernel；有明显空白 → 先搞清楚空白期在等什么。
2. 空白期看 **CPU** 行在干什么。CPU 忙着发 launch → CPU-bound (Python 开销 / kernel 太碎)，对策是 `torch.compile`、CUDA graph、fused optimizer。CPU 也闲 → 在等外部：等 dataloader (能看到 `## data ##` 那段变长) 或等通信。
3. 找 `ncclAllReduce` 之类的通信 **kernel** 在哪一行。如果它和 compute kernel 在不同 **stream** 且时间上重叠，overlap 生效了；如果 compute 那行在通信期间是空的，overlap 没生效(第 18 章)。
4. 找占时最长的几个 **kernel**。看名字就能判断类别：`*_gemm_*` / `cutlass*` 是 matmul，`elementwise_kernel` / `vectorized_elementwise` 是 elementwise，`reduce_kernel` 是规约。如果 elementwise 的总时间和 GEMM 一个量级，说明有大量没融合的小 op，`torch.compile` 会有明显收益。
5. 看 **kernel** 之间有没有 **Memcpy** / **Memset**。Memcpy DtoH 出现在训练 loop 里几乎总是 bug(某处在 `.item()` 或 `.cpu()`)。Memcpy DtoD 多说明有大量 `.contiguous()` 或不必要的 copy。

`nsys` 和 `ncu` 的分工一句话：`nsys` 看时间线上的宏观分布 (哪个 kernel、多少 gap、通信有没有重叠)，能力和 `torch.profiler` 的 trace 重叠但更底层、能看到驱动和 NCCL 内部；`ncu` 看单个 **kernel** 的微观指标 (achieved occupancy、DRAM 吞吐、是 memory-bound 还是 compute-bound)，一次只看一个 kernel，开销极大。判断“该不该 fuse 这个 op”用 `ncu`，判断“该优化哪个 op”用 `nsys` 或 `torch.profiler`。具体命令和指标解读见 `books/cuda/`。

roofline 直觉：这个 op 值不值得 fuse

一个 kernel 的算术强度 (arithmetic intensity) 定义为它做的浮点运算数除以它搬运的字节数：

$$I = \frac{\text{FLOPs}}{\text{bytes moved}} \quad [\text{单位: FLOP/byte}]$$

拿它和硬件的“分界强度” $I^* = \frac{P_{\text{peak}}}{B}$ 比：小于 I^* 就是 memory-bound (HBM 带宽是瓶颈，SM 在等数据)，大于就是 compute-bound。

A100-SXM4-80GB：bf16 稠密峰值约 312 TFLOPS，HBM 带宽约 2.0 TB/s。

$$I_{\text{A100,bf16}}^* = \frac{312 \times 10^{12}}{2.0 \times 10^{12}} \approx 156 \text{ FLOP/byte}$$

156 是个很高的门槛，所以绝大多数 **op** 都是 **memory-bound**。代几个例子看：

op	I 量级	结论
----	------	----

$y = x + 1$ (bf16)	0.25	读 2 B 写 2 B 做 1 FLOP, 极度 memory-bound
LayerNorm	$O(1)$	几次遍历、每元素十几个 FLOP, memory-bound
Softmax	$O(1)$	同上, 而且要多遍 (max、sum、除)
GEMM $n \times n \times n$ (bf16)	$n/3$	$n = 4096$ 时约 1365, compute-bound
GEMM, $n = 468$	156	恰好在分界线上

GEMM 那一行值得单独说: bf16 方阵乘法读写 3 个 n^2 矩阵共 $6n^2$ 字节、做 $2n^3$ 次运算, 所以 $I = 2n^3/(6n^2) = n/3$ 。解 $n/3 = 156$ 得 $n \approx 470$ 。也就是说 **A100** 上小于约 **470** 的方阵 **GEMM** 也是 **memory-bound**——这解释了为什么小 batch、小 hidden size 的模型再怎么优化 kernel 也吃不满算力。

Key insight. 这个直觉的实际用法: 一串连续的 elementwise / norm op (比如 $x = \text{norm}(x)$; $x = x * w$; $x = \text{gelu}(x)$) 每个都要把整个张量从 HBM 读一遍写一遍。融合成一个 kernel 之后, 中间结果留在寄存器和 shared memory 里, 省掉的是 $k-1$ 次 **HBM** 往返。所以 fusion 的收益上界就是“省掉的 HBM 流量 $\div$ 带宽”, 可以直接估出来。反过来, 两个大 GEMM 之间融合没什么意义——它们本来就 compute-bound。这就是 torch.compile (第 14 章) 主要在优化 memory-bound 部分的原因。

MFU: 一个数说清训练效率

MFU (Model FLOPs Utilization) = 模型有效计算量 $\div$ 硬件峰值。分子用著名的 $6N$ 近似: 训练一个 token, forward 约 $2N$ 次乘加 (每个参数一次乘一次加), backward 约 $4N$ (对输入和对权重各一遍), N 是非 embedding 参数量。

$$\text{MFU} = \frac{6N \cdot T}{P_{\text{peak}}}, \quad T = \text{每卡每秒处理的 token 数}$$

长序列时还要补上 attention 的部分, 它不含参数但要算: 每 token 每层约 $12sh$ 次运算 (s 是 seq len, h 是 hidden), 所以完整式子是 $6N + 12Lsh$ 。当 s 和 h 同量级时这一项不可忽略。

算一个例子 (假设测到每卡 3000 tokens/s, 7B 模型, A100 bf16):

$$\text{MFU} = \frac{6 \times 7 \times 10^9 \times 3000}{312 \times 10^{12}} = \frac{1.26 \times 10^{14}}{3.12 \times 10^{14}} \approx 40\%$$

公开的大模型训练报告里, A100 上调好的稠密 Transformer 通常落在 35%–50% 这个区间; 低于 30% 说明有明显的可优化空间, 高于 55% 在稠密训练上很少见。

Note. 面试被问“你的训练 MFU 多少”时, 答案里必须包含怎么算的: 分子用 $6N$ 还是 $6N + 12Lsh$ 、 N 含不含 embedding、分母用稠密峰值还是稀疏峰值 (NVIDIA 标的稀疏峰值是稠密的 2 倍, 用它会令 MFU 看起来腰斩)。还有 MFU 和 HFU 的区别: HFU (Hardware FLOPs Utilization) 把 activation checkpointing 重算的那部分也算进分子, 所以开了 checkpoint 之后 HFU 会比 MFU 高一截。能主动说清这些, 比报一个漂亮数字更有说服力。

优化清单, 按性价比排序

版本	核心思路	相对成本 / 收益
----	------	-----------

AMP / bf16	激活和参数的读写都减半，且走 Tensor Core，峰值算力高一个档	几乎必做
<code>torch.compile</code>	把连续的 memory-bound op 融合，省 HBM 往返和 launch 次数	一行代码，见第 15 章
SDPA / Flash	不 materialize $s \times s$ 的 score 矩阵，省显存也省 HBM 往返	长序列最大单项
fused optimizer	<code>AdamW(..., fused=True)</code> ，把 per-param 的 Python 循环压成少量 kernel	CPU-bound 时明显
<code>set_to_none</code>	grad 不填 0 而是直接释放	省显存，少一批 memset
<code>channels_last</code>	NHWC 布局才能让 cudnn 走 Tensor Core 路径	仅 CNN，LLM 无关
加大 batch	小 batch 下 GEMM 是 memory-bound，Tensor Core 吃不满	受显存限制
dataloader 调优	<code>num_workers</code> + <code>pin_memory</code> + <code>persistent_workers</code>	确认是瓶颈再做
activation ckpt	用算力换显存，换来的显存拿去加大 batch	算力 +30%，见第 8 章
CUDA graph	消除 CPU 侧的 launch 开销	仅 CPU-bound，见第 9 章
通信 overlap	让 AllReduce 藏进 backward 的计算里	分布式，见第 18 章

前三条基本无脑上，中间几条要先确认瓶颈在那一层，后面几条都有明确的前置条件——用错了不但不会快，还会因为多出来的转换开销变慢。

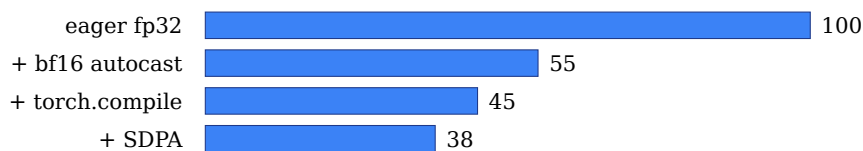


图 19 示意图，非实测数据。数值是归一化的 step time (`eager fp32` = 100)，只表达“逐项叠加、收益递减”的形状。真实数字强烈依赖模型结构、shape 和 GPU 型号——同一套优化在小模型上可能只有几个百分点（因为它本来就 CPU-bound），在长序列模型上 SDPA 一项可能就占大半收益。每一项都要自己测。

“优化了但没变快”

Warning. 排查顺序：

1. 瓶颈不在那里。你优化了 GEMM，但 40% 的时间在等 dataloader。回到第一节，先做那张 step time 分解图。
2. 被 CPU 掩盖了。GPU kernel 快了一倍，但 CPU 发 launch 的速度没变，step time 一点不动——因为原本就是 CPU-bound。判据：`torch.profiler` 里 GPU 总忙时远小于 wall-clock。
3. `torch.compile` 的编译时间没排除。warmup 不够，或者输入 shape 每个 batch 都在变（变长 seq）导致反复重编译。看 `torch._dynamo.utils.compile_times()` 或日志里的 recompile 计数。
4. **batch** 太小。 $n < 470$ 的 GEMM 在 A100 bf16 上是 memory-bound（见 roofline 那节），Tensor Core 的峰值根本用不上。这时候正确的优化是加大 batch，不是调 kernel。

5. 测量噪声大于收益。Timer 报的 IQR 超过 median 的 10% 时,“提升 5%”是噪声。把别的进程停掉、锁定时钟、多测几轮。
6. 优化被别的地方吃掉了。典型是加了 `channels_last` 之后模型里某个 op 不支持这个布局,于是每次都要转回 contiguous,转换的开销抵消了收益。在 trace 里找突然多出来的 Memcpy DtoD 或 contiguous kernel。

面试考点

面试考点. Q1: 你会怎么定位一个训练任务的性能瓶颈?

A: 分三层。先量端到端 step time 和 tokens/s;再用 `torch.profiler` 加 `record_function` 把 step 拆成 data / forward / backward / optimizer / comm,看哪块占比异常;最后才看单个 kernel。看 trace 的第一眼是 GPU 行有没有空隙:没空隙 → GPU-bound,去看 kernel;有空隙且 CPU 忙 → CPU-bound (launch 开销、Python 开销);有空隙且 CPU 也闲 → 在等 data 或等通信。

面试考点. Q2: 为什么用 `time.time()` 测 GPU 算子时间是错的?正确怎么做?

A: kernel launch 是异步的,`time.time()` 测到的是 launch 开销(微秒级),不是 kernel 执行时间。正确做法: warmup 若干次 → `torch.cuda.synchronize()` → 用 `torch.cuda.Event(enable_timing=True)` 打 start/end → 多次取中位数。生产上直接用 `torch.utils.benchmark.Timer.blocked_autorange()`,它自动处理这三件事,还会报 IQR 让你知道测量不可信。

面试考点. Q3: 算术强度是什么?怎么用它判断一个 op 该不该优化?

A: 算术强度 $I = \text{FLOPs} / \text{搬运字节数}$ 。和硬件的分界强度 $I^* = \text{峰值算力} / \text{带宽}$ 比较: A100 bf16 是 $312 \text{ TFLOPS} / 2.0 \text{ TB/s} \approx 156 \text{ FLOP/byte}$ 。低于它是 memory-bound。elementwise ($I \approx 0.25$) LayerNorm、softmax 都远低于 156,优化方向是减少 HBM 往返(融合);大 GEMM 的 $I \approx n/3$, $n = 4096$ 时约 1365,是 compute-bound,优化方向是提高 Tensor Core 利用率。

面试考点. Q4: kernel fusion 的收益上限怎么估?

A: k 个连续的 memory-bound op,每个都要把张量从 HBM 读一遍写一遍。融合后中间结果留在寄存器/shared memory,省掉 $k-1$ 次往返。收益上界 = 省掉的字节数 ÷ HBM 带宽,可以直接算出来。所以 `torch.compile` 对“一长串 elementwise + norm”收益大,对“两个大 GEMM”基本没收益——后者本来就 compute-bound。

面试考点. Q5: MFU 怎么算?你的训练 MFU 多少?

A: $\text{MFU} = 6NT/P_{\text{peak}}$, N 是非 embedding 参数量、 T 是每卡每秒 token 数、 P_{peak} 用稠密峰值 (A100 bf16 是 312 TFLOPS), $6N$ 来自 forward $2N$ + backward $4N$;长序列要补 attention 的 $12Lsh$ 。回答时必须交代清楚分子用了哪个式子、 N 含不含 embedding、分母用稠密还是稀疏峰值。公开报告里 A100 上调好的稠密 Transformer 常在 35%-50%。另外要能区分 MFU 和 HFU: 后者把 activation checkpointing 的重算算进分子。

面试考点. **Q6** : `torch.profiler` 的 `schedule` 为什么要设 `wait / warmup / active` ?

A : `profiler` 自身有开销 (尤其 `with_stack=True`), 全程开着会把 `timeline` 拖变形, 而且产生的 `trace` 大到打不开。所以只在稳态的少数几个 `step` 上采集 : `skip_first` 跳过 `warmup` 阶段, `warmup` 让 `profiler` 自己预热 (这几步数据丢弃), `active` 才是真正采集。同时必须在每个 `step` 末尾调 `prof.step()` 推进 `schedule`——忘了写是最常见的错误, 结果是什么都没采到。

面试考点. **Q7** : `nsys` 和 `ncu` 分别什么时候用 ?

A : `nsys` 是时间线级别的, 看整体分布——哪些 `kernel`、`gap` 在哪、通信有没有和计算重叠、多卡之间有没有 `straggler`。用它回答“该优化哪个 `op`”。`ncu` 是单 `kernel` 级别的, 给 `achieved occupancy`、`DRAM` 吞吐、`L2` 命中率这些微观指标, 用它回答“这个 `kernel` 离硬件上限还差多远、瓶颈是访存还是算力”。`ncu` 会 `replay kernel`, 开销极大, 只对少数几个热点 `kernel` 用。

面试考点. **Q8** : 优化之后 `step time` 没变, 可能是什么原因 ?

A : 最常见是瓶颈不在你优化的地方——比如 `GPU kernel` 快了但整体是 `CPU-bound` 或在等 `dataloader`, 判据是 `profiler` 里 `GPU` 忙时远小于 `wall-clock`。其次 : `torch.compile` 的编译时间没从测量里排除, 或输入 `shape` 在变导致反复重编译 ; `batch` 太小以致 `GEMM` 本身就 `memory-bound` ; 测量噪声大于收益 (`Timer` 的 `IQR` 超过 `median` 的 10%); 以及优化引入了新开销 (比如 `channels_last` 遇到不支持的 `op`, 每次都要转回 `contiguous`)。

TorchDynamo : 从 Python 字节码到 FX Graph

面试里问 `torch.compile` 通常只有两个方向：它是怎么把 **Python** 变成 **kernel** 的，和它为什么在我的模型上不快。两个问题的答案都从 Dynamo 开始——它是唯一直接面对你写的 Python 代码的那一层，也是 graph break 和 recompile 这两个最常见性能杀手的发源地。这一章讲 Dynamo 的机制、guard 与 graph break 的成因与观测方法；AOTAutograd 见第 13 章，Inductor 见第 14 章。本章所有输出都在 torch 2.10.0+cu128 / A100-SXM4-80GB 上实跑得到，做了裁剪但没有改写。

四层流水线

`torch.compile(model)` 不是一个编译器，是四层组件串起来的一条流水线。



图 20 `torch.compile` 的四层：Dynamo 抓图，AOTAutograd 补反向，Inductor 生成 kernel。

层	输入 → 输出	核心职责
TorchDynamo	CPython frame(字节码) → FX Graph + guards + 残余字节码	符号执行字节码，抓出 tensor 计算，其余留在 Python
AOTAutograd	FX Graph (只有前向) → 前向图 + 反向图	提前 trace 出反向、functionalize、decompose、切分
PrimTorch	2000+ 个 ATen op → 一小组 prim op	收窄编译器要支持的 op 集合
TorchInductor	切分后的 FX Graph → Triton (GPU) / C++ OpenMP (CPU)	lowering、fusion、autotune、codegen、缓存

只有 Dynamo 这一层是“必须理解 Python 语义”的。后面三层拿到的都已经是纯粹的张量计算图，不再有 Python 对象、控制流和副作用。这个分工决定了后面所有的坑都长什么样。

Dynamo 怎么接管 Python 执行

CPython 3.6 引入的 PEP 523 允许把解释器求值一个 code object 的入口函数替换掉。Dynamo 就是注册了这样一个自定义 frame evaluation 回调：每当 Python 要执行一个被 `torch.compile` 包住的函数的 frame，控制权先交给 Dynamo。

Dynamo 拿到 frame 后做符号执行 (symbolic evaluation)：逐条解释字节码，但栈上放的不是真实值，而是 `VariableTracker` 这类符号对象。

- 遇到 `TensorVariable` 上的 `torch` 操作 → 记一个节点到 FX Graph，返回一个新的 `TensorVariable` (带 fake 的 shape/dtype/device 元信息，不算真实数据)
- 遇到纯 Python 逻辑 (for 展开、常量折叠、属性访问、`nn.Module` 的 `__call__` 派发) → 直接在符号执行里消化掉，不进图。这就是为什么编译后的图里看不到 `for layer in self.layers` 这个循环，只看到被展开的一串 op。
- 遇到看不懂的东西 → graph break (下面细讲)

符号执行结束后，Dynamo 产出三样东西：一张 FX Graph、一组 guard、一段重写过的字节码。重写后的字节码把原来那段计算替换成“调用编译好的 callable”，其余部分照原样保留。

```
import torch

def f(x, w):
    return (x @ w).relu().sum()

gm = []
torch.compile(f, backend=lambda g, ex: (gm.append(g), g.forward)[1])(
    torch.randn(8, 8), torch.randn(8, 8))
print(gm[0].code)
```

实跑输出（Dynamo 抓到的图，注意这时还是 torch 层的 method call，不是 ATen）：

```
def forward(self, L_x_ : torch.Tensor, L_w_ : torch.Tensor):
    l_x_ = L_x_
    l_w_ = L_w_
    matmul = l_x_ @ l_w_; l_x_ = l_w_ = None
    relu = matmul.relu(); matmul = None
    sum_1 = relu.sum(); relu = None
    return (sum_1,)
```

Note. 自定义 backend 就是一个 (gm, example_inputs) -> callable 的函数。这是最省事的“我到底抓到了什么图”调试手段，也是 backend="eager" / "aot_eager" 这些调试后端的本质：把图原样跑掉，不做 codegen，用来判断问题出在 Dynamo 还是 Inductor。

guard：编译产物是特化的

Dynamo 抓的图是在一组具体假设下才正确的。这些假设被记录成 guard，挂在编译产物上；每次调用先跑一遍 guard 检查，全通过才复用编译结果。

guard 覆盖的东西比大多数人以为的多：

guard 类型	记录了什么
TENSOR_MATCH	输入 tensor 的 dtype / device / 维数 / 各维 size / requires_grad / layout
EQUALS_MATCH, CONSTANT_MATCH	被当成常量参与 trace 的 Python 标量的具体值
TYPE_MATCH, ID_MATCH	Python 对象的类型，或某个 nn.Module 实例的 id
GRAD_MODE	当前是否在 no_grad / inference_mode 下
DEFAULT_DEVICE, DETERMINISTIC_ALGORITHMS	全局状态
SHAPE_ENV	动态 shape 的符号约束（s0 >= 2 这类）

torch._dynamo.explain(f)(args) 会把 guard 全部打印出来。下面是实跑里一条 guard 的原样输出：

```
Guard 29:
  Name: "L['b']"
  Source: local
  Create Function: TENSOR_MATCH
  Guard Types: ['TENSOR_MATCH']
  Code List: ["hasattr(L['b'], '_dynamo_dynamic_indices') == False"]
```

guard 失败就 recompile。用 `TORCH_LOGS="recompiles"` 能直接看到原因，实跑输出：

```
[0/1] [__recompiles] Recompiling function f in /tmp/v3_logs.py:3
[0/1] [__recompiles]      triggered by the following guard failure(s):
[0/1] [__recompiles]      - 0/0: tensor 'x' size mismatch at index 0. expected 4,
actual 5
```

[0/1] 这个编号要会读：斜杠前是 **frame** 序号（同一个函数第几段图），斜杠后是 该段图的第几份编译产物。看到 [0/7] 就说明第 0 段图已经编了 8 份。

recompile 上限与静默 fallback

同一段代码的编译产物数量有上限。torch 2.6 起这个配置的正式名字是 `torch._dynamo.config.recompile_limit`（旧名 `cache_size_limit` 仍是别名），默认 8；另有 `accumulated_recompile_limit`，默认 256，管一个进程内的总量。

超限后 Dynamo 不报错，只打一条 warning，然后把这个函数标记成“以后别再编了”，永久 fallback 到 eager：

```
W convert_frame.py:1676] [0/8] torch._dynamo hit config.recompile_limit (8)
W convert_frame.py:1676] [0/8]      function: 'f' (/tmp/v4_cachelimit.py:3)
W convert_frame.py:1676] [0/8]      last reason: 0/7: tensor 'x' size mismatch
at index 0. expected 11, actual 12
W convert_frame.py:1676] [0/8] To log all recompilation reasons, use
TORCH_LOGS="recompiles"
```

Warning. 这是生产里最阴的一类“compile 没效果”：日志里只有一行 warning，训练照样跑，速度悄悄退回 eager。上线前务必确认 stderr 里没有 hit `config.recompile_limit`。正确修法不是把 limit 调大（那只是把编译时间浪费掉），而是找到 recompile 的源头——通常是变长 shape，见第 15 章的动态 shape 一节。

Key insight. guard 是 `torch.compile` 能“既激进特化又保证正确”的全部原因。TorchScript 没有 guard，所以它必须在编译期把所有情况都表达出来（于是要求你写受限的静态类型代码）；Dynamo 选择“先假设，再运行时校验”，代价是 recompile，收益是你完全不用改代码。

graph break：图被切开

Dynamo 符号执行时遇到无法表达成图的东西，就把图切断：把已经攒到的 op 编译成一段图，然后回到 Python 逐字节码执行那段不认识的代码，再从下一条指令开始新的一段图。结果是 [编译段] → [Python 段] → [编译段] 交替。

常见成因，按遇到频率排：

成因	为什么
<code>print</code> / <code>logging</code> 调用	有副作用且返回值不是张量，无法进图
<code>.item()</code> / <code>.tolist()</code> / <code>float(t)</code>	要求真实数据；trace 时只有 fake tensor，没有值
<code>.cpu()</code> / <code>.numpy()</code>	同上，跨设备取值意味着同步
依赖张量值的 <code>if</code> / <code>while</code>	分支走哪边取决于运行时数据，图里表达不了
不支持的第三方库调用（ <code>numpy</code> 之外的 C 扩展等）	Dynamo 没有对应的 <code>VariableTracker</code>
<code>try</code> / <code>except</code> 里抛出并被捕获的路径	异常控制流的支持有限
部分生成器 / 闭包 / <code>**kwargs</code> 的动态用法	符号执行无法静态解析
数据相关的 <code>shape</code> （ <code>x[mask]</code> 、 <code>nonzero</code> ）	输出 <code>shape</code> 依赖数据

Note. `.item()` 与数据相关 `shape` 这两类不是绝对不行：`torch._dynamo.config.capture_scalar_outputs = True` 和 `capture_dynamic_output_shape_ops = True` 可以让它们进图，代价是引入 `unbacked symbol`，下游要么加 `torch._check` 约束，要么在 Inductor 里退化。默认关掉是因为它们经常把编译搞得更糟。

三种观测手段

一、`torch._dynamo.explain` —— 一次拿到图数量、break 数量、每段的 op、以及每个 break 的原因。它返回 `ExplainOutput` 对象，字段是 `graph_count` / `graph_break_count` / `op_count` / `break_reasons` / `ops_per_graph` / `out_guards` / `graphs` / `compile_times`。

```
import torch
import torch._dynamo as dynamo

def f(x, y):
    a = torch.sin(x) + y
    print("hello")           # graph break 1
    b = a * 2
    if b.sum().item() > 0:    # graph break 2
        b = b + 1
    return b.relu()

exp = dynamo.explain(f)(torch.randn(8, 8), torch.randn(8, 8))
print(exp.graph_count, exp.graph_break_count, exp.op_count)
```

实跑输出（裁掉了 guard 部分，那是几十行）：

```
Graph Count: 3
Graph Break Count: 2
Op Count: 4
Break Reasons:
  Break Reason 1:
    Reason: Failed to trace builtin operator
    Explanation: Dynamo does not know how to trace builtin operator `print` with
                 argument types ['str'] (has_kwargs False)
```

```

Hint: Avoid calling builtin `print` with argument types ['str']. ...
Hint: If you are attempting to call a logging function (e.g. `print`), you can
      try adding it to torch._dynamo.config.reorderable_logging_functions.
User Stack:
  <FrameSummary file /tmp/v1_explain.py, line 7 in f>
Break Reason 2:
  Reason: Unsupported Tensor.item() call with capture_scalar_outputs=False
  Explanation: Dynamo does not support tracing `Tensor.item()` with
               config.capture_scalar_outputs=False.
Ops per Graph:
  Ops 1:
    <built-in method sin of type object at 0x747b04929b40>
    <built-in function add>
  Ops 2:
    <built-in function mul>
  Ops 3:
    <built-in function add>

```

两个 break 把代码切成 3 段图，一共只有 4 个 op——每段图都小到 Inductor 几乎没得优化。

二、**TORCH_LOGS="graph_breaks"** —— 训练跑起来之后被动观测，不用改代码。它按发生顺序打，还带 user code traceback，能直接定位到行。和 **recompiles** 一起开最有用：

```
TORCH_LOGS="graph_breaks,recompiles" python train.py
```

三、**fullgraph=True** —— 开发期首选。它把第一个 graph break 变成异常抛出，逼你当场处理：

```
torch.compile(f, fullgraph=True)(torch.randn(4))
```

实跑得到的异常（类型是 `torch._dynamo.exc.Unsupported`）：

```

Failed to trace builtin operator
  Explanation: Dynamo does not know how to trace builtin operator `print` with
               argument types ['str'] (has_kwargs False)
...
from user code:
  File "/tmp/v2_fullgraph.py", line 5, in f
    print("hi")

Set TORCHDYNAMO_VERBOSE=1 for the internal stack trace ...

```

Key insight. 工作流应该是：开发时 **fullgraph=True** 把图打通，确认能整图编译；上生产再决定要不要放开成 **fullgraph=False**（换取对偶发不可 trace 代码的容忍）。反过来先上 **fullgraph=False** 的人，通常到最后都不知道自己的模型被切成了几十段。

graph break 的代价

1. **fusion** 机会消失。Inductor 只能在一段图内融合。切开之后每段图边界上的中间张量必须落到 HBM，第 14 章会算这笔账。
2. 每段之间要回 **Python**。除了字节码执行本身的开销，进出编译产物还要跑一遍 guard 检查。

3. **CUDA graph** 用不了。 `mode="reduce-overhead"` 需要一段连续的、地址固定的 kernel 序列；中间插了 Python 就断了。
4. 编译产物数量翻倍。每段图独立编译、独立 guard、独立计入 `recompile_limit`。上面那个例子里 `torch_dynamo_resume_in_f_at_6` 就是 break 之后那段图的名字，它有自己的 `[1/0]`、`[1/1]` 编号。

不是所有 break 都要消灭。在 `forward` 之外、每个 step 只发生一次的 break（比如 dataloader 逻辑、metric 打印）代价可以忽略。要消灭的是在热点 **forward** 内部、每层都发生一次的那种。

手动控制编译边界

API	用途
<code>@torch.compiler.disable</code>	标记“这个函数别 trace”。Dynamo 直接在此 break 并原样调用它，比让它 trace 失败干净
<code>torch.compiler.allow_in_graph(fn)</code>	反过来：让 Dynamo 不要看函数内部，把它当成一个不透明的 op 塞进图。前提是它输入输出都是张量且无副作用
<code>torch._dynamo.mark_dynamic(t, dim)</code>	声明 <code>t</code> 的第 <code>dim</code> 维是动态的，别为它生成 size guard。见第 15 章
<code>torch.compiler.nested_compile_region</code>	标记一个可复用的子区域，让重复结构（比如 N 个同构 layer）少编几次
<code>torch.compiler.set_stance("...")</code>	运行时切换编译策略，比如 <code>"force_eager"</code> 、 <code>"eager_on_recompile"</code> 。用来快速做 A/B 对比

实测 `@torch.compiler.disable` 的效果：把带 `print` 和 `.item()` 的日志函数标上之后，`explain` 报 `graph_count 2, breaks 1`——一次干净的 break，而不是函数内部每个不支持的操作各来一次。

`allow_in_graph` 的效果同样可验证，被标记的函数在图里变成一个单节点：

```
def forward(self, L_x_ : torch.Tensor):
    L_x_ = L_x_
    sin = L_x_.sin(); L_x_ = None
    custom = __main__custom(sin); sin = None
    return (custom,)
```

Warning. `allow_in_graph` 是“我保证”级别的 API。如果被标记的函数里有副作用、有 inplace 修改、或者对同样的输入返回不同结果，编译后的行为会静默地和 `eager` 不一致——因为 AOTAutograd 之后的每一层都相信这个节点是纯函数。自定义 CUDA op 的正确做法是注册成 `custom op ( torch.library )`，不是 `allow_in_graph`。

为什么 **Dynamo** 成了，**TorchScript** 没成

这是高频对比题。答案不是“Dynamo 更快”，而是采纳成本。

版本	核心思路	相对成本 / 收益
<code>torch.jit.trace</code>	跑一遍记录 op 序列	控制流被固化成一条路径，静默出错
<code>torch.jit.script</code>	解析源码到 TorchScript 子集	要求你改代码，第三方库全部不可用

<code>torch.compile</code>	字节码符号执行 + guard + graph break	不改代码，能 fallback，可能 recompile
----------------------------	-------------------------------	------------------------------

三条关键差别：

1. 不要求改代码。`jit.script` 只支持 Python 的一个静态类型子集：不能用 `**kwargs`、不能用大多数第三方库、`Optional` 要显式标注、类要写 `@torch.jit.export`。一个真实模型迁过去经常要重写几百行。Dynamo 直接吃你现在的代码。
2. 能优雅降级。`jit.script` 遇到不支持的语法就是编译失败，你要么改代码要么放弃。Dynamo 遇到就 graph break：不支持的部分回 Python 跑，其余照样编译。这让“部分收益”成为可能，而部分收益是能推广的前提。
3. 不需要静态类型推断。Dynamo 在符号执行时手上有真实的第一批输入，`shape`、`dtype`、`device` 全都是已知的具体值，特化后由 guard 兜住。`jit.script` 必须在没有输入的情况下静态推断类型，这在 Python 里是打不赢的仗。

`torch.jit.trace` 的坑另说：它记录的是一次执行的 op 序列，条件分支和循环次数被固化，换一条路径的输入结果就是错的，而且不报错。第 16 章给了实测复现。

Note. TorchScript 目前处于维护模式（不再加新特性）。面试里问到“你们模型怎么部署”，答 `jit.script` 会被追问为什么不用 `torch.export`；反过来能说清“trace 丢控制流、script 采纳成本高、export 是现在的正确答案”就是加分项。

面试考点

面试考点. Q1：`torch.compile` 内部有哪几层？各层负责什么？

A：四层。Dynamo 用 PEP 523 接管 CPython frame，符号执行字节码，产出 FX Graph + guard + 重写的字节码；AOTAutograd 提前 trace 出反向，做 functionalization 和 decomposition，切成前向图和反向图；PrimTorch 把 2000+ ATen op 收窄成一小组 prim op；Inductor 把图 lowering 成 loop-level IR，做 fusion 和 codegen，GPU 出 Triton、CPU 出 C++/OpenMP。

面试考点. Q2：什么是 guard？为什么必须有？

A：guard 是编译时假设的运行时代验条件，覆盖输入 tensor 的 `dtype/device/维数/各维 size/requires_grad`、被折成常量的 Python 值、`nn.Module` 的 `id`、`grad_mode` 等全局状态。因为 Dynamo 抓的图是对第一批输入特化过的，guard 是“特化”与“正确”之间的唯一保险。guard 检查失败就 recompile。

面试考点. Q3：什么是 graph break？列 4 个常见成因。

A：Dynamo 遇到无法表达成图的代码，就把图切断成“编译段 - Python 段 - 编译段”。常见成因：`print/logging` 这类有副作用的调用；`.item()/ .tolist()/ .cpu()` 这类需要真实数据的操作；依赖张量值的 `if/while`；Dynamo 不认识的第三方库调用。数据相关 `shape(x[mask])`、`nonzero`) 和部分 `try/except` 也会。

面试考点. Q4：graph break 有什么代价？为什么不能只看“反正还能跑”？

A：四条。fusion 只能发生在一段图内，切开后段边界的中间张量必须写回 HBM；每段之间要回 Python 执行字节码并重跑 guard；CUDA graph (mode="reduce-overhead") 需要连续 kernel 序列，插了 Python 就用不了；每段图独立编译独立 guard，各自计入 recompile_limit。极端情况下切成几十段，编译时间全付了、收益接近零。

面试考点. Q5：怎么定位 graph break？

A：开发期用 fullgraph=True，第一个 break 直接抛 Unsupported 异常并指到源码行；要看全部 break 用 torch._dynamo.explain(fn)(args)，它给 graph_count / graph_break_count / break_reasons 和每段的 op；线上被动观测用 TORCH_LOGS="graph_breaks,recompiles"，带 user traceback 且不用改代码。

面试考点. Q6：模型每个 step 的 shape 都不一样，会发生什么？

A：每种新 shape 触发一次 guard 失败 → recompile。Dynamo 有 automatic dynamic shapes：同一维度出现第二个值时会自动把它符号化，所以通常第 3 种 shape 起就不再重编。但如果 shape 变化模式复杂（多维同时变、或 dynamic=False 强制静态），编译份数会撞到 recompile_limit（默认 8），之后 Dynamo 打一条 warning 就永久 fallback 到 eager——训练照跑，加速全没了。修法是 mark_dynamic 标注动态维，或把输入 pad 到固定的几个桶。

面试考点. Q7：为什么 torch.compile 比 torch.jit.script 成功？

A：采纳成本。jit.script 要求代码落在 TorchScript 的静态类型子集里，第三方库基本不可用，迁移经常要重写；遇到不支持就是编译失败，没有中间状态。Dynamo 不要求改代码，遇到不认识的就 graph break 回 Python，收益可以是部分的；而且它在符号执行时手上有真实输入，shape/dtype 都是已知值，靠 guard 兜住特化，不需要静态类型推断。

面试考点. Q8：torch.compiler.disable 和 torch.compiler.allow_in_graph 分别什么时候用？

A：disable 是“别 trace 这段”，用在日志、metric、调试代码上，让 Dynamo 干净地 break 一次而不是在函数内部反复失败。allow_in_graph 是反向：让 Dynamo 不看函数内部，把它当不透明 op 塞进图，用在 Dynamo 追不进去但语义是纯函数的调用上。后者危险——下游所有层都假设该节点无副作用，有 in-place 或随机性就会和 eager 静默不一致；自定义 kernel 的正确做法是用 torch.library 注册 custom op。

AOTAutograd：提前把反向图也编译出来

Dynamo 交出来的图只有前向。如果流水线到这里就结束，反向仍然由 eager 的 autograd engine 一个 op 一个 kernel 地跑——而训练里反向占约 2/3 的计算量，收益直接砍掉一大半。AOTAutograd 这一层的任务就是提前（ahead-of-time）把前向和反向一起 trace 成一张 joint graph，做完 functionalization 和 decomposition，再切成前向图和反向图交给 Inductor。

这一层还顺手解释了两件生产里绕不开的事：为什么 compile 之后显存占用会变、以及为什么 `torch.compile` 和 activation checkpointing 会打架。

先算清楚“只编前向”值多少

以最基本的 $y = x @ W$ （ x 是 (M, K) ， W 是 (K, N) ）为例：

$$\text{fwd} : y = xW \rightarrow 2MNK \text{ FLOPs}$$

反向要算两个梯度，各是一次同规模 GEMM：

$$\text{bwd} : dx = dyW^T, \quad dW = x^T dy \rightarrow 4MNK \text{ FLOPs}$$

Key insight. 反向的计算量是前向的 2 倍，占训练总量的 2/3。只编译前向，理论上限就是 1/3 的加速空间；而 elementwise/norm 的反向同样是 memory-bound，正是 fusion 收益最大的地方。这就是“为什么反向也要编译”的标准答案。

怎么在 dispatcher 层 trace 出反向

AOTAutograd 不解析 Python，它站在 PyTorch 的 dispatcher 上工作（dispatcher 见第 7 章）：

1. 用 `FakeTensor` 按 Dynamo 给的输入元信息造出一批“只有 shape/dtype/device 没有数据”的输入。`FakeTensor` 走真实的 meta kernel 做 shape 推导，所以不用分配一字节 HBM 就能算出每一步的 shape。
2. 用 `__torch_dispatch__` 挂在 dispatcher 上，把每个落到 dispatcher 的 ATen op 记录成 FX 节点。关键是它挂的位置在 **Autograd key** 之下：autograd 已经把反向展开成了具体的 ATen 调用，所以记录下来的图里反向是显式的 **op**，不是一个黑盒的 `backward()`。
3. 前向 trace 完之后，用一个 dummy 的输出梯度（图里叫 `tangents`）调 `.backward()`，让反向也被同一个 `__torch_dispatch__` 记下来。前向 + 反向记在一张图里，就是 joint graph。

看真实的 joint graph（`TORCH_LOGS="aot_joint_graph"`，`f(x, w) = gelu(x @ w).sum()`，A100 上实跑）：

```
===== Joint graph 0 =====
class inner_f(torch.nn.Module):
    def forward(self, primals, tangents):
        primals_1: "f32[8, 8][8, 1]cuda:0"; primals_2: "f32[8, 8][8, 1]cuda:0";
        tangents_1: "f32[8][8]cuda:0";
```

```

mm: "f32[8, 8]" = torch.ops.aten.mm.default(primals_1, primals_2)
gelu: "f32[8, 8]" = torch.ops.aten.gelu.default(mm)
sum_1: "f32[]" = torch.ops.aten.sum.default(gelu); gelu = None
expand: "f32[8, 8][0, 0]" = torch.ops.aten.expand.default(tangents_1, [8,
8])
gelu_backward: "f32[8, 8]" = torch.ops.aten.gelu_backward.default(expand,
mm)
t: "f32[8, 8][1, 8]" = torch.ops.aten.t.default(primals_1); primals_1 =
None
mm_1: "f32[8, 8]" = torch.ops.aten.mm.default(t, gelu_backward); t = None
t_1: "f32[8, 8][1, 8]" = torch.ops.aten.t.default(primals_2)
mm_2: "f32[8, 8]" = torch.ops.aten.mm.default(gelu_backward, t_1)
return pytree.tree_unflatten([sum_1, mm_2, mm_1], self._out_spec)

```

读法：`primals_*` 是前向输入（含参数），`tangents_*` 是输出梯度。`gelu_backward`、`t`、两个 `mm` 都是反向——它们在图里和前向的 `op` 平级，Inductor 完全不知道也不需要知道哪些是反向。`f32[8, 8][1, 8]` 这个标注里第二个方括号是 `stride`，可以看出 `t` 是零拷贝的 `stride` 变换。

Note. `aten.t` 出现在图里而不是被折进 `mm`，是因为 AOTAutograd 只负责如实记录；把 `t + mm` 变成一次 `mm` 的 `transposed` 读取是 Inductor 的活（第 14 章里能看到它生成 `reinterpret_tensor` 而不是真的转置拷贝）。

functionalization：把副作用消掉

Dynamo 抓到的图里 `in-place` 操作原样保留。用一个自定义 backend 打印 Dynamo 图，实跑输出：

```

def forward(self, L_x_ : torch.Tensor):
    l_x_ = L_x_
    y = l_x_.clone(); l_x_ = None
    add_ = y.add_(1.0); add_ = None
    mul_ = y.mul_(2.0); mul_ = None
    sum_1 = y.sum(); y = None
    return (sum_1,)

```

同一段代码经过 AOTAutograd (`TORCH_LOGS="aot_graphs"`)，`in-place` 全部被改写：

```

===== Forward graph 0 =====
def forward(self, primals_1: "f32[4][1]cpu"):
    clone: "f32[4][1]cpu" = torch.ops.aten.clone.default(primals_1)
    add: "f32[4][1]cpu" = torch.ops.aten.add.Tensor(clone, 1.0); clone =
None
    mul: "f32[4][1]cpu" = torch.ops.aten.mul.Tensor(add, 2.0); add = None
    sum_1: "f32[]cpu" = torch.ops.aten.sum.default(mul); mul = None
    return (sum_1,)

```

`add_` → `add`，`mul_` → `mul`。这个过程叫 `functionalization`：所有 `in-place`（`add_`、`copy_`、`index_put_`）和 `view` 上的写入（`x[0] = v`、`x.transpose(0,1).mul_(2)`）都被重写成“读旧值 → 算新值 → 产生新张量”的纯函数形式；如果被修改的张量是图的输入，最后再补一个显式的 `copy_` 把结果写回去。

为什么必须做这一步：

- 下游做的每个变换都假设无副作用。fusion、重排、CSE（公共子表达式消除）死代码消除，都建立在“节点之间只有数据依赖”这个前提上。有 in-place 就多了隐式的顺序依赖，而 FX 图里并不表达这种依赖。
- **min-cut partitioner** 要重算某些中间结果。只有纯函数才能被安全地重算（见下节）。
- 别名(**aliasing**)分析是不可判定的噩梦。与其在每个 pass 里处理“这两个张量是否共享 storage”，不如一次性把别名消掉。

Warning. functionalization 不等于“in-place 白写”。它只是让编译器内部看到纯函数；如果你在 forward 里 in-place 修改了一个外部张量(比如往一个全局 buffer 里写), AOTAutograd 会在图末尾补 copy_ 保证语义, 但这个 copy_ 是真实的额外 HBM 写。而 in-place 修改图的输入且该输入 **requires grad** 常常直接触发 graph break 或报错。

decomposition 与 PrimTorch

ATen 有 2000 多个 op (算上 overload 更多)。让每个后端都实现 2000 个 kernel 是不现实的。decomposition 就是一组 Python 写的规则，把复合 op 拆成更基础的 op：

```
# 概念示意：真实实现在 torch/_decomp/decompositions.py
def gelu_decomp(x):
    return x * 0.5 * (1.0 + torch.erf(x * 0.7071067811865476))
```

第 14 章里贴的真实 Triton kernel 中就能看到 0.5、0.7071067811865476、libdevice.erf 这几个常量——那正是 aten.gelu 被分解后的样子。

规模上，torch 2.10 里实测：torch.ops.prims 下有 125 个 prim op，而 torch._decomp.core_aten_decompositions() 返回 1004 条分解规则。也就是说一千多个 ATen op 的语义被一百多个 prim 表达出来了。

```
import torch
print(len([n for n in dir(torch.ops.prims) if not n.startswith("_")])) # 125
print(len(torch._decomp.core_aten_decompositions())) # 1004
```

有了这层收窄，一个新后端只要实现这一百多个 prim 就能跑全部 PyTorch 模型——这也是 PrimTorch 存在的商业理由（硬件厂商接入成本）。

Note. decomposition 是可选的、可配置的。torch.export 默认保留较高层的 ATen op（导出的图里能看到 aten.linear.default 而不是 mm + add，见第 16 章），而给 Inductor 的图分解得更彻底。同一个模型在不同下游会看到不同粒度的图，这不是 bug。

min-cut partitioner：显存和重算的取舍

joint graph 要切成前向图和反向图。切在哪里等价于回答一个问题：哪些中间结果在前向算完后保存下来给反向用，哪些在反向里重算？

- 保存 → 占显存，从前向一直活到反向。
- 重算 → 省显存，多花计算。

把 joint graph 看成一个有向图，“前向侧”和“反向侧”之间必须传递的张量就是一个割（cut）。割的容量是这些张量的总字节数。要最小化保存的显存，就是求最小割——这就是 min_cut_rematerialization_partition 的名字来源。

它就是 Inductor 路径上的默认切法：`torch/_inductor/compile_fx.py` 里的 `partition_fn` 在 `config.custom_partitioner_fn` is None 时直接调它。`torch._functorch.partitioners` 里另有一个 `default_partition`，是朴素切法（只保存 autograd 本来就要 `save_for_backward` 的那些），一般只在调试对比时用。

实际的策略比纯最小割精细：便宜的 `elementwise op`（`add`、`mul`、`gelu`、`sigmoid`）倾向于重算，因为它们 `memory-bound`，重算的成本远低于多存一份中间张量的代价；GEMM 这类贵的 `op` 一定保存。上面那张 `joint graph` 切完之后的前向图返回值就体现了这一点：

```
===== Forward graph 0 =====
    mm      = aten.mm.default(primals_1, primals_2)
    gelu     = aten.gelu.default(mm)
    sum_1    = aten.sum.default(gelu); gelu = None
    t        = aten.t.default(primals_1)
    t_1      = aten.t.default(primals_2)
    return (sum_1, mm, t, t_1)
```

它把 `mm` 的结果存了下来（因为 `gelu_backward` 需要它，而重算 `mm` 太贵），但没有存 `gelu` 的输出（反向里由 `gelu_backward(expand, mm)` 直接重算）。

实测：显存确实变少了

A100-SXM4-80GB，bf16，`x`、`w` 都是 (4096, 4096)，函数体是一次 GEMM 后接 4 轮 `sigmoid(tanh(y) * 1.5)`，做完整的 fwd + bwd，测 `torch.cuda.max_memory_allocated()`：

eager	368 MiB
torch.compile	176 MiB

图 21 峰值显存实测。eager 每个 `elementwise` 的中间结果都被 autograd 存下来；partitioner 判断它们该重算。

eager 里 8 个 `elementwise op` 各存一份 (4096, 4096) bf16 中间结果（32 MiB 一份）；编译后 partitioner 只保存 GEMM 的输出，其余在反向重算。

Warning. “compile 之后显存一定变少”是错的。partitioner 这一层的目标是“在不慢于 default 策略的前提下不比 eager 用更多 activation 显存”，但整个 `torch.compile` 的峰值显存可能上升，原因在别处：`mode="reduce-overhead"` 的 CUDA graph 要独占一个显存池；`max-autotune` 在 autotune 期间要为每个候选实现分配 workspace；Inductor 的 buffer 复用决策与 `caching allocator` 的实际行为不总是一致。所以换配置之后必须重测峰值，别靠推理。

手动控制取舍

`torch._functorch.config.activation_memory_budget` 是这一层最有用的旋钮，取值 0.0 到 1.0：

值	含义
1.0（默认）	纯按运行时最优切，只做那些“既省显存又不明显变慢”的重算
0.4	保存的 activation 压到默认策略的 40%，用 0-1 背包求“达标所需的最小重算量”
0.0	相当于对整个被编译区域做 activation checkpointing

它比手写 `checkpoint()` 好的地方是粒度：`checkpoint()` 是“整个 block 的中间结果全丢，反向从入口重算全部”，`budget` 是“按每个 `op` 的实际字节数和实际耗时求解，只丢该丢的”。

与 **activation checkpointing** 的关系

两者做的是同一件事——用重算换显存——所以叠加时要小心。

Warning. `torch.utils.checkpoint.checkpoint` 必须用 `use_reentrant=False` (`non-reentrant` 实现, 基于 `saved-tensor hooks`) 才能和 `torch.compile` 正常配合。老的 `reentrant` 实现会在反向里再进行一次 `autograd engine`, `Dynamo` 追不进去, 典型症状是 `graph break` 或者编译报错。这个参数在较新版本已经开始强制显式传值。

叠加时的实际后果: `checkpoint` 区域内部的 `activation` 已经被丢弃了, `AOTAutograd` 的 `partitioner` 在这个区域里能腾挪的空间就很小——你实际上是用一个粗粒度的手工决策覆盖了一个细粒度的自动决策。所以对已经能整图编译的模型, 先试 `activation_memory_budget` 调到需要的显存水位, 往往比手写 `checkpoint` 更快; `checkpoint` 的价值在于它跨 `graph break` 也有效、并且显存上限可预测。

超大模型上两者常常必须共存 (`checkpoint` 保证显存能装下, `compile` 负责 `fusion`)。这时把 `checkpoint` 的粒度放大到整个 `transformer block`, 让 `compile` 在 `block` 内部自由发挥。FSDP / 分布式场景下的组合见第 15 章和第四部分。

反向不总能提前编出来: **compiled autograd**

`AOTAutograd` 的前提是“前向能整图抓下来”。前向一旦有 `graph break`, 反向就被切成对应的若干段, 段与段之间仍然由 `eager` 的 `autograd engine` 串起来。更麻烦的是有些反向逻辑根本不在前向的 `trace` 范围内: 注册在张量上的 `backward hook`、DDP 的 `bucket AllReduce hook`、跨编译段的 `AccumulateGrad` ——这些是 `autograd engine` 在运行时才知道的。

`torch._dynamo.config.compiled_autograd = True` 换一个思路: 不在前向时提前 `trace` 反向, 而是在 `backward()` 真正开始时, 把 `autograd engine` 即将执行的整张反向图抓下来, 当场编译。代价是每次 `backward` 前多一次 `trace/guard` 检查, 收益是拿到一张真正完整的反向图 (含 `hook`、含跨段部分)。

按 `torch/_dynamo/config.py` 里的注释, 它还会放松前向 `trace` 的一些限制, 比如允许在编译区域内的张量上注册 `backward hook`。它也是 DDP 的 `optimize_ddp="python_reducer"` 模式的前提——那个模式让 DDP 关掉 C++ `reducer` 改用 Python `reducer`, 好让 `compiled autograd` 能把通信 `trace` 进反向图, 从而不靠 `graph break` 就实现 `comm/compute overlap`。

Note. `compiled autograd` 目前默认关闭, 属于“知道它存在、知道它解决什么问题”的层次。面试里被问“前向有 `graph break` 时反向怎么办”, 能说出“`AOTAutograd` 只能按段编, 完整反向图要靠 `compiled autograd` 在 `backward` 时现抓”就足够了。分布式场景下的取舍见第四部分。

怎么看 **AOTAutograd** 产出的图

开关	看到什么
<code>TORCH_LOGS="graph_code"</code>	<code>Dynamo</code> 抓到的图 (<code>AOTAutograd</code> 之前), Python 代码形式
<code>TORCH_LOGS="aot_joint_graph"</code>	切分之前的 <code>joint graph</code> 。判断“反向到底有没有被 <code>trace</code> 到”
<code>TORCH_LOGS="aot_graphs"</code>	切分之后的前向图和反向图。判断“哪些中间结果被保存了”
<code>TORCH_LOGS="post_grad_graphs"</code>	<code>post-grad pass</code> 之后、真正交给 <code>Inductor lowering</code> 的图

TORCH_LOGS="graph_sizes"	Dynamo 图里每个节点的 shape，动态 shape 调试用
TORCH_LOGS_OUT=/tmp/log.txt	同时把日志写到文件。这些图动辄几千行，必须重定向

配套的调试 backend (都是 `torch.compile(f, backend=...)`):

backend	行为
"eager"	Dynamo 抓图后原样用 eager 跑。验证“问题是不是 Dynamo 引起的”
"aot_eager"	走完 AOTAutograd(含 functionalization、partition)但不 codegen。定位“是 AOTAutograd 还是 Inductor”
"inductor"	默认，全流程

Key insight. 三分法排查 `torch.compile` 引起的数值错误或崩溃：`backend="eager"` 通过说明 Dynamo 没问题；`backend="aot_eager"` 也通过说明 AOTAutograd 没问题，锅在 Inductor 的 codegen；`aot_eager` 就挂了说明是 functionalization 或 partition 的问题(通常源头是你代码里的 in-place / 别名操作)。这套方法在面试里被问“你怎么 debug compile”时是标准答案。

面试考点

面试考点. Q1：AOTAutograd 解决什么问题？为什么必须有它？

A：Dynamo 只抓到前向，反向仍由 eager 的 autograd engine 逐 op 跑。而反向的计算量是前向的 2 倍 ($dx = dy W^T$ 和 $dW = x^T dy$ 各一次同规模 GEMM)，占训练总量 2/3。AOTAutograd 提前把前向和反向 trace 成一张 joint graph 再切开，让 Inductor 也能对反向做 fusion。

面试考点. Q2：AOTAutograd 是怎么 trace 出反向图的？

A：用 FakeTensor 造出只有元信息没有数据的输入 (走 meta kernel 做 shape 推导，不分配显存)，用 `__torch_dispatch__` 挂在 dispatcher 上记录每个 ATen op。关键是挂的位置在 Autograd key 之下，autograd 已经把反向展开成具体的 ATen 调用，所以图里的 `mm`、`t`、`gelu_backward` 都是显式节点。前向 trace 完用一个 dummy 的输出梯度调 backward，把反向也记进同一张图。

面试考点. Q3：什么是 functionalization？为什么必须做？

A：把 in-place 和 view 写入重写成纯函数形式 (`y.add_(1)` 变成 `add(y, 1)` 产生新张量；如果改的是图输入，末尾补一个显式 `copy_`)。必须做是因为下游的 fusion、重排、CSE、死代码消除都假设节点间只有数据依赖；in-place 引入的隐式顺序依赖在 FX 图里没法表达。另外 min-cut partitioner 要重算中间结果，只有纯函数才能安全重算。

面试考点. Q4：decomposition 和 PrimTorch 是干什么的？

A：ATen 有 2000+ 个 op，让每个后端都实现是不现实的。decomposition 是一组 Python 规则把复合 op 拆成基础 op (`gelu` 拆成 `mul + erf + add`)，PrimTorch 定义了目标的那一小组 prim op (两百个上下量级，覆盖 elementwise / reduction / view / 数据搬运)。新后端只

实现这一小组就能跑全部模型。注意分解粒度是可配的：`torch.export` 默认保留较高层的 ATen op，给 Inductor 的图分解得更彻底。

面试考点. Q5：min-cut partitioner 在决定什么？为什么叫 min-cut？

A：决定 joint graph 里哪些中间结果在前向保存、哪些在反向重算。把“前向侧到反向侧必须传递的张量集合”看成图的一个割，割的容量是这些张量的总字节数，最小化保存显存就是求最小割。实际策略还带成本模型：便宜的 memory-bound elementwise 倾向重算，GEMM 输出一定保存。实测一个 GEMM + 8 个 elementwise 的 bf16 例子，A100 上峰值显存从 eager 的 368 MiB 降到 176 MiB。

面试考点. Q6：`torch.compile` 之后显存一定变少吗？

A：不一定。partitioner 这一层的目标是不比 eager 用更多 activation 显存，但整体峰值可能上升：`mode="reduce-overhead"` 的 CUDA graph 要独占显存池，`max-autotune` 期间每个候选实现要 workspace，Inductor 的 buffer 复用与 caching allocator 的实际行为也不总一致。换配置必须重测 `max_memory_allocated()`。想主动压显存用 `torch._functorch.config.activation_memory_budget`（1.0 是默认的运行时最优，0.0 相当于对整个编译区域做 checkpointing）。

面试考点. Q7：activation checkpointing 和 `torch.compile` 一起用要注意什么？

A：`checkpoint` 必须传 `use_reentrant=False`；老的 reentrant 实现在反向里会再进行一次 autograd engine，Dynamo 追不进去，会 graph break 或直接报错。语义上两者做同一件事（重算换显存），叠加时 `checkpoint` 会用粗粒度的手工决策覆盖 partitioner 的细粒度自动决策，所以能整图编译的模型优先调 `activation_memory_budget`。超大模型两者共存时，把 `checkpoint` 的粒度放到整个 transformer block，让 compile 在 block 内自由发挥。

TorchInductor : codegen 与 fusion

前两层只是把 Python 变成了图,一个 kernel 都没省。真正产生加速的是 Inductor :它把图 lowering 成一个 loop-level 的 IR, 决定哪些 op 可以塞进同一个 kernel, 然后生成代码——GPU 上生成 Triton, CPU 上生成 C++/OpenMP。

面试里“`torch.compile` 为什么快”的标准答案只有一句：大部分逐点和归约操作是 **memory-bound** 的,融合成一个 **kernel** 就省掉了中间张量往返 **HBM** 的开销。这一章把这句话拆开算清楚,并贴上实测的 Triton 代码。

流水线

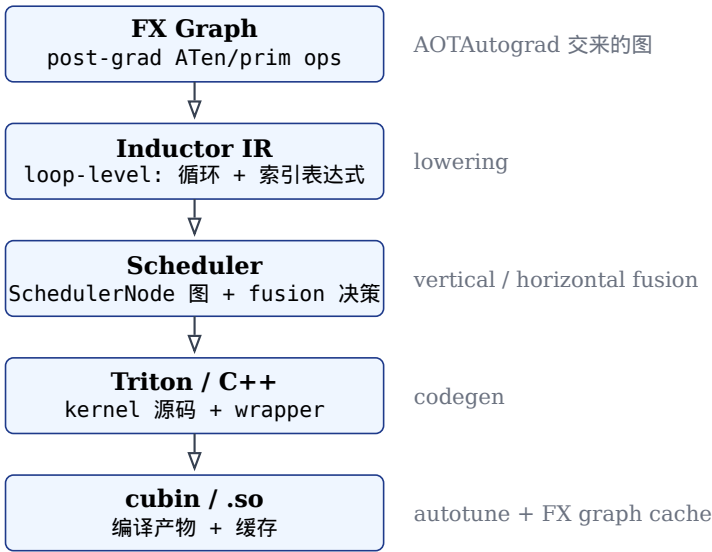


图 22 Inductor 内部五步。关键在中间两步：lowering 到 loop-level IR 让 fusion 变成“合并循环”这个可判定的问题。

lowering 这一步的核心是把每个 op 表示成“输出的每个元素怎么由输入索引算出来”。`add` 变成 `out[i] = a[i] + b[i]`, `gelu` 变成一串标量运算, `layer_norm` 变成一个带归约的循环。表示成这个形式之后, **fusion** 就等价于“能不能把两个循环合并成一个”——一个有明确判定条件的问题,而不是靠一堆手写的 pattern match 规则。

为什么 fusion 是主要收益来源

A100-SXM4-80GB 的两个关键数字：HBM 带宽 **2.04 TB/s**, bf16 tensor core 峰值 **312 TFLOPS**。两者的比值给出这块卡的 ridge point (算术强度拐点)：

$$I_{\text{ridge}} = \frac{312 \times 10^{12}}{2.04 \times 10^{12}} \approx 153 \text{ FLOP/byte}$$

意思是：一个 kernel 每从 HBM 读写 1 字节,必须做到 153 次浮点运算,才能把 tensor core 喂饱。低于这个强度的 kernel 一定是 memory-bound——加速的唯一途径是少读写字节。

操作	FLOP	bytes (bf16)	算术强度
<code>tanh(x)</code> (单独一个 kernel)	N	$4N$	约 0.25, 比 ridge 低 600 倍

$x + y$	N	$6N$	约 0.17
layer_norm (一行 1024)	约 $5N$	$4N$	约 1.25
mm 4096^3	$2 \cdot 4096^3$	$3 \cdot 4096^2 \cdot 2$	约 1365, 远高于 ridge

Key insight. elementwise、activation、norm、dropout 全部低于 ridge point 两三个数量级，它们的耗时就等于“读写了多少字节 ÷ 带宽”。 N 个这样的 op 串起来，eager 下要往返 HBM 大约 $2N$ 次；融合成一个 kernel 后只有一次读、一次写。省下来的时间就是全部收益。GEMM 完全在另一侧，它的优化路径不是 fusion（见后文）。

实测
A100-SXM4-80GB，bf16， x 是 (8192, 8192)（每次读或写 134.2 MB），函数是

```
def f(x):
    return torch.sigmoid(torch.tanh(x) * 2.0 + 1.0) * x
```

用 torch.profiler 数 kernel、用 CUDA event 计时（no_grad，50 次 warmup，200 次取平均）：

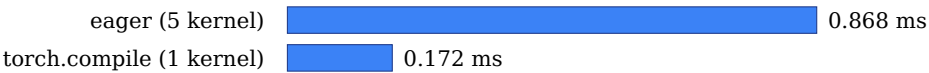


图 23 A100-SXM4-80GB / bf16 / (8192, 8192) 实测。5.06× 的来源全部是省下的 HBM 往返。对得上账：eager 的 5 个 kernel 一共约 11 次 134.2 MB 的 HBM 往返（tanh 读写各一次、mul 一次、add 一次、sigmoid 一次，最后 * x 要读两个输入写一个输出），共约 1476 MB，除以 0.868 ms 得约 1.70 TB/s——已经接近这块卡的带宽上限，说明 eager 的每个 kernel 本身写得没问题。融合后只剩 2 次往返共 268 MB，除以 0.172 ms 得约 1.56 TB/s。带宽利用率几乎没变，变的是要搬的字节数：11 次 → 2 次。

Note. 这也解释了为什么 torch.compile 在小张量上经常没有收益甚至变慢：张量小到 kernel 时间被 launch 开销和 kernel 内在效率主导时，省 HBM 往返省不出什么，而 Inductor 生成的 Triton kernel 未必比 ATen 手调的 kernel 更高效。实测 `gelu(LayerNorm(x)) * 1.5`、张量 (8, 512, 1024) fp32（每次往返 16.7 MB），eager 3 个 kernel 0.0657 ms、compile 1 个 kernel 0.0632 ms，只有 1.04×。小 shape 的正确解法是 `mode="reduce-overhead"`（第 15 章）。

fusion 的种类与阻碍

vertical fusion (producer-consumer)： $a = f(x)$ 后紧跟 $b = g(a)$ ，把 g 的计算接在 f 的寄存器里做完， a 根本不写回 HBM。这是绝大多数收益的来源。

horizontal fusion：两个互不依赖但迭代空间相同的 op（比如同 shape 的两个 elementwise），塞进同一个 kernel 分摊 launch 开销和索引计算。收益小得多。

什么阻止 fusion：

阻碍	为什么
----	-----

归约后接不同迭代空间的 op	归约把 (B, S, H) 变成 (B, S, 1), 后面再作用回 (B, S, H) 需要跨 block 广播; 只有当归约维能整块放进一个 program 时 (persistent reduction) 才融得动
中间结果有图外的使用者	该张量是图的输出、或被 .item() 之类读走, 必须真的物化到 HBM
内存布局冲突	两个 op 的最优 tiling/循环序不同, 硬融进去会导致非合并访存, Inductor 的启发式会拒绝
graph break	段边界上的张量必须落盘。第 12 章的账在这里结算
寄存器压力 / spill	融合链太长会超出寄存器预算, spill 到 local memory 反而更慢; Inductor 有 spill_threshold 之类的保护

看真实生成的 Triton kernel

把 LayerNorm(1024) + GELU 作用在 (8, 512, 1024) fp32 上, TORCH_LOGS="output_code" 实跑, Inductor 生成的是一个 Triton kernel (eager 是两个: vectorized_layer_norm_kernel 加一个 GELU 的 vectorized_elementwise_kernel):

```
@triton_heuristics.persistent_reduction(
    size_hints={'x': 4096, 'r0_': 1024},
    reduction_hint=ReductionHint.INNER,
    triton_meta={'signature': {'in_out_ptr0': '*fp32', 'in_ptr0': '*fp32',
                              'in_ptr1': '*fp32', 'in_ptr2': '*fp32',
                              'xnumel': 'i32', 'r0_numel': 'i32', 'XBLOCK': 'constexpr'}, ...},
    inductor_meta={'kernel_name': 'triton_per_fused_gelu_native_layer_norm_0',
                  'num_load': 3, 'num_store': 1, 'num_reduction': 4, ...},
)
@triton.jit
def triton_per_fused_gelu_native_layer_norm_0(in_out_ptr0, in_ptr0, in_ptr1,
                                              in_ptr2, xnumel, r0_numel,
                                              XBLOCK : tl.constexpr):

    xnumel = 4096                                # 8 * 512 行
    r0_numel = 1024                              # 归约维 (normalized_shape)
    R0_BLOCK: tl.constexpr = 1024                # 整行放进一个 program → persistent reduction
    xoffset = tl.program_id(0) * XBLOCK
    xindex = xoffset + tl.arange(0, XBLOCK)[: , None]
    r0_index = tl.arange(0, R0_BLOCK)[None, :]
    r0_1 = r0_index
    x0 = xindex
    tmp0 = tl.load(in_ptr0 + (r0_1 + 1024*x0), None) # x
    tmp21 = tl.load(in_ptr1 + (r0_1), None, eviction_policy='evict_last') # weight
    tmp23 = tl.load(in_ptr2 + (r0_1), None, eviction_policy='evict_last') # bias
    tmp1 = tl.broadcast_to(tmp0, [XBLOCK, R0_BLOCK])
    tmp3 = tl.broadcast_to(tmp1, [XBLOCK, R0_BLOCK])
    tmp5 = tl.sum(tmp3, 1)[: , None].to(tl.float32)
    tmp8 = (tmp5 / 1024.0) # mean
    tmp9 = tmp1 - tmp8
    tmp10 = tmp9 * tmp9
    tmp11 = tl.broadcast_to(tmp10, [XBLOCK, R0_BLOCK])
    tmp13 = tl.sum(tmp11, 1)[: , None]
    tmp16 = (tmp13 / 1024.0) # var
```



```

tmp19 = libdevice.rsqrt(tmp16 + 1e-05)
tmp20 = (tmp0 - tmp8) * tmp19      # 归一化
tmp24 = tmp20 * tmp21 + tmp23      # affine
tmp26 = tmp24 * 0.5                # ↓ gelu 的 decomposition
tmp28 = tmp24 * 0.7071067811865476
tmp29 = libdevice.erf(tmp28)
tmp32 = tmp26 * (tmp29 + 1.0)
tl.store(in_out_ptr0 + (r0_1 + 1024*x0), tmp32, None)

```

三件事值得注意：

1. 一次 `tl.load`，一次 `tl.store`。LayerNorm 的两趟归约、affine、GELU 的四步全在寄存器里完成。这就是前面那笔账的具体形态。
2. GELU 被分解了。`0.5`、`0.7071067811865476`、`libdevice.erf` 就是第 13 章讲的 decomposition 的产物——Inductor 从来没见过 `aten.gelu`。
3. **kernel** 名字自带来源。`triton_per_fused_gelu_native_layer_norm_0`：per 表示 persistent reduction (poi 是 pointwise, red 是普通 reduction, tem 是 template)，后面是被融进来的 ATen op 名。profiler timeline 里看到 kernel 名就知道融了什么，这是排查 fusion 的第一手信息。

看代码的两个入口：

```

TORCH_LOGS="output_code" python train.py 2>&1 | tee /tmp/oc.log # 打到 stderr
TORCH_COMPILE_DEBUG=1    python train.py                       # 落盘，含各阶段 IR
TORCH_LOGS="ir_pre_fusion,ir_post_fusion,fusion,schedule" python train.py # 看融合决策本身

```

matmul 不靠 fusion

GEMM 的算术强度远在 ridge point 之上，它需要的是 tiling、双缓冲、tensor core 指令编排——这些 cuBLAS/CUTLASS 已经做到极致了。Inductor 的策略是选，而不是自己生成。

默认 `mode` 下，`Linear(2048, 2048) + ReLU (bf16)` 实跑生成的 wrapper 是：

```

buf0 = empty_strided_cuda((2048, 2048), (2048, 1), torch.bfloat16)
# Original ATen: [aten.t, aten.addmm]
extern_kernels.mm(arg2_1,
    reinterpret_tensor(arg0_1, (2048, 2048), (1, 2048), 0), out=buf0)
buf1 = buf0; del buf0 # reuse
triton_poi_fused_addmm_relu_0.run(buf1, arg1_1, 4194304, stream=stream0)

```

读法：GEMM 交给 `extern_kernels.mm`（就是 cuBLAS）；权重的转置用 `reinterpret_tensor` 换 stride 完成，没有拷贝；bias 加法和 ReLU 融进一个 Triton pointwise kernel，并且直接写回 `buf0` 复用显存（`buf1 = buf0`）。

`mode="max-autotune"` 会额外把 Triton GEMM template 拉进候选池，benchmark 之后选最快的。同一个例子实跑，autotune 日志和选中的 kernel：

```

SingleProcess AUTOTUNE benchmarking takes 0.7233 seconds and 0.0031 seconds
precompiling for 20 choices

```



```
inductor_meta={'kernel_name': 'triton_tem_fused_addmm_t_0', ...,
  'config_args': {'EVEN_K': True, 'USE_FAST_ACCUM': False, 'ACC_TYPE':
'tl.float32',
                  'BLOCK_M': 64, 'BLOCK_N': 128, 'BLOCK_K': 64, 'GROUP_M': 8,
                  'ALLOW_TF32': False}}
```

tem 前缀说明这次选中了 Triton template (不是 cuBLAS), 并且 autotune 定下了 tile 尺寸。

epilogue fusion 就发生在这个 template 上: Triton template 的尾部留了一个可以插入 pointwise 计算的钩子, 于是 bias、activation、scale 可以在 GEMM 的累加器还在寄存器里的时候就算掉, 省掉一次 (M, N) 的往返。

Warning. epilogue fusion 会不会真的发生取决于 scheduler 的 benchmark 结果, 不是必然。上面那次 max-autotune-no-cudagraphs 的实跑里, Inductor 选了 Triton template 但把 bias + ReLU 留在了独立的 triton_poi_fused_addmm_relu_1 里。判断方法只有一个: 看 output_code 里 kernel 的数量和名字, 不要靠推断。

max-autotune 的代价

同一个 Linear(4096,4096) + GELU + Linear(4096,4096), bf16, 输入 (4096, 4096), A100 上用干净的 TORCHINDUCTOR_CACHE_DIR 实测:

配置	稳态 step	冷启动编译
eager	1.314 ms	—
mode 默认	1.331 ms	4.8 s
mode="max-autotune"	1.438 ms	13.6 s

Key insight. 这个例子里 max-autotune 更慢, 编译时间还贵了 3 倍。原因是这个 shape 上 cuBLAS 已经接近最优 (1.314 ms 对应约 209 TFLOPS, A100 bf16 峰值 312), Triton template 赢不了, 而 autotune 的候选池里挑出来的那个还带了额外的 epilogue 开销。结论是: max-autotune 是要实测才能用的选项, 尤其在标准 shape 的 GEMM 上没有免费午餐。它真正有价值的场景是不规则 shape、小 batch 的 GEMM、以及能吃到 epilogue fusion 的地方。

编译缓存

Inductor 的产物落在 TORCHINDUCTOR_CACHE_DIR, 默认 /tmp/torchinductor_\$USER。实测目录结构:

```
/tmp/torchinductor_duo.an/
├─ fxgraph/          # FX graph cache: 图 + 配置的 hash → 编译结果
├─ aotautograd/      # AOTAutograd 阶段的缓存
├─ triton/           # 编译好的 Triton kernel (cubin)
└─ <hash>/          # 生成的 Python wrapper 与 kernel 源码
```

缓存	键与作用
FX graph cache 配置项 fx_graph_cache, 默认开	图结构 + 输入元信息 + Inductor 配置 + PyTorch 版本 + GPU 架构的 hash → 整个 Inductor 编译结果

Triton / autotune cache	kernel 源码 hash → cubin ; autotune 选出的最优 config 也缓存，避免重复 benchmark
remote cache 配置项 <code>fx_graph_remote_cache</code>	同上但存在共享后端，供集群里多个 job 复用；默认未启用

为什么第一次慢：Dynamo 符号执行 + AOTAutograd trace + Inductor lowering/fusion + Triton 编译（要调 `ptxas`）+ 可能的 autotune benchmark，全部串在第一次调用里。

缓存命中的条件很严：图结构、输入的 shape/dtype/device、Inductor 与 Dynamo 的配置、PyTorch 版本、GPU compute capability，任何一项变了都不命中。所以改一行模型代码、换一张卡型号、升一次 **torch** 版本都会全量重编。

Warning. 缓存只省掉 Inductor 及以后的部分。同一个例子实测：冷缓存 4.8 s，热缓存（新进程，同 `TORCHINDUCTOR_CACHE_DIR`）3.8 s——只快了 1 s。Dynamo 的符号执行和 AOTAutograd 的 trace 每个进程都要重做一遍，不进 FX graph cache。所以“加了缓存冷启动就不慢了”是错的；真正想省启动时间要靠 `torch.export` + AOTInductor 把编译彻底移到离线（第 16 章）。

Warning. `/tmp` 在很多集群上是 tmpfs 或有清理策略，容器重启就没了；多个 job 并发写同一个 cache dir 也见过踩坏的案例。生产上把 `TORCHINDUCTOR_CACHE_DIR` 指到持久盘上的一个按“torch 版本 + GPU 型号”分开的路径。

Inductor 和手写 Triton 的边界

Inductor 生成的 Triton 已经能覆盖绝大多数 elementwise/reduction 的融合。什么时候还要自己写：

场景	为什么 Inductor 不够
FlashAttention 这类算法级重写	它不是“把几个循环合并”，而是改变了算法（online softmax + 分块，把 $O(S^2)$ 的中间矩阵彻底消掉）。Inductor 只做保语义的循环变换，想不出这种改写。用 <code>F.scaled_dot_product_attention</code> ，它会派发到 FlashAttention 后端
跨归约的复杂融合	归约 → 广播 → 再归约这种链，Inductor 的启发式常常拒绝融合
特殊数据布局 / 量化	4-bit 打包权重、block-scaled fp8、稀疏格式，索引逻辑无法用 Inductor 的 IR 表达
极致占用率调优	手动控制 <code>num_warps</code> 、 <code>num_stages</code> 、 <code>swizzle</code> 、异步拷贝流水

手写 Triton 的写法、tl 语义和 autotune 见仓库里的 `src/triton/`；CUDA 层面的 tiling 与 tensor core 见 `books/cuda/`。融进 `torch.compile` 的正确姿势是把手写 kernel 注册成 custom op（`torch.library`），这样 Dynamo 不会 break、AOTAutograd 也能拿到它的反向。

面试考点

面试考点. Q1：torch.compile 为什么快？

A : 主要收益是 kernel fusion。elementwise、activation、norm、dropout 的算术强度在 0.2 到几 FLOP/byte，而 A100 的 ridge point 是 $312 \text{ TFLOPS} \div 2.04 \text{ TB/s} \approx 153 \text{ FLOP/byte}$ ，低了两三个数量级，全是 memory-bound。eager 下 N 个这样的 op 要往返 HBM 约 2N 次，融成一个 kernel 只剩一次读一次写。次要收益是省 Python 和 kernel launch 开销（mode="reduce-overhead" 上 CUDA graph 才是这部分的主力）。

面试考点. Q2 : 给个具体数字，fusion 能省多少？

A : A100-SXM4-80GB、bf16、(8192, 8192)，sigmoid(tanh(x)*2+1)*x 实测：eager 5 个 kernel 0.868 ms，compile 1 个 kernel 0.172 ms，5.06x。对账：eager 约 11 次 134 MB 的 HBM 往返，共 1476 MB，除以耗时约 1.70 TB/s（已接近带宽上限）；融合后 2 次往返共 268 MB，约 1.56 TB/s。带宽利用率没变，字节数少了 5.5 倍。

面试考点. Q3 : Inductor 是怎么决定能不能融合的？

A : lowering 时把每个 op 表示成 loop-level IR（“输出每个元素怎么由输入索引算出来”），fusion 就变成“两个循环能不能合并”。最常见是 vertical fusion（producer-consumer，中间结果留在寄存器）；horizontal fusion（同迭代空间的独立 op）收益小。阻碍：归约改变了迭代空间、中间结果是图的输出必须物化、tiling/布局冲突导致非合并访存、graph break 切断了图、融合链太长导致寄存器 spill。

面试考点. Q4 : matmul 会被 fusion 优化吗？

A : 不会，GEMM 的算术强度远高于 ridge point，是 compute-bound，靠的是 tiling 和 tensor core 编排。Inductor 默认直接调 `extern_kernels.mm(cuBLAS)`；mode="max-autotune" 会把 Triton GEMM template 加进候选池 benchmark 后选最快的。GEMM 上唯一的 fusion 是 epilogue fusion——把 bias/activation/scale 接在累加器还在寄存器时算掉，省一次 (M, N) 的往返，但只有走 Triton template 时才可能。

面试考点. Q5 : mode="max-autotune" 一定更快吗？

A : 不一定。实测 `Linear(4096,4096)+GELU+Linear` bf16 在 A100 上：eager 1.314 ms，默认 mode 1.331 ms，max-autotune 1.438 ms，而冷启动编译从 4.8 s 涨到 13.6 s。标准 shape 的 GEMM 上 cuBLAS 已经接近最优（1.314 ms 约 209 TFLOPS，峰值 312），Triton template 赢不了。max-autotune 有价值的场景是不规则 shape、小 batch GEMM、以及能吃到 epilogue fusion 的地方——必须实测。

面试考点. Q6 : 怎么看 Inductor 生成了什么？

A : `TORCH_LOGS="output_code"` 打出完整的 Python wrapper + Triton kernel 源码；`TORCH_COMPILE_DEBUG=1` 落盘且包含各阶段 IR；`TORCH_LOGS="ir_pre_fusion,ir_post_fusion,fusion,schedule"` 看融合决策本身。看不全部代码时先看 kernel 名字：`triton_poi_*` 是 pointwise，`triton_per_*` 是 persistent reduction，`triton_red_*` 是普通 reduction，`triton_tem_*` 是 GEMM template，名字后半段直接列出被融进来的 ATen op。

面试考点. **Q7** : 为什么第一次调用很慢? 缓存能解决吗?

A : 第一次要跑完 Dynamo 符号执行、AOTAutograd trace、Inductor lowering/fusion、Triton 编译(调 `ptxas`), 可能还有 autotune benchmark。FX graph cache(默认开, 落在 `/tmp/torchinductor_$USER`)只缓存 Inductor 及以后, 键是“图结构 + 输入元信息 + 配置 + torch 版本 + GPU 架构”的 hash。实测冷 4.8 s、热 3.8 s——Dynamo 和 AOTAutograd 每个进程都要重做。要真正消掉启动开销得用 `torch.export` + AOTInductor 把编译移到离线。

面试考点. **Q8** : 什么时候 `torch.compile` 不够, 得自己写 Triton?

A : Inductor 只做保语义的循环变换, 想不出算法级的改写。FlashAttention 是典型: 它靠 `online softmax` + 分块消掉 $O(S^2)$ 的中间矩阵, 这是换算法不是换循环(实践中直接用 `F.scaled_dot_product_attention` 派发到 FlashAttention 后端)。其他要手写的场景: 跨归约的复杂融合链、4-bit 打包权重或 block-scaled fp8 这类特殊布局、需要手动控制 `num_warps` / `num_stages` / 异步流水的极致调优。手写 kernel 要用 `torch.library` 注册成 custom op 才能干净地嵌进 `torch.compile`。

生产里用 torch.compile

前三章讲机制，这章讲怎么用不踩坑。面试里这部分问得比机制更细，因为它直接暴露你有没有真的把 torch.compile 推上过生产：参数怎么选、动态 shape 怎么办、数值对不上算不算 bug、和 AMP/checkpoint/DDP 怎么组合、什么时候干脆别用。

参数逐个说清

```
torch.compile(model=None, *, fullgraph=False, dynamic=None,
              backend="inductor", mode=None, options=None, disable=False)
```

mode

torch 2.10 实测的合法取值（传错会报 Unrecognized mode=...）：default、lite、reduce-overhead、max-autotune-no-cudagraphs、max-autotune。

mode	做什么 / 适用场景	代价
default	fusion + codegen，不开 CUDA graph、不做 GEMM autotune。训练的默认选择	编译时间数秒到数十秒
reduce-overhead	default + CUDA graph。小 batch、decode、CPU 开销占比高的场景	shape 必须固定、要独占显存池、限制多（见下）
max-autotune	reduce-overhead + GEMM/卷积 autotune + coordinate descent 调参	编译时间常涨到 3 倍以上，收益不保证
max-autotune-no-cudagraphs	要 autotune 但不能用 CUDA graph（有动态 shape、有 CPU 同步、PP 通信）时用这个	同上，减去 CUDA graph 的限制
lite	torch 2.10 新增。默认对所有区域 fallback，只编译显式标注的区域，同时关掉大部分图级 pass	收益也相应小；适合“只想编一小块”的渐进式落地

dynamic

- dynamic=None（默认）：先按静态 shape 编一份；同一维度出现第二个不同的值时，Dynamo 自动把它符号化重编一份动态的（automatic dynamic shapes，由 torch._dynamo.config.automatic_dynamic_shapes = True 控制）。
- dynamic=True：一上来就假设 shape 是符号的。少编几次，但生成的 kernel 拿不到具体 size，某些 tiling 决策会保守。
- dynamic=False：禁止符号化，每种 shape 都单独编一份。这是最容易撞 recompile_limit 的设置。

fullgraph / backend / options

- fullgraph=True：遇到第一个 graph break 直接抛 Unsupported。开发期必用，见第 12 章。

- `backend` : `torch._dynamo.list_backends()` 在这套环境上实测返回 `['cudagraphs', 'inductor', 'openxla', 'tvm']` ;调试用的 `"eager"` / `"aot_eager"` 不在这个列表里但可用 (第 13 章)
- `options` : 直接塞 Inductor 配置, 比 `mode` 细。 `mode` 和 `options` 不能同时给。

```
model = torch.compile(model, options={"max_autotune_gemm": True,
                                     "triton.cudagraphs": False})
```

reduce-overhead 就是 CUDA graph

`mode="reduce-overhead"` 的实质是把 Inductor 生成的那串 kernel launch 用 CUDA graph 捕获下来, 之后每步只 replay 一次。收益是彻底消掉 per-kernel 的 CPU launch 开销。

实测: 8 层 `Linear(512, 512) + GELU, bf16, no_grad, batch = 1`, A100-SXM4-80GB:

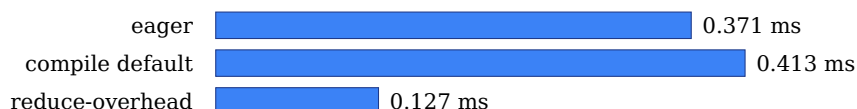


图 24 `batch=1` 时 GPU 几乎没活干, 耗时全是 launch 开销。CUDA graph 直接把它砍掉 2.9×; 而 `default` 因为 kernel 数没少多少, 反而比 `eager` 慢一点。

这组数字是理解 `reduce-overhead` 的关键: 它和 **fusion** 是两条独立的收益来源。 `batch` 大、GPU 饱和时它几乎没用; `batch` 小、层数多、单 kernel 很短时它是唯一的解。LLM 推理的 `decode` 阶段 (每步只算一个 token) 是最典型的受益场景。

限制 (都是 CUDA graph 本身的限制):

限制	后果与对策
输入 shape 必须固定	每种 shape 要重新捕获一张图。动态 shape 场景先 pad 到桶
输入张量地址必须固定	PyTorch 用一个静态输入 buffer + <code>copy_</code> 处理这件事; 但如果你自己持有输出张量的引用跨 step 使用, 会读到被下一次 replay 覆盖的数据
捕获期间不能有 CPU 同步	<code>.item()</code> 、 <code>.cpu()</code> 、 <code>print(tensor)</code> 都会让捕获失败或触发 fallback
独占一个显存池	峰值显存可能上升。换配置必须重测 <code>max_memory_allocated()</code>
动态 P2P 通信不兼容	pipeline parallel 的 <code>send/recv</code> 在 graph 里地址固定不了, PP 场景用 <code>default</code> 或 <code>max-autotune-no-cudagraphs</code>

Warning. 最常见的 CUDA graph 事故: 把编译后模型的输出直接存进一个 list 攒起来 (比如收集每步的 logits), 过几步之后发现前面的值全被改了。CUDA graph 的输出写在固定地址上, 下一次 replay 就覆盖。要跨 step 保留就 `.clone()`。需要手动切分 step 边界时用 `torch.compiler.cudagraph_mark_step_begin()`。

动态 shape

变长序列是 `torch.compile` 在生产里最大的敌人。每个新 shape 都是一次 guard 失败:


```
[0/1] [__recompiles] Recompiling function f in train.py:3
[0/1] [__recompiles]      triggered by the following guard failure(s):
[0/1] [__recompiles]      - 0/0: tensor 'x' size mismatch at index 0. expected 4,
actual 5
```

automatic dynamic shapes

好消息是默认行为已经帮了大忙。实跑一个函数、输入长度依次为 4、5、6：第 5 次触发一次 recompile（上面那条日志），到 6 就不再重编了——Dynamo 发现这一维变过，自动把它换成符号 `s0`，编出一份对该维通用的图。

坏消息是这个机制只在“变化模式简单”时有效。多维同时变、或者 batch 与 seq 组合出很多种情况时，编译份数照样爆。用 `dynamic=False` 强制静态则完全没有这层保护——实测 16 种不同长度的输入，编到第 8 份就撞上限：

```
W [0/8] torch._dynamo hit config.recompile_limit (8)
W [0/8]      function: 'f' (/tmp/v4_cachelimit.py:3)
W [0/8]      last reason: 0/7: tensor 'x' size mismatch at index 0. expected 11,
actual 12
```

之后这个函数永久 fallback 到 eager，不报错。

用 `mark_dynamic` 精确标注

想精确控制哪一维动态，用 `torch._dynamo.mark_dynamic`。它的完整签名（实测）：

```
torch._dynamo.mark_dynamic(t, index, *, hint_override=None,
                           min=None, max=None, specialize_on=None)
```

```
x = torch.randn(6, 16)
torch._dynamo.mark_dynamic(x, 0, min=2, max=128)  # 第 0 维动态，范围给出来
```

标注之后 Dynamo 抓到的图多了一个 `SymInt` 参数，实跑输出：

```
def forward(self, s13 : torch.SymInt, l_t_ : torch.Tensor):
    l_t_ = l_t_
    sin = l_t_.sin(); l_t_ = None
    sum_1 = sin.sum(-1); sin = None
    return (sum_1,)
```

给 `min` / `max` 不是可选的装饰：范围能让 Inductor 选更合理的 tiling，也能让 shape 相关的断言在编译期就求解掉，而不是在图里留一堆运行时检查。`maybe_mark_dynamic` 是“能动态就动态，不行就算了”的软版本。

0/1 specialization

shape 为 0 或 1 的维度总是被特化，即使你写了 `dynamic=True`。实测：`dynamic=True` 编译一个函数，依次喂 batch = 1、2、3、4，只在 1 → 2 时发生一次 recompile：


```
Recompiling function g in <string>:3
  triggered by the following guard failure(s):
  - 0/0: tensor 'x' size mismatch at index 0. expected 1, actual 2
```

原因是 size 1 会改变广播语义((1, N) 能广播到 (M, N), (2, N) 不能), size 0 会让很多 op 走空张量的特殊路径。所以符号 shape 的隐含约束是 $s \geq 2$ 。

Warning. 这个规则在推理服务里会咬人: batch=1 的请求和 batch=8 的请求走两份不同的编译产物。如果你的 warmup 只用 batch=1 跑过, 线上第一个非 1 的 batch 会在请求路径上触发一次几十秒的编译。warmup 必须覆盖所有会出现的桶, 包括 1。

bucketing: 朴素但最常用

比 mark_dynamic 更常见的生产解法是把输入 pad 到固定的几个长度:

```
BUCKETS = (128, 256, 512, 1024, 2048)

def pad_to_bucket(x, lengths):
    s = x.shape[1]
    tgt = next(b for b in BUCKETS if b >= s)
    return torch.nn.functional.pad(x, (0, 0, 0, tgt - s))
```

代价是浪费一部分计算(平均而言, 桶分得越粗浪费越多), 换来的是编译份数固定为桶的个数、每份都是静态 shape (于是 reduce-overhead 也能用)。桶的数量要留在 recompile_limit 以内, 或者相应调大它。

编译时间

冷启动从几秒到几分钟。实测 Linear(4096,4096)+GELU+Linear(4096,4096) 这么小的模型, 干净缓存下默认 mode 就要 4.8 s, max-autotune 要 13.6 s。真实的 LLM 训练脚本第一步花几分钟是常态。

缓解手段, 按性价比排:

1. 只编热点模块, 不编整个 **model**。一个 transformer 的 N 层是同构的, 编一层就够——nested_compile_region 或者直接 compile 单个 block 类, 能把编译时间从 O(N) 压到 O(1)。

```
for blk in model.blocks:
    blk.forward = torch.compile(blk.forward, dynamic=False)
```

2. 把动态维标好。每次 recompile 都是一次完整的编译时间。这是最大的单项收益。
3. 缓存落在持久盘上。TORCHINDUCTOR_CACHE_DIR 指到持久目录, 按 torch 版本 + GPU 型号分开。注意缓存只覆盖 Inductor 及以后, 实测冷 4.8 s 热 3.8 s (第 14 章)。
4. 别在 CI 或调参循环里开 max-autotune。它的收益需要实测确认, 编译时间的代价是确定的。

调试流程

按这个顺序走, 不要跳步:

1. fullgraph=True —— 先确认能不能整图编。挂了就有异常和源码行号, 直接改代码消除 graph break。

2. `torch._dynamo.explain(fn)(args)` —— 不想让它抛异常时，一次拿到 `graph_count` / `graph_break_count` / 每个 `break` 的原因和 `user stack`。
3. `TORCH_LOGS="graph_breaks,recompiles"` —— 跑真实训练，看有没有稳态阶段还在反复重编。`recompiles_verbose` 能看到所有失败的 `guard` 检查，不只是第一个。
4. 确认没撞上限 —— `grep "hit config.recompile_limit"`。撞了就先解决 `shape`，别急着调大 `limit`。
5. `TORCH_LOGS="output_code"` —— 怀疑“编了但没融合”时看生成的 `kernel` 数量和名字。
6. 怀疑正确性 —— `torch._dynamo.config.suppress_errors`（默认就是 `False`，别去改它）确保编译错误抛出来而不是静默 `fallback`；然后用 `backend="eager" → "aot_eager" → "inductor"` 三段夹逼定位是哪一层的问题（第 13 章）。

```
TORCH_LOGS="graph_breaks,recompiles" TORCH_LOGS_OUT=/tmp/compile.log python
train.py
grep -c "__graph_breaks" /tmp/compile.log
grep "hit config.recompile_limit" /tmp/compile.log
```

数值差异不是 bug

编译后的结果不会和 `eager bitwise` 相同。三个原因：

1. **fusion** 改了运算顺序。归约的累加顺序、中间结果是否落回低精度，都会变。
2. 可能换了 **GEMM** 实现。`max-autotune` 下尤其明显（`cuBLAS → Triton template`，`split-k` 策略不同）。
3. **TF32** / 精度策略。Inductor 生成的 Triton `kernel` 里的 `ALLOW_TF32` 和 `ACC_TYPE` 未必和 ATen `kernel` 的选择一致（第 14 章那段 `config_args` 里能直接看到这两项）。

实 测 `Linear(1024,4096)+GELU+Linear(4096,1024)`、`bf16`、`A100`：
`torch.equal(eager, compiled)` 为 `False`，最大绝对误差 `0.0078125`（正好是该量级下 `bf16` 的一个 `ulp`）。用 `torch.testing.assert_close` 卡容差，`rtol=atol=1e-2` 通过。

怎么验证“等价”：

- 单步：`torch.testing.assert_close(a, b, rtol=..., atol=...)`，容差按 `dtype` 定（`bf16` 放到 `1e-2` 量级是合理的，`fp32` 可以到 `1e-5`）。不要用 `torch.equal`，也不要再在 `bf16` 上要求 `rtol=1e-5`。
- 训练：比 `loss` 曲线。跑几百步，`compile` 和 `eager` 的 `loss` 应该在噪声范围内重合。这是唯一有说服力的判据——单步 `1e-2` 的相对误差看着大，但它和“换一个随机种子”的影响是同一量级。
- 怀疑真有 **bug** 时：用 `backend="aot_eager"` 对比，把 Inductor 摘出去。

Warning. 比 `loss` 曲线时先把随机性对齐：`dropout` 的 `RNG`、`dataloader` 的 `shuffle`、`torch.use_deterministic_algorithms`。否则你比的是两条不同的随机轨迹，看不出编译到底有没有引入偏差。相关细节见第 10 章。

常见坑清单

Warning. 在 `forward` 里 `print / .item() / .cpu()`。现象：`explain` 报一堆 `break`，加速接近零。修法：调试打印挪到 `forward` 外，或者包一层 `@torch.compiler.disable`；`loss` 的 `.item()` 放到 `backward()` 之后的训练循环里。

Warning. 每步换 **shape**。现象：日志里 `__recompiles` 刷屏，稳态阶段还在编译。修法：`mark_dynamic` 标注动态维，或 `pad` 到桶。别用 `dynamic=False`。

Warning. `recompile_limit` 静默 **fallback**。现象：训练速度慢慢退回 `eager`，没有报错。修法：上线前 `grep "hit config.recompile_limit"`；根治靠减少 `shape` 变化。

Warning. 第一步慢被当成“训练变慢”。现象：加了 `compile` 之后第一个 `step` 几十秒。修法：性能统计从第 10 步之后开始算；先跑一次覆盖所有桶的 `warmup`。

Warning. 编译 **optimizer step** 但忘了 `capturable=True`。现象：`optimizer` 里的 `step` 计数、`lr` 之类是 CPU 标量，每次都触发 `guard` 失败或 CPU 同步。修法：`torch.optim.AdamW(..., capturable=True)` 让状态全在 GPU 上。实测在 `torch 2.10` 上 `torch.compile` 包住 `opt.step()` 配 `capturable=True` 可以正常跑。

Warning. `if self.training:` 导致两份图。这不是坑而是必然：`training` 是 `guard` 的一部分。实测 `model.eval()` 之后再调用，日志给出 `- 0/0: fn._modules['1'].training == True`，触发一次重编。`train` 和 `eval` 各一份编译产物是正常的，只要别在训练循环里反复来回切。

Warning. **compile** 与 **hook** 的交互。注册在编译区域内部张量上的 `backward hook`，`AOTAutograd` 在前向 `trace` 时看不到，可能被忽略或引起 `graph break`。要靠它就开 `torch._dynamo.config.compiled_autograd = True`（第 13 章），或者把 `hook` 挪到编译边界之外的参数上。

Warning. `torch.compile` 包了 **dataloader** 或 **metric** 逻辑。这些代码里全是 Python 对象操作，`Dynamo` 会疯狂 `break` 且毫无收益。只 `compile` 纯计算的部分。

与其他特性组合

特性	怎么配
AMP / autocast	推荐 <code>autocast</code> 在外、 <code>compile</code> 在内(<code>compile(model)</code> 然后在训练循环里 <code>with autocast(...)</code>)。放在被编译的函数里面也能 <code>trace</code> ——实测 <code>graph_count 1, breaks 0</code> ——但放外面语义更清楚，且 <code>autocast</code> 的 <code>dtype</code> 会进 <code>guard</code>
activation checkpoint	必须 <code>use_reentrant=False</code> 。粒度放到整个 <code>block</code> 。能整图编译的模型优先改用 <code>activation_memory_budget</code> （第 13 章）
DDP	先 <code>DDP(model)</code> 再 <code>torch.compile</code> 。 <code>torch._dynamo.config.optimize_ddp</code> 默认 <code>True</code> （等价于 <code>"ddp_optimizer"</code> ），按 <code>DDP bucket</code> 大小把图切成多段，让通信能早发从而与计算 <code>overlap</code> 。另有 <code>"python_reducer"</code> （需要 <code>compiled autograd</code> ，不靠 <code>graph break</code> 做 <code>overlap</code> ）和 <code>"no_optimization"</code> （不切图，但没有 <code>overlap</code> ）

FSDP2	较新版本支持度已经不错，DTensor 的 sharding 元信息在编译期可见。仍建议对单个 block compile 而不是整个 FSDP 包装体
optimizer	<code>torch.compile</code> 可以编 <code>opt.step()</code> ，把逐参数的 <code>elementwise</code> 更新融成少量 kernel(对参数多的模型收益明显)。前提是 <code>capturable=True</code> 或用 <code>foreach</code> 实现
pipeline parallel	别用 <code>reduce-overhead</code> ；动态 P2P 与 CUDA graph 不兼容

生产推荐 idiom：

```

model = MyModel().cuda()

# 1) 分布式包装在最外，compile 在里面（或直接 compile 每个 block）
model = DDP(model, device_ids=[local_rank])

# 2) 开发期先用 fullgraph=True 把图打通，确认后再放开
model = torch.compile(model, dynamic=False) # shape 稳定时；变长就去掉或
mark_dynamic

opt = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True)

# 3) autocast 在 compile 外层
for step, batch in enumerate(loader):
    with torch.autocast("cuda", dtype=torch.bfloat16):
        loss = model(batch).loss
    loss.backward()
    opt.step()
    opt.zero_grad(set_to_none=True)
    if step == WARMUP: # 4) 性能统计从 warmup 之后开始
        timer.reset()

```

配套的环境变量：

```

export TORCHINDUCTOR_CACHE_DIR=/persistent/inductor_cache/torch2.10-a100
export TORCH_LOGS="graph_breaks,recompiles" # 只在验证阶段开
export TORCH_LOGS_OUT=/tmp/compile.log

```

什么时候不该用

1. **shape** 高度动态且没法 **bucket**。比如输入 shape 由数据内容决定（检测框数量、稀疏图的节点数）。编译份数控制不住，收益被重编吃光。
2. 模型很小、**CPU** 开销占主导，且 **shape** 不固定。CPU 开销大本该用 `reduce-overhead`，但 shape 不固定又用不了 CUDA graph，两头堵。
3. 频繁改代码的实验阶段。改一行模型就全量重编。用 `torch.compiler.set_stance("force_eager")` 或 `disable=True` 一键关掉，跑通了再开。
4. 编译时间不可接受。短时任务（跑几十个 step 的 ablation）或者按请求冷启动的 serverless 推理。后者的正确答案是 `torch.export` + AOTInductor（第 16 章）。
5. 已经被通信或数据加载卡住。step time 的 60% 花在 AllReduce 或 dataloader 上时，把计算再快 20% 也看不出来。先 profile（第 11 章）。

Key insight. 面试里被问“你怎么决定要不要上 compile”，答案的结构应该是：先 profile 确认瓶颈在 GPU 计算上；看 kernel 分布判断是 memory-bound 的碎 kernel 多 (fusion 有戏) 还是 launch 开销占主 (CUDA graph 有戏)；再看 shape 稳不稳定决定 dynamic 和能不能用 reduce-overhead；最后用 loss 曲线验证数值等价。给不出这条链的人，通常是抄了别人的 `torch.compile(model)` 然后发现没变快。

面试考点

面试考点. Q1：torch.compile 的 mode 有哪些？分别什么时候用？

A：torch 2.10 上是 default、lite、reduce-overhead、max-autotune-no-cudagraphs、max-autotune。default 是训练默认，只做 fusion 和 codegen。reduce-overhead 额外开 CUDA graph，用在小 batch / decode / CPU 开销占主的场景。max-autotune 再加 GEMM autotune，编译时间常涨 3 倍以上且收益不保证，要实测。有动态 shape 或 CPU 同步、不能用 CUDA graph 但想 autotune 时用 max-autotune-no-cudagraphs。

面试考点. Q2：mode="reduce-overhead" 的原理和限制？

A：原理是用 CUDA graph 捕获 Inductor 生成的 kernel launch 序列，之后每步 replay 一次，消掉 per-kernel 的 CPU launch 开销。实测 8 层 Linear(512,512) bf16 batch=1 在 A100 上：eager 0.371 ms、默认 compile 0.413 ms、reduce-overhead 0.127 ms。限制：shape 必须固定、输入输出地址固定（跨 step 保留输出必须 `.clone()`）、捕获期不能有 CPU 同步、独占一个显存池、与动态 P2P 通信不兼容（PP 别用）。

面试考点. Q3：变长序列为什么会反复 recompile？怎么解决？

A：shape 是 guard 的一部分，每个新 shape 触发 guard 失败 → 重编。Dynamo 的 automatic dynamic shapes 会在某一维出现第二个值时自动把它符号化，所以简单情况第 3 种 shape 起就不再重编；但多维同时变时编译份数照样爆，撞到 recompile_limit（默认 8）就永久 fallback 到 eager 且只打一条 warning。解法：`torch._dynamo.mark_dynamic(x, dim, min=..., max=...)` 标注动态维并给出范围；或者更常用的 pad 到固定的几个桶（桶数留在 limit 以内，且每桶都是静态 shape 所以还能用 reduce-overhead）。

面试考点. Q4：什么是 0/1 specialization？

A：shape 为 0 或 1 的维度总是被特化，即使写了 `dynamic=True`。因为 size 1 改变广播语义（(1,N) 能广播到 (M,N)，(2,N) 不能）size 0 会走空张量的特殊路径。所以符号 shape 隐含 `s >= 2`。实测 `dynamic=True` 下 batch 依次为 1、2、3、4，只在 1 → 2 时重编一次。工程后果：推理服务里 batch=1 和 batch>1 是两份产物，warmup 必须覆盖包括 1 在内的所有桶，否则线上第一个非 1 的 batch 会在请求路径上触发几十秒编译。

面试考点. Q5：compile 之后结果和 eager 不一样，是 bug 吗？

A：不 bitwise 相同是正常的。原因：fusion 改变了归约的累加顺序和中间结果的精度、可能换了 GEMM 实现（max-autotune 下尤其）、TF32 与累加 dtype 的选择可能

和 ATen kernel 不同。实测 bf16 的两层 MLP，最大绝对误差 0.0078（约一个 bf16 ulp），`assert_close(rtol=1e-2, atol=1e-2)` 通过。验证方式：单步用 `assert_close` 放宽容差（bf16 到 $1e-2$ 量级），训练用 loss 曲线在几百步内重合，怀疑真有 bug 时用 `backend="aot_eager"` 把 Inductor 摘出去对比。

面试考点. Q6：讲一下你排查 `torch.compile` 性能问题的流程。

A：先 `fullgraph=True` 确认能不能整图编，挂了就按异常里的源码行改；不想抛异常就用 `torch._dynamo.explain` 看 `graph_break_count` 和每个 break 的原因；跑真实训练开 `TORCH_LOGS="graph_breaks,recompiles"`（配 `TORCH_LOGS_OUT` 落盘）看稳态还在不在重编；`grep hit config.recompile_limit` 确认没静默 fallback；怀疑“编了但没融合”就看 `TORCH_LOGS="output_code"` 里 kernel 的数量和名字；怀疑正确性用 `eager` → `aot_eager` → `inductor` 三段夹逼定位层。

面试考点. Q7：DDP 和 `torch.compile` 谁包谁？为什么？

A：先 DDP(model) 再 `torch.compile`。因为 `torch._dynamo.config.optimize_ddp` 默认开（`"ddp_optimizer"`），它会按 DDP 的 bucket 大小把图切成多段，让每个 bucket 的梯度一算完就能发 AllReduce，从而和后面的计算 overlap；如果反过来 compile 出一整张大图，所有梯度到图的最后才出来，通信完全串行在计算之后。另有 `"python_reducer"` 模式配合 compiled autograd，能不靠切图就 overlap。

面试考点. Q8：怎么压缩编译时间？

A：性价比从高到低：只 compile 热点模块——transformer 的 N 层同构，compile 单个 block 或用 `nested_compile_region` 把编译时间从 $O(N)$ 压到 $O(1)$ ；把动态维标好，因为每次 recompile 都是一次完整编译；`TORCHINDUCTOR_CACHE_DIR` 指到持久盘并按 torch 版本 + GPU 型号分开（注意只覆盖 Inductor 及以后，实测冷 4.8 s 热 3.8 s）；别在 CI 和调参循环里开 `max-autotune`。彻底消掉启动开销要靠 `torch.export` + AOTInductor。

面试考点. Q9：什么情况下不该用 `torch.compile`？

A：shape 由数据内容决定又没法 bucket（检测框数、稀疏图节点数）——编译份数控制不住；模型小、CPU 开销占主但 shape 又不固定——想用 CUDA graph 用不了，两头堵；频繁改模型代码的实验阶段——改一行全量重编，用 `set_stance("force_eager")` 或 `disable=True` 关掉；编译时间不可接受的短时任务或按请求冷启动的推理（后者该用 AOTInductor）；瓶颈本来就在通信或 dataloader 上——先 profile 再说。

导出与部署

`torch.compile` 解决的是“在 Python 里跑得更快”。部署常常有另一个要求：把模型变成一个不依赖 **Python** 的产物——因为推理服务是 C++ 写的、因为要跑在没有 Python 的设备上、因为几十秒的 JIT 冷启动放在请求路径上不可接受。这是 `torch.export` 和 AOTInductor 的地盘。

面试里这块的高频题就两类：三代方案的关系与取舍，以及 `trace` 和 `script` 的区别。

三代方案

版本	核心思路	相对成本 / 收益
TorchScript (2019)	<code>jit.trace</code> 记录执行 / <code>jit.script</code> 解析源码	维护模式，不再加新特性
<code>torch.compile</code> (2023)	Dynamo + Inductor, JIT, 可 graph break	仍需要 Python runtime
<code>torch.export</code> + AOTInductor	AOT 严格导出成图,再编成 .so	无 Python 依赖,但要求整图

方案	时机	graph break	产物 / 用途
<code>torch.jit.trace</code>	AOT	不存在（直接丢控制流）	<code>ScriptModule</code> ，历史遗留
<code>torch.jit.script</code>	AOT	不允许（语法不支持就失败）	<code>ScriptModule</code> ，历史遗留
<code>torch.compile</code>	JIT	允许，自动 fallback	进程内的编译产物，训练与推理加速
<code>torch.export</code>	AOT	不允许，遇到就报错	<code>ExportedProgram</code> （图 + 签名 + 权重）
AOTInductor	AOT	不允许	.so / .pt2 包, C++ runtime 加载
ONNX	AOT	不允许	.onnx，交给 ORT / TensorRT 等

Key insight. 一句话总结取舍：`torch.compile` 是 JIT 且宽容（有 fallback，所以能上任何模型，但要 Python）；`torch.export` 是 AOT 且严格（必须整图，所以要改代码，但换来一个可序列化、可脱离 Python 的产物）。两者共享 Dynamo 和 AOTAutograd 的前半段，所以把模型改到 `fullgraph=True` 能过，就是把它改到能 **export** 的九成工作量。

jit.trace vs jit.script

这题几乎必问，因为它检验你是否理解“记录一次执行”和“理解程序”的区别。

```
def g(x):
    if x.sum() > 0:
        return x * 2
    return x * 3

t = torch.jit.trace(g, torch.ones(3))
print(t.code)
```

实跑输出——`if` 整个消失了：


```
def g(x: Tensor) -> Tensor:
    return torch.mul(x, CONSTANTS.c0)
```

于是换一条分支的输入结果就是错的，而且不抛任何异常：

```
trace(ones) -> [2.0, 2.0, 2.0]    eager: [2.0, 2.0, 2.0]    ✓
trace(-ones) -> [-2.0, -2.0, -2.0] eager: [-3.0, -3.0, -3.0] x
```

`torch.jit.script(g)` 解析源码，保留了 `if`，`script(-ones)` 实跑得到 `[-3.0, -3.0, -3.0]`，正确。

Warning. `jit.trace` 会固化的东西不止 `if : for` 的循环次数被展开成固定值(变长序列直接错) `.item()` 的结果变成常量、`tensor.shape[0]` 变成一个整数字面量、`if self.training` 被定死在 `trace` 时的模式。它只在 `trace` 时对 `shape` 变化和不同分支发出 `TracerWarning` ——而这个 `warning` 在真实项目的日志里几乎注定被淹掉。这是 PyTorch 历史上制造过最多线上事故的 API 之一。

`jit.script` 正确但采纳成本高：只支持 Python 的一个静态类型子集，第三方库基本不可用，`Optional` 要显式标注。这是它没能普及的原因，也是 Dynamo 设计时刻意避开的坑（第 12 章）。

torch.export

```
torch.export.export(mod, args, kwargs=None, *, dynamic_shapes=None,
                    strict=False, preserve_module_call_signature=(),
                    prefer_deferred_runtime_asserts_over_guards=False)
```

产出的 `ExportedProgram` 有三个组成部分：图、**graph signature**、**state dict**。

```
import torch, torch.nn as nn
from torch.export import export, Dim

class M(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(8, 4)
    def forward(self, x):
        return torch.relu(self.fc(x))

m = M().eval()
B = Dim("B", min=1, max=1024)
ep = export(m, (torch.randn(2, 8),), dynamic_shapes={"x": {0: B}})
print(ep.graph_signature)
print(ep.graph_module.code)
print(list(ep.state_dict.keys()))
print(ep.module()(torch.randn(5, 8)).shape)    # 用别的 batch 跑
```

实跑输出：

```

# inputs
p_fc_weight: PARAMETER target='fc.weight'
p_fc_bias:   PARAMETER target='fc.bias'
x:           USER_INPUT

# outputs
relu: USER_OUTPUT

def forward(self, p_fc_weight, p_fc_bias, x):
    linear = torch.ops.aten.linear.default(x, p_fc_weight, p_fc_bias)
    relu = torch.ops.aten.relu.default(linear)
    return (relu,)

['fc.weight', 'fc.bias']
torch.Size([5, 4])

```

三件事：

1. 参数被提升成图的输入。 graph signature 记录了“第 1 个输入其实是 `fc.weight` 这个 parameter”。图本身是纯函数，权重通过 `state_dict` 单独携带——这是能序列化、能换权重、能量化的前提。
2. **op** 层级比给 **Inductor** 的图高。这里是 `aten.linear.default`，不是 `mm + add`。想彻底分解调 `ep.run_decompositions()`，实跑变成 `permute + addmm + relu`。
3. **ep.module()** 能直接当普通 **module** 跑，方便做数值对比。

序列化用 `torch.export.save / load`（实测上面这个模型存成 `.pt2` 是 8356 字节）：

```

torch.export.save(ep, "/tmp/ep.pt2")
ep2 = torch.export.load("/tmp/ep.pt2")

```

用 **dynamic_shapes** 声明动态维

用 `Dim` 对象声明哪一维动态。三种写法：

```

B = Dim("B", min=1, max=1024)      # 具名，给范围；同名 Dim 表示"这几处必须相等"
Dim.AUTO                            # 让 export 自己推断这一维是否动态
Dim.STATIC                          # 显式声明静态
dynamic_shapes={"x": {0: B}, "mask": {0: B}}  # 两个输入的第 0 维绑定成同一个符号

```

导出后的约束可以直接看出来：

```

print(ep.range_constraints)      # {s77: VR[2, int_oo]}

```

`VR[2, int_oo]` 的下界是 2 而不是 1——这就是第 15 章讲的 0/1 specialization 在 export 里的体现：符号维度隐含 `>= 2`，要支持 `batch=1` 得单独导一份。

必须整图

export 不允许 graph break。碰到数据相关的控制流就直接报错。实跑 `if x.sum() > 0:`：

```

GuardOnDataDependentSymNode: Could not guard on data-dependent expression
Eq(u0, 1) (unhinted: Eq(u0, 1)). (Size-like symbols: none)

```

```
consider using data-dependent friendly APIs such as guard_or_false,
guard_or_true and statically_known_true.
...
The following call raised this error:
File "/tmp/vll_export.py", line 27, in forward
    if x.sum() > 0:
```

`u0` 是 `unbacked symbol`——一个“运行时才知道值”的符号。`export` 拿它没法决定走哪条分支，于是失败。修法：把数据相关控制流改写成张量运算（`torch.where`）或用 `torch.cond` 这类显式的控制流算子、或把这段逻辑挪到模型外面。

Note. `torch 2.10` 里 `strict` 的默认值是 `False`。非严格模式直接在 Python runtime 下 `trace`，仍然校验 `shape` 安全这类关键假设，但不验证全部隐式假设；`strict=True` 走 `TorchDynamo`，保证图的 `soundness`，代价是 Python 特性覆盖更窄、更容易报错。文档明确说两者产出的 **IR** 规格相同，序列化方式也一样。调试导出失败时可以用 `torch.export.draft_export` 先拿到一份带诊断信息的结果。

AOTInductor

把 `ExportedProgram` 编成一个自包含的包，用 C++ runtime 加载，完全不需要 Python：

```
ep = torch.export.export(m, (x,), dynamic_shapes={"x": {0: Dim("B", max=64)}})
path = torch._inductor.aoti_compile_and_package(ep, package_path="/tmp/m.pt2")
runner = torch._inductor.aoti_load_package(path) # Python 侧也能加载，方便验证
runner(torch.randn(7, 64, device="cuda")) # batch 7，动态维生效
```

实测这个小模型的 `.pt2` 包是 434824 字节，`aoti_load_package` 加载后用 `batch=7` 调用正常。C++ 侧用 `torch::inductor::AOTIModelPackageLoader` 加载同一个包。

适用场景：

- **C++ 推理服务。** 已有的 C++ 服务框架，不想引入 Python 解释器和 GIL。
- 冷启动敏感。编译在离线做完，上线加载 `.so` 是毫秒级，而 `torch.compile` 的首次调用是几十秒。
- 交付一个二进制。不想把模型代码和一堆 Python 依赖发给客户。

Warning. AOTInductor 的产物是绑定 **GPU** 架构和 **torch** 版本的。为 A100 (sm80) 编的包不能拿去 H100 (sm90) 跑，`torch` 小版本升级也可能不兼容。CI 里要按“`torch 版本 × GPU 架构`”的矩阵产出多份包，并且把这两个维度写进产物的文件名。

ONNX

`torch.onnx.export` 在较新版本里已经切到基于 `torch.export` 的新路径，`dynamo` 参数在 `torch 2.10` 上默认就是 `True`：

```
torch.onnx.export(model, (x,), "m.onnx", dynamo=True,
                  dynamic_shapes={"x": {0: Dim("B", max=128)}}),
                  opset_version=18, optimize=True)
```

新旧两条路径的区别：

路径	行为
旧 tracer (<code>dynamo=False</code>)	基于 <code>jit.trace</code> , 继承了它丢控制流的全部问题 ; 动态轴用 <code>dynamic_axes</code> 声明 (只是给轴起名 , 不做符号推理)
新路径 (<code>dynamo=True</code> , 默认)	先 <code>torch.export</code> 拿到 <code>ExportedProgram</code> 再转 ONNX ; 动态维用 <code>dynamic_shapes</code> + <code>Dim</code> , 有真正的符号 shape 约束

常见导出失败原因 :

1. **op** 在目标 **opset** 里没有对应实现。报“unsupported operator”。对策 : 升 `opset_version`、换等价写法、或写 custom translation。
2. **data-dependent shape**。 `nonzero`、`x[mask]`、`unique` 的输出 shape 依赖数据 , ONNX 的 shape 推理表达不了。
3. **data-dependent** 控制流。和 export 同一个问题 , 先在 PyTorch 侧改掉。
4. 动态轴声明不全。导出时用一个具体 batch , 忘了声明动态 , 结果 ONNX 图里 batch 被写死成常量。这类问题不报错 , 只在换 batch 时才暴露。

Note. `dynamo=True` 这条路径依赖 `onnxscript` 包。这套环境里没装 , 实跑直接得到 `ModuleNotFoundError: No module named 'onnxscript'` ——所以本节的 ONNX 代码没有在本机跑通 , 签名是从 `inspect.signature(torch.onnx.export)` 实测读出来的 , 行为描述来自文档。部署链路里 ONNX 通常只是中转格式 , 真正跑的是 ORT / TensorRT , 它们各自还有一层图优化和 op 支持范围。

推理侧的常规动作

动作	作用
<code>model.eval()</code>	切 dropout 到恒等、BN 用 running stats。忘了这句是最经典的“推理结果抖动”故障
<code>with torch.inference_mode():</code>	比 <code>no_grad</code> 更强 : 不建 autograd 图 , 还关掉 version counter 和 view 追踪 , 省一点开销。代价是产出的张量不能再参与 autograd
fuse conv + bn	推理时 BN 是仿射变换 , 可以吸进前面卷积的权重和 bias , 省一个 kernel 和一次往返。 <code>torch.ao.nn.intrinsic</code> 和 <code>torch.fx.experimental.optimization.fuse</code> 有现成实现 ; <code>torch.compile</code> 也会做类似的常量折叠 (<code>freezing</code> 配置)
KV cache	自回归解码里避免重算历史 token 的 K/V。这是 LLM 推理的第一优化 , 见第 26 章
量化	PTQ (训练后量化 , 校准几百条数据就能上) 与 QAT (训练中插伪量化 , 精度更好但要重训)。现在的推荐入口是 <code>torchao</code> , 老的 <code>torch.quantization</code> / <code>torch.ao.quantization</code> eager 模式路径已不推荐
batching	静态 batching 攒够就发 ; continuous batching 让每个请求独立进出 , GPU 利用率高得多。这是推理引擎 (<code>vLLM</code> / <code>SGLang</code>) 的职责 , 不是 PyTorch 层要解决的问题

Key insight. 面试里问“你怎么部署 LLM”，正确的答案层次是：单模型的图优化归 `torch.compile` / `torch.export`；调度、KV cache 管理、continuous batching、prefix caching 归推理引擎（vLLM / SGLang）。自己拿 `torch.export` 从零搭一个 LLM 服务是重复造轮子，说得这个边界比说得 API 更重要。

state_dict VS 整模型保存

```
torch.save(model.state_dict(), "sd.pt")      # 推荐：只存张量
torch.save(model, "whole.pt")                # 不推荐：pickle 整个对象
```

`torch.save(model)` 用 pickle 序列化整个对象图，文件里只存了类的引用路径（`mypkg.models.MyModel`），不存类的定义。后果：

- 加载时必须能 import 到同名同路径的类。重构包结构、改类名、换代码分支，checkpoint 就打不开了。
- pickle 能执行任意代码，加载不可信文件等于执行不可信代码。

`torch.load` 在 torch 2.6 起把 `weights_only` 的默认行为改成了只允许张量和少数安全类型。实测直接 `torch.load("whole.pt")` 现在会失败：

```
UnpicklingError: Weights only load failed. This file can still be loaded, to do so
you have two options, do those steps only if you trust the source of the
checkpoint.
```

```
(1) In PyTorch 2.6, we changed the default value of the `weights_only` argument
    in `torch.load` from `False` to `True`. Re-running `torch.load` with
    `weights_only=False` ...
```

Warning. 看到这个报错的正确反应不是马上加 `weights_only=False`。先问：这个文件是谁产的？如果是自己训练产出的 `state_dict`，它本来就該能以 `weights_only=True` 加载，报错说明存的是整模型或者混了自定义对象——改存 `state_dict`。只有确认来源可信、且真的需要反序列化自定义类时才关掉，或者用 `torch.serialization.add_safe_globals` 把需要的类逐个加白名单。

一个完整的 checkpoint 应该是可自解释的 `dict`，全部由张量和基础类型组成：

```
torch.save({
    "model": model.state_dict(),
    "optim": opt.state_dict(),
    "step": step,
    "config": asdict(cfg),      # dataclass → dict, 不要存 dataclass 对象本身
}, path)
```

分布式场景下的 sharded checkpoint（`torch.distributed.checkpoint`）见第 22 章。

面试考点

面试考点. Q1：`torch.jit.trace` 和 `torch.jit.script` 的区别？

A: `trace` 跑一遍记录 `op` 序列, 控制流被固化——`if` 只保留 `trace` 时走的那条分支、`for` 的次数被展开成常量、`.item()` 和 `shape[0]` 变成字面量。实测 `if x.sum() > 0` 的函数 `trace` 出来只剩 `torch.mul(x, c0)`, 喂负输入结果错且不报错。`script` 解析源码保留控制流, 但只支持 `TorchScript` 的静态类型子集, 第三方库基本不可用, 采纳成本高。两者都已进入维护模式。

面试考点. Q2: `torch.export` 和 `torch.compile` 的区别?

A: `compile` 是 JIT 且宽容——运行时编译, 遇到不能 `trace` 的就 `graph break` 回 Python, 产物在进程内、依赖 Python runtime。`export` 是 AOT 且严格——不允许 `graph break`, 遇到数据相关控制流直接报 `GuardOnDataDependentSymNode`, 产出可序列化的 `ExportedProgram` (图 + `graph signature` + `state dict`), 能脱离 Python。两者共享 `Dynamo/AOTAutograd` 的前半段, 所以把模型改到 `fullgraph=True` 能过基本就等于改到能 `export`。

面试考点. Q3: `ExportedProgram` 里有什么?

A: 三部分。图 (ATen 层级的 FX graph, 默认保留 `aten.linear.default` 这样较高层的 `op`, 要彻底分解得调 `run_decompositions()`); `graph signature` (记录每个图输入是 `user input` 还是 `parameter/buffer`, 以及对应的 `state_dict key`——参数被提升成图的输入, 图本身是纯函数); `state dict` (权重)。用 `torch.export.save / load` 序列化成 `.pt2`。

面试考点. Q4: `torch.export` 怎么声明动态 shape?

A: `dynamic_shapes` 参数配 `Dim` 对象, 比如 `{"x": {0: Dim("B", min=1, max=1024)}}`。同一个 `Dim` 用在多处表示这几维必须相等。也可以用 `Dim.AUTO` 让 `export` 自己推断、`Dim.STATIC` 显式声明静态。导出后 `ep.range_constraints` 能看到实际约束, 实测是 `{s77: VR[2, int_oo]}`——下界是 2 不是 1, 因为 `0/1 specialization` 让符号维度隐含 `>= 2`, 要支持 `batch=1` 得单独导一份。

面试考点. Q5: `AOTInductor` 是什么? 什么时候用?

A: 把 `ExportedProgram` 编译成自包含的 `.so / .pt2` 包, 用 C++ `runtime (AOTIModelPackageLoader)` 加载, 完全不需要 Python。API 是 `torch._inductor.aoti_compile_and_package(ep, package_path=...)` 和 `aoti_load_package`。用在三种场景: 已有 C++ 推理服务不想引入 Python 和 GIL; 冷启动敏感 (编译离线做完, 加载是毫秒级, 而 `torch.compile` 首次调用几十秒); 要交付一个二进制而不是一堆 Python 依赖。注意产物绑定 GPU 架构和 `torch` 版本, `sm80` 编的包不能拿去 `sm90` 跑。

面试考点. Q6: ONNX 导出常见失败原因? 新旧路径的区别?

A: 失败原因: 目标 `opset` 里没有对应 `op`; `data-dependent shape` (`nonzero`、`x[mask]`) `ONNX` 的 `shape` 推理表达不了; `data-dependent` 控制流; 动态轴声明不全导致 `batch` 被写死 (这类不报错, 换 `batch` 才暴露)。路径上, 旧 `tracer` 基于 `jit.trace`, 继承丢控制流的问题, 动态轴用 `dynamic_axes` 只是给轴起名; 新路径 (`torch.onnx.export(dynamo=True)`, `torch 2.10` 已是默认) 先走 `torch.export` 再转 `ONNX`, 动态维用 `dynamic_shapes + Dim`, 有真正的符号 `shape` 约束。

面试考点. Q7: `torch.save(model.state_dict())` 和 `torch.save(model)` 有什么区别?

A: `state_dict` 只存张量, 与代码解耦。 `torch.save(model)` 用 pickle 序列化整个对象图, 但只存类的引用路径不存定义——加载时必须能 import 到同名同路径的类, 重构包结构或改类名 checkpoint 就废了; 而且 pickle 能执行任意代码, 加载不可信文件等于执行不可信代码。 torch 2.6 起 `torch.load` 默认只允许张量和少数安全类型, 加载整模型会直接抛 `UnpicklingError`。正确做法是存一个全由张量和基础类型组成的 dict (model / optim / step / config)。

面试考点. Q8: 推理侧除了图优化还会做什么?

A: `model.eval()` (切 dropout 和 BN) `inference_mode()` (比 `no_grad` 更强, 连 version counter 和 view 追踪都关掉) `fuse conv+bn` (BN 在推理时是仿射变换, 可以吸进卷积权重) `KV cache` (自回归解码避免重算历史 K/V, 第 26 章) 量化 (PTQ 校准几百条就能上, QAT 精度更好但要重训, 现在的入口是 `torchao`)。调度层面的 `continuous batching`、`prefix caching` 归推理引擎 (vLLM / SGLang), 不是 PyTorch 这一层的事——说清这个边界比背 API 重要。

torch.distributed 基础：通信原语与进程组

分布式面试的第一关不是 DDP，是“你知道 AllReduce 到底传了多少字节吗”。这一章把 `torch.distributed` 拆成三层：底下是集合通信原语（谁给谁发多少数据）、中间是进程组与 rendezvous（这些进程怎么互相找到）、上面才是 DDP / FSDP / TP 这些并行策略。原语层的通信量公式是所有并行策略成本分析的地基，也是被问到最多的地方——第 18、19、20 章的每一个“通信量 = 几倍参数量”的结论，都是从这一章的几条公式推出来的。

为什么要分布式：两道墙

显存墙。混合精度 AdamW 训练一个 Φ 参数的模型，每卡至少要放这些（ Φ 是参数个数，下面都是字节）：

$$M = \underbrace{2\Phi}_{\text{bf16 参数}} + \underbrace{2\Phi}_{\text{bf16 梯度}} + \underbrace{4\Phi}_{\text{fp32 master}} + \underbrace{4\Phi + 4\Phi}_{\text{Adam } m, v} = 16\Phi$$

7B 模型： $16 \times 7 \times 10^9 = 112 \text{ GB}$ 。A100 只有 80 GB，连一步都跑不了，activation 还没算。这条墙靠 ZeRO / FSDP 拆（第 19 章），或者靠 TP / PP 把模型本身切开（第 20 章）。

时间墙。Transformer 训练的 FLOPs 有个好用的估算式（forward 2、backward 4）： $\text{FLOPs} \approx 6\Phi \cdot N_{\text{tokens}}$ 。7B 模型训 1T token 就是 4.2×10^{22} FLOPs；单张 A100 bf16 峰值 312 TFLOPS、取 50% MFU 得 1.56×10^{14} FLOPS，需要 2.7×10^8 秒 ≈ 8.5 年。1024 卡才压到 3 天。这条墙靠 data parallel 拆。

并行策略全景

策略	切什么	详见
DP (data parallel)	切 batch，每卡持完整模型，梯度 AllReduce	第 18 章
ZeRO / FSDP	DP 的显存优化版：把参数/梯度/优化器状态沿 DP 组切片	第 19 章
TP (tensor parallel)	切单个矩阵乘的权重，一层内部就要通信	第 20 章
PP (pipeline parallel)	按层切成 stage，stage 间只传 activation	第 20 章
CP / SP (context / sequence)	切 sequence 维，长上下文时 activation 装不下才用	第 20 章
EP (expert parallel)	MoE 的 expert 分到不同卡，用 <code>all_to_all</code> 路由 token	仓库的 MoE 书

实际的大规模训练是这些的乘积（“3D / 4D 并行”）：`world_size = dp × tp × pp`。怎么组合、每一维放多大，取决于下面要讲的硬件拓扑。

Key insight. 记住这条分工：DP 通信的是梯度（与 batch 无关，与参数量成正比），TP / CP 通信的是 activation（与 batch·seq 成正比），PP 通信的是 stage 边界的 activation（最省，但有 bubble）。面试里几乎所有“该用哪个并行”的问题都能从这一句推出来。

集合通信原语的精确语义

先统一记号： $N = \text{world size}$ ， $D = \text{这次通信涉及的全量数据字节数}$ 。所有原语都用同一个 D 表示，这样它们的通信量才能直接比。

原语	每卡输入	每卡输出	每卡收发量	典型用途
<code>all_reduce</code>	D	D (全卡相同)	$2(N-1)/N \cdot D$	DDP 梯度同步；求全局 loss
<code>reduce_scatter</code>	D	D/N	$(N-1)/N \cdot D$	FSDP 梯度规约到自己那片
<code>all_gather</code>	D/N	D	$(N-1)/N \cdot D$	FSDP unshard 参数；收集 logits
<code>broadcast</code>	src 有 D	D	$\approx D$	初始参数同步；广播 seed
<code>reduce</code>	D	dst 有 D	$\approx D$	把 metric 聚到 rank 0
<code>all_to_all</code>	D (N 块)	D (N 块)	$(N-1)/N \cdot D$	MoE dispatch；SP 的 activation 重排
<code>gather</code>	D/N	dst 有 D	dst 收 $(N-1)/N \cdot D$	收集每卡的评测结果
<code>scatter</code>	src 有 D	D/N	src 发 $(N-1)/N \cdot D$	从 rank 0 分发数据
<code>barrier</code>	无	无	≈ 0	同步点（写 checkpoint 前后）

几个容易被追问的点：

- `all_reduce` / `reduce` 都是 **in-place**：直接改传入的张量，不返回新张量。`reduce` 只有 `dst` 上的结果有意义，其他 rank 的张量是未定义的中间值——别去读。
- `gather` / `scatter` 的形状不对称：只有 `dst` 需要传 `gather_list`（长度 N 的张量列表），其他 rank 传 `None`。
- `all_to_all` 的语义是“转置”：rank i 的第 j 块发给 rank j ，rank j 收到的第 i 块来自 rank i 。
- `op` 支持 **SUM / AVG / MAX / MIN / PRODUCT**。`ReduceOp.AVG` 由 NCCL 原生支持，比“SUM 完再除”省一次 elementwise kernel。

```
import torch, torch.distributed as dist

# 每 rank 一个 (4,) 张量；world_size = 2
t = torch.full((4,), float(dist.get_rank() + 1), device="cuda")
dist.all_reduce(t) # rank0/1 都得到 [3,3,3,3]
dist.all_reduce(t, op=dist.ReduceOp.AVG) # 都得到 [1.5,...] (NCCL 原生支持)

# 扁平版本比 list 版本快：少一次 stack/split
full = torch.arange(8, device="cuda", dtype=torch.float32) # D = 8 个元素
shard = torch.empty(4, device="cuda")
dist.reduce_scatter_tensor(shard, full) # 每卡拿到求和后的一半

out = torch.empty(8, device="cuda")
dist.all_gather_into_tensor(out, shard) # 拼回全量
```

Note. list 版本的 `all_gather(tensor_list, t) / reduce_scatter(out, input_list)` 要在 Python 侧构造张量列表、NCCL 内部还得拼成连续 buffer。一律优先用 `all_gather_into_tensor / reduce_scatter_tensor`，它们接受扁平张量，也是 FSDP 内部实际调用的入口。

核心恒等式：**AllReduce = ReduceScatter + AllGather**

这是本章最高频的一道题。AllReduce 的语义是“每卡都拿到全体的规约结果”，实现上分两半：

1. **ReduceScatter**：把 D 切成 N 片，rank i 负责规约第 i 片。结束后 rank i 手里是“第 i 片的全局和”。
2. **AllGather**：每个 rank 把自己那片全局和广播给所有人。结束后每卡都有完整结果。

两个阶段各自的每卡收发量都是 $(N-1)/N \cdot D$ ，所以：

$$D_{AR} = D_{RS} + D_{AG} = 2 \frac{N-1}{N} \cdot D \approx 2D$$

Key insight. AllReduce 的每卡通信量 $\approx 2D$ ，与 N 几乎无关（ $N \rightarrow \infty$ 时收敛到 $2D$ ， $N=2$ 时是 D ）。这是 DDP 能扩到上千卡的根本原因，也是“为什么 ZeRO-1/2 通信量和 DDP 一样”的直接推论：ZeRO 只是把 AllReduce 拆回它的两个组成部分，各留一半在不同位置用，总量没变（第 19 章）。

NCCL 的 `all_reduce_perf` 报的 `busbw` 就是按这个系数换算的： $\text{busbw} = D/T \times 2(N-1)/N$ 。所以估时间可以直接写：

$$T_{AR} \approx 2 \frac{N-1}{N} \cdot \frac{D}{B_{\text{bus}}}$$

Ring AllReduce：为什么与 N 无关

朴素做法：所有 rank 把数据发给 rank 0，rank 0 求和再广播。rank 0 要收发 $2(N-1)D$ ，通信量随 N 线性增长，而且它的网卡是唯一瓶颈。这就是 `nn.DataParallel` 的病根（第 18 章）。

Ring 的做法：把 N 个 rank 排成环，每个 rank 只跟左邻居收、右邻居发。

Ring：每卡只与两个邻居通信，带宽全用满

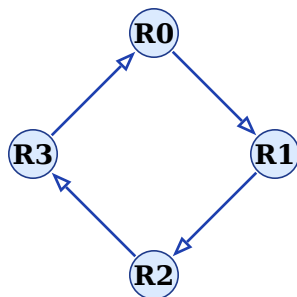


图 25 4 卡 ring。每条边同时在传不同的数据分片，所以 N 张网卡是并行工作的，而不是像 rank-0 聚合那样串行挤在一张卡上。

把数据切成 N 片，跑两个各 $N-1$ 步的阶段：

1. **ReduceScatter**：第 k 步，rank i 把手上第 $(i - k) \bmod N$ 片发给右邻居，同时把左邻居发来的片加到自己对应的片上。 $N - 1$ 步后，rank i 的第 i 片已累加了全部 N 个 rank 的贡献。
 2. **AllGather**：同样绕环 $N - 1$ 步，但这次是覆盖而不是累加，把每片的最终值传遍全环。
- 每步每卡收发 D/N 字节，两阶段共 $2(N - 1)$ 步：

$$D_{\text{ring}} = 2(N - 1) \cdot \frac{D}{N} = 2 \frac{N - 1}{N} \cdot D$$

和上一节的公式完全一致——**Ring** 是通信量最优的 **AllReduce** 实现，达到了理论下界。

为什么必须分两个阶段？因为规约和分发是两件事： $N - 1$ 步只够让每片的和汇聚到某一个 rank（这就是 ReduceScatter 的输出），要让所有 rank 都看到所有片，必须再绕一圈。想省掉第二圈，只能是 ReduceScatter 本身就够用的场景——那正是 ZeRO-2 / FSDP 干的事。

Note. Ring 的代价是延迟： $2(N - 1)$ 步，每步一个网络往返，延迟随 N 线性增长。小消息（几十 KB 级）上延迟项主导，NCCL 会自动切到 **Tree** 算法（延迟 $O(\log N)$ ，通信量略高），或者在有 NVSwitch 的机器上用 **NVLS**（NVLink SHARP，在交换机里做规约）。可以用 `NCCL_ALGO` 强制，但默认自动选择通常是对的。

`init_process_group` 到底做了什么

```
import os, torch, torch.distributed as dist

def setup():
    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)          # 必须在建 PG 之前
    dist.init_process_group(backend="nccl")     # 其余参数从环境变量读
    return local_rank
```

一行 `init_process_group(backend="nccl")` 背后是四件事：

1. 读环境变量（`init_method="env://"` 是默认）：`RANK`（全局唯一编号 $0..N - 1$ ）、`WORLD_SIZE`（ N ）、`MASTER_ADDR`、`MASTER_PORT`。注意 `LOCAL_RANK`（机内编号）不被 `init_process_group` 使用，它是给你自己调 `set_device` 用的。
2. **Rendezvous**：所有进程连到 `MASTER_ADDR:MASTER_PORT` 上的一个 **TCPStore**（rank 0 起 server，其他 rank 当 client）。TCPStore 是个简单的分布式 KV，用来交换“每个 rank 的 NCCL unique id、网卡地址”这些 bootstrap 信息，并做一次 barrier 确认所有 N 个 rank 都到齐。
3. 初始化 **backend**：NCCL 在这一步探测拓扑（谁和谁之间有 NVLink、有几张 IB 网卡）、建 ring / tree、分配通信 buffer。
4. 建默认 **ProcessGroup**：之后所有不带 `group=` 参数的 collective 都走它。

Warning. `init_process_group` 是阻塞的：它会等到所有 `WORLD_SIZE` 个进程都完成 rendezvous 才返回，默认超时 30 分钟（NCCL backend 是 10 分钟）。所以“卡在 `init_process_group` 不动”永远是有 **rank** 没起来或者 **WORLD_SIZE** 设错了，不是 NCCL 的问题。先去数进程个数。

TCPStore 只在初始化和少量元数据交换时用，真正的数据通信不走它——建好之后是 NCCL 直接走 NVLink / IB。所以 `MASTER_ADDR` 的网络质量对训练吞吐没影响。

backend 怎么选

- **nccl** : GPU 训练的唯一选择。走 NVLink / IB , 支持全部原语 , 是唯一能打满带宽的。
- **gloo** : CPU-only 环境、gather_object 这类 CPU 侧集合 , 以及本地调试——没有多卡的机器上用 backend="gloo" + CPU 张量 , torchrun --nproc_per_node=4 起 4 个进程 , collective 语义与 NCCL 完全一致。
- **mpi** : 需要编译时开启 , 只在和已有 MPI 作业集成时用。

一个进程组可以同时挂两个 backend (backend="cpu:gloo,cuda:nccl"), DDP 内部就靠这个在 CPU 上做一些控制流相关的 collective。

torchrun : 谁来设这些环境变量

手写 MASTER_ADDR 和逐个 rank 起进程太痛苦 , torchrun (torch.distributed.run) 就是那个 launcher :

```
# 单机 2 卡 ( 本仓库的环境 )
torchrun --nproc_per_node=2 train.py

# 2 机 × 8 卡 , c10d rendezvous ( 推荐 , 不需要额外服务 )
torchrun --nnodes=2 --nproc_per_node=8 \
    --rdzv_backend=c10d --rdzv_endpoint=node0:29500 \
    --rdzv_id=my_job train.py
```

参数	作用
--nproc_per_node	每机起几个进程。GPU 训练就等于每机 GPU 数 (也可写 gpu 自动取)
--nnodes	机器数。可写 2:4 表示弹性范围 (配合 --max_restarts)
--rdzv_backend=c10d	用 rank 0 上内建的 TCPStore 做 rendezvous。取代老的 --master_addr/--master_port 写法
--rdzv_endpoint	rendezvous 服务地址 host:port。所有节点填同一个
--rdzv_id / --max_restarts	作业标识 ; worker 挂掉后允许整组重启几次 (弹性训练)

torchrun 为每个子进程设的环境变量 : RANK、LOCAL_RANK、WORLD_SIZE、LOCAL_WORLD_SIZE、GROUP_RANK (第几台机器)、MASTER_ADDR、MASTER_PORT、TORCHELASTIC_RUN_ID。你的脚本只需要读 LOCAL_RANK , 其余交给 init_process_group。

Warning. 忘记 `torch.cuda.set_device(local_rank)` 是最高频的坑。不设的话所有进程的 “current device” 都是 `cuda:0` : 8 个进程的模型和 NCCL buffer 全挤在 GPU 0 上 , 表现是 “GPU 0 OOM、其他 7 张卡 0% 利用率” , 或者在 NCCL 初始化时直接 hang (同一张卡上多个 rank 建 ring 会死锁)。

```
# 错
dist.init_process_group("nccl")
model = Model().cuda() # 全去 cuda:0

# 对
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank) # 放在最前面
```

```
dist.init_process_group("nccl")
model = Model().to(f"cuda:{local_rank}")
```

也可以以用 `dist.init_process_group("nccl", device_id=torch.device(f"cuda:{local_rank}"))` 显式绑定，较新版本还能让 NCCL 提前 eager 初始化，把建 communicator 的开销从第一次 collective 挪到这里。

ProcessGroup 与 sub-group

3D 并行需要“只在某几个 rank 之间通信”——TP 组内 AllReduce activation、DP 组内 AllReduce 梯度。这就是 sub-group：

```
# world_size = 8, 想切成 tp=2 × dp=4
rank = dist.get_rank()
tp_groups, dp_groups = [], []

# 关键：所有 rank 都要走完这两个循环，创建全部 group
for i in range(4):                                # tp 组：{0,1} {2,3} {4,5} {6,7}
    g = dist.new_group(ranks=[2 * i, 2 * i + 1])
    tp_groups.append(g)
for i in range(2):                                # dp 组：{0,2,4,6} {1,3,5,7}
    g = dist.new_group(ranks=[i, i + 2, i + 4, i + 6])
    dp_groups.append(g)

my_tp = tp_groups[rank // 2]
my_dp = dp_groups[rank % 2]
dist.all_reduce(grad, group=my_dp)                # 只在 DP 组内规约
```

Warning. `dist.new_group` 必须被所有 rank 以相同顺序调用，哪怕这个 rank 不在 ranks 里。它内部要做一次跨全 world 的元数据交换（分配 group id、建 NCCL communicator），少一个 rank 参与就永久 hang。

```
# 错：只有偶数 rank 调用 → 全体 hang 在这里
if rank % 2 == 0:
    g = dist.new_group(ranks=[0, 2])

# 对：全体调用，各自挑自己的
g02 = dist.new_group(ranks=[0, 2])
g13 = dist.new_group(ranks=[1, 3])
my_group = g02 if rank % 2 == 0 else g13
```

这也是手搓 3D 并行最容易翻车的地方。生产代码用 `init_device_mesh("cuda", (4, 2), mesh_dim_names=("dp", "tp"))` 让 torch 建好全部 sub-group，再用 `mesh["dp"].get_group()` 取（第 21 章）。

另一个易错点：sub-group 里 rank 编号有两套。dist.broadcast(t, src=...) 的 src 是全局 rank；手里只有 group 内下标时改用 group_src= 参数，或 dist.get_global_rank(group, group_rank) 转换。搞混会 broadcast 到错的源，而且不会报错。

同步、异步，以及 **collective** 的调用契约

async_op=True 让 collective 立刻返回一个 work handle：

```
handle = dist.all_reduce(grad, async_op=True)
other_work()           # 与通信 overlap
handle.wait()          # 之后 grad 才可读
```

Key insight. NCCL 的 collective 是在另一条 **CUDA stream** 上排的，所以“同步”和“异步”跟你想的可能不一样：async_op=False 时 wait() 被隐式调用，而 NCCL 的 wait() 做的是 cudaStreamWaitEvent ——给当前 **stream** 插一个依赖，CPU 并不阻塞。真正让 CPU 阻塞要设 TORCH_NCCL_BLOCKING_WAIT=1（调试用）。这就是为什么 DDP 能把 AllReduce 藏进 backward：GPU 侧的依赖关系已经表达清楚了，CPU 只管往下发 kernel。

Collective 的调用契约（这条比任何 API 细节都重要）：

1. 组内每个 rank 都必须调用，一个不落。
2. 调用的顺序必须一致。
3. 每次调用的 **shape**、**dtype**、**op**、**group** 必须一致。

违反任何一条 → hang，或者更糟：数据被静默错配。

Warning. 最常见的违反方式是控制流在不同 rank 上分叉：

```
# 错：不同 rank 的 loss 不同 → 有的 rank 进 if 有的不进 → 集合调用不匹配 → hang
if loss.item() > threshold:
    dist.all_reduce(stats)

# 错：rank 0 多做一次 collective
if rank == 0:
    dist.all_reduce(extra)      # 其他 rank 没有对应调用

# 对：先把判据同步成全卡一致的值
flag = torch.tensor([1.0 if loss.item() > threshold else 0.0], device="cuda")
dist.all_reduce(flag, op=dist.ReduceOp.MAX)
if flag.item() > 0:
    dist.all_reduce(stats)
```

排查手法：设 TORCH_NCCL_ASYNC_ERROR_HANDLING=1 让超时后进程带栈退出而不是永久 hang，再看每个 rank 停在第几个 collective——用 flight recorder 直接 dump 未完成的 collective（第 22 章）。

dist.barrier() 是个纯同步点，没有数据。用在“rank 0 写完文件其他 rank 才能读”这种地方：

```
if rank == 0:
    torch.save(state, "ckpt.pt")
```



```
dist.barrier(device_ids=[local_rank])    # 其他 rank 等 rank 0 写完
state = torch.load("ckpt.pt", map_location=f"cuda:{local_rank}")
```

NCCL 的 `barrier()` 实际是拿一个 dummy 张量做 AllReduce，会用到 current device——所以显式传 `device_ids` 比依赖 `set_device` 更稳。

硬件拓扑决定并行策略怎么摆

链路	量级	说明
NVLink 3.0 (A100 SXM4)	单向 300 GB/s	12 条 link × 25 GB/s；NVIDIA 标称的 600 GB/s 是双向合计
NVSwitch (DGX A100 内 8 卡)	任意两卡同上	全互联，不用担心“哪两张卡更近”
PCIe 4.0 x16	单向 ~32 GB/s	没有 NVLink 的机器（如 A100 PCIe 版）走这条
InfiniBand / RoCE 单网卡	100–400 Gbps = 12.5–50 GB/s	跨机。DGX 级机器有 8 张，聚合后仍与机内差一个数量级

机内比跨机快一个数量级，结论就一句话：通信最密集的并行维度放在机内。TP 每层要做两次 activation AllReduce，必须压在 NVLink 域内（所以 `tp` 一般不超过单机卡数，A100/H100 上就是 8）；DP 的梯度 AllReduce 每 step 只做一次而且能和 backward overlap，跨机是可以接受的。

以 2 机 × 4 卡、`tp=2 × dp=4` 为例（`world_size = 8`，rank 编号按 `tp` 最内层排）：

TP 组：每 2 卡一组，全部在机内 NVLink

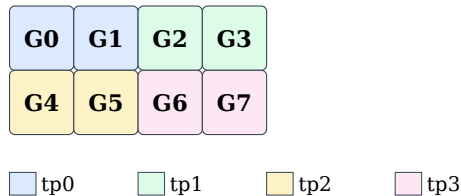


图 26 每行是一台机器。TP 组 {G0,G1} {G2,G3} {G4,G5} {G6,G7} 都不跨行——每层的 activation AllReduce 全走 NVLink。

DP 组：跨机，但每 step 只 AllReduce 一次

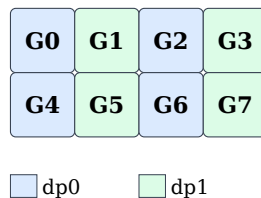


图 27 DP 组 {G0,G2,G4,G6} 和 {G1,G3,G5,G7} 跨了机器边界。梯度 AllReduce 每 step 一次且能与 backward overlap，跨机延迟被藏住。

排 rank 的通用原则：把通信频率最高的维度放在 **rank** 编号的最内层，这样同组的 rank 编号连续、物理上也最近。Megatron 的默认顺序是 `tp` 最内、然后 `cp`、`dp`、`pp` 最外，正是按通信频率从高到低排的。用 `nvidia-smi topo -m` 看每对 GPU 之间实际是 NV12（12 条 NVLink）、PIX（同 PCIe switch）还是 SYS（跨 NUMA），别凭机型猜。

NCCL 环境变量速查

变量	用途
<code>NCCL_DEBUG=INFO</code>	打印拓扑探测、ring 构建、用了哪张网卡。排查一切分布式问题的第一步；太吵就用 <code>NCCL_DEBUG_SUBSYS=INIT,GRAPH</code> 过滤
<code>NCCL_SOCKET_IFNAME=eth0</code>	指定 bootstrap 用的网卡。多网卡机器上不设可能选到 <code>docker0</code> 之类的死网卡然后 hang
<code>NCCL_IB_DISABLE=1</code>	禁用 IB/RoCE 退回 TCP socket。用来确认“是不是 IB 配置的问题”，别在生产开
<code>NCCL_P2P_DISABLE=1</code>	禁用 GPU 间直连 (NVLink / PCIe P2P)，退回经主机内存。同上，诊断用
<code>TORCH_NCCL_ASYNC_ERROR_HANDLING=1</code>	torch 侧 watchdog：collective 超时后主动 abort 进程并打栈，而不是永久 hang。默认值随版本变化，显式设 1 最稳
<code>TORCH_NCCL_BLOCKING_WAIT=1</code>	让 <code>work.wait()</code> 真的阻塞 CPU。定位“哪一行卡住”时用，会掉性能
<code>NCCL_ALGO=Ring</code>	强制算法 (Ring / Tree / NVLS / CollNet)。默认自动选，一般只在对比测试时手动设。同族的 <code>NCCL_PROTO</code> 强制协议 (LL / LL128 / Simple)

Note. `TORCH_NCCL_*` 前缀是 torch 2.2 起的命名 (旧名 `NCCL_ASYNC_ERROR_HANDLING` 等已废弃)。这些是 **PyTorch** 读的变量，不是 NCCL 库读的——所以在 NCCL 文档里查不到。

通信量估算：一套通用方法

任何并行策略的每 step 通信量都可以套这个式子：

$$D_{\text{step}} = \underbrace{\Phi}_{\text{元素个数}} \times \underbrace{b}_{\text{dtype 字节}} \times \underbrace{c}_{\text{策略系数}}$$

c 就是各章要推的那个数：DDP 是 2 (一次 AllReduce)，ZeRO-3 是 3 (两次 AllGather + 一次 ReduceScatter)，TP 是 $4 \times L$ 量级 (每层前后各一次 AllReduce，且乘的不是 Φ 而是 activation 大小)。

具体算一遍：7B 模型、bf16 梯度、DP 组大小 N ：

- 一份梯度 $D = 7 \times 10^9 \times 2 = 14 \text{ GB}$
- $N = 2$ (本仓库环境)： $2 \times (2 - 1) / 2 \times 14 = 14 \text{ GB}$
- $N = 8$ ： $2 \times (8 - 1) / 8 \times 14 = 24.5 \text{ GB}$
- $N = 64$ ： $2 \times (64 - 1) / 64 \times 14 = 27.6 \text{ GB}$

从 8 卡到 64 卡通信量只涨 13%，这就是“**AllReduce** 与 N 无关”的实感。时间上代入 $T \approx D_{\text{AR}} / B_{\text{bus}}$ 。若机内 NVLink 域的 busbw 是 200 GB/s 量级，8 卡的 24.5 GB 约 120 ms；跨机换成单网卡 25 GB/s 量级就要 1 s 量级。所以跨机 **DP** 必须靠 **overlap** 把这一秒藏进 **backward**，藏不住就得上 gradient accumulation 减少 AllReduce 次数 (第 18 章)，或者上 HYBRID_SHARD 把大部分通信压回机内 (第 19 章)。

Warning. 上面的时间是基于假设带宽的量级估算，不是实测。要报数就自己跑 `nccl-tests` (`./build/all_reduce_perf -b 8M -e 2G -f 2 -g 2`) 看 `busbw` 列。别拿标称带宽当实际带宽——小消息受延迟主导，实测值可能只有标称的一半。

更深的推导 (3D 并行的通信量矩阵、bubble 分析、大规模拓扑感知调度) 见仓库的《大模型分布式训练面试通关手册》。

面试考点

面试考点. **Q1** : AllReduce 和 ReduceScatter + AllGather 是什么关系？通信量各是多少？

A : AllReduce 就是这两个的组合。ReduceScatter 把数据切 N 片，让 rank i 拿到第 i 片的全局和，每卡收发 $(N-1)/N \cdot D$ ；AllGather 再把每片广播回全环，同样 $(N-1)/N \cdot D$ 。合起来 $2(N-1)/N \cdot D \approx 2D$ 。这个恒等式直接解释了 ZeRO-1/2 为什么通信量和 DDP 一样：它只是把这两半用在了不同位置。

面试考点. **Q2** : Ring AllReduce 为什么通信量与 world size 几乎无关？

A : 数据切成 N 片，每卡每步只发 D/N ，两阶段共 $2(N-1)$ 步，总量 $2(N-1)/N \cdot D$ ， $N \rightarrow \infty$ 时收敛到 $2D$ 。关键是 N 张网卡是并行工作的——环上每条边同时在传不同的片，不像“都发给 rank 0”那样串行挤在一张卡的带宽上。代价是延迟： $2(N-1)$ 步，延迟随 N 线性增长，所以小消息 NCCL 会切到 Tree ($O(\log N)$ 延迟)。

面试考点. **Q3** : `init_process_group` 做了哪些事？

A : 四件。读环境变量 `RANK` / `WORLD_SIZE` / `MASTER_ADDR` / `MASTER_PORT`；所有 rank 连到 rank 0 起的 TCPStore 做 rendezvous，交换 NCCL unique id 等 bootstrap 信息并 barrier 确认全员到齐；初始化 backend (NCCL 探测 NVLink/IB 拓扑、建 ring)；建默认 ProcessGroup。它是阻塞的，卡住基本都是 rank 没起来或 `WORLD_SIZE` 设错。数据通信不走 TCPStore。

面试考点. **Q4** : 为什么必须 `torch.cuda.set_device(local_rank)`？不做会怎样？

A : 不说的话所有进程的 current device 都是 `cuda:0`，模型、activation、NCCL buffer 全挤在 GPU 0 上——现象是 GPU 0 OOM、其他卡零利用率，或者 NCCL 建 communicator 时直接 hang (同一张卡上多个 rank 建 ring 会死锁)。要在 `init_process_group` 之前调用。注意 `LOCAL_RANK` 是机内编号，`RANK` 是全局编号，`set_device` 用的是前者。

面试考点. **Q5** : 两个 rank 调用 collective 的顺序不一致会发生什么？怎么排查？

A : hang。collective 的契约是“组内每个 rank 都调用、顺序一致、shape/dtype/op/group 一致”，NCCL 按调用顺序配对，顺序错位就永久互等。最常见的诱因是控制流在不同 rank 上分叉 (`if loss.item() > x:`、`if rank == 0:` 里做了 collective)。修法是先把判据 AllReduce 成全卡一致的值再分支。排查用 `TORCH_NCCL_ASYNC_ERROR_HANDLING=1` 让超时后带栈退出，再用 flight recorder dump 未完成的 collective。

面试考点. Q6 : `dist.new_group(ranks=[0,1])` 只需要 rank 0 和 1 调用吗 ?

A : 不。所有 **rank** 都必须调用, 且顺序一致, 哪怕自己不在 `ranks` 里——因为它内部有一次跨全 world 的元数据交换。少一个 rank 参与就永久 hang。这也是手搓 3D 并行 mesh 最容易出错的地方, 生产代码用 `init_device_mesh` 让 torch 建。

面试考点. Q7 : NCCL 和 gloo 怎么选 ?

A : GPU 训练一律 nccl——它走 NVLink / IB, 支持全部原语, 是唯一能打满带宽的选择。gloo 用在三个地方: CPU-only 环境、没有多卡的本地调试 (`gloo` + CPU 可以在单机跑 4 个进程验证分布式逻辑) 以及 `gather_object` 这类需要 CPU 侧序列化的集合。也可以 `backend="cpu:gloo,cuda:nccl"` 同时挂两个。

面试考点. Q8 : 为什么 TP 要放在机内, DP 可以跨机 ?

A 通信频率和通信量的量级差。TP 每层 forward / backward 各要一次 activation AllReduce , 一个 32 层模型一个 step 就是上百次集合通信, 而且通信量与 `batch·seq` 成正比 ; DP 只在 backward 末尾对梯度做一次 AllReduce , 与 batch 无关且能和 backward overlap。机内 NVLink 单向 300 GB/s, 跨机单张 IB 网卡 12.5-50 GB/s, 差一个数量级——所以把通信最密集的维度压进 NVLink 域, `tp` 一般不超过单机卡数。

面试考点. Q9 : `async_op=True` 之后的 `work.wait()` 在等什么 ? 会阻塞 CPU 吗 ?

A : NCCL collective 排在一条独立的 CUDA stream 上, `wait()` 默认做的是 `cudaStreamWaitEvent` ——给当前 stream 插一条依赖, **CPU** 不阻塞, 只是保证后续 kernel 在通信完成后才跑。想让 CPU 真阻塞要设 `TORCH_NCCL_BLOCKING_WAIT=1` (调试用)。这个机制正是 DDP 能把梯度 AllReduce 藏进 backward 计算的基础 : 依赖关系在 GPU 侧表达, CPU 只管继续发 kernel。

DDP：原理、实现与调优

DDP 是分布式面试的必答题，而且面试官问的几乎总是同一个点：梯度 **AllReduce** 怎么和 **backward** 计算重叠。答案是 bucket + backward hook 这一套机制，它把“backward 算完再通信”的串行结构改成了流水线。这一章把 DDP 从数学（为什么是梯度平均）讲到实现（bucket 怎么分、hook 什么时候触发），再到那一长串会让训练 hang 住的坑。上一章的 $D_{AR} \approx 2D$ 是这里所有成本分析的前提。

为什么 `nn.DataParallel` 被废弃

`nn.DataParallel`（DP，注意和“data parallel”这个泛称区分）是单进程多线程的实现，一个 step 干这些事：

1. rank 0 把 batch scatter 到各卡
2. 每卡各起一个 Python 线程跑 forward
3. 输出 gather 回 GPU 0，在 GPU 0 上算 loss
4. backward 的梯度 gather 回 GPU 0，GPU 0 上做 optimizer step
5. 更新后的参数 broadcast 回其他卡

四个致命问题：

- **GIL 争抢**：多个线程在同一个 Python 解释器里发 kernel，CPU 侧的 launch 被 GIL 串行化。模型层数越多、kernel 越小，这个开销越致命。
- **GPU 0 是瓶颈也是热点**：所有 output、loss、梯度都汇聚到 GPU 0，它的显存占用显著高于其他卡（经常是“7 张卡 40 GB，GPU 0 OOM”），算力和带宽也不均衡。
- **每 step 重新 broadcast 参数**：因为只有 GPU 0 更新了参数，别的卡必须重新拿一份。这是纯粹的浪费——DDP 里每卡都能自己更新出相同的参数。
- **无 overlap**，且不能跨机：gather 必须等 backward 全部结束；单进程模型天然只能单机。

DDP 的答案是每张卡一个独立进程：绕开 GIL、原生跨机、用 AllReduce 替代 gather + broadcast，让每卡对称。

Key insight. 一句话概括差异：**DP** 是“一个 master 收发数据”，**DDP** 是“所有卡对称地交换数据”。前者的通信量随卡数线性增长且集中在一张网卡上，后者按上一章的 Ring AllReduce 是 $2(N-1)/N \cdot D$ 且带宽全用满。

DDP 的数学基础

DDP 一个 step 的语义：

1. 全局 batch 按 rank 切开，rank i 拿到 micro-batch B_i （大小 b ）
2. rank i 在 B_i 上算 loss $\mathcal{L}_i$ ，backward 得到 local 梯度 $g_i = \nabla \mathcal{L}_i$
3. 对 g_i 做 AllReduce 求平均，每卡得到相同的 $\bar{g}$
4. 每卡各自跑 optimizer step

为什么是平均而不是求和

因为要和“单卡跑一个 Nb 的大 batch”等价。单卡大 batch 的 loss 通常定义成样本均值：

$$\mathcal{L} = \frac{1}{Nb} \sum_{i=1}^N \sum_{x \in B_i} \ell(x) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i$$

对它求梯度就是 $\bar{g} = \frac{1}{N} \sum_i g_i$ ——正是梯度的算术平均。如果 AllReduce 用 SUM 不除，得到的梯度是正确值的 N 倍，等价于把学习率悄悄放大了 N 倍，训练很快就炸。

DDP 内部自动做这个除法：ReduceOp.SUM 之后除 `world_size`（较新版本直接用 NCCL 原生的 ReduceOp.AVG，省一次 elementwise kernel）。所以你的训练脚本里 **loss** 不需要手动除 **world_size**——它已经是 local batch 的均值了。

Warning. 这个等价性有个前提：每个 **rank** 的 **local batch** 大小必须相同。如果最后一个 batch 各 rank 样本数不等，简单平均就给小 batch 的样本更高的权重了。用 `DistributedSampler` 时它会自动 pad 到整除；用 token 级 loss (LLM 训练里 `sum(loss) / num_tokens`) 时各 rank 的有效 token 数天然不等，严格做法是先 **AllReduce token** 数再归一化，否则 loss 和梯度都有偏。

为什么初始参数必须一致

AllReduce 只同步梯度，不同步参数。只要初始参数一致、梯度一致、optimizer 状态一致，每卡独立 step 就必然算出相同的参数——DDP 全靠这个不变量维持一致性。所以：

- 构造时 **broadcast** 参数：`DDP(model)` 在 `__init__` 里把 rank 0 的参数和 buffer 广播给所有 rank（由 `init_sync=True` 控制，默认开）。这样即使你每个 rank 用了不同的随机种子，也不会跑飞。
- **broadcast_buffers=True**（默认）：每次 forward 前把 rank 0 的 **buffer**（`register_buffer` 注册的东西，如 BatchNorm 的 `running_mean` / `running_var`）广播给所有 rank。buffer 不是参数、没有梯度，不参与 AllReduce，所以只能靠广播保持一致。

Note. 纯 Transformer 模型（LayerNorm / RMSNorm，没有 running stats）通常没有需要同步的 buffer，`broadcast_buffers=False` 可以省掉每 step 一次 broadcast。有 BatchNorm 时不能关。

bucket + backward hook：本章核心

Naive 的实现是“backward 全部跑完，再对所有梯度做一次大 AllReduce”。问题是这两段完全串行：通信期间 GPU 空转，一个 step 的时间是 $T_{\text{compute}} + T_{\text{comm}}$ 。

DDP 的做法是把梯度切成若干 **bucket**，每个 **bucket** 一就绪就立刻发一次 **async AllReduce**，让通信和还没算完的 backward 重叠。

bucket 把 AllReduce 藏进 backward

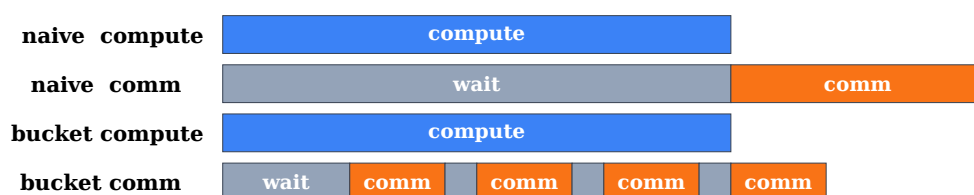


图 28 上两行 naive DDP : backward 结束才开始通信, $\text{step time} = T_{\text{compute}} + T_{\text{comm}}$ 。下两行 bucket DDP : 每个 bucket 就绪就发 async AllReduce, 通信藏在后续 backward 里, 理想情况 $\text{step time} \approx T_{\text{compute}} + \text{最后一个 bucket 的通信时间}$ 。

机制拆开看：

1. 构造时分桶：DDP.__init__ 把所有 requires_grad=True 的参数按顺序装进 bucket, 默认每桶 25 MB (bucket_cap_mb)。同一个 bucket 里的梯度会被拷进一块连续的 buffer, 这样只发一次 AllReduce 而不是每个参数一次。
2. 注册 autograd hook : 给每个参数的 AccumulateGrad 节点挂钩子。参数的梯度算完 → hook 触发 → 把这个梯度标记 ready, 并拷进它所属 bucket 的 buffer。
3. bucket 齐了就发 : 某个 bucket 里所有参数都 ready → 立刻 all_reduce(bucket.buffer, async_op=True), 不等 backward 结束。
4. backward 结束时收 : DDP 在 backward 末尾等所有 pending 的 work handle ,然后把 bucket buffer 里的平均梯度写回各个 p.grad。

为什么按参数的反向顺序分桶

backward 是从 loss 往输入走的, 最后一层的梯度最先算出来。而 model.parameters() 的顺序是前向定义顺序 (第一层在前)。所以 DDP 分桶时把参数列表倒过来 : 第 0 个 bucket 装的是模型最后几层的参数。

这样第 0 个 bucket 在 backward 刚开始几毫秒就齐了, AllReduce 立刻发出去, 有整个 backward 的时间去藏它。如果按正向顺序分, 第 0 个 bucket 要等 backward 走到第一层才齐——那一刻 backward 已经结束, 退化成 naive 实现。

Key insight. “为什么反向顺序”是这道题的题眼 : **overlap** 的可用窗口 = 从这个 bucket 就绪到 backward 结束的时间。让最先算出梯度的参数分在最先发的 bucket ,就是让每个 bucket 的窗口最大化。

bucket 大小的取舍

单次 AllReduce 的时间可以粗略写成“延迟 + 带宽项” : $T \approx \alpha + D/B$ 。 n 个 bucket 就是 $n\alpha + D/B$ 。

- 桶太小 : n 大, $n\alpha$ 主导, 而且每次 AllReduce 都要走一遍 NCCL 的 kernel launch 和同步开销。
- 桶太大 : backward 走到很后面这个桶才齐, 能藏的时间变短; 极端情况一个桶装下整个模型, 就是 naive 实现。

25 MB 是 PyTorch 的通用默认值。7B+ 模型或跨机 (α 大) 时调大到 100–500 MB 通常更好, 但具体值必须实测——用 profiler 看 AllReduce kernel 是不是压在 compute kernel 下面。

```
model = DDP(
    model,
    device_ids=[local_rank],
```



```

bucket_cap_mb=100,
gradient_as_bucket_view=True,      # 省一次拷贝，大模型建议开
broadcast_buffers=False,          # 纯 Transformer 没有需要同步的 buffer
)

```

`gradient_as_bucket_view=True` 让 `p.grad` 直接是 bucket buffer 上的一个 view，而不是独立张量。默认 (`False`) 时每个梯度要从 `p.grad` 拷进 bucket buffer、AllReduce 完再拷回来——开了这个选项就省掉这两次拷贝，同时也省下一份梯度的显存。

Warning. 开了 `gradient_as_bucket_view=True` 之后不要对 `p.grad` 做原地替换 (`p.grad = some_new_tensor`)，那会切断 view 关系让 DDP 拷贝失效。用 `p.grad.copy(...)` 或 `optimizer.zero_grad(set_to_none=True)` (DDP 会在下一次 backward 重建 view)。

find_unused_parameters 的代价

DDP 判断“bucket 齐了”的依据是“这个 bucket 里每个参数的 hook 都触发过”。如果某个参数这一步没参与 forward，它的 hook 永远不触发，那个 bucket 永远不齐，AllReduce 永远不发——其他 rank 在等这次 AllReduce，全体 hang。

`find_unused_parameters=True` 的做法是：backward 一开始就从 loss 的 autograd 图往回遍历一遍，找出哪些参数不在图里，提前把它们标记成 ready。

代价有两块：

- 每 **step** 一次图遍历：纯 CPU 开销，模型越大越明显。
- 破坏 **overlap**：这次遍历要在 backward 起步阶段完成才能确定 bucket 的就绪条件，实践中会推迟第一批 AllReduce 的发出时机，overlap 窗口变窄。

什么时候不得不开：模型里真有条件分支 (`if task == "a": use head_a`)、MoE 只激活部分 expert、多任务模型每 step 只用一个 head。

更好的做法，按优先级排：

1. 改模型让所有参数都参与。MoE 的常见手法是在 aux loss 里加一个乘 0 的项，让所有 expert 权重都在图里 (梯度是 0，但 hook 会触发)。
2. **static_graph=True**：如果“哪些参数参与”在每个 step 都一样 (只是不是全部参与)，用这个。它在第一个 step 记录下参与的参数集合和 backward 的执行顺序，之后每 step 复用，不需要重新遍历。还能顺带优化 activation checkpointing 下的重入问题。
3. 实在动态就只能 `find_unused_parameters=True`。

Warning. `static_graph=True` 和 `find_unused_parameters=True` 不要同时开——`static_graph` 已经涵盖了 `unused` 参数的情况，同时设会被忽略或报错。而且 `static_graph=True` 要求图真的静态：如果 step 之间参与的参数集合会变，行为是未定义的 (表现为梯度错或 hang)。

`no_sync()`：梯度累积时别通信

梯度累积的语义是“累 K 个 micro-batch 的梯度再 step 一次”。累积中间的梯度是不完整的，同步它没有意义——只在最后一个 micro-step 做一次 AllReduce 就够了，因为梯度累加和 AllReduce 都是线性算子，交换顺序不影响结果。

```
from contextlib import nullcontext

ACCUM = 8
optimizer.zero_grad(set_to_none=True)
for i, (x, y) in enumerate(loader):
    is_last = (i % ACCUM == ACCUM - 1)
    # 前 K-1 步进 no_sync: hook 照样触发累加梯度, 但不发 AllReduce
    ctx = nullcontext() if is_last else model.no_sync()
    with ctx:
        loss = criterion(model(x), y) / ACCUM      # 除以 ACCUM 保持梯度是均值
        loss.backward()
    if is_last:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)
```

要点三条：

- `loss / ACCUM` 是必须的，否则梯度是 K 个 micro-batch 的和而不是均值，等于把 LR 放大 K 倍。
- `no_sync()` 省掉 $K - 1$ 次全模型 AllReduce。 $K = 8$ 时通信次数降到 $1/8$ 。
- `clip_grad_norm` 必须在最后一步、**AllReduce** 之后做。累积期间梯度是 partial 的，对它 clip 得到的 norm 没有意义。

$$\text{GBS} = \text{MBS} \times \text{accum} \times N_{\text{dp}}$$

Note. 如果 backward 本来就能完全藏住 AllReduce，`no_sync()` 的收益不大——省下的通信本来就是免费的。它在跨机、通信藏不住的场景才是关键优化。反过来，用了 `no_sync()` 就相当于把 K 步的通信压到一步，那一步的通信更难藏，需要重新调 `bucket_cap_mb`。

DDP 与 BatchNorm

BatchNorm 在每卡上只用本卡的 **local batch** 算 mean / var。8 卡、每卡 MBS=4 时，BN 看到的实际是 batch size 4 的统计量，不是 32——DDP 下 BN 的 **effective batch** 被切小了。batch 小到个位数时 BN 的统计噪声会明显伤害收敛。

`SyncBatchNorm` 让 BN 的统计量跨卡同步：

```
model = torch.nn.SyncBatchNorm.convert_sync_batchnorm(model)  # 必须在包 DDP 之前
model = DDP(model, device_ids=[local_rank])
```

代价：每个 BN 层每次 **forward** 都要做一次 **AllReduce**（同步 $\sum x$ 和 $\sum x^2$ ，可以合并成一次）。ResNet-50 有 53 个 BN 层，就是每 step 多 53 次小消息 AllReduce——小消息受延迟主导，开销比通信量看起来严重得多，而且这些 AllReduce 在 forward 里，没有 **backward** 计算可以拿来 **overlap**。

结论：CV 模型 per-GPU batch 很小时才值得开；LLM 一律用 LayerNorm / RMSNorm，没有这个问题。

DDP + torch.compile

推荐顺序是先 **DDP** 再 **compile**：

```
model = DDP(model, device_ids=[local_rank])
model = torch.compile(model)
```

DDP 的 AllReduce 是靠 autograd hook 触发的，而 Dynamo 默认会把整个 backward 编译成一个大的 fused graph——如果 backward 是一整块，所有梯度是在图末尾一次性产出的，hook 全在同一时刻触发，**bucket** 的渐进就绪性质就消失了，**overlap** 归零。

所以 Inductor 里有个专门的 `ddp_optimizer`（由 `torch._dynamo.config.optimize_ddp` 控制，默认开）：它读取 DDP 的 `bucket_cap_mb`，按 **bucket** 边界把图切成多个子图。每个子图的 backward 一跑完就产出对应 bucket 的全部梯度、触发 AllReduce，overlap 结构被保留下来。代价是引入了 graph break（第 15 章讲了怎么权衡）。

Note. 这也解释了为什么 DDP + compile 的 bucket 数量会影响编译产物：桶越多、切的子图越多、fusion 机会越少。相比 eager，compile 场景下可以适度调大 `bucket_cap_mb` 换更大的子图。

手写一个极简 DDP

白板题常见问法：“不用 `nn.parallel.DistributedDataParallel`，你怎么实现梯度同步？”

```
import os, torch, torch.nn as nn, torch.distributed as dist

class MiniDDP(nn.Module):
    """极简 DDP：参数 broadcast + 每个梯度就绪即 async AllReduce。"""

    def __init__(self, module):
        super().__init__()
        self.module = module
        self.world = dist.get_world_size()
        self._works = []
        # 1) 初始参数对齐：所有 rank 从 rank 0 拿一份
        for p in module.parameters():
            dist.broadcast(p.data, src=0)
        # 2) 每个参数挂 post-accumulate hook，梯度写好后触发
        for p in module.parameters():
            if p.requires_grad:
                p.register_post_accumulate_grad_hook(self._on_grad)

    def _on_grad(self, param):
        param.grad.div_(self.world)  # 先除，再 SUM = 平均
        self._works.append(dist.all_reduce(param.grad, async_op=True))

    def forward(self, *a, **kw):
        return self.module(*a, **kw)
```

```

def finish_backward(self):
    """在 loss.backward() 之后、optimizer.step() 之前调用。"""
    for w in self._works:
        w.wait()
    self._works.clear()

def main():
    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)
    dist.init_process_group("nccl")

    torch.manual_seed(dist.get_rank())    # 故意每卡不同：数据不同，初始化靠 broadcast
    对齐
    model = MiniDDP(nn.Sequential(nn.Linear(64, 64), nn.ReLU(),
                                   nn.Linear(64, 8)).cuda())
    opt = torch.optim.SGD(model.parameters(), lr=0.1)

    for _ in range(5):
        x = torch.randn(16, 64, device="cuda")
        loss = model(x).square().mean()
        opt.zero_grad(set_to_none=True)
        loss.backward()
        model.finish_backward()           # 等梯度 AllReduce 完成
        opt.step()

    # 校验：所有 rank 的参数应完全一致
    w = next(model.module.parameters()).data
    ref = w.clone(); dist.broadcast(ref, src=0)
    assert torch.equal(w, ref), "参数发散了"
    if local_rank == 0:
        print("ok")
    dist.destroy_process_group()

if __name__ == "__main__":
    main()

```

`torchrun --nproc_per_node=2 mini_ddp.py` 运行。这个版本相比真 DDP 缺三样东西，正好是加分回答：没有分桶（每个参数一次 AllReduce，小消息太多）、没有 **buffer** 同步、没有 **unused parameter** 处理。

Note. 用 `register_post_accumulate_grad_hook` (torch 2.1+) 而不是 `p.register_hook`：后者拿到的是“即将被累加的梯度”，在梯度累积场景下会在 `p.grad` 更新之前触发，语义不对。前者保证 `p.grad` 已经写好。

DDP 常见坑清单

Warning. 1. 忘记 `torch.cuda.set_device(local_rank)`。所有进程挤在 `cuda:0`，表现是 GPU 0 OOM 或 NCCL 初始化 hang。第 17 章有详细说明。

2. 忘记 `sampler.set_epoch(epoch)`。 `DistributedSampler` 的 `shuffle` 种子是 `seed + epoch`，不调用就每个 `epoch` 洗出同样的顺序——训练看起来正常，但等价于反复看同一个排列的数据。

```
for epoch in range(E):
    sampler.set_epoch(epoch)      # 少这一行，静默降低数据多样性
    for batch in loader: ...
```

3. 不同 **rank** 走了不同分支 → **collective** 不匹配 → **hang**。典型是 `if loss.item() > thr:` 里做 `log / eval / 存 checkpoint`。判据必须先同步成全卡一致（第 17 章）。

4. 只在 **rank 0** 存 **checkpoint** 但没 **barrier**。rank 0 在写盘（几十秒），其他 rank 已经冲进下一个 step 并发起 `AllReduce` → 超时。存完要 `dist.barrier(device_ids=[local_rank])`。反过来，不要让所有 **rank** 都写同一个路径，会互相截断文件。

5. `state_dict` 的 `module.` 前缀。 `DDP(model).state_dict()` 的 `key` 全带 `module.` 前缀，直接 `load_state_dict` 到裸 `model` 会 `key` 不匹配。存的时候用 `model.module.state_dict()`。

6. 各 **rank** 的 **batch** 数不同 → 最后一步 **hang**。数据集不能被 `world_size × batch_size` 整除时，快的 rank 已经退出循环，慢的 rank 还在 `AllReduce`。三种修法：`DistributedSampler(..., drop_last=True)`（丢尾部数据）、`DataLoader(..., drop_last=True)`、或者用 `model.join()` context（让提前结束的 rank 继续参与 shadow collective）。

```
with model.join():
    for batch in loader:
        loss = model(batch).mean(); loss.backward(); opt.step()
```

7. **eval** 时忘记关梯度同步。 `with torch.no_grad():` 下没有 `backward`，`DDP` 不会通信，但 `broadcast_buffers=True` 时 `forward` 前仍有 `broadcast`。eval 直接用 `model.module` 更干净。

扩展性与大 batch

`DDP` 的通信量 $2(N-1)/N \cdot D$ 与 N 几乎无关，但延迟不是：Ring 有 $2(N-1)$ 步，每步一个网络往返，所以 N 越大 `AllReduce` 的固定延迟越高（`NCCL` 用 Tree 算法把它压到 $O(\log N)$ ）。这是 `DDP` 在千卡规模仍能工作、但边际收益递减的原因。

`DDP` 扩卡是 **weak scaling**：卡数翻倍，`global batch` 也翻倍。GBS 变了就要重调超参：

$$\text{lr} = \text{lr}_{\text{base}} \times \frac{\text{GBS}}{\text{GBS}_{\text{base}}}$$

这是 linear scaling rule (Goyal et al. 2017)：batch 翻倍则梯度的方差减半，可以走更大的步子。两个必须配套的东西：

- **warmup**：训练开头参数变化剧烈，直接用放大后的 LR 会炸。前几百到几千步从很小的 LR 线性升到目标值。

- 上限：linear scaling 在 batch 特别大时失效（ResNet-50 上大约 8k 之后）。LLM 预训练更常用 $lr \propto \sqrt{GBS}$ 或者干脆按经验表查。

大 batch 还有个泛化问题：同样的 **epoch** 数下，大 **batch** 的 **optimizer step** 次数更少，模型倾向收敛到更“尖”的极小值，测试集表现变差。实践中靠 LR warmup + 更长训练 + LARS / LAMB 这类 layer-wise 自适应优化器缓解。更细的推导见仓库的《大模型分布式训练面试通关手册》。

面试考点

面试考点. Q1：DDP 里 backward 和 AllReduce 怎么 overlap？

A：DDP 在构造时把参数按反向顺序分成 bucket（默认 25 MB），给每个参数挂 autograd hook。某个参数的梯度算完就触发 hook 标记 ready 并拷进 bucket buffer；一个 bucket 全 ready 就立刻发 `async_op=True` 的 AllReduce，不等 **backward** 结束。因为 backward 是从最后一层往前算，最先就绪的 bucket 有整个 backward 的时间窗口去藏它的通信。backward 末尾统一 `wait()` 所有 handle。

面试考点. Q2：为什么 bucket 要按参数的反向顺序分？

A：overlap 的可用窗口 = 从 bucket 就绪到 backward 结束的时间。backward 从 loss 往输入走，最后一层的梯度最先出来，所以把最后几层的参数放在第 0 号 bucket，它就在 backward 刚开始时就绪，窗口最大。如果按 `model.parameters()` 的正向顺序分，第 0 号 bucket 要等 backward 走到第一层才齐，那时已经没有计算可以重叠了，退化成 naive 实现。

面试考点. Q3：`nn.DataParallel` 为什么被废弃？

A：四点。单进程多线程被 GIL 卡住 kernel launch；所有 output / loss / 梯度 gather 到 GPU 0，它成为算力、带宽和显存的三重瓶颈（经常只有 GPU 0 OOM）；每 step 要把更新后的参数 broadcast 回其他卡，纯浪费；gather 必须等 backward 全部结束，无法 overlap，且单进程不能跨机。DDP 用“每卡一进程 + 梯度 AllReduce”把这四条全解决了。

面试考点. Q4：DDP 的梯度是平均还是求和？为什么？

A：平均。要和“单卡跑 N 倍大的 batch”等价——大 batch 的 loss 是样本均值， $\mathcal{L} = \frac{1}{N} \sum_i \mathcal{L}_i$ ，求梯度就是各 rank 梯度的算术平均。用 SUM 不除相当于把 LR 放大 N 倍。DDP 内部自动做（`ReduceOp.SUM` 后除 world size，或直接用 NCCL 的 `ReduceOp.AVG`），所以脚本里 loss 不用手动除。前提是各 rank local batch 大小相同；LLM 里按 token 归一化时各 rank token 数不等，严格做法是先 AllReduce token 数。

面试考点. Q5：`bucket_cap_mb` 调大调小分别有什么影响？

A：单次 AllReduce 时间 $\approx \alpha + D/B$ ， n 个桶就是 $n\alpha + D/B$ 。调小 $\rightarrow$ 桶多，延迟项 $n\alpha$ 累积、NCCL launch 开销上升。调大 $\rightarrow$ 单次通信效率高，但要等更多参数就绪，overlap 窗口变窄；极端情况一个桶装完整个模型就是 naive 实现。25 MB 是通用默认；7B+ 或跨机（ α 大）时 100–500 MB 通常更好，但必须用 profiler 实测确认 AllReduce 压在 compute 下面。

面试考点. Q6：`find_unused_parameters=True` 为什么慢？有什么替代？

A : DDP 判断 bucket 就绪的依据是“桶里每个参数的 hook 都触发过”，某参数没参与 forward 就永远不触发、AllReduce 永远不发、全体 hang。这个选项通过每 step 遍历一次 autograd 图找出未参与的参数并提前标 ready——CPU 开销 + 推迟第一批 AllReduce 的发出时机、overlap 变窄。替代方案按优先级：改模型让所有参数都在图里（MoE 常加一个乘 0 的 aux 项）；参与集合固定的话用 `static_graph=True`（第一步记录后复用，不再遍历）；真动态才开它。

面试考点. Q7 : `no_sync()` 做什么？为什么这样是对的？

A : 梯度累积时让前 $K - 1$ 个 micro-step 只累加梯度、不发 AllReduce，只在最后一步同步一次，省掉 $K - 1$ 次全模型 AllReduce。正确性来自“梯度累加和 AllReduce 都是线性算子，交换顺序不影响结果”：先本地累加再 AllReduce 一次，等于每步都 AllReduce 再累加。两个配套点：loss 要除 ACCUM 保持均值语义；`clip_grad_norm_` 只能在最后一步、AllReduce 之后做。

面试考点. Q8 : DDP 下用 BatchNorm 有什么问题？SyncBatchNorm 代价是什么？

A : BN 只用本卡 local batch 算 mean / var，8 卡每卡 MBS=4 时 BN 看到的是 batch 4 而不是 32，统计噪声大伤收敛。`SyncBatchNorm.convert_sync_batchnorm(model)`（要在包 DDP 之前调）让统计量跨卡同步。代价是每个 BN 层每次 **forward** 一次 **AllReduce**：ResNet-50 有 53 个 BN 层就是 53 次小消息通信，受延迟主导，而且在 forward 里没有 **backward** 计算可以拿来 **overlap**。per-GPU batch 很小时才值得。LLM 用 LayerNorm，没这个问题。

面试考点. Q9 : 为什么不同 rank 的 batch 数不同会 hang？怎么修？

A : AllReduce 要求组内所有 rank 都调用。数据集不能被 `world_size × batch_size` 整除时，数据少的 rank 先退出循环，多的 rank 还在发 AllReduce，等不到配对就一直等到超时。修法三种：`DistributedSampler(drop_last=True)` 或 `DataLoader(drop_last=True)` 丢掉尾部数据；或者用 `with model.join():`，让提前结束的 rank 继续参与 shadow collective，直到所有 rank 都跑完。

面试考点. Q10 : DDP 扩到更多卡时，为什么要调学习率？

A : DDP 是 weak scaling——卡数翻倍 global batch 也翻倍。GBS 变大后梯度方差变小，可以走更大的步子，linear scaling rule 说 $lr \propto GBS$ (Goyal 2017)，必须配 warmup（开头几百到几千步线性升 LR，否则放大后的 LR 会在参数剧变阶段炸掉）。这个规则在 batch 特别大时失效，LLM 预训练更常用 $\sqrt{GBS}$ 缩放。另外同 epoch 数下大 batch 的 step 数更少，泛化会变差，需要更长训练或 LARS / LAMB 这类 layer-wise 优化器。

ZeRO 与 FSDP：把训练状态切开

DDP 的假设是“模型能装进一张卡”。7B 模型混合精度训练要 112 GB，A100 只有 80 GB——这个假设一开始就不成立。ZeRO (Rajbhandari 2020) 和它的 PyTorch 原生实现 FSDP 干的事只有一句话：**DDP** 在每张卡上存了 N 份完全相同的参数、梯度和优化器状态，把它们沿 **DP** 组切成 N 片，每卡只存一片。这一章要说清三件事：三级切分各省多少显存、各多付多少通信、以及 FSDP 那几个必须调对的旋钮。上一章的 $2(N-1)/N \cdot D$ 和 “AllReduce = ReduceScatter + AllGather” 是这里全部推导的工具。

动机：DDP 冗余了什么

混合精度 AdamW 训练一个 Φ 参数的模型，每卡的常驻显存（不含 activation）：

$$M_{\text{DDP}} = \underbrace{2\Phi}_{\text{bf16 参数}} + \underbrace{2\Phi}_{\text{bf16 梯度}} + \underbrace{4\Phi}_{\text{fp32 master}} + \underbrace{4\Phi}_{\text{Adam } m} + \underbrace{4\Phi}_{\text{Adam } v} = 16\Phi$$

为什么要 fp32 master weight：bf16 只有 8 位 mantissa，`w += lr * update` 里当 update 比 w 小 3 个数量级以上时加法直接被舍掉，参数就再也动不了。所以 optimizer 在一份 fp32 副本上更新，再 cast 成 bf16 给 forward 用。 m 、 v 同理保 fp32。

7B 模型： $16 \times 7 = 112$ GB。这 112 GB 里，每张卡的内容完全相同——8 卡就是同一份数据存了 8 遍。ZeRO 就是把这份冗余吃掉。

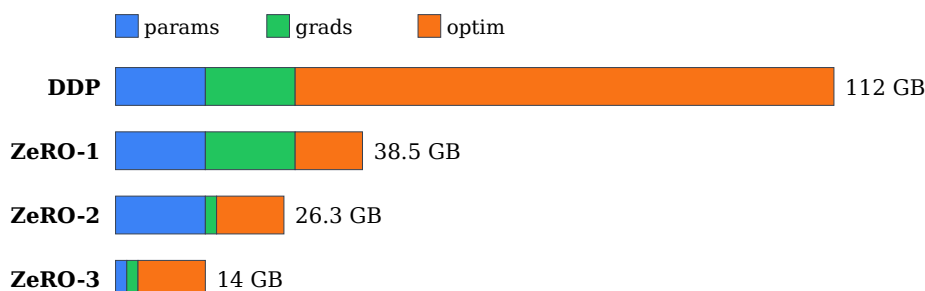


图 29 7B 模型、8 卡、bf16 + AdamW 的每卡常驻显存（不含 activation）。DDP 需要 112 GB 直接 OOM；ZeRO-3 降到 14 GB，activation 反而成了主导项。注意 `optim` 那一段（ $12\Phi = 84$ GB）占了 DDP 的 75%，所以先切它收益最大。

ZeRO 三级：切什么、省多少、多付多少

三级是递进的，每级在前一级基础上再切一样东西：

Stage	优化器状态	梯度	参数	每卡显存	每 step 通信量
DDP	复制	复制	复制	16Φ	$2M$
ZeRO-1	切	复制	复制	$4\Phi + 12\Phi/N$	$2M$
ZeRO-2	切	切	复制	$2\Phi + 14\Phi/N$	$2M$
ZeRO-3	切	切	切	$16\Phi/N$	$3M$

$M = 2\Phi$ 是一份 **bf16** 参数（或梯度）的字节数，7B 时 $M = 14$ GB。通信量都是每卡每 step 的收发量，已经把 $(N-1)/N \approx 1$ 近似掉了。

通信量为什么是这些数

DDP : backward 末尾对全量梯度做一次 AllReduce。上一章的公式给出 $2(N-1)/N \cdot M \approx 2M$ 。

ZeRO-1 / ZeRO-2 :关键在于每张卡只需要自己那片优化器状态对应的梯度,不需要全量梯度。所以把 AllReduce 拆回它的两个组成部分,各用在不同地方:

1. backward 末尾 **ReduceScatter** 梯度: M 。rank i 拿到第 i 片梯度的全局平均。
2. 本地更新自己那片 fp32 master weight, cast 成 bf16。
3. **AllGather** 更新后的参数片: M 。每卡拿回完整的新参数,给下一步 forward 用。

$$D_{\text{ZeRO-1/2}} = \underbrace{M}_{\text{RS 梯度}} + \underbrace{M}_{\text{AG 参数}} = 2M = D_{\text{DDP}}$$

完全等于 **DDP**。区别只是 AllGather 的对象从“梯度”换成了“更新后的参数”,总量分文不差。

ZeRO-3 :参数也切了,所以用的时候必须先拼回来:

1. forward 逐层 **AllGather** 参数:合计 M
2. backward 逐层再 AllGather 一次参数:合计 M
3. backward 末尾 **ReduceScatter** 梯度: M

$$D_{\text{ZeRO-3}} = \underbrace{M}_{\text{fwd AG}} + \underbrace{M}_{\text{bwd AG}} + \underbrace{M}_{\text{RS 梯度}} = 3M = 1.5 \times D_{\text{DDP}}$$

Key insight. **ZeRO-1** 和 **ZeRO-2** 的通信量与 **DDP** 完全相同——显存收益是“免费”的。只有 **ZeRO-3** 多付 50% 通信 ($3M$ vs $2M$),换来的是显存从 $O(1)$ 变成 $O(1/N)$ 。

这是本章最高频的一道题。追问“多 50% 为什么还用”的答案是:你本来就装不下。DDP 跑 70B 根本起不来,多 50% 通信换来能训,这不是权衡,是没得选。反过来,模型明明装得下还上 `FULL_SHARD`,就是白亏 50% 通信。

为什么反向还要 **AllGather** 一次

ZeRO-3 的 forward 是“AllGather 本层参数 → 算 → 立刻 reshard 释放”。释放之后这块显存就被拿去装下一层的参数了。backward 算这一层的 $\partial\mathcal{L}/\partial W$ 和 $\partial\mathcal{L}/\partial x$ 都需要 W 本身,而 W 已经不在了一——只能再 AllGather 一次。

那为什么不留着不释放?留着就等于没切:所有层的参数同时驻留,显存回到 2Φ ,ZeRO-3 退化成为 ZeRO-2。这恰好就是 `SHARD_GRAD_OP` 策略在做的事——它是一个显式的“多花 2Φ 显存、省一次 AllGather”的选项。

FSDP 的执行流程

FSDP 把模型切成若干 **unit** (由 `auto_wrap_policy` 决定,一般一个 Transformer block 一个),每个 unit 的生命周期:

阶段	集合操作	显存变化
forward 进入 unit	<code>all_gather</code> 参数	本 unit 参数从 $2\Phi_u/N$ 涨到 $2\Phi_u$
forward 计算	无	产出 activation (要留给 backward)
forward 退出 unit	无 (reshard)	立刻释放 unshard 的参数,回到 $2\Phi_u/N$

backward 进入 unit	all_gather 参数	再次 unshard
backward 计算	无	算出本 unit 的全量梯度
backward 退出 unit	reduce_scatter 梯度	只留自己那片梯度，释放参数和全量梯度

所以任意时刻 GPU 上只有 1-2 个 unit 的完整参数，显存峰值是 $16\Phi/N + O(2\Phi_u)$ 而不是 $16\Phi/N + 2\Phi$ 。这个 Φ_u （单个 unit 的参数量）就是 auto_wrap_policy 在控制的量。

prefetch 是让通信藏进计算的机制：在算第 l 层的时候就把第 $l+1$ 层的 AllGather 发出去。

FSDP forward：第 $l+1$ 层的 AllGather 与第 l 层计算 overlap



图 30 只有第一层的 AllGather 暴露在关键路径上，之后每层的通信都藏在前一层的计算里。藏得住的前提是单层计算时间 > 单层 AllGather 时间——这就是“unit 不能太小”的量化标准。

Key insight. FSDP 能跑得快，全靠“参数 AllGather 与计算 overlap”。这也给出了 unit 大小的判据：unit 太小则单次 AllGather 消息小、带宽利用率低、次数多，藏不住；unit 太大则一次 unshard 的显存峰值高，而且第一层的暴露延迟变长。一个 **Transformer block** 是被反复验证的甜点。

FSDP 的关键配置

```
import functools, torch, torch.distributed as dist
from torch.distributed.fsd import (
    FullyShardedDataParallel as FSDP,
    ShardingStrategy, MixedPrecision, BackwardPrefetch,
)
from torch.distributed.fsd.wrap import transformer_auto_wrap_policy

model = FSDP(
    model,
    sharding_strategy=ShardingStrategy.FULL_SHARD,           # = ZeRO-3
    auto_wrap_policy=functools.partial(
        transformer_auto_wrap_policy,
        transformer_layer_cls={MyDecoderLayer},             # 按 block 包
    ),
    mixed_precision=MixedPrecision(
        param_dtype=torch.bfloat16,                          # AllGather 出来的参数用 bf16
        reduce_dtype=torch.float32,                          # 梯度规约用 fp32 更稳，见下
        buffer_dtype=torch.bfloat16,
    ),
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
    limit_all_gathers=True,
    use_orig_params=True,
    device_id=torch.cuda.current_device(),
)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True) # 必须在 wrap 之后
```

sharding_strategy

策略	等价于	什么时候用
FULL_SHARD	ZeRO-3	模型装不下。切参数 + 梯度 + 优化器状态，通信 $3M$
SHARD_GRAD_OP	ZeRO-2	参数装得下但优化器状态紧。参数复制、forward 后不 reshard，通信 $2M$
HYBRID_SHARD	机内 ZeRO-3 + 机间 DDP	多机大规模，见下一节
NO_SHARD	DDP	调试对比用；正经跑 DDP 就直接用 DDP
_HYBRID_SHARD_ZER02	机内 ZeRO-2 + 机间 DDP	前缀有下划线，是实验性 API

auto_wrap_policy：最重要的旋钮

不设 `auto_wrap_policy` 是新手最容易犯的错。此时整个模型是一个 **unit**：forward 一开始就 AllGather 全部参数、直到 forward 结束才释放——峰值显存回到 2Φ ，显存基本没降，还白付了通信。

三种包法：

- `transformer_auto_wrap_policy(transformer_layer_cls={MyDecoderLayer})`：按 Transformer block 包。首选。
- `ModuleWrapPolicy({MyDecoderLayer})`：同上，更简洁的新写法。
- `size_based_auto_wrap_policy(min_num_params=1e8)`：按参数量凑。适用于不知道模型结构的通用脚本，但切出的边界可能横穿一个 block，导致通信碎片化。

反面例子：按 `nn.Linear` 包。7B 模型每层有 7 个 Linear，一个 block 就要 7 次 AllGather，每次只有几十 MB——小消息带宽利用率低、NCCL launch 开销累积，而且藏不住。

mixed_precision

三个 dtype 分开设，因为它们的数值需求不同：

- `param_dtype=bf16`：AllGather 出来的参数用什么精度算。用 bf16 直接把参数通信量减半。
- `reduce_dtype`：梯度 ReduceScatter 用什么精度。bf16 只有 7 位 mantissa，大 world size 下累加误差会积累。小规模（百卡以内）bf16 一般够；上千卡建议 fp32，通信量翻倍但数值稳。
- `buffer_dtype`：register_buffer 注册的东西（如 rotary embedding 的 cache）。不设的话 buffer 留在 fp32，会和 bf16 的 activation 类型不匹配报错。

limit_all_gathers 与 prefetch 的取舍

prefetch 越激进，overlap 越好，但同时 **unshard** 的 **unit** 越多、显存峰值越高。这一组参数就是在调这个取舍：

- `limit_all_gathers=True`（默认）：限制同时 in-flight 的 AllGather 数量，靠一个 rate limiter 让 CPU 不要跑得太快导致 caching allocator 一直在为未来的 unshard 预留显存。**OOM** 时确认它是开的。
- `backward_prefetch=BACKWARD_PRE`（默认）：在第 l 层 backward 开始前就发第 $l-1$ 层的 AllGather。overlap 最好、显存峰值最高。`BACKWARD_POST` 更保守，`None` 最省显存最慢。

与 activation checkpointing 组合

FSDP 省的是参数/梯度/优化器状态，**activation** 一点没省——forward 时参数是完整的，activation 大小和单卡一样。所以 ZeRO-3 之后 activation 常常变成新的瓶颈（看回图 29 的 14 GB，一个长序列的 activation 轻松超过它）。

```

from torch.distributed.algorithms._checkpoint.checkpoint_wrapper import (
    apply_activation_checkpointing, checkpoint_wrapper, CheckpointImpl,
)

# 顺序：先 AC，再 FSDP wrap
apply_activation_checkpointing(
    model,
    checkpoint_wrapper_fn=functools.partial(
        checkpoint_wrapper, checkpoint_impl=CheckpointImpl.NO_REENTRANT),
    check_fn=lambda m: isinstance(m, MyDecoderLayer),
)
model = FSDP(model, ...)

```

两个要点：先 **AC** 再 **wrap**，以及 `CheckpointImpl.NO_REENTRANT`（reentrant 版本的 backward 重入行为与 FSDP 的 unshard / reshard 时机配合不好）。recompute 时需要的参数由 FSDP 自动重新 AllGather。

HYBRID_SHARD：把通信压回机内

FULL_SHARD 在 1024 卡上的问题：每层 AllGather 要跨越 128 台机器，走的是 IB（单网卡 12.5–50 GB/s），比 NVLink 慢一个数量级（第 17 章）。而且 N 大到一定程度，每卡那一片参数已经小到 AllGather 全是延迟开销。

HYBRID_SHARD 的做法：机内 **shard**、机间 **replicate**。一台 8 卡机内部做完整的 ZeRO-3，机器之间做 DDP。

```

from torch.distributed.device_mesh import init_device_mesh

n_nodes = dist.get_world_size() // 8
mesh = init_device_mesh("cuda", (n_nodes, 8),
                        mesh_dim_names=("dp_replicate", "dp_shard"))
model = FSDP(model, sharding_strategy=ShardingStrategy.HYBRID_SHARD,
              device_mesh=mesh, auto_wrap_policy=..., use_orig_params=True)

```

代价与收益：每卡显存变成 $16\Phi/8$ 而不是 $16\Phi/1024$ （省得少了），但所有参数 **AllGather** 都在 **NVLink** 域内，跨机只剩梯度的 AllReduce——而且这个 AllReduce 作用在已经 shard 过的梯度上，量只有 $2M/8$ 。大规模训练（Llama 3 405B 就是这个路子）几乎都用它。

FSDP1 vs FSDP2

FSDP1 的核心数据结构是 **FlatParameter**：把一个 unit 里所有参数拼成一个巨大的一维张量再切片。这个设计带来一串麻烦：

- `state_dict` 里看到的是 `FlatParameter`，不是原始参数名，要靠一层复杂的映射还原（`use_orig_params=True` 缓解了这一点，但仍是打补丁）。
- 和 TP 组合别扭：TP 已经把单个权重切了，FSDP1 再拼平再切，两层 sharding 元数据无法统一表达。
- `torch.compile` 不友好：`FlatParameter` 的 view / slice 操作会造成 graph break。
- 逐参数的控制（frozen、per-param LR、不同 dtype）很难做。

FSDP2(`fully_shard` , torch 2.4 起可用 , 2.6+ 进入 `torch.distributed.fsdp` 公开命名空间 , 逐步成为推荐路径) 改成 **per-parameter sharding** : 每个 `nn.Parameter` 独立在 dim-0 上切 , 用 **DTensor** 表达“这是一个逻辑上完整、物理上分片的张量”。

方面	FSDP1	FSDP2
切分单位	FlatParameter (unit 内拼平)	每个 parameter 独立切 dim-0
元数据	自定义 handle	DTensor + DeviceMesh
<code>state_dict</code>	需要 <code>StateDictType</code> 上下文转换	直接就是 DTensor , key 是原始参数名
与 TP 组合	别扭	两层 sharding 都用 DTensor 表达 , 自然叠加 (第 21 章)
<code>torch.compile</code>	容易 graph break	友好
API 形态	包一层 <code>FSDP(model)</code>	原地改造 <code>fully_shard(module)</code> , 不换类型

```
import torch
from torch.distributed.fsdp import fully_shard, MixedPrecisionPolicy
from torch.distributed.device_mesh import init_device_mesh

mesh = init_device_mesh("cuda", (dist.get_world_size(),))
mp = MixedPrecisionPolicy(param_dtype=torch.bfloat16,
                          reduce_dtype=torch.float32)

# 逐 block 调用, 最后包 root — 顺序很重要, root 必须最后
for block in model.layers:
    fully_shard(block, mesh=mesh, mp_policy=mp)
fully_shard(model, mesh=mesh, mp_policy=mp)

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True)
```

三个 API 差异值得记 :

- **fully_shard** 是原地改造 , 返回的还是原来那个 module (类型上多混入了 `FSDPModule`) 。所以没有 `.module` 前缀问题 , `state_dict` 的 key 就是原始名字——这是 FSDP2 最直接的体验改善。
- **auto_wrap_policy** 没了 , 改成你自己循环调用 `fully_shard` 。更显式 , 也更难包错。
- **reshard_after_forward** 取代 `sharding_strategy : True = ZeRO-3` , `False = ZeRO-2` (forward 后不释放) 。默认对 root module 是 `False` (反向马上要用 , 留着省一次 AllGather) , 其他 module 是 `True` 。传一个 2D mesh 就是 HSDP。

梯度累积的等价写法 : FSDP1 用 `model.no_sync()` , FSDP2 用 `model.set_requires_grad_sync(False)` 。

Warning. FSDP1 的 `no_sync()` 在 ZeRO-3 下有个反直觉的代价 : 不做 ReduceScatter 就意味着要保留全量未分片的梯度 , 显存从 $2\Phi/N$ 涨回 2Φ 。所以 FSDP 下的梯度累积通常不用 `no_sync` ——反正 ReduceScatter 本来就和 backward overlap , 每个 micro-step 都做的额外成本远小于多占 2Φ 显存。

checkpoint：分片状态怎么存

FSDP 每卡只有一片参数，`state_dict()` 直接存下来就是“这一片”。两种模式：

- **FULL_STATE_DICT**：所有 rank AllGather 出完整参数，汇聚到 rank 0（通常配 `cpu_offload=True`，否则一张卡装不下）。产物和单卡 checkpoint 格式一致，可以直接给推理用，但存的瞬间 rank 0 的 CPU 内存要能装下整个模型，而且很慢。
- **SHARDED_STATE_DICT**：每卡各存自己那片，产物是“ N 个分片文件 + 一份元数据”。存得快、内存友好，但直接加载要求相同的 world size。

解法是 **DCP**（`torch.distributed.checkpoint`）：它把分片写成带全局坐标元数据的格式，加载时可以 **resharding**——用 8 卡存的 checkpoint 能在 16 卡上恢复。现代做法是统一走 DCP：

```
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint.state_dict import get_state_dict, set_state_dict

model_sd, optim_sd = get_state_dict(model, optimizer)
dcp.save({"model": model_sd, "optim": optim_sd}, checkpoint_id="ckpt/step_1000")
```

完整的存/恢复流程、resharding 细节和常见坑见第 22 章。

ZeRO-Offload / ZeRO-Infinity

思路是把显存压力转移到更慢更大的存储：**ZeRO-Offload** 把优化器状态和 fp32 master weight 放 CPU 内存，optimizer step 在 CPU 上跑（DeepSpeed 有 SIMD 优化的 CPU Adam）；**ZeRO-Infinity** 再往下放到 NVMe。

代价是带宽：A100 的 HBM 有 2 TB/s 量级，PCIe 4.0 x16 只有 32 GB/s 量级，NVMe 更低一个数量级。每步都要把梯度搬下去、把更新后的参数搬上来，瓶颈从算力变成 **PCIe**。所以它的定位很明确：单卡 / 少卡想跑本来跑不了的模型（单张 24 GB 卡 fine-tune 7B），而不是提升吞吐。FSDP 侧的对应物是 `CPUOffload(offload_params=True)`（FSDP1）/ `CPUOffloadPolicy`（FSDP2）。

FSDP 常见坑

Warning. 1. 没设 `auto_wrap_policy`。整个模型成为一个 unit，forward 一开始 AllGather 全部参数，显存基本不降。现象是“上了 FULL_SHARD 还是 OOM”。FSDP2 里对应的错是只调了 root 的 `fully_shard` 而没有逐 block 调。

2. 优化器在 `wrap` 之前创建。 `AdamW(model.parameters())` 拿到的是完整尺寸的参数引用；FSDP wrap 会把参数替换成分片（FSDP1 是 `FlatParameter`，FSDP2 是 `DTensor`），结果是优化器状态按全量分配（显存没省）而且更新写到了游离的旧张量上，模型根本不动。一律 wrap 完再建 optimizer。

3. 用 `torch.nn.utils.clip_grad_norm_` 裁剪 FSDP1 的梯度。全局 norm 是 $\sqrt{\sum_i g_i^2}$ ，需要跨 rank 规约；每卡只有梯度的一片，本地算出来的 norm 是错的（偏小），裁剪系数也就错了。FSDP1 要用它自己的方法 `model.clip_grad_norm_(1.0)`。FSDP2 因为梯度是 `DTensor`，`torch.nn.utils.clip_grad_norm_` 会自动走 `DTensor` 的规约路径，可以直接用。

4. 直接 `print(param)` 或做自定义初始化。分片状态下参数是不完整的。FSDP1 用 `with FSDP.summon_full_params(model):` 临时 AllGather 出完整参数（退出时自动

reshard), 适合做初始化、打印、或者调用不理解 sharding 的第三方代码。FSDP2 用 `model.unshard()` 或对 DTensor 调 `param.full_tensor()`。

5. **buffer_dtype** 没设。buffer 留在 fp32、activation 是 bf16, 某些 op 会报 dtype 不匹配。

6. **AC** 和 **FSDP** 的顺序搞反。先 AC 再 FSDP wrap, 且用 `CheckpointImpl.NO_REENTRANT`。

7. **frozen** 参数和可训练参数混在一个 **unit** 里。LoRA / partial fine-tune 时, 如果一个 unit 内既有 `requires_grad=True` 又有 `False` 的参数, FSDP1 的 FlatParameter 无法表达“只对一部分做 ReduceScatter”。要么把 frozen 部分单独 wrap, 要么用 FSDP2 (per-parameter sharding 天然支持)。

DDP 还是 FSDP : 决策清单

按顺序问自己 :

1. 16Φ 加上 **activation** 能装进一张卡吗? 能 → **DDP**。它最简单、通信最少 ($2M$) 没有 unshard 开销。1B 以下的 dense 模型基本都该用 DDP。
2. 参数装得下, 但优化器状态挤爆了? → **SHARD_GRAD_OP** (ZeRO-2)。通信量还是 $2M$, 显存从 16Φ 降到 $2\Phi + 14\Phi/N$, 是最划算的一档。
3. 参数也装不下? → **FULL_SHARD** (ZeRO-3)。付 50% 额外通信换 $16\Phi/N$ 。
4. 卡数超过一台机器, 跨机 **AllGather** 拖慢了? → **HYBRID_SHARD**, 把参数通信压回 NVLink 域。
5. **activation** 成了新瓶颈? → 叠 activation checkpointing; 序列很长则上 CP / SP (第 20 章)。
6. 单层的权重大到一张卡放不下, 或者要压低单 **token** 延迟? → 这已经超出 data parallel 的范畴, 需要 TP / PP (第 20 章)。

Note. 一个常被忽略的选项: 先试 **ZeRO-2**。很多人一上手就 **FULL_SHARD**, 其实 12Φ 的优化器状态占了 DDP 显存的 75%, 切掉它 (外加梯度) 往往就够了, 而且不花额外通信。

更深的推导 (ZeRO 与 TP 的分工、HSDP 的通信量矩阵、超大规模下的拓扑感知配置) 见仓库的《大模型分布式训练面试通关手册》。

面试考点

面试考点. **Q1** : 混合精度 AdamW 训练每个参数要多少字节? 为什么?

A : 16 字节。bf16 参数 2 + bf16 梯度 2 + fp32 master weight 4 + Adam 的 m 4 + v 4。fp32 master 是必须的: bf16 只有 8 位 mantissa, 当 `lr * update` 比参数本身小 3 个数量级以上时, bf16 加法直接把更新舍掉, 参数就再也动不了。7B 模型就是 112 GB, A100 的 80 GB 一步都跑不了——这是 ZeRO 存在的理由。

面试考点. **Q2** : ZeRO 三级各切什么、显存和通信各是多少?

A : Stage 1 切优化器状态 (12Φ), 每卡 $4\Phi + 12\Phi/N$; Stage 2 再切梯度, $2\Phi + 14\Phi/N$; Stage 3 再切参数, $16\Phi/N$ 。通信量: Stage 1/2 都是 $2M$, 和 **DDP** 完全一样; Stage 3 是 $3M$, 多 50%。所以“先切优化器状态”是最划算的一步——它占了 DDP 显存的 75% 且完全不花额外通信。

面试考点. **Q3** : 为什么 ZeRO-1/2 的通信量和 DDP 一样?

A :因为 AllReduce 本身就是 ReduceScatter + AllGather ,各 M 。DDP 是“RS 梯度 + AG 梯度” ;ZeRO-1/2 是“RS 梯度(每卡只要自己那片)+ 本地更新 + AG 更新后的参数”。AllGather 的对象从梯度换成了参数 ,字节数一样 ,总量都是 $2M$ 。显存收益是免费的。

面试考点. **Q4** : ZeRO-3 通信量多 50% ,为什么还用 ?

A :因为你本来就装不下。DDP 跑 70B 需要每卡 1120 GB ,根本起不来 ;ZeRO-3 把它降到 $16\Phi/N$,64 卡就是 17.5 GB。多 50% 通信换来“能训” ,这不是权衡而是没得选。反过来说 ,模型明明装得下还上 FULL_SHARD 就是白亏 50% 通信——那种情况该用 DDP 或 SHARD_GRAD_OP。

面试考点. **Q5** : FSDP 的 forward 已经 AllGather 过参数了 ,为什么 backward 还要再 AllGather 一次 ?

A :因为 forward 算完就 reshard 释放了 ,那块显存拿去装下一层了。backward 求 $\partial\mathcal{L}/\partial W$ 和 $\partial\mathcal{L}/\partial x$ 都需要 W ,只能重新拼一次。不释放当然可以 ,但那就等于没切——所有层参数同时驻留 ,显存回到 2Φ ,退化成 ZeRO-2。SHARD_GRAD_OP 就是这个显式选项 :多花 2Φ 显存换省一次 AllGather。

面试考点. **Q6** : auto_wrap_policy 为什么必须按 Transformer block 包 ? 包错了会怎样 ?

A :FSDP 的 unshard / reshard 是以 unit 为粒度的 ,unit 大小决定了显存峰值和单次 AllGather 的消息大小。不设 **policy** → 整个模型一个 unit ,forward 开头 AllGather 全部参数直到结束才释放 ,显存基本不降 (最常见的“开了 FULL_SHARD 还是 OOM”)。按 **nn.Linear** 包 → 一个 block 7 次几十 MB 的 AllGather ,小消息带宽利用率低、launch 开销累积、藏不住。一个 block 一个 unit 让单次 AllGather 足够大又不会把峰值拉太高。

面试考点. **Q7** : FSDP2 相比 FSDP1 改了什么 ? 为什么这个改动重要 ?

A :FSDP1 用 FlatParameter——把一个 unit 内所有参数拼成一维大张量再切 ;FSDP2 用 **per-parameter sharding** ,每个 **nn.Parameter** 独立切 dim-0 ,用 DTensor 表达。重要性在四点 :state_dict 的 key 就是原始参数名 (不再需要 StateDictType 转换 ,也没有 module. 前缀) ;和 TP 组合时两层 sharding 都用 DTensor 表达 ,可以自然叠加 ;torch.compile 不再因 FlatParameter 的 view 操作 graph break ;逐参数的 frozen / 不同 dtype / per-param LR 都能做 (LoRA 场景)。API 上 fully_shard 是原地改造而不是包一层。

面试考点. **Q8** : HYBRID_SHARD 解决什么问题 ?

A :FULL_SHARD 在多机上每层 AllGather 都要跨 IB ,比 NVLink 慢一个数量级 ,而且 N 很大时每片参数小到 AllGather 全是延迟。HYBRID_SHARD 改成机内 ZeRO-3、机间 DDP :所有参数 AllGather 都在 NVLink 域内完成 ,跨机只剩梯度 AllReduce ,且作用在已分片的梯度上 (量是 $2M/8$)。代价是显存只降到 $16\Phi/8$ 而不是 $16\Phi/N$ 。大规模训练 (Llama 3 405B) 的标准做法。

面试考点. **Q9** : FSDP 下为什么不能直接用 torch.nn.utils.clip_grad_norm_ ?

A: 全局 norm 是 $\sqrt{\sum_i g_i^2}$, 要跨所有 rank 求和。FSDP1 下每卡只有梯度的一片, 本地算出的 norm 偏小, 裁剪系数就错了——训练不会报错, 只是 clip 失效。FSDP1 要用 `model.clip_grad_norm_(1.0)`, 它内部做跨 rank 规约。FSDP2 下梯度是 DTensor, `torch.nn.utils.clip_grad_norm_` 会走 DTensor 的规约路径, 可以直接用。同理, `summon_full_params(FSDP1)/unshard()` (FSDP2) 用于需要看到完整参数的场景, 比如自定义初始化或调用不理解 sharding 的第三方代码。

面试考点. **Q10**: DDP 和 FSDP 怎么选?

A: 按“装不装得下”决策。 $16\Phi + \text{activation}$ 装得下 $\rightarrow$ DDP, 最简单、通信最少 ($2M$)、没有 unshard 开销。参数装得下但优化器状态紧 $\rightarrow$ SHARD_GRAD_OP (ZeRO-2), 通信还是 $2M$ 但显存降到 $2\Phi + 14\Phi/N$, 性价比最高的一档。参数也装不下 $\rightarrow$ FULL_SHARD, 付 50% 通信换 $16\Phi/N$ 。跨机后 AllGather 拖慢 $\rightarrow$ HYBRID_SHARD。别忘了 FSDP 不省 **activation**, 切完状态之后 activation 常常是新瓶颈, 要叠 activation checkpointing。

张量并行与流水并行

DDP 和 FSDP 切的都是“样本”和“状态”：DP 沿 batch 维复制计算，FSDP 再把参数/梯度/优化器状态沿 DP 组切开。它们从来没有把一层的计算本身切开。当一层已经装不下、或者 batch 小到不能再切时，只剩两条路：把单个 matmul 横向切给多卡（Tensor Parallel），或者把层纵向切成 stage 排流水（Pipeline Parallel）。这一章讲这两种“模型并行”的机制、通信量、以及为什么它们在集群里的摆放位置不能随便定。DTensor 的声明式写法见第 21 章。

什么时候必须超越 FSDP

三个信号，任何一个成立就该考虑 TP / PP：

1. 单层装不下。FSDP 是按 module 做 AllGather 的：forward 到某一层时，这一层的完整参数必须一张卡上物化。 $H = 16K$ 的 MLP 单个权重就是 $16K \times 64K \times 2 = 2 \text{ GB}$ ，加上 fp32 master weight 和优化器状态峰值直接爆。FSDP 无法把“一次 matmul”切开。
2. **batch** 已经小到不能再切。DP 的并行度上限是 global batch size。1024 卡而 global batch 只有 1024 个样本时，`micro_bsz = 1`，再加卡只能靠模型并行。而且 micro batch 太小 GEMM 的 M 维退化，SM 利用率掉。
3. 激活显存是瓶颈。FSDP 不切激活：每张卡跑完整的 (B, S, H) 前向，激活量与单卡训练同阶。TP 把中间激活沿 hidden 维切成 $\frac{1}{T}$ ，PP 让每张卡只持有 $\frac{L}{P}$ 层的激活。

策略	切什么	不切什么	每 step 通信
DDP	batch	参数、梯度、激活	梯度 AllReduce，1 次
FSDP	batch + 参数/梯度/优化器状态	激活、单层计算	每层 AllGather + ReduceScatter
TP	单层的权重与中间激活	batch、层数	每层 2 次 AllReduce (fwd)
PP	层 (stage)	单层计算、batch	stage 边界 P2P

Key insight. 四种并行切的是四个正交的维度，所以可以叠加。选择顺序是固定的：先用 DP/FSDP（通信最轻），装不下再加 TP（限机内），层数太多再加 PP（可跨机）。

Tensor Parallel : MLP 的 column-then-row

以 Transformer 的 MLP 为例。忽略 bias，两层权重是

$$Y = \text{GELU}(XW_1)W_2, \quad W_1 \in \mathbb{R}^{H \times 4H}, \quad W_2 \in \mathbb{R}^{4H \times H}$$

TP 组大小记作 T 。有两种切法，选哪一种、以什么顺序组合，决定了通信次数。

Linear1 : column parallel (沿输出维切)

把 W_1 沿输出维 ($4H$) 切成 T 块 $W_1 = [W_1^{(0)}, \dots, W_1^{(T-1)}]$ ，每块 $H \times 4H/T$ 。输入 X 在 TP 组内是 **replicate** 的，每张卡算

$$Z^{(i)} = XW_1^{(i)} \in \mathbb{R}^{B \times S \times 4H/T}$$

没有任何通信：每张卡拿到输出的一段切片。

W1: column parallel (沿 out 维切给 4 张卡)

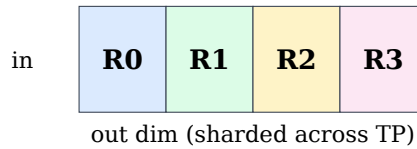


图 31 column parallel : W_1 沿输出维切, 输入 replicate, 输出天然沿 hidden 维 shard。前向零通信。

激活函数：正好能在 **shard** 上直接做

GELU / SiLU 是逐元素的, $\text{GELU}(Z)^{(i)} = \text{GELU}(Z^{(i)})$ ——每张卡对自己手上那 $4H/T$ 列直接做, 结果就是完整结果的对应切片。这就是 **column** 必须在前的原因: column parallel 的输出切分方式恰好与逐元素算子兼容。

Linear2 : row parallel (沿输入维切)

W_2 沿输入维 ($4H$) 切成 T 块, 每块 $4H/T \times H$ 。输入必须已经沿同一维切开——上一步给的正是这个形状。每张卡算

$$Y^{(i)} = \text{GELU}(Z^{(i)})W_2^{(i)} \in \mathbb{R}^{B \times S \times H}$$

每块都是完整形状, 但只是部分和。要拿到 $Y = \sum_i Y^{(i)}$ 必须做一次 **AllReduce**。

W2: row parallel (沿 in 维切给 4 张卡)

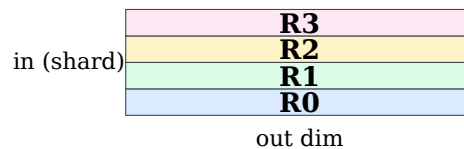


图 32 row parallel : W_2 沿输入维切, 输入必须已按同一维 shard。每卡得到形状完整但数值为部分和的输出, 需要一次 AllReduce 求和。

为什么这个顺序只要一次 **AllReduce**

整条链: X (replicate) $\rightarrow W_1$ column (无通信) $\rightarrow \text{GELU}$ (无通信) $\rightarrow W_2$ row $\rightarrow$ **AllReduce** $\rightarrow Y$ (replicate)。前向一次 AllReduce。

反过来 (row-then-column) 会怎样: W_1 若是 row parallel, 输入 X 得先沿 H 维切开, 输出是部分和 $\rightarrow$ 必须先 AllReduce 才能算 GELU (逐元素算子需要完整的数值, 部分和上做 GELU 是错的: $\text{GELU}(a+b) \neq \text{GELU}(a) + \text{GELU}(b)$); AllReduce 之后 W_2 若是 column parallel, 输出又是 shard 的, 下一层 LayerNorm 需要完整 hidden 才能算 $\rightarrow$ 再 AllReduce 一次。同样的数学结果, 通信次数 $2\times$ 。

Key insight. Megatron-LM 的核心 trick 一句话: 让唯一的非线性算子落在“输出已经切好、且切法与逐元素兼容”的位置上, 这样两个 GEMM 之间不需要同步, 只在最后把部分和合起来。高频题, 能把“GELU 在部分和上不成立”说出来就到位了。

手写一遍: **f** 和 **g** 两个对偶算子

TP 的全部通信可以收进两个自定义 autograd Function, 它们互为对偶: **f** 前向恒等、反向 AllReduce; **g** 前向 AllReduce、反向恒等。


```

# torchrun --nproc-per-node 2 tp_mlp.py    (2x A100, TP=2)
import torch, torch.nn as nn, torch.nn.functional as F
import torch.distributed as dist

class _CopyToTP(torch.autograd.Function):    # f: fwd identity / bwd AllReduce
    @staticmethod
    def forward(ctx, x, group):
        ctx.group = group
        return x

    @staticmethod
    def backward(ctx, g):
        g = g.contiguous()
        dist.all_reduce(g, group=ctx.group)    # 各卡的 dL/dX 求和
        return g, None

class _ReduceFromTP(torch.autograd.Function): # g: fwd AllReduce / bwd identity
    @staticmethod
    def forward(ctx, x, group):
        x = x.contiguous()
        dist.all_reduce(x, group=group)        # 部分和求和
        return x

    @staticmethod
    def backward(ctx, g):
        return g, None                        # 每卡的 dL/dY 本来就相同

```

为什么反向是这两个方向 :前向 replicate 的 X 被 T 张卡各用了一次 ,链式法则要求把 T 份 $\partial L / \partial X$ 加起来 (f 的反向 AllReduce);前向做过 AllReduce 的 Y 在每张卡上数值相同 ,反向传下来的 $\partial L / \partial Y$ 也相同 ,直接透传 (g 的反向恒等)。

```

class ColumnParallelLinear(nn.Module):
    """weight: (out_f // tp, in_f); 输入 replicate, 输出沿 out 维 shard."""
    def __init__(self, in_f, out_f, group):
        super().__init__()
        self.group, tp = group, dist.get_world_size(group)
        assert out_f % tp == 0, "out_features 必须能被 tp 整除"
        self.weight = nn.Parameter(torch.empty(out_f // tp,
in_f).normal_(std=0.02))
        self.bias = nn.Parameter(torch.zeros(out_f // tp))

    def forward(self, x):
        # x: (... , in_f) replicated
        x = _CopyToTP.apply(x, self.group)
        return F.linear(x, self.weight, self.bias)    # (... , out_f // tp)

class RowParallelLinear(nn.Module):
    """weight: (out_f, in_f // tp); 输入已沿 in 维 shard, 输出 replicate."""
    def __init__(self, in_f, out_f, group):
        super().__init__()
        self.group, tp = group, dist.get_world_size(group)
        assert in_f % tp == 0

```



```

self.weight = nn.Parameter(torch.empty(out_f, in_f //
tp).normal_(std=0.02))
self.bias = nn.Parameter(torch.zeros(out_f))

def forward(self, x):
    # x: (... , in_f // tp) sharded
    y = _ReduceFromTP.apply(F.linear(x, self.weight), self.group)
    return y + self.bias
    # bias 必须在 AllReduce 之后加

```

Warning. RowParallelLinear 的 bias 必须在 **AllReduce** 之后加。放在前面就会被加 T 次。这是手写 TP 最经典的静默错误：不报错、shape 全对、loss 也在降，只是慢慢偏掉。

校验方式：单进程构造完整的 `nn.Linear`，把 `weight` 按对应维度切给两张卡，比较 TP 版输出与单卡输出的 `torch.allclose(rtol=1e-5)`，以及两边 `weight.grad` 拼回后是否一致。数值不会逐位相同（求和顺序不同），但应在 $1e-5$ 量级内。

Attention 的 TP：按 head 切

Multi-head attention 天然按 head 分块，因为不同 head 之间没有任何计算耦合：

- `q_proj / k_proj / v_proj` :column parallel(等价于按 head 切),每卡持有 `num_heads / tp` 个 head
- QK^T 、softmax、 PV ：完全本地，不需要通信（这是按 head 切的全部意义）
- `o_proj`：row parallel + 一次 AllReduce

约束：`num_heads % tp == 0`。GQA 还多一条：`num_kv_heads % tp == 0`。Llama-3 的 `num_kv_heads = 8` 就是照着 $TP \leq 8$ 设的；`tp = 16` 时 KV head 不够分，只能让每 2 个 rank 共享一份 KV（复制存储），或者把 TP 调回 8。

Warning. QKV 通常 fuse 成一个 $(3H, H)$ 的权重。切它的时候不能直接按 $3H$ 维均分成 T 段——那会把 Q 的后半和 K 的前半切进同一张卡。必须先 reshape 成 $(3, \text{num_heads}, \text{head_dim}, H)$ ，按 `num_heads` 维切，再 flatten 回去。改 TP 数时加载老 checkpoint 出乱码，一多半是这里错了。

通信量：为什么 **TP** 必须在 **NVLink** 域内

每个 transformer layer 的 AllReduce 次数：

阶段	AllReduce 次数 / 层	张量	触发点
forward	2	(B, S, H)	attention 的 <code>o_proj</code> 之后 + MLP 的 down proj 之后
backward	2	(B, S, H)	两个 column-parallel 入口的 <code>f</code> 反向

一层 4 次， L 层一个 step 就是 $4L$ 次 AllReduce。对比 DDP：整个 step 只有一次梯度 AllReduce（分 bucket 但总量固定）。**TP** 的同步频率是 **DP** 的两三个数量级。

单次 AllReduce 每张卡进出的字节数（ring 算法）：

$$\text{bytes} = 2 \cdot \frac{T-1}{T} \cdot B \cdot S \cdot H \cdot \text{itemsize}$$

代入 $B = 1$ 、 $S = 4096$ 、 $H = 8192$ 、bf16、 $T = 4$ ： $BSH \cdot 2 = 64$ MiB，乘 $2 \times 3/4$ 得约 96 MiB 每次。32 层 $\times$ 4 次 = 128 次，一个 step 搬约 12 GiB。这是数量级估算（假设理想 ring、忽略 kernel 启动与非连续拷贝）。

两个结论直接跟着出来：

1. T 翻倍，通信量几乎不变（ $2(T-1)/T$ 从 1.5 涨到 1.75、1.875），但每卡计算量减半。所以通信/计算比随 T 线性恶化，TP 的扩展性天生有限。
2. 这些 **AllReduce** 全在关键路径上：`o_proj` 的结果 AllReduce 完才能算 LayerNorm，没法像 DDP 梯度那样和反向计算 overlap。

Warning. 铁律： $tp_size \leq$ 单机 NVLink 域内的卡数。跨机做 TP 等于把每层两次全量激活搬上 IB/以太网，step time 会被通信吃掉大半。本书环境是 $2 \times$ A100-SXM4 NVLink 直连，所以 `tp_size` 上限是 2。8 卡 A100/H100 机器上是 8。

Embedding 与 loss

vocab 大（128K+）时这两处是显存与通信的另一个大头。**vocab-parallel embedding**：把 embedding 表沿 vocab 维切成 T 份，rank i 持有 $[v_i, v_{i+1})$ 这段。前向每张卡把落在自己区间外的 token id mask 掉、查表、把 mask 掉的行置零，然后一次 AllReduce 求和——因为每个 token 只会命中一张卡，求和等价于选择。

parallel cross-entropy：logits 是 (B, S, V) ， $V = 128K$ 、 $S = 4096$ 、bf16 时单个张量就是 1 GiB，AllGather 回完整 logits 再算 loss 非常浪费。做法是让 logits 保持沿 vocab 维 shard，只在 TP 组内交换两个标量：每个 token 的 $\max$ （数值稳定用）和 $\sum \exp$ （softmax 分母）。通信量从 $O(BSV)$ 降到 $O(BS)$ 。Megatron 的 `vocab_parallel_cross_entropy` 就是这个；PyTorch 侧对应 `torch.distributed.tensor.parallel.loss_parallel`（配合 DTensor 用，见第 21 章）。

Warning. LayerNorm / RMSNorm 的 weight 在 TP 组内是 **replicate** 的，梯度必须在 TP 组内额外 AllReduce 一次，否则各 rank 的这份参数会慢慢 drift（现象只是 loss 不降，没有任何报错）。dropout 同理：replicate 区间的 mask 必须在 TP 组内一致，shard 区间的 mask 必须不同，所以 seed 要按 `rank // tp` 和 `rank` 分别设。

Sequence Parallel：把 TP 没切到的那部分切掉

TP 只切了 QKV/MLP 中间那段激活。LayerNorm、Dropout、residual 这些区间的张量仍是完整的 (B, S, H) ——每张卡都存一份完整副本，TP 完全没帮上忙。 $B = 1$ 、 $S = 32K$ 、 $H = 8192$ 、bf16 时一层的 LN 输入就是 512 MiB，80 层光这一项就是 40 GB。

Sequence Parallel 的做法：这些区间的张量沿 **sequence** 维切成 $(B, S/T, H)$ 。LayerNorm 沿 hidden 维归一化、逐 token 独立，Dropout 也逐元素，所以在 seq 切片上直接算就是对的，零通信。

通信随之改造：进入 TP 区间前把 $(B, S/T, H)$ AllGather 成 (B, S, H) ；TP 区间出来的部分和直接 ReduceScatter 回 $(B, S/T, H)$ 。而 $\text{AllReduce} = \text{ReduceScatter} + \text{AllGather}$ ，所以：

Key insight. SP 的通信量与纯 TP 完全相同（一次 AR 换成一次 RS + 一次 AG），但把 LayerNorm / Dropout / residual 的激活显存从 BSH 降到 BSH/T 。零通信代价换显存，长序列训练必开。

代价只有实现复杂度：所有跨 token 的计算（loss 分母、grad norm、pooling）都要意识到 seq 已经被切开。PyTorch 侧对应 `SequenceParallel` 这个 `ParallelStyle`，见第 21 章。

Pipeline Parallel：按层切 stage

把 L 层切成 P 段，每段一张卡（或一个 TP 组），stage 之间只在边界传一个激活张量，通信量与层数无关——这是 PP 唯一的优点，也是它能跨机的原因。代价是流水线气泡。把一个 global batch 拆成 M 个 micro-batch 排流水，前向记 t_f 、反向记 t_b ：每个 stage 都要等前面 $P-1$ 个 stage 先把第一个 micro-batch 传过来，也要等后面 $P-1$ 个 stage 把最后一个 micro-batch 的梯度传回来，这段等待是 $(P-1)(t_f + t_b)$ ；总时长是 $(M + P - 1)(t_f + t_b)$ 。

$$\text{bubble} = \frac{(P-1)(t_f + t_b)}{(M + P - 1)(t_f + t_b)} = \frac{P-1}{M + P - 1}$$

$P=4, M=8$ 是 $3/11 \approx 27.3\%$ ， $M=32$ 时是 $3/35 \approx 8.6\%$ ——降 **bubble** 唯一便宜的办法是加大 M 。

Warning. PP 有一件事任何时空图都不会告诉你：**autograd** 图在 **send / recv** 处断开。loss 只在最后一个 stage 上存在，stage 0 上调 `loss.backward()` 是不可能的。每个 stage 必须手工把 recv 到的激活 `detach().requires_grad_(True)` 变成新 leaf，反向时用 `torch.autograd.backward(y, grad_y)` 从下游给的梯度接着往回传，再把 `x.grad` 发给上游。用 `torch.distributed.pipelineing` 时这些由框架代劳，但面试会问。

GPipe：所有 F 完再所有 B

GPipe: P=4, M=8

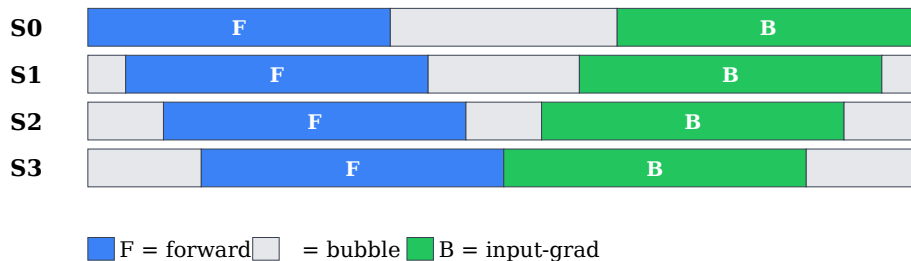
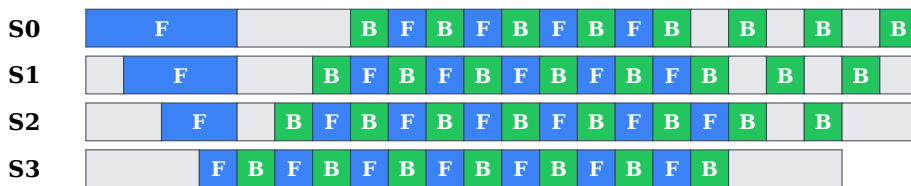


图 33 GPipe：每个 stage 把 8 个 micro-batch 的前向全部做完再开始反向。空闲集中在中间一整块， $\text{bubble} = 3/11 \approx 27.3\%$ 。stage 0 必须同时持有 8 份激活。图中取 $t_f = t_b$ ，每个 B 格是一次完整的 backward（真实比例约 $t_b \approx 2t_f$ ，不影响 bubble 公式）。

问题在显存：stage 0 在第一次反向之前已经跑完 M 个前向， M 份激活全部滞留。于是“加大 M 降 bubble”和“激活爆显存”被绑在了一起。

1F1B：稳定期一前一后

1F1B: $P=4, M=8$



■ F = forward ■ = bubble ■ B = input-grad

图 34 1F1B :warm-up 阶段 stage s 先做 $P-s$ 个前向,之后进入“一前一后”的稳定期,最后 cool-down 排空。总时长与 GPipe 完全相同 (22 个单位, bubble 仍是 27.3%), 空闲只是被打散了。

Key insight. 1F1B 的收益不是 bubble——它和 GPipe 的 bubble 一模一样。收益是激活峰值 :stage s 在第一次反向前只跑了 $P-s$ 个前向,所以峰值从 M 份降到 $P-s$ 份 ($P=4, M=8$ 时 stage 0 是 4 份而不是 8 份)。这解开了“降 bubble”与“爆显存”的耦合,让你敢把 M 开到 $4P$ 以上。这是高频题,答错的人极多。

interleaved 1F1B：每卡持有多个非连续 stage

把模型切成 vP 段,每张卡持有 v 段 (rank r 持有第 $r, r+P, r+2P, \dots$ 段)。micro-batch 在环上绕 v 圈,每段的计算量变成 $1/v$,等效于把流水线拉长而 t_f 变小:

$$\text{bubble}_{\text{interleaved}} = \frac{P-1}{vM+P-1}$$

$P=4, M=8, v=2: 3/19 \approx 15.8\%$,比 $v=1$ 的 27.3% 差不多减半。代价是 P2P 通信次数 $\times v$ (每个 micro-batch 每圈都要过一遍所有 stage 边界),而且激活峰值比 1F1B 更高而不是更低。所以先把 M 加满, M 被 global batch 卡住之后才动 v 。

zero-bubble / DualPipe

一次 backward 其实是两件事:算 $\partial L/\partial X$ (记 B,会阻塞上游,必须尽早做)和算 $\partial L/\partial W$ (记 W,不阻塞任何人)。把 W 拆出来塞进 cool-down 的空隙, bubble 能再降一半左右——这是 zero-bubble 的全部思路。填不满是因为 warm-up 阶段还没有任何梯度回来,手上没有 W 可做。DeepSeek-V3 的 DualPipe 更激进:从流水线两端同时注入 micro-batch, bubble 系数从 $P-1$ 降到 $P/2-1$,代价是 2 份权重副本。

PP 的工程难点

1. $M \geq 4P$ 才划算。 $M=P$ 时 bubble 高达 $(P-1)/(2P-1) \approx 50\%$ 。而 M 的上限是 global batch size 除以 micro batch size,所以 PP 深度受 batch 大小约束。
2. **stage** 负载必须均衡。流水线跑在最慢那一段的速度上,不均衡是每个 micro-batch 都要付的减速,与 bubble 相乘而不是被 bubble 吸收。主要不均衡来源是首尾:embedding 很便宜,而 lm_head 要做 (B, S, V) 的 GEMM 加 cross-entropy, $V=128K$ 时相当于好几个 transformer block。按耗时分层而不是按层数平均分,或者给首尾 stage 少放几层。
3. **P2P** 的死锁。1F1B 稳定期里 stage s 要往下发激活、stage $s+1$ 同时要往上发梯度,两边都“先 send 后 recv”就双向卡死。解法是 `batch_isend_irecv` 把这一步所有 send/recv 先 post 再一起 wait。

4. 只有最后一个 **stage** 有 **loss**。dataloader 只在首尾两端起作用 (stage 0 要 token、末 stage 要 label), 而同一个 PP 组内所有 rank 必须拿到同一个样本。tied embedding (首尾共享) 的梯度还要在首尾两个 rank 之间单独 AllReduce 一次。

PyTorch 原生 `torch.distributed.pipelining` (2.4+)

三个概念：`pipeline()` 把模型按 `split_spec` 切成 stage (基于 `torch.export` 追踪)、`PipelineStage` 描述某一段在拓扑里的位置、`Schedule*` 负责排程并驱动 `step()`。

```
# torchrun --nproc-per-node 2 pp_demo.py      (2 卡 = 2 stage)
import os, torch, torch.nn as nn, torch.distributed as dist
from torch.distributed.pipelining import pipeline, SplitPoint, ScheduleGPIPE

class Toy(nn.Module):
    def __init__(self, h=512, n=4):
        super().__init__()
        self.layers = nn.Sequential(*[nn.Linear(h, h) for _ in range(n)])
    def forward(self, x): return self.layers(x)

dist.init_process_group("nccl")
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)
dev = torch.device("cuda", local_rank)
rank = dist.get_rank()                # 单机时与 local_rank 相同; stage 编号用全局 rank

model = Toy().to(dev)
n_mb = 8                             # micro-batch 数
mb_x = torch.randn(4, 512, device=dev) # 一个 micro-batch 的样例输入

pipe = pipeline(model, mb_args=(mb_x,),
                 split_spec={"layers.2": SplitPoint.BEGINNING})
stage = pipe.build_stage(rank, dev)    # 本 rank 那一段
sched = ScheduleGPIPE(stage, n_microbatches=n_mb, loss_fn=nn.MSELoss())

x = torch.randn(4 * n_mb, 512, device=dev) # 完整 batch, 框架自己切 micro-batch
y = torch.randn(4 * n_mb, 512, device=dev)
opt = torch.optim.AdamW(stage.submod.parameters(), lr=1e-3)
opt.zero_grad()

if rank == 0:
    sched.step(x)                    # 第一个 stage 只喂输入
else:
    losses = []
    out = sched.step(target=y, losses=losses) # 最后一个 stage 提供 label
    print("loss", torch.stack(losses).mean().item())
opt.step()
dist.destroy_process_group()
```

把 `ScheduleGPIPE` 换成 `Schedule1F1B` 就切换排程, 接口不变; `ScheduleInterleaved1F1B` 需要一个 rank 持有多个 stage (传 stage 列表)。

Note. pipelining 的 API 在 2.4 到 2.10 之间仍有调整 (`build_stage` / `PipelineStage` 两条构造路径、`scale_grads` 等参数), 确切签名以对应版本文档为准。稳定的部分是三个概念本身 : `split`、`stage`、`schedule`。生产上 Megatron-LM / torchtitan 的实现更成熟。

组合并行 : 3D / 4D 的维度排布

排布原则只有一条 : 通信最重的维度放在最内层 (**rank** 相邻、同 **NVLink** 域), 最轻的放最外层 (可跨机)。按通信重量排序是 $TP > EP > CP > PP > DP$, 所以标准顺序是

$$\text{rank} = \text{dp} \cdot (P \cdot T) + \text{pp} \cdot T + \text{tp}$$

也就是 TP 变化最快、DP 最慢。给定 world size 16、 $(d, p, t) = (2, 2, 4)$:

world=16, dp=2, pp=2, tp=4

G0	G1	G2	G3
G4	G5	G6	G7
G8	G9	G10	G11
G12	G13	G14	G15

dp=0 dp=1

图 35 每一行 4 张卡是一个 TP 组, 必须落在同一台机器的 NVLink 域内 ($G_0..G_3$ 一台, $G_4..G_7$ 另一台)。上两行是 pp stage 0、下两行是 stage 1。DP 组是跨行取同一列, 例如 $\{G_0, G_4\}$; PP 组是 $\{G_0, G_8\}$, 跨机走 P2P。

具体映射 (`tp` 最快):

rank	dp	pp	tp	所属组
0-3	0	0	0-3	TP 组 $\{0,1,2,3\}$, 同机
4-7	0	1	0-3	TP 组 $\{4,5,6,7\}$, 同机
8-11	1	0	0-3	DP 组 $\{0,8\} / \{1,9\} / \dots$
12-15	1	1	0-3	PP 组 $\{0,4\} / \{8,12\} / \dots$

Warning. 手写这套映射最常见的 bug : pipeline 邻居是 $\text{rank} \pm T$ (这里 ± 4), 不是 ± 1 。而且 `dist.send/recv` 即使传了子进程组, `src/dst` 参数要的仍是全局 rank, 用 `dist.get_global_rank(group, i)` 翻译。算错了如果那个 rank 恰好存在, 表现是挂住而不是报错。

用 `DeviceMesh` 描述同一件事只要一行, 而且不用自己算 rank (见第 21 章):

```
from torch.distributed.device_mesh import init_device_mesh
mesh = init_device_mesh("cuda", (2, 2, 4), mesh_dim_names=("dp", "pp", "tp"))
mesh["tp"]      # 本 rank 所在的 TP 子 group (相邻 4 个 rank)
mesh["dp"]      # 本 rank 所在的 DP 子 group (stride = 8)
```


怎么选并行配置

给定模型大小和集群，按这个顺序走，每一步只在上一步不够时才启用：

1. 先算显存。参数 + 梯度 + Adam 状态在混合精度下约 $16P$ 字节（bf16 参数与梯度各 $2P$ 、fp32 master weight 与两个 moment 各 $4P$ ）。单卡放不下就上 FSDP，看 $16P/N + \text{激活}$ 是否放得下。
2. 能只用 **FSDP** 就只用 **FSDP**。通信最轻、代码最简单、checkpoint 最省事。
3. 单层放不下，或激活显存爆，才加 **TP**，且 `tp_size` 不超过单机卡数（本书环境 2）。优先 $T = 8$ （8 卡机）； $T > 8$ 几乎总是亏的。开 TP 就顺手开 SP。
4. 层数多到 **TP + FSDP** 仍装不下，才加 **PP**，让 PP 跨机。检查 $M \geq 4P$ 是否成立、stage 是否均衡。
5. 剩下的卡全给 **DP/FSDP**（ $\text{dp} = \text{world} / (t \cdot p)$ ），然后跑 10 个 step 看 step time 与显存峰值，用 profiler 看通信占比（见第 11 章）：TP 通信占比超过 20% 说明 T 太大或跨了 NVLink 域。

本书的 $2 \times \text{A100}$ 环境能真跑的只有 $(d, p, t) \in \{(2, 1, 1), (1, 2, 1), (1, 1, 2)\}$ 这几种组合，但概念和调试手段与 512 卡完全一样。

面试考点

面试考点. Q1：Megatron 的 MLP 为什么是 column-then-row？反过来行不行？

A：行是行，但通信翻倍。column parallel 的输出沿 hidden 维 shard，而 GELU 是逐元素的，在 shard 上直接算就对，所以两个 GEMM 之间零通信，只在 row parallel 之后把部分和 AllReduce 一次。反过来的话，row parallel 的输出是部分和，而 $\text{GELU}(a+b) \neq \text{GELU}(a) + \text{GELU}(b)$ ，必须先 AllReduce 才能过激活函数；后面 column parallel 的输出又是 shard 的，下一层 LayerNorm 需要完整 hidden，还得再 AllReduce。语义等价，通信 $2\times$ 。

面试考点. Q2：TP=8 的一个 transformer layer，前向反向一共几次 AllReduce？

A：前向 2 次（attention 的 `o_proj` 之后、MLP 的 `down proj` 之后），反向 2 次（两个 column-parallel 入口处对 $\partial L / \partial X$ 求和），一层共 4 次。32 层就是 128 次，而 DDP 整个 step 只有 1 次梯度 AllReduce——TP 的同步频率高两三个数量级，这就是它必须待在 NVLink 域里的原因。

面试考点. Q3：为什么 `tp_size` 不能超过单机卡数？ T 从 8 加到 16 会发生什么？

A：单次 AllReduce 的字节数是 $2(T-1)/T \cdot BSH \cdot b$ ， T 从 8 到 16 只从 1.75 涨到 1.875 倍——几乎不变，但每卡计算量减半，通信/计算比翻倍。同时这些 AllReduce 在关键路径上无法与计算 overlap。跨机之后带宽掉一个数量级，step time 直接被通信吃掉。所以 `tp_size` $\leq$ NVLink 域大小，实践上 $T \leq 8$ 。

面试考点. Q4：Sequence Parallel 的通信量和纯 TP 一样，那它省了什么？

A：省激活显存。TP 只切了 QKV/MLP 中间那段，LayerNorm / Dropout / residual 区间的张量仍是完整 (B, S, H) 。SP 把这些区间沿 seq 维切成 $(B, S/T, H)$ ——LN 沿 hidden 归一化、逐 token 独立，所以在 seq 切片上零通信直接算。原来的一次 AllReduce 拆成 ReduceScatter + AllGather，二者之和等于 AllReduce，通信量不变，但这部分激活降到 $\frac{1}{T}$ 。 $S = 32\text{K}$ 时能省 GB 级。

面试考点. Q5：GPipe 和 1F1B 的 bubble 分别是多少？1F1B 到底赢在哪？

A : 完全相同, 都是 $(P-1)/(M+P-1)$ ($P=4, M=8$ 时 27.3%)。1F1B 赢的是激活峰值: stage s 在第一次反向前只跑 $P-s$ 个前向, 峰值从 M 份降到 $P-s$ 份。为什么这很重要: 降 bubble 最便宜的手段就是加大 M , 而 GPipe 的显存正比于 M , 把两件事绑死了; 1F1B 解开耦合, 才让 $M \geq 4P$ 变得可行。

面试考点. Q6 : interleaved 1F1B 把 bubble 降到多少? 代价是什么?

A : $(P-1)/(vM+P-1)$, 每卡持有 v 个非连续 stage。 $P=4, M=8, v=2$ 时从 27.3% 降到 15.8%。代价: micro-batch 要在环上绕 v 圈, P2P 通信次数 $\times v$; 激活峰值比普通 1F1B 更高。所以先加 M (不花显存), M 被 global batch 卡住了才动 v 。

面试考点. Q7 : PP 里为什么不能直接 `loss.backward()` ?

A : autograd 图在 `send/recv` 处断成 P 段, `loss` 只在最后一个 stage 上, stage 0 上没有任何从 `loss` 连回自己权重的路径。做法是手工提供每段的起点: 把 `recv` 到的激活 `detach().requires_grad_(True)` 变成新 leaf, 反向时 `recv` 下游给的 `grad_output`, 调 `torch.autograd.backward(y, grad_y)` (不是 `y.backward()`, 非标量会抛异常), 它会同时填好本段的 `weight.grad` 和 `x.grad`, 再把 `x.grad` 发给上游。第一个 stage 的输入不要 `requires_grad_`, 它常是 int64 的 token id。

面试考点. Q8 : PP 的 stage 怎么切? 为什么不能按层数平均分?

A : 按耗时分。embedding 很便宜, `lm_head` 要做 (B, S, V) 的 GEMM 加 cross-entropy, $V=128K$ 时相当于好几个 transformer block。流水线跑在最慢那段的速度上, 所以不均衡是每个 micro-batch 都要付的减速, 与 bubble 相乘而不是被吸收。常见做法: 给首尾 stage 少放几层, 或把 `loss` 单独放一个 virtual chunk。另外大模型不能“先建完整模型再切”, 分层方案必须在分配权重之前就定下来。

面试考点. Q9 : TP、PP、DP 三种并行在集群里怎么摆?

A : 通信最重的放最内层。顺序 `rank = dp*(P*T) + pp*T + tp`, TP 变化最快、落在连续 rank (同 NVLink 域), 因为它每层搬两次完整激活; PP 给最大 stride 让它跨机, 因为它只在 stage 边界传一次激活、而且是 P2P 不占整个 fabric; DP/FSDP 最外层, 它对带宽最宽容且能与反向计算 overlap。所以 pipeline 邻居是 `rank  $\pm T$` 而不是 `$\pm 1$` 。

面试考点. Q10 : 给一个 70B 模型和 64 张 A100-80G, 你怎么配并行?

A : 先算显存: $70B \times 16$ 字节约 1.1 TB 状态, 除以 64 卡是 17.5 GB, FSDP 装得下参数部分。但单层 $H=8192$ 的权重和激活加上通信 buffer 会紧, 且激活不切。我会从 `tp=8` (机内 NVLink, 顺带开 SP 把 LN 激活切掉) + `fsdp=8` 起步, 先不上 PP——64 卡这个规模 PP 的 bubble 和实现复杂度都不值得。跑 10 步看显存峰值和通信占比: TP 通信超过 20% 就把 `tp` 降到 4; 显存还是不够就加 activation checkpointing (比加 PP 便宜得多)。只有层数多到 TP+FSDP 也装不下、且卡数上千时才引入 PP, 并保证 $M \geq 4P$ 。

DeviceMesh 与 DTensor

第 20 章的 TP 是“命令式”的：每个 op 前后自己插 AllReduce，每种切法写一个 module。这套写法能跑，但它把张量的分布信息藏在了类名里——ColumnParallelLinear 的输出是 shard 的、RowParallelLinear 的输入必须是 shard 的，这些约束只存在于文档和你的脑子里，编译器和框架都不知道。DTensor 反过来：把分布信息写进张量本身，让通信由“两个张量的分布不一致”自动推导出来。这是 PyTorch 分布式在 2.x 时代的主线，FSDP2、TP、DCP、torch.compile 全部落在这一层上。

手写并行的痛点

Megatron 风格的代码有四个具体问题：

1. 组合并行时代码爆炸。TP 一套 module，加上 FSDP 又一套包装，再加 PP 又一层。 k 种并行两两交互，要考虑的组合是 $O(k^2)$ 而不是 $O(k)$ 。“这个 tensor 现在是 TP shard 还是 FSDP shard 还是两者都是”，只能靠人脑追。
2. **checkpoint** 与并行度绑死。权重按 `tp=8` 切成 8 份存下来，换成 `tp=4` 就读不回来——因为文件里只有裸的 local tensor，没有“我是第几片、总共几片、沿哪一维切”这些元信息。
3. 每个 **op** 都要手写通信。加一个新算子（比如某种 fused norm）就要重新想一遍它的输入输出该是什么分布、要不要补通信。
4. **torch.compile** 看不懂。手写的 `dist.all_reduce` 对 Dynamo 是一个黑盒副作用，图会在这里断开（见第 12 章的 graph break）

DTensor 要解决的就是“分布信息缺失”这一个根因。

DeviceMesh：给 world 的每一维起名字

DeviceMesh 把 world 的 N 个 rank 组织成一个 n 维网格，每一维起一个名字，然后你就可以按名字取子通信组，不用自己算 rank 了。

```
# torchrun --nproc-per-node 8 mesh_demo.py
import torch
from torch.distributed.device_mesh import init_device_mesh

mesh = init_device_mesh("cuda", (2, 4), mesh_dim_names=("dp", "tp"))
# mesh.mesh 是 arange(8).reshape(2, 4):
#   dp=0 -> [0, 1, 2, 3]
#   dp=1 -> [4, 5, 6, 7]

mesh["tp"]          # 本 rank 所在的 TP 子 mesh (rank 3 -> [0,1,2,3])
mesh["dp"]          # 本 rank 所在的 DP 子 mesh (rank 3 -> [3, 7])
mesh["tp"].get_group() # 拿到底层 ProcessGroup, 可以直接喂 dist.all_reduce
mesh["tp"].get_local_rank() # 本 rank 在 TP 维的坐标
mesh.ndim, mesh.shape # 2, (2, 4)
```

rank 到坐标的映射是行优先：`arange(world).reshape(mesh_shape)`，所以最后一维变化最快，最后一维上相邻的坐标对应连续的 global rank。

Key insight. 连续的 global rank 通常落在同一台机器(同一个 NVLink 域),所以最后一维必须留给通信最重的并行。标准写法是 ("dp", "pp", "tp") 或 ("dp", "tp") ——TP 放最后。把 mesh 写成 ("tp", "dp") 不会报错,也能算出正确结果,但 TP 的 AllReduce 会跨机,step time 差几倍。这是 DeviceMesh 唯一的性能陷阱。

`init_device_mesh("cuda", (2, 4), ("dp", "tp"))`

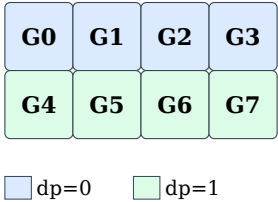


图 36 每一行是一个 TP 组 (rank 连续, 走机内 NVLink); 每一列是一个 DP 组 (stride = 4, 跨机走 IB)。 `mesh["tp"]` 取行、 `mesh["dp"]` 取列。

DTensor : local tensor + mesh + placements

一个 DTensor 逻辑上是一个全局张量,物理上由每个 rank 手里的一块 local tensor 拼成。它携带三样东西 :

- `_local_tensor` : 本 rank 实际持有的那块内存 (用 `to_local()` 取)
- `device_mesh` : 分布在哪个 mesh 上
- `placements` : 一个长度等于 `mesh.ndim` 的元组,逐维说明这个张量沿该 mesh 维怎么分布

三种 placement :

placement	含义	local shape (mesh 维大小 = n)
<code>Shard(d)</code>	沿张量的第 d 维切给这一 mesh 维上的 n 个 rank	第 d 维变成 1/n
<code>Replicate()</code>	每个 rank 持有完整副本,数值相同	与全局 shape 相同
<code>Partial(op)</code>	每个 rank 持有部分和,还没规约;全局值 = 各 rank 按 op 规约的结果	与全局 shape 相同

`Shard` 和 `Replicate` 是直觉的, `Partial` 是这套设计里真正关键的一环。

Key insight. `Partial` 让“需要 AllReduce”这件事变成一个可以延迟兑现的标记。第 20 章的 row-parallel matmul 输出是部分和,手写版本必须立刻 AllReduce 才敢往下传;DTensor 里它只是 `Partial("sum")`,什么时候必须变成 `Replicate` 或 `Shard`,由后面的算子决定。如果下一步正好是 `ReduceScatter` 能满足的分布 (SP 场景),就一次 `ReduceScatter` 解决,省掉一次 `AllGather`;如果连着几个逐元素线性算子,规约还能继续往后推。手写代码做不到这种“通信重排”,因为信息不在张量里。

placement 转换与 collective 的对应关系

这张表是全章的核心。DTensor 不需要为每个算子手写通信,它只需要知道“当前 placement”和“算子要求的 placement”,剩下的是一次查表 :

从	到	collective	为什么
---	---	------------	-----

Shard(i)	Replicate()	AllGather	每个 rank 缺别人的切片，要全部收齐
Replicate()	Shard(i)	无通信（本地切片）	完整数据已在手上，按坐标切一刀丢掉其余
Partial	Replicate()	AllReduce	部分和求和，且结果每个 rank 都要
Partial	Shard(i)	ReduceScatter	求和 + 只保留自己那一段，比 AllReduce 更省
Shard(i)	Shard(j)	AllToAll	沿 i 维收拢、同时沿 j 维散开
Replicate()	Partial	本地缩放（除以 n ）	反向里出现；不是通信

三个能直接背的推论：

1. **AllReduce = ReduceScatter + AllGather** 在这张表里是显式的：Partial $\rightarrow$ Replicate 等于 Partial $\rightarrow$ Shard(i) 再 Shard(i) $\rightarrow$ Replicate。这正是 Sequence Parallel 省显存不加通信的原因（第 20 章）。
2. **Replicate $\rightarrow$ Shard** 是免费的，所以“能 shard 就早点 shard”。
3. **TP** 的一整套通信不需要任何新概念：column-parallel 的输入要 Replicate、输出是 Shard(-1)；row-parallel 的输入要 Shard(-1)、输出是 Partial。把这两条接起来，中间那一步是 Shard(-1) $\rightarrow$ Shard(-1)（无通信），末尾 Partial $\rightarrow$ Replicate 就是那唯一一次 AllReduce。第 20 章手写的 f / g 两个 Function，在 DTensor 里是查表的结果。

matmul 的三条规则就够解释整个 TP

转换表说的是“怎么改分布”，还差一半：算子怎么从输入分布推出输出分布。以 $C = AB$ 为例，只看被切的那一维落在哪：

A 的分布	B 的分布	C 的分布	对应什么
Shard(0)	Replicate	Shard(0)	沿 batch/token 维切，DP 与 SP 的日常
Replicate	Shard(1)	Shard(1)	column parallel：切输出维
Shard(1)	Shard(0)	Partial("sum")	row parallel：切了收缩维，结果是部分和

第三条是全章最值得记的一句：被切的维如果正好是 **matmul** 的收缩维（ A 的列 = B 的行），结果一定是 **Partial**。所有“什么时候需要 AllReduce”的问题都归到这一条——不是因为 TP 有个约定，而是因为求和被分到了不同的卡上。

Note. 反向传播的 placement 是前向的对偶：前向 Replicate 的输入，反向的梯度是 Partial（每个 rank 算出一份部分梯度，需要求和）；前向 Partial $\rightarrow$ Replicate 的输出，反向梯度是 Replicate（直接透传）。这恰好复现了手写版本里 f 反向 AllReduce、 g 反向恒等的结构——不是巧合，是同一条规则的两种写法。

用 DTensor 表达 TP

parallelize_module 接受一个“module 名 $\rightarrow$ ParallelStyle”的字典，把对应 module 的参数换成 DTensor 并注册输入/输出的 placement 转换。

```
# torchrun --nproc-per-node 2 tp_dtensor.py (2x A100, tp=2)
import torch, torch.nn as nn, torch.distributed as dist
from torch.distributed.device_mesh import init_device_mesh
from torch.distributed.tensor.parallel import (
    parallelize_module, ColwiseParallel, RowwiseParallel,
```

```

)

class MLP(nn.Module):
    def __init__(self, h=1024):
        super().__init__()
        self.up = nn.Linear(h, 4 * h)
        self.down = nn.Linear(4 * h, h)
        def forward(self, x): return self.down(torch.nn.functional.gelu(self.up(x)))

dist.init_process_group("nccl")
rank = dist.get_rank()
torch.cuda.set_device(rank)
mesh = init_device_mesh("cuda", (dist.get_world_size(),), mesh_dim_names=("tp",))

model = MLP().cuda()
model = parallelize_module(model, mesh["tp"], {
    "up": ColwiseParallel(),      # weight → Shard(0), 输出 Shard(-1)
    "down": RowwiseParallel(),    # weight → Shard(1), 输入 Shard(-1), 输出
Replicate
})

# up.weight 现在是 DTensor: 全局 shape (4h, h), local shape (4h/tp, h)
w = model.up.weight
print(rank, type(w).__name__, tuple(w.shape), tuple(w.to_local().shape),
w.placements)

x = torch.randn(8, 1024, device="cuda")  # 普通 tensor, 被当作 Replicate 处理
y = model(x)                             # 内部只有一次 AllReduce
y.sum().backward()
dist.destroy_process_group()

```

生成的通信与第 20 章手写版本完全等价：ColwiseParallel 默认 `input_layouts=Replicate()`、`output_layouts=Shard(-1)`；GELU 在 `Shard(-1)` 上逐元素做，不触发转换；RowwiseParallel 期望输入 `Shard(-1)`（正好匹配，无通信），输出 `Partial` 再按 `output_layouts=Replicate()` 落地成一次 AllReduce。区别只是这些通信是查表推出来的，不是你写出来的。

几个常用的 style：

style	参数怎么切	默认 in / out placement
ColwiseParallel	weight 沿 dim 0 (输出维) 切	in Replicate / out Shard(-1)
RowwiseParallel	weight 沿 dim 1 (输入维) 切	in Shard(-1) / out Replicate
SequenceParallel	LayerNorm / RMSNorm 的参数 Replicate	in/out Shard(1) (沿 seq 维)
PrepareModuleInput	不切参数，只转换输入	由 <code>desired_input_layouts</code> 指定
loss_parallel	context manager, 让 CE 在 vocab-shard 的 logits 上算	logits Shard(-1)

开 SP 时把 norm 层标成 `SequenceParallel()`，再用 `ColwiseParallel(input_layouts=Shard(1))` 告诉 attention 入口“输入是沿 seq 切的”——转换

表自动把它变成 AllGather，出口的 `RowwiseParallel(output_layouts=Shard(1))` 自动变成 ReduceScatter。第 20 章那一整段手写 AG/RS 改造，在这里是两个参数。

Warning. `use_local_output=True` (`Colwise/Rowwise` 的默认值)会把模块输出转回普通 Tensor，方便和不感知 DTensor 的代码对接，但也丢掉了分布信息——后面的算子无法再自动推导通信。要串联多个 TP 模块并让 DTensor 全程接管，中间层要传 `use_local_output=False`。`SequenceParallel` 的默认值本来就是 `False`。

FSDP2 就是 mesh 上的 `Shard(0)`

`fully_shard` (`FSDP2`, torch 2.6+ 稳定) 把每个参数换成 `placements=(Shard(0),)` 的 DTensor，前向按 module AllGather 成 `Replicate`、用完再丢掉，梯度以 `Partial -> Shard(0)` (`ReduceScatter`) 落地。所以 ZeRO-3 在这套语言里不是一个特殊机制，只是“参数沿 DP 维 `Shard(0)`”这一个 placement。

于是 TP + FSDP 的组合变成了在 2D mesh 上给参数写两维 placement：

```
mesh = init_device_mesh("cuda", (2, 4), mesh_dim_names=("dp", "tp"))

for layer in model.layers:
    # 先 TP: 沿 tp 维切
    parallelize_module(layer, mesh["tp"], {
        "attn.wq": ColwiseParallel(), "attn.wk": ColwiseParallel(),
        "attn.wv": ColwiseParallel(), "attn.wo": RowwiseParallel(),
        "mlp.up": ColwiseParallel(), "mlp.down": RowwiseParallel(),
    })
    # 再 FSDP: 沿 dp 维切
    fully_shard(layer, mesh=mesh["dp"])
    fully_shard(model, mesh=mesh["dp"])

# attn.wq.weight.placements == (Shard(0), Shard(0))
# 第 0 个是 dp 维 (FSDP 沿参数 dim 0 切), 第 1 个是 tp 维 (Colwise 沿 dim 0 切)
```

`placements` 的第 k 项对应 `mesh` 的第 k 维，两维互不干扰。这就是“组合并行变简单”的全部：不需要为每种组合写新类，只需要多写一维 **placement**。

Note. FSDP2 与 TP 组合必须用 `fully_shard`，老的 FSDP 包装类 (`FSDP1`) 与 TP 混用有已知问题。判断代码是哪一代：`FSDP1` 是 `FSDP(model, ...)` 返回包装对象，`FSDP2` 是 `fully_shard(model)` 原地改造。

手工构造与转换 DTensor

四个 API 覆盖绝大多数用法，语义上的区别是“输入是全局张量还是本地张量”：

```
import torch
from torch.distributed.tensor import (
    DTensor, Shard, Replicate, Partial, distribute_tensor,
)

# 1) distribute_tensor: 输入是【全局】张量，由 src rank 广播/切分下去
big = torch.randn(1024, 512)  # 每个 rank 上都有 (或只有 rank 0 有效)
```

```

dt = distribute_tensor(big, mesh["tp"], [Shard(0)])
dt.shape, dt.to_local().shape # (1024,512) / (256,512) at tp=4

# 2) from_local: 输入是【本地】张量,你声称它已经是某个分布的一片
# 不做任何通信,也不校验(run_check=False 是默认),全局 shape 由 local shape 推出
local = torch.randn(256, 512)
dt2 = DTensor.from_local(local, mesh["tp"], [Shard(0)])
dt2.shape # (1024, 512)

# 3) redistribute: 改 placement,按转换表插 collective
dt3 = dt.redistribute(mesh["tp"], [Replicate()]) # AllGather
dt4 = dt3.redistribute(mesh["tp"], [Shard(1)]) # 本地切片,无通信

# 4) to_local: 拿回本 rank 的那块内存,脱离 DTensor 体系
plain = dt.to_local() # 普通 Tensor,autograd 仍能穿过

```

三个容易踩的语义细节：

- `distribute_tensor` 要求所有 **rank** 传进来的全局张量在逻辑上是同一个（默认 `src_data_rank=0`，从 rank 0 分发），常用于初始化；`from_local` 相反，它信任你手里的 local 片，不通信、不校验，用错了会静默算错。
- `redistribute` 是可微的：反向会自动插入对偶的 `collective`（`AllGather` 的反向是 `ReduceScatter`）。
- `to_local()` 之后你就自己负责了。想让梯度以特定 `placement` 回来，用 `to_local(grad_placements=[...])` 显式声明。

Warning. DTensor 上的所有操作都是 **SPMD** 的：每个 rank 必须执行同一串 DTensor 操作。在 `if rank == 0:` 里 `redistribute` 或者对 DTensor 做 `print(dt)`（会触发 `AllGather`）都会导致部分 rank 参与 `collective`、部分不参与，直接 hang。排查见第 22 章。

收益与代价

收益：

1. 声明式：通信从“手写”变成“由 `placement` 不一致推导”，加新算子不用重新想通信。
2. **checkpoint** 可 **resharding**：DTensor 自带 `mesh` 与 `placement`，`torch.distributed.checkpoint` 存的是“这块内存是全局张量的哪一段”，所以 `tp=8` 存的 checkpoint 能用 `tp=4` 加载（见第 22 章）。这是 DTensor 最实际的好处。
3. 组合并行是加维度而不是加代码： $O(k)$ 而不是 $O(k^2)$ 。
4. **torch.compile** 友好：`collective` 以正规算子的形式进图，Inductor 能把通信与计算重排、`overlap`，而不是遇到 `dist.all_reduce` 就 graph break。

代价：

1. **placement** 推导有 **CPU 侧开销**：每个 op 都要查 sharding 规则、可能构造新的 DTensor 对象。小模型/小 batch 时这部分 Python overhead 可能变得可见，`torch.compile` 能吃掉大部分。
2. 调试栈更深：一个报错要穿过 DTensor dispatch、sharding propagation、DeviceMesh 三层，而分不清全局 shape 与 local shape 是最常见的困惑。
3. 算子覆盖度仍在补齐：自定义算子和冷门算子可能没有 sharding rule，报“operator does not have a sharding strategy”。解法是自己注册规则，或者在这一段 `to_local()` 手写。

调试 DTensor 的第一步固定是把这三样一起打印：

```
def dbg(name, t):
    if isinstance(t, DTensor):
        print(f"[{dist.get_rank()}] {name} global={tuple(t.shape)} "
              f"local={tuple(t.to_local().shape)} {t.placements} {t.device_mesh}")
    else:
        print(f"[{dist.get_rank()}] {name} plain={tuple(t.shape)}")
```

`global` 与 `local` 相同而 `placement` 是 `Shard(...)`，说明某处 `from_local` 的声明和实际数据不符；`placement` 长期停在 `Partial` 说明规约被推迟到了你没预期的位置。

`torch.titan` 是 DTensor path 的官方参考实现（FSDP2 + TP + PP + DCP 的组装方式），要看生产写法直接读它。

面试考点

面试考点. Q1：DeviceMesh 解决什么问题？为什么 TP 要放在 mesh 的最后一维？

A：它把 world 组织成命名的 n 维网格，`mesh["tp"]` 直接给出子通信组，不用手算 `rank = dp*(P*T) + pp*T + tp` 这类映射。`rank` 到坐标是行优先，最后一维变化最快，也就是最后一维上相邻的坐标对应连续的 **global rank**、通常在同一台机器的 NVLink 域内。TP 每层要搬两次完整激活，通信最重，必须落在这里。写成 `("tp", "dp")` 不报错但 TP 会跨机，慢几倍。

面试考点. Q2：DTensor 由哪几部分组成？`placements` 的长度是多少？

A：`local tensor + device mesh + placements`。`placements` 的长度等于 `mesh.ndim`，第 k 项描述这个张量沿 mesh 第 k 维怎么分布。所以 2D mesh 上 TP + FSDP 的参数是 `(Shard(0), Shard(0))`——第一项是 FSDP 沿 `dp` 维切，第二项是 Colwise 沿 `tp` 维切，互不干扰。

面试考点. Q3：Partial 是什么？为什么它是关键？

A：`Partial(op)` 表示每个 rank 手上是部分和，全局值等于各 rank 按 `op` 规约后的结果。它把“这里需要一次 AllReduce”从“必须立刻执行的动作”变成“可以延迟兑现的标记”。于是 `row-parallel matmul` 的输出可以先带着 `Partial` 往下走，等真正需要完整数值时才规约；如果下游要的是 `seq-shard`，一次 `ReduceScatter` 就够，省掉 `AllGather`。手写代码做不到这种重排，因为分布信息不在张量里。

面试考点. Q4：写出 placement 转换与 collective 的对应关系。

A：`Shard(i) -> Replicate` 是 `AllGather`；`Replicate -> Shard(i)` 是本地切片、无通信；`Partial -> Replicate` 是 `AllReduce`；`Partial -> Shard(i)` 是 `ReduceScatter`；`Shard(i) -> Shard(j)` 是 `AllToAll`。DTensor 就是靠这张表把“当前 placement”和“算子要求的 placement”之间的差异翻译成 collective。顺带能看出 `AllReduce = ReduceScatter + AllGather`。

面试考点. **Q5** : `ColwiseParallel` + `RowwiseParallel` 生成的通信和 Megatron 手写的一样吗 ?

A : 一样。Colwise 的输入要 `Replicate`、输出 `Shard(-1)` ; 逐元素激活在 `Shard(-1)` 上不触发转换 ; Rowwise 期望输入 `Shard(-1)` (正好匹配, 零通信), 输出是 `Partial` , 落地成 `Replicate` 时就是那唯一一次 `AllReduce`。前向一次、反向一次, 与手写的 `f / g` 完全对应。区别只在于通信是查表推出来的还是人写的。

面试考点. **Q6** : `distribute_tensor` 和 `DTensor.from_local` 有什么区别 ?

A : `distribute_tensor` 的输入是全局张量, 默认从 `src_data_rank=0` 分发并按 placement 切开, 有通信 ; `from_local` 的输入是本地片, 你声称它已经是某个分布的一片, 不通信也不校验 (`run_check` 默认 `False`), 全局 shape 由 local shape 与 mesh 推出。前者用于初始化/加载, 后者用于把已有的手写并行代码接进 `DTensor`。 `from_local` 声明错了不会报错, 会静默算错。

面试考点. **Q7** : FSDP2 和 `DTensor` 什么关系 ? 为什么它让“组合并行”变简单了 ?

A : `fully_shard` 就是把参数变成 mesh 上 `Shard(0)` 的 `DTensor` , 前向 `AllGather` 成 `Replicate` 用完即丢, 梯度 `Partial -> Shard(0)` 即 `ReduceScatter`。所以 ZeRO-3 不是特殊机制, 只是一个 placement。TP + FSDP 于是变成 2D mesh 上的两维 placement (`Shard(0)`, `Shard(0)`) ——不需要为每种组合写新类, 只需要多写一维。注意必须用 FSDP2 的 `fully_shard` , FSDP1 与 TP 混用有已知问题。

面试考点. **Q8** : `DTensor` 的代价是什么 ? 什么时候不该用 ?

A : 三点。placement 推导有 CPU 侧开销, 小模型小 batch 时可能可见 (`torch.compile` 能吃掉大部分); 调试栈更深, 报错要穿过 `dispatch / sharding propagation / mesh` 三层, 看 shape 时要分清全局还是 local ; 算子覆盖度仍在补齐, 自定义算子可能没有 sharding rule, 需要自己注册或局部 `to_local()` 手写。如果只是单机 DDP 或纯 FSDP 且不打算改并行度, 用不上 `DTensor` 的表达力 ; 一旦要组合两种以上并行、或者要求 checkpoint 能换并行度恢复, `DTensor` 就是当前最省事的路。

分布式 debug、checkpoint 与容错

前面几章讲的是“怎么把训练跑起来”，这一章讲“跑起来之后它挂了怎么办”。这是面试里区分“看过教程”和“真跑过大规模训练”的地方：hang 的成因分类、flight recorder 怎么用、checkpoint 为什么不能用 `torch.save`、timeout 该设多长——这些问题背不出来就骗不过做过训练平台的面试官。核心事实只有一条：**collective** 是同步的，所有 **rank** 必须以相同顺序、相同 **shape**、相同次数调用相同的 **collective**。这条被破坏时 PyTorch 不会报错，它会等。

hang 的排查：最高频的分布式面试题

hang 的本质：某个 rank 进了 collective 在等其他人，而其他人永远不会来。NCCL 的 kernel 已经在 GPU 上排好队等数据，Python 侧看起来完全正常——没有 traceback、没有 CPU 占用异常，`nvidia-smi` 里 GPU 利用率可能还是 100%（因为 NCCL 的 spin-wait 也算利用率）。

成因分类

成因	典型来源	现象特征
collective 调用不匹配	<code>if rank == 0:</code> 里 做 了 collective / log / 打印 DTensor	每次都挂在同一个位置，第一个 step 就挂
各 rank step 数不同	数据量不同（最后一个 batch 被 drop 了一部分 rank）、 <code>IterableDataset</code> 各 shard 长度不等	跑很多 step 之后才挂，挂在 epoch 末尾
走了不同分支	按 loss / 数据内容决定是否 <code>continue</code> ；OOM 后 <code>try/except</code> 跳过这一步；早停	偶发，与数据相关，重跑换种子就变
shape 不一致	变长序列没 pad 齐、动态 batch size	<code>TORCH_DISTRIBUTED_DEBUG=DETAIL</code> 直接报 shape mismatch
DDP 的 unused parameter	只有部分 rank 走到某个 module（MoE、条件分支）， <code>find_unused_parameters=False</code>	挂在 backward，且与数据分布有关
部分 rank 已经死了	某 rank OOM / segfault 退出，其余 rank 继续等它	日志里能找到一个 rank 的 traceback，其他 rank 静默

Warning. 最经典的写法错误：

```
if dist.get_rank() == 0:
    total = torch.tensor(len(dataset), device="cuda")
    dist.all_reduce(total)          # 只有 rank 0 参与 → 全体 hang
```

正确写法是所有 rank 都调 collective，只让 rank 0 使用结果：

```
total = torch.tensor(len(dataset), device="cuda")
dist.all_reduce(total)          # 所有 rank 都调
```

```
if dist.get_rank() == 0:
    print(total.item())
```

同一类错误的变体：if rank == 0: dcp.save(...) (DCP 内部有 collective)、print(dtensor) (触发 AllGather) 只在 rank 0 上做 model.eval() 后跑一遍带 BN 的验证 (BN 的 buffer 同步是 collective)。

排查工具

按“先让它报错、再定位到哪一行”的顺序：

1. 让它别死等。init_process_group(timeout=timedelta(minutes=30)) 加上 TORCH_NCCL_ASYNC_ERROR_HANDLING=1 (torch 2.x 默认已开), 超时后 watchdog 会 abort 进程并打出栈, 而不是挂到你手动 kill。没有这一步, 后面所有工具都用不上。
2. TORCH_DISTRIBUTED_DEBUG=DETAIL。给每个 collective 包一层 wrapper, 校验各 rank 的 op 类型与 shape 是否一致, 不一致直接报错并指出是哪个 collective。代价是每个 collective 多一次同步, 只在排查时开。
3. **flight recorder** (TORCH_NCCL_TRACE_BUFFER_SIZE=2000 + TORCH_NCCL_DUMP_ON_TIMEOUT=1)。这是排查 hang 最有效的工具: 每个 rank 维护一个环形 buffer 记录最近 N 个 collective 的类型、shape、seq id、开始/完成状态, 超时时各 rank 把 buffer dump 到 TORCH_NCCL_DEBUG_INFO_TEMP_FILE 指定的路径。用 torch.distributed.flight_recorder.fr_trace 合并所有 rank 的 dump, 它会直接告出“seq 12345 这个 AllReduce 有 63 个 rank 参与、rank 17 没到”——谁少调了一个 **collective** 一眼可见。
4. NCCL_DEBUG=INFO (配 NCCL_DEBUG_SUBSYS=INIT,COLL)。看的是 NCCL 自己的视角: ring/tree 拓扑怎么建的、走了哪块网卡、用了什么算法。初始化阶段的问题 (选错网卡、P2P 不可用) 主要靠它。日志量很大, 别在生产长期开。
5. **py-spy dump --pid <PID>**。不需要重启进程、不需要改代码, 直接打出目标进程当前的 Python 栈。对每个 rank 都 dump 一次然后 diff: 绝大多数 hang 都表现为“63 个 rank 停在 all_reduce 那一行, 1 个 rank 停在 dataloader.__next__”。PID 用 nvidia-smi 或 ps 找。
6. TORCH_NCCL_DESYNC_DEBUG=1。超时时分析各 rank 的最后一个 collective, 直接报告谁不同步。

Warning. TORCH_NCCL_BLOCKING_WAIT=1 和 watchdog 是互斥的: 开了 blocking wait 就不会创建 **watchdog** 线程, 超时行为完全不同。两个都设会让排查行为变得难以预测。生产上选 TORCH_NCCL_ASYNC_ERROR_HANDLING=1 (默认), 不要开 blocking wait。

排查流程

1. 确认真的是 **hang** 而不是慢。看日志时间戳: step time 是不是突然从 2 s 变成无穷, 还是逐渐变慢 (后者是 straggler 或数据供不上, 不是 hang)。
2. 确认所有 **rank** 都活着。有没有某个 rank 已经退出 (OOM / segfault)。torchrun 通常会报 “one of the processes exited”, 但如果它自己也卡住了, 就直接 ps 数进程数。
3. **py-spy dump** 每个 **rank**, 按栈分组。落单的那个 rank 就是罪魁。
4. 如果所有 **rank** 的栈都在同一个 **collective**, 说明是 shape 或 op 不匹配 (大家都在等, 但等的不是同一件事), 或者有 rank 已经死了。上 flight recorder dump 对比 seq id。
5. 定位到具体代码后问三个问题: 这个 collective 是不是在 rank 条件分支里? 各 rank 的 step 数是不是一样? shape 是不是数据相关的?

6. 复现。把 world size 缩到 2、数据固定，用 `TORCH_DISTRIBUTED_DEBUG=DETAIL` 单机跑。80% 的 collective 不匹配在 2 卡上就能复现。

其他典型故障

现象	成因	处理
只有某个 rank OOM	rank 间显存本来就不均：PP 的 stage 负载不均、rank 0 额外持有 dataloader 的 prefetch buffer / logger / 完整 state_dict 的临时副本	先确认“该不该不均”。PP 就应该给首尾 stage 少放层 rank 0 的额外占用要挪到 CPU
step time 忽快忽慢，某 rank 总是最慢	straggler：单卡降频（温度/功耗）、ECC 错误重试、host 侧被别的进程抢 CPU、IB link 掉速	collective 是同步的，一张慢卡拖垮全局。按 rank 打点 step time 找出 outlier，nvidia-smi -q 看 clock 与 ECC，确认后把节点换掉
init_process_group 就卡住或超时	多网卡机器 NCCL 选错网卡（挑了个不通的 docker0 / 管理网口）	<code>NCCL_SOCKET_IFNAME=eth0</code> 显式指定； <code>NCCL_DEBUG=INFO</code> 看它实际选了哪块
单机正常、跨机就挂	<code>MASTER_ADDR</code> 不可达、端口被防火墙拦、IB 没起来	先用 <code>nc -zv \$MASTER_ADDR 29500</code> 和 <code>ibstat</code> 验证链路，再跑 <code>nccl-tests</code> 的 <code>all_reduce_perf</code> 排除框架
Address already in use	上一次训练的进程没清干净，还占着 <code>MASTER_PORT</code>	<code>ps aux grep torchrun</code> 清残留；或换端口 / 用 <code>--rdzv-backend=c10d</code> 让它自己选

Note. 遇到疑似通信问题，先跑 `nccl-tests (all_reduce_perf -b 8 -e 1G -f 2 -g 8)`，它能在 5 分钟内把问题分成两半：带宽达不到标称值是集群/驱动问题，找 SRE；带宽正常那就是你的代码问题。跳过这一步会浪费很多天。

环境变量生产模板

```
#!/usr/bin/env bash
# ---- 让故障变成报错，而不是永久 hang（必开） ----
export TORCH_NCCL_ASYNC_ERROR_HANDLING=1 # watchdog 检测到超时/异常就 abort 进程
# torch 2.x 默认为 1，显式写防止被环境覆盖
# 注意：不要同时开 TORCH_NCCL_BLOCKING_WAIT，它会禁掉 watchdog
```

```
# ---- flight recorder:hang 之后能知道谁少调了 collective(强烈建议常开)----
export TORCH_NCCL_TRACE_BUFFER_SIZE=2000 # 每 rank 记录最近 2000 个 collective
export TORCH_NCCL_DUMP_ON_TIMEOUT=1 # 超时时自动 dump
export TORCH_NCCL_DEBUG_INFO_TEMP_FILE=/shared/fr/rank # 必须是所有节点都能写的共享路径

# ---- 网络:多网卡/多 HCA 的机器必须显式指定,否则 NCCL 可能挑错 ----
export NCCL_SOCKET_IFNAME=eth0 # 控制面(bootstrap)走哪块网卡
export NCCL_IB_DISABLE=0 # 有 IB 就别关
# export NCCL_IB_HCA=mlx5 # 多 HCA 时限定用哪些

# ---- 只在排查时打开,日志量大 / 有额外同步开销 ----
# export NCCL_DEBUG=INFO
# export NCCL_DEBUG_SUBSYS=INIT,COLL
# export TORCH_DISTRIBUTED_DEBUG=DETAIL # 校验各 rank 的 collective shape
# export TORCH_NCCL_DESYNC_DEBUG=1 # 超时时报告谁不同步
# export CUDA_LAUNCH_BLOCKING=1 # 只用于定位 CUDA 报错的真实位置,极慢

torchrun --nnodes 2 --nproc-per-node 8 \
    --rdzv-backend c10d --rdzv-endpoint "$MASTER_ADDR:29500" \
    --max-restarts 3 \
    train.py
```

NCCL_SOCKET_IFNAME 是最常见的坑:它管的是建连阶段用哪块网卡,多网卡机器上 NCCL 的自动选择经常挑到 docker 网桥或管理网口,表现是 `init_process_group` 卡住若干分钟然后超时。数据面走 IB 是另一套变量 (`NCCL_IB_*`)。

数值不一致的排查

先分清哪些不一致是正常的:

- 各 **rank** 的 **loss** 不同:正常。每个 rank 吃不同的数据。要看全局 loss 就自己 AllReduce 求平均后再打印,不要拿 rank 0 的 loss 当全局指标(噪声大,且会掩盖某个 rank 的数据问题)。
- 各 **rank** 的梯度在 **AllReduce** 之前不同:正常。之后应该相同。
- 各 **rank** 的参数不同:不正常。DDP 语义要求所有 rank 的参数逐位相同(FSDP/TP 下则是“同一个 replica 组内的对应分片相同”)。

参数 drift 是最难查的一类 bug,因为它不报错,只表现为“loss 降得比预期慢”或者“eval 结果诡异”。定期校验的成本很低:

```
@torch.no_grad()
def check_replica_consistency(model, group=None):
    """校验 group 内所有 rank 的参数逐位相同。DDP 传 None(整个 world);
    TP 场景传 TP 组,只校验 replicate 的参数(LayerNorm、bias)"""
    flat = torch.cat([p.detach().reshape(-1).float() for p in model.parameters()])
    sig = torch.stack([flat.sum(), (flat * flat).sum(), flat.abs().max()])
    ref = sig.clone()
    src = 0 if group is None else dist.get_global_rank(group, 0)
    dist.broadcast(ref, src=src, group=group) # src 要的是全局 rank
    if not torch.equal(sig, ref):
```

```
raise RuntimeError(f"[rank {dist.get_rank()}] param drift: "
                  f"{sig.tolist()} != {ref.tolist()}")
```

每 500 step 调一次，开销可以忽略。三种最常见的 drift 成因：

1. 自定义 `autograd.Function` 的 **forward** 做了通信，**backward** 没做对偶通信（或反过来）。第 20 章的 `f/g` 一旦写错一个方向，梯度就少加了一部分，各 rank 的更新量不同。
2. **buffer** 没同步。`nn.Parameter` 由 DDP 的梯度 AllReduce 保证一致，但 `register_buffer` 的东西（BN 的 `running_mean`、KV cache、自己维护的计数器）不在梯度路径上。DDP 有 `broadcast_buffers=True` 每次 forward 广播一次，FSDP / 手写并行要自己管。
3. **replicate** 参数的梯度漏了组内 **AllReduce**。TP 组内 LayerNorm 的 weight 是复制的，梯度必须在 TP 组内额外 AllReduce（第 20 章）。漏掉就是每张卡各自朝不同方向更新同一个参数。

另外两个必须一致的东西：随机数（dropout mask、数据增强的 seed 策略）和 optimizer 的超参（LR schedule 必须由 step 数唯一决定，不要依赖 wall clock）。

Checkpoint

朴素做法为什么不行

`rank0 gather 全量 state_dict 再 torch.save` 有三个问题，规模越大越致命：

1. 显存/内存峰值：rank 0 要装下完整的模型 + 优化器状态。70B 模型混合精度下约 1.1 TB，rank 0 装不下。
2. 慢且不并行：所有分片先汇到一张卡，再由一个进程串行写盘。别的 rank 全程干等。
3. 与并行度绑死：存下来的是裸张量，没有“这一片是全局张量的哪一段”的元信息，换并行度就读不回来。

DCP：每 **rank** 存自己的分片 + 元数据

`torch.distributed.checkpoint`（DCP）的模型是：每个 rank 只写自己持有的分片，另外写一份共享的 metadata 记录每个张量的全局 shape 与各分片的覆盖范围。加载时按 metadata 做区间匹配，把需要的字节读到当前并行度要求的位置上——所以改变并行度也能恢复。

```
# torchrun --nproc-per-node 2 ckpt_demo.py
import torch, torch.distributed as dist
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint.state_dict import get_state_dict, set_state_dict
from torch.distributed.checkpoint.stateful import Stateful

class AppState(Stateful):
    """DCP 会调用 state_dict()/load_state_dict()，把 model 与 optim 一起处理。
    用 get_state_dict / set_state_dict 是为了拿到 DTensor 形态的 sharded
    state_dict，并处理 FSDP/TP 的参数名与优化器状态映射。"""
    def __init__(self, model, optim, step=0):
        self.model, self.optim, self.step = model, optim, step

    def state_dict(self):
        msd, osd = get_state_dict(self.model, self.optim)
        return {"model": msd, "optim": osd, "step": self.step}

    def load_state_dict(self, sd):
```

```

set_state_dict(self.model, self.optim,
               model_state_dict=sd["model"], optim_state_dict=sd["optim"])
self.step = sd["step"]

# ---- save: 所有 rank 都要调, 内部有 collective ----
state = {"app": AppState(model, optim, step)}
dcp.save(state, checkpoint_id=f"/shared/ckpt/step-{step}")

# ---- load: 先按当前并行度建好 model/optim, 再原地填回 ----
state = {"app": AppState(model, optim)}
dcp.load(state, checkpoint_id="/shared/ckpt/step-1000")
start_step = state["app"].step

```

Key insight. DCP 的核心卖点是 **resharding** : tp=8 存的 checkpoint 可以用 tp=4, dp=2 加载, 512 卡训练挂了 8 张卡之后能用 504 卡 (或任意新配置) 继续。前提是张量的分布信息被记了下来——也就是第 21 章的 DTensor。这两章是一件事的两半。

Note. DCP 的 API 在 2.x 期间演进过 (save_state_dict 已废弃、 checkpoint_id 与 storage_writer 两种指定方式、 async_save 的返回类型), 确切签名以对应版本文档为准。稳定的是数据模型 : 分片 + metadata + 按区间匹配加载。

必须存什么

漏一样就是“恢复之后曲线不接续”。按重要性 :

内容	漏了会怎样
model 参数	显然
optimizer 状态 (Adam 的两个 moment、fp32 master weight)	loss 会明显跳一下再慢慢恢复 ; 这部分比参数还大
LR scheduler 状态 (或 step 本身)	学习率跳回起点 , 最容易被忽略
GradScaler 状态 (用 fp16 时)	前若干 step 白跑在 scale 探测上
dataloader 进度 (shard 位置 / 已消费样本数)	重复训练同一批数据 , 大规模下会造成可观的过拟合
RNG 状态 (CPU + CUDA , 每 rank 一份)	dropout / 数据增强序列变了 , 不可复现
step 与 epoch	日志、eval 节奏、以上全部的对齐基准

异步 checkpoint

70B 模型一次 checkpoint 是 TB 量级 , 写共享存储要好几分钟。同步存的话这几分钟所有 GPU 全闲。

sync vs async checkpoint



图 37 同步 checkpoint：训练停在原地等落盘完成。异步：先把张量 D2H 拷到 CPU pinned memory（图中那一小格，只有这段挡住训练），再由后台线程慢慢写盘，训练立刻继续。

```
fut = dcp.async_save(state, checkpoint_id=path)  # D2H 之后立刻返回
...                                              # 训练继续
fut.result()                                    # 下次 save 之前确认落盘完成
```

代价是 CPU 内存要放得下一份 checkpoint 的暂存副本，以及不能在上一次没写完时就发起下一次（所以要留住 future）。

Warning. checkpoint 与 `init_process_group(timeout=...)` 的关系是高频题。存 checkpoint 期间 DCP 内部有 collective，而不同 rank 写盘速度可能差很多（共享存储争抢），写得慢的 rank 让快的 rank 停在 collective 里等——一旦超过 timeout，watchdog 就把这次等待判成 hang 并 abort 整个 job。NCCL 后端的默认 timeout 是 10 分钟，对大模型 checkpoint 明显不够。生产上把它拉到 30-60 分钟：

```
from datetime import timedelta
dist.init_process_group("nccl", timeout=timedelta(minutes=45))
```

这也是异步 checkpoint 的另一个好处：GPU 侧的等待窗口从“几分钟写盘”缩短到“几秒 D2H”，撞 timeout 的概率大幅下降。

容错与弹性

几百卡以上，硬件故障是日常而不是例外：单卡 MTBF 按年算，几千张卡一乘就是每天若干次。所以系统必须假设“随时有卡会挂”。

`torchrun` 提供的部分：

- `--max-restarts N`：某个 worker 挂了，agent 把整组 worker 全部重启（不是只重启挂的那个——collective 的通信组已经废了，只能整组重建），最多 N 次。
- `--rdzv-backend c10d --rdzv-endpoint host:port`：rendezvous，重启后各节点重新会合并重新分配 rank。
- `--nnodes 4:8` 这种弹性区间：允许在 4 到 8 个节点之间伸缩。

弹性能生效有两个硬前提，缺一个就白搭：

1. 进程重启后能从 **checkpoint** 恢复。`torchrun` 只负责把进程拉起来，恢复训练状态是你的代码的事。所以启动逻辑必须是“扫描 checkpoint 目录 → 找最新的 → `dcp.load` → 从那个 step 继续”，而不是“从头开始”。
2. **checkpoint** 能 **resharding**。节点数变了并行度就变了，固定分片的 checkpoint 读不回来。这就是 DCP 的价值所在。

工程上还要加三件 `torchrun` 不管的事：

1. 启动前健康检查：跑一个小的 AllReduce + 每卡一个小 GEMM，验证所有卡可用、带宽正常。有问题的节点在开训前就踢掉，比训到一半挂掉便宜得多。
2. 按 **rank** 上报指标：step time、loss、grad norm、显存峰值都要带 rank 打点。straggler 和“某个 rank 数据坏了”只能靠 per-rank 指标发现。
3. 自动重启的止损：反复重启同一个坏节点是最常见的运维事故。重启要记录原因，同一节点连续失败就把它拉出资源池，而不是无限 `--max-restarts`。

Key insight. 面试被问“怎么保证几千卡训几个月不断”，答三层：检测（timeout + watchdog + flight recorder + per-rank 指标）恢复（异步 DCP checkpoint，15-30 分钟一次，能 resharding）调度（健康检查 + 坏节点隔离 + spare 节点替换 + 从最新 checkpoint 自动续跑）checkpoint 频率的取舍就是一句话：期望损失 $\approx$ 故障率 $\times$ 半个 checkpoint 间隔。

面试考点

面试考点. Q1：训练 hang 了，你的排查顺序是什么？

A：先分清 hang 和慢（看 step time 是突然变无穷还是逐渐变慢）。然后确认所有进程都活着——有没有 rank 已经 OOM 退出而其余在等它。接着 `py-spy dump --pid` 每个 rank 打栈并按栈分组，落单的那个就是罪魁祸首。如果所有栈都停在同一个 collective，说明是 shape/op 不匹配或有 rank 已死，用 flight recorder 的 dump 对比各 rank 的 seq id。定位到代码后问三件事：collective 是不是写在 rank 条件分支里、各 rank 的 step 数是否相同、shape 是否数据相关。最后缩到 2 卡加 `TORCH_DISTRIBUTED_DEBUG=DETAIL` 复现。

面试考点. Q2：hang 最常见的根因是什么？举个具体例子。

A：collective 调用不匹配。最经典的是把 collective 写进了 `if rank == 0`：——比如只在 rank 0 上 `all_reduce` 一个统计量、只在 rank 0 上 `dcp.save`、或者 `print(dtensor)` 触发了 AllGather。正确写法是所有 rank 都调 collective，只让 rank 0 使用结果。第二常见的是各 rank step 数不同（数据量不等、`IterableDataset` 的 shard 长度不齐），特征是跑很久之后挂在 epoch 末尾。

面试考点. Q3：flight recorder 是什么？为什么它比看日志有效？

A：`TORCH_NCCL_TRACE_BUFFER_SIZE=N` 开启后，每个 rank 用一个环形 buffer 记录最近 N 个 collective 的类型、shape、seq id 和完成状态；配 `TORCH_NCCL_DUMP_ON_TIMEOUT=1`，超时各 rank 自动把 buffer dump 到共享路径。用 `fr_trace` 合并所有 rank 的 dump，它按 seq id 对齐，直接告诉你“这个 AllReduce 有 63 个 rank 到了、rank 17 没到”。日志做不到这件事，因为 hang 的时候没有人报错，你只有一堆看起来正常的日志。

面试考点. Q4：为什么某个 rank OOM 而其他 rank 好好的？

A：rank 间显存本来就可能不均。三类来源：PP 的 stage 负载不均（lm_head 那段最重）；rank 0 额外持有东西（dataloader prefetch、logger、gather 全量 state_dict 的临时副本）；数据长度不均（变长序列没按 token 数均衡分桶）。排查先问“该不该不均”——PP 就应该给首尾 stage 少放层。另外 `torch.cuda.max_memory_allocated()` 要按 rank 打点，不然根本看不到不均。

面试考点. **Q5** : 什么是 straggler ? 一张慢卡为什么能拖垮整个集群 ?

A : collective 是同步的, AllReduce 要等最后一个 rank 到达, 所以全局 step time 等于最慢那个 rank 的 step time——一张卡慢 20%, 整个 job 慢 20%, 加卡完全无用。常见成因: 温度或功耗墙导致降频、ECC 错误重试、host 侧 CPU 被抢、IB link 掉速。定位靠 per-rank step time 打点找 outlier, 再用 `nvidia-smi -q` 看 clock 和 ECC 计数、`ibstat` 看链路速率。确认是硬件就换节点, 这类问题修不好。

面试考点. **Q6** : `NCCL_SOCKET_IFNAME` 是干什么的? 什么时候必须设?

A : 指定 NCCL bootstrap (建连阶段) 走哪块网卡。多网卡机器上 NCCL 的自动选择经常挑到 docker 网桥或不通的管理网口, 表现是 `init_process_group` 卡住然后超时, 且日志里没有任何有用信息。所以容器环境、多网卡机器一律显式设置, 用 `NCCL_DEBUG=INFO` 确认它实际选了哪块。数据面走 IB 是另一套 `NCCL_IB_*` 变量。

面试考点. **Q7** : `rank0 gather` 之后 `torch.save` 有什么问题? DCP 怎么解决?

A : 三个问题: rank 0 要装下完整模型加优化器状态 (70B 是 TB 量级, 装不下); 汇聚加串行写盘很慢且其他 rank 全程干等; 存下来的是裸张量, 没有分布元信息, 换并行度就读不回来。DCP 让每个 rank 只写自己的分片, 另外写一份 metadata 记录每个张量的全局 shape 和各分片的覆盖范围; 加载时按区间匹配读到当前并行度需要的位置, 所以支持 resharding。写盘是各 rank 并行的, 也没有显存峰值问题。

面试考点. **Q8** : checkpoint 必须存哪些东西?

A : model 参数、optimizer 状态 (Adam 的两个 moment 和 fp32 master weight, 比参数还大)、LR scheduler 状态或等价的 step、GradScaler 状态 (fp16 时)、dataloader 进度、每 rank 的 RNG 状态、以及 step/epoch 本身。最常被漏的是 scheduler 和 dataloader 进度: 前者让 LR 跳回起点, 后者让你重复训同一批数据。

面试考点. **Q9** : 为什么大模型训练一定要异步 checkpoint? 它和 timeout 有什么关系?

A : TB 级 checkpoint 写共享存储要几分钟, 同步存意味着这几分钟所有 GPU 全闲; 15 分钟存一次的话开销可观。异步的做法是先把张量 D2H 拷到 CPU pinned memory (秒级, 这段确实挡住训练), 再由后台线程写盘, 训练立刻继续。和 timeout 的关系: 存 checkpoint 期间 DCP 内部有 collective, 各 rank 写盘速度差异会让快的 rank 停在 collective 里等, 超过 `init_process_group(timeout=...)` 就被 watchdog 判成 hang 并 abort 整个 job。NCCL 后端默认 10 分钟, 对大模型不够, 生产设 30-60 分钟; 异步 checkpoint 把 GPU 侧的等待窗口从几分钟缩到几秒, 也顺带降低了撞 timeout 的概率。

面试考点. **Q10** : `torchrun --max-restarts` 能实现弹性训练吗? 还缺什么?

A : 它只解决“进程被重新拉起来”, 而且是整组 worker 一起重启 (通信组已经废了, 没法只补一个 rank)。要真的弹性还差两件事: 代码启动时必须自动扫描并从最新 checkpoint 恢复, 否则重启等于从头训; checkpoint 必须能 resharding, 因为节点数变了并行度就变了, 固定分片读不回来——这是 DCP 加 DTensor 的价值。工程上还要加启动前健康检查、per-rank 指标上报、以及坏节点隔离, 否则会反复重启同一个坏节点。

手写基础层：从零实现 torch.nn

前面四部分讲的是原理，这一部分开始是白板题。面试官最爱的开场是“来，手写一个 LayerNorm”——不给 IDE、不给文档、二十行之内写完，然后追问三个细节。这一章覆盖九个最高频的层。每个层都控制在白板能写完的规模，重点不在代码有多长，而在那几个只有真写过才知道的细节：`eps` 放在 `sqrt` 里还是外、方差除 `N` 还是 `N-1`、padding 补 0 还是补 `-inf`。

本章所有代码的可运行版本 + `pytest` 对齐测试在 `python/pytorch/interview/test_layers.py` (31 个测试, `pytest python/pytorch/interview/test_layers.py -q` 直接跑)。章节里的实现去掉了 `jaxtyping` 标注，逻辑与该文件一致。

先说验收标准。面试里写完代码，下一句一定要是“我会这样验证它”。手写层的验证标准只有一个：把官方层的权重拷进你的实现，同一个输入下 `torch.testing.assert_close` 通过——`shape` 对了不算对，数值一致才算对。

```
def copy_params(dst: nn.Module, src: nn.Module) -> None:
    """把 src 的同名 parameter / buffer 拷进 dst，然后就可以逐元素比了。"""
    with torch.no_grad():
        dst.load_state_dict(
            {k: v.clone() for k, v in src.state_dict().items()}, strict=False
        )
```

`strict=False` 容忍两边 key 不完全一样(比如你没实现 `num_batches_tracked`)。注意权重不能用默认初始化后直接比：两个模块各自 `reset_parameters` 一次，随机数不同，输出当然不同。带 `buffer` 的层 (`BatchNorm`) 除了输出，`running_mean` / `running_var` 也要逐步对齐——很多错误只在第二次 `forward` 才暴露。下面每个层结尾都给一句针对性的自证方法。

Key insight. `assert_close` 默认的 `rtol/atol` 是按 `dtype` 定的 (`fp32` 是 `rtol=1.3e-6`, `atol=1e-5`)。如果你的实现和官方只是运算顺序不同 (比如先乘 `weight` 再除 `std` vs 反过来), `fp32` 下会差到 `1e-6` 量级，默认容差刚好能过。差到 `1e-3` 就不是浮点误差，是公式错了。

Linear

最简单的一题，但下面两个追问能筛掉一半人。

```
import math
from torch import nn

class MyLinear(nn.Module):
    def __init__(self, in_features, out_features, bias=True):
        super().__init__()
        self.weight = nn.Parameter(torch.empty(out_features, in_features))
        self.bias = nn.Parameter(torch.empty(out_features)) if bias else None
        self.reset_parameters()

    def reset_parameters(self):
        # a=sqrt(5) 是 nn.Linear 的历史默认值
```

```

nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
if self.bias is not None:
    bound = 1 / math.sqrt(self.weight.shape[1])
    nn.init.uniform_(self.bias, -bound, bound)

def forward(self, x):
    # matmul 自动把前面所有维度当 batch，不需要手动 reshape 成 2D
    out = x @ self.weight.t()
    return out if self.bias is None else out + self.bias

```

追问一：**weight** 为什么是 **(out, in)** 而不是 **(in, out)**？前向写成 $y = xW^T + b$ ，**W.t()** 看起来是“白白多一次转置”。四个理由：

1. 转置是免费的。底层落到 `addmm(bias, x, W.t())`，cuBLAS 的 GEMM 本来就带 `transa / transb` 标志位，**W.t()** 只是改 stride，不产生任何拷贝。
2. 行主序下 **cache** 更友好。**(out, in)** 让“同一个输出神经元的所有输入权重”在内存里连续。反向算 `grad_input = grad_out @ W` 时按行扫 **W**，一次 cache line 拿到的都是有用数据。
3. 量化 / 融合天然对齐。per-output-channel 的 scale 正好按行切，`weight[i]` 就是第 i 个通道的全部权重。
4. 现实原因：所有预训练权重都是这个布局，改了就没法 `load_state_dict`。

追问二：默认初始化到底是什么分布？`nn.Linear` 用的是 `kaiming_uniform_(weight, a=math.sqrt(5))`。kaiming uniform 的边界代入 $a = \sqrt{5}$ 后， $1 + a^2 = 6$ 与分子的 6 正好约掉：

$$\text{bound} = \sqrt{\frac{6}{(1 + a^2) \cdot \text{fan_in}}} = \sqrt{\frac{1}{\text{fan_in}}} = \frac{1}{\sqrt{\text{fan_in}}}$$

也就是 $U(-1/\sqrt{\text{fan_in}}, +1/\sqrt{\text{fan_in}})$ ——和 bias 的初始化区间完全一样。**a** 本意是 leaky-ReLU 的负斜率，但这里根本没有 leaky ReLU，`sqrt(5)` 纯粹是从 Torch7 继承的历史常数，作用只是把 kaiming 的公式凑回老的 $1/\sqrt{\text{fan_in}}$ 均匀分布。“为什么是 `sqrt(5)`”是高频追问，答案就是“历史遗留，等价于 $U(\pm 1/\sqrt{\text{fan_in}})$ ”。

Warning. 白板上直接写 `self.weight = nn.Parameter(torch.randn(out, in))` 是最常见的失分点：`randn` 的方差是 1，经过 D 维内积后输出方差放大 D 倍，堆几层就溢出。至少要写 `torch.randn(...)/math.sqrt(in_features)`，并说明这是在保持前向方差。

自证：`copy_params` 后与 `nn.Linear` 对齐；再断言 `weight.shape == (out, in)` 和 `weight.abs().max() <= 1 / sqrt(fan_in)`。

Softmax

这题考的是数值稳定性，不是公式。

```

class MySoftmax(nn.Module):
    def __init__(self, dim=-1):
        super().__init__()
        self.dim = dim

```

```
def forward(self, x):
    x_max = x.amax(dim=self.dim, keepdim=True)
    e = torch.exp(x - x_max)
    return e / e.sum(dim=self.dim, keepdim=True)
```

为什么减 **max** 后结果不变. 对任意常数 c :

$$\text{softmax}(x + c)_i = \frac{e^{x_i + c}}{\sum_k e^{x_k + c}} = \frac{e^c \cdot e^{x_i}}{e^c \cdot \sum_k e^{x_k}} = \text{softmax}(x)_i$$

分子分母同乘 e^c , 约掉。所以可以自由挑 c , 取 $c = -\max_k x_k$ 是最优选择 :

这样所有指数的输入都 ≤ 0 , `exp` 结果落在 $(0, 1]$, 永远不会上溢 ; 同时分母里至少有一项是 $e^0 = 1$, 永远不会下溢成 **0**, 也就不会出现 $0/0 = \text{NaN}$ 。

溢出的具体数字. `exp` 的溢出门槛就是 $\ln(\text{dtype 的最大值})$ 。fp32 的 max 是 `3.40e38`, 阈值 **88.72** (`torch.exp(tensor(89.))` 就是 `inf`) ; fp16 的 max 只有 `65504`, 阈值 **11.09** ; bf16 的指数位和 fp32 一样宽, 阈值同样 88.72, 只是尾数少。

fp16 只有 11.09 的余量, 这在实际训练里太容易撞上 : 没加 $1/\sqrt{d_k}$ 缩放的 attention logits、loss 爆炸的中间态、长序列上没归一化的分数, 随便一个都能超。这就是混合精度里 softmax 一律在 fp32 里算的原因。

Warning. 朴素写法 `torch.exp(x) / torch.exp(x).sum(-1, keepdim=True)` 在 `x = [1e4, 1e4+1, 0]` 上会算出 `inf / inf = NaN` ; 稳定版返回正确结果。另一个隐蔽错误是 `keepdim=False`, 广播维度错位后 shape 还可能是对的 (比如方阵输入), 数值全错。

最后一个高频细节 : 分类任务里不要写 `log(softmax(x))`, 要用 `log_softmax`。后者把 `log` 和 `exp` 解析地约掉了, 直接算 $x - \max - \log \sum e^{x - \max}$, 少一次精度损失, 也不会在概率接近 0 时 `log(0) = -inf`。

自证 : 与 `torch.softmax` 对齐 ; 额外断言 `y.sum(-1) == 1` 和大输入下 `torch.isfinite(y).all()`。

LayerNorm

```
class MyLayerNorm(nn.Module):
    def __init__(self, normalized_shape, eps=1e-5, elementwise_affine=True):
        super().__init__()
        if isinstance(normalized_shape, int):
            normalized_shape = (normalized_shape,)
        self.normalized_shape = tuple(normalized_shape)
        self.eps = eps
        if elementwise_affine:
            self.weight = nn.Parameter(torch.ones(self.normalized_shape))
            self.bias = nn.Parameter(torch.zeros(self.normalized_shape))
        else:
            self.register_parameter("weight", None)
            self.register_parameter("bias", None)
```

```
def forward(self, x):
    dims = tuple(range(-len(self.normalized_shape), 0))
    mean = x.mean(dim=dims, keepdim=True)
    var = x.var(dim=dims, keepdim=True, unbiased=False) # 除以 N
    x_hat = (x - mean) * torch.rsqrt(var + self.eps)
    if self.weight is not None:
        x_hat = x_hat * self.weight + self.bias
    return x_hat
```

三个必答点。第一，**eps** 在 **sqrt** 里面。

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} \cdot w + b \quad (\text{对}) \quad y = \frac{x - \mu}{\sqrt{\sigma^2}} \cdot w + b \quad (\text{错})$$

区别在 $\sigma^2 \rightarrow 0$ 的时候。放外面时 $\sqrt{\sigma^2}$ 的导数 $1/(2\sqrt{\sigma^2})$ 会爆炸；放里面则被 ε 兜住，整条式子在常数输入上仍然有限。这是最常被抓的一个细节，而且可以一行代码验：喂一个常数张量 (`var = 0`)，eps 在里面时输出恒为 0，在外面时是 `NaN` 或 `inf`。

必答点二：方差用有偏估计。x.var(dim, keepdim=True) 默认是 `unbiased=True` (除以 $N-1$)，直接用就和官方对不上。LayerNorm 要的是 `unbiased=False` (除以 N)：它不是在估计总体方差，只是在把这一条样本缩放到单位模长，除以 N 才让 `x_hat` 的 (有偏) 方差正好是 1。归一化维度小的时候差异很明显——`head_dim=4` 时 $N/(N-1) = 1.33$ ，肉眼可见。

必答点三：归一化的维度是最后若干维。`normalized_shape=(3, 8)` 表示沿最后两维一起做归一化，它必须与输入的尾部 shape 完全匹配。因为是逐样本的，LN 与 batch size 无关，也就没有 **running stats**，train / eval 行为完全一致——这是它和 BatchNorm 最本质的差别，也是它在变长序列和小 batch 上通吃的原因。

Warning. 混合精度下的坑：官方 LN 会把统计量提升到 fp32 再算。bf16 只有 8 位尾数 ($\varepsilon \approx 0.0078$)， $D = 4096$ 上直接 `x.mean()` 的累计误差足以让归一化结果偏掉。生产实现一律 `x.float()` 算完 `rstd` 再 cast 回去，RMSNorm 同理 (而且它还要先平方，动态范围翻倍，更容易溢出)。

自证：与 `nn.LayerNorm` 对齐 (记得给 `weight` / `bias` 灌随机值，否则 $w=1, b=0$ 掩盖了 affine 的 bug)；再喂常数张量验 eps 位置。

RMSNorm

去掉均值中心化和 bias 的 LayerNorm，Llama / T5 / Gemma 都用它。

```
class MyRMSNorm(nn.Module):
    def __init__(self, normalized_shape, eps=1e-5):
        super().__init__()
        if isinstance(normalized_shape, int):
            normalized_shape = (normalized_shape,)
        self.normalized_shape = tuple(normalized_shape)
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(self.normalized_shape))

    def forward(self, x):
```

```

dims = tuple(range(-len(self.normalized_shape), 0))
ms = x.pow(2).mean(dim=dims, keepdim=True)
return x * torch.rsqrt(ms + self.eps) * self.weight

```

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{D} \sum_i x_i^2 + \varepsilon}} \odot w$$

为什么 **LLM** 都换成它。RMSNorm 论文的核心论点：LayerNorm 的收益主要来自 **re-scaling** 不变性，而不是 re-centering 不变性。既然减均值那一步贡献不大，就砍掉它，换来两个实打实的好处：

少一趟 **reduction**——LayerNorm 要先求 mean 再求 var(或者用一趟 Welford)，RMSNorm 只需要一个平方和，而归一化在 LLM 里是 memory-bound 的 kernel，省一趟 reduce 就是省一趟带宽。以及少一组参数——没有 bias，Llama 系列全是 weight-only，实测效果与 LayerNorm 相当。

考点：`nn.RMSNorm` 的 `eps` 默认不是 **1e-5**。`nn.RMSNorm` (torch 2.4+) 的 `eps` 默认值是 **None**，不是 `1e-5`。None 时它取 `torch.finfo(x.dtype).eps`——fp32 下约 `1.19e-7`，比 `1e-5` 小两个数量级。

```

ref = nn.RMSNorm(8)
ref.eps          # None
# 实际用的是 torch.finfo(torch.float32).eps == 1.1920928955078125e-07

```

对齐测试时不显式传 `eps`，你的 `1e-5` 版本和官方会差 `1e-5` 量级——刚好大过 `assert_close` 的默认 `atol=1e-5`，测试红了，你以为公式写错了，其实只是 `eps` 不同。两边都显式写 **`eps=1e-5`** 就好了。

Warning. `mean(x^2)` 不等于 `var(x)`，两者只在 `mean(x) = 0` 时才相等 ($\text{var} = \mathbb{E}[x^2] - (\mathbb{E}[x])^2$)。写成 `x.var(...)` 是把 RMSNorm 又写回了半个 LayerNorm。这个错在随机初始化的激活上几乎看不出来 (均值本来就接近 0)，一旦上游有 bias 漂移就原形毕露。

自证：与 `nn.RMSNorm(8, eps=1e-5)` 对齐；再用手算参考值 `x / sqrt(x.pow(2).mean(-1, keepdim=True) + eps)` 交叉验证，同时断言它不等于 `var` 版本。

BatchNorm1d

面试考点密度最高的一个层。

```

class MyBatchNorm1d(nn.Module):
    def __init__(self, num_features, eps=1e-5, momentum=0.1, affine=True):
        super().__init__()
        self.eps, self.momentum = eps, momentum
        if affine:
            self.weight = nn.Parameter(torch.ones(num_features))
            self.bias = nn.Parameter(torch.zeros(num_features))
        else:
            self.register_parameter("weight", None)
            self.register_parameter("bias", None)
        # buffer 而非 parameter: 进 state_dict、跟着 .to(device)，但不参与梯度
        self.register_buffer("running_mean", torch.zeros(num_features))

```



```

self.register_buffer("running_var", torch.ones(num_features))
self.register_buffer("num_batches_tracked", torch.tensor(0))

def forward(self, x):
    dims = (0,) if x.dim() == 2 else (0, 2)  # 除通道维 C 外全 reduce
    if self.training:
        mean = x.mean(dim=dims)
        var_biased = x.var(dim=dims, unbiased=False)  # 用于归一化
        with torch.no_grad():
            var_unbiased = x.var(dim=dims, unbiased=True)  # 用于 running
            self.running_mean.mul_(1 - self.momentum).add_(
                self.momentum * mean.detach())
            self.running_var.mul_(1 - self.momentum).add_(
                self.momentum * var_unbiased)
            self.num_batches_tracked += 1
    else:
        mean, var_biased = self.running_mean, self.running_var

    shape = (1, -1) if x.dim() == 2 else (1, -1, 1)
    x_hat = (x - mean.view(shape)) * torch.rsqrt(var_biased.view(shape) +
self.eps)
    if self.weight is not None:
        x_hat = x_hat * self.weight.view(shape) + self.bias.view(shape)
    return x_hat

```

考点 a：归一化用有偏方差，`running_var` 存无偏方差。这不是笔误，是官方行为，也是最容易被追问到底的一点：

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma_{\text{biased}}^2 + \varepsilon}}, \quad \text{running_var} \leftarrow (1 - m) \cdot \text{running_var} + m \cdot \sigma_{\text{unbiased}}^2$$

两个分母不一样：归一化除 N ，running 除 $N - 1$ 。理由是两者的目的不同——`running_var` 是在估计总体方差，无偏才对；而归一化只是对当前 batch 做缩放，除 N 才让 $\hat{x}$ 的样本方差正好是 1。

后果是一个真实存在的坑：同一个 **batch** 在 **train** 和 **eval** 下的输出有系统性差异，比例大约是 $\sqrt{N/(N-1)}$ 。 $N = 32$ 时是 1.6%， $N = 4$ 时就是 15%。很多人报告“切到 eval 后指标掉了”，除了 running stats 没跑够，这个偏差也贡献了一部分。

```

mine = MyBatchNorm1d(3, momentum=1.0)  # momentum=1 => running 直接等于本 batch
mine.train(); mine(x)                  # x: (N, 3)
mine.running_var / x.var(0, unbiased=False)  # 恒等于 N/(N-1)

```

考点 b：PyTorch 的 **momentum** 和 Keras 的方向相反。PyTorch：
`running = (1 - momentum) * running + momentum * batch`。所以 `momentum=0.1` 的含义是新观测占 10% 权重。

TensorFlow / Keras：`moving = momentum * moving + (1 - momentum) * batch`，默认 `momentum=0.99` 的含义是旧值占 99%。

两者的关系是 $m_{\text{torch}} = 1 - m_{\text{keras}}$ 。迁移代码时把 0.99 原样抄进 PyTorch, 就是“新观测占 99%”——running stats 变成了几乎只记得最后一个 batch, eval 时抖得一塌糊涂。反过来把 0.1 抄进 Keras, 则是几乎完全不更新。

Warning. PyTorch 还允许 `momentum=None`, 此时改用累积平均 (`1 / num_batches_tracked` 作为权重), 这就是 `num_batches_tracked` 这个 buffer 存在的唯一理由。手写版如果不实现它, `load_state_dict(strict=True)` 会因为缺 key 而失败。

考点 c: 小 **batch** 与分布式. BN 的统计量是在 batch 维上算的, 所以它的质量直接取决于 batch size:

- **batch_size = 1** 时 **train** 模式下方差为 0, `x_hat` 全是 0, 输出恒等于 `bias`。检测目标检测这类每卡 batch 只有 2 的任务, BN 基本失效, 所以后来都换成了 GroupNorm / LayerNorm。
- **DDP** 下每张卡各算各的 **BN**。`nn.BatchNorm2d` 的统计量只覆盖本卡的 local batch, 8 卡每卡 4 张图, 等效 BN batch 是 4 而不是 32。梯度会被 AllReduce, 但统计量不会。
- 需要跨卡统计就用 `nn.SyncBatchNorm` (`SyncBatchNorm.convert_sync_batchnorm(model)` 一键替换)。代价是每个 BN 层每次 forward 多一次 AllGather (同步 `sum` 和 `sum_sq`), 网络里 BN 层多的话开销可观。

Warning. 忘了 `model.eval()` 是线上事故的经典原因: 推理时 BN 仍在用当前 batch 的统计量, 输出随 batch 组成变化, 同一张图放在不同 batch 里结果不同。另一个方向的错误是 eval 时还在更新 `running_mean` / `running_var` ——手写版一定要把更新逻辑放在 `if self.training` 里面。

自证: train 模式下连续跑 3 个 batch, 每步都比对输出 + `running_mean` + `running_var` (只比第一步抓不到 momentum 方向写反); 再切 eval 比一次, 并断言 eval 下 `running_mean` 不变。

Dropout

```
class MyDropout(nn.Module):
    def __init__(self, p=0.5):
        super().__init__()
        assert 0.0 <= p <= 1.0
        self.p = p

    def forward(self, x):
        if not self.training or self.p == 0.0:
            return x  # eval 是恒等, 直接返回原张量
        if self.p == 1.0:
            return torch.zeros_like(x)  # 否则 1/(1-p) 除零
        mask = torch.empty_like(x).bernoulli_(1 - self.p)
        return x * mask / (1 - self.p)
```

为什么放大放在训练时 (**inverted dropout**)。设 $m \sim \text{Bernoulli}(1 - p)$ 。

- 朴素 **dropout**: 训练输出 $m \odot x$, 期望是 $(1 - p)x$ 。为了让推理和训练的分布对上, 推理时必须乘 $(1 - p)$ 。
- **inverted dropout**: 训练输出 $m \odot x / (1 - p)$, 期望正好是 x 。于是推理路径就是纯恒等。

$$\mathbb{E}\left[m \odot \frac{x}{1-p}\right] = \frac{(1-p) \cdot x}{1-p} = x$$

把系数挪到训练侧有三个好处：推理零开销、零分支（部署 / 导出 ONNX / 图优化时 dropout 直接消失，不留任何算子，而推理是要跑几十亿次的那一侧）；**p** 可以随时改（dropout schedule、per-layer 不同的 **p**）而不影响已保存的推理逻辑；以及只有一套权重语义，不会出现“训练好的模型忘了乘 $(1-p)$ ”这种事故——而这在朴素方案下是必然会发生的。

Warning. 两个高频错法。其一：eval 时还在 drop（忘了检查 `self.training`），表现是推理结果每次都不同。其二：用 `torch.rand(x.shape) > p` 得到 mask 后忘了除 $(1-p)$ ，推理期望值偏小 $(1-p)$ 倍，模型输出整体缩水，loss 看起来还在降但指标很差。

mask 必须每次前向重新采样，且同一个 mask 要同时用在 forward 和 backward（autograd 保存 mask 自动保证）。手写 CUDA kernel 时要么显式存 mask，要么在 backward 里用同一个 RNG offset 重放——这就是 Philox RNG 要记录（seed, offset）的原因，见第 10 章。

自证：`eval()` 下断言 `mine(x) is x`；`train()` 下用大张量统计置零比例 $\approx p$ 、`y.mean() \approx x.mean()`、非零元素恰好等于 $1/(1-p)$ ；再连调两次断言两次输出不同。

Embedding

```
class MyEmbedding(nn.Module):
    def __init__(self, num_embeddings, embedding_dim, padding_idx=None):
        super().__init__()
        self.padding_idx = padding_idx
        self.weight = nn.Parameter(torch.empty(num_embeddings, embedding_dim))
        nn.init.normal_(self.weight)  # 官方默认 N(0, 1)
        if padding_idx is not None:
            with torch.no_grad():
                self.weight[padding_idx].fill_(0)

    def forward(self, idx):
        weight = self.weight
        if self.padding_idx is not None:
            # 前向数值不变，但这一行不在计算图里，梯度自然是 0
            weight = weight.clone()
            weight[self.padding_idx] = self.weight[self.padding_idx].detach()
        return weight[idx]
```

本质是一次 **gather**，不是矩阵乘。“embedding 等价于 one-hot 乘权重矩阵”在数学上对，实现上绝对不能这么做。one-hot 是 (N, V) 的稠密矩阵， V 通常 5 万到 15 万，而 gather 是 $O(ND)$ 、one-hot matmul 是 $O(NVD)$ —— $V = 128000, D = 4096$ 时差了五个数量级。反向也一样：gather 的反向是 **scatter-add**（同一个 token 在 batch 里出现多次，梯度累加到同一行），天然稀疏，可以配 `sparse=True` + `SparseAdam`。

padding_idx 不只是初始化为 0。很多人以为 `padding_idx` 就是“把这一行初始化成全 0”。不够——还必须屏蔽它的梯度。理由：

- 如果 padding embedding 参与了 attention 或 pooling，梯度照样会流回这一行，把它从 0 推走；

- 就算梯度是 0，带 **weight decay** 或 **momentum** 的优化器仍然会更新它。AdamW 的 decoupled weight decay 是 $p -= lr * wd * p$ ， $p = 0$ 时确实不动；但 momentum 缓冲里一旦攒了历史梯度，后续 step 会持续把它推离 0。

上面的实现用 `clone()` + `detach()` 把那一行从计算图里摘出去：前向数值一模一样，反向时这一行压根没有 `grad_fn` 指向 `self.weight`，梯度精确为 0。

Warning. “我把 padding 位置的 loss mask 掉了”不等价于设了 `padding_idx`。loss mask 只挡住了从 loss 直接过来的那一条路径，如果 padding token 的 embedding 还进了 attention 的 K/V、或者被 mean-pooling 平均进去了，梯度照样从别的路径流回来。

weight tying：很多 LM 让 input embedding 和输出投影共享同一个 (V, D) 矩阵 (`lm_head.weight = embed.weight`)。能共享正是因为这两者是同一个矩阵的两种用法——一边 gather，一边 matmul。省下 $V \times D$ 个参数，在 $V = 128k$ 、 $D = 4096$ 时是 5 亿参数。

自证：与 `nn.Embedding(..., padding_idx=pad)` 对齐前向；再各自 backward 一次，比对 `weight.grad` 全等，并断言 `weight.grad[pad]` 全 0 而非 padding 行非 0。

Conv2d : im2col / unfold

手写卷积是最高频的一题。核心思路一句话：把卷积变成一次矩阵乘。

把每个滑窗位置上的 (C_{in}, K_H, K_W) 立方体拉平成一个长度 $K = C_{in} K_H K_W$ 的列向量，整张图就变成 (K, L) 的矩阵 ($L = O_H O_W$ 是滑窗个数)。卷积核 reshape 成 (C_{out}, K) ，一次矩阵乘就算完了。



图 38 im2col 的 shape 变化。关键是第二步：F.unfold 把不规则的滑窗访存 pattern 展平成规则的列，之后就完全是 GEMM 的地盘了。

```

import torch.nn.functional as F

def _pair(v):
    return (v, v) if isinstance(v, int) else v

def my_conv2d_unfold(x, weight, bias=None, stride=1, padding=0):
    n, _, h, w = x.shape
    c_out, _, kh, kw = weight.shape
  
```

```

sh, sw = _pair(stride)
ph, pw = _pair(padding)
oh = (h + 2 * ph - kh) // sh + 1
ow = (w + 2 * pw - kw) // sw + 1

cols = F.unfold(x, (kh, kw), stride=(sh, sw), padding=(ph, pw)) # (N, K, L)
out = weight.reshape(c_out, -1) @ cols # (Cout,K) @ (N,K,L) -> (N,Cout,L)
if bias is not None:
    out = out + bias.view(1, -1, 1)
return out.view(n, c_out, oh, ow)

```

输出尺寸公式（要能默写）。带 dilation 的完整版：

$$O_H = \left\lfloor \frac{H + 2p_h - d_h(K_H - 1) - 1}{s_h} \right\rfloor + 1$$

无 dilation ($d = 1$) 时化简成常用形式：

$$O_H = \left\lfloor \frac{H + 2p_h - K_H}{s_h} \right\rfloor + 1$$

推论：kernel=3, padding=1, stride=1 恰好保持尺寸不变（“same”卷积），这是 3×3 卷积统治 CNN 的实用原因之一。

im2col 的显存代价。`cols` 的元素个数是 $N \times C_{in} K_H K_W \times O_H O_W$ 。原图是 $N \times C_{in} \times HW$ 。stride=1、padding 保尺寸时， $O_H O_W \approx HW$ ，于是

$$\text{显存放大倍数} \approx K_H \times K_W$$

3×3 卷积就是 9 倍。一个 (32, 256, 56, 56) 的 fp16 activation 本身 51 MB，展开后 460 MB。这是 im2col 最大的缺点，也是别的卷积算法存在的理由：

显式 **im2col**（物化 col 矩阵后调 GEMM）只适合教学和 CPU 参考实现；隐式 **GEMM** 不物化 col，读 shared memory 时按需算索引，是 cuDNN 在绝大多数 shape 上的默认选择；**Winograd** 在 3×3、stride=1 上用变换换掉乘法次数；**FFT** 走频域点乘，只在大 kernel（7×7 以上）划算。面试里能说出“cuDNN 的 IMPLICIT_GEMM 就是不物化的 im2col”，比背公式加分。

Warning. 最难查的一个错：`F.unfold` 展开后 `K` 维的排列顺序是 `(Cin, KH, KW)`，`weight.reshape(Cout, -1)` 的顺序也必须是 `(Cin, KH, KW)`。如果你在 `reshape` 之前 `permute` 过 `weight`（比如从 NHWC 权重转过来），shape 依然完全正确、程序不报错，但算出来的数完全是错的。所以卷积这题一定要跑数值对齐，不能只看 shape。

自证：
`torch.testing.assert_close(my_conv2d_unfold(x, w, b, s, p), F.conv2d(x, w, b, s, p), rtol=1e-5, atol=1e-5)`
 遍历 `stride` / `padding` 的几种组合以及非方形 kernel；再对两边 backward 比 `x.grad` 和 `w.grad`（unfold 本身可微，反向白拿）。

MaxPool2d

同样用 `unfold` 展开滑窗，只是把 `matmul` 换成沿窗口维取 `max`。但有一个陷阱。

```
class MyMaxPool2d(nn.Module):
    def __init__(self, kernel_size, stride=None, padding=0):
        super().__init__()
        self.kernel_size = _pair(kernel_size)
        self.stride = _pair(stride) if stride is not None else self.kernel_size
        self.padding = _pair(padding)

    def forward(self, x):
        n, c, h, w = x.shape
        kh, kw = self.kernel_size
        sh, sw = self.stride
        ph, pw = self.padding
        oh = (h + 2 * ph - kh) // sh + 1
        ow = (w + 2 * pw - kw) // sw + 1

        # 关键：手动补 -inf，而不是让 unfold 补 0
        if ph or pw:
            x = F.pad(x, (pw, pw, ph, ph), value=float("-inf"))
        cols = F.unfold(x, (kh, kw), stride=(sh, sw))  # (N, C*kh*kw, L)
        cols = cols.view(n, c, kh * kw, -1)
        return cols.amax(dim=2).view(n, c, oh, ow)
```

Warning. `F.unfold` 的 `padding` 参数补的是 `0`。对 `conv` 这正好符合语义（补 `0` 的位置贡献 `0`），但对 `max pooling` 是错的：如果特征图全是负数，补进来的 `0` 会成为窗口最大值，输出里凭空出现 `0`。

```
x = -torch.rand(1, 2, 5, 5) - 1.0  # 全部 < -1
ninf = float("-inf")

# 错：让 unfold 补 0，边缘窗口的最大值变成 0
bad = F.unfold(x, 3, stride=2, padding=1).view(1, 2, 9, -1).amax(2)
# 对：先补 -inf，padding 位置不参与竞争
good = F.unfold(F.pad(x, (1, 1, 1, 1), value=ninf), 3, stride=2)
good = good.view(1, 2, 9, -1).amax(2)

bad.max()  # tensor(0.) x
torch.testing.assert_close(good, F.max_pool2d(x, 3, 2, 1).flatten(2))  # ✓
```

官方 `max_pool2d` 的语义是“padding 位置不参与竞争”，等价于补 $-\infty$ 。这个 bug 只在输入全负时暴露，随机初始化的测试数据（有正有负）抓不到，必须专门构造。

反向是稀疏的 **scatter**：只有每个窗口里的 `argmax` 位置收到梯度，其余位置为 `0`——这就是 `return_indices=True` 存在的意义（喂给 `MaxUnpool2d`）；窗口重叠时（`stride < kernel`）同一个输入位置可能被多个窗口选中，梯度要累加。对比 `AvgPool`：它的反向是把梯度均摊到窗口内所有位置，处处非零；`MaxPool` 的梯度稀疏，训练早期容易让部分区域完全收不到信号。

- `ceil_mode=True` 会把输出尺寸公式里的 `floor` 换成 `ceil`，并保证最后一个窗口的起点仍在输入内。本实现只支持 `floor`，白板上说明这一点即可。

自证：遍历 (k, s) 组合与 `F.max_pool2d` 对齐；再专门用全负输入 + **padding** 跑一次，断言输出全 < 0 。

汇总：九个层的对照表

手写实现	官方对照	最容易错的点
<code>MyLinear</code>	<code>nn.Linear</code>	<code>weight</code> 是 (out, in) ；默认初始化是 $U(\pm 1/\sqrt{fan_in})$ ，别写 <code>randn</code>
<code>MySoftmax</code>	<code>torch.softmax</code>	不减 <code>max</code> $\rightarrow$ fp32 在 88.7、fp16 在 11.1 就 <code>inf</code>
<code>MyLayerNorm</code>	<code>nn.LayerNorm</code>	<code>eps</code> 要在 <code>sqrt</code> 内； <code>var</code> 要 <code>unbiased=False</code>
<code>MyRMSNorm</code>	<code>nn.RMSNorm</code>	<code>nn.RMSNorm</code> 的 <code>eps</code> 默认是 <code>None</code> $\rightarrow$ <code>finfo.eps</code> ； <code>mean(x^2) != var(x)</code>
<code>MyBatchNorm1d</code>	<code>nn.BatchNorm1d</code>	归一化用有偏方差、 <code>running_var</code> 存无偏； <code>momentum</code> 语义与 Keras 相反
<code>MyDropout</code>	<code>nn.Dropout</code>	<code>inverted</code> ：训练时放大 $1/(1-p)$ ； <code>eval</code> 必须恒等
<code>MyEmbedding</code>	<code>nn.Embedding</code>	<code>padding_idx</code> 要屏蔽梯度，不只是初始化为 0
<code>my_conv2d_unfold</code>	<code>F.conv2d</code>	K 维顺序必须是 (C_{in}, K_H, K_W) ； <code>im2col</code> 显存放大 $K_H K_W$ 倍
<code>MyMaxPool2d</code>	<code>F.max_pool2d</code>	<code>padding</code> 必须补 $-\infty$ ，不能靠 <code>unfold</code> 补 0

Key insight. 这九个层里，有七个的“最容易错的点”都是数值语义而不是 `shape` 逻辑。所以白板写完之后，主动说一句“我会把官方权重拷进来跑 `assert_close`，并且专门构造能触发边界的输入（常数张量验 `eps`、全负输入验 `pooling padding`、大 logits 验 `softmax`）”——这一句话往往比代码本身更能说明你写过。

面试考点

面试考点. Q1：`nn.Linear` 的默认初始化是什么分布？`a=math.sqrt(5)` 是什么意思？

A：`kaiming_uniform(weight, a=sqrt(5))`。`kaiming uniform` 的边界是 `sqrt(6 / ((1+a^2) * fan_in))`，代入 `a=sqrt(5)` 后 `1+a^2=6`，约掉就是 `1/sqrt(fan_in)`，即 $U(-1/\sqrt{fan_in}, 1/\sqrt{fan_in})$ 。`a` 本意是 leaky ReLU 的负斜率，但这里没有 leaky ReLU，`sqrt(5)` 纯粹是从 Torch7 继承的历史常数，作用就是把公式凑回老的均匀分布。

面试考点. Q2：`softmax` 为什么要减最大值？不减会怎样？

A：因为 `softmax(x + c) = softmax(x)`（分子分母同乘 e^c 约掉），减 `max` 后所有指数输入都 ≤ 0 ，`exp` 落在 $(0, 1]$ 永不上溢，分母至少含一项 1 也不会下溢成 0。不减的话 fp32 在 $x > 88.7$ 就 `inf`，fp16 在 $x > 11.1$ 就 `inf`，最后 `inf/inf = NaN`。fp16 那 11 的余量在没做 QK 缩放的 attention 上很容易撞上。

面试考点. Q3：BatchNorm 归一化时用有偏方差还是无偏方差？`running_var` 呢？

A：归一化用有偏（除 N），`running_var` 存无偏（除 N-1）。因为 `running_var` 是在估计总体方差，要无偏；而归一化只是给当前 batch 做缩放，除 N 才让输出方差正好是 1。副作用是同一个 batch 在 train / eval 下输出差 $\sqrt{N/(N-1)}$ —— N=4 时约 15%。

面试考点. Q4：PyTorch 的 BN `momentum=0.1` 和 Keras 的 `0.99` 是一回事吗？

A：不是，语义正好相反。PyTorch 是 `running = (1-m)*running + m*batch`，`m` 是新观测的权重；Keras 是 `moving = m*moving + (1-m)*batch`，`m` 是旧值的权重。换算关系是 `m_torch = 1 - m_keras`。迁移时把 0.99 直接抄进 PyTorch，等于让 running stats 只记得最后一个 batch。

面试考点. Q5：为什么是 inverted dropout（训练时放大）而不是推理时缩小？

A：训练输出 $m \cdot x / (1-p)$ 的期望正好是 x ，于是推理路径退化成纯恒等：零开销、零分支，导出 ONNX 或做图优化时 dropout 直接消失。而且 `p` 可以随时改（dropout schedule）而不影响推理逻辑，也不会出现“训练好的模型忘了乘 $(1-p)$ ”这种事故。另外 `p=1` 要特判成全 0，否则 $1/(1-p)$ 除零。

面试考点. Q6：手写 conv2d，讲讲 im2col 的思路和代价；MaxPool 能不能照抄？

A：`F.unfold` 把每个滑窗的 (C_{in}, KH, KW) 立方体展平成一行，得到 (N, K, L) ，权重 reshape 成 (C_{out}, K) ，一次 batched matmul 就是卷积；输出尺寸 $\text{floor}((H + 2p - k)/s) + 1$ 。代价是 col 矩阵放大约 $KH \cdot KW$ 倍显存，所以 cuDNN 用的是不物化 col 的隐式 GEMM，小 kernel 还有 Winograd。MaxPool 不能照抄：`unfold` 的 padding 补 0，特征图全负时补进来的 0 会成为窗口最大值，必须先 `F.pad(x, ..., value=-inf)`。

手写 autograd Function : backward 全靠自己推

上一章手写的是 forward , autograd 帮你把 backward 免费算了。这一章反过来 : **forward** 和 **backward** 都你写。面试官问这块 , 考的 not API , 是“你知不知道 autograd 到底在做什么”——backward 收到的是什么、要返回几个东西、返回的东西必须是真导数吗、你怎么证明你推对了。

前三个问题有标准答案 , 第四个问题的标准答案是一句话 : 跑 **gradcheck** 。能主动说出这句话的候选人 , 和只会写 `return grad_out * something` 的候选人 , 是两个段位。本章代码的可运行版本 + pytest 在 `python/pytorch/interview/test_custom_autograd.py` (33 个测试 , `pytest python/pytorch/interview/test_custom_autograd.py -q` 直接跑) 。章节实现去掉了 `jaxtyping` 标注 , 逻辑一致。autograd 引擎本身的机制 (VJP、`grad_fn`、拓扑序) 见第 6 章 , 这里只讲怎么往里插自己的节点。

标准模板与五条规则

```
import torch
from torch.autograd import Function

class MyOp(Function):
    @staticmethod
    def forward(ctx, x, scale):          # scale 是 python float , 非 tensor
        ctx.save_for_backward(x)        # tensor 走 save_for_backward
        ctx.scale = scale               # 非 tensor 直接挂 ctx
        return x * scale

    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        dx = grad_out * ctx.scale if ctx.needs_input_grad[0] else None
        return dx, None                # 两个输入 -> 两个返回值

y = MyOp.apply(x, 3.0)                # 用 apply , 不是 MyOp()(x, 3.0)
```

规则	说明
forward / backward 都是 <code>staticmethod</code>	状态全靠 <code>ctx</code> 传递。入口是 <code>MyOp.apply(...)</code> ; <code>MyOp()(...)</code> 在 torch 2.x 上直接报 “Legacy autograd function with non-static forward method is deprecated”。
backward 返回值个数 = forward 输入个数	严格相等、位置一一对应 , 包括非 tensor 参数。少返回就报 “returned an incorrect number of gradients (expected 2, got 1)” , 多返回同样报错。这是最高频的运行时错误。
不需要梯度的位置返回 <code>None</code>	非 tensor 参数 (<code>float</code> / <code>int</code> / <code>bool</code> / <code>str</code>) 永远返回 <code>None</code> 。

<code>ctx.save_for_backward</code> 只存 tensor	存非 tensor 会报错 ;反过来把 tensor 挂 <code>ctx.xxx</code> 会绕过版本检查和引用计数。
<code>ctx.needs_input_grad</code> 是 bool 元组	长度等于 forward 输入个数。用它跳过不必要的计算，对应位置返回 <code>None</code> 。

为什么 **tensor** 必须走 **save_for_backward**，两个理由，第二个是面试的落点。一是引用计数：`save_for_backward` 让引擎知道这个 tensor 被反向图持有，`backward()` 跑完自动释放（这也是第二次 backward 会报 “Trying to backward through the graph a second time” 的原因）；挂在 `ctx.x` 上则一直活着，等于内存泄漏。二是版本号检查：`saved_tensors` 读取时会比对 `version counter`，如果这个 tensor 在 forward 之后被原地改过，直接报 “modified by an inplace operation”，而不是静默算错。挂 `ctx.x` 完全绕过这层保护。

另外两个 `ctx` 方法不常用，但一被问到就能拉开差距。`ctx.mark_dirty(x)` 用在 forward 里原地修改了输入的时候：它给 `version counter +1` 并更新图里的引用，否则任何 `saved_tensors` 持有它的节点都会拿到被改过的数据；调了之后 `apply` 的返回值和输入是同一个 **Python** 对象，这是实现 `relu_` / 原地量化这类算子唯一的正确姿势。`ctx.mark_non_differentiable(i)` 用在某个输出天然没有梯度（整数索引、掩码、计数）的时候：标记后它的 `requires_grad` 恒为 `False`，用户对它 backward 会直接报错而不是拿到一堆 0——`torch.max(x, -1).indices.requires_grad` 永远是 `False` 就是这么实现的。注意即使标记了，**backward** 的形参个数仍然等于 **forward** 的输出个数，那个用不到的 `grad` 也要写在签名里。

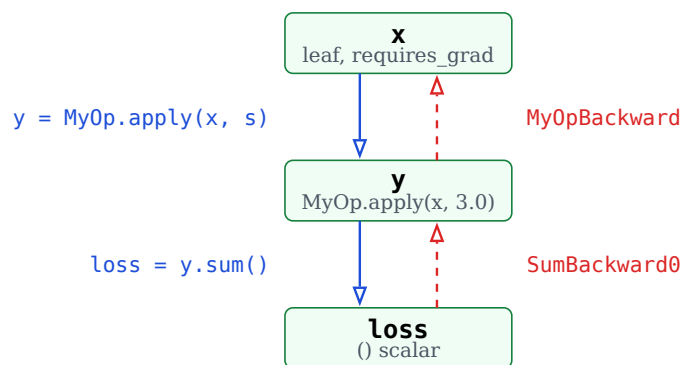


图 39 自定义 Function 在图里就是一个普通节点。`apply` 时引擎自动生成一个名为 `<类名>Backward` 的 `grad_fn` 并把 `ctx` 塞进去；`backward()` 走到它时调用你写的 `backward`。

Note. torch 2.0+ 推荐把 “存 context” 从 `forward` 拆到独立的 `setup_context(ctx, inputs, output)`，让 `forward` 成为纯函数，从而能被 `torch.func (vmap)` 正确处理。两种写法都还支持：纯训练脚本用旧写法没问题，写库就用新写法。签名字节见第 6 章。

MyReLU：最简模板

```

class MyReLU(Function):
    @staticmethod
    def forward(ctx, x):
        mask = x > 0 # 存 bool (1 byte) 比存 x (4 byte) 省 4 倍
        ctx.save_for_backward(mask)
        return x * mask

```

```

@staticmethod
def backward(ctx, grad_out):
    (mask,) = ctx.saved_tensors
    if not ctx.needs_input_grad[0]:
        return None
    return grad_out * mask

```

存输入还是存输出,是设计 **activation kernel** 的第一个决定。官方 `relu` 比上面更省:它存输出 `y`, 因为 $y > 0 \iff x > 0$, 而 `y` 本来就要传给下一层,等于零额外显存。这也正是 ReLU 能安全 inplace (`relu_`) 的原因——反向不需要原始输入。同理 `sigmoid` / `tanh` 也存输出(导数能用输出表达: $y(1-y)$ 、 $1-y^2$); 而 `GELU` / `SiLU` 只能存输入, 它们的导数无法只用输出写出来。

$x = 0$ 处 ReLU 不可导, PyTorch 约定次梯度取 0, 实现上就是 `x > 0` 而不是 `x >= 0`。所以 `gradcheck` 的输入要避开 0 附近——中心差分会跨过折点, 两边斜率不一致, 必然失败; 测试里用 `x += 0.5 * x.sign()` 把输入推离折点。

MyGELU : 手推导数

BERT / GPT-2 用的是 `tanh` 近似版:

$$s(x) = c(x + ax^3), \quad c = \sqrt{\frac{2}{\pi}} \approx 0.7979, \quad a = 0.044715$$

$$\text{gelu}(x) = \frac{1}{2}x(1 + \tanh s(x))$$

反向手推(面试要求当场写)。令 $t = \tanh s$, 用 $\frac{dt}{ds} = 1 - t^2$ 和 $\frac{ds}{dx} = c(1 + 3ax^2)$, 对 $\frac{1}{2}x(1 + t)$ 用乘法法则:

$$\frac{d \text{ gelu}}{dx} = \underbrace{\frac{1}{2}(1 + t)}_{\text{对前面的 } x \text{ 求导}} + \underbrace{\frac{1}{2}x(1 - t^2)c(1 + 3ax^2)}_{\text{对 } \tanh \text{ 求导}}$$

漏掉第二项是最常见的错误——它是“门控因子”的贡献, x 大的时候不可忽略。

```

import math
_C = math.sqrt(2.0 / math.pi)
_A = 0.044715

class MyGELU(Function):
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)          # 存 x, backward 里重算 s 和 t
        s = _C * (x + _A * x.pow(3))
        return 0.5 * x * (1.0 + torch.tanh(s))

    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        s = _C * (x + _A * x.pow(3))
        t = torch.tanh(s)

```

```
dsdx = _C * (1.0 + 3.0 * _A * x.pow(2))
local = 0.5 * (1.0 + t) + 0.5 * x * (1.0 - t * t) * dsdx
return grad_out * local
```

这里存 `x` 然后在 `backward` 里重算 `s` 和 `t`，是 `activation checkpointing` 的微观版本：多两次 `elementwise`（在 `memory-bound` 的层上几乎免费），省掉一个和输入等大的张量。

考点一：F.gelu 默认不是 tanh 近似

精确版是 $0.5x(1 + \operatorname{erf}(x/\sqrt{2}))$ 。2018 年选 `tanh` 近似是因为当时部分硬件上 `erf` 没有快速实现；今天 `erf` 已经不慢，所以 PyTorch 把精确版设成了默认。

```
F.gelu(x) # 默认 approximate='none'，走精确的 erf 版本
F.gelu(x, approximate='tanh') # BERT / GPT-2 用的是这个
```

Warning. 两者的最大绝对差约 $4.7\text{e-}4$ （出现在 $x \approx \pm 2.70$ ）。落在激活值上完全不影响收敛，但做数值对齐时一定会被 `assert_close` 抓到。所以 `assert_close(MyGelu.apply(x), F.gelu(x))` 会失败，不是因为你推错了，是因为默认走 `erf`——对齐必须写 `F.gelu(x, approximate="tanh")`。复现 HuggingFace 老模型时发现输出对不上小数点后三位，先查这个。

考点二：GELU 不单调

这和 ReLU 的直觉完全不同，是个很好的追问。GELU 的导数在 $x < -0.752$ 的整个区间上都是负的（之后单调回升趋向 0），最小值约 -0.129 ，出现在 $x \approx -1.419$ 。

```
x = torch.tensor([-1.42, -1.0, 0.0, 1.0], requires_grad=True)
MyGelu.apply(x).sum().backward()
x.grad # [-0.1290, -0.0830, 0.5000, 1.0830] 前两个是负的
```

Warning. 网上常见的说法是“GELU 的导数在 $x \approx -0.75$ 附近为负”——这是把导数的零点（ $x \approx -0.752$ ，导数由正转负的地方，也正是 GELU 函数自身的极小值点）当成了导数的极小点（ $x \approx -1.419$ ）。极小点在更左边。被问到时把这两个数分清楚，比记住一个模糊的“大约 -0.75 ”强得多。

非单调的意义是“轻度的负输入被小幅保留，更负的输入反而被推回 0”——一种平滑的软门控，比 ReLU 的硬截断多一点表达力。

Softmax 的 backward：最高频的手推题

设 $y_i = e^{x_i} / \sum_k e^{x_k}$ 。 $i = j$ 时商法则给出 $y_i - y_i^2$ ， $i \neq j$ 时给出 $-y_i y_j$ ，两种情况合并成一条：

$$\frac{\partial y_i}{\partial x_j} = y_i(\delta_{ij} - y_j), \quad J = \operatorname{diag}(y) - yy^T$$

现在做 VJP。记上游梯度 $g_i = (\partial L) / (\partial y_i)$ ：

$$\frac{\partial L}{\partial x_j} = \sum_i g_i y_i (\delta_{ij} - y_j) = g_j y_j - y_j \sum_i g_i y_i = y_j \left(g_j - \sum_i g_i y_i \right)$$

向量化就是那条要背下来的式子：

$$dx = (dy - (dy \odot y) \cdot \text{sum_1}) \odot y$$

```
class MySoftmaxFunction(Function):
    @staticmethod
    def forward(ctx, x):
        y = torch.exp(x - x.amax(dim=-1, keepdim=True))
        y = y / y.sum(dim=-1, keepdim=True)
        ctx.save_for_backward(y)          # 只需要 y, 不需要 x
        return y

    @staticmethod
    def backward(ctx, grad_out):
        (y,) = ctx.saved_tensors
        return (grad_out - (grad_out * y).sum(dim=-1, keepdim=True)) * y
```

四个要点：

1. 绝不显式构造雅可比。 D 维输入的 J 是 (D, D) ；对 (B, S, D) 的输入就是 (B, S, D, D) ， $S = D = 4096$ 时单是这一个张量就上百 GB。上面那条式子把它压成两次 elementwise + 一次 reduction， $O(D)$ 显存。这是“VJP 而不是 Jacobian”最具体的一次体现。
2. **backward** 只需要 **y**，不需要 **x**，所以存输出。attention 的反向能只保存 softmax 结果就是这个道理；FlashAttention 更狠，连 **y** 都不存，只存 (B, H, S) 的 logsumexp 在反向里重算。
3. $(dy \odot y)$ 求和这一项的物理含义是“上游梯度的加权平均”（权重就是概率 y ）。减掉它保证了 $\sum_j dx_j = 0$ ：softmax 的输出被约束在单纯形上，沿全 1 方向扰动输入不改变输出，所以那个方向上的梯度必然为 0。这是一条免费的自检。
4. 和 cross-entropy 融合时， $g = -\text{onehot} / y$ 代入整条式子恰好塌缩成 $dx = y - \text{onehot}$ 。这就是 softmax_cross_entropy 的反向如此简洁、也如此适合融成一个 kernel 的原因。

Warning. 最经典的错法： $dx = dy * y * (1 - y)$ 。这只保留了雅可比的对角线，是 **sigmoid** 的导数，不是 softmax 的。它丢掉了 $-y_j \sum_i g_i y_i$ 这一项，也就丢掉了“各个类别之间此消彼长”的耦合。这个错误在 shape 上完全正确，loss 也会下降（方向大致对）， $D = 2$ 时数值上还很接近——只有 gradcheck 能立刻抓出来。它还有个特征：不满足 $\sum_j dx_j = 0$ 。

LayerNorm 的 backward：难度最高的手推题

面试里能完整推到最后那条紧凑形式，这题基本就过了。设一行输入 $x \in \mathbb{R}^D$ ：

$$\mu = \frac{1}{D} \sum_i x_i, \quad \sigma^2 = \frac{1}{D} \sum_i (x_i - \mu)^2, \quad r = \frac{1}{\sqrt{\sigma^2 + \varepsilon}}$$

$$\hat{x} = (x - \mu)r, \quad y = w \odot \hat{x} + b$$

参数梯度是简单的部分，在除最后一维外的所有维度上求和（ w/b 是 $(D,)$ ，而 dy 是 (B, S, D) ）：

$$\frac{\partial L}{\partial w} = \sum_{\text{batch}} (dy \odot \hat{x}), \quad \frac{\partial L}{\partial b} = \sum_{\text{batch}} dy$$

输入梯度要走三条路径。先把 affine 剥掉：令 $g = dy \odot w = (\partial L)/(\partial \hat{x})$ 。关键在于 x_i 影响 $\hat{x}$ 有直接、经 μ 、经 σ^2 三条路，三条都要算：

$$\frac{\partial L}{\partial \sigma^2} = \sum_i g_i (x_i - \mu) \cdot \left(-\frac{1}{2}\right) (\sigma^2 + \varepsilon)^{-3/2}$$

$$\frac{\partial L}{\partial \mu} = -r \sum_i g_i \quad (\text{经 } \sigma^2 \text{ 的那一项含 } \sum_i (x_i - \mu) = 0, \text{ 自动消失})$$

$$\frac{\partial L}{\partial x_i} = \underbrace{g_i r}_{\text{直通}} + \underbrace{\frac{\partial L}{\partial \sigma^2} \cdot \frac{2}{D} (x_i - \mu)}_{\text{经方差}} + \underbrace{\frac{\partial L}{\partial \mu} \cdot \frac{1}{D}}_{\text{经均值}}$$

化简：把 $x_i - \mu = \hat{x}_i / r$ 代回去，方差那一项变成

$$\frac{\partial L}{\partial \sigma^2} \cdot \frac{2}{D} (x_i - \mu) = -\frac{1}{D} r^3 (x_i - \mu) \sum_j g_j (x_j - \mu) = -r \hat{x}_i \cdot \frac{1}{D} \sum_j g_j \hat{x}_j$$

均值那一项直接是 $-r \cdot (1/D) \sum_j g_j$ 。三项合并，得到那条著名的紧凑形式（两个 `mean` 都是沿归一化维，不是全局均值）：

$$dx = r(g - \text{mean}(g) - \hat{x} \odot \text{mean}(g \odot \hat{x}))$$

```
class MyLayerNormFunction(Function):
    @staticmethod
    def forward(ctx, x, weight, bias, eps=1e-5):
        mu = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        rstd = torch.rsqrt(var + eps)
        xhat = (x - mu) * rstd
        ctx.save_for_backward(xhat, rstd, weight)  # 存 xhat/rstd, 不是 x/mu/var
        return xhat * weight + bias

    @staticmethod
    def backward(ctx, grad_out):
        xhat, rstd, weight = ctx.saved_tensors
        g = grad_out * weight  # dL/dxhat

        dx = dw = db = None
        if ctx.needs_input_grad[0]:
```

```

dx = rstd * (
    g
    - g.mean(dim=-1, keepdim=True)
    - xhat * (g * xhat).mean(dim=-1, keepdim=True)
)
reduce_dims = tuple(range(grad_out.dim() - 1))
if ctx.needs_input_grad[1]:
    dw = (grad_out * xhat).sum(dim=reduce_dims)
if ctx.needs_input_grad[2]:
    db = grad_out.sum(dim=reduce_dims)
return dx, dw, db, None      # 第 4 个输入 eps 是 float

```

几何直观（能说出来就是加分项）。归一化把 x 投影到“均值为 0、模长为 $\sqrt{D}$ ”的球面上。沿两个方向移动 x 根本不改变 y ：全 1 方向（整体平移， μ 跟着变， $x - \mu$ 不变）和 $\hat{x}$ 方向（整体缩放， σ 跟着变， $(x - \mu)/\sigma$ 不变）。既然输出不变，梯度在这两个方向上的分量就必须为 0——公式里的 $-\text{mean}(g)$ 正是在减掉全 1 方向的分量， $-\hat{x} \cdot \text{mean}(g \odot \hat{x})$ 正是在减掉 $\hat{x}$ 方向的分量。这条式子就是一个正交投影。推论是两条免费自检：

$$\sum_j dx_j = 0 \quad \text{且} \quad \sum_j dx_j \hat{x}_j = 0$$

Warning. 最常见的错法：只写 `dx = g * rstd`，漏掉 `mu` 和 `var` 两条路径。这个错误极其隐蔽——漏掉的两项是投影分量，量级通常比主项小，方向大体是对的，小 batch 上 **loss** 照样会下降，小模型甚至能训到收敛，只是慢一截、上限低一截。没有报错、没有 NaN，只有“我的实现好像比官方差一点”。`gradcheck` 一秒抓出来（`test_layernorm_fn_naive_wrong_backward_is_detected` 就是专门演示这个的反面教材）。

两个工程细节值得一起记住：kernel 里存（`mean`, `rstd`）这两个（ BS ,）的小张量而不是 `var`，省一次 `sqrt` 和除法；`dw` / `db` 是跨 batch 的 reduction，要做两阶段规约（每个 block 先累加到 partial buffer，第二个 kernel 汇总），直接 `atomicAdd` 到（ D ,）上会被 BS 个 block 抢同一批地址的写冲突拖垮。

gradcheck：怎么证明你推对了

`gradcheck` 用数值差分（中心差分）逐元素算出雅可比，和你的 `backward` 给出的解析雅可比逐项比对。它是唯一能抓出“公式推错但 shape 正确”的手段。

```

from torch.autograd import gradcheck, gradgradcheck

x = torch.randn(3, 4, dtype=torch.double, requires_grad=True)
assert gradcheck(MyGELU.apply, (x,))          # 一阶
assert gradgradcheck(MyGELU.apply, (x,))      # 一阶 + 二阶
ok = gradcheck(RoundSTE.apply, (x,), raise_exception=False)  # 返回 False 而不抛异常

```

必须用 float64

这是最容易被追问、也最容易答错的一点。中心差分的误差有两个来源：

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \underbrace{O(h^2)}_{\text{截断误差}} + \underbrace{O(\varepsilon_{\text{mach}}/h)}_{\text{舍入误差}}$$

两项一个随 h 减小、一个随 h 增大，最优 $h \approx \varepsilon_{\text{mach}}^{1/3}$ ，此时总误差约 $\varepsilon_{\text{mach}}^{2/3}$ ：

dtype	机器 eps	可达相对误差	够不够 atol=1e-5
fp32	1.19e-7	约 1e-5	不够：数值误差本身就到了容差
fp64	2.22e-16	约 1e-11	足够，留了 6 个数量级余量

用 fp32 跑 gradcheck，数值微分自己的误差会掩盖真实的梯度错误——公式错了 1% 它也可能通过，公式全对它也可能失败。所以 gradcheck 的输入一律 `dtype=torch.double`。

参数上还有三点：`eps`（差分步长）默认 `1e-6`，配 fp64 刚好，不要因为“失败了”就去调大它，先怀疑公式；`raise_exception=False` 返回 `bool` 而不抛异常，用途是断言某个实现应该失败（测 STE、测反面教材时必须用它，否则 `pytest` 直接被异常打断）；`nondet_tol` 只在算子有非确定性（`atomicAdd` 之类）时才用。

Warning. gradcheck 失败时先排除三个“不是你的错”的原因：输入落在不可导点上（ReLU 的 0、abs 的 0、round 的 .5）；用了 fp32；算子里有随机性（dropout 之类）导致两次 forward 结果不同。这三个排除完，才是公式真的推错了。

二阶导：白拿与显式禁用

用 `create_graph=True` 反向时，autograd 会把 `backward()` 里面的运算本身也录进计算图，再对这张图求一次导。所以只要满足两个条件，二阶导自动可得、你什么都不用写：`backward` 全部用可微的 **torch** 算子写（不要用 `.data` / `.detach()` / `numpy` / 原地写 `buffer`——这些会把链条切断，二阶导静默变成 0，比报错更可怕），并且没有加 `@once_differentiable`。

```
class MyCube(Function):
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return x.pow(3)

    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        return 3.0 * x.pow(2) * grad_out      # 全是可微算子

x = torch.tensor([2.0, -3.0, 0.5], requires_grad=True)
(g1,) = torch.autograd.grad(MyCube.apply(x).sum(), x, create_graph=True)
(g2,) = torch.autograd.grad(g1.sum(), x)
# g1 = 3x^2, g2 = 6x — 二阶导没写一行额外代码
```

反过来，`@torch.autograd.function.once_differentiable`（加在 `@staticmethod` 下面）做两件事：在 `no_grad()` 下执行 `backward`（省掉建图开销），并给输出打标记，二阶导时直接报错而不是静默给错误结果。什么时候该加：`backward` 里调了 C++/CUDA 扩展、`numpy` 或任何不可微的东西时——这时二阶导本来就是错的，与其让用户拿到一个静默错误的 Hessian，不如显式报错。

真的需要二阶导的场景：WGAN-GP 的梯度惩罚（对梯度范数再求导）、MAML 等元学习、Hessian-vector product（见第 6 章）以及一些二阶优化器。这些链路里只要有一个 `@once_differentiable` 的算子，整条就断了。

Straight-Through Estimator : backward 可以不是真导数

```
class RoundSTE(Function):
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return torch.round(x)

    @staticmethod
    def backward(ctx, grad_out):
        (x,) = ctx.saved_tensors
        return grad_out * (x.abs() <= 1.0)  # clipped STE : 量化区间外不透传
```

`round` / `sign` / `argmax` 这类算子的真导数几乎处处为 **0**，梯度一到这里就断了，前面的层永远学不到东西。STE 的做法是“前向用离散值保证和推理一致，反向假装这一步是恒等函数”。两大用途：量化感知训练（**QAT**）（前向 fake-quantize 到 int8，反向按 fp32 更新）和离散 **latent**（VQ-VAE 的 codebook 查找、Gumbel-softmax 的硬采样分支）。

它是一个有偏估计器，不对应任何函数的真导数。能 work 的直觉是把 `round(x)` 看成 $x + \text{noise}$ ——一个加了零均值扰动的恒等映射，那么恒等的导数 1 就是个合理的代理。而 `grad * (|x| <= 1)` 这个 clip 不是可有可无的。落在量化区间外的输入已经饱和，再怎么推也不会改变输出，透传梯度纯属噪声，还会让权重一路漂向无穷。这叫 clipped STE 或 hardtanh STE，实践里几乎都用这个版本。

一行写法（面试里能说出来是加分项）：`y = x + (x.round() - x).detach()`。前向等于 `x.round()`，反向因为括号被 detach 了、梯度直接以系数 1 从 `y` 流到 `x`。但它不带 **clip**（区间外照样透传 1），也没法定制门函数，所以生产代码还是写 `Function`。

Warning. STE 必然通不过 `gradcheck`，这是设计不是 **bug**。`round` 是分段常数函数，数值差分给出的雅可比处处为 0（只要采样点不跨过 .5 边界），而你的解析梯度是 1，永远对不上。所以不要因为 **gradcheck fail** 就去“修”STE。被问到“你怎么验证 STE”，正确答案不是 `gradcheck`，而是：(1) 断言前向严格等于 `torch.round`；(2) 断言区间内梯度按预期透传、区间外为 0；(3) 端到端跑一个小 QAT 看量化后精度是否收敛。

GradientReversal : DANN 的核心

```
class GradientReversal(Function):
    @staticmethod
    def forward(ctx, x, lambd=1.0):
        ctx.lambd = lambd  # float 不能 save_for_backward
        return x.view_as(x)  # 恒等，返回新 view 避免 inplace 告警

    @staticmethod
    def backward(ctx, grad_out):
        return -ctx.lambd * grad_out, None
```



图 40 DANN 的结构。forward 时 GRL 什么也不做；backward 时它把 domain classifier 回传的梯度乘 $-\lambda$ ，于是特征提取器在最大化 domain loss，而 classifier 在最小化它。一次前向、一个 optimizer 就实现了 min-max 对抗。

用途. domain adaptation 的目标是让特征提取器学到的表示分不出样本来自源域还是目标域。做法是接一个 domain classifier 去分辨，然后让特征提取器和它对着干。按 GAN 的思路做要交替更新两个网络、维护两个 optimizer、调交替比例；GRL 把这一切压成一个层：classifier 正常最小化自己的 loss，回传到特征提取器的梯度被翻转，等价于它在最大化同一个 loss。一次 **forward**、一次 **backward**、一个 **optimizer step**。

这是“**backward** 不必是真导数”的最好例证。autograd 只是忠实地按你写的 **backward** 做链式传播，它没有任何机制去校验你返回的东西是不是真的 VJP。理解这一点，就理解了 STE、梯度裁剪、detach 断路里各种“作弊”手法的合法性来源——它们全都是在合法地篡改反向传播。一个有趣的性质可以直接验证这件事：

```
gradcheck(GradientReversal.apply, (x, 1.0), raise_exception=False)    # False: 符号反了
gradcheck(GradientReversal.apply, (x, -1.0))                          # True: 退化成恒等
```

$\lambda = -1$ 时 backward 返回 $+g$ ，正好是恒等函数的真导数，于是 gradcheck 通过了。同一段代码，只因为一个常数就从“数学上错”变成“数学上对”——这恰好反衬出失败的唯一原因就是符号，也说明 gradcheck 检查的是一致性而不是正确性。

λ 通常带 schedule：训练初期设 0（等价于 detach，先让 domain classifier 自己学好），再按 $2/(1 + e^{-10p}) - 1$ （ p 是训练进度）逐渐升到 1。早期就开满会让噪声梯度毁掉特征。

Warning. 常见错法：把 forward 写成 `return -x`。那样前向就变了，domain classifier 看到的是取反后的特征，整个训练目标都不一样了。forward 必须是恒等，改的只有 backward。

什么时候真的需要自定义 Function

面试官问完实现，常会追一句“实际项目里你什么时候会写这个”。只有三类，其他情况都不需要：

1. **backward** 有闭式解，比 **autograd** 逐 **op** 展开更省显存或更快。autograd 会把每个中间量都存下来，而手写 backward 可以只存 `(mean, rstd)` 这种小量、或者用重算换显存。例子：fused LayerNorm / RMSNorm、fused softmax-cross-entropy、FlashAttention。
2. **forward** 里调了不可微的外部代码。autograd 看不进去，只能你自己告诉它梯度是什么。例子：自定义 CUDA / Triton kernel、通信原语（TP 里的 all-reduce）、第三方 C++ 库。
3. 需要“假梯度”，即数学上的真导数没用或不存在。例子：STE（QAT / BNN）、GRL（DANN）、各种梯度手术。

什么时候不需要，这一半更重要。绝大多数情况 autograd 自动展开就够了，而且更不容易错。只是“我觉得手写会快一点”就去写 Function，通常是净亏损：你放弃了 autograd 的正确性保证、放弃了白拿的二阶导、放弃了 `vmap` 兼容，还大概率被 `torch.compile` 甩在后面——Inductor 会把你手工融合的那几个 op 自动融掉。先 profile，确认这一段真的是瓶颈、且瓶颈真的来自 activation 显存或 kernel 数量，再动手。

Key insight. 自定义 Function 的成本是“你从此对这个节点的正确性全权负责”。收益必须能说清是省显存、省 kernel、还是实现了 autograd 表达不出来的语义。说不清就别写——这也是这道题的正确回答方式。

与 `torch.compile` 的关系一句话：`autograd.Function` 会导致 **graph break**。Dynamo 没法 trace 进你的 Python `backward`，只能把图切开，前后两段分别编译，中间回到 eager，跨算子 fusion 的机会就没了。torch 2.4+ 推荐的新路线是 `torch.library.custom_op + register_autograd`：把 op 注册成一个对编译器“不透明但已知”的算子（有 schema、有 `register_fake` 给出的 meta 实现、有单独注册的反向），Dynamo 就能把它当普通 op 放进图里而不 break。见第 7 章（dispatcher 与 op 注册）和第 12 章（graph break）。

面试考点

面试考点. Q1：写一个 `autograd.Function` 要注意什么？`backward` 返回几个值？

A：forward / backward 都是 `staticmethod`，调用用 `.apply()`（`MyOp()(x)` 在 torch 2.x 上直接报错）。**backward** 返回值的个数严格等于 **forward** 输入的个数（不含 `ctx`），非 tensor 参数（float/int/bool）返回 `None` 占位。tensor 一律用 `ctx.save_for_backward` 存，非 tensor 直接挂 `ctx.xxx`。用 `ctx.needs_input_grad` 跳过不必要的计算。forward 里原地改了输入要 `ctx.mark_dirty()`，返回不可微输出要 `ctx.mark_non_differentiable()`。

面试考点. Q2：手推 softmax 的 backward。

A：雅可比是 $dy_i/dx_j = y_i * (\delta_{ij} - y_j)$ ，矩阵形式 $\text{diag}(y) - y y^T$ 。做 VJP： $dx_j = \sum_i g_i y_i (\delta_{ij} - y_j) = y_j (g_j - \sum_i g_i y_i)$ ，向量化就是 $dx = (dy - (dy*y).sum(-1, keepdim=True)) * y$ 。关键是不显式构造雅可比（ (B, S, D, D) 会 OOM），并且 backward 只需要输出 y ——这就是 attention 反向只存 softmax 结果的原因。自检性质： $dx.sum(-1) == 0$ 。常见错法是写成 $dy*y*(1-y)$ ，那是 sigmoid 的导数。

面试考点. Q3：手推 LayerNorm 的 backward。

A：令 $g = dy * w$ ， $xhat = (x - \mu) * rstd$ 。x 影响输出有三条路径（直通、经 μ 、经 var ），三条都算完合并化简，得 $dx = rstd * (g - \text{mean}(g) - xhat * \text{mean}(g * xhat))$ ，两个 mean 都沿归一化维。参数梯度 $dw = \text{sum_batch}(dy * xhat)$ 、 $db = \text{sum_batch}(dy)$ 。几何上这就是把 g 对全 1 方向和 $xhat$ 方向做正交投影——沿这两个方向移动 x 不改变输出，梯度分量必须为 0，所以 $dx.sum(-1)$ 和 $(dx * xhat).sum(-1)$ 都是 0。常见错法是只写 $dx = g * rstd$ ，loss 照样降、只有 gradcheck 抓得到。

面试考点. Q4：什么是 STE？它能够通过 gradcheck 吗？

A：Straight-Through Estimator：前向做 `round / sign` 这类离散化，反向直接把梯度透传（通常还带 $|x| \leq 1$ 的 clip，因为区间外已饱和、透传的是纯噪声）。用在 QAT 和 VQ-VAE 这类离散 latent 上，因为这些算子的真导数几乎处处为 0，不作弊梯度就断了。它必然通不过 **gradcheck**——数值差分对分段常数函数给出的雅可比是 0，解析梯度是 1。这是设计不是 bug，验证要靠“前向严格等于 round + 梯度按预期透传 + 端到端收敛”。

面试考点. Q5 : gradcheck 为什么必须用 float64 ?

A : 中心差分的截断误差是 $O(h^2)$ 、舍入误差是 $O(\text{eps}/h)$, 最优步长下总相对误差约 $\text{eps}^{(2/3)}$ 。fp32 的 eps 是 $1.2\text{e-}7$, 能达到的精度只有 $1\text{e-}5$ 量级, 正好等于 gradcheck 默认的 atol ——数值微分自身的误差就把真实的梯度错误淹没了。fp64 的 eps 是 $2.2\text{e-}16$, 可达 $1\text{e-}11$, 留了六个数量级余量。所以 gradcheck 的输入一律 `dtype=torch.double` 。

面试考点. Q6 : 自定义 Function 什么时候支持二阶导 ?

A : 两个条件 :backward 里全部用可微的 torch 算子写(不碰 `.data` / `.detach()` / numpy / 原地写 buffer), 并且没加 `@once_differentiable` 。满足了就白拿—— `create_graph=True` 时 autograd 会把 backward 里的运算本身也录进图再求一次导。加了 `@once_differentiable` 就是显式声明只支持一阶, 二阶时直接报错(好过静默给错误的 Hessian)。用 `gradgradcheck` 一次验完一阶和二阶。需要二阶导的场景 : WGAN-GP、MAML、Hessian-vector product。

面试考点. Q7 : backward 必须是 forward 的真导数吗 ? 举个反例。

A : 不必须。autograd 只是忠实地按你写的 backward 做链式传播, 引擎没有任何机制校验你返回的是不是真 VJP。反例 : GRL (forward 恒等、backward 乘 $-\lambda$, 用一次 forward 实现 DANN 的 min-max 对抗, 不用像 GAN 那样交替更新)、STE (forward 离散化、backward 透传)。判断标准从“数学上对不对”变成“训练动力学上想要什么”。代价是正确性由你承担——这类 Function 不能用 gradcheck 验, 只能断言“梯度等于我设计的那个值”。

面试考点. Q8 : 什么时候不该写自定义 Function ?

A : 绝大多数时候。只是“觉得手写会快一点”就写通常是净亏损 : 放弃了 autograd 的正确性保证、放弃白拿的二阶导、放弃 `vmap` 兼容, 还会导致 **graph break** (Dynamo 没法 trace 进你的 Python backward), 反而拿不到 Inductor 的 fusion。真正的三个理由是 :backward 有闭式解且能显著省显存(fused LayerNorm)、forward 调了不可微的外部代码、需要假梯度。新代码如果只是想包一个自定义 kernel , 更推荐 `torch.library.custom_op + register_autograd` , 它不会 break 图。

经典模型：MLP / CNN / RNN / Seq2Seq

这一章开始是白板题。面试官不会再问“`view` 和 `reshape` 有什么区别”，他会说：“来，手写一个 ResNet BasicBlock。”这类题的评分点不在“能不能写出来”——大部分候选人都能写个七八成——而在那几个只有真正实现过才知道的细节：ReLU 到底加在残差相加之前还是之后、GRU 的重置门乘在矩阵乘之前还是之后、`h_n` 的形状受不受 `batch_first` 影响。这一章把这些点全部挑出来。

配套的可运行代码 + 权重级对齐测试在 `python/pytorch/interview/test_classic_models.py` (31 个 `pytest`，全部通过)。RNN 部分全都和 `torch.nn` 官方实现做了逐元素数值比对——因为“我能证明我写的和 `nn.LSTMCell` 完全一致”是这道题最有说服力的答案。

本章的代码是白板规模：省掉了 `jaxtyping` 标注和长 `docstring`，逻辑与配套文件一字不差。Transformer 全家桶在第 26 章。

MLP：送分题里的失分点

题目：写一个可配置深度的 MLP，输入 `in_dim`、隐层列表 `hidden_dims`、输出 `out_dim`。

关键考点。这道题真正在考三件事：容器怎么选、激活/norm/dropout 的顺序、以及最后一层能不能加激活。

```
class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dims, out_dim,
                 activation=nn.ReLU, dropout=0.0):
        super().__init__()
        layers = []
        prev = in_dim
        for h in hidden_dims:
            layers.append(nn.Linear(prev, h))
            layers.append(activation())
            if dropout > 0:
                layers.append(nn.Dropout(dropout))
            prev = h
        layers.append(nn.Linear(prev, out_dim))  # 输出层：不激活、不 dropout
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)
```

`nn.Sequential` 还是 `nn.ModuleList`？判据只有一条：**forward** 是不是纯串联。是就用 `Sequential`（自带 `forward`，少写一个循环）；有跳连、多分支、需要拿中间层输出的必须用 `ModuleList`——它只注册子模块、不定义 `forward`，怎么连由你写。

Warning. 用 Python 原生 `list` 存子模块是致命错误：

```
self.layers = [nn.Linear(d, d) for _ in range(4)]  # 参数不会被注册！
```

`nn.Module.__setattr__` 只拦截 `Parameter` / `Module` / `ModuleList` 这些类型，普通 `list` 里的东西对它是不可见的。后果是 `model.parameters()` 收不到它们（optimizer 不更新）

`model.to("cuda")` 搬不动它们 (forward 直接报 device mismatch) `state_dict()` 存不下来。这个 bug 在小网络上表现为“loss 降得很慢”，极易被误判成学习率问题。见第 3 章。

激活放在 **norm** 前还是后？加了 norm 之后的标准顺序是 `Linear → Norm → Act → Dropout`。norm 的作用是把进入非线性之前的分布拉到零均值单位方差，让激活工作在梯度最好的区间；放到激活之后就失去了这个意义。附带一条：此时前一层 `Linear` 的 `bias` 可以省掉，它会被 norm 的减均值完全吃掉。dropout 则放在激活之后——对 ReLU 前后等价（被置零的位置过 ReLU 仍是 0），但换成 GELU / Sigmoid 就不等价了，`sigmoid(0) = 0.5` 相当于给网络注入一个常数偏置。

Warning. 最后一层不加激活也不加 **dropout**，这是这道送分题最常见的失分点。分类头后面接的 `nn.CrossEntropyLoss` 内部已经含 `log_softmax`，它要的是原始 logits（可正可负、无界），再套一层 ReLU 会把所有负 logit 压成 0，模型永远无法表达“这个类不可能”；套一层 `softmax` 则是做了两次 softmax，梯度被压得极小，训练看起来“能动但学不出来”。

怎么自证：断言 `isinstance(m.net[-1], nn.Linear)`，再喂一批随机输入检查输出能取到负值（有激活的话 ReLU 后恒非负，断言会挂）；层序直接比对 `[type(x).__name__ for x in m.net]`。

ResNet BasicBlock：ReLU 加在哪

题目：手写 ResNet-18/34 的残差块，支持 stride 和通道数变化。

关键考点：ReLU 的位置、shortcut 什么时候需要投影、conv 后接 BN 时为什么 `bias=False`。

```
def conv3x3(in_c, out_c, stride=1):
    return nn.Conv2d(in_c, out_c, 3, stride=stride, padding=1, bias=False)

class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_planes, planes, stride=1):
        super().__init__()
        self.conv1 = conv3x3(in_planes, planes, stride)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = conv3x3(planes, planes)
        self.bn2 = nn.BatchNorm2d(planes)

        self.downsample = None
        if stride != 1 or in_planes != planes:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_planes, planes, 1, stride=stride, bias=False),
                nn.BatchNorm2d(planes),
            )

    def forward(self, x):
        identity = x if self.downsample is None else self.downsample(x)
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out)) # 注意：这里没有 relu
        return F.relu(out + identity)
```

Warning. 最高频的错法：在加法之前多写一个 **ReLU**。

```
out = F.relu(self.bn2(self.conv2(out))) # 错
return out + identity
```

后果是残差主分支的输出恒非负，残差只能“加”不能“减”，表达能力直接砍掉一半——网络再也无法学出“把这个通道的响应压下去”。它不会报错，shape 完全正确，loss 也会下降，只是精度上不去。正确顺序是 `relu(bn2(conv2(relu(bn1(conv1(x))))) + shortcut(x))` : **ReLU** 必须在相加之后。

shortcut 什么时候需要 **downsample** : `stride != 1` (空间尺寸变了) 或 `in_planes != planes` (通道数变了) 时，恒等映射的 shape 对不上，必须用 `1×1 conv` (同样的 stride) 把它投影过去。注意这个 `1×1 conv` 也要配一个 **BN**，否则两条分支的数值尺度不匹配，相加后主分支会被淹掉。

Note. 能加分的一句：`1×1 conv + stride=2` 的 **downsample** 丢掉了 **3/4** 的像素 (每 `2×2` 窗口只采左上角那个)。ResNet-D / bag-of-tricks 的改法是先 `2×2 AvgPool` 再 `1×1 conv stride=1`，把被跳过的像素平均进去。这说明你真读过论文而不只是抄过代码。

conv 后接 **BN** 时为什么 **bias=False** : BN 做的是 $(x - \mu) / \sigma \cdot \gamma + \beta$ ，conv 的 bias 是逐通道常数，先被 $x - \mu$ 的减均值项完全抵消，再由 BN 自己的 β 重新提供。它既不改变函数空间，还白占参数和一次访存。ResNet 全网的 conv 都是 **bias=False**。

为什么残差能训深。设块的映射是 $y = F(x) + x$ ，则 $\partial y / \partial x = \partial F / \partial x + I$ 。那个 $+I$ 是关键： L 层堆起来，梯度的展开式里存在一项等于恒等 (把所有 $\partial F / \partial x$ 取零那一项)，梯度可以原样传回最底层；没有它的话梯度是 L 个 Jacobian 连乘，谱范数略小于 1 就指数衰减到 0。“残差提供了一条恒等通路”是标准答案，能顺手写出这个式子会好得多。

怎么自证：把两个 conv 的权重全部置零、BN 设成恒等 (`running_mean=0`、`running_var=1`、eval 模式)，此时主分支恒为 0，block 的输出必须精确等于 `F.relu(x)` ——这一条同时验证了残差连线接对和 ReLU 在加法之后。另一条：随机权重下检查 `bn2(conv2(...))` 的输出能取到负值，证明加法前没有 ReLU。

LSTMCell : 4 个门与两套 bias

题目：手写 `nn.LSTMCell`，要求能直接 `load_state_dict(nn.LSTMCell(...).state_dict())` 并数值对齐。

关键考点：门的公式与顺序、为什么一次 `matmul` 算完 4 个门、为什么有两套 bias。

$i = \sigma(W_{ii}x + b_{ii} + W_{hi}h + b_{hi})$	输入门：新信息写入多少
$f = \sigma(W_{if}x + b_{if} + W_{hf}h + b_{hf})$	遗忘门：旧 cell 保留多少
$g = \tanh(W_{ig}x + b_{ig} + W_{hg}h + b_{hg})$	候选值 (<code>tanh</code> ，不是 <code>sigmoid</code>)
$o = \sigma(W_{io}x + b_{io} + W_{ho}h + b_{ho})$	输出门：cell 露出多少
$c' = f \odot c + i \odot g$	
$h' = o \odot \tanh(c')$	

```
class MyLSTMCell(nn.Module):
    def __init__(self, input_size, hidden_size, bias=True):
```

```

super().__init__()
self.hidden_size = hidden_size
self.weight_ih = nn.Parameter(torch.empty(4 * hidden_size, input_size))
self.weight_hh = nn.Parameter(torch.empty(4 * hidden_size, hidden_size))
if bias:
    self.bias_ih = nn.Parameter(torch.zeros(4 * hidden_size))
    self.bias_hh = nn.Parameter(torch.zeros(4 * hidden_size))
else:  # 声明成 None, state_dict 的键才和官方一致
    self.register_parameter("bias_ih", None)
    self.register_parameter("bias_hh", None)
std = 1.0 / hidden_size ** 0.5  # 官方 RNN 系列的初始化
for p in self.parameters():
    nn.init.uniform_(p, -std, std)

def forward(self, x, state=None):
    if state is None:
        z = x.new_zeros(x.shape[0], self.hidden_size)
        state = (z, z)
    h, c = state
    # 一次算完 4 个门: (B,I)@(I,4H) + (B,H)@(H,4H) -> (B,4H)
    gates = (F.linear(x, self.weight_ih, self.bias_ih)
              + F.linear(h, self.weight_hh, self.bias_hh))
    i, f, g, o = gates.chunk(4, dim=1)
    i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
    g = torch.tanh(g)
    c_next = f * c + i * g
    h_next = o * torch.tanh(c_next)
    return h_next, c_next

```

为什么一次大 **matmul** 而不是 4 次小的。两条理由，面试官通常要听到第二条：算术强度——4 次 $(B, I) @ (I, H)$ 变成 1 次 $(B, I) @ (I, 4H)$ ，小矩阵乘是彻底 memory-bound 的，分开写要把 x 从 HBM 读 4 遍，合并后只读一遍、算术强度提升约 4 倍；**kernel launch**——RNN 要跑 T 个时间步， $T = 100$ 时省下的是几百次 launch，短序列小 batch 下 launch 开销能占到整体一半以上。这就是 `nn.LSTMCell` 的 `weight_ih` 形状是 $(4 * H, I)$ 而不是四个 (H, I) 的原因：行方向按 **i, f, g, o** 顺序拼接。

Warning. 门顺序记成 “ifgo” (**IFGO**)。记错了权重照样能 `load_state_dict` 进去、shape 全对、不报任何错，但结果完全不对——比如把遗忘门当输入门用，网络行为彻底改变。这是“能跑但错”的典型，只有做数值对齐才能发现。

Key insight. PyTorch 有两套 **bias** (`bias_ih` 和 `bias_hh`)，数学上完全冗余。 $b_{ii} + b_{hi}$ 可以合并成一个数，参数量白多一份 $4H$ 。留着纯粹是为了对齐 cuDNN 的 API——cuDNN 分别存输入侧和隐层侧的 bias，PyTorch 的 `nn.LSTM` 直接把它的权重布局透出来了。手写时只写一个 bias，功能上没问题，但和官方做数值对齐时会差一个常数，然后你会花半小时怀疑自己的门顺序。

c 和 **h** 的区别。**c** 是“内部长期记忆”，只被逐元素的加法和乘法更新，从不过矩阵乘；**h** 是“对外输出”，是 **c** 经 `tanh` 压缩再被输出门筛选的结果。维度相同但角色完全不同：下一层和输出层看到的是 **h**，**c** 只在时间维上流动。

为什么 **LSTM** 能缓解梯度消失。cell state 的更新是 $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ ，对 c_{t-1} 求导得 $\partial c_t / \partial c_{t-1} = \text{diag}(f_t)$ ——逐元素相乘，不是矩阵乘。跨 T 步的梯度是 $\prod_t f_t$ ，只要遗忘门保持接近 1，梯度就能近似无损地沿 c 传回去，和残差连接的 $+I$ 是同一个思路。而原始 RNN 的 $\partial h_t / \partial h_{t-1} = W_{hh}^T \text{diag}(\tanh')$ 是 T 个矩阵连乘， W_{hh} 的谱半径稍微偏离 1 就指数爆炸或消失。

Note. 工程 trick：把遗忘门的 bias 初始化成 1 (bias_ih 的第 2 段填 1)，让训练早期 $f \approx 1$ ，cell state 默认“记住”。PyTorch 默认没这么做，但 Jozefowicz et al. 2015 系统验证过这个 trick 有效。

怎么自证：mine.load_state_dict(nn.LSTMCell(I,H).state_dict()) 之后喂同一个 (x, h0, c0)，torch.testing.assert_close 比对 h' 和 c'。验门顺序可以更狠：把 weight 全置零、只用 bias_ih 把四段分别设成 [+20, -20, +20, +20] (sigmoid(± 20) 已经饱和到 1/0)，则 $i = 1, f = 0, g \approx 1, o = 1$ ，于是 $c' = 1, h' = \tanh(1)$ 。顺序记错的话这两个值都对不上。

LSTM：沿时间步展开，以及 h_n 的形状陷阱

题目：用上面的 cell 实现单层单向 LSTM，支持 batch_first。

```
class MyLSTM(nn.Module):
    def __init__(self, input_size, hidden_size, batch_first=False):
        super().__init__()
        self.hidden_size = hidden_size
        self.batch_first = batch_first
        self.cell = MyLSTMCell(input_size, hidden_size)

    def forward(self, x, state=None):
        if self.batch_first:
            x = x.transpose(0, 1)          # (B,T,I) -> (T,B,I)
            t, b, _ = x.shape
        if state is None:
            h = x.new_zeros(b, self.hidden_size)
            c = x.new_zeros(b, self.hidden_size)
        else:
            h, c = state[0][0], state[1][0]  # 去掉 num_layers 维

        outputs = []
        for step in range(t):
            h, c = self.cell(x[step], (h, c))
            outputs.append(h)
        output = torch.stack(outputs, dim=0)  # (T,B,H)

        if self.batch_first:
            output = output.transpose(0, 1)
        # h_n / c_n 始终是 (num_layers, B, H)
        return output, (h.unsqueeze(0), c.unsqueeze(0))
```

batch_first=False 为什么是默认值——不是反人类，是内存布局。循环里每步要取 $x[\text{step}]$ ，(T,B,I) 布局下这是一整块连续内存，直接喂给 GEMM；(B,T,I) 下每个时间步都是跨 stride 的切片，GEMM 前得先物化一次。所以实现上先 transpose 成 (T,B,I) 再循环，比在循环里反复做非连续索引快。

Key insight. `h_n` / `c_n` 的形状不受 `batch_first` 影响，恒为 `(num_layers * num_directions, B, H)`。只有 `output` 会在 `(T,B,H)` 和 `(B,T,H)` 之间切换。这个不一致坑过所有人——写 `h_n[0]` 想拿第一个样本，实际拿到的是第一层。这是本题最常被追问的一点。

Warning. 时间步之间有严格的串行依赖，这是 RNN 打不过 Transformer 的根本原因： T 步无法并行，GPU 利用率被时间维锁死。cuDNN 的优化手段是把“输入侧的 $W_{i*}x$ ”在所有时间步上一次性算完（它不依赖 h ），只把 $W_{h*}h$ 留在循环里——这是很好的追问点，因为它解释了为什么 `nn.LSTM` 比你手写的 `for` 循环快好几倍，而不是靠什么魔法。

Note. 双向 LSTM 不是“反着再跑一遍就完事”：反向那一支的 `h_n` 取的是序列起点那一步的状态；而且变长序列必须配 `pack_padded_sequence`，否则反向会从 padding 开始跑，把 pad 的信息混进真实 token。

怎么自证：和 `nn.LSTM(I, H, num_layers=1)` 逐权重对拷（`ref.weight_ih_l0` 等四个），比对 `output`、`h_n`、`c_n` 三个返回值；再断言 `out[:, -1] == h_n[0]`，最后一个时间步的输出就是 `h_n`，这条恒等式在单向单层下必须成立。

GRUCell：重置门乘在哪

题目：手写 `nn.GRUCell` 并与官方数值对齐。

$$\begin{aligned} r &= \sigma(W_{ir}x + b_{ir} + W_{hr}h + b_{hr}) && \text{重置门} \\ z &= \sigma(W_{iz}x + b_{iz} + W_{hz}h + b_{hz}) && \text{更新门} \\ n &= \tanh(W_{in}x + b_{in} + r \odot (W_{hn}h + b_{hn})) && \text{候选状态} \\ h' &= (1 - z) \odot n + z \odot h \end{aligned}$$

```
class MyGRUCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.hidden_size = hidden_size
        self.weight_ih = nn.Parameter(torch.empty(3 * hidden_size, input_size))
        self.weight_hh = nn.Parameter(torch.empty(3 * hidden_size, hidden_size))
        self.bias_ih = nn.Parameter(torch.zeros(3 * hidden_size))
        self.bias_hh = nn.Parameter(torch.zeros(3 * hidden_size))
        std = 1.0 / hidden_size ** 0.5
        for p in self.parameters():
            nn.init.uniform_(p, -std, std)

    def forward(self, x, h=None):
        if h is None:
            h = x.new_zeros(x.shape[0], self.hidden_size)
        gi = F.linear(x, self.weight_ih, self.bias_ih) # (B, 3H)
        gh = F.linear(h, self.weight_hh, self.bias_hh) # (B, 3H)
        i_r, i_z, i_n = gi.chunk(3, dim=1)
        h_r, h_z, h_n = gh.chunk(3, dim=1)
```

```

r = torch.sigmoid(i_r + h_r)
z = torch.sigmoid(i_z + h_z)
n = torch.tanh(i_n + r * h_n)      # 关键：r 乘的是 (W_hn h + b_hn) 整体
return (1 - z) * n + z * h

```

Key insight. 本题唯一也是最大的考点： r 到底乘在矩阵乘之前还是之后。原论文 (Cho et al. 2014) 写的是 $n = \tanh(W_{in}x + W_{hn}(r \odot h))$ ——先把 r 作用在 h 上，再做矩阵乘。PyTorch / cuDNN 实现的是 $n = \tanh(W_{in}x + b_{in} + r \odot (W_{hn}h + b_{hn}))$ —— r 乘在矩阵乘之后，而且把 b_{hn} 也一起乘进去了。动机纯粹是性能：这样 $W_{hn}h + b_{hn}$ 就能和 r 、 z 两个门的隐层项合并成一次 $(B,H) @ (H,3H)$ 的 GEMM；按原论文写， n 分支必须等 r 算出来才能做它自己的矩阵乘，被迫拆成两次 GEMM 且串行。因为 r 是逐元素的， $r \odot (Wh) \neq W(r \odot h)$ ，两者数学上不等价。实际效果差异很小，但做权重对齐时会直接对不上。

Warning. 第二个坑是更新门的方向。PyTorch 的 $h' = (1 - z)n + zh$ 里， z 是“保留旧状态”的比例： $z \rightarrow 1$ 时 $h' = h$ 。有些教材和框架写成 $zn + (1 - z)h$ ，语义正好相反。权重能加载进去，行为完全不同。

另外两点：GRU 只有 3 组权重（`weight_ih` 是 $(3H, I)$ ），参数量是 LSTM 的 3/4；没有独立的 cell state，长期记忆和输出合并成一个 h ，短序列上和 LSTM 基本打平、超长依赖上通常略逊。

怎么自证：`load_state_dict(nn.GRUCell(...).state_dict())` 后逐元素对齐。更能体现理解的是把两种写法都算出来：用官方的 `weight_hh[2H:]` 手动构造 `paper_style = tanh(i_n + F.linear(r * h, w_hn, b_hn))`，断言它不等于 `ref(x, h)` 而 PyTorch 写法相等。这一条测试就是这道题的分水岭。

Bahdanau vs Luong : Transformer 之前的 attention

题目：实现加性 (additive) 和乘性 (multiplicative) 两种 attention，支持 padding mask。

打分函数的对比就是这道题的全部：

$$\text{Bahdanau (additive): } \text{score}(q, k) = v^T \tanh(W_q q + W_k k)$$

$$\text{Luong (general): } \text{score}(q, k) = q^T W k$$

$$\text{Luong (dot): } \text{score}(q, k) = q^T k$$

```

class BahdanauAttention(nn.Module):
    def __init__(self, query_dim, key_dim, attn_dim):
        super().__init__()
        self.w_q = nn.Linear(query_dim, attn_dim, bias=False)
        self.w_k = nn.Linear(key_dim, attn_dim, bias=False)
        self.v = nn.Linear(attn_dim, 1, bias=False)

    def forward(self, query, keys, values=None, mask=None):
        if values is None:
            values = keys
        # (B,Tq,1,A) + (B,1,Tk,A) -> (B,Tq,Tk,A)

```

```

feats = torch.tanh(self.w_q(query).unsqueeze(2)
                  + self.w_k(keys).unsqueeze(1))
scores = self.v(feats).squeeze(-1)          # (B,Tq,Tk)
return _attend(scores, values, mask)

class LuongAttention(nn.Module):
    def __init__(self, query_dim, key_dim, scaled=False):
        super().__init__()
        self.w = nn.Linear(query_dim, key_dim, bias=False)
        self.scaled, self.key_dim = scaled, key_dim

    def forward(self, query, keys, values=None, mask=None):
        if values is None:
            values = keys
        scores = self.w(query) @ keys.transpose(-1, -2)  # (B,Tq,Tk)
        if self.scaled:
            scores = scores / self.key_dim ** 0.5
        return _attend(scores, values, mask)

    def _attend(scores, values, mask):
        if mask is not None:          # mask: (B,Tk), True = 有效
            scores = scores.masked_fill(~mask.unsqueeze(1), float("-inf"))
        weights = torch.softmax(scores, dim=-1)
        return weights @ values, weights  # (B,Tq,Dv), (B,Tq,Tk)

```

参数量与计算量的差异，这是选择的依据：

	Bahdanau (加性)	Luong (乘性)
参数	W_q, W_k, v , 共 $(D_q + D_k)A + A$	W , 共 $D_q D_k$
中间张量	(B, T_q, T_k, A)	(B, T_q, T_k)
算子	broadcast add + tanh + 一次 $(\cdot, 1)$ 投影	两次 batched matmul
$D_q \neq D_k$	天然支持	靠 W 桥接
硬件友好度	elementwise 为主, memory-bound	纯 GEMM, 能打满 tensor core

关键差别是那个多出来的 A 维：加性注意力必须显式物化 (B, T_q, T_k, A) 的中间张量，显存和访存都是乘性版的 A 倍。Transformer 选乘性，论文原话就是“乘性在实践中快得多且省显存”——因为它能直接调用高度优化的 matmul。

Note. 一个反直觉的点： d_k 很大时加性反而更稳。点积的方差随 d_k 线性增长，不缩放的话 softmax 会饱和；加性因为外面套了 tanh，打分天然被压在 $[-1, 1]$ ，没有这个问题。Transformer 的解法不是换回加性，而是乘性 $+ 1/\sqrt{d_k}$ 缩放——推导见第 26 章。Luong 原文没有这一项，因为当年 d_k 只有几百且输入是被 tanh 压过的 RNN 输出，问题不明显。

mask 的处理有两条规则。一是在 **softmax** 之前把无效位置的 score 设成 $-\infty$ ，softmax 之后自然是精确的 0；如果 softmax 之后再乘 0，剩下的权重就不再和为 1 了，等于给不同样本用了不同的温度。二是 mask 形状为 (B, Tk) ，`unsqueeze(1)` 成 $(B, 1, Tk)$ 对所有 query 位置广播——padding 是 key 侧的属性，与 query 无关。

Warning. -inf 的陷阱：某一行被全部 mask 掉时（比如 batch 里混进一个长度为 0 的样本），softmax(全 -inf) 是 $\exp(-\text{inf})$ 全 0 除以 0，结果是 NaN，然后 NaN 顺着反向传播污染掉整个模型的梯度。两个解法：要么在 dataloader 侧保证每条序列至少一个有效位置，要么用 `torch.finfo(scores.dtype).min` 代替 -inf（全 mask 行会得到均匀分布，是无意义但有限的值）。配套测试里专门留了一个 `test_attention_fully_masked_row_produces_nan` 断言 NaN 会出现——把坑固化成反面教材，比在注释里写一句提醒有用。

怎么自证：断言 `weights.sum(-1) == 1`；断言 masked 位置的权重精确为 0（不是“很小”）；最强的一条是把 padding 位置的 value 换成 `1e4` 这种大数，检查 context 完全不变——这证明 mask 真的挡住了信息流，而不只是让权重看起来小。

Seq2Seq : encoder + decoder + attention

题目：用上面的组件拼一个带 attention 的 seq2seq，支持 teacher forcing 和 source mask。

```
class Seq2Seq(nn.Module):
    def __init__(self, src_vocab, tgt_vocab, emb_dim, hidden_dim):
        super().__init__()
        self.src_emb = nn.Embedding(src_vocab, emb_dim)
        self.tgt_emb = nn.Embedding(tgt_vocab, emb_dim)
        self.encoder = MyLSTM(emb_dim, hidden_dim, batch_first=True)
        self.decoder_cell = MyLSTMCell(emb_dim, hidden_dim)
        self.attention = BahdanauAttention(hidden_dim, hidden_dim, hidden_dim)
        self.out = nn.Linear(hidden_dim * 2, tgt_vocab)  # 吃 [h ; context]

    def forward(self, src, tgt, src_mask=None):
        enc_out, (h_n, c_n) = self.encoder(self.src_emb(src))  # (B,Ts,H)
        h, c = h_n[0], c_n[0]  # 桥接初始状态

        tgt_emb = self.tgt_emb(tgt)  # (B,Tt,E)
        logits, attns = [], []
        for t in range(tgt.shape[1]):
            h, c = self.decoder_cell(tgt_emb[:, t], (h, c))  # teacher forcing
            context, attn = self.attention(h.unsqueeze(1), enc_out, mask=src_mask)
            logits.append(self.out(torch.cat([h, context.squeeze(1)], dim=-1)))
            attns.append(attn.squeeze(1))
        return torch.stack(logits, dim=1), torch.stack(attns, dim=1)
```

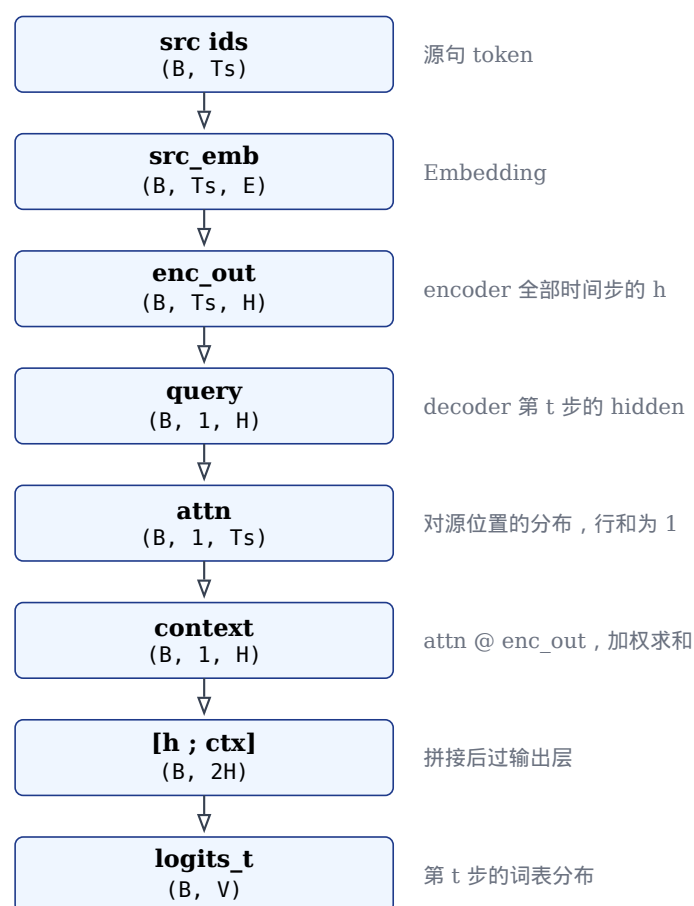


图 41 seq2seq 的 shape 流。decoder 每步都要重跑虚线以下的四步， T_t 步串行——这正是 Transformer decoder 用 causal mask 一次算完、而 RNN decoder 做不到的地方。

decoder 为什么必须逐步展开，而 encoder 可以一把跑完？因为每一步的 attention query 是上一步的 **decoder hidden state**，而 encoder 的输入是已知的完整源句，没有这个依赖。这正是 RNN decoder 训练慢的根源：Transformer decoder 训练时所有位置的 query 都来自已知的 target 序列，配上 causal mask 可以一次算完 T_t 步。

teacher forcing 与 **exposure bias**。训练时喂真实的上一个 token（代码里的 `tgt_emb[:, t]`），推理时只能喂模型自己的预测。两者输入分布不一致，这就是 **exposure bias**：模型从没见过“自己犯了错之后的状态”，推理早期一偏就一路崩下去。常见缓解手段是 scheduled sampling（训练中按概率混入自己的预测）。反过来不用 teacher forcing 的话训练初期几乎学不动，因为随机的预测提供不了有效梯度。

decoder 的初始 **hidden** 从哪来。这里用 encoder 的最后状态 `h_n[0]`，叫“桥接”。但有个反直觉的事实：有了 **attention** 之后直接零初始化也能训得差不多好——源句信息主要通过 attention 流过去，而不是挤在这个定长向量里；attention 诞生的动机本来就是“不要把整个源句压进一个定长向量”。桥接更多是历史惯例加一点热启动的好处。

context 怎么用：这里把 context 和 decoder hidden 拼起来过输出层（Luong 的做法）；Bahdanau 的做法是把 context 拼进下一步的 decoder 输入，让 context 参与状态更新，理论上更强但实现更绕。

Warning. `src_mask` 必须传到 **attention**。变长 batch 里如果 padding 位置参与了 softmax，模型会学会去看 padding——而 padding 的分布在训练集和验证集里可能完全不同（比如验证集句子更长、padding 更少），于是验证指标和训练时对不上，而且这种偏差随 batch 组成随机波动，非常难排查。

怎么自证：`logits.shape == (B, Tt, V)`、`attns.sum(-1) == 1`；传 mask 后断言 `attns[:, :, valid_len:]` 精确为 0；最后跑一次 `cross_entropy(...).backward()`，检查每一个 named parameter 都拿到了有限梯度——seq2seq 组件多，最容易出的问题是某个模块因为写错连线而完全没参与 forward。

RNN 家族对比

	门数	状态	参数量 (weight + bias)	适用
RNN	0	<code>h</code>	$HI + H^2 + 2H$	几乎不用了，梯度消失严重
LSTM	4	<code>h, c</code>	$4(HI + H^2 + 2H)$	长依赖、需要显式记忆通路
GRU	3	<code>h</code>	$3(HI + H^2 + 2H)$	参数敏感场景、小数据集

表里的“门数”按 PyTorch 的权重布局口径数（候选值 g/n 也算一路，因为它在 `weight_ih` 里占一段）；严格说 LSTM 是 3 个门 + 1 个候选，GRU 是 2 个门 + 1 个候选。参数量的推导很简单：每一路需要一个 (H, I) 的输入侧权重、一个 (H, H) 的隐层侧权重、两个长度 H 的 bias，乘上路数就是全部。所以 GRU 的参数量恰好是 LSTM 的 $3/4$ ，这是个常被要求口算的数。

面试考点

面试考点. Q1：ResNet BasicBlock 里 ReLU 加在残差相加之前还是之后？写错了会怎样？

A：之后，`relu(bn2(conv2(...)) + shortcut(x))`。加在相加之前的话主分支输出恒非负，残差就只能加不能减，网络无法学出“抑制某个通道”，表达能力砍半。它不报错、shape 全对、loss 也会降，只是精度上不去，属于最难发现的一类错。

面试考点. Q2：conv 后面接 BN 时为什么要 `bias=False`？

A：BN 是 $(x - \mu) / \sigma \cdot \gamma + \beta$ ，conv 的 bias 是逐通道常数，先被减均值项完全抵消，再由 BN 的 β 重新提供。所以它既不扩大函数空间，还多占参数和一次访存。同理 `Linear` → `LayerNorm` 时 `Linear` 的 bias 也可以省。

面试考点. Q3：为什么残差连接能训很深的网络？

A： $\partial y / \partial x = \partial F / \partial x + I$ ，那个 $+I$ 给梯度留了一条恒等通路， L 层堆起来梯度里始终存在一项等于恒等。没有它的话梯度是 L 个 Jacobian 连乘，谱范数略小于 1 就指数衰减。LSTM 的 cell state ($\partial c_t / \partial c_{t-1} = \text{diag}(f_t)$) 是同一个思路的时间维版本。

面试考点. Q4：LSTM 为什么把 4 个门拼成一次 matmul？

A：两点。一是算术强度：4 次 $(B, I) @ (I, H)$ 要把 x 从 HBM 读 4 遍，合成 1 次 $(B, I) @ (I, 4H)$ 后只读一遍，小 GEMM 本来是 memory-bound 的，这一改直接提升 4 倍强度。二是 kernel launch： T 个时间步 × 省 3 次 launch，短序列小 batch 下 launch 开销能占一半。这也是 `nn.LSTMCell.weight_ih` 形状是 $(4H, I)$ 、按 i/f/g/o 顺序拼接的原因。

面试考点. Q5：`nn.LSTMCell` 为什么有 `bias_ih` 和 `bias_hh` 两套 bias？

A：数学上完全冗余， $b_{ii} + b_{hi}$ 可以合并成一个。留着纯粹是为了对齐 cuDNN 的 API (cuDNN 分别存输入侧和隐层侧的 bias)，是历史包袱。实际影响：手写版只写一个 bias 的话，功能没问题但和官方做数值对齐会差一个常数。

面试考点. Q6：nn.LSTM 的 batch_first=True 会改变哪些返回值的形状？

A：只有 **output**，从 (T,B,H) 变成 (B,T,H)。h_n 和 c_n 恒为 (num_layers * num_directions, B, H)，不受影响。这个不一致性的后果是有人写 h_n[0] 想拿第一个样本，实际拿到的是第一层。默认 batch_first=False 是因为循环里取 x[t] 在 time-first 布局下是连续内存。

面试考点. Q7：PyTorch 的 GRU 和原论文有什么区别？为什么？

A：重置门 **r** 的作用位置。论文是 $\tanh(W_{in}x + W_{hn}(r \odot h))$ ，PyTorch/cuDNN 是 $\tanh(W_{in}x + b_{in} + r \odot (W_{hn}h + b_{hn}))$ ——**r** 乘在矩阵乘之后，还把 b_{hn} 一起乘了进去。动机是让三路 gate 的隐层项合并成一次 (B,H)@(H,3H) GEMM；按论文写，**n** 分支得等 **r** 算完才能做矩阵乘，被迫拆两次且串行。因为 **r** 是逐元素的，两者数学上不等价，做权重对齐会直接对不上。

面试考点. Q8：Bahdanau 和 Luong attention 的区别？Transformer 为什么选后者？

A：打分函数不同。Bahdanau 是加性 $v^T \tanh(W_q q + W_k k)$ ，要物化 (B,Tq,Tk,A) 的中间张量；Luong 是乘性 $q^T W k$ ，只要 (B,Tq,Tk)。Transformer 选乘性是因为它能直接落到高度优化的 matmul 上，硬件友好度天差地别。代价是点积的方差随 d_k 增长会让 softmax 饱和，所以补了 $1/\sqrt{d_k}$ ；加性因为有 tanh 压着，天然没这个问题。

面试考点. Q9：为什么 Transformer 取代了 RNN？

A：核心是并行性，不是表达能力。RNN 的 h_t 依赖 h_{t-1} ， T 步严格串行，GPU 利用率被时间维锁死，且任意两个位置的信息要走 $O(T)$ 步才能相遇（梯度也要穿过 $O(T)$ 个 Jacobian）。Transformer 训练时所有位置一次算完，任意两位置的路径长度是 $O(1)$ ，唯一代价是 attention 的 $O(S^2)$ 复杂度——在 S 不太大时换来的并行度和硬件利用率完全值得。这也解释了为什么 RNN 在推理上反而有优势：它的状态是 $O(1)$ 的，而 Transformer 的 KV cache 是 $O(S)$ ，这是 Mamba / RWKV 这条线的出发点。

Transformer 全家桶：从 attention 到 generate

这是全书最重要的一章。“手写 multi-head attention”是 LLM 相关岗位的必考题，“手写 RoPE”、“实现 KV cache”、“写一个 top-p 采样”紧随其后。这些题的共同特点是看起来简单、有一堆静默出错的坑：reshape 顺序写反不报错但学不出来、mask 语义取反会让 loss 降得离奇地快、KV cache 的位置偏移错了只在长序列上才暴露。

配套的可运行代码在 `python/pytorch/interview/test_transformer.py` (32 个 pytest, 全部通过), 每个实现都和 `F.scaled_dot_product_attention` / `nn.MultiheadAttention` / `nn.RMSNorm` 做了数值对齐——“我能证明我写的和官方逐元素相同”是这类题最有力的答案。

全文的 `mask` 约定：`bool mask` 里 `True` = 可见, `False` = 屏蔽。这和 `F.scaled_dot_product_attention` 一致, 和 `nn.MultiheadAttention` 相反 (后者 `True` = 屏蔽), 后面会专门讲这个坑。

Scaled dot-product attention

题目：手写 `F.scaled_dot_product_attention`, 支持 `bool / float mask`、`is_causal`、`dropout`。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

```
def scaled_dot_product_attention(query, key, value, attn_mask=None,
                                dropout_p=0.0, is_causal=False, training=True):
    d_k = query.shape[-1]
    # (B,H,Sq,D) @ (B,H,D,Sk) -> (B,H,Sq,Sk)
    logits = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)

    if is_causal:
        assert attn_mask is None, "is_causal 与 attn_mask 只能二选一"
        s_q, s_k = query.shape[-2], key.shape[-2]
        # decode 阶段 Sq < Sk, query 要对齐到序列末尾
        attn_mask = causal_mask(s_q, s_k, start_pos=s_k - s_q, device=query.device)

    if attn_mask is not None:
        if attn_mask.dtype == torch.bool:
            logits = logits.masked_fill(~attn_mask, float("-inf"))
        else:
            logits = logits + attn_mask  # float mask 按加性 bias 处理

    attn = torch.softmax(logits, dim=-1)
    if dropout_p > 0.0 and training:
        attn = F.dropout(attn, p=dropout_p, training=True)

    return torch.matmul(attn, value), attn
```

为什么要除以 $\sqrt{d_k}$

这是本章最高频的追问，而且面试官想听的是推导，不是“为了数值稳定”。

设 $q, k \in \mathbb{R}^{d_k}$ 的各分量独立同分布、均值 0、方差 1（这正是常见初始化 + LayerNorm 之后的近似状态），内积是 d_k 项之和：

$$\begin{aligned} q \cdot k &= \sum_{i=1}^{d_k} q_i k_i, \quad \mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0 \\ \text{Var}(q_i k_i) &= \mathbb{E}[q_i^2 k_i^2] = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1 \\ \text{Var}(q \cdot k) &= \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k, \quad \sigma = \sqrt{d_k} \end{aligned}$$

所以 logits 的标准差随 $\sqrt{d_k}$ 增长： $d_k = 128$ 时典型幅度已经是 ± 11 ，softmax 里最大项和次大项能差好几个 e 的量级，分布接近 one-hot。

关键在下一步：**softmax** 饱和时梯度会消失。它的 Jacobian 是 $\text{diag}(p) - pp^T$ ，迹为 $\sum_i p_i(1 - p_i)$ ——这个量就是“能往下游传的梯度通道量”， p 接近 one-hot 时趋于 0， p 接近均匀时趋于上界 $1 - 1/S$ 。

$$\text{tr}(J_{\text{softmax}}) = \sum_i p_i(1 - p_i) \rightarrow \begin{cases} 0 & \text{one-hot (饱和, 梯度死)} \\ 1 - 1/S & \text{均匀 (梯度最健康)} \end{cases}$$

除以 $\sqrt{d_k}$ 把 logits 方差拉回 1，softmax 就停在梯度良好的区域。配套测试把这个论断量化了： $d_k = 256$ 、64 个 key 的随机 q/k 下，不缩放时注意力分布的熵 < 0.5 （上界是 $\log 64 \approx 4.16$ ），Jacobian 迹 < 0.25 ；缩放后熵 $> 0.8 \log 64$ 、迹 > 0.9 ——梯度通道量差了 5 倍以上。

Warning. 两个常见错法：除以 d_k （过度缩放，logits 全被压到 0 附近，attention 永远接近均匀，学不出选择性）；除以 $\sqrt{d_{\text{model}}}$ （当 $d_{\text{model}} = n_{\text{head}} \times d_{\text{head}}$ 时缩放过头 $\sqrt{n_{\text{head}}}$ 倍），必须是每个 **head** 的维度 d_{head} 。

mask 用 `-inf` 还是 `finfo.min`

Warning. 整行被 **mask** 掉时 `-inf` 会产生 **NaN**。softmax 对全 $-\infty$ 的一行算的是 $\exp(-\infty) = 0$ 全零，再除以和 0，得到 $0/0 = \text{NaN}$ 。然后 NaN 顺着反向传播把整个模型的梯度污染成 NaN，而 loss 曲线上看起来就是“训到某一步突然变 nan”。

什么时候会遇到整行被 mask？padding mask 碰上“整条序列都是 pad”的样本（变长 batch + 数据清洗不干净）；或者 cross-attention 里源句为空。生产代码的两个选择：

- 保证每行至少一个可见位置（在 dataloader 侧过滤空序列，或强制 pad 位置的第 0 列可见）；
- 用 `torch.finfo(logits.dtype).min` 代替 `-inf`。全 mask 行会得到均匀分布——是无意义的值，但有限，不会污染梯度。

另一个相关的错法是用魔数 `-1e4`。它在 fp16 里是可表示的“安全值”，但如果真实 logits 也在 10^4 量级，被 mask 的位置不会被完全屏蔽，信息悄悄泄漏。要么 `-inf`，要么 `finfo.min`，不要自己编数。

dropout 加在哪，以及怎么自证

dropout 加在 softmax 之后、乘 V 之前，即随机丢掉一部分 attention 权重。PyTorch 的 inverted dropout 会除以 $1-p$ ，推理时什么都不用做。常见错法是加在 logits 上——那等价于随机屏蔽 key（被丢的概率会重新分配给其他 key），不是“减弱”而是“删除”，语义完全不同。

怎么自证：无 mask、is_causal、bool mask、float mask 四条路径分别和 `F.scaled_dot_product_attention` 比对；断言 `attn.sum(-1) == 1`；causal 下断言严格上三角精确为 0（`attn.triu(diagonal=1).abs().max() == 0.0`，不是“很小”）。

MultiHeadAttention：那个静默出错的 reshape

题目：手写 MHA，要求能加载 `nn.MultiheadAttention` 的权重并数值对齐。

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_head, dropout=0.0, bias=True):
        super().__init__()
        assert d_model % n_head == 0
        self.d_model, self.n_head = d_model, n_head
        self.d_head = d_model // n_head
        self.dropout = dropout
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=bias) # fused
        self.out_proj = nn.Linear(d_model, d_model, bias=bias)

    def forward(self, x, attn_mask=None, is_causal=False):
        b, s, _ = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1) # 每个 (B,S,H)

        # 唯一正确的 split-head 写法
        q = q.view(b, s, self.n_head, self.d_head).transpose(1, 2)
        k = k.view(b, s, self.n_head, self.d_head).transpose(1, 2)
        v = v.view(b, s, self.n_head, self.d_head).transpose(1, 2)

        out, _ = scaled_dot_product_attention(
            q, k, v, attn_mask=attn_mask, dropout_p=self.dropout,
            is_causal=is_causal, training=self.training)

        # merge: 先 transpose 回去, 再 contiguous 才能 view
        out = out.transpose(1, 2).contiguous().view(b, s, self.d_model)
        return self.out_proj(out)
```

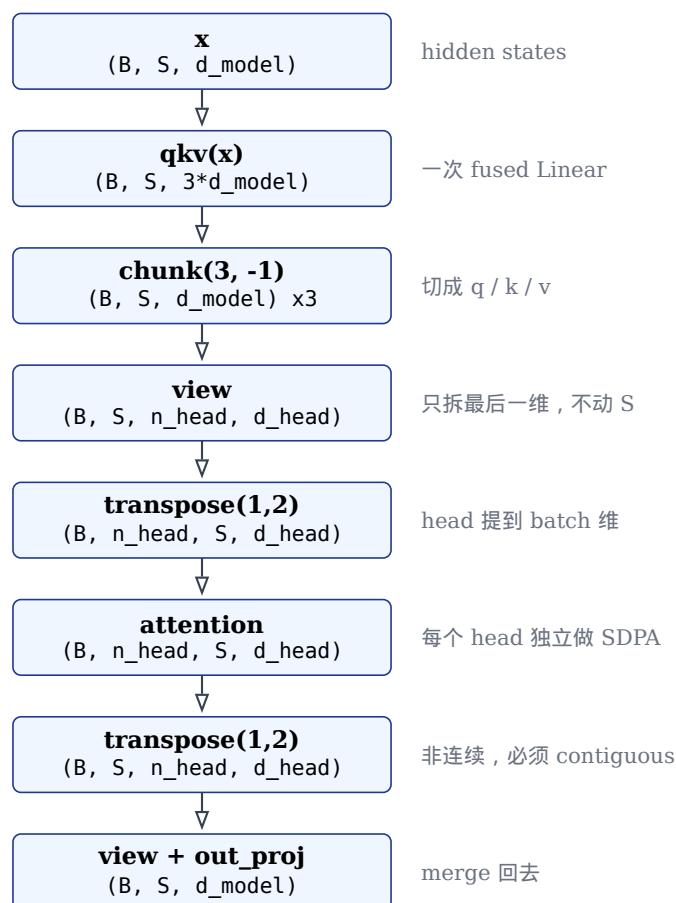


图 42 MHA 的 shape 流。第 4→5 步必须是 `view` 再 `transpose` 两步，不能一步 `view(B, n_head, S, d_head)`。

Warning. 全章最经典的 bug：

```
q = q.view(b, self.n_head, s, self.d_head) # 错！但 shape 完全正确
```

Linear 输出最后一维的内存排布是 `[head0_d0 ... head0_d(dh-1), head1_d0, ...]`，即 **head** 维在 **d_head** 维外侧、在 **S** 维内侧。正确写法只切最后一维，语义是“把每个 token 的 hidden 分给各个 head”。错误写法把 `S * d_model` 这段连续内存重新切成 `n_head * (S * d_head)`，语义变成“第 0 个 head 拿走了前 S/n_{head} 个 token 的全部通道”——相邻 hidden 维被错切给不同 head，序列也被切碎。

它之所以经典：**shape** 完全正确、不报任何错、**loss** 也会降一点（毕竟还是个可微变换），只是永远学不到该有的水平。配套测试用 `wrong[0,0].reshape(-1) == x[0].reshape(-1)[:s*dh]` 把它钉死，直接展示错法拿到的是哪一段内存。

merge 回去时同理：`out.transpose(1,2)` 之后 stride 不连续，少了 `.contiguous()` 会让 `view` 直接报错。这反而是好事——它至少会响。用 `reshape` 能绕过报错，但那是隐式拷贝，不如显式写清楚。

fused qkv 的好处：一次 GEMM 代替三次，kernel launch 少 2/3，且 $M/N/K$ 更大更容易打满 tensor core。代价是加载别人的预训练权重时要自己拼 `cat([Wq, Wk, Wv], dim=0)`，顺序必须是 q, k, v。

Key insight. 考点：`nn.MultiheadAttention` 的权重是怎么存的。当 `embed_dim == kdim == vdim`（默认情况）时，它把三个投影打包成一个 `in_proj_weight`，形状 `(3E, E)`，行方向按 `[Wq; Wk; Wv]` 顺序拼接（`in_proj_bias` 同理是 `(3E,)`），此时 `q_proj_weight / k_proj_weight / v_proj_weight` 全是 `None`。只有 `kdim / vdim` 与 `embed_dim` 不同时才拆成三个独立的 `*_proj_weight`，那时 `in_proj_weight` 是 `None`。所以如果你的 `self.qkv` 也是 `Linear(E, 3E)`，可以整块 `copy_(ref.in_proj_weight)` 直接对齐；写权重迁移脚本时不知道这个分支逻辑，就会在 `q_proj_weight is None` 上翻车。

Warning. 考点：两套 API 的 **bool mask** 语义正好相反。

- `F.scaled_dot_product_attention(attn_mask=bool)`：**True** = 保留
- `nn.MultiheadAttention(attn_mask=bool)`：**True** = 屏蔽

所以同一个 causal 约束，一边要传 `tril`（下三角 True），另一边要传 `triu(diagonal=1)`（上三角 True）。迁移时忘了取反，得到的是反向 **causal**：每个位置只能看未来、看不到过去。这个事故的特征非常好记——**loss** 降得离奇地快，因为位置 t 能看到 $t+1$ 的 token，而训练目标恰好就是预测 $t+1$ ，模型直接在偷看答案。开发集指标也很好（同样的 mask），只有真正自回归生成时才发现全是胡言乱语。看到“loss 比论文低一大截”，第一件事就是查 mask 方向。

怎么自证：`mine.qkv.weight.copy_(ref.in_proj_weight)` 等四行对拷之后，双方 `eval()`，比对 `ref(x, x, x, need_weights=False)[0]` 和 `mine(x)`；causal 情况用上面说的 `triu / tril` 分别传，同样要逐元素相等。

MHA / MQA / GQA：一条连续的谱

三者的区别只有一个数字：`n_kv_head`。

	<code>n_kv_head</code>	KV cache 相对大小	代表模型
MHA	<code>n_head</code>	1	GPT-2、Llama-1、Llama-2 7B/13B
GQA	$1 < n_{kv} < n_{head}$	n_{kv}/n_{head}	Llama-2 70B、Llama-3 全系(8)、Mistral
MQA	1	$1/n_{head}$	PaLM、Falcon、早期 StarCoder

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, d_model, n_head, n_kv_head, dropout=0.0, bias=False):
        super().__init__()
        assert n_head % n_kv_head == 0
        self.d_model, self.n_head, self.n_kv_head = d_model, n_head, n_kv_head
        self.n_rep, self.d_head = n_head // n_kv_head, d_model // n_head
        self.dropout = dropout
        out_dim = (n_head + 2 * n_kv_head) * self.d_head # q 全量 + kv 缩减
        self.qkv_proj = nn.Linear(d_model, out_dim, bias=bias)
        self.out_proj = nn.Linear(d_model, d_model, bias=bias)

    @staticmethod
    def repeat_kv(x, n_rep): # (B,Hkv,S,D) -> (B,Hkv*n_rep,S,D)
        if n_rep == 1:
```

```

        return x
    b, h_kv, s, d = x.shape
    return (x[:, :, None, :, :] # expand 不拷贝内存
            .expand(b, h_kv, n_rep, s, d).reshape(b, h_kv * n_rep, s, d))

def forward(self, x, cos=None, sin=None, attn_mask=None, is_causal=False,
            kv_cache=None, start_pos=0):
    b, s, _ = x.shape
    nh, nk, dv = self.n_head, self.n_kv_head, self.d_head
    q, k, v = self.qkv_proj(x).split([nh * dv, nk * dv, nk * dv], dim=-1)
    q = q.view(b, s, nh, dv).transpose(1, 2)
    k = k.view(b, s, nk, dv).transpose(1, 2)
    v = v.view(b, s, nk, dv).transpose(1, 2)

    if cos is not None: # RoPE 只作用于 q 和 k, 不作用于 v
        q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)
    if kv_cache is not None: # cache 里存 RoPE 之后的 k/v
        k, v = kv_cache.update(k, v)
    k, v = self.repeat_kv(k, self.n_rep), self.repeat_kv(v, self.n_rep)

    if is_causal and attn_mask is None: # 自己算 mask 以带上 start_pos
        attn_mask = causal_mask(s, k.shape[-2], start_pos, x.device)
        is_causal = False
    out, _ = scaled_dot_product_attention(
        q, k, v, attn_mask=attn_mask, dropout_p=self.dropout,
        is_causal=is_causal, training=self.training)
    out = out.transpose(1, 2).contiguous().view(b, s, self.d_model)
    return self.out_proj(out)

```

Key insight. GQA 省的是 **KV cache** 显存和访存带宽，不是计算量。repeat_kv 之后 attention 本身的 FLOPs 一模一样：query head 数没变， QK^T 依然是 n_{head} 个 (S, S) 矩阵，省的只有 qkv 投影那一小块参数。那为什么推理明显更快？因为 **decode** 阶段是彻底 **memory-bound** 的：每生成一个 token 都要把整个 KV cache 从 HBM 读一遍，而计算量只有 $O(Sd)$ ，算术强度极低；KV cache 小 $n_{\text{head}}/n_{\text{kv}}$ 倍，读的字节数就少同样的倍数，端到端时间基本线性跟着降。MQA ($n_{\text{kv}} = 1$) 压得最狠但质量掉得明显，GQA 取 8 之类的中间值是“几乎不掉点 + 拿到大部分收益”的折中，所以成了现在的默认。

KV cache 的显存公式必须能当场算出来（前面的 2 是 K、V 两份）：

$$\begin{aligned}
 M_{\text{KV}} &= 2 \cdot L \cdot B \cdot S \cdot n_{\text{kv}} \cdot d_{\text{head}} \cdot \text{bytes} \\
 &= 2 \times 32 \times 1 \times 4096 \times 32 \times 128 \times 2 \text{ B} = 2 \text{ GiB}
 \end{aligned}$$

第二行代的是 **Llama-2 7B** ($L = 32$ ，MHA 所以 $n_{\text{kv}} = 32$ ， $d_{\text{head}} = 128$) fp16、 $B = 1$ 、 $S = 4096$ 。7B 的 fp16 权重本身是 14 GB，一条 **4K** 序列的 **KV cache** 就要 **2 GiB**——batch 开到 8 就是 16 GiB，比权重还多。换成 GQA-8 ($n_{\text{kv}} = 8$) 直接降到 512 MiB。这就是 KV cache 优化 (GQA / MQA / MLA / KV 量化 / PagedAttention) 成为推理主战场的原因。

Warning. `repeat_kv` 必须是 `repeat_interleave` 语义，不是 `repeat`。
`repeat_interleave(2)` 给 `[k0, k0, k1, k1]`，`repeat(2)` 给 `[k0, k1, k0, k1]`。query head 是按组连续排布的：第 0 组的 `n_rep` 个 query head 要对上同一个 kv head。用 `repeat` 会让 query head 配错 kv head——又是一个 shape 全对、不报错、静默掉点的坑。上面的实现用 `expand + reshape`：`expand` 只改 stride 不拷贝内存，`reshape` 时才物化一次，比 `repeat_interleave` 少一个 kernel。

怎么自证：`n_kv_head == n_head` 时权重对拷给 MHA，两者输出必须逐元素相等（GQA 是 MHA 的严格推广）；`repeat_kv(x, n)` 断言等于 `x.repeat_interleave(n, dim=1)` 且不等于 `x.repeat(1, n, 1, 1)`。

RoPE：为什么它是相对位置编码

题目：实现旋转位置编码，并说明为什么它能表达相对位置。

旋转公式。把 d 维特征拆成 $d/2$ 个二维平面，第 i 个平面按角度 $m\theta_i$ 旋转（ m 是 token 的绝对位置）：

$$\theta_i = \text{base}^{-2i/d}, \quad i = 0, 1, \dots, d/2 - 1$$

$$\begin{pmatrix} x'_a \\ x'_b \end{pmatrix} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix} \begin{pmatrix} x_a \\ x_b \end{pmatrix} = R(m\theta_i) \begin{pmatrix} x_a \\ x_b \end{pmatrix}$$

写成复数最清爽：把 (x_a, x_b) 看成复数 z ，RoPE 就是 $z \rightarrow ze^{im\theta}$ 。

相对性的证明

这是本题的核心，务必能当场推出来。二维旋转矩阵是正交的（ $R(\alpha)^T = R(\alpha)^{-1} = R(-\alpha)$ ）而且可加（ $R(\alpha)R(\beta) = R(\alpha + \beta)$ ），于是位置 m 的 query 和位置 n 的 key 在第 i 个平面上的内积是

$$\begin{aligned} \langle R(m\theta)q, R(n\theta)k \rangle &= q^T R(m\theta)^T R(n\theta)k \\ &= q^T R(-m\theta)R(n\theta)k \\ &= q^T R((n-m)\theta)k = f(q, k, n-m) \end{aligned}$$

只依赖 $n-m$ 。用复数写更短： $\langle R_m q, R_n k \rangle = \text{Re}[(qe^{im\theta})(\overline{ke^{in\theta}})] = \text{Re}[q\bar{k}e^{i(m-n)\theta}]$ 。

Key insight. RoPE 的精妙之处：编码时用绝对位置（每个 token 独立旋转，不需要成对处理），注意力里自动变成相对位置。这让它兼有绝对编码的实现简单性和相对编码的泛化能力，而且不占任何参数、不改变 attention 的计算图形状——这就是它彻底取代可学习绝对位置编码的原因。

实现

```
def build_rope_cache(seq_len, dim, base=10000.0, device=None,
                    dtype=torch.float32):
    assert dim % 2 == 0
    inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2, device=device,
                                             dtype=torch.float32) / dim))
    t = torch.arange(seq_len, device=device, dtype=torch.float32)
    freqs = torch.outer(t, inv_freq) # (S, dim/2)
    return freqs.cos().to(dtype), freqs.sin().to(dtype)
```

```
def apply_rope(x, cos, sin):
    # x: (B, H, S, D)
    x1, x2 = x.chunk(2, dim=-1)          # 各 (B, H, S, D/2)
    cos = cos.to(x.dtype).unsqueeze(0).unsqueeze(0)  # (1, 1, S, D/2)
    sin = sin.to(x.dtype).unsqueeze(0).unsqueeze(0)
    return torch.cat([x1 * cos - x2 * sin,
                      x1 * sin + x2 * cos], dim=-1)
```

三个实现细节：

- **cos/sin cache**。 θ_i 只依赖维度、 $m\theta_i$ 只依赖位置，都和数据无关，所以整张 $(\text{max_seq}, \text{d_head}/2)$ 的表在 `__init__` 里算一次，用 `register_buffer(..., persistent=False)` 存：随 `.to(device)` 一起搬，但不进 `state_dict`——它是可重算的常量，存进 checkpoint 只是浪费。频率必须在 fp32 里算，bf16 的 mantissa 不足以区分相邻位置的角度。
- **base=10000**。 θ_i 从 1 ($i=0$) 指数衰减到 $1/\text{base}$ (最高维)。低维通道转得快、编码局部位置；高维通道转得慢、波长最长约 $2\pi \cdot \text{base}$ ，编码全局位置。base 越大，最长波长越长，能覆盖的上下文越长。
- 维度配对有两种排布。**interleaved** (原论文) 把相邻两维 $(x_0, x_1), (x_2, x_3), \dots$ 配成复数；**split-half** (GPT-NeoX / HF-Llama，也是上面的实现) 把前一半和后一半配对 $(x_0, x_{d/2}), (x_1, x_{d/2+1}), \dots$ 。

Warning. 两种排布都是正确的 RoPE (都是逐平面旋转，相对性证明照样成立)，但必须和权重训练时用的排布一致。混用不报错、shape 完全一样、模型还能输出通顺的文字，只是质量静默下降——因为 W_q 学到的通道配对关系被打乱了。这是移植 checkpoint 时的经典事故，`convert_llama_weights_to_hf.py` 里那段看起来莫名其妙的权重 permute 就是在做这件事。

两个正确性测试

RoPE 有两条能直接写成断言的数学性质，比“跑一下看 loss 降不降”强得多：

1. 旋转不改变范数。 R 正交，所以 `apply_rope(x).norm(dim=-1) == x.norm(dim=-1)`；更细一点，每个二维子平面的范数也单独守恒 (`y1**2 + y2**2 == x1**2 + x2**2`)，这证明它确实是逐平面旋转，而不是某种混了通道的线性变换。
2. 内积只依赖相对位置。固定 q, k ，让 $m - n = \delta$ 不变而 m 变，`(rope(q, m) * rope(k, n)).sum()` 必须是常数。配套测试对 $\delta \in \{0, 1, 5, -3\}$ 各扫 8 个起点断言极差 $< 10^{-4}$ ，同时断言不同的 δ 给出不同的内积 (否则位置信息就白编码了)。

再加一条便宜的：位置 0 的旋转角是 0，`cos[0] == 1`、`sin[0] == 0`，`apply_rope(x, cos[:1], sin[:1]) == x`。

Note. 长上下文外推一句话：训练时只见过 S_{train} 以内的角度，直接推到更长会因为高频通道的角度进入没见过的区间而崩。三条主流做法都是在动角度——位置插值 (把位置索引缩放到训练区间)、NTK-aware (调大 base，让高频通道少转一点)、YaRN (按通道波长分段处理，短波长插值、长波长保留，再配一个温度修正)。

mask : causal、padding，以及它们的组合

```
def causal_mask(q_len, kv_len=None, start_pos=0, device=None):
    """True = 可见。用 q_pos >= k_pos 的通式覆盖所有情况。"""
```

```

if kv_len is None:
    kv_len = q_len
q_pos = torch.arange(q_len, device=device).unsqueeze(1) + start_pos # (Sq,1)
k_pos = torch.arange(kv_len, device=device).unsqueeze(0)           # (1,Sk)
return q_pos >= k_pos

def padding_mask(lengths, max_len, device=None):
    """(B,) 的真实长度 -> (B, 1, 1, Sk) 的 key mask, True = 可见。"""
    ar = torch.arange(max_len, device=device)
    return (ar.unsqueeze(0) < lengths.to(device).unsqueeze(1)).view(-1, 1, 1,
max_len)

```

训练时 `causal_mask(S)` 就是最经典的 (S, S) 下三角全 True，等价于 `torch.ones(S, S, dtype=torch.bool).tril()`。

Warning. decode 阶段的 causal mask 不是方阵。 用 KV cache 逐 token 解码时 `q_len=1`、`kv_len=已缓存长度 P`，这个 query 的绝对位置是 `start_pos = P-1`，它能看到全部 P 个历史 key，所以 mask 应该是全 True 的 $(1, P)$ 。常见 bug 是直接套 `tril(ones(1, P))`——得到“只能看第 0 个 key”，模型每步都只看到序列开头。症状很有欺骗性：prefill 阶段 ($q_{len} = S$) 完全正常，只有开始逐 token 解码后输出才变成重复的胡话。上面 `q_pos + start_pos >= k_pos` 的通式一次覆盖 prefill 和 decode，不需要特判。

两者的组合是逻辑与，不是相加。padding mask 的形状故意做成 $(B, 1, 1, S_k)$ ，causal mask reshape 成 $(1, 1, S_q, S_k)$ ，直接 & 广播成 $(B, 1, S_q, S_k)$ ——语义是“两个条件都可见才可见”：

```
mask = padding_mask(lengths, S) & causal_mask(S).view(1, 1, S, S)
```

用 float 加性 mask 相加也能得到同样效果 ($-\infty + (-\infty) = -\infty$)，但 bool 版更省显存且没有数值问题。

怎么自证：causal 下断言 attention 权重的严格上三角精确为 0；组合 mask 下断言“长度为 3 的样本在 key 位置 3 之后权重全 0”且“前 3 个 query 行的权重仍归一”。后半句很重要——它验证了归一化是在剩下的可见位置上重新做的。

KV cache：为什么需要，怎么实现，怎么验

自回归生成时，第 t 步要算 q_t 对 $k_1..k_t$ 的 attention。朴素做法是把整条 (B, t) 序列重新 forward 一遍再取最后一个 logit——但 $k_1..k_{t-1}$ 只依赖前缀 token 和它们的绝对位置，和这一步无关，上一步已经算过了。缓存下来，每步的投影计算量从 $O(t)$ 降到 $O(1)$ ，生成 N 个 token 的总量从 $O(N^2)$ 降到 $O(N)$ 。这是推理侧收益最大、也最不可能不做的优化。

预分配 buffer + 游标

```

class KVCache:
    def __init__(self, batch_size, n_kv_head, max_seq_len, d_head,
dtype=torch.float32, device=None):
        shape = (batch_size, n_kv_head, max_seq_len, d_head)
        self.k = torch.zeros(shape, dtype=dtype, device=device)

```

```

self.v = torch.zeros(shape, dtype=dtype, device=device)
self.max_seq_len = max_seq_len
self.pos = 0          # 已填充长度，也是下一个 token 的绝对位置

def reset(self):      # 换一条 prompt 时复用 buffer，不重新分配
    self.pos = 0

def update(self, k, v):
    """写入新 k/v (prefill 时 S>1, decode 时 S=1), 返回全部历史。"""
    n = k.shape[2]
    assert self.pos + n <= self.max_seq_len, "KV cache 溢出"
    self.k[:, :, self.pos : self.pos + n] = k
    self.v[:, :, self.pos : self.pos + n] = v
    self.pos += n
    return self.k[:, :, : self.pos], self.v[:, :, : self.pos]  # view, 不拷贝

```

为什么不用 `torch.cat` : `cat` 每步都重新分配一块更大的显存并拷贝整个历史, decode N 步的总拷贝量是 $O(N^2)$, 还不停触发 caching allocator 去找新 block(碎片化)。预分配后每步只写一个 slot, 总量 $O(N)$, 返回的还是 view。代价是要提前知道 `max_seq_len`, 且显存按最坏情况占满——不管实际生成多长。vLLM 的 PagedAttention 就是为了解决这个“预留浪费”而生: 把 cache 切成固定大小的 block 按需分配, 像操作系统的分页。

cache 里存 **RoPE** 之前还是之后的 **k** : 之后。RoPE 只依赖该 token 的绝对位置, 写入时算一次即可; 存之前的话每步都要对整个历史重算旋转, 白付 $O(N)$ 开销, KV cache 的意义就没了。

prefill 与 decode : 两个完全不同的阶段

Key insight. 这是整个推理优化领域的核心认识。**prefill** (喂 prompt) 一次输入 S 个 token, 所有位置并行算, GEMM 的 M 维是 $B \times S$, **compute-bound**, 能打满 tensor core, 指标是 TTFT (time to first token)。**decode** (逐 token 生成) 每步只有 1 个 token, GEMM 退化成 GEMV ($M = B$), 且每步都要把全部权重 + 全部 **KV cache** 从 HBM 读一遍, **memory-bound**, 指标是 TPOT (time per output token)。

瓶颈不同, 优化手段就完全不同: decode 侧要提高 batch (continuous batching 把不同请求的 decode 步凑成一个大 GEMM) \ 压 KV cache (GQA / 量化) \ 投机解码 (用小模型一次猜多个 token, 把 memory-bound 的串行步换成 compute-bound 的并行验证); prefill 侧要 chunked prefill (把长 prompt 切块插到 decode 的空隙里, 避免长 prompt 把所有请求的 decode 卡住)。

验证正确性的黄金测试

Key insight. causal 下, “**prefill 一段 + 逐 token decode**” 每一步的 **logits** 必须与“一次性 **forward** 整条序列”取对应位置的 **logits** 逐位相同——两者数学上完全等价。这一条测试同时覆盖了 KV cache 实现的所有常见 bug: mask 的 `start_pos`、RoPE 的位置切片 `cos[start:start+s]`、cache 写入的下标, 三者中任何一处错了它都会挂。

```

full = model(idx)                                # (B, S, V) 一次算完
caches, prefill = model.make_caches(b), 4
out = [model(idx[:, :prefill], caches=caches, start_pos=0)]

```



```

for t in range(prefill, s):
    out.append(model(idx[:, t:t+1], caches=caches, start_pos=t))
torch.testing.assert_close(torch.cat(out, dim=1), full, rtol=1e-5, atol=1e-5)

```

同一个思路在 `generate` 层面还有一条：`use_cache=True` 和 `use_cache=False` 的 greedy 结果必须逐 **token** 相同。

TransformerBlock : pre-norm、RMSNorm、SwiGLU

```

class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x):
        dtype = x.dtype
        xf = x.float()
        xf = xf * torch.rsqrt(xf.pow(2).mean(-1, keepdim=True) + self.eps)
        return xf.to(dtype) * self.weight

class SwiGLU(nn.Module):
    def __init__(self, dim, hidden_dim=None, bias=False):
        super().__init__()
        if hidden_dim is None:
            hidden_dim = int(8 * dim / 3)
        self.w1 = nn.Linear(dim, hidden_dim, bias=bias)
        self.w3 = nn.Linear(dim, hidden_dim, bias=bias)
        self.w2 = nn.Linear(hidden_dim, dim, bias=bias)

    def forward(self, x):
        return self.w2(F.silu(self.w1(x)) * self.w3(x))

class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_head, n_kv_head=None, ffn_hidden=None):
        super().__init__()
        self.attn_norm = RMSNorm(d_model)
        self.attn = GrouperQueryAttention(d_model, n_head, n_kv_head or n_head)
        self.ffn_norm = RMSNorm(d_model)
        self.ffn = SwiGLU(d_model, ffn_hidden)

    def forward(self, x, cos=None, sin=None, is_causal=True,
                kv_cache=None, start_pos=0):
        x = x + self.attn(self.attn_norm(x), cos, sin, is_causal=is_causal,
                          kv_cache=kv_cache, start_pos=start_pos)
        return x + self.ffn(self.ffn_norm(x))

```

pre-norm vs post-norm

	post-norm (原论文)	pre-norm (现在的默认)
公式	$x \leftarrow \text{Norm}(x + \text{Sub}(x))$	$x \leftarrow x + \text{Sub}(\text{Norm}(x))$
残差通路	每层被 Norm 重新缩放	纯恒等映射
warmup	必需, 去掉直接发散	不敏感
深度	超过 20 层难训	百层量级没问题
代价	—	残差流方差随层数累加, 需补 final norm

为什么 **pre-norm** 稳：残差路径上没有任何操作，梯度可以从最后一层无损直达第一层（和 ResNet 的 $+I$ 完全同理）。post-norm 的梯度要穿过 L 个 Norm，每个 Norm 的 Jacobian 都会重新缩放梯度，期望范数随深度指数变化，所以必须靠 warmup 让训练早期别走大步。代价是 pre-norm 每层都往残差流上加东西而没人归一化， x 的方差随层数累加，深层子层的输出相对于已经很大的残差流只是“小扰动”，相对贡献被稀释（有人叫 representation collapse）。所以 pre-norm 结构最后必须补一个 final norm，否则 `lm_head` 拿到的输入尺度会随层数漂移。DeepNorm、sandwich-norm 是两种折中。

Key insight. **pre-norm** 的残差通路是恒等这句话可以直接写成断言：把 `attn.out_proj.weight` 和 `ffn.w2.weight` 全部置零（两个子层输出恒为 0），则 `block(x) == x` 精确成立。post-norm 做同样的事得到的是 `Norm(x) != x`。一行代码就把两种结构的本质差别测出来了。

RMSNorm 与 SwiGLU

RMSNorm 与 LayerNorm 的差别：不减均值、没有 **bias**，只除以均方根。少一次 reduce（对 memory-bound 的 norm kernel 是实打实的收益），实测效果与 LayerNorm 相当，所以 Llama 之后基本都用它。`mean( $x^2$ )` 要在 fp32 里算——bf16 下平方容易损失精度，而且这个量在分母上，误差会被放大。

SwiGLU 为什么是 3 个矩阵：GLU 家族用一路做“门控”、一路做“内容”，逐元素相乘，即 $W_2(\text{SiLU}(W_1x) \cdot W_3x)$ 。门控让 FFN 有了乘性交互（普通 FFN 只有加性），实测比同参数量的 ReLU/GELU FFN 好一截。

中间维为什么取 $8d/3$ 。普通 2 矩阵 FFN 取 `hidden = 4d` 时参数量是 $2 \cdot d \cdot 4d = 8d^2$ ；SwiGLU 有 3 个 $d \times h$ 规模的矩阵，要参数量持平就得

$$3dh = 8d^2 \Rightarrow h = \frac{8}{3}d \approx 2.67d$$

Llama 还会把它向上对齐到 256 的倍数（对 GEMM 的 tile 友好），所以 Llama-2 7B ($d = 4096$) 的 FFN hidden 是 11008 而不是 $8 \times 4096/3 = 10922.7$ 。

Warning. 常见错法：直接把 SwiGLU 的 hidden 设成 $4d$ 。参数量变成 $3 \cdot 4d^2 = 12d^2$ ，比对照组多 50%——然后你拿它和 GELU FFN 比说“SwiGLU 更好”，实际上比的是一个更大的模型。这是论文复现里的经典不公平对比。

MiniGPT 与参数量估算

```
class MiniGPT(nn.Module):
    def __init__(self, vocab_size, d_model, n_layer, n_head, n_kv_head=None,
                  ffn_hidden=None, max_seq_len=512, tie_weights=False,
                  rope_base=10000.0):
        super().__init__()
        self.n_kv_head, self.d_head = n_kv_head or n_head, d_model // n_head
        self.max_seq_len = max_seq_len
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, n_head, self.n_kv_head, ffn_hidden)
            for _ in range(n_layer)])
        self.final_norm = RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        if tie_weights:
            self.lm_head.weight = self.tok_emb.weight
        cos, sin = build_rope_cache(max_seq_len, self.d_head, base=rope_base)
        self.register_buffer("rope_cos", cos, persistent=False)  # 不进 ckpt
        self.register_buffer("rope_sin", sin, persistent=False)

    def forward(self, idx, caches=None, start_pos=0):
        s = idx.shape[1]
        x = self.tok_emb(idx)
        cos = self.rope_cos[start_pos : start_pos + s]
        sin = self.rope_sin[start_pos : start_pos + s]
        for i, block in enumerate(self.blocks):
            x = block(x, cos, sin, is_causal=True, start_pos=start_pos,
                     kv_cache=None if caches is None else caches[i])
        return self.lm_head(self.final_norm(x))

    def make_caches(self, batch_size, max_seq_len=None):
        w = self.tok_emb.weight  # 跟着模型的 dtype/device
        return [KVCache(batch_size, self.n_kv_head,
                        max_seq_len or self.max_seq_len, self.d_head,
                        w.dtype, w.device) for _ in self.blocks]
```

注意没有位置 **embedding**——位置信息全靠 RoPE 在每层的 attention 里注入。这是 Llama 系的标准做法，也解释了为什么 RoPE 模型换上下文长度不需要改任何参数形状。

tied embedding 的取舍：`lm_head.weight = tok_emb.weight` 省下 Vd 个参数。小模型上这块能占很大比例（ $V = 32000$ 、 $d = 768$ 时是 24.6M，而 12 层 block 才 85M），tie 了显然值得，还附带“输入输出共享同一套词表示”的正则效果。大模型（Llama 2/3）反而不 tie，因为 Vd 相对总量已经很小，解绑能多一点自由度。注意 tie 之后 embedding 的梯度会同时来自查表和输出投影两条路径。

参数量估算公式

按上面的实现逐块数（忽略 norm 的 d 个参数和所有 bias）：

$$\begin{aligned}\text{attn} &= \underbrace{(n_{\text{head}} + 2n_{\text{kv}})d_{\text{head}} \cdot d}_{\text{fused qkv}} + \underbrace{d^2}_{\text{out proj}} \rightarrow 4d^2 \quad (\text{MHA 时}) \\ \text{ffn} &= \underbrace{3dh}_{\text{SwiGLU 三个矩阵}} = 8d^2 \quad (h = 8d/3) \\ \text{per layer} &= 4d^2 + 8d^2 = 12d^2 \\ N_{\text{total}} &\approx 12Ld^2 + 2Vd \quad (Vd \text{ 只算一次, 如果 tie})\end{aligned}$$

$12Ld^2$ 是必须记住的数。验算 Llama-2 7B ($L = 32, d = 4096, V = 32000$): $12 \times 32 \times 4096^2 = 6.44 \times 10^9$, 加 $2Vd = 0.26 \times 10^9$ 得 6.70×10^9 , 实际是 6.74B (差异来自 FFN hidden 取了 11008 而非 10922)。两个直接推论: 参数量对 d 是平方、对 L 是线性, 所以加宽比加深涨得快得多; $12d^2$ 里 attention 只占 $4d^2$, **FFN** 才是参数量的大头——这也是 MoE 只把 FFN 换成专家的原因。

配套测试把公式落到了代码上: 手算 `per_block = d + (n_head + 2*n_kv)*d_head*d + d*d + d + 3*d*ffn_h`, 断言它乘层数再加 `emb / final norm / head` 之后精确等于 `sum(p.numel() for p in model.parameters())`。手算和 `numel()` 对不上, 说明你對自己模型的结构理解有偏差。

generate : greedy / temperature / top-k / top-p

```
def top_k_top_p_filter(logits, top_k=None, top_p=None):
    if top_k is not None and top_k > 0:
        k = min(top_k, logits.shape[-1])
        kth = logits.topk(k, dim=-1).values[..., -1, None] # (B, 1)
        logits = logits.masked_fill(logits < kth, float("-inf"))

    if top_p is not None and 0.0 < top_p < 1.0:
        sorted_logits, sorted_idx = torch.sort(logits, descending=True, dim=-1)
        probs = sorted_logits.softmax(-1)
        cumprobs = probs.cumsum(-1)
        remove = cumprobs - probs >= top_p # 累计概率已超过 p 之后的才删
        remove[..., 0] = False # 永远保留概率最高的那个
        remove = remove.scatter(-1, sorted_idx, remove) # 还原到原始序
        logits = logits.masked_fill(remove, float("-inf"))
    return logits

@torch.no_grad()
def generate(model, idx, max_new_tokens, temperature=1.0, top_k=None,
            top_p=None, use_cache=True, generator=None):
    model.eval()
    caches = model.make_caches(idx.shape[0]) if use_cache else None
    cur = idx
    for step in range(max_new_tokens):
        if use_cache:
            if step == 0: # prefill
                inp, start = cur, 0
            else: # decode: 只喂 1 个
                inp, start = cur[:, -1:], caches[0].pos
        logits = model(inp, caches=caches, start_pos=start)
```

```

else:
    logits = model(cur[:, -model.max_seq_len:])

logits = logits[:, -1, :].float() # 采样统一在 fp32 做
if temperature == 0.0:
    next_tok = logits.argmax(-1, keepdim=True)
else:
    logits = logits / temperature
    logits = top_k_top_p_filter(logits, top_k, top_p)
    probs = logits.softmax(-1)
    next_tok = torch.multinomial(probs, num_samples=1,
                                generator=generator)
cur = torch.cat([cur, next_tok], dim=1)
return cur

```

顺序是固定的：先 temperature，再 top-k / top-p，最后 softmax 采样。先截断再除温度会改变截断的边界，得到的不是你想要的候选集。

temperature：softmax(logits / T)。 $T \rightarrow 0$ 时分布退化成 one-hot，等价于 **greedy** (argmax)； $T \rightarrow \infty$ 时变成词表上的均匀分布。 $T < 1$ 更保守、 $T > 1$ 更发散。实现上 $T = 0$ 必须单独走 argmax 分支，不然会除零。

top-k vs top-p：

	top-k	top-p (nucleus)
规则	保留概率最大的 k 个	保留累计概率刚超过 p 的最小集合
候选集大小	固定	自适应
分布很尖时	强行引入本该排除的低概率词	集合自动缩小到 1-2 个
分布很平时	砍得太狠	集合自动放大

Warning. top-p 有两个必须写对的细节。

一是截断判据。要删的是“排在我前面的累计概率已经够了”的 token，即 `cumprobs - probs >= top_p`；写成 `cumprobs >= top_p`（含自己）会把恰好跨过阈值的那个 token 也删掉，候选集少一个。

二是至少保留 1 个 **token**。 p 取得极小时（ $p < \text{最大概率}$ ）上面的判据会把整行都标成 remove，所以必须显式 `remove[..., 0] = False` 兜底。漏了这行的后果是 `multinomial` 拿到全 0 的概率向量，报 `invalid multinomial distribution`。

还有一步容易忘：排序后的 mask 要用 `scatter(-1, sorted_idx, remove)` 还原到原始词表顺序才能 apply 到 logits 上。忘了它会屏蔽掉一批随机 token，采样结果“还算通顺但偶尔莫名其妙”。

怎么自证：三条互相独立的等价性——`temperature=0` 与 `temperature=1e-5` 结果逐 token 相同；`top_k=1` 与 `greedy` 相同（只剩一个候选，采样必然选中它）；`top_p` 取极小值也与 `greedy` 相同（兜底逻辑生效）。加上前面说的 `use_cache=True/False` 一致性，`generate` 的正确性基本被锁死了。

复杂度：什么时候 attention 才是瓶颈

按 forward、每层、batch 为 1 算（ S 是序列长度， d 是 hidden， $h = 8d/3$ ）：

$$\begin{aligned}\text{FLOPs}_{\text{attn-score}} &= \underbrace{2S^2d}_{QK^T} + \underbrace{2S^2d}_{\text{attn} \cdot V} = 4S^2d \\ \text{FLOPs}_{\text{linear}} &= \underbrace{2S \cdot 4d^2}_{\text{qkv} + \text{out proj}} + \underbrace{2S \cdot 8d^2}_{\text{SwiGLU}} = 24Sd^2 \\ \text{ratio} &= \frac{4S^2d}{24Sd^2} = \frac{S}{6d}\end{aligned}$$

也就是说 S 超过 $6d$ 之后 **attention** 的打分部分才成为 **FLOPs** 主项；causal mask 下只算下三角，这一项再减半，门槛推到 $S \approx 12d$ 。 $d = 4096$ 时那是 $S \approx 5 \times 10^4$ ——所以在 8K、32K 这些常见上下文下，**FLOPs** 主体仍然是那些 d^2 的线性层，说“attention 是 $O(S^2)$ 所以长上下文算不动”并不准确。

真正的瓶颈是激活显存：attention logits 是 $B \cdot n_{\text{head}} \cdot S^2$ 个元素，每层都要为反向保存一份。 $B = 1$ 、 $n_{\text{head}} = 32$ 、 $S = 8192$ 、fp16 就是 $32 \times 8192^2 \times 2 = 4 \text{ GiB}$ ，每层；32 层 128 GiB，单卡直接爆掉。这才是长上下文的真瓶颈。

Key insight. FlashAttention 不改变 **FLOPs**，改变的是 **HBM** 访存。它把 QK^T 分块、用 online softmax 在 SRAM 里增量归约，从不把 $S \times S$ 的矩阵物化到 **HBM**，把 attention 的激活显存从 $O(S^2)$ 压到 $O(S)$ （只存 softmax 的 statistics），同时因为省掉了那几次巨大的 HBM 往返，实际也更快。反向重算需要的中间量而不是读回来——用 FLOPs 换带宽，正是 memory-bound kernel 的标准套路。tiling、online softmax、访存分析的细节见 [books/cuda/](#)。

面试考点

面试考点. **Q1**：attention 为什么要除以 $\sqrt{d_k}$ ？除以 d_k 行不行？

A： q, k 各分量独立零均值单位方差时， $q \cdot k = \sum_i q_i k_i$ 是 d_k 个方差为 1 的独立项之和，方差是 d_k 、标准差 $\sqrt{d_k}$ 。不缩放的话 logits 幅度随 $\sqrt{d_k}$ 增长，softmax 进入饱和区，而它的 Jacobian $\text{diag}(p) - pp^T$ 在 one-hot 附近趋于 0，梯度消失、训练早期就学不动。除 $\sqrt{d_k}$ 正好把方差拉回 1。除以 d_k 是过度缩放，logits 全被压到 0 附近，attention 永远接近均匀分布，学不出选择性。注意用的是每个 **head** 的 d_{head} ，不是 d_{model} 。

面试考点. **Q2**：MHA 里 split head 为什么必须 `view(B,S,H,D).transpose(1,2)` 而不能 `view(B,H,S,D)`？

A：Linear 输出最后一维的排布是 head 在 d_{head} 外侧、在 S 内侧，所以只能切最后一维再把 head 换到前面。直接 `view(B,H,S,D)` 会把 $S * d_{\text{model}}$ 这段内存重新切分，语义变成“第 0 个 head 拿走前 S/H 个 token 的全部通道”——相邻 hidden 维被错切给不同 head，序列也被切碎。它 shape 完全正确、不报错、loss 还会降一点，只是永远学不到该有的水平，属于最难发现的一类 bug。

面试考点. **Q3**：`nn.MultiheadAttention` 的 `attn_mask` 和 `F.scaled_dot_product_attention` 的有什么区别？

A：bool mask 的语义正好相反。`F.scaled_dot_product_attention` 里 `True` = 保留，`nn.MultiheadAttention` 里 `True` = 屏蔽。同一个 causal 约束一边传 `tril` 一边传

`triu(diagonal=1)`。迁移时忘了取反会得到反向 causal——每个位置只能看未来，而训练目标就是预测下一个 token，等于直接偷看答案，症状是 loss 降得离奇地快、开发集也很好，只有真正自回归生成才暴露。

面试考点. Q4：GQA 相比 MHA 省了什么？为什么推理更快？

A：省的是 **KV cache** 显存和访存带宽，不是计算量。`repeat_kv` 之后 attention 的 FLOPs 一模一样（query head 数没变）。之所以更快，是因为 decode 阶段彻底 memory-bound：每生成一个 token 要把整个 KV cache 从 HBM 读一遍，KV cache 小 $n_{\text{head}}/n_{\text{kv}}$ 倍，读的字节数就少同样的倍数。顺带能省下的还有 qkv 投影里 k/v 那部分参数。

面试考点. Q5：估算一下 7B 模型的 KV cache 有多大。

A： $M = 2LBSn_{\text{kv}}d_{\text{head}} \cdot \text{bytes}$ 。Llama-2 7B： $L = 32$ 、MHA 所以 $n_{\text{kv}} = 32$ 、 $d_{\text{head}} = 128$ ，fp16、 $B = 1$ 、 $S = 4096$ ，代入是 2 GiB。权重本身 14 GB，batch 开到 8 时 KV cache（16 GiB）就超过权重了。这就是 GQA / MQA / MLA / KV 量化 / PagedAttention 这一整条优化线存在的理由。

面试考点. Q6：RoPE 为什么是相对位置编码？

A：RoPE 把特征拆成 $d/2$ 个二维平面，位置 m 的向量在第 i 个平面上乘旋转矩阵 $R(m\theta_i)$ 。二维旋转正交且可加，所以 $R(m\theta)^T R(n\theta) = R(-m\theta)R(n\theta) = R((n-m)\theta)$ ，于是 $\langle R_m q, R_n k \rangle = q^T R((n-m)\theta) k$ ，只依赖 $n-m$ 。精妙之处是编码时用绝对位置（每个 token 独立旋转），注意力里自动变成相对位置，兼有两者优点且不占参数。

面试考点. Q7：怎么验证自己写的 RoPE 是对的？

A：两条数学性质直接写成断言。一是旋转不改变范数：`apply_rope(x).norm(-1) == x.norm(-1)`，而且每个二维子平面的范数单独守恒（证明它确实是逐平面旋转）。二是内积只依赖相对位置：固定 q, k ，让 $m-n$ 不变而 m 变，内积必须是常数，同时不同的 $m-n$ 要给出不同的内积。再加一条便宜的：位置 0 的旋转是恒等。

面试考点. Q8：KV cache 为什么要预分配，而不是每步 `torch.cat`？

A：`cat` 每步重新分配更大的显存并拷贝整个历史，decode N 步的总拷贝量 $O(N^2)$ ，还不停触发 allocator 造成碎片。预分配 $(B, n_{\text{kv}}, \text{max_seq}, d_{\text{head}})$ 的 buffer + 一个 pos 游标，每步只写一个 slot、返回 view，总量 $O(N)$ 。代价是要提前知道 `max_seq_len` 且按最坏情况占满显存——PagedAttention 就是把 cache 切成固定 block 按需分配来解决这个浪费。另外 cache 里存的应该是 **RoPE** 之后的 k/v，否则每步都要对整个历史重算旋转。

面试考点. Q9：prefill 和 decode 有什么区别？为什么这个区别重要？

A：prefill 一次处理 S 个 token，GEMM 的 M 维是 $B \times S$ ，**compute-bound**，指标是 TTFT；decode 每步只有 1 个 token，GEMM 退化成 GEMV，每步都要把全部权重和 KV cache 从 HBM 读一遍，**memory-bound**，指标是 TPOT。瓶颈不同所以优化手段完全不同：decode 靠

提高 batch (continuous batching) 压 KV cache、投机解码；prefill 靠 chunked prefill 避免长 prompt 卡住其他请求的 decode。这是 LLM 推理优化的第一性认识。

面试考点. **Q10**：pre-norm 和 post-norm 的区别？为什么现在都用 pre-norm？

A：post-norm 是 $x \leftarrow \text{Norm}(x + \text{Sub}(x))$ ，pre-norm 是 $x \leftarrow x + \text{Sub}(\text{Norm}(x))$ 。pre-norm 的残差通路是纯恒等映射，梯度能从顶层无损直达底层，训练稳定、对 warmup 不敏感；post-norm 的梯度要穿过 L 个 Norm，期望范数随深度指数变化，必须靠 warmup 才能训起来，深了容易发散。代价是 pre-norm 的残差流方差随层数累加，深层子层的相对贡献被稀释，所以最后必须补一个 final norm。可以一行代码验证：把两个子层的输出投影置零，pre-norm 的 `block(x)` 精确等于 `x`。

面试考点. **Q11**：SwiGLU 为什么有 3 个矩阵？中间维为什么取 $8d/3$ ？

A：GLU 家族一路做门控、一路做内容，逐元素相乘引入乘性交互，所以是 gate / up / down 三个矩阵： $W_2(\text{SiLU}(W_1x) \cdot W_3x)$ 。普通 2 矩阵 FFN 取 hidden = $4d$ 时参数量 $8d^2$ ；SwiGLU 要持平就解 $3dh = 8d^2$ 得 $h = 8d/3$ 。直接设 $h = 4d$ 会让参数量变成 $12d^2$ ，多 50%，那样比出来的“SwiGLU 更好”是不公平对比。Llama 还会把 $8d/3$ 向上对齐到 256 的倍数，所以 7B 的 FFN hidden 是 11008。

面试考点. **Q12**：Transformer 的复杂度是 $O(S^2)$ ，那 8K 上下文的瓶颈是 attention 吗？

A：**FLOPs** 上不是。每层 attention 打分是 $4S^2d$ ，线性层是 $24Sd^2$ ，比值 $S/(6d)$ ；causal 下再减半，门槛是 $S \approx 12d$ 。 $d = 4096$ 要 $S \approx 5 \times 10^4$ 才让 attention 成为 FLOPs 主项，8K 时线性层仍占绝大多数。真正的瓶颈是激活显存：attention logits 是 $Bn_{\text{head}}S^2$ 个元素， $B = 1、32$ head、 $S = 8192$ 、fp16 就是每层 4 GiB。FlashAttention 解决的正是这个——它不改变 FLOPs，而是分块 + online softmax 让 $S \times S$ 矩阵从不物化到 HBM，激活从 $O(S^2)$ 降到 $O(S)$ ，顺带因为省掉巨大的 HBM 往返而更快。

进阶模型：ViT / CLIP / MoE / Diffusion / LoRA

第 26 章把 Transformer 拆到了零件级别。这一章往上走一层：面试里真正被要求“现场写一个”的进阶模块——ViT 的 patch embedding、CLIP 的对比损失、MoE 的 top-k 路由、DDPM 的加噪与采样、LoRA 的低秩旁路。

这五个模块的共同点是：代码都不长（**20-40** 行），但每一个都埋着一两个“写出来能跑、跑出来是错的”的坑。面试官问这些题不是想看你会不会调库，是想看你能不能说清“为什么这一行必须这么写”。所以下面每题都按同一个模板走：题目 → 考点 → 参考实现 → 常见错法 → 怎么自证。最后那一步最值钱：能当场说出一个可验证的数值断言（“随机初始化时 CLIP loss 应该正好是 $\log B$ ”），比任何口头解释都有说服力。

配套的可运行实现与全部断言在 `python/pytorch/interview/test_advanced_models.py`。

ViT：把图像变成 token 序列

题目. 手写 `PatchEmbed` 和一个最小可用的 ViT 前向，说清 patch 数、序列长度、cls token 和位置编码各自的作用。

考点. conv 实现切 patch 的等价性；序列长度为什么是 $N + 1$ ；ViT 为什么吃数据。

参考实现

```
class PatchEmbed(nn.Module):
    def __init__(self, img_size=32, patch_size=8, in_chans=3, embed_dim=48):
        super().__init__()
        assert img_size % patch_size == 0
        self.num_patches = (img_size // patch_size) ** 2
        # 关键: kernel == stride == patch_size
        self.proj = nn.Conv2d(in_chans, embed_dim,
                               kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        # (B, C, H, W)
        # (B, D, H/P, W/P) -> (B, D, N) -> (B, N, D)
        return self.proj(x).flatten(2).transpose(1, 2)
```

为什么 `Conv2d(kernel=P, stride=P)` 等价于“切块 + Linear”？ViT 的原始定义是：把 (C, H, W) 切成 $N = (H/P) \times (W/P)$ 个互不重叠的 (C, P, P) 块，每块展平成 CP^2 维，再过一个所有 patch 共享的 `Linear( $C \times P \times P$ , D)`。而卷积的每个输出位置做的事就是“取一个 $P \times P$ 的感受野、与卷积核做内积”。当 `stride == kernel_size` 时感受野恰好不重叠、不遗漏，于是每个输出位置正好对应一个 patch；卷积核 (D, C, P, P) reshape 成 $(D, C \times P \times P)$ 就是那个 Linear 的权重矩阵，而卷积的权重共享正好对应“所有 patch 过同一个 Linear”。

好处是省掉显式的 `unfold + reshape`（`unfold` 会实体化一个 $CP^2 \times N$ 的大矩阵，纯内存搬运），一个 cudnn kernel 搞定。

cls token 与位置编码

```
def __init__(self, ..., embed_dim=48, depth=2, num_classes=10):
    self.patch_embed = PatchEmbed(img_size, patch_size, 3, embed_dim)
    n = self.patch_embed.num_patches
    self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
    self.pos_embed = nn.Parameter(torch.zeros(1, n + 1, embed_dim)) # ← N+1
    nn.init.trunc_normal_(self.cls_token, std=0.02) # ViT 原论文的初始化
    nn.init.trunc_normal_(self.pos_embed, std=0.02)

def forward(self, x):
    b = x.shape[0]
    x = self.patch_embed(x) # (B, N, D)
    cls = self.cls_token.expand(b, -1, -1) # expand 不拷贝内存
    x = torch.cat([cls, x], dim=1) # (B, N+1, D)
    x = x + self.pos_embed
    for blk in self.blocks: # pre-norm LN + MHA + MLP
        x = blk(x)
    return self.head(self.norm(x)[: , 0]) # 只取 cls
```

ViT-B/16 在 224 分辨率下： $N = (224/16)^2 = 196$ ，加 cls token 后序列长 197，pos_embed 也是 197 行。ViT block 与 GPT block 的差别只有两点：没有 **causal mask**（图像 patch 双向可见），以及用 LayerNorm + GELU MLP 而不是 RMSNorm + SwiGLU（历史原因，非本质）。

cls token 是什么。一个可学习的、与输入无关的向量，拼在序列最前面。它自己不携带任何图像内容，只能通过 attention 从各个 patch 聚合信息，因此天然成为“全局表示”。替代方案是对所有 patch 做 mean pooling（DeiT III、SigLIP 用这个），效果相当甚至更好还省一个 token；cls token 更多是从 BERT 继承的传统。

位置编码为什么必须有。attention 对输入顺序是置换等变的：把 patch 序列打乱，输出只是跟着一起打乱，每个 token 的表示完全不变——图像的空间结构就丢了。ViT 用可学习的绝对位置编码而不是 RoPE，因为图像位置是二维的。

Warning. ViT 上最常踩的三个错：

1. **pos_embed** 少了一行：写成 $(1, N, D)$ 会和 $(B, N+1, D)$ 广播不上；更糟的是顺手写成 $x[:, 1:] += \text{pos_embed}$ ，变成 cls token 没有位置信息、所有 patch 整体偏移一格。
2. 换分辨率忘了插值位置编码。pos_embed 的长度和 N 绑死。从 224 微调到 384， N 从 196 变成 576，必须把 patch 部分 reshape 成 14×14 做二维双三次插值再拉平（cls 那一行单独保留），否则 load_state_dict 报 size mismatch，强行截断则精度崩盘。
3. **cls_token.repeat(b, 1, 1)** 代替 **expand**。功能一样但会真拷贝 B 份内存；expand 只改 stride（见第 1 章），后面 cat 时才实体化一次。

怎么自证

```
# 1) conv 路径 == unfold + 共享 Linear, 逐元素相同
patches = F.unfold(x, kernel_size=p, stride=p).transpose(1, 2) # (B, N, C*p*p)
w        = pe.proj.weight.reshape(d, c * p * p)                # (D, C*p*p)
torch.testing.assert_close(pe(x), patches @ w.t() + pe.proj.bias)

# 2) 关掉位置编码 => 打乱 patch 顺序输出不变（置换等变性）
```

```
torch.testing.assert_close(model(x, use_pos=False),
                           model(shuffle_patches(x, perm), use_pos=False))
assert not torch.allclose(model(x), model(shuffle_patches(x, perm)))
```

第二个测试是“位置编码为什么必须有”的直接实验证据，比说一遍定义有力得多。

与 CNN 的对比：归纳偏置与复杂度

	CNN	ViT
归纳偏置	局部性 + 平移等变 + 层级感受野，硬编码在结构里	几乎没有；只有“patch 内是局部的”
感受野	随深度线性增长	第一层就是全局
单层复杂度	$O(HWC^2k^2)$ ，与分辨率线性	$O(N^2D) + O(ND^2)$ ，与 N 平方
数据需求	小数据集也能训	JFT-300M 级别才超过 CNN

“ViT 需要大数据”的准确说法：CNN 把“相邻像素相关、物体平移后还是同一个物体”这两条先验直接写进结构，等于免费拿到一堆样本。ViT 把这些先验全交给数据去学——数据够多时它学到的比手工先验更好（这是 ViT 赢的地方），数据不够时它连“局部性”都学不出来。DeiT 用蒸馏 + 强增强在 ImageNet-1k 规模补掉了这个 gap。复杂度那一行的实际后果： P 从 16 减到 8， N 变 4 倍，attention 计算量变 16 倍——这就是 ViT 上不去高分辨率、要靠 Swin 的窗口注意力把 $O(N^2)$ 降回 $O(N)$ 的原因。

CLIP：对称对比损失

题目. 手写 `clip_contrastive_loss(image_emb, text_emb, logit_scale)`。

考点. L2 normalize 为什么必须；`logit_scale` 为什么可学习且用 `exp`；为什么两个方向都要算；batch size 为什么是命门。

```
def clip_contrastive_loss(image_emb, text_emb, logit_scale=1.0):
    image_emb = F.normalize(image_emb, dim=-1) # (B, D)
    text_emb = F.normalize(text_emb, dim=-1) # (B, D)

    logits = logit_scale * (image_emb @ text_emb.t()) # (B, B)
    labels = torch.arange(logits.shape[0], device=logits.device)

    loss_i2t = F.cross_entropy(logits, labels) # 按行：每张图找文本
    loss_t2i = F.cross_entropy(logits.t(), labels) # 按列：每段文本找图
    return (loss_i2t + loss_t2i) / 2, logits
```

整个 loss 就是“把 (B, B) 相似度矩阵按行和按列都逼近单位阵”。`labels = arange(B)` 是因为第 i 张图的正样本就是第 i 段文本——对角线是正样本，同一行其余 $B - 1$ 个都是负样本。

为什么必须先 L2 normalize

归一化后内积就是余弦相似度，取值被限死在 $[-1, 1]$ 、与向量模长无关。不归一化的话，模型可以靠“把正样本对的模长一起撑大”来降 loss——这是一条完全不学语义的捷径，训练会不稳定甚至发散。归一化把“学表示（方向）”和“学置信度（尺度）”解耦，后者统一交给 `logit_scale`。

Key insight. L2 normalize 有一个直接可测的后果：**loss** 对 **embedding** 的整体缩放完全不敏感。把 `image_emb * 100`、`text_emb * 0.01` 喂进去，loss 一位不变。这是验证“你真的归一化了”最快的检查。

logit_scale：为什么可学习、为什么 **exp**、为什么 **clamp**

余弦相似度只在 $[-1, 1]$ 。直接送进 softmax， $B = 32768$ 时最大和最小 logit 只差 2，softmax 输出几乎是均匀分布，梯度小到学不动。所以必须乘一个大的“温度倒数”，CLIP 把它设成可学习参数，训练收敛到约 100（即 $\text{temperature} \approx 0.01$ ）。

参数化成 $s = \exp(t)$ 而不是直接学 s 有两个理由： $\exp$ 保证 $s > 0$ （温度必须正）；优化在对数尺度上进行，对这种要跨 1-100 两个数量级的标量更稳（等价于对 s 做乘性更新而不是加性更新）。clamp 上界（CLIP 官方 clamp 到 $\ln 100$ ）是为了防跑飞——只要模型已经把对角线做成最大值，无脑增大 s 就能继续压 loss， s 会单调爆炸到 softmax 溢出。

为什么两个方向都要算

单向 `image→text` 只约束“给定图片能选对文本”。模型可以把所有文本 embedding 挤到一小块区域、只让图片彼此可分，照样把这个方向的 loss 降下去——表示坍塌的一种。双向同时约束两个检索方向，得到的才是真正对齐的联合空间。实践上两个方向的 loss 数值通常差不多，但去掉一个会明显掉检索指标。

batch size 为什么对对比学习至关重要

每个样本的负样本数就是 $B - 1$ ，全部来自 **batch** 内部，没有别的来源。InfoNCE 是互信息的一个下界，而这个下界本身的上限是 $\log B$ ：

$$I(X; Y) \geq \log B - \mathcal{L}_{\text{InfoNCE}}$$

所以 B 越大，loss 能提供的信息量上界越高。这就是 CLIP 用 32k batch 的原因，也是 MoCo（memory bank / queue）与 SimCLR（暴力大 batch）两条路线的分歧点。分布式实现要用 all-gather 把所有 GPU 的 embedding 聚起来算全局 (N, N) 矩阵。

Warning. 分布式对比学习的经典 bug：只 **gather embedding**，没让梯度流回其他 **rank**。`dist.all_gather` 是不可微的，返回的 tensor 没有 `grad_fn`，所以本 rank 只会收到“自己那一段 embedding”的梯度，其他 rank 贡献的负样本对本地梯度的贡献全丢了——loss 数值是对的（所以很难发现），梯度是错的。修法是用可微的 `torch.distributed.nn.all_gather`（它的 backward 里做 reduce-scatter），或手动把本地那一段拼回去。

怎么自证

两个理论值都能算出来，这是这一题最漂亮的地方：

```
# 1) 完美对齐: img == txt, 对角线相似度 1、其余近 0 (高维随机向量近似正交)
emb = F.normalize(torch.randn(32, 64), dim=-1)
loss, logits = clip_contrastive_loss(emb, emb.clone(), logit_scale=30.0)
assert loss.item() < 1e-3
assert (logits.argmax(-1) == torch.arange(32)).all() # top-1 检索 100%

# 2) 随机 embedding: loss ≈ log(B)
```



```
for b in (32, 64):
    loss, _ = clip_contrastive_loss(torch.randn(b, 512), torch.randn(b, 512),
                                    logit_scale=1.0)
    assert abs(loss.item() - math.log(b)) < 0.05
```

第二条的推导（面试时现场给出来是加分项）： $D = 512$ 维的两个独立高斯向量归一化后，内积期望 0、标准差 $1/\sqrt{D} \approx 0.044$ 。logit_scale = 1 时整个 (B, B) 矩阵几乎全是 0，softmax 退化均匀分布 $1/B$ ，于是 $\mathcal{L} = -\log(1/B) = \log B$ （ $B = 32$ 是 3.47， $B = 64$ 是 4.16）。工程价值：**loss** 卡在 $\log B$ 不降，说明模型根本没在学对齐——常见原因是学习率炸了、logit_scale 初始化太小、或者图文配对本身错位。这是对比学习最好用的健康度指标。

MoE : top-k gating 与负载均衡

题目. 手写一个 top-k sparse MoE 层，返回输出和 load-balancing aux loss。

考点. dispatch 怎么写才不慢；aux loss 两项各是什么；router 为什么必须 fp32；MoE 省了什么、没省什么。

Note. 本仓库另有一本 MoE 专书 `books/moe/`，把 dispatch、capacity、expert parallel、all-to-all 的工程细节讲透了。这一节只给面试需要的深度：写出正确的 20 行、说清 aux loss 的数学、回答“省显存吗”。

参考实现

```
class MoELayer(nn.Module):
    def __init__(self, d_model, d_ff, num_experts, top_k):
        super().__init__()
        self.num_experts, self.top_k = num_experts, top_k
        self.gate = nn.Linear(d_model, num_experts, bias=False)
        self.experts = nn.ModuleList([
            nn.Sequential(nn.Linear(d_model, d_ff), nn.GELU(),
                          nn.Linear(d_ff, d_model)) for _ in range(num_experts)])

    def forward(self, x):
        # (B, S, H)
        b, s, h = x.shape
        flat = x.reshape(-1, h)  # (T, H), 逐 token 路由

        gate_probs = F.softmax(self.gate(flat).float(), dim=-1)  # 强制 fp32
        topk_probs, topk_idx = torch.topk(gate_probs, self.top_k, -1)
        topk_probs = topk_probs / topk_probs.sum(-1, keepdim=True)  # 重归一化
        topk_probs = topk_probs.to(x.dtype)

        out = torch.zeros_like(flat)
        for e in range(self.num_experts):
            # 按 expert 循环, 不是按 token
            token_ids, slot_ids = torch.where(topk_idx == e)
            if token_ids.numel() == 0:
                continue
            y = self.experts[e](flat[token_ids])  # (M, H) 一个大 GEMM
            out.index_add_(0, token_ids,
                          y * topk_probs[token_ids, slot_ids, None])
```

```
aux = load_balancing_loss(gate_probs, topk_idx, self.num_experts)
return out.view(b, s, h), aux
```

top-1 路由，E0 拿了一半 token

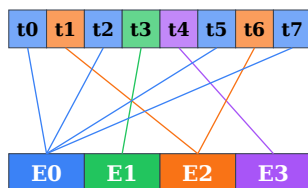


图 43 负载不均衡的样子：E0 拿到 4 个 token，E1/E3 各只有 1 个。expert parallel 下 E0 所在的卡成为 straggler，其余卡空转等它——这就是 aux loss 存在的现实动机。

为什么按 **expert** 循环而不是按 **token** 循环。按 token 循环是 Python 级 $O(T)$ 次小 kernel launch， T 是几万，慢到不能用。按 expert 循环是 $O(E)$ 次、每次一个大 GEMM，而 E 通常只有 8-64。生产实现会再进一步：先按 expert 做一次 argsort 把 token 排好序，变成一次 grouped GEMM (Megablocks、grouped_gemm)，彻底消掉 Python 循环。

top-k 内为什么要重归一化。让每个 token 的权重和为 1，保持输出量级与 dense 一致（否则 top-2 的权重和可能只有 0.3，输出被系统性缩小）。顺带一个好性质：top_k == num_experts 时这一步是恒等操作，整层精确退化成 dense 的概率加权——这就是这一题的自证方法。

load-balancing aux loss

```
def load_balancing_loss(gate_probs, expert_indices, num_experts):
    t, k = expert_indices.shape
    one_hot = F.one_hot(expert_indices.reshape(-1), num_experts).float()
    f = one_hot.sum(0) / (t * k)      # (E,) 硬分配频率, sum(f) == 1
    p = gate_probs.mean(0)            # (E,) 软概率均值, sum(p) == 1
    return num_experts * torch.sum(f * p)
```

$$f_e = \frac{1}{TK} \sum_{t,k} \mathbb{1}[\text{token } t \text{ 的第 } k \text{ 个 slot 选了 } e], \quad P_e = \frac{1}{T} \sum_t p_{te}, \quad \text{aux} = E \sum_e f_e P_e$$

两项的分工是这个设计的全部精髓。 f_e 来自 topk，是硬分配的计数、不可导——它决定真实的显存占用和通信量。 P_e 是 softmax 的输出、可导——梯度全部从这一项流回 router。相乘的效果是：某个 expert 如果既被大量选中（ f_e 大）又被 router 给了高概率（ P_e 大），乘积就大、被重罚，router 因此被推着降低对它的打分。用一个不可导的“实际负载”去加权一个可导的“意愿”，是这类 straight-through 风格设计的通用套路。

为什么均匀时最小值恰好是 **1.0**。关键观察是 f 和 P 不是独立的两个分布：硬分配是从软概率里 top-k 出来的，训练中 $f \approx P$ 。代入后 $\text{aux} = E \sum_e f_e^2$ ，而 f 是概率向量，由 Cauchy-Schwarz

$$\sum_e f_e^2 \geq \frac{(\sum_e f_e)^2}{E} = \frac{1}{E} \Rightarrow \text{aux} \geq E \cdot \frac{1}{E} = 1$$

等号成立当且仅当 f 均匀。另一端：全挤到一个 expert 时 $f = P = (1, 0, \dots, 0)$ ， $\text{aux} = E \cdot 1 = E$ 。所以 **aux** 天然落在 $[1, E]$ ，训练时盯着它离 1 有多远就知道路由健不健康。总 loss 通常是 `task_loss + 0.01 * aux`。

Warning. 两个常见错法。

第一，只用 P_e 的方差当 **loss**（“让软概率均匀”）。这只约束了意愿，实际的硬分配仍可能极不均衡——而决定显存和 all-to-all 通信的是硬分配。

第二，**router** 的 **softmax** 用 **bf16/fp16**。两个后果：(a) bf16 只有 8 位尾数，**exp** 之后 top-1 和 top-2 的概率可能变成完全相同的浮点数；(b) **topk** 的 tie-breaking 依赖精确比较，一旦并列，不同 rank 可能选出不同 expert，expert-parallel 下 all-to-all 的收发形状直接对不上——表现为随机 hang 或 shape 错，是最难查的一类非确定性 bug。fp16 更糟：router logits 会被 aux loss 推得很尖，**exp(11.1)** 就已经超过 fp16 的 65504 上限，直接 **inf**。所以 Switch / Mixtral 的实现里 router 一律 **.float()**，算完再 cast 回去。

expert collapse 与“省了什么”

collapse 现象。少数 expert 拿走绝大多数 token，其余几乎不被选中 → 收不到梯度 → 更不会被选中，正反馈跑飞。诊断按顺序看三个指标：aux loss 离 1 有多远（贴着 E 就是彻底坍缩）每个 expert 的 token 计数直方图、被丢弃 token 的比例（有 capacity 时）。只看 **loss** 曲线看不出来——坍缩后的 MoE 就是一个参数量虚高的 dense 模型，task loss 照样降。修法：调大 aux 系数（0.01 → 0.1，代价是牺牲一点 task loss）；router logits 加噪声（Noisy top-k gating）；加 router z-loss（惩罚 logits 的 log-sum-exp，防止整体变尖）；换 Expert Choice 路由（让 expert 挑 token，负载天然均衡）。

Key insight. MoE 省 FLOPs，不省显存。top-2/64 的 MoE 每个 token 只过 2 个 expert，FLOPs 是 dense 的 2/64；但所有 **64** 个 **expert** 的权重都得装在显存里，参数和优化器状态一分不少（AdamW 下每参数 12-16 字节）。这就是 MoE 必须配 expert parallel 的原因：把 expert 拆到不同卡上，用 all-to-all 换显存。完整答法还要加一句：也不省 **activation**，每个 token 的中间激活照存，反向还多一层 dispatch/combine 的 index 记账。所以 MoE 的真实收益是“同等训练 FLOPs 下容纳更多参数”，不是“同等显存下更快”。

怎么自证

```
# 1) aux 的两个端点值都是精确的
uniform = torch.full((64, 4), 0.25)
idx      = (torch.arange(64) % 4).unsqueeze(1)      # 完美均匀的硬分配
assert abs(load_balancing_loss(uniform, idx, 4).item() - 1.0) < 1e-6
worst = torch.zeros(64, 4); worst[:, 0] = 1.0      # 全挤 expert 0 => aux == E
assert abs(load_balancing_loss(worst, torch.zeros(64,1).long(), 4) - 4.0) < 1e-6

# 2) top_k == num_experts 时精确等于 dense 加权和 —— dispatch 的杀手测试
probs = F.softmax(moe.gate(flat).float(), -1)
dense = sum(probs[:, i, None] * moe.experts[i](flat) for i in range(E))
torch.testing.assert_close(moe(x)[0].reshape(-1, H), dense, rtol=1e-5, atol=1e-5)

# 3) 稀疏性：改一个没被选中的 expert 的权重，输出必须不变
```

第 2 条能一次抓住 **index_add_** 的下标、**slot_ids** 的对应关系、top-k 重归一化——任何一处写错它都过不了。

Diffusion : DDPM 的前向闭式解与反向采样

题目. 写出前向加噪的闭式解、DDPM 的训练目标、反向采样循环, 并说清 linear 与 cosine beta schedule 的区别。

考点. 闭式解怎么推; 为什么预测噪声; 最后一步为什么不加噪; schedule 与 T 的耦合。

前向: 一步跳到任意时刻

逐步加噪的定义是一条马尔可夫链($\alpha_t = 1 - \beta_t$): $x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{\beta_t}\varepsilon_t, \varepsilon_t \sim \mathcal{N}(0, I)$ 。递归展开一步就是推导的关键一步, 面试时把它写出来就够了:

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}x_{t-2} + \underbrace{\sqrt{\alpha_t\beta_{t-1}}\varepsilon_{t-1} + \sqrt{\beta_t}\varepsilon_t}_{\text{两个独立高斯之和}}$$

独立高斯的线性组合仍是高斯、方差直接相加: $\alpha_t\beta_{t-1} + \beta_t = \alpha_t(1 - \alpha_{t-1}) + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}$ 。也就是展开后系数仍满足“信号系数平方 + 噪声系数平方 = 1”。归纳下去, 记 $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \Rightarrow x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$$

```
def q_sample(x0, t, noise, alphas_cumprod):
    abar = alphas_cumprod[t]                                # (B,)
    shape = (-1,) + (1,) * (x0.dim() - 1)                 # (B, 1, 1, ...)
    return abar.view(shape).sqrt() * x0 + (1.0 - abar.view(shape)).sqrt() * noise
```

Key insight. 这个闭式解是 DDPM 能被训练的根本原因: 训练时随机采一个 t , 一步就能构造出 (x_t, ε) 样本对, 不用真跑 t 步前向, 否则训练成本要乘 $O(T)$ 。它也顺带解释了系数为什么是 $\sqrt{\bar{\alpha}}$ 和 $\sqrt{1 - \bar{\alpha}}$: 两者平方和为 1, 所以 $x_0 \sim \mathcal{N}(0, I)$ 时任意 t 下 x_t 的方差都保持 1。这叫 variance-preserving (VP) 扩散, 好处是网络在所有时间步看到的输入量级一致。

Warning. 广播是这一题最容易写错的地方。abar[t] 是 (B,), 直接和 (B, C, H, W) 相乘会广播成 (B,C,H,B) 的灾难或直接报错。必须补维成 (B,1,1,1); 用 view(shape) 动态算而不是硬编码 4 维, 才能同时兼容 (B, D) 和 (B, C, H, W)。

训练目标: 为什么是“预测噪声的 MSE”

```
def ddpm_loss(model, x0, alphas_cumprod, t=None):
    if t is None:      # 每个样本独立采 t, 不能整个 batch 共用一个
        t = torch.randint(0, alphas_cumprod.shape[0], (x0.shape[0],),
                           device=x0.device)
    noise = torch.randn_like(x0)
    x_t = q_sample(x0, t, noise, alphas_cumprod)
    return F.mse_loss(model(x_t, t), noise)
```

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$$

这个朴素 MSE 是简化的变分下界：完整 ELBO 是一串 KL 项 $\sum_t D_{\text{KL}}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t))$ ，两边都是方差固定的高斯，KL 退化成均值之差的平方；把均值用 ε_θ 重参数化后每项变成 $w_t \|\varepsilon - \varepsilon_\theta\|^2$ ，权重 $w_t = \beta_t^2 / (2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t))$ 。DDPM 的关键经验是把 w_t 直接设为 **1** 反而效果更好——原权重给低噪声时刻（任务只是“去掉一点点噪声”）压了极大的权重，扔掉它相当于把注意力挪到中高噪声区间。

为什么预测 ε 而不是 x_0 。两者可互换 ($\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\varepsilon}) / \sqrt{\bar{\alpha}_t}$)，数学上等价，差别在隐含加权：以 ε 为目标等价于给高噪声时刻更大权重，实测样本质量更好。后来还有 v-prediction（预测 $\sqrt{\bar{\alpha}} \varepsilon - \sqrt{1 - \bar{\alpha}} x_0$ ），高 SNR 区间数值更稳，是 SD 2.x 与蒸馏常用的参数化。**t** 必须逐样本采，整个 batch 共用一个 t 会让梯度方差巨大。

反向采样

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

```
@torch.no_grad()
def ddpm_sample(model, shape, betas, device=None, generator=None):
    alphas, abar = 1.0 - betas, torch.cumprod(1.0 - betas, dim=0)
    x = torch.randn(shape, device=device, generator=generator)
    for i in reversed(range(betas.shape[0])):
        t = torch.full((shape[0],), i, device=device, dtype=torch.long)
        eps = model(x, t)
        mean = (x - betas[i] / (1.0 - abar[i]).sqrt() * eps) / alphas[i].sqrt()
        if i > 0:
            z = torch.randn(shape, device=device, generator=generator)
            x = mean + betas[i].sqrt() * z
        else:
            x = mean
    return x  # ← 最后一步不加噪
```

$\sigma_t^2 = \beta_t$ 是论文给的两个选择之一（另一个是后验方差 $\tilde{\beta}_t = (1 - \bar{\alpha}_{t-1}) / (1 - \bar{\alpha}_t) \cdot \beta_t$ ，效果接近）。最后一步为什么不加噪： x_0 就是最终输出，再加一次噪声等于白往结果里掺噪点。对应 `if i > 0` 那一行——实现里最常漏的一行，漏了的表现是生成图有一层可见颗粒噪声。为什么慢：必须串行走 T 步、每一步一次完整前向， $T = 1000$ 就是 1000 次调用且无法并行。DDIM 把采样改写成确定性的非马尔可夫过程 ($\sigma_t = 0$)，允许跳步（只走 50 步），且同一个训练好的模型直接复用、不用重训。

beta schedule : linear 与 cosine

```
def linear_beta_schedule(timesteps, beta_start=1e-4, beta_end=0.02):
    return torch.linspace(beta_start, beta_end, timesteps)

def cosine_beta_schedule(timesteps, s=0.008):
    t = torch.linspace(0, timesteps, timesteps + 1) / timesteps  # ← 归一化 t/T
    abar = torch.cos((t + s) / (1 + s) * math.pi * 0.5) ** 2
    abar = abar / abar[0]
    return (1 - abar[1:] / abar[:-1]).clamp(0.0, 0.999)
```

cosine 先定义累积信号强度 $\bar{\alpha}(t)$ ，再反推 $\beta_t = 1 - \bar{\alpha}_t / \bar{\alpha}_{t-1}$ 。它更好的原因：linear 下 $\bar{\alpha}$ 在前 20% 步就掉到接近 0，绝大部分时间步都在“几乎纯噪声”里空转，样本利用率低；cosine 让 $\bar{\alpha}$ 在中段近似线性下降，噪声水平分布更均匀。可测量的体现是 $\bar{\alpha}_{T/2}$ ：cosine 明显高于 linear。

Warning. `linear_beta_schedule` 的默认 `beta` 区间是为 $T = 1000$ 调的，换 T 会静默失效——这是自己实现 DDPM 最容易踩的坑，而且不报错。

推导： β 从 10^{-4} 线性到 0.02 时 $\bar{\alpha}_T = \prod(1 - \beta_t) \approx \exp(-\sum \beta_t)$ ，而 $\sum \beta_t \approx T(10^{-4} + 0.02)/2 = 0.01005T$ ——与 T 成正比。 $T = 1000$ 时 $\sum \beta \approx 10.05$ 、 $\bar{\alpha}_T \approx e^{-10.05} \approx 4 \times 10^{-5}$ （合格，终点是纯噪声）； $T = 100$ 时 $\sum \beta \approx 1.005$ 、 $\bar{\alpha}_T \approx e^{-1.005} \approx 0.36$ ，前向过程根本没到纯噪声—— x_T 里还留着 $\sqrt{0.36} \approx 0.6$ 倍的原始信号。

而反向采样是从 $\mathcal{N}(0, I)$ 起步的。训练时模型只见过“还带 60% 信号”的 x_T ，采样时喂给它纯噪声，训练/采样分布对不上，生成结果系统性偏灰、偏糊——但 loss 曲线完全正常，因为训练本身自洽。

cosine schedule 因为按归一化的 t/T 定义（看代码里的 `t / timesteps`）， $\bar{\alpha}$ 的形状与 T 无关，天然免疫。自证：`torch.cumprod(1 - linear_beta_schedule(T), 0)[-1]` 在 $T = 1000$ 时 $< 10^{-3}$ 、 $T = 100$ 时 > 0.3 ；cosine 在两个 T 下都 $< 10^{-3}$ 。

怎么自证

```
# 1) schedule 合法性: 0 < beta < 1, abar 单调递减, 首尾到端点
abar = torch.cumprod(1 - betas, 0)
assert (betas > 0).all() and (betas < 1).all()
assert (abar.diff() < 0).all() and abar[0] > 0.99 and abar[-1] < 0.01

# 2) q_sample 两端: t=T-1 时是纯噪声 — 用统计量而不是逐元素比
x_end = q_sample(x0, torch.full((512,), T - 1), noise, abar)
assert abs(x_end.std().item() - 1.0) < 0.1      # 标准正态
assert abs(corrcoef(x_end, x0)) < 0.1          # 与信号无关
assert corrcoef(x_end, noise) > 0.99          # 就是噪声本身

# 3) 最后一步不加噪: 把网络换成恒返回零的 ZeroEps, 反向单步退化
# x / sqrt(alpha_t) + sigma_t * z, 固定 generator 后可手算轨迹逐位比对
```

第 3 条的技巧值得记：把随机过程的一个分支固定成确定性，是测试生成模型的通用手法——`if i > 0` 写错立刻被抓出来。

LoRA：低秩旁路

题目. 手写 `LoRALinear`，说清 A/B 的初始化、参数量、以及 `alpha / r` 这个缩放到底在干什么。

考点. B 为什么必须为 0；参数量省了多少；`alpha / r` 是不是真的让量级与 r 无关（这题能筛掉背公式的人）。

$$y = xW^\top + b + \left(\frac{\alpha}{r}\right)(xA^\top)B^\top, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, B \in \mathbb{R}^{d_{\text{out}} \times r}$$

```
class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, r=4, alpha=8.0, bias=True):
        super().__init__()
        self.r, self.scaling = r, alpha / r
        self.base = nn.Linear(in_features, out_features, bias=bias)
```



```

self.base.weight.requires_grad_(False)          # 冻结主干
if bias:
    self.base.bias.requires_grad_(False)
self.lora_A = nn.Parameter(torch.empty(r, in_features))
self.lora_B = nn.Parameter(torch.zeros(out_features, r))  # ← 必须 0
nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))

def forward(self, x):
    return self.base(x) + F.linear(F.linear(x, self.lora_A),
                                    self.lora_B) * self.scaling

def merged_weight(self):
    return self.base.weight + self.scaling * (self.lora_B @ self.lora_A)

```

注意 `forward` 里是 `F.linear(F.linear(x, A), B)` 两次小 GEMM，不是先算 `B @ A` 再乘 `x`。前者计算量 $O(Br(d_{\text{in}} + d_{\text{out}}))$ ；后者要实体化一个 $d_{\text{out}} \times d_{\text{in}}$ 矩阵、计算量 $O(d_{\text{in}}d_{\text{out}}(r + B))$ —— 写反了 LoRA 就一点都不省了。

B 为什么必须初始化为 0

训练开始时必须有 $\Delta W = BA = 0$ ，让模型精确等价于原模型（不是“接近”，是逐位相同）。否则一上来就给预训练权重叠了个随机扰动，前几百步全在修复它，微调效果明显变差 —— 数据少时尤其明显。

但 **A** 和 **B** 不能都置零：那样 $\partial \mathcal{L} / \partial A \propto B^\top (\dots) = 0$ 、 $\partial \mathcal{L} / \partial B \propto (\dots) A^\top = 0$ ，两边梯度恒为 0，对称性未破缺，永远学不动。标准做法是 **A** 随机（**kaiming uniform**）**B** 全零：第一步只有 **B** 收到梯度（它的梯度依赖 **A**，而 $A \neq 0$ ），**B** 动起来之后 **A** 才开始收梯度。反过来（ $A=0$ 、**B** 随机）数学上也可行，但 PEFT 库统一用前者。

参数量：给具体数字

$$\text{可训练} = r(d_{\text{in}} + d_{\text{out}}) \quad \text{vs} \quad \text{原权重} = d_{\text{in}}d_{\text{out}}$$

拿 LLaMA-7B 的一个 attention 投影 $d_{\text{in}} = d_{\text{out}} = 4096$ 举例：full FT 是 $4096^2 = 16.8 \text{ M}$ ； $r = 8$ 是 $8 \times 8192 = 65.5 \text{ K}$ （**0.39%**，省 256 倍）； $r = 16$ 是 131 K（0.78%，常用起点）； $r = 64$ 是 524 K（3.1%，接近 full FT 表现的经验上限）。

Key insight. LoRA 省得最多的其实是优化器状态，不是参数本身。AdamW 给每个可训练参数存 m 、 v 两份 fp32（参数量的 8 倍），加上 fp32 master weight 是 12 倍。冻结 base 后这 12 倍几乎全部归零。

LoRA 不省 **activation** 显存。反向仍要经过冻结的 base 权重把梯度传到输入，所以前向激活照存不误（`requires_grad=False` 只让参数不建图，不影响 activation 保存，见第 6 章）。“用了 LoRA 还是 OOM”通常就是这个原因，得靠 gradient checkpointing 解决。

alpha / r：一个流传很广的错误理解

标准说法是“ α/r 让 ΔW 的量级与 r 无关，所以换 r 不用重调 lr”。这个说法是错的，而且能当场用两行代码证伪。

设 A 、 B 的元素独立、标准差分别 σ_A 、 σ_B ，则 $(BA)_{ij} = \sum_{k=1}^r B_{ik}A_{kj}$ 是 r 个独立项之和，方差相加、标准差按 $\sqrt{r}$ 长：

$$\text{std}[(BA)_{ij}] = \sqrt{r}\sigma_A\sigma_B \propto \sqrt{r} \quad (\text{不是 } \propto r)$$

于是三种缩放的实际量级：

缩放	ΔW 量级	后果
不缩放	$\propto \sqrt{r}$	r 变大更新失控
LoRA 的 α/r	$\propto 1/\sqrt{r}$	矫枉过正： r 越大更新越小
rsLoRA 的 $\alpha/\sqrt{r}$	$\propto 1$	真正与 r 无关

Warning. α/r 除多了一个 $\sqrt{r}$ 。这直接解释了一个非常常见的现象：“把 r 从 8 提到 64，效果不但没涨反而略掉，除非同时把 lr 也调大”——有效更新量级被压了 $\sqrt{64/8} \approx 2.8$ 倍。这正是 rsLoRA (rank-stabilized LoRA) 主张改用 $\alpha/\sqrt{r}$ 的全部理由；如果你在用标准 LoRA，“提高 r 时把 lr 按 $\sqrt{r}$ 一起放大”就是这个修正的手工版本。

自证 (test_lora_scaling_and_the_rslora_critique): 固定 σ_B ，对 $r \in \{2, 8, 32\}$ 各算一次 $(B @ A).std()$ 。不缩放时 r 从 2 到 32 (16 倍) 量级涨 $\sqrt{16} = 4$ 倍；乘上 α/r 后反过来缩小 4 倍；用 $\alpha/\sqrt{r}$ 则三个 r 基本持平 (相对差 < 15%)。

合并回主干、QLoRA、以及怎么自证

推理时 $W' = W + (\alpha/r) * B @ A$ 合并进主干，之后就是一个普通 Linear，零额外延迟。不合并的代价是每层多两次小 GEMM (对 batch=1 解码尤其不划算，全是 memory-bound 的 kernel launch)，约束是合并后没法在同一个 base 上热切换多个 adapter——多租户 LoRA 服务 (S-LoRA、vLLM 的 multi-LoRA) 反而故意不合并，把多个 adapter 的 A 、 B 批起来做一次 grouped GEMM。

QLoRA 一句话：把冻结的 base 权重量化到 4-bit (NF4) 存着，前向时反量化成 bf16 参与计算，LoRA 旁路仍是 bf16 全精度训练。因为 base 不更新，量化误差是固定偏置而不是累积误差，所以撑得住——效果是 65B 模型能在单张 48GB 卡上微调。

```
# 1) 初始等价性：必须是 rtol=0, atol=0 的逐位相同，不是“接近”
torch.testing.assert_close(lora(x), lora.base(x), rtol=0, atol=0)
assert (lora.lora_B == 0).all() and (lora.lora_A != 0).any()

# 2) 只有 A/B 有梯度；且初始时 A 的梯度恒为 0 (因为它依赖 B)
lora(x).pow(2).sum().backward()
assert lora.base.weight.grad is None and lora.lora_B.grad.abs().sum() > 0
torch.testing.assert_close(lora.lora_A.grad, torch.zeros_like(lora.lora_A))

# 3) 参数量与合并等价性
assert lora.num_trainable() == r * (in_f + out_f)
merged = F.linear(x, lora.merged_weight(), lora.base.bias) # B 已改成非零
torch.testing.assert_close(lora(x), merged, rtol=1e-5, atol=1e-6)
```

第 2 条的最后一行断言一举两得：既验证了“ $B=0 \Rightarrow A$ 的梯度为 0”这个数学结论，也证明了“两边都置零会死锁”的推理是对的。

面试考点

面试考点. **Q1** : `Conv2d(kernel=P, stride=P)` 为什么等价于“切 patch + 共享 Linear” ? ViT 的序列长度是多少 ?

A : stride 等于 kernel 时卷积的感受野恰好互不重叠、不遗漏, 每个输出位置对应一个 patch ; 卷积核 (D, C, P, P) reshape 成 $(D, C \cdot P \cdot P)$ 就是那个 Linear 的权重, 卷积的权重共享对应“所有 patch 过同一个 Linear”。序列长度是 $N + 1 = (H/P)(W/P) + 1$, 所以 `pos_embed` 是 $(1, N+1, D)$ —— cls token 也占一个位置槽。换分辨率微调时 N 变了, 必须把 `pos_embed` 的 patch 部分 reshape 成 $\sqrt{N} \times \sqrt{N}$ 做二维插值再拉平 (cls 那一行单独留), 这是最容易漏的一步。

面试考点. **Q2** : 为什么说 ViT 需要大数据 ?

A : CNN 把“局部性”和“平移等变”硬编码在结构里, 等于免费拿到一堆先验 ; ViT 除了“patch 内是局部的”之外几乎没有归纳偏置, 全靠数据学。所以在 ImageNet-1k 规模从头训会输给 CNN, 到 JFT-300M 级别才反超 —— 数据够多时学出来的先验比手工的更好。DeiT 用蒸馏 + 强增强在 1k 规模补上了这个 gap。另外 attention 是 $O(N^2)$, P 从 16 减到 8 计算量涨 16 倍, 这是 ViT 上不去高分辨率、Swin 要做窗口注意力的原因。

面试考点. **Q3** : CLIP 的 loss 为什么要 L2 normalize、为什么对称、`logit_scale` 为什么可学习且用 `exp` ?

A : 归一化让内积变成余弦相似度、限死在 $[-1, 1]$, 堵死“靠撑大模长降 loss”这条不学语义的捷径, 把表示(方向)和置信度(尺度)解耦。对称是因为单向只约束一个检索方向, 模型可以把所有文本挤到一起、只让图片可分就把 loss 降下去 ; 双向把 (B, B) 矩阵按行和按列都逼近单位阵才是真对齐。`logit_scale` 可学习是因为余弦相似度太窄、softmax 几乎均匀、梯度学不动, 必须乘一个大温度倒数 (CLIP 收敛到约 100) ; `exp` 参数化保证恒正且让优化在对数尺度进行 ; 还要 clamp 到 $\ln 100$, 否则“无脑增大 scale”本身就能降 loss, s 会爆炸到 softmax 溢出。

面试考点. **Q4** : 为什么 batch size 对对比学习特别重要 ? 随机初始化时 loss 应该是多少 ?

A : 负样本数就是 $B - 1$, 全部来自 batch 内、没有别的来源 ; InfoNCE 作为互信息下界, 这个下界的上限是 $\log B$, 所以 B 越大信息量上界越高 (CLIP 用 32k)。随机初始化时高维 embedding 两两近似正交, logits 矩阵近似全 0, softmax 退化成 $1/B$, 于是 $\text{loss} = \log B$ ($B = 32$ 是 3.47)。训练时 loss 卡在 $\log B$ 不降, 说明根本没学到对齐 —— 这是最好用的 sanity check。分布式实现要注意 `dist.all_gather` 不可微, 只 `gather embedding` 会静默算出错梯度。

面试考点. **Q5** : MoE 的 load-balancing aux loss 是什么 ? 为什么均匀时最小值是 1 ?

A : $\text{aux} = E \sum_e f_e P_e$, 其中 f_e 是硬分配频率 (来自 `topk`, 不可导), P_e 是软概率均值 (可导, 梯度从这里流回 router)。相乘的效果是“既被大量选中又被给高分”的 expert 被重罚。因为硬分配来自软概率、训练中 $f \approx P$, $\text{aux} = E \sum_e f_e^2$, 由 Cauchy-Schwarz $\sum_e f_e^2 \geq 1/E$, 所以 $\text{aux} \geq 1$ 、均匀时取等 ; 全挤一个 expert 时是 E 。所以 aux 落在 $[1, E]$, 是现成的路由健康度指标。常见错法是只用 P_e 的方差做 loss —— 那只约束了软概率, 真正决定显存和通信的硬分配仍可能极不均衡。

面试考点. **Q6** : MoE 的 router softmax 为什么必须在 fp32 里算 ?

A : 数值上 fp16 撑不住尖锐的 router logits , $\exp(11.1)$ 就超过 65504 直接 inf ; bf16 虽不溢出但只有 8 位尾数 , top-1 和 top-2 的概率可能变成同一个浮点数。更致命的是 topk 的 tie-breaking 依赖精确比较 , 一旦并列 , 不同 rank 可能选出不同 expert , expert-parallel 下 all-to-all 的收发形状对不上 , 表现为随机 hang 或 shape 错 —— 非常难查。所以 Switch / Mixtral 一律 `.float()` 算完再 cast 回去。

面试考点. **Q7** : MoE 省显存吗 ? expert collapse 怎么诊断 ?

A : 不省。它省的是 FLOPs (top-2/64 只算 2/64 的计算量) , 但所有 64 个 expert 的权重都得装在显存里 , 参数和优化器状态一分不少 ; activation 也不省 , 还多一层 dispatch/combine 的记账。所以 MoE 必须配 expert parallel 把 expert 拆到不同卡上 , 真实价值是“同等训练 FLOPs 下容纳更多参数”。collapse 的诊断不能看 loss 曲线 (坍缩后就是个参数虚高的 dense 模型 , loss 照样降) , 要看 aux 离 1 有多远、expert 的 token 计数直方图、丢弃 token 比例。修法 : 调大 aux 系数、router 加噪、加 z-loss、或换 Expert Choice 路由。

面试考点. **Q8** : 写出 DDPM 前向的闭式解 , 并说明训练目标为什么是“预测噪声的 MSE”。

A : $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$, $\bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s)$ 。推导的关键一步是把逐步加噪递归展开 , 两个独立高斯之和仍是高斯、方差相加 , 得 $\alpha_t \beta_{t-1} + \beta_t = 1 - \alpha_t \alpha_{t-1}$, 归纳即得 ; 系数平方和为 1 所以是方差保持的。重要性 : 训练时随机采一个 t 就能一步构造 (x_t, ε) 样本对 , 否则训练成本乘 $O(T)$ 。目标是 MSE 因为完整 ELBO 展开后每项是 $w_t \|\varepsilon - \varepsilon_\theta\|^2$, 而 DDPM 发现把 w_t 直接设为 1 效果反而更好 (原权重给低噪声时刻压了过大权重)。预测 ε 和预测 x_0 可互换 , 但前者隐含给高噪声时刻更大权重 , 实测样本质量更好。

面试考点. **Q9** : linear_beta_schedule 换 T 会出什么问题 ? 最后一步采样为什么不加噪 ?

A : 默认 $\beta \in [10^{-4}, 0.02]$ 是为 $T = 1000$ 调的。 $\bar{\alpha}_T \approx \exp(-\sum \beta_t)$, 而 $\sum \beta_t \approx 0.01T$ 与 T 成正比 : $T = 1000$ 时终点 $\bar{\alpha} \approx 4 \times 10^{-5}$ (合格) , $T = 100$ 时变成 0.36 , x_T 里还留着 60% 的原始信号 , 前向根本没到纯噪声。而反向采样从 $\mathcal{N}(0, I)$ 起步 , 训练和采样的分布对不上 , 生成结果偏灰偏糊 , 但 loss 曲线完全正常 —— 静默失效。cosine 按归一化的 t/T 定义 , 天然对 T 免疫。最后一步不加噪是因为 x_0 就是最终输出 , 再加噪等于往结果里掺噪点 , 对应 `if i > 0` 那一行 , 漏了会得到一层可见颗粒。

面试考点. **Q10** : LoRA 的 B 为什么必须为 0 ? α / r 真的让 ΔW 的量级与 r 无关吗 ? LoRA 省了什么 ?

A : $B=0$ 保证初始 $\Delta W = BA = 0$ 、模型逐位等价于原模型 , 否则一上来就叠了个随机扰动破坏预训练知识。两边都置零不行 : $\partial \mathcal{L} / \partial A \propto B^\top (\dots) = 0$ 且 $\partial \mathcal{L} / \partial B \propto (\dots) A^\top = 0$, 梯度恒零、对称性未破缺。 α / r 不是无关 —— $(BA)_{ij}$ 是 r 项之和、标准差 $\propto \sqrt{r}$ 而不是 $\propto r$, 所以除以 r 后实际量级 $\propto 1/\sqrt{r}$, 矫枉过正 , 这正是“提高 r 却不涨点、必须同时调大 lr”的原因 , rsLoRA 的修正是改用 $\alpha/\sqrt{r}$ 。省的是可训练参数 (4096 方阵、 $r = 8$ 时 0.39%) 以及占大头的优化器状态 (AdamW 的 m 、 v 加 master weight 是参数量的 12 倍) ; 不省 **activation** , 因为反向仍要过冻结的 base 把梯度传到输入。推理时可合并回主干做到零延迟 , 代价是没法热切换多 adapter。

训练技巧类手写题

这一类题的共同特点是：公式不难，细节全是坑。label smoothing 的分母是 C 还是 $C - 1$ 、SGD 第一步的 momentum buffer 到底等于什么、AdamW 的 `eps` 加在开方前还是开方后 —— 每一个都能让你的实现和 `torch.optim` 差出可观的数值，而且不报错。

好消息是这一类题几乎每一道都能和官方实现跑数值对齐。面试时能说出“我写完会用 `assert_close` 和 `F.cross_entropy(label_smoothing=eps)` 对一遍”，比解释十分钟公式都有用。所以下面每题都给对齐方式。

第 5 章讲过训练循环的整体骨架（AMP、`zero_grad`、`accum` 的写法），这一章只讲怎么把这些组件自己写出来。下面的 loss 实现都省略了末尾那段相同的 `reduction` 收尾（`mean` / `sum` / `none` 三支），只留逐样本的 `(N,)` 张量。配套实现与 33 个断言在 `python/pytorch/interview/test_training_tricks.py`。

Label Smoothing Cross Entropy

题目. 手写 label smoothing 交叉熵, 要求与 `F.cross_entropy(label_smoothing=eps)` 数值对齐。

$$q_c = (1 - \varepsilon) \cdot \mathbb{1}[c = y] + \frac{\varepsilon}{C}, \quad C = \text{类别总数}$$

注意两件事：真实类的概率是 $1 - \varepsilon + \varepsilon/C$ ，不是 $1 - \varepsilon$ ；分母是类别总数 C ，不是 $C - 1$ 。展开后有一个等价且好记的形式 —— “真标签 **CE**”与“均匀分布 **CE**”的凸组合：

$$\mathcal{L} = (1 - \varepsilon) \cdot \text{NLL} + \varepsilon \cdot \frac{1}{C} \sum_c (-\log \text{softmax}(x)_c) = (1 - \varepsilon)H(p_{\text{true}}, p) + \varepsilon H(p_{\text{uniform}}, p)$$

按这个形式写实现最短：

```
def label_smoothing_cross_entropy(logits, target, smoothing=0.0):
    logp = F.log_softmax(logits, dim=-1)
    nll = -logp.gather(-1, target.unsqueeze(-1)).squeeze(-1) # (N,)
    smooth = -logp.mean(dim=-1) # (N,) 对均匀分布的交叉熵
    return (1.0 - smoothing) * nll + smoothing * smooth
```

Warning. 流传极广的那个版本是另一个 **loss**：

```
q = torch.full_like(logp, eps / (c - 1)) # x 其余类 eps/(C-1)
q.scatter_(1, target[:, None], 1.0 - eps) # x 真实类 1-eps
```

它也是合法的平滑分布（不少论文用的就是它），但数值上和 `F.cross_entropy(label_smoothing=eps)` 对不上 —— $\varepsilon = 0.1$ 、 $C = 5$ 时差值已到 10^{-2} 量级。被追问“你确定分母是 C 吗”，答案是：PyTorch 里就是 C ，理由是它等价于上面那个凸组合（均匀分布覆盖全部 C 类，包括真实类自己）。要对齐官方就必须用 C 。

它解决什么问题。one-hot 目标要求正确类的 logit 与其他类拉开无穷大的间距，模型会不断增大权重范数去逼近这个不可达的目标，结果是过度自信（预测概率 0.99 但实际准确率只有 0.8）、校准变差、泛化下降。平滑后最优 logit 间距变成一个有限值（让 softmax 输出恰好等于 q 所需的间距）： $\Delta = \log(q_y/q_{\text{other}}) = \log(1 + C(1 - \varepsilon)/\varepsilon)$ 。 $C = 1000$ 、 $\varepsilon = 0.1$ 时 $\Delta \approx 9.1$ —— 模型有了可达的最优解，正则效果就来自这里。什么时候不该用：知识蒸馏时（teacher 的软标签已经在做同样的事，叠加互相干扰）；需要精确概率的检索/排序任务（平滑会系统性压低 top-1 置信度）；标签本身有噪声时（平滑假设标签可信）。

怎么自证。对 `eps` 和 `reduction` 做叉乘对齐，`eps=0` 必须精确退化成普通 CE。`reduction="none"` 一定要覆盖 —— 很多实现只在 `mean` 下对得上，因为把归约写进了公式里。

```
for eps in (0.0, 0.05, 0.1, 0.3, 0.9):
    for red in ("mean", "sum", "none"):
        torch.testing.assert_close(
            label_smoothing_cross_entropy(logits, target, eps, reduction=red),
            F.cross_entropy(logits, target, label_smoothing=eps, reduction=red))
```

Focal Loss

题目。手写 focal loss，说明 γ 和 α 各解决什么问题。

$$p_t = \text{softmax}(\text{logits})_y, \quad \text{FL} = -\alpha_t(1 - p_t)^\gamma \log p_t$$

```
def focal_loss(logits, target, alpha=None, gamma=2.0):
    logp = F.log_softmax(logits, dim=-1)
    logpt = logp.gather(-1, target.unsqueeze(-1)).squeeze(-1) # (N,)
    pt = logpt.exp()
    loss = -((1.0 - pt) ** gamma) * logpt
    if alpha is not None: # 标量 或 (C,) 逐类别权重
        at = alpha.to(logits.device)[target] if isinstance(alpha, Tensor) \
            else torch.full_like(loss, float(alpha))
        loss = at * loss
    return loss
```

它解决什么问题。一阶段检测器里负样本（背景框）有 10^5 个、正样本只有几十个。问题不只是数量不均衡，更致命的是：那些“容易分对的负样本”单个 loss 很小，但数量太多，求和之后压倒了少数难样本的梯度，训练被简单样本主导。 $(1 - p_t)^\gamma$ 这个调制因子把权重从易分样本转移到难分样本： $p_t = 0.9$ 的易分样本在 $\gamma = 2$ 时 loss 被压到 $1/100$ ， $p_t = 0.1$ 的难样本几乎不受影响。 $\gamma = 0$ 且 `alpha=None` 时精确退化为（加权）CE。

α 和 γ 分工不同，别混为一谈。 α 是按类别的静态权重，处理“数量”不均衡； γ 是按样本难度的动态权重，处理“难度”不均衡。两者正交。RetinaNet 论文里 $\alpha = 0.25$ 、 $\gamma = 2$ —— 有意思的是最优 α 反而给正样本更低的权重，因为 γ 已经大幅压低了负样本，再用大 α 就矫枉过正了。

Warning. 验证退化时最容易踩的坑：`F.cross_entropy(weight=w)` 的 `reduction="mean"` 是加权平均，不是简单平均。它除的是权重之和 $\sum_i w_{y_i}$ 而不是样本数 N ，所以必须用 `'none'` 或 `'sum'` 才能直接比：


```
# x 对不上：左边除以 N，右边除以 sum(w[target])
focal_loss(logits, target, alpha=w, gamma=0.0, reduction="mean")
F.cross_entropy(logits, target, weight=w, reduction="mean")
# ✓
torch.testing.assert_close(
    focal_loss(logits, target, alpha=w, gamma=0.0, reduction="sum"),
    F.cross_entropy(logits, target, weight=w, reduction="sum"))
```

纯 API 细节，但对齐测试写不出来就会以为自己公式错了，白查半小时。

怎么自证。除了 $\gamma = 0$ 的退化，还要测那个核心性质——相对权重发生两个数量级的转移，这就是 focal loss 全部的作用：

```
logits = torch.tensor([[3.0, 0.0], [0.0, 3.0]]); target = torch.tensor([0, 0])
ce = focal_loss(logits, target, gamma=0.0, reduction="none") # 样本 0 易分
fl = focal_loss(logits, target, gamma=2.0, reduction="none") # 样本 1 难分
assert (fl[0] / ce[0]) < 0.01 # 易分样本压到 1% 以下
assert (fl[1] / ce[1]) > 0.85 # 难样本基本保留
```

Mixup 与 CutMix

题目. 实现 `mixup` 和 `cutmix`，并解释 `mixup_criterion` 为什么写成两个 CE 的线性组合。

```
def mixup_data(x, y, alpha=1.0, generator=None):
    lam = 1.0 if alpha <= 0 else float(
        torch.distributions.Beta(alpha, alpha).sample())
    index = torch.randperm(x.shape[0], device=x.device, generator=generator)
    mixed = lam * x + (1.0 - lam) * x[index]
    return mixed, y, y[index], lam # (x', y_a, y_b, lam)

def mixup_criterion(criterion, pred, y_a, y_b, lam):
    return lam * criterion(pred, y_a) + (1.0 - lam) * criterion(pred, y_b)
```

$\lambda \sim \text{Beta}(\alpha, \alpha)$ 是对称分布， $\alpha = 1$ 时就是 `Uniform(0, 1)`； α 越小越倾向抽到接近 0 或 1 的值（“几乎不混合”）。配对用 `randperm(B)` 在 **batch** 内部完成，不需要额外取数据——这是这个 trick 极其廉价的原因。原论文每个 batch 采一个标量 λ （逐样本采也能跑、方差更小，但和原实现不一致，说自己用哪种即可）。

为什么不直接混合 **label**。因为 CE 对目标分布是线性的：

$$\lambda \cdot \text{CE}(\hat{y}, y_a) + (1 - \lambda) \text{CE}(\hat{y}, y_b) = \text{CE}(\hat{y}, \lambda \text{onehot}(y_a) + (1 - \lambda) \text{onehot}(y_b))$$

两者数学上完全相等。返回 `(y_a, y_b, lam)` 的写法更好：不用把 `y` 展开成 `(B, C)` 的 one-hot（`C` 大时省不少显存）。对任何 criterion 都适用、天然兼容 `label_smoothing`。为什么有效：它强制模型在样本之间做线性插值的行为，大幅收缩决策边界附近的置信度，等价于一种数据依赖的正则。代价是目标变难了，训练要更多 epoch 才收敛。

```
def cutmix_data(x, y, alpha=1.0, generator=None):
    lam = 1.0 if alpha <= 0 else float(
```

```

    torch.distributions.Beta(alpha, alpha).sample()
b, _, h, w = x.shape
index = torch.randperm(b, device=x.device, generator=generator)

ratio = math.sqrt(1.0 - lam)          # ← sqrt: 控制的是面积比例
cut_h, cut_w = int(h * ratio), int(w * ratio)
cy, cx = int(torch.randint(h, (1,))), int(torch.randint(w, (1,)))
y1, y2 = max(cy - cut_h // 2, 0), min(cy + cut_h // 2, h)
x1, x2 = max(cx - cut_w // 2, 0), min(cx + cut_w // 2, w)

out = x.clone()
out[:, :, y1:y2, x1:x2] = x[index, :, y1:y2, x1:x2]
lam = 1.0 - ((y2 - y1) * (x2 - x1) / (h * w))    # ← 用实际面积修正
return out, y, y[index], lam, (y1, y2, x1, x2)

```

mixup 是全图加权叠加，产生“半透明重影”，不自然；cutmix 是区域替换，每个像素都来自某张真实图片，局部统计量正常，对卷积网络更友好，还天然带 Cutout 的遮挡正则。实践中两者常随机二选一交替用（timm 的 `mixup_prob` / `switch_prob`）。

Warning. CutMix 的两个必踩点。第一，`cut_w = W * sqrt(1 - lam)` 里的开方不能忘——要控制的是面积比例而不是边长比例，忘了开方替换面积就变成 $(1 - \lambda)^2$ 。第二， λ 必须用实际面积重算一遍：框中心均匀采样、边界会被 clip 到图像内，实际替换面积比目标小。不修正的话标签权重和图像内容对不上——这是一种静默的标签噪声，不报错、不崩溃，只是精度上不去。自证方法：用“每个样本一个常数值”的假图，断言框内逐位等于配对样本、框外保持原样，且 `lam` 精确等于未替换面积比例。

ModelEMA

题目. 实现权重的指数滑动平均，讨论 `decay` 怎么选、buffer 要不要 EMA。

$$\text{shadow} \leftarrow d \cdot \text{shadow} + (1 - d) \cdot \theta$$

```

class ModelEMA:
    def __init__(self, model, decay=0.999, warmup_steps=0):
        self.decay, self.warmup_steps, self.num_updates = decay, warmup_steps, 0
        self.shadow = copy.deepcopy(model).eval()
        for p in self.shadow.parameters():
            p.requires_grad_(False)

    def current_decay(self):          # timm 的 warmup 修正
        if self.warmup_steps <= 0:
            return self.decay
        return self.decay * (1.0 - math.exp(-self.num_updates / self.warmup_steps))

    @torch.no_grad()
    def update(self, model):
        d = self.current_decay()
        for s, p in zip(self.shadow.parameters(), model.parameters()):
            s.lerp_(p.detach(), 1.0 - d)    # s = (1-w)*s + w*end, w = 1-d

```

```
for s, b in zip(self.shadow.buffers(), model.buffers()):
    s.copy_(b) # buffer 直接拷贝, 不做 EMA
self.num_updates += 1
```

为什么有用. 训练末期参数在最优点附近震荡, EMA 相当于对参数轨迹做低通滤波, 近似取“平均权重”, 在 ViT / 检测 / 扩散上通常能白捡 0.2-1 个点。和 SWA 的区别是 EMA 给近期权重更高权重 (指数衰减), SWA 是等权平均。

Warning. $1/(1-d)$ 就是有效平均窗口长度 —— `decay=0.9999` 相当于平均最近 1 万步。所以如果总共只训 **5000** 步, 影子权重还停在初始化附近, 评测会惨不忍睹。这是 EMA 最常见的翻车方式, 现象非常迷惑: 主模型好好的, EMA 模型像没训过。

修法就是 warmup 修正: 让有效 decay 从 0 慢慢升到目标值, 早期几乎完全跟随当前权重。上面用的是 timm 的 $d_t = d(1 - e^{-t/\tau})$; 另一种思路是像 Adam 那样做偏差校正 `shadow/(1 - dt)`。选 decay 的经验法则: 让 $1/(1-d)$ 明显小于总步数, 典型 0.999 (窗口 1000 步) 到 0.9999 (窗口 1 万步)。

buffer 要不要 EMA. 这是个真实的分歧点。BN 的 `running_mean` / `running_var` 本身已经是滑动平均了, 再 EMA 一次相当于二重平滑, 会让统计量严重滞后于影子权重对应的激活分布 —— 所以 timm 的默认 (也是上面的实现) 是直接拷贝。也有实现对 buffer 也做 EMA, 差别通常很小。但 `num_batches_tracked` 这种整型 buffer 绝对不能 EMA (会被转成浮点或截断出错), 必须直接 copy。无论选哪种, 评测前一定要确认 **buffer** 被同步过 —— 忘了同步 BN buffer 会导致 EMA 模型精度离奇地差, 非常难查。

怎么自证. EMA 有一个可解析验证的性质: 目标权重固定不动时 shadow 以 d^t 的速度指数逼近它。再加三个端点: `decay=0` 时 shadow 完全等于当前权重; `decay=1` 时永不移动; 带 warmup 时第一次 `update` 的有效 decay 是 0, shadow 直接对齐当前权重。

```
ema = ModelEMA(model, decay=0.9)
for s in ema.shadow.parameters():
    s.zero_() # shadow 从 0 出发, target 固定
for _ in range(20):
    ema.update(model)
for s, t in zip(ema.shadow.parameters(), target):
    torch.testing.assert_close(s, t * (1 - 0.9 ** 20)) # 解析解
```

Warmup + Cosine Scheduler

题目. 实现 linear warmup + cosine 退火到 `min_lr`, 给出 `LambdaLR` 版和继承 `LRScheduler` 版。

$$\text{lr}(s) = \begin{cases} \text{lr}_{\text{base}} \cdot s/W & s < W \\ \text{lr}_{\text{min}} + (\text{lr}_{\text{base}} - \text{lr}_{\text{min}}) \cdot \frac{1}{2}(1 + \cos(\pi p)) & s \geq W, p = \frac{s-W}{T-W} \end{cases}$$

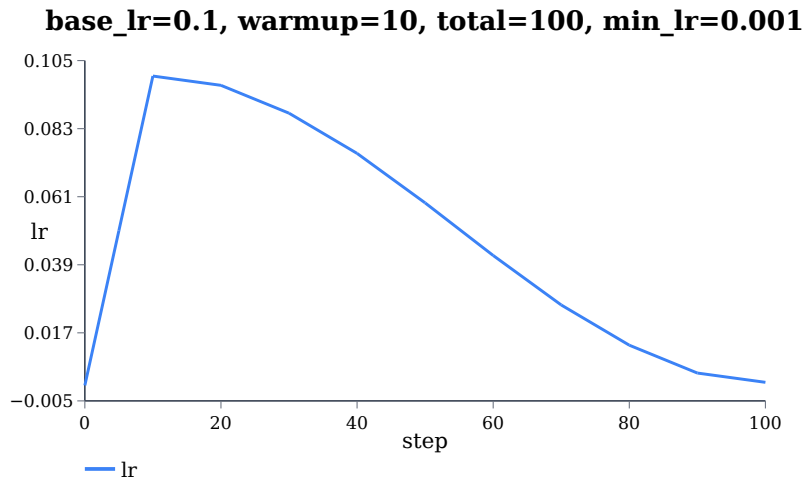


图 44 三个必须对得上的关键点：step=0 时 lr 为 0、step=W 时正好是 base_lr（两段在此处连续）、step=T 时正好是 min_lr。中点约为 $(lr_{base} + lr_{min})/2$ ，来自 cosine 的对称性。

LambdaLR 版三行就能写完，是实战首选：

```
def warmup_cosine_lambda(warmup, total, min_ratio=0.0):
    def fn(step):
        # 返回的是 base_lr 的【倍率】
        if warmup > 0 and step < warmup:
            return step / warmup
        p = min(1.0, max(0.0, (step - warmup) / max(1, total - warmup)))
        return min_ratio + (1 - min_ratio) * 0.5 * (1 + math.cos(math.pi * p))
    return fn

sched = torch.optim.lr_scheduler.LambdaLR(opt,
    warmup_cosine_lambda(warmup=2000, total=100_000, min_ratio=0.01))
```

Note. LambdaLR 的 lambda 返回的是相对 base_lr 的倍率，所以只能表达“按比例 min_lr”。多个 param group 的 base_lr 不同时，各组的绝对 min_lr 也会不同。要一个所有组共享的绝对 min_lr，就得用下面的继承版。

```
class WarmupCosineScheduler(LRScheduler):
    def __init__(self, optimizer, warmup_steps, total_steps, min_lr=0.0,
        last_epoch=-1):
        assert 0 <= warmup_steps < total_steps
        self.warmup_steps, self.total_steps, self.min_lr = (
            warmup_steps, total_steps, min_lr)
        super().__init__(optimizer, last_epoch) # ← 属性必须先设好

    def _lr_at(self, step, base_lr):
        if self.warmup_steps > 0 and step < self.warmup_steps:
            return base_lr * step / self.warmup_steps
        p = (step - self.warmup_steps) / max(1, self.total_steps
            - self.warmup_steps)
        p = min(1.0, max(0.0, p))
        return self.min_lr + (base_lr - self.min_lr) * 0.5 * (
            1.0 + math.cos(math.pi * p))
```

```
def get_lr(self):
    return [self._lr_at(self.last_epoch, b) for b in self.base_lrs]
```

Warning. 三个实现细节，写错了很难发现：

1. 属性必须在 `super().__init__()` 之前赋值。父类构造函数内部会立刻调一次 `step()` → `get_lr()`，此时 `self.warmup_steps` 还不存在就是 `AttributeError`。
2. 构造完成后 **optimizer** 里就已经是 **step 0** 的 **lr** 了，不需要（也不应该）在训练循环开始前额外调一次 `scheduler.step()` —— 多调一次会把整条曲线整体前移一格。
3. `lr(0) = 0` 是 HF transformers 的约定，意味着第一步完全不更新。不想要就改成 `(step + 1) / W` 或设一个 `warmup_start_lr`。

warmup 为什么必要。训练初期参数随机、梯度方向噪声极大，大 lr 会直接把参数推飞。对 **Adam** 系更关键：二阶矩 v 在最初几步只有极少样本，估计方差巨大， $1/\sqrt{\hat{v}}$ 可能是个离谱的大数，导致巨大的更新步长（RAdam 正是为了从理论上修掉这一点提出的，warmup 是它的经验近似）。对 **post-norm Transformer** 更是刚需，没有 warmup 直接发散。大 batch 训练时 lr 按线性法则放大，warmup 长度也要相应加长。

为什么 **cosine** 而不是 **step decay**。cosine 中段下降平缓、末段快速趋近 0，没有 step decay 那种突变造成的 loss 抖动，而且只有一个超参（总步数），不用调 milestone。代价是它依赖准确的总步数 —— 中途改变训练长度会让曲线整体错位，续训时必须把 `total_steps` 和已完成步数都对上。

梯度累积的等价性证明

题目。证明“累积 K 步、每步 `loss/K`”与“一次大 batch”的梯度数值相同，并说出例外。

设大 batch 有 N 个样本、切成 K 个等大的 micro-batch（每个 $m = N/K$ 个）。大 batch 的平均 loss 等于各 micro-batch 平均 loss 的算术平均：

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \ell_i = \frac{1}{K} \sum_{k=1}^K \underbrace{\frac{1}{m} \sum_{i \in \text{batch } k} \ell_i}_{\mathcal{L}_k}$$

梯度是线性算子，而 `backward()` 是把梯度累加到 `.grad`（见第 6 章），所以对每个 $\mathcal{L}_k/K$ 各 `backward` 一次，累加结果就是 $\nabla \mathcal{L}$ ：

$$\sum_{k=1}^K \nabla \left(\frac{\mathcal{L}_k}{K} \right) = \nabla \left(\frac{1}{K} \sum_k \mathcal{L}_k \right) = \nabla \mathcal{L}$$

两个前提必须说清：必须除以 K （忘了就相当于把学习率放大 K 倍，梯度恰好差 K 倍）；**micro-batch** 必须等大（最后一个不满的 batch 会让加权变形，正确做法是按样本数加权而不是均匀除 K ）。

Warning. 等价性的三类例外：

1. **BatchNorm**。BN 用的是当前 **batch** 内的均值/方差：大 batch 用 N 个样本统计，micro-batch 只用 N/K 个，归一化后的激活本身就不同，梯度自然对不上。这不是实现 **bug**，是数

学上就不等价。工程解法是换 GroupNorm / LayerNorm / RMSNorm (不依赖 batch 统计, 等价性立刻恢复), 或者用 SyncBN。

2. 任何跨样本的 **op**。in-batch 对比损失最典型: CLIP 的负样本数就是 $B - 1$ (见第 27 章), 把 batch 切成 K 份, 每份的负样本只剩 $N/K - 1$ 个, loss 的定义都变了。三元组挖掘、batch-level 归一化同理。
3. **DDP**。默认每次 backward() 都触发一次梯度 AllReduce, 累积 K 步就白白通信 K 次。必须用 no_sync() 包住前 $K - 1$ 步:

```
for i, batch in enumerate(loader):
    ctx = model.no_sync() if (i + 1) % K != 0 else contextlib.nullcontext()
    with ctx:
        (loss_fn(model(batch)) / K).backward()
    if (i + 1) % K == 0:
        opt.step(); opt.zero_grad(set_to_none=True)
```

怎么自证。一次大 batch backward 对比 K 次 loss/K backward, 逐位比 `.grad`。两个配套的反面测试同样值得写: 忘了除 K 时断言 `p_acc.grad == p_big.grad * K`; 插一个 BatchNorm1d 时断言 `max_diff > 1e-4`, 换成 LayerNorm 后等价性立刻恢复。能把“不等价”也测出来, 说明你真的理解了边界。

手写优化器

题目。继承 `torch.optim.Optimizer` 实现 `step()`, 与 `torch.optim.SGD` / `AdamW` 逐位对齐。

SGD with momentum

PyTorch 官方文档的伪代码 (顺序很重要, λ 是 weight decay、 τ 是 dampening):

$$g_t = g + \lambda \theta_{t-1}; \quad b_t = \begin{cases} g_t & t = 1 \\ \mu b_{t-1} + (1 - \tau) g_t & t > 1 \end{cases}; \quad \theta_t = \theta_{t-1} - \eta \cdot \begin{cases} g_t + \mu b_t & \text{nesterov} \\ b_t & \text{else} \end{cases}$$

```
class MySGDMomentum(torch.optim.Optimizer):
    # __init__ 只是把 lr/momentum/dampening/weight_decay/nesterov 塞进 defaults
    @torch.no_grad()
    def step(self, closure=None):
        for group in self.param_groups:
            lr, mu = group["lr"], group["momentum"]
            damp, wd = group["dampening"], group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad
                if wd != 0:
                    g = g.add(p, alpha=wd) # 耦合的 L2: 加进梯度
                if mu != 0:
                    state = self.state[p]
                    buf = state.get("momentum_buffer")
                    if buf is None:
                        buf = g.clone().detach() # 第一步: buffer = 梯度本身
```



```

state["momentum_buffer"] = buf
else:
    buf.mul_(mu).add_(g, alpha=1 - damp)
    g = g.add(buf, alpha=mu) if group["nesterov"] else buf
p.add_(g, alpha=-lr)

```

Warning. 三个几乎人人写错的细节。

1. 第一步的动量 **buffer** 直接等于梯度本身,不是 $(1 - \tau)g$ 。PyTorch 就是这么实现的,写成 $(1 - \tau)g$ 时 `dampening != 0` 的配置第一步就对不上。附带的好性质: `momentum=0.9` 的第一步和纯 SGD 完全一样,可以直接断言。
2. **PyTorch** 的动量不是教科书形式。教科书写 $v = \mu*v + lr*g$; $\theta -= v$, PyTorch 是 $v = \mu*v + g$; $\theta -= lr*v$ 。两者只在 **lr** 恒定时等价:PyTorch 版本改 `lr` 会立刻用新 `lr` 缩放整个历史动量,教科书版本里旧动量还带着当时的 `lr`。这解释了 `scheduler` 与 `momentum` 的交互——`cosine` 退火末期,PyTorch 版本的有效步长下降得更快。从别的框架搬公式时要留意。
3. **weight_decay** 是加到梯度上的(耦合的 **L2**),所以它会被动量累积、也会被 `lr` 缩放。这正是 AdamW 要“解耦”的东西。

momentum 为什么能加速. 它对梯度做指数滑动平均,在震荡方向上正负相消、在一致方向上累加,有效步长约放大 $1/(1 - \mu)$ 倍($\mu = 0.9$ 时约 10 倍)。Nesterov 则是在“预估的下一个位置”求梯度,多一阶修正项。

AdamW

$$\theta \leftarrow \theta(1 - \eta\lambda); \quad m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t; \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$$

$$\hat{m} = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v} = \frac{v_t}{1 - \beta_2^t}, \quad \theta \leftarrow \theta - \eta \cdot \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}$$

```

state["step"] += 1
t = state["step"]
m, v = state["exp_avg"], state["exp_avg_sq"] # 都初始化为 0

if wd != 0:
    p.mul_(1.0 - lr * wd) # 解耦: 直接作用在参数上
m.mul_(b1).add_(g, alpha=1 - b1)
v.mul_(b2).addcmul_(g, g, value=1 - b2)

bc1, bc2 = 1.0 - b1 ** t, 1.0 - b2 ** t
denom = (v.sqrt() / math.sqrt(bc2)).add_(eps) # eps 在开方之后
p.addcddiv_(m, denom, value=-lr / bc1)

```

Warning. AdamW 的三个对齐陷阱。

1. **eps** 加在开方之后: PyTorch 是 $\hat{m}/(\sqrt{\hat{v}} + \varepsilon)$ 。有些实现(TF 的 `epsilon_hat`)写成 $\hat{m}/\sqrt{\hat{v} + \varepsilon}$, 在 v 很小时行为差别明显。对不齐官方实现时先查这里。

2. 解耦衰减量是 $\text{lr} * \text{wd} * \theta$, 和 lr 相乘。所以调 lr 会连带改变实际衰减强度 —— 做 lr 扫描时这是一个隐藏的耦合变量。(AdamW 原论文的形式其实不乘 lr ; PyTorch 选了乘 lr 的版本, 因为这样和 scheduler 配合更自然。)
3. **bias correction** 不能省。 m_t 是 t 个梯度的加权和、权重之和是 $1 - \beta_1^t$, 除掉它才是无偏估计。 $\beta_2 = 0.999$ 时 $1 - \beta_2^t$ 在 $t = 1$ 只有 0.001 , 少了这一步第一步的 v 会小 1000 倍、步长直接爆炸。

AdamW vs Adam + L2 , 再强调一次. Adam + L2 把 $\lambda\theta$ 加进梯度, 于是它会被 $1/\sqrt{\hat{v}}$ 自适应地缩放 —— 后果是梯度大的参数 (v 大) 实际受到的衰减反而更小, 梯度小的被衰减得更狠, 与“weight decay 应该均匀地把权重拉向 0”的初衷完全相反。AdamW 把衰减从梯度里拿出来直接作用在参数上, 与 v 无关、对所有参数一视同仁。等价衰减强度差了 $\sqrt{\hat{v}}$ 倍, 所以从 **Adam** 换到 **AdamW** 必须重调 **weight_decay** (典型值从 10^{-4} 量级变到 10^{-2} 量级)

怎么自证

对齐测试是这一题的全部价值 —— 在固定数据和初始化上跑 8 步, 逐位比参数:

```
for cfg in [dict(lr=0.1, momentum=0.9),
            dict(lr=0.05, momentum=0.9, weight_decay=0.01),
            dict(lr=0.05, momentum=0.9, dampening=0.2),
            dict(lr=0.1, momentum=0.9, nesterov=True)]:
    got = run(m1, MySGDMomentum(m1.parameters(), **cfg))
    ref = run(m2, torch.optim.SGD(m2.parameters(), **cfg))
    torch.testing.assert_close(got, ref, rtol=1e-6, atol=1e-7)
```

三个更精巧的单点断言, 能直接验证公式细节:

- **SGD 第一步 == 纯 SGD**: `MySGDMomentum(lr=0.1, momentum=0.9)` 跑 1 步 $\equiv$ `SGD(lr=0.1)` 跑 1 步, 证明 buffer 初值就是梯度。
- **AdamW weight_decay=0 == Adam**: 最快的“是否真的解耦了”检查。反过来 `MyAdamW` 与 `Adam(weight_decay=wd)` 必须不同 (`max_diff > 1e-4`), 这正是 AdamW 论文的全部意义。另外无梯度时参数每步恰好乘 $(1 - \eta\lambda)$, 解耦衰减是精确可预测的。
- **AdamW 第一步的更新量恰好是 lr**: $t = 1$ 时 $\hat{m} = g$ 、 $\hat{v} = g^2$, 比值是 $\text{sign}(g)$, 与梯度大小完全无关。所以 `lr=0.1` 时参数正好走 0.1, 把梯度放大 1000 倍结果不变。这是 bias correction 正确的直接证据 (少了它第一步会小 $(1 - \beta_1)/\sqrt{1 - \beta_2} \approx 3.2$ 倍)

手写 `clip_grad_norm`

题目. 手写全局梯度范数裁剪, 与 `torch.nn.utils.clip_grad_norm` 对齐。

$$\text{total} = \|\text{concat}[g_1, \dots, g_n]\|_p, \quad c = \min\left(1, \frac{\text{max_norm}}{\text{total} + 10^{-6}}\right), \quad g_i \leftarrow cg_i$$

```
def my_clip_grad_norm_(params, max_norm, norm_type=2.0):
    grads = [p.grad for p in params if p.grad is not None]
    if not grads:
        return torch.tensor(0.0)
    device = grads[0].device
    norms = torch.stack([g.detach().norm(norm_type).to(device) for g in grads])
```

```
total_norm = norms.norm(norm_type) # 对"各张量范数"再取一次范数

clip_coef = (max_norm / (total_norm + 1e-6)).clamp(max=1.0)
for g in grads:
    g.detach().mul_(clip_coef)
return total_norm # 返回裁剪【前】的范数
```

“全局”两个字是重点，必须先把所有参数的梯度当成一个大向量算总范数，再统一缩放。对每个 tensor 单独调 `clip_grad_norm` 会改变各层梯度之间的相对比例，等于扭曲了梯度方向——这是最常见的错误实现。代码里的技巧： norm_p 的可加性让“先算每个张量的范数、再对这些范数取范数”等于“拼起来算一个范数”，不用真的 `torch.cat`（对 70B 模型省一次巨大的内存分配）。

clip by norm vs clip by value. by norm 保持梯度方向不变、只缩短长度，是默认选择；by value 逐元素截断会改变方向，一般只在特殊场景用。FSDP 下不能直接用这个函数，因为每个 rank 只有梯度的分片，必须用 `FSDP.clip_grad_norm`（内部对局部范数的平方做 AllReduce，见第 19 章）。

Warning. 四个必须记住的语义。

1. 只在 `clip_coef < 1` 时缩放，范数小于阈值时不放大（所以要 `clamp(max=1.0)`）。写成无条件乘 `clip_coef` 就变成了“归一化”而不是“裁剪”，小梯度会被放大，训练行为完全不同。
2. 返回的是裁剪前的范数，不是裁剪后的。所以日志里看到 `grad_norm` 超过 `max_norm` 是正常的——这个返回值是最好用的训练健康度指标（突然的尖峰通常紧跟着 loss spike）。
3. `+ 1e-6` 是防止 `total_norm` 为 0 时除零。
4. 必须在 `backward()` 之后、`optimizer.step()` 之前调用；用 **AMP** 时还必须先 `scaler.unscale_(optimizer)`——否则裁的是被 loss scale 放大过的梯度，阈值完全失效（见第 5 章）。这是混合精度训练里最经典的坑。

怎么自证。

```
for max_norm in (0.01, 0.5, 1e6): # 覆盖"要裁"和"不用裁"
    for norm_type in (2.0, 1.0, float("inf")):
        got = my_clip_grad_norm(list(m1.parameters()), max_norm, norm_type)
        ref = torch.nn.utils.clip_grad_norm(m2.parameters(), max_norm, norm_type)
        torch.testing.assert_close(got, ref) # 返回值
        for p1, p2 in zip(m1.parameters(), m2.parameters()):
            torch.testing.assert_close(p1.grad, p2.grad) # 裁剪后的梯度

# 全局性：两个张量梯度范数 10 : 0.1，裁剪后比例必须不变、总范数正好等于阈值
my_clip_grad_norm_([a, b], max_norm=1.0)
assert abs((a.grad.norm() / b.grad.norm()) - 100.0) < 1e-4
assert abs(torch.cat([a.grad, b.grad]).norm() - 1.0) < 1e-5
# 对比：分别裁剪，比例被彻底破坏（10:0.1 变成 1:0.1）
```

面试考点

面试考点. Q1：写出 PyTorch label smoothing 的确切公式。真实类的概率是多少？为什么能防过拟合？

A: $q_c = (1 - \varepsilon)1[c = y] + \varepsilon/C$, 分母是类别总数 C , 所以真实类的概率是 $1 - \varepsilon + \varepsilon/C$ 而不是 $1 - \varepsilon$ 。等价的好记形式是 $(1 - \varepsilon) \cdot \text{NLL} + \varepsilon \cdot \text{mean}_c(-\log \text{softmax}_c)$, 即“真标签 CE”与“均匀分布 CE”的凸组合。流传很广的“真实类 $1 - \varepsilon$ 、其余 $\varepsilon/(C - 1)$ ”是另一个 loss, 也合法但和 `F.cross_entropy(label_smoothing=)` 数值对不上。防过拟合的机理: one-hot 要求正确类 logit 与其他类拉开无穷大间距, 模型会不停增大权重范数去追一个不可达的目标; 平滑后最优间距变成有限值 $\log(1 + C(1 - \varepsilon)/\varepsilon)$, 有了可达的最优解。不该用的场景是知识蒸馏 (teacher 软标签已在做同样的事)、需要精确概率的检索/排序、以及标签本身有噪声时。

面试考点. Q2: focal loss 解决什么问题? γ 和 α 分别管什么?

A: 一阶段检测器里负样本 10^5 个、正样本几十个, 致命的不只是数量, 而是“容易分对的负样本”单个 loss 很小但数量太多, 求和后压倒了难样本的梯度、训练被简单样本主导。 $(1 - p_t)^\gamma$ 把权重从易分样本转移到难分样本: $p_t = 0.9$ 在 $\gamma = 2$ 时 loss 压到 $1/100$, $p_t = 0.1$ 几乎不动。 α 是按类别的静态权重 (管数量), γ 是按样本难度的动态权重 (管难度), 两者正交。RetinaNet 用 $\alpha = 0.25, \gamma = 2$ —— α 反而给正样本更低权重, 因为 γ 已经压狠了负样本。

面试考点. Q3: 怎么验证 focal loss 在 $\gamma = 0$ 时退化为 CE? 有什么陷阱?

A: `gamma=0` 且 `alpha=None` 时应精确等于 `F.cross_entropy`, 三种 reduction 都要过。陷阱在带 `alpha` 的版本: `F.cross_entropy(weight=w)` 的 `reduction="mean"` 是加权平均, 除的是权重之和 $\sum_i w_{y_i}$ 而不是样本数 N 。所以要验证等价必须用 `reduction='none'` 或 `'sum'`, 用 `'mean'` 直接比会以为自己公式错了。

面试考点. Q4: `mixup_criterion` 为什么写成 $\lambda \text{CE}(y_a) + (1 - \lambda)\text{CE}(y_b)$ 而不是混合 one-hot 标签? CutMix 有哪两个必须做对的细节?

A: 两者数学上完全相等, 因为 CE 对目标分布是线性的。写成两项之和的好处是不用把 y 展开成 (B, C) 的 one-hot (C 大时省显存) 对任何 criterion 都适用、天然兼容 `label_smoothing`。CutMix: (1) `cut_w = W * sqrt(1 - lam)` 里必须开方, 控制的是面积比例不是边长比例; (2) λ 必须用实际面积重算 —— 框被图像边界 clip 后实际替换面积更小, 不修正就是静默的标签噪声, 不报错只是精度上不去。

面试考点. Q5: EMA 的 `decay` 怎么选? 为什么需要 warmup? buffer 要不要 EMA?

A: $1/(1 - d)$ 就是有效平均窗口长度, `decay=0.9999` 相当于平均最近 1 万步。所以总步数只有 5000 时影子权重还停在初始化附近, 评测会惨不忍睹 —— 这是 EMA 最常见的翻车, 现象是主模型正常、EMA 模型像没训过。warmup 就是修这个冷启动: 让有效 `decay` 从 0 升到目标值 (timm 用 $d(1 - e^{-t/\tau})$), 早期几乎完全跟随当前权重。buffer 通常直接拷贝不做 EMA, 因为 BN 的 running stats 本身已经是滑动平均、再平滑一次会严重滞后于影子权重对应的激活分布; `num_batches_tracked` 这种整型 buffer 更是绝对不能 EMA。

面试考点. Q6: 手写 warmup + cosine scheduler 有哪些实现细节?

A: 继承 `LRScheduler` 时三个属性必须在 `super().__init__()` 之前赋值, 因为父类构造函数会立刻调 `get_lr()`。构造完成后 optimizer 里已经是 step 0 的 lr, 不要在训练循环前再多调一次 `scheduler.step()` (会把曲线整体前移一格)。三个必须对上的关键点: `step=0` 为 0 (HF 的

约定 λ `step=W` 正好是 `base_lr` (两段连续) `step=T` 正好是 `min_lr`。LambdaLR 版更短但返回的是 `base_lr` 的倍率,只能表达按比例 `min_lr`;要跨 param group 共享绝对 `min_lr` 就得用继承版。

面试考点. Q7: 证明梯度累积与大 batch 等价。有哪些例外?

A: 大 batch 的平均 loss 等于各等大 micro-batch 平均 loss 的算术平均, $\mathcal{L} = (1/K) \sum_k \mathcal{L}_k$; 梯度是线性算子而 `backward()` 是累加到 `.grad`, 所以对每个 $\mathcal{L}_k/K$ 各 `backward` 一次, 累加就是 $\nabla \mathcal{L}$ 。前提是必须除 K 、micro-batch 必须等大。三类例外: **BatchNorm** (用的是当前 batch 内统计量, 数学上就不等价, 换 `LayerNorm/GroupNorm` 立刻恢复); 任何跨样本的 **op** (in-batch 对比损失的负样本数从 $N-1$ 变成 $N/K-1$, loss 定义都变了); **DDP** (每次 `backward` 都触发 `AllReduce`, 必须用 `no_sync()` 包住前 $K-1$ 步, 否则通信量白涨 K 倍)。

面试考点. Q8: 手写优化器时有哪三个细节最容易和 `torch.optim` 对不上?

A: (1) **SGD** 第一步的 **momentum buffer** 直接等于梯度本身, 不是 $(1-\tau)g$ —— 所以 `momentum=0.9` 的第一步和纯 SGD 完全一样。(2) **PyTorch** 的动量形式是 `$\mathbf{v} = \mu * \mathbf{v} + \mathbf{g}$` ; `$\theta -= \text{lr} * \mathbf{v}$` , 不是教科书的 `$\mathbf{v} = \mu * \mathbf{v} + \text{lr} * \mathbf{g}$` ; `$\theta -= \mathbf{v}$` ; 两者只在 `lr` 恒定时等价, `lr` 变化时 PyTorch 版本会立刻用新 `lr` 缩放整个历史动量, 这解释了 `scheduler` 与 `momentum` 的交互。(3) **AdamW** 的 **eps** 加在开方之后 $\hat{m}/(\sqrt{\hat{v}} + \epsilon)$, 而 TF 的 `epsilon_hat` 是 $\hat{m}/\sqrt{\hat{v} + \epsilon}$; 另外 PyTorch 的解耦衰减量是 `$\text{lr} * \text{wd} * \theta$` , 和 `lr` 相乘, 所以调 `lr` 会连带改变实际衰减强度。验证手段是在固定数据和初始化上跑 8 步逐位比参数; 单点断言用“AdamW 第一步的更新量恰好是 `lr`、与梯度大小无关”。

面试考点. Q9: 手写 `clip_grad_norm_` 要注意什么?

A: 四点。全局范数 —— 所有参数的梯度当成一个大向量算一个 `norm` 再统一缩放, 逐张量分别裁会改变各层梯度的相对比例、扭曲梯度方向。只在 `clip_coef < 1` 时缩放, 范数小于阈值不放大, 否则就变成“归一化”而不是“裁剪”。返回裁剪前的范数, 所以日志里看到超过 `max_norm` 是正常的, 而且这个值是最好用的健康度指标 (尖峰常常紧跟 `loss spike`)。顺序: `backward` 之后、`step()` 之前; AMP 下必须先 `scaler.unscale_(optimizer)`, 否则裁的是被放大过的梯度、阈值完全失效。FSDP 下要用 `FSDP.clip_grad_norm_`, 因为每个 rank 只有梯度分片。

快问快答题库

前面 28 章每章结尾的「面试考点」是按主题深挖的题：一道题能聊五分钟。这一章是另一种东西 —— 收口题库，全是“一句话判断”型的短题，用于最后一周的冲刺自测。

怎么用它：先盖住答案，出声把答案说出来，再对照。默读会骗自己 —— 你以为知道，但说不出流畅的两句话，面试时就会卡。一轮下来把说不出的标记下来，只复习标记的那些，第二轮通常只剩 20% 不到。每道题的目标是 15 秒内说完；说到 30 秒还没说完，说明你在现场组织语言而不是在回忆结论，这一条要回去重看对应章节。

最后一节「开放题与故障排查」不一样：那 10 道题的答案是结构化的排查/决策清单，不是一句话。这类题是高级岗位的分水岭 —— 面试官不在意你猜的根因对不对，在意你有没有一套“从现象到根因”的系统方法。练它们的方式是把清单的顺序和每一步的判据说出来，而不是背结论。

基础与 Tensor

面试考点. Q：x.T 能用在 3-D 张量上吗？

A：不能。T 对非 2-D 张量已经废弃（新版本直接报错），因为它的语义是“反转所有维度”，对高维几乎总不是你想要的。要转最后两维用 x.mT 或 x.transpose(-2, -1) —— batched matmul 场景下一律用这两个。

面试考点. Q：torch.empty 和 torch.zeros 差在哪？什么时候必须用 zeros？

A：empty 只分配显存不写值，内容是上一次这块显存的残留；zeros 多一次 memset。所以只有“马上就会被整块覆盖”的 buffer 才用 empty（比如 torch.empty(n, out=...) 的接收端）。要累加进去的、要当 mask 的、要作为 index_add_ 目标的，一律 zeros —— 用 empty 会得到随机 NaN，而且是不可复现的。

面试考点. Q：nn.Parameter(t) 会拷贝 t 吗？

A：不会，它和 t 共享 storage。所以之后原地改 t 会改到参数上。要独立就写 nn.Parameter(t.clone())。同理 torch.as_tensor(x) 尽量不拷贝，而 torch.tensor(x) 总是拷贝 —— 热路径上用 as_tensor 或 from_numpy。

面试考点. Q：x.contiguous() 在 x 已经连续时会 copy 吗？

A：不会，直接返回 x 本身（同一个对象）。所以在不确定的地方无脑加 .contiguous() 的代价是零 —— 除非真的不连续。反过来说，看到 .contiguous() 出现在 profiler 的热点里，说明上游确实产生了非连续张量，该去看的是那里。

面试考点. Q：x.numel() * x.element_size() 一定等于它占的显存吗？

A：不一定，这算的是逻辑元素占的字节。如果 x 是一个 view/slice，它背后的 storage 可能大得多（x = big[0] 会让整个 big 都活着）。看真实占用要用 x.untyped_storage().nbytes()。这也是“切了一小片却没省显存”这类问题的根源。

面试考点. Q : `x.storage_offset()` 什么时候非 0 ?

A : 切片跳过了开头的元素时, 比如 `x[1:]`、`x[:, 2:]`。它和 `shape`、`stride` 一起构成 view 的完整描述: `storage[offset + sum(i_k * stride_k)]`。手写 kernel 或做 `from_blob` 时忘了 `offset` 是经典 bug。

面试考点. Q : `torch.cat` 和 `torch.stack` 的区别?

A : `cat` 在已有维度上拼接, 输出维度数不变; `stack` 新建一个维度, 输出维度数 +1, 且要求所有输入 `shape` 完全相同。`stack([a, b], 0)` 等价于 `cat([a[None], b[None]], 0)`。

面试考点. Q : `a[mask]` 的输出 `shape` 是静态的吗?

A : 不是, 它是数据相关的 —— 输出长度等于 `mask.sum()`, 只有真的跑一遍才知道。后果是: `torch.compile` 遇到它会 `graph break` 或引入 `unbacked symint`, `torch.export` 需要显式约束, CUDA graph 直接不支持。想保持静态 `shape` 就改成“乘 `mask` 保留原形状”或 `masked_fill`。

面试考点. Q : 为什么不建议给 `torch.arange` 传浮点 `step` ?

A : 元素个数是 `ceil((end - start) / step)` 算出来的, 浮点误差会让它在边界上多一个或少一个元素, 而且跨平台不一致。要固定个数就用 `torch.linspace(start, end, steps)`。

面试考点. Q : `torch.tensor(1) / torch.tensor(2)` 得到什么?

A : `0.5` (一个 float 张量), 不是整数 0。/ 是 `true division`, 会把整型提升成默认浮点 `dtype`。要整除用 `//` 或 `torch.div(..., rounding_mode="floor")`。这在算 `index` 时最容易出事 —— 得到 float 索引会直接报错。

面试考点. Q : `torch.equal` 和 `torch.eq` 的区别?

A : `torch.equal(a, b)` 返回一个 Python `bool`, 同时比较 `shape` 和全部数值, 不广播; `torch.eq(a, b)` (即 `==`) 返回逐元素的 `bool` 张量并且会广播。所以 `if a == b:` 在张量上会报 “Boolean value of Tensor is ambiguous”。浮点比较一律用 `torch.allclose` 或 `torch.testing.assert_close`。

面试考点. Q : `model.half()` 和 AMP 有什么区别?

A : `half()` 把所有参数和 buffer 永久转成 `fp16`, 优化器状态也变成 `fp16`, 很容易在几百步后发散。AMP 保留 `fp32` 的参数, 只在 forward 时对选定的 **op** (`matmul`、`conv`) `autocast` 成低精度, `softmax` / `LayerNorm` / `loss` 归约仍在 `fp32`。训练用 AMP, 纯推理才考虑 `half()`。

面试考点. Q : 为什么混合精度训练需要 `fp32` 的 master weight ?

A : 因为 `lr * grad` 常常比权重本身的 ULP (最小可分辨间隔) 还小。`fp16` 只有 10 位尾数、`bf16` 只有 7 位, `w + tiny` 会被直接舍回 `w`, 更新静默丢失 (`swamping`)。用 `fp32` 存权重、低精度只用于计算, 就避开了这个问题。

面试考点. Q : `nn.Module.float()` / `half()` 是原地的吗？

A : 是 (`nn.Module.to()` 系列对 module 都是原地的, 返回 `self` 只是为了链式写法)。而 `Tensor.to()` 不是原地的, 必须接返回值。这个不对称是新手最常翻车的地方之一。

autograd 与训练

面试考点. Q : `loss.backward()` 之后 `loss` 这个张量还能用吗？

A : 能, `loss` 本身的数值不变, `loss.item()` 照常。被释放的是图里的中间 buffer (`save_for_backward` 存的东西), 所以不能再 `backward` 第二次。但要注意 : 不要把 `loss` 张量存进 `list` 做日志 —— 那会让整张图跟着活着, 几十步后 OOM。存 `loss.item()` 或 `loss.detach()`。

面试考点. Q : `model.train()` / `model.eval()` 具体改变了哪些层的行为？

A : 只有依赖“当前是训练还是推理”的层 : Dropout 系 (`eval` 时变恒等) 和 BatchNorm 系 (`eval` 时用 running stats 而不是当前 batch 统计)。LayerNorm、Linear、Conv 完全不受影响。所以一个只有 LayerNorm 的 Transformer, 忘了 `eval()` 唯一的后果是 dropout 没关。

面试考点. Q : `scheduler.step()` 和 `optimizer.step()` 的顺序？搞错会怎样？

A : 先 `optimizer.step()`, 再 `scheduler.step()`, 每个 iteration 各一次 (epoch 级 scheduler 则每个 epoch 一次)。反过来会让第一步用错误的 lr, 整条曲线偏移一格 ; 更常见的错是在构造完 scheduler 后又手动多调一次 `step()`, 效果一样。torch 2.x 会对明显的顺序错误发 warning。

面试考点. Q : `p.grad` 的 dtype 一定和 `p` 一样吗？

A : 是的, autograd 保证 `p.grad` 和 `p` 的 dtype、shape、device 都一致。AMP 下参数是 fp32, 所以梯度也是 fp32 —— 低精度只存在于 forward 的中间计算里。这也是为什么 AMP 省的是 activation 显存, 不是梯度显存。

面试考点. Q : 一个参数同时出现在两个 `param_group` 里会怎样？

A : 它会被更新两次, 等效 lr 翻倍, 而且不报错。这在“按 weight decay 分组”时很容易发生 —— 分组逻辑用 `name` 匹配, 某个参数同时命中两条规则。自检方法 : `sum(len(g["params"]) for g in opt.param_groups)` 必须等于 `len(list(model.parameters()))`, 并且把所有 `id(p)` 放进 set 检查无重复。

面试考点. Q : weight decay 该不该作用在 bias 和 LayerNorm 的参数上？

A : 不该。这些参数是“平移/缩放”性质的, 把它们拉向 0 没有正则意义, 反而会损害表达能力 (LayerNorm 的 `weight` 被拉到 0 等于关掉这一层)。标准做法是分两个 param group : `ndim >= 2` 的 (权重矩阵、embedding) 给 wd, `ndim < 2` 的 (bias、norm 的 `weight/bias`) wd 设 0。GPT-2 / LLaMA 的官方训练脚本都是这么分的。

面试考点. Q : `F.dropout` 忘了传 `training=self.training` 会怎样？

A : `F.dropout` 的 `training` 默认是 `True` ,不会跟着 `model.eval()` 变。所以推理时 `dropout` 照样在丢,输出带随机性、指标莫名偏低。这是用 functional API 而不用 `nn.Dropout` 模块的最大风险——`nn.Dropout` 会自动读 `self.training`。

面试考点. Q : Dropout 在 eval 时要不要除以 `1-p` ？

A : 不用,因为 PyTorch 用的是 **inverted dropout** : 训练时保留下来的激活已经被除过 `1-p` 了,所以期望和 eval 时一致,eval 直接恒等输出即可。老教材里“训练不缩放、推理乘 `1-p`”是另一种等价写法,但推理时多一次乘法,所以现代框架都选前者。

面试考点. Q : `model.zero_grad()` 和 `optimizer.zero_grad()` 有区别吗？

A : 作用范围不同。`optimizer.zero_grad()` 只清 `param_groups` 里的参数,`model.zero_grad()` 清这个 module 的所有参数。绝大多数情况两者等价;不等价的场景是“optimizer 只管了模型的一部分”(比如冻结了 backbone、或者有多个 optimizer),这时候用哪个要想清楚。两者的 `set_to_none` 默认都是 `True`。

面试考点. Q : `cross_entropy` 的 `target` 能是 float 吗？

A : 能,但语义不同。整型 (N,) 是类别索引,走 NLL 路径;浮点 (N, C) 被当成概率分布 (torch 1.10+ 支持),走软标签路径。所以知识蒸馏、mixup 的软标签可以直接喂。混淆这两种会得到“expected scalar type Long but found Float”或者一个数值完全不对的 loss。

面试考点. Q : `nn.Embedding` 的 `sparse=True` 有什么用?代价是什么？

A : 让它的梯度变成稀疏张量 (只包含这一步真正用到的 row),省掉“整个 (V, D) 稠密梯度”的显存和 memset——词表 10 万时这是几百 MB。代价是只有少数 **optimizer** 支持稀疏梯度 (SGD、SparseAdam、Adagrad),AdamW 不支持;而且和 DDP 的梯度桶机制配合很差。所以大模型基本不用它,靠 tied embedding 和 TP 切分解决。

面试考点. Q : Adam 的 `beta2` 调大 (比如 `0.999` $\rightarrow$ `0.9999`) 会怎样？

A : 二阶矩的平均窗口从约 1000 步拉长到约 1 万步,对梯度尺度的突变反应更迟钝——好处是更稳 (大 batch、长训练常这么调),坏处是遇到真实的 loss spike 恢复更慢,而且前期需要更长的 warmup (因为 bias correction 的分母 $1 - \beta_2^t$ 更久才接近 1)。

显存与性能

面试考点. Q : batch size 翻倍,显存一定翻倍吗？

A : 不会。只有 activation 部分随 batch 线性增长;参数、梯度、优化器状态是固定开销,与 batch 无关。所以小模型大 batch 时接近线性,大模型小 batch 时几乎不变。这个拆分也是估显存的正确方式:先算固定项 (每参数 16 字节量级的 AdamW 混合精度),剩下的才是 activation 预算。

面试考点. Q : 两个进程共用一张卡,为什么特别容易 OOM ？

A：因为每个进程有独立的 **caching allocator**，各自缓存的空闲块对另一个进程完全不可见、也不会让出。加上每个 CUDA context 本身就要几百 MB，两个进程的“可用显存”远小于总量的一半。所以同卡多进程要么设 `PYTORCH_CUDA_ALLOC_CONF` 限制单进程上限，要么干脆用 MPS / 单进程多流。

面试考点. Q：显存碎片怎么判断？`expandable_segments` 干什么的？

A：判 据 是 `memory_reserved() - memory_allocated()` 很大却仍然 OOM —— 说明 空 闲 显 存 被 切 成 了 很 多 小 块，凑 不 出 一 个 连 续 的 大 块。
`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` 让 allocator 用可增长的虚拟地址段而不是固定大小的 segment，变长 shape 的训练上通常能显著缓解碎片。

面试考点. Q：`nvidia-smi` 显示 GPU-Util 100%，说明算力跑满了吗？

A：完全不能这么推。GPU-Util 的定义是“过去一段采样窗口里，至少有一个 kernel 在执行的时间比例”。一个极小的 kernel 连续不断地跑，利用率也是 100%，但 SM 占用率可能只有 2%。要看真实算力利用得算 MFU，或者用 `ncu` 看 achieved occupancy 和 memory throughput。

面试考点. Q：KV cache 占多少显存？给公式。

A： $2 \times L \times S \times H_{kv} \times d_h \times B \times \text{bytes}$ —— 系数 2 是 K 和 V 两份。LLaMA-2-7B ($L = 32$ 、 $H_{kv}d_h = 4096$ 、bf16) 算下来是每 **token** 每层 **16 KB**、全模型 **512 KB/token**，4k 上下文单条序列就是 2 GB。这就是 GQA / MQA (减小 H_{kv}) 和 PagedAttention (减少碎片浪费) 要解决的问题。

面试考点. Q：`torch.inference_mode()` 能省显存吗？

A：能省一点，但主要不是靠它。它不建图，所以中间 activation 用完即释放(这部分和 `no_grad()` 一样)；额外省的是 view 记账和 version counter 的元数据，量很小。它真正的收益是速度 —— 少了记账开销。要在推理上省显存得靠低精度、量化、KV cache 管理。

面试考点. Q：gradient checkpointing 和 CPU offload 怎么选？

A：先 checkpointing。它换的是计算 (多一次 forward，吞吐掉 20%-30%)，而 offload 换的是 **PCIe** 带宽 —— 单卡 A100 的 PCIe 是 32 GB/s 量级，比 HBM 慢 60 倍，很容易让 GPU 空转等数据。只有 checkpointing 已经开满还是不够、或者要 offload 的是“整个 step 只用一次”的优化器状态时，offload 才划算。

面试考点. Q：`persistent_workers=True` 解决什么问题？

A：默认每个 epoch 结束时 DataLoader 会销毁所有 worker 进程，下个 epoch 重新 fork —— 每次都要重新 import、重建 Dataset 对象、重开文件句柄。epoch 短或 Dataset 初始化重(比如要加载索引文件)时，这个开销能占到几个百分点。设成 `True` 就让 worker 活到 DataLoader 被销毁为止。代价是 worker 一直占着内存。

面试考点. Q：为什么推理时 batch 越大吞吐越高但延迟越差？

A：因为解码阶段是 memory-bound —— 每生成一个 token 都要把全部权重从 HBM 读一遍，这个代价与 batch 无关。所以把 B 条请求批在一起，权重只读一次，吞吐几乎线性涨。但单条请求必须等整个 batch 一起算完，还可能要等凑批，所以 P99 延迟变差。continuous batching (vLLM) 就是在这个权衡上做优化。

面试考点. Q：max_memory_allocated() 什么时候需要 reset？

A：它记录的是进程启动以来的峰值，所以想测“某一段代码的峰值”必须先 torch.cuda.reset_peak_memory_stats()。典型用法：warmup 几步（让 allocator 缓存稳定）→ reset → 跑一个 step → 读峰值。不 reset 就会一直读到 warmup 或模型加载时的旧峰值，得出错误结论。

面试考点. Q：non_blocking=True 拷到 GPU 后立刻在 CPU 上改源张量会怎样？

A：数据竞争 —— 拷贝还在 DMA 引擎里跑，你已经把源 buffer 改了，GPU 上可能拿到新旧混合的内容，而且不报错、不可复现。规则是：异步拷贝的源 buffer 在拷贝完成前不能碰。要么用 torch.cuda.current_stream().synchronize()，要么用双 buffer 轮换 (DataLoader prefetch 就是这么做的)。

面试考点. Q：为什么大模型训练要把 fused / foreach optimizer 打开？

A：因为朴素实现是“逐参数一个小 kernel”。一个 7B 模型有几百个参数张量，AdamW 每步要对每个做 5-6 次 elementwise op，就是几千次 kernel launch，纯 CPU 开销能占到 step 的百分之几十。foreach=True（默认）用 multi-tensor apply 把同 dtype 同 device 的张量批成一个 kernel；fused=True 更进一步，把整个 AdamW 更新融进一个 kernel。代价是 fused 要求参数都在 CUDA 上且是浮点，且对 state_dict 的兼容性略敏感。

torch.compile

面试考点. Q：torch.compile(model) 和 model.compile() 有区别吗？

A：有，而且是个实际的坑。torch.compile(model) 返回一个 OptimizedModule 包装，它的 state_dict 的 key 全部多了 _orig_mod。前缀 —— 存下来的 checkpoint 加载回未编译的模型会全部 key mismatch。model.compile() (torch 2.2+) 是原地编译，key 不变。存 checkpoint 时要么用后者，要么存 model._orig_mod.state_dict()。

面试考点. Q：torch.compile 能加速 backward 吗？

A：能，而且反向往往是收益更大的一半。AOTAutograd 在编译时就把反向图一起 trace 出来交给 Inductor（见第 13 章），所以反向也享受 fusion 和 kernel 生成。这也是为什么 compile 只需要包住 forward：loss.backward() 走的是编译好的反向图，不需要（也不能）单独编译。附带效应是反向的 activation 保存策略由 min-cut partitioner 决定，所以显存占用可能和 eager 不同。

面试考点. Q：fullgraph=True 是干什么的？

A：把 graph break 从“静默降级”变成“直接报错”。它不会让代码变快，它是个调试开关：想确认整个模型真的编成了一张图（CUDA graph、export 的前置条件）就打开它，报错信息会指出是哪一行导致了 break。日常训练不要开，因为很多 break 是无害的。

面试考点. Q：dynamic=True / False / None 分别什么意思？

A：None（默认）是自动：第一次按静态 shape 编译，看到第二种 shape 时把变化的维度标成动态、重编译一次，之后不再重编。True 从一开始就假设动态，省掉那次多余的静态编译（变长输入场景推荐）。False 强制静态，每种 shape 都单独编一份——shape 种类少且想要极致性能时用。

面试考点. Q：编译过的模型能直接 torch.save 吗？

A：不能可靠地存——OptimizedModule 里挂着编译后的 callable，pickle 不了。正确做法是只存 state_dict（注意上面说的 key 前缀问题），加载时重新构造模型再 compile。要真正序列化编译产物得走 torch.export + AOTInductor（见第 16 章）。

面试考点. Q：编译区里 print(x) 会发生什么？

A：一次 graph break。print 需要张量的真实值，而 Dynamo 在 trace 时手里只有 FakeTensor，所以它只能把图切开、退回 eager 执行这一行、再重新开图。调试时想看中间值，用 TORCH_LOGS="+graph_breaks,output_code" 或者 torch._dynamo.graph_break() 显式标记，别靠 print。

面试考点. Q：Inductor 的编译缓存存在哪？换机器能复用吗？

A：默认在 /tmp/torchinductor_\$USER（可以用 TORCHINDUCTOR_CACHE_DIR 改）。同机器上二次启动能命中，冷启动时间从几十秒降到几秒。换机器有条件可以复用：需要相同的 torch 版本、相同的 GPU 架构（sm_80 的产物不能给 sm_90 用）、相同的编译配置。CI 里预热这个目录是很常见的优化。

面试考点. Q：torch._dynamo.config.cache_size_limit 是什么？超了会怎样？

A：同一个代码位置允许缓存的已编译版本数上限（默认 8）。每种不同的 guard 组合（shape、dtype、Python 类型、某个 bool 标志）都占一份。超过之后 Dynamo 放弃编译、静默退回 eager，只在 TORCH_LOGS="recompiles" 里留一行日志——表现就是“跑了一会儿突然变慢且没人报错”。看到它超限不要直接调大上限，先去查为什么有这么多个版本（通常是 shape 没 bucket，见第 15 章）。

分布式

面试考点. Q：torchrun 和 mp.spawn 怎么选？

A：新代码一律 torchrun。它帮你设好 RANK / WORLD_SIZE / LOCAL_RANK / MASTER_ADDR 等环境变量，支持多机、支持 --max-restarts 做故障重启、和 rendezvous 后端集成。mp.spawn

是在一个 Python 进程里 fork 子进程，只能单机，而且父进程一旦提前 import 了 CUDA 就会出问题。

面试考点. Q：RANK、LOCAL_RANK、WORLD_SIZE 分别是什么？

A：RANK 是全局进程编号（0 到 WORLD_SIZE - 1），WORLD_SIZE 是总进程数，LOCAL_RANK 是本机内的编号。绑设备用 LOCAL_RANK（torch.cuda.set_device(local_rank)），判断“是不是主进程”用 RANK == 0。混用是新手最常见的错：用 RANK 绑设备，第二台机器上就会去访问不存在的 cuda:8。

面试考点. Q：哪些事只该 rank0 做？

A：写日志和 TensorBoard、保存 checkpoint（除非用 DCP，见第 22 章）、下载数据集或预训练权重、打印进度条、上报监控指标。核心判断是有外部副作用的操作。反过来，所有 collective 必须全 rank 都调用——把 all_reduce 写在 if rank == 0: 里就是直接 hang。

面试考点. Q：dist.barrier() 什么时候真的需要？

A：只有当“某个 rank 的非通信副作用”必须先于其他 rank 的动作时才需要——典型是 rank0 下载/解压数据集，其他 rank 等它完成再打开文件。普通训练循环里不需要 barrier，因为 collective 本身就是同步点，多余的 barrier 只会掩盖负载不均衡（把 straggler 的等待从通信挪到 barrier 上，profiler 里更难看出来）。NCCL 后端调 barrier 记得传 device_ids。

面试考点. Q：DDP 下各 rank 的 dropout mask 应该一样吗？

A：不应该一样，而且默认就不一样（各 rank 的 RNG 状态独立演化）。因为各 rank 处理的是不同数据，dropout 本来就该独立采样——强行同步等于把有效 batch 的正则化多样性砍掉。必须一样的是模型初始化，DDP 构造时会从 rank0 broadcast 一次参数和 buffer 来保证这一点。

面试考点. Q：用了 DistributedSampler 还要不要给 DataLoader 传 shuffle=True？

A：不要，会直接报错（sampler 和 shuffle 互斥）。shuffle 交给 sampler：DistributedSampler(ds, shuffle=True)，并且每个 epoch 调 sampler.set_epoch(epoch)，否则每个 epoch 的顺序完全一样。

面试考点. Q：8 卡 DDP、per-GPU batch 16、梯度累积 4 步，effective batch size 是多少？

A： $16 \times 8 \times 4 = 512$ 。三个因子都要算进去——这是调 lr 时最容易少算一项的地方（尤其是加了累积之后忘了同步调整 lr 和 warmup 步数）。另外要注意“总 step 数”的定义：optimizer 更新的次数是 dataloader 迭代次数的 1/4，scheduler 的 total_steps 必须按前者算。

面试考点. Q：为什么 DDP 要求所有 rank 的模型结构完全相同？

A：因为 DDP 在构造时按参数的注册顺序划分梯度 bucket，所有 rank 必须得到完全一致的 bucket 划分和 AllReduce 顺序。哪怕只是某个 rank 多了一个 buffer 或者层的定义顺序不同，通信就会错位——表现为 hang（等一个永远不来的 collective）或者梯度被张冠李戴（更可怕，因为不报错）。

面试考点. Q : `gradient_as_bucket_view=True` 干什么的 ?

A : 让 `p.grad` 直接指向 DDP 通信 bucket 里的一段 (view), 而不是单独分配一份再拷进 bucket。省掉一整份梯度的显存 (约 $|P|$) 和一次拷贝。副作用是 `p.grad` 的生命周期和 bucket 绑定, 不要长期持有它的引用, 也不要对它做改变 storage 的原地操作。

面试考点. Q : 多机训练 `MASTER_ADDR` 该填谁 ?

A : **rank 0** 所在节点的地址 (且必须是其他节点真正能连上的那个网卡地址, 不是 `127.0.0.1`)。所有节点填同一个值。配套的 `MASTER_PORT` 要选一个空闲端口, 同一台机器上跑多个任务时必须错开。连不上通常是防火墙或者填了内网/外网中不通的那一个。

面试考点. Q : NCCL 报 “unhandled system error” 第一步查什么 ?

A : 先加 `NCCL_DEBUG=INFO` 拿到真实原因 —— 这个报错本身几乎不含信息。最常见的三类根因 : 网卡选错 (设 `NCCL_SOCKET_IFNAME`)、共享内存不足 (容器里 `/dev/shm` 太小, `--shm-size` 调大或设 `NCCL_SHM_DISABLE=1` 验证)、P2P/IB 不可用 (`NCCL_P2P_DISABLE=1` 试一下能不能跑通来定位)。

手写题速答

面试考点. Q : LayerNorm 和 RMSNorm 差几个操作 ?

A : RMSNorm 去掉了减均值和偏置 β , 只除以均方根 : $x / \sqrt{\text{mean}(x^2) + \varepsilon} \cdot \gamma$ 。省一次求均值的 reduction 和一次减法, 反向也少一项。实测在 LLM 上效果不掉, 所以 LLaMA 之后基本成了默认选择。

面试考点. Q : attention 为什么要除以 $\sqrt{d_k}$?

A : $q \cdot k$ 是 d_k 个乘积之和, 输入方差为 1 时它的方差是 d_k 、标准差 $\sqrt{d_k}$ 。 $d_k = 128$ 时 logits 的量级就到 ± 11 , softmax 严重饱和、梯度接近 0。除以 $\sqrt{d_k}$ 把方差归一化回 1。注意除的是 $\sqrt{d_k}$ (单头维度) 而不是 $\sqrt{d_{\text{model}}}$ 。

面试考点. Q : multi-head attention 的 head 数变多, 参数量变吗 ?

A : 不变。 W_Q 、 W_K 、 W_V 、 W_O 都是 $d_{\text{model}} \times d_{\text{model}}$, 与 head 数无关 —— head 只是把这个矩阵的输出在通道维上切分成 h 段, $d_h = d_{\text{model}}/h$ 。变的是 attention 矩阵的个数 (h 个 $S \times S$) 和 softmax 的粒度。所以 head 数是纯粹的“表达方式”超参, 不影响参数预算。

面试考点. Q : KV cache 为什么只 cache K/V 不 cache Q ?

A : 因为解码时只需要当前这一个 **token** 的 q , 它每步都是新的、下一步用不上; 而所有历史 token 的 k 、 v 每一步都要被重新用一次。这个不对称正是“prefill 是 compute-bound、decode 是 memory-bound”的根源。

面试考点. Q : RoPE 和可学习绝对位置编码的关键区别 ?

A: 绝对 PE 是加在 embedding 上的一张可学习表, 长度写死; RoPE 是对 q 、 k 做一个与位置相关的旋转, 使得注意力分数只依赖相对位置 $i - j$ 。后果是 RoPE 没有可学习参数、外推到更长上下文更容易 (配合 NTK / YaRN 插值), 而绝对 PE 换长度必须重训或插值 (ViT 那一套, 见第 27 章)。

面试考点. Q: causal mask 应该填 `-inf` 还是一个很大的负数?

A: 用 `torch.finfo(dtype).min` 或直接 `-inf`, 不要硬编码 `-1e9` —— fp16 的最大值是 65504, `-1e9` 会直接变成 `-inf` 之外的溢出行为。用 `-inf` 的唯一风险是“整行全被 mask”时 softmax 得到 $0/0 = \text{NaN}$ (padding 全掩的行会遇到), 所以变长序列要么保证每行至少有一个可见位置, 要么事后把 NaN 行清零。

面试考点. Q: `nn.Embedding` 的反向是什么操作?

A: `index_add_ / scatter_add_` —— 把上游梯度按 token id 累加进一个 (V, D) 的零张量。所以同一个 token 在 batch 里出现多次, 梯度会正确累加; 也所以这个反向是原子加, 在 GPU 上是非确定性的 (见第 10 章, `use_deterministic_algorithms` 会换一个慢但确定的实现)。

面试考点. Q: dropout 放在 residual 相加之前还是之后?

A: 之前 —— $x = x + \text{Dropout}(\text{Sublayer}(\text{LN}(x)))$ 。dropout 作用在子层的输出上, residual 那条通路必须保持干净, 否则梯度高速公路被随机切断, 深层网络训不起来。放错位置的表现是收敛明显变慢但不报错。

面试考点. Q: weight tying 能省多少参数?

A: 省一整个 $V \times d$ 的输出投影。GPT-2 small ($V = 50257$ 、 $d = 768$) 省 38.6 M, 占 124 M 总参数的 **31%**; LLaMA-7B ($V = 32000$ 、 $d = 4096$) 省 131 M, 只占 **2%**。所以它对小模型是关键优化、对大模型基本可忽略 —— LLaMA 就没有 tie。

面试考点. Q: SwiGLU 的 FFN 中间维度为什么常取 $8d/3$ 而不是 $4d$?

A: 为了保持参数量不变。标准 FFN 有 2 个 $d \times h$ 矩阵, $h = 4d$ 时参数是 $8d^2$; SwiGLU 有 3 个 (gate、up、down), 要让 $3dh = 8d^2$ 就得 $h = 8d/3$ 。实现里还会把它向上取整到 128 或 256 的倍数以对齐 tensor core。

面试考点. Q: AdamW 的优化器状态占多少?

A: m 和 v 各一份、都是 fp32, 所以是参数数量的 **2 倍元素**、**8 字节** 每参数。混合精度训练下还要加 fp32 master weight (4 字节), 合计 12 字节; 再加 bf16 的参数和梯度各 2 字节, 就是常说的每参数 **16 字节**。7B 模型光这部分就是 112 GB —— 这是 ZeRO / FSDP 存在的全部理由 (见第 19 章)。

面试考点. Q: 手写一个 op 之后, 怎么快速确认它和 `torch.nn` 的实现等价?

A: 三步。数值对齐: 随机输入上 `torch.testing.assert_close`, 覆盖多组 shape 和边界 (batch=1、序列长 1、含负数)。梯度对齐: 同一份输入分别 backward, 比对所有参数的 `.grad`

；自定义 `autograd.Function` 还要在 `float64` 上过 `gradcheck`。退化对齐：找一个超参使你的实现精确退化成官方的某个已知情形（`gamma=0` 退化成 CE、`top_k=E` 退化成 dense）——这类测试抓 bug 的效率最高。

开放题与故障排查

这一节的题没有一句话答案。面试官想看的是你有没有一套从现象到根因的系统方法，以及你会不会在信息不足时先问清楚。回答的通用结构是：先复述我的理解和假设 → 给出排查顺序和每步的判据 → 说明什么情况下推翻假设换方向。

面试考点. Q：训练前 20 万步都正常，某一步 loss 突然变 NaN，怎么排查？

A：第 10 章给了 NaN 的高发位置清单（`log(0)`、除零、`sqrt` 的反向、softmax 整行被 mask、fp16 溢出）。这道题的重点不是那个清单，而是“跑了很久才炸”这个时间信息该怎么用——它排除了“实现本身就错”，把嫌疑集中在“某个特定 batch”或“某个缓慢累积的量”上。

第一步是把范围缩到一个 **step**。三个判据同时看：`loss.item()`、`grad_norm`（`clip_grad_norm` 的返回值本来就要算）、以及 AMP 的 `scaler.get_scale()`。如果 `grad_norm` 在炸之前几十步就在持续攀升 → 是缓慢发散，看 `lr / warmup /  $\beta_2$` ，属于超参问题。如果前一步一切正常、这一步直接 NaN → 是这个 **batch** 的问题，去 dump 它。如果 `scaler` 的 `scale` 在反复减半 → 是 fp16 溢出，换 bf16。

第二步是定位到层。用 `register_full_backward_hook` 给每层挂一个 `isfinite` 检查，找到第一个出 NaN 的层；需要精确到行就开 `set_detect_anomaly(True)` 复现一次（慢好几倍，只用于复现）。注意区分“forward 就 NaN”和“forward 正常但 backward NaN”——后者指向导数在边界上发散的 op。

第三步是确认根因可复现：把那个 batch 单独喂进去，看能不能稳定重现。能重现就是数据问题（越界 label、全 padding 的序列、脏样本）；不能重现就要怀疑非确定性来源（原子加、多卡不一致、显存越界的自定义 kernel）。

最后说工程上的预防：`grad_norm` 和每层激活的 **max** 应该是常态监控指标。NaN 几乎总有几十步的前兆，有这两条曲线就能在事后五分钟定位，没有就只能重跑。再加一道廉价守卫：`if not torch.isfinite(total_norm): skip this step`，别让 NaN 进参数——一旦进了参数，后面所有 step 都是 NaN，现场就被破坏了。

面试考点. Q：给你 8 卡 A100-80G，训一个 13B 模型，你怎么配并行？

A：先算显存账。13B 参数在 bf16 混合精度 + AdamW 下，固定开销约 $13 \times 10^9 \times 16 = 208 \text{ GB}$ ，8 卡总共 640 GB——装得下，但单卡 80 GB 放不下 26 GB 的固定开销加 activation，所以必须切。

方案排序：(1) 首选 **FSDP / ZeRO-3**，`auto_wrap_policy` 按 transformer block 包，`HYBRID_SHARD` 在单机 8 卡上通常不必要（NVLink 带宽足够）。8 卡全切之后每卡固定开销降到 26 GB 左右，剩下 50+ GB 给 activation，配 gradient checkpointing 就能开到不错的序列长度和 batch。(2) 如果序列很长导致 activation 仍然爆，再叠 **TP=2** 或 **4**（机内 NVLink，绝不跨机）+ FSDP 在剩下的维度上做 DP，并开 sequence parallel 摊掉 LayerNorm 的 activation。(3) **PP** 在单机 8 卡上一般不用——bubble 和实现复杂度都不划算，PP 是跨机才有价值。

然后要问清的前提：序列长度多少（决定 activation 是不是主要矛盾）、要不要全参微调还是 LoRA（LoRA 的话优化器状态几乎归零，单卡 DDP 就够了）、吞吐目标是多少。最后一定说：先跑一个 **profile** 拿 **MFU**，再决定继续调哪一维——配置是量出来的不是猜出来的。

面试考点. Q：如何判断训练是 CPU bound 还是 GPU bound？

A：三个层次，从便宜到贵。

(1) 最快的判据：把 dataloader 换成一个反复吐同一个预先搬到 GPU 上的假 batch，其他不动。step time 明显变快 → 瓶颈在数据侧（CPU / IO）；几乎不变 → 瓶颈在 GPU 计算或通信。这个实验五分钟能做完，应该是第一步。

(2) 看指标：nvidia-smi dmon 或 DCGM 看 GPU-Util 的时间序列而不是瞬时值——数据瓶颈的典型特征是“忙一段、空一段”的锯齿。同时看 CPU 的 %usr（如果某个 worker 进程吃满一个核，说明预处理是瓶颈）和磁盘 iostat。注意 GPU-Util 100% 不代表算力跑满（见前面那题）。

(3) 看 **profile**：torch.profiler 开 CPU + CUDA，看 timeline 上 GPU 那一行有没有空隙，以及空隙对齐到哪个 CPU 事件。空隙前面是 enumerate(loader) 就是数据；是 all_reduce 就是通信；GPU 密不透风但吞吐还是低，就去算 MFU、看是不是 kernel 本身效率低。

确认是 CPU bound 之后的优化顺序：加 num_workers → pin_memory + non_blocking → persistent_workers → 把预处理挪到 GPU 或离线预处理成二进制格式（webdataset / mmap）→ 最后才考虑换更快的解码库（见第 4 章）。

面试考点. Q：训练 loss 一路正常下降，但验证指标很差，怎么查？

A：先分清是过拟合还是有 **bug**，两者的曲线形状不同：过拟合是 val loss 先降后升，bug 是 val 从一开始就不对或者和 train 差一个数量级。

bug 优先级：(1) **train/eval** 预处理不一致——归一化的均值方差、resize 方式、tokenizer 的截断策略，这是最高频的原因。(2) 忘了 **model.eval()**——dropout 还在丢、BN 还在用当前 batch 统计。(3) **BN** 的 **running stats** 有问题——batch 太小导致统计量噪声大，或者用了 EMA 却没同步 buffer（见第 28 章）。(4) 数据泄漏或标签错位——划分时同一个来源的样本同时进了 train 和 val，或者 val 的 label 顺序被 shuffle 打乱了。(5) 评测代码本身写错——先用“训练集的一小部分”当验证集跑一遍，如果这样指标也差，问题一定在评测路径而不是泛化。最后这个技巧值得单独记：用 **train** 的子集当 **val** 是分离“泛化问题”和“代码问题”最快的实验。

面试考点. Q：复现一篇论文，指标差 2 个点，怎么定位？

A：按“影响量级”从大到小查，别从超参开始猜。(1) 数据——版本、预处理、增强、划分方式；数据差异通常就能解释好几个点。(2) 有效 **batch size** 和总训练步数——论文常报“8 卡 × bs 32”，你在 2 卡上跑 bs 32 就差了 4 倍，lr 和 warmup 都得跟着改。(3) 学习率 **schedule** 的细节——warmup 步数、min_lr、是按 step 还是按 epoch。(4) 正则项——weight decay 有没有排除 bias/norm、label smoothing、EMA、mixup 的概率。(5) 评测协议——是否用 EMA 权重、是否多尺度/TTA、指标定义是否一致。(6) 最后才是初始化和随机种子——单个种子的方差常有 0.3-0.5 个点，先用 3 个种子确认这 2 个点是不是噪声。

方法论上说一句会加分：一次只改一个变量，并且优先改那些能把差距一次性抹掉大半的。

面试考点. Q：训练跑了几百步之后才 OOM，怎么办？

A：“跑了一会儿才 OOM”这个现象本身就是最强的线索 —— 说明有东西在单调增长，不是单步峰值不够。

查这几处：(1) 把带图的张量存进了 **list** —— `losses.append(loss)` 而不是 `loss.item()`，整张 autograd 图被保活，这是第一嫌疑人。(2) 变长输入 —— 序列变长时峰值随之涨，加上碎片就在某个长 batch 上炸；判据是看 `memory_reserved` 是否远大于 `memory_allocated`，修法是按长度分桶、或开 `expandable_segments`。(3) 误用 `retain_graph=True` 或者跨 step 带 hidden state 没 `detach()`。(4) 缓存类的字典无界增长 —— 自己写的 KV cache、metrics 累加器。(5) **eval** 循环没包 `no_grad()` —— 每次验证都攒一堆 activation。

工具：`torch.cuda.memory._record_memory_history()` + `_dump_snapshot()` 拿到分配栈，在 pytorch.org/memory_viz 上看是哪一行的分配在持续增长（见第 8 章）。这是这类问题的决定性手段，比逐行注释快得多。

面试考点. Q：同一份代码换到新集群慢了 30%，怎么查？

A：先分层：单卡慢，还是只有多卡慢？跑一个单卡 step time 对比就能分开，这决定了后面查计算还是查通信。

单卡就慢：查 GPU 型号和功耗墙（`nvidia-smi -q -d PERFORMANCE` 看有没有 SW Power Cap 或热降频）、CUDA / cuDNN / torch 版本差异（TF32 默认值、cuDNN benchmark、新版 SDPA 后端选择都会变）、CPU 侧的 dataloader（新集群的存储可能慢得多，或者 `num_workers` 相对新的 CPU 核数不合适）。

只有多卡慢：查拓扑（`nvidia-smi topo -m`，是不是从 NVLink 掉到了 PCIe）、网卡选择（`NCCL_DEBUG=INFO` 看它实际用了哪个 interface，有没有走上了慢的管理网口）、是否有 straggler（每 rank 记 step time，看是不是固定某张卡慢）、`/dev/shm` 大小。

两边都要做的一件事：在两个集群上跑同一个 **profile**，对比 **step time** 的 **breakdown**（compute / comm / dataloader 三段的绝对值）。哪一段变长了，答案就在那一段里，不用继续猜。

面试考点. Q：单卡训练正常，换 DDP 之后 loss 不降（不是 hang），怎么查？

A：DDP 下“能跑但不收敛”的根因集中在几处。(1) **lr** 没跟着有效 **batch** 调 —— 8 卡的有效 batch 是 8 倍，沿用单卡 lr 相当于 lr 小了 8 倍，表现就是“降得特别慢”。(2) 各 **rank** 的初始化不一致 —— 正常情况 DDP 构造时会 broadcast，但如果你在构造 DDP 之后才改了参数（比如加载 checkpoint、重新初始化某一层），各 rank 就分叉了；验证方法是对所有参数做一次 `all_reduce` 求和再比对哈希。(3) 数据没正确切分 —— 忘了 `DistributedSampler`，每个 rank 都在跑全量数据，等于一个 epoch 重复了 8 次；或者忘了 `set_epoch`，每个 epoch 顺序完全一样。(4) **BatchNorm** —— 每张卡只用自己那 1/8 的 batch 算统计量，小 batch 下噪声大，换 `SyncBatchNorm` 或 `GroupNorm`。(5) **loss** 归约方式不一致 —— 各 rank 的 batch 大小不等时，简单平均梯度就不等于全局平均。

自检的一招：把 **world size** 设成 1 用 `torchrun` 跑。如果这样正常、2 卡就不对，问题一定在“跨 rank 的一致性”这一类；如果 world size 1 就已经不对，问题在你为分布式改的那些代码本身。

面试考点. Q：给你一个别人写的训练脚本，怎么在一天内把吞吐提一倍？

A：先测量，按投入产出比排序，每改一步重测。

第一小时只做一件事：拿到 step time 的 breakdown (dataloader / compute / comm) 和当前 MFU。没有这个数字，后面全是瞎猜。

然后按“改动小、收益大”排：(1) 确认开了 **AMP** (bf16)，这通常是最大的单项收益。(2) **dataloader**：num_workers、pin_memory、persistent_workers，如果 breakdown 显示数据占比高，这里几行代码就能吃掉一大块。(3) 消掉训练循环里的隐式同步（每步的 .item()、print、assert 张量条件）——见第 9 章。(4) zero_grad(set_to_none=True)、torch.backends.cuda.matmul.allow_tf32 = True。(5) **torch.compile** ——收益通常 10%-40%，但要先确认 shape 稳定、没有密集 graph break。(6) 如果显存有余量，加大 **batch** 并按比例调 lr (memory-bound 的 op 直接受益)。(7) 通信占比高就调 bucket_cap_mb、关掉不需要的 find_unused_parameters。

最后说一句边界：不改变数值语义的优化先做完，再考虑会动收敛行为的（换 **dtype**、换 **batch size**、换 **fused optimizer**），而且后者必须跑一段曲线确认收敛没退化。

面试考点. Q：估算训练一个 7B 模型需要多少卡多少天。

A：用 Chinchilla + MFU 两步就能给出量级。

(1) 总 **FLOPs**：经验公式 $C \approx 6ND$ (N 参数量、 D 训练 token 数，系数 6 来自 forward 2 倍加 backward 4 倍)。7B 按 Chinchilla 最优配 20 token/参数是 140B token， $C \approx 6 \times 7 \times 10^9 \times 1.4 \times 10^{11} \approx 5.9 \times 10^{21}$ FLOPs。

(2) 算力：A100 的 bf16 峰值是 312 TFLOPS，实际 MFU 取 40%-50% (这是 7B 稠密模型在良好实现下的合理区间)，即约 1.3×10^{14} FLOP/s 每卡。

(3) 单卡时间 $\approx 5.9 \times 10^{21} / 1.3 \times 10^{14} \approx 4.5 \times 10^7$ 秒 ≈ 525 卡·天。所以 **64 张 A100 约 8 天**，**128 张约 4 天** (假设扩展效率接近线性，实际要打 85%-95% 的折扣)。

面试时要主动交代三件事：系数 **6** 的来源、**MFU** 是假设值必须实测、没算上 **checkpoint** 恢复和失败重跑的开销 (真实项目按 **1.2-1.5 倍** 留余量)。愿意把假设摆出来、并说“这些数我会先用小规模跑一个 scaling 实验校准”，比给一个精确到小数点的答案更让人放心。

附录 A：常见报错速查

拿着报错原文来查这一章。小节标题就是报错里最有辨识度那段字符串，直接搜关键字。每条给三样：报错原文（torch 2.10 实际触发的输出，老版本措辞可能略有出入，为排版做了换行）、成因（通常不止一种）、最小修法。

面试也考这些——“你遇到过什么棘手的 bug”、“CUDA OOM 怎么排查”本质上就是这张表。答的时候别只说修法，要说怎么定位到成因。

view size is not compatible with input tensor's size and stride

```
RuntimeError: view size is not compatible with input tensor's size and stride
(at least one dimension spans across two contiguous subspaces).
Use .reshape(...) instead.
```

成因. view 要求新 shape 能用一组合法 stride 走完原 storage，前提是被合并的那些维度连续。来源几乎总是这三个之一：前面做过 transpose / permute（stride 被交换）；做过非最外维的切片如 `x[:, 1:3]`；或者是 expand 出来的广播视图（stride=0）

修法. `x.reshape(12)`（能 view 就 view，不能就自动 copy）或 `x.contiguous().view(12)`（显式表明这里有一次拷贝）。两者结果一样，差别是意图表达：性能敏感路径用后者，让 review 的人一眼看到开销。判定规则见第 1 章。

Expected all tensors to be on the same device

两种措辞，来自不同的检查点：

```
RuntimeError: Expected all tensors to be on the same device, but found at least
two devices, cuda:0 and cpu!
RuntimeError: Expected all tensors to be on the same device, but got mat1 is on
cpu, different from other tensors on cuda:0 (... wrapper_CUDA_addmm)
```

成因. 模型 `.cuda()` 了但 batch 没有，或反过来（`.to()` 对 module 是原地改、对 tensor 是返回新对象，`x.to("cuda")` 不写等号就白做了，见第 3 章）；forward 里 `torch.zeros(...)` / `torch.arange(...)` 没指定 device，默认落 CPU；常量存成普通属性而不是 `register_buffer`，`model.cuda()` 搬不动它；多卡时没设 `torch.cuda.set_device(local_rank)`，新建张量全跑到 `cuda:0`。

修法. 新建张量一律从已有张量取 device，不要硬编：

```
pos = torch.arange(S, device=x.device)          # 不是 device="cuda"
mask = torch.zeros_like(x, dtype=torch.bool)    # *_like 继承 device + dtype
self.register_buffer("freqs", freqs)           # 常量走 buffer，跟着 .to() 走
```

dtype 不匹配：must have the same dtype / should be the same

```
RuntimeError: mat1 and mat2 must have the same dtype, but got Half and Float
RuntimeError: Input type (c10::Half) and bias type (float) should be the same
```

```
RuntimeError: Expected query, key, and value to have the same dtype, but got
query.dtype: c10::Half key.dtype: float and value.dtype: float instead.
```

老版本里 `expected scalar type Float but found Half` 是同一类问题的另一种措辞。

成因. 手动 `.half()` 了模型但输入还是 `fp32` (最常见); **AMP** 边界——`autocast` 只对白名单 op 自动 cast, 其低精度输出流出上下文后再和 `fp32` 做 `matmul` 就炸; 自定义 `autograd.Function` 不在白名单里; 加载 `bf16` checkpoint 到 `fp32` 模型, 或 `LoRA` 与 `base` 精度不一致。

修法. 让 `AMP` 管精度, 不要手动 `.half()` 整个模型; 需要局部对齐时显式 `cast` 到目标那一边:

```
with torch.autocast("cuda", dtype=torch.bfloat16):
    loss = model(x)          # 边界只在这里
loss.backward()             # backward 不放进 autocast
y = y.to(w.dtype)          # 局部对齐, 别指望隐式提升
```

另外, `autocast` 里不要缓存中间结果给下个 iteration 用——存下来的是低精度张量, 下次可能落在 `autocast` 外面。AMP 完整用法见第 5 章。

CUDA out of memory. Tried to allocate ...

```
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 4096.00 GiB.
GPU 0 has a total capacity of 79.25 GiB of which 78.83 GiB is free. ... Of the
allocated memory X is allocated by PyTorch, and Y is reserved by PyTorch but
unallocated. If reserved but unallocated memory is large try setting
PYTORCH_ALLOC_CONF=expandable_segments:True to avoid fragmentation.
```

先读这条消息本身, 答案已经写在里面。关键是两个数的差: `allocated` 是真正被张量占着的, `reserved` 是 `caching allocator` 从驱动拿到、还没还回去的。 `allocated` 逼近上限是真 **OOM**; `reserved` 远大于 `allocated` (比如 70 GB vs 40 GB) 却申请不到几百 MB 连续块, 是碎片。

排查顺序。

1. 看 `Tried to allocate` 的数值。异常巨大 (几百 GB) 说明是 `shape` 算错了, 不是显存不够——去查那一行的 `reshape / broadcast`。
2. `torch.cuda.memory_allocated()` 对比 `memory_reserved()`, 区分真 OOM 与碎片。
3. 真 OOM : 减 batch、开 AMP、开 `activation checkpointing`、换 `FSDP` (性价比排序见第 8 章)。碎片: 设 `PYTORCH_ALLOC_CONF=expandable_segments:True`, 对变长 `shape` 场景效果显著。
4. 定位谁吃的: `torch.cuda.memory._record_memory_history()` + `_dump_snapshot()`, 用官方 `memory viz` 页面看。

Warning. `torch.cuda.empty_cache()` 只把 `reserved` 还给驱动, 不减少 **allocated**, 对真 OOM 毫无帮助, 还会让下次分配走慢路径。只在“把显存让给同机另一个进程”时才用。

a leaf Variable that requires grad is being used in an in-place operation

```
RuntimeError: a leaf Variable that requires grad is being used in an in-place
operation.
```

成因. 直接原地修改 `requires_grad=True` 的叶子张量 (也就是 `nn.Parameter`), `autograd` 需要叶子的值稳定, 禁止这么干。最常见是手动改权重: `w += 1`。

修法. 用 `torch.no_grad()` 包起来——optimizer 内部就是这么做的:

```
with torch.no_grad():
    w += 1; w.clamp_(-1, 1)
```

`w.data += 1` 也能跑, 但绕过了 version counter, 不推荐。初始化用 `nn.init.*` (内部已在 `no_grad` 里)。

one of the variables needed for gradient computation has been modified by an inplace operation

```
RuntimeError: one of the variables needed for gradient computation has been
modified by an inplace operation: [torch.FloatTensor [3]], which is output 0 of
ExpBackward0, is at version 1; expected version 0 instead. Hint: enable anomaly
detection ... torch.autograd.set_detect_anomaly(True).
```

成因. forward 时某个 op 把张量存进了 `ctx (save_for_backward)`, 之后这个张量被原地改了, backward 时 version counter 对不上。报错里的 `output 0 of ExpBackward0` 是关键线索: 被改的是 **exp** 的输出。典型来源: 残差写成 `x += block(x); relu_ / add_` 紧跟在保存输出的 op (`exp`、`sigmoid`、`tanh`、`softmax`) 后面; 自写 Function 里 `save_for_backward` 存了会被复用的 buffer。

修法. 把那处原地操作改成非原地。定位靠报错里的 `XxxBackward0` 名字, 找不到就开 `anomaly detection`:

```
x = x + block(x)          # 不是 x += block(x)
h = F.relu(h)              # 不是 F.relu(h, inplace=True)
with torch.autograd.set_detect_anomaly(True): # 只在 debug 时开, 很慢
    loss.backward()
```

不是所有原地操作都会炸: `relu_` 接在 `Linear` 后面是安全的, 因为 `addmm` 的 backward 不需要它的输出。会炸的只有“输出被 backward 用到”的那些 op (第 6 章的 version counter 一节)。

Trying to backward through the graph a second time

```
RuntimeError: Trying to backward through the graph a second time (or directly
access saved tensors after they have already been freed). ... Specify
retain_graph=True if you need to backward through the graph a second time ...
```

成因. 报错文本建议加 `retain_graph=True`, 但绝大多数情况下这是错的建议。真实成因按概率排: (1) 把带图的张量跨 **step** 累积了, 最经典是 `total_loss += loss` —— `loss` 带着整张图, 累加后上一步的图仍被引用, 下次 backward 会走回去; (2) RNN / 状态机里 hidden state 跨 step 传下去忘了 `detach()`; (3) 一次 forward 配了两次 `backward()` (GAN 里 D 和 G 共用一次 forward)。只有第 3 种是 `retain_graph=True` 的正当用法, 而且更好的解法通常是重新 forward 一次或用 `autograd.grad`。

修法. `total_loss += loss.item()` (记录标量用 `.item()` 或 `.detach()`); `h = h.detach()` (RNN 跨 step 截断)。

Warning. 盲目加 `retain_graph=True` 会让图一直不释放，症状是显存随 step 单调上涨、几十步后 OOM——一个报错换成另一个更难查的报错。

element 0 of tensors does not require grad and does not have a grad_fn

```
RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn
```

成因。你 `backward()` 的那个张量根本不在图上：整段 forward 被包在 `torch.no_grad()` / `inference_mode()` 里（推理代码复制过来忘了删）；中间 `.detach()` 了，或走了 `.item()` / `.numpy()` / `.tolist()` 再转回张量；所有参数都被 `requires_grad_(False)` 冻住（只训 LoRA 但忘了给 A/B 打开）；loss 是 `torch.tensor(...)` 现造的常量。

修法。从 loss 往回查一步：

```
print(loss.requires_grad, loss.grad_fn) # 期望 True, <SumBackward0 ...>
print([n for n, p in model.named_parameters() if p.requires_grad]) # 别是空的
```

`grad_fn` 是 `None` 说明断点在 loss 这一层；不为 `None` 但某个参数没梯度，看下一节。

一个细节：`inference_mode()` 产生的张量比 `no_grad()` 更“死”，它带了 inference tensor 标记，之后即使离开上下文也不能参与 autograd。验证代码用 `no_grad()` 更保险。

.grad 是 None（不报错，但更难查）

没有报错，`optimizer.step()` 照跑，就是参数不动。按这个顺序查：

1. 它是叶子吗？只有叶子的 `.grad` 会被填。`p = w.cuda()` 之后 `p` 不是叶子——正确写法是先建 module 再 `.cuda()`。访问非叶子的 `.grad` 会有一条 `UserWarning` 提示你用 `.retain_grad()`。
2. `requires_grad` 是 `True` 吗？路径上有没有 `detach()` / `no_grad()` / `.data` ？
3. 它参与 **forward** 了吗？定义了但 forward 里没用到的层梯度当然是 `None`（DDP 下还会直接 hang，见第 18 章 `find_unused_parameters`）。
4. 是不是在 `backward()` 之前、或 `zero_grad(set_to_none=True)` 之后看的？那时本来就是 `None`。
5. **optimizer** 拿到这个参数了吗？`model.cuda()` 在 `SGD(model.parameters())` 之后执行的话，optimizer 里存的是旧对象。顺序永远是先 `.to(device)` 再建 optimizer。

— 行 定 位 :

```
print([n for n, p in model.named_parameters() if p.requires_grad and p.grad is None])
```

CUDA error: device-side assert triggered

```
.../ATen/native/cuda/Indexing.cu:1515: indexSelectSmallIndex: block: [0,0,0],
thread: [0,0,0] Assertion `srcIndex < srcSelectDimSize` failed.
torch.AcceleratorError: CUDA error: device-side assert triggered
CUDA kernel errors might be asynchronously reported at some other API call, so
the stacktrace below might be incorrect.
For debugging consider passing CUDA_LAUNCH_BLOCKING=1.
```

成因. 几乎总是索引越界。两个高频来源：`nn.Embedding` 的某个 `token id` $\geq$ `num_embeddings` (词表和 `tokenizer` 对不上、忘了 `special token`)；`cross_entropy` / `nll_loss` 的某个 `label` $\geq$ `num_classes` 或为负数。同一个 bug 在 CPU 上报的是可读得多的 `IndexError: index out of range in self` 和 `IndexError: Target 5 is out of bounds.`。

修法. 先用 `CUDA_LAUNCH_BLOCKING=1 python train.py` 让 `stack trace` 指向真正出错那一行，再在数据入口补上 `assert ids.max() < emb.num_embeddings` 和 `assert labels.min() >= 0 and labels.max() < num_classes`。

Warning. CUDA context 被 `assert` 打挂后不可恢复，后续任何 CUDA 调用都报同一个错。看到它必须重启进程，不要在同一进程里继续 debug。异步语义见第 9 章。

Expected more than 1 value per channel when training

```
ValueError: Expected more than 1 value per channel when training, got input size torch.Size([1, 4])
```

成因. `BatchNorm` 在 `train` 模式下要算 batch 内方差，batch size 为 1 时方差无定义。最常出现在：最后一个不完整 batch 只剩 1 个样本；或者 `eval` 时忘了 `model.eval()`。

修法. `DataLoader(..., drop_last=True)` 丢掉尾 batch；推理前 `model.eval()`；小 batch 场景换 `nn.GroupNorm` / `nn.LayerNorm`。

Sizes of tensors must match except in dimension 0 / stack expects each tensor to be equal size

```
RuntimeError: Sizes of tensors must match except in dimension 0. Expected size 3 but got size 4 for tensor number 1 in the list.
RuntimeError: stack expects each tensor to be equal size, but got [2, 3] at entry 0 and [3, 3] at entry 1
```

成因. 两个函数契约不同，混用是主因：`cat` 沿已有维度拼接，除拼接维外其余维度必须完全一致，不增加维度；`stack` 新建一维，所有输入 **shape** 必须完全一致。变长序列走 `collate_fn` 时最容易撞上——不同样本长度不同却直接 `torch.stack`。

修法. 变长序列用 `torch.nn.utils.rnn.pad_sequence(seqs, batch_first=True)` 先 `pad` 再 `stack`。报错里的“tensor number 1”是列表下标，`print([t.shape for t in xs])` 直接看出是哪一个。

Error(s) in loading state_dict: Missing key(s) / Unexpected key(s)

```
RuntimeError: Error(s) in loading state_dict for Linear:
  Missing key(s) in state_dict: "weight", "bias".
  Unexpected key(s) in state_dict: "module.weight", "module.bias".
```

成因. 看两组 `key` 的差异形状就能判断：整齐地差一个 `module.` 前缀 $\rightarrow$ `checkpoint` 是从 `DDP` / `DataParallel` 包装后的模型存的；差一个 `_orig_mod.` 前缀 $\rightarrow$ 是从 `torch.compile` 包装后的模型存的；只多不少（多出 `num_batches_tracked` 之类） $\rightarrow$ 版本差异或 `buffer` 的 `persistent` 设置变了；结构真变了（加层、改名）才需要改代码。

修法. 存的时候就存干净的, 这是根治: `torch.save(model.module.state_dict(), path)` (DDP 存内层) `torch.save(model._orig_mod.state_dict(), path)` (compile 存原模型)。已经存脏了就在 load 时剥前缀: `sd = {k.removeprefix("module."): v for k, v in sd.items()]`。

Warning. `strict=False` 能让它跑起来, 但会静默忽略所有对不上的 key——权重没加载完, 训练看着正常但效果莫名其妙差。真要用就检查返回值 `missing, unexpected = model.load_state_dict(sd, strict=False)` 并打印出来人工确认。

DataLoader worker (pid xxx) is killed by signal

```
RuntimeError: DataLoader worker (pid 12345) is killed by signal: Killed.
It is possible that dataloader's workers are out of shared memory. Please try
to raise your shared memory limit.
RuntimeError: DataLoader worker (pid(s) 12345) exited unexpectedly
```

成因. worker 是子进程, 它死了主进程只看得到信号。三种来源: 共享内存不足——worker 通过 `/dev/shm` 把 batch 传回主进程, Docker 默认 `--shm-size=64m`, 大 batch 或大图像立刻爆, 这是容器里最常见的原因; 宿主机 OOM 被内核 `kill` (`signal: Killed = SIGKILL`), worker 是 fork 出来的, Python 对象的引用计数会让 copy-on-write 失效, 实际内存远超预期; worker 里抛了异常, 这种通常能看到 `Caught ValueError in DataLoader worker process 0`。加原始 traceback, 好查得多。

修法. 容器加 `--shm-size=8g` 或 `--ipc=host`; debug 第一步永远是 `num_workers=0`, 报错会原样抛出来, 八成问题当场清楚。降 `num_workers`、把 Dataset 里的大 Python list 换成 numpy array 也能缓解内存问题 (第 4 章)。

Cannot re-initialize CUDA in forked subprocess

```
RuntimeError: Cannot re-initialize CUDA in forked subprocess. To use CUDA with
multiprocessing, you must use the 'spawn' start method
```

成因. CUDA context 不能跨 fork 继承。父进程只要初始化过 CUDA (哪怕只是建了一个 cuda 张量), fork 出来的子进程碰 CUDA 就炸。Linux 上 multiprocessing 默认就是 fork。

修法. 用 `spawn`: `mp.set_start_method("spawn", force=True)` (放在 `if __name__ == "__main__"` 里), 或直接 `mp.spawn(worker, nprocs=world_size)`。DataLoader 的 worker 里也不要碰 CUDA: augment 留在 CPU, `.cuda(non_blocking=True)` 在主进程做。生产上更简单的路子是用 `torchrun` 起独立进程, 从根上避开 fork。

Address already in use

```
torch.distributed.DistNetworkError: The server socket has failed to listen on
any local network address. port: 29500, useIpv6: false, code: -98,
name: EADDRINUSE, message: address already in use
```

成因. rank 0 要在 `MASTER_PORT` 上建 TCPStore, 端口被占了: 上次训练崩了进程没清干净; 同机跑了两个 job 都用默认的 29500; 或端口被别的服务占用。

修法.

```
pkill -f train.py # 先清残留
torchrun --rdzv-backend=c10d --rdzv-endpoint=localhost:0 ... # 让它自己挑端口
```

或者 `MASTER_PORT=$((20000 + RANDOM % 10000))` 随机一个。多 job 共机时把端口按 job id 算进启动脚本，不要靠人记。

NCCL : `unhandled system error` 与 `Watchdog caught collective operation timeout`

```
NCCL WARN Bootstrap : no socket interface found
ncclInternalError: Internal check failed. ... unhandled system error
[Rank 3] Watchdog caught collective operation timeout: WorkNCCL(...,
OpType=ALLREDUCE, Timeout(ms)=600000) ran for 600351 milliseconds before
timing out.
```

这两条是分布式面试的高频题，成因完全不同。

unhandled system error 一般是环境问题：网卡选错（多网卡机器上 NCCL 挑了张不通的，比如 `docker0`）→ `NCCL_SOCKET_IFNAME=eth0` 指定；IB 不可用却在走 IB → `NCCL_IB_DISABLE=1` 先退回 socket 验证连通性；容器 `/dev/shm` 太小或没开 `--ipc=host`。排查第一步永远是 `NCCL_DEBUG=INFO`，它会打印挑了哪张网卡、走什么传输。

Watchdog caught collective operation timeout 是 **collective** 对不齐，按概率排：

1. 某个 **rank** 没调这个 **collective**。 `if rank == 0: dist.all_reduce(...)` 这种写法必挂——所有 rank 必须以相同顺序调用相同的 collective。
2. **shape / dtype** 不一致。变长 batch 下不同 rank 算出的 tensor 大小不同。
3. 某个 **rank** 挂在别处：等 IO、OOM 重试、或已经抛异常退出——其他 rank 集体超时。
4. 慢卡：某张卡降频或被别的进程占用，超过 timeout（默认 10 分钟）。

定位。 `TORCH_NCCL_DESYNC_DEBUG=1` 会在超时打印每个 rank 卡在哪个 collective，直接看出谁没跟上；配合 `py-spy dump --pid <pid>` 看 Python 栈。完整 hang 排查流程见第 22 章。

`torch._dynamo.exc.Unsupported`

```
torch._dynamo.exc.Unsupported: Data-dependent branching
Explanation: Detected data-dependent branching (e.g. `if my_tensor.sum() > 0:`).
Dynamo does not support tracing dynamic control flow.
Hint: Use `torch.cond` to express dynamic control flow.
from user code: File "model.py", line 42, in forward
    if x.sum() > 0:
```

成因。这是一个 **graph break**，只在 `fullgraph=True` 下才变成异常（默认 `fullgraph=False` 时它静默切图，代价是编译收益打折）。常见触发源：依赖张量值的控制流（`if x.sum() > 0`、`while loss > tol`）；`print` / `pdb` / 写日志；`.item()` / `.tolist()` / `.cpu()` 这类强制同步；Dynamo 不认识的 C 扩展和第三方库。

修法。报错里的 `from user code:` 已经指到具体行。

```
y = torch.where(cond, a, b) # 张量条件用 where，不用 if
torch._dynamo.graph_break() # 主动在这里切，接受两段图
```

```
@torch._dynamo.disable          # 整个函数不编译
def log_metrics(loss): ...
```

先不加 `fullgraph=True` 跑一遍，用 `TORCH_LOGS="graph_breaks"` 看有几个 break、在哪，确认没有致命的再打开 `fullgraph=True` 锁住。机制见第 12 章。

hit config.recompile_limit

```
torch._dynamo hit config.recompile_limit (8)
  function: 'forward' (model.py:42)
  last reason: 0/7: tensor 'x' size mismatch at index 0. expected 16, actual 17
```

成因。同一个函数被反复重编译，超过上限后 Dynamo 静默 **fallback** 到 **eager**。这是“加了 `torch.compile` 但没变快”的头号原因——它是 warning 不是 error，很容易被日志淹掉。触发源：输入 shape 一直变（变长序列、尾 batch）；forward 依赖一个每次都不同的 Python 标量（`step`、`temperature`）；每次传进来新建的 Python 对象（`config dataclass`、`dict`）。

修法。 `torch.compile(model, dynamic=True)` 编动态 shape 版本；`DataLoader(..., drop_last=True)` 消掉尾 batch；把变动的标量做成张量参数；或者把 shape bucket 化（pad 到 128 的倍数）。定位一律用 `TORCH_LOGS="recompiles"`，它会打印每次重编的具体 guard 原因。调参见第 15 章。

loss 变成 NaN

不是报错，但排查成本最高。按这个顺序查，每一步都能二分定位：

1. 定位第一个 NaN 在哪一步。`assert torch.isfinite(loss)`，`step`。是某一步突然开始（数据 / lr spike）还是逐渐发散（lr 太大）？
2. 查输入与数值稳定性。`assert torch.isfinite(x).all()` 排除脏数据、除零归一化、全 pad 样本（softmax 整行被 mask 会出 NaN）；再查有没有手写 `log(softmax(x))` 而不用 `log_softmax`、`log(p)` 时 `p` 可能为 0、`sqrt(x)` 在 0 处梯度是 inf。
3. 查 **lr** 与 **warmup**。lr 太大、warmup 太短、加载 checkpoint 时 scheduler 状态没恢复导致 lr 跳回峰值。
4. 查 **AMP**。fp16 动态范围只到 65504，激活或梯度溢出会变 inf 再变 NaN。`GradScaler` 处理的是梯度下溢，处理不了前向溢出。换 **bf16** 是最快的验证手段——bf16 指数位和 fp32 一样宽，换过去就好了基本可确诊是 fp16 range 问题。
5. 查梯度爆炸。打印 `clip_grad_norm_` 的返回值（它返回裁剪前的范数），看是不是某步突然涨了几个数量级。
6. 定位到具体 **op**：`torch.autograd.set_detect_anomaly(True)` 会在产生 NaN 梯度的 backward 节点抛异常并打印 forward 调用栈。很慢，只在 debug 时开。

Key insight. 前向正常、反向 NaN，几乎总是“梯度公式在某点无定义”：`sqrt(0)`、`log(0)`、`norm()` 在零向量处，以及 mask 用 `-inf` 而不是 `-1e9` 导致整行被 mask 时 softmax 出 NaN。加 `eps` 或换成有限大的负数即可。

报错关键字 → 去看哪一章

报错关键字	相关章节
-------	------

view size is not compatible is invalid for input of size	/	第 1、2 章
Expected all tensors to be on the same device		第 1、3 章
must have the same dtype Input type ... and bias type	/	第 1 章、第 5 章 (AMP)
CUDA out of memory / 碎片 / expandable_segments		第 8 章
leaf Variable ... in-place modified by an inplace operation	/	第 1、6 章
backward through the graph a second time retain_graph	/	第 6 章
does not have a grad_fn / .grad 是 None		第 6 章
device-side assert triggered / CUDA_LAUNCH_BLOCKING		第 9 章
Expected more than 1 value per channel		第 3 章、第 18 章 (SyncBN)
Sizes of tensors must match stack expects each tensor	/	第 2、4 章
Missing key(s) / Unexpected key(s) / module. 前缀		第 3、18、22 章
DataLoader worker ... killed by signal / shm 不足		第 4 章
Cannot re-initialize CUDA in forked subprocess		第 4、9 章
address already in use / MASTER_PORT		第 17 章
NCCL WARN Watchdog caught collective operation timeout	/	第 17、22 章
torch._dynamo.exc.Unsupported / graph break		第 12、15 章
hit config.recompile_limit / 反复重编译		第 12、15 章
loss NaN / inf / 数值不稳定		第 2、5、10 章
两次跑结果不一致 / TF32 / 加了 compile 没变快		第 10、11、15 章

附录 B：自测清单与复习路线

这一章不讲新知识，只做三件事：让你知道自己漏了什么、该按什么顺序补、面试前一晚看什么。

自测清单里的每一条都是“能力项”不是“知识点”——判据是合上书能不能讲出来，讲得磕磕巴巴等于不会。勾不上的按右边的章号回去读。

自测清单

Part 1 基础（第 1-5 章）

能力项	章
☐ 能用 stride 公式解释 tensor 的内存布局，说清哪些 op 是纯元数据操作	1
☐ 能说清 view 和 reshape 的区别，以及 view 失败的确切条件	1
☐ 能区分 expand 和 repeat（stride=0 的视图 vs 真拷贝）	1
☐ 能列出“共享内存”和“复制”两类操作，说清 clone / detach / .data 的差别	1
☐ 能说清 bf16 和 fp16 的差别，以及为什么大模型训练偏好 bf16	1
☐ 能手推广播规则，说清 matmul 的批维广播怎么算	2
☐ 能区分 basic indexing（返回视图）和 advanced indexing（返回拷贝）	2
☐ 能用 gather / scatter_add_ 写出 top-k 路由这类操作	2
☐ 能解释 softmax 为什么要减 max，log_softmax 为什么比 log(softmax) 稳	2
☐ 能说清 nn.Module 的四张注册表，以及为什么 python list 装子模块是致命错误	3
☐ 能说清 register_buffer 与普通属性的差别、persistent=False 的用途	3
☐ 能说清 train() / eval() 只影响哪两类层，以及 eval() 和 no_grad() 的正交性	3
☐ 能说清 num_workers 的进程模型、pin_memory 为什么能加速	4
☐ 能写一个处理变长序列的 collate_fn，并说清多 worker 下 RNG 的坑	4
☐ 能判断 dataloader 是不是瓶颈（给出具体的测量方法）	4
☐ 能默写训练循环七行，并解释每一行为什么在那个位置	5
☐ 能说清 zero_grad(set_to_none=True) 省了什么、有什么行为差异	5
☐ 能写出正确的 AMP + GradScaler + 梯度裁剪的调用顺序（顺序错了会怎样）	5
☐ 能说清梯度累积的等价性条件，以及 BatchNorm 为什么是例外	5
☐ 能说清 AdamW 和 Adam+L2 的区别（不是同一个东西）	5

Part 2 原理（第 6-11 章）

能力项	章
☐ 能画出一个两层网络的反向图，标出 grad_fn 和叶子	6
☐ 能说清 leaf / requires_grad / .grad 三者的关系，以及 requires_grad 的传播规则	6
☐ 能给出 .grad 是 None 的完整排查清单（至少 5 条）	6
☐ 能说清 retain_graph 什么时候真的需要，以及乱加的后果	6
☐ 能区分 detach / no_grad / inference_mode 三者的语义差别	6
☐ 能写一个 autograd.Function 并用 gradcheck 验证（含为什么必须 float64）	6, 24
☐ 能解释 version counter 是什么、什么样的原地操作会炸	6
☐ 能用 60 秒讲清一次 a + b 从 Python 到 kernel 的分层路径	7
☐ 能说清 dispatcher 的 op × DispatchKey 二维分发，以及 autograd 是一层 key	7

☐ 能算出一个 Transformer 训练的显存构成（参数/梯度/优化器/激活各占多少）	8
☐ 能解释 caching allocator，说清为什么 <code>empty_cache()</code> 通常没用	8
☐ 能区分真 OOM 和碎片，并给出各自的处理手段	8
☐ 能说清 kernel launch 是异步的，并列出至少 4 个隐式同步点	9
☐ 能说清 <code>non_blocking=True</code> 生效的前提（pinned memory）	9
☐ 能判断一个训练是 CPU-bound 还是 GPU-bound，并说出判据	9, 11
☐ 能解释“设了 seed 结果还是不一样”的至少 3 种原因	10
☐ 能说清 TF32 是什么、在 A100 上默认状态如何、什么时候要关	10
☐ 能用 <code>torch.profiler</code> 抓一段 trace 并读出瓶颈在哪	11
☐ 能算 MFU，并说清它为什么是判断训练效率的单一指标	11

Part 3 torch.compile（第 12-16 章）

能力项	章
☐ 能说出 compile 的四层（Dynamo / AOTAutograd / 后端 / Inductor）各自负责什么	12-14
☐ 能解释 Dynamo 怎么接管 Python 执行（字节码层面），以及为什么它比 TorchScript 成功	12
☐ 能解释 guard 是什么，为什么编译产物是特化的	12
☐ 能说清 graph break 是什么、常见触发源、怎么定位（TORCH_LOGS）	12
☐ 能解释 AOTAutograd 为什么要在 dispatcher 层 trace，而不是在 Python 层	13
☐ 能解释 functionalization 解决什么问题	13
☐ 能说清 min-cut partitioner 在权衡什么（显存 vs 重算）	13
☐ 能说清 Inductor 的主要收益来自 fusion，以及为什么 matmul 不靠 fusion	14
☐ 能说清 <code>mode="reduce-overhead"</code> 就是 CUDA graph，以及它的使用前提	15
☐ 能说清动态 shape 的三种处理方式，以及重编译超限的症状	15
☐ 知道 compile 后数值有微小差异是预期行为，能解释为什么	15
☐ 能说清 <code>torch.export</code> 和 <code>torch.compile</code> 的定位差别	16
☐ 能说清 <code>jit.trace</code> 和 <code>jit.script</code> 各自的失败模式，以及为什么新项目不该用	16

Part 4 分布式（第 17-22 章）

能力项	章
☐ 能精确说出 AllReduce / AllGather / ReduceScatter / AllToAll 的语义和通信量	17
☐ 能推 Ring AllReduce 的 $2\frac{N-1}{N} \cdot P $ ，并解释为什么与 N 基本无关	17
☐ 能说清 <code>init_process_group</code> 干了什么、 <code>torchrun</code> 设了哪些环境变量	17
☐ 能说清 collective 的调用契约（所有 rank、相同顺序、相同 shape）	17
☐ 能说清 <code>nn.DataParallel</code> 为什么被废弃	18
☐ 能讲清 DDP 的 bucket + backward hook 机制，以及为什么要分 bucket	18
☐ 能说清 <code>find_unused_parameters=True</code> 的代价和替代方案	18
☐ 能说清 <code>no_sync()</code> 在梯度累积里的作用	18
☐ 能说清 ZeRO 三级各切什么、省多少显存、多付多少通信	19
☐ 能描述 FSDP 一次 forward/backward 的 gather-compute-reshard 流程	19
☐ 能说清 HYBRID_SHARD 的动机，以及 FSDP1 与 FSDP2 的差别	19
☐ 能画出 MLP 的 TP column-then-row 切法，说清为什么这个顺序只需一次 AllReduce	20
☐ 能说清 Sequence Parallel 补了 TP 的哪块短板	20

☐	能算 PP 的 bubble 比例，说清 1F1B 相比 GPipe 省的是显存不是 bubble	20
☐	能说清 DeviceMesh / DTensor 的 placement 与 collective 的对应关系	21
☐	能给出分布式 hang 的完整排查流程（这是最高频的分布式面试题）	22
☐	能说清分片 checkpoint 怎么存、怎么在不同并行度之间 resume	22

Part 5 手写题（第 23-29 章）

能力项	章
☐ 写完任何一个层，能马上说出“我会怎么验证它和官方实现一致”	23
☐ 能说清 LayerNorm 的 eps 在 sqrt 里面还是外面，以及为什么	23
☐ 能说清 BatchNorm 的有偏/无偏方差分别用在哪里（train vs running stats）	23
☐ 能说清 Dropout 为什么在训练时除以 $1 - p$ （inverted dropout）	23
☐ 能说清 autograd.Function 的 save_for_backward 与 ctx.xxx = 的区别	24
☐ 能解释 STE 为什么过不了 gradcheck，而这是设计使然	24
☐ 能默写 LSTM 的四个门公式，说清 PyTorch 的门顺序和双 bias	25
☐ 能说清 GRU 的 reset gate 作用位置与原论文的差别	25
☐ 能默写 SDPA，说清 $\sqrt{d_k}$ 缩放为什么必要	26
☐ 能默写 MHA，并说出 reshape/transpose 顺序写错时为什么不报错却算错	26
☐ 能说清 GQA/MQA 省的是什么（KV cache 显存，不是计算）	26
☐ 能说清 RoPE 的两条核心性质（保范数、内积只依赖相对位置）	26
☐ 能写 KV cache 并说清预填充与解码两阶段的差别	26
☐ 能说清 ViT 的 PatchEmbed 为什么可以用一个 Conv2d(k=P, s=P) 实现	27
☐ 能写 CLIP 的对称 InfoNCE loss，说清随机初始化时 loss 应该约等于 $\ln B$	27
☐ 能写 MoE 的 top-k gating 和 load-balancing aux loss，说清后者的最小值是多少	27
☐ 能写 LoRA，说清 B 为什么零初始化、merge 之后为什么零推理开销	27
☐ 能写 label smoothing CE 并与 F.cross_entropy(label_smoothing=) 对齐	28
☐ 能手写 AdamW 的一步更新（含 bias correction），并说清与 Adam+L2 的差异	28
☐ 能说清 clip_grad_norm_ 是全局范数而不是逐张量，且不会放大梯度	28

按岗位定制复习路线

岗位	重点	可以浅一点
算法 / 模型 (偏训练调优)	Part 1 全部、Part 2 的第 6/8/10 章、Part 5 全部。AMP、梯度累积、显存优化、手写模型是主战场	Part 3 会用 torch.compile 即可 ;Part 4 只需 DDP 的完整机制 + FSDP 的概念（切什么、省多少）
AI Infra / 训练框架	Part 2 全部(dispatcher、显存、CUDA 执行、profiling 是核心)、Part 3 全部、Part 4 全部。Part 5 的第 26 章 Transformer 必须能手写	Part 5 的第 25/27 章（经典模型、扩散/多模态）了解即可
推理 / 部署	Part 1、Part 2 的第 8/9 章（显存与 CUDA 执行）Part 3 全部（尤其第 16 章 export/AOTInductor）第 26 章的 KV cache 与 GQA	Part 4 只需知道 TP 在推理里怎么用；Part 5 的训练技巧类（第 28 章）可跳

应届 / 转行	Part 1 打牢（这是筛人的第一道关）第 6 章 autograd 讲清楚、Part 5 的第 23/25/26 章能手写	Part 3、Part 4 知道名词和一句话定位即可，不要在这上面花时间——面试官不会指望应届生答
---------	--	---

Key insight. 岗位不确定时按这个优先级：第 1 章 → 第 6 章 → 第 5 章 → 第 26 章 → 第 8 章 → 第 18 章。这六章覆盖了 PyTorch 面试里出现频率最高的一批题，不管什么岗位都会问。

时间预算

预算	读什么	跳过什么
1 天	本章最后的“面试前一晚速览”完整过一遍；第 29 章题库快速自测；错的题只翻对应小节，不读整章	所有原理章的推导过程、所有代码。这一档的目标是“不在送分题上翻车”，不是“答出难题”
1 周	第 1、5、6 章精读（约 2 天）；第 8、18、19 章（1 天）；第 12、15 章（1 天）；第 23、26 章边读边写代码（2 天）；最后半天做第 29 章题库	第 7、10、13、14、16、20、21 章；Part 5 的第 25、27 章
1 个月	顺序读完全书。Part 5 的每道题自己敲一遍再跑对应 pytest；Part 2、Part 4 的每章读完合上书复述一遍面试考点框	不跳。有余力的话按前言里的指引去读 <code>books/distributed-training/</code> 或 <code>books/cuda/</code>

手写题清单

难度分三档：**L1** 必须能写（写不出来直接挂）**L2** 应该能写（写不出来要能讲清思路和坑）**L3** 能讲思路即可（面试官通常不会真让你写完）。

代码列给的是 `python/pytorch/interview/` 下的文件名。

题目	难度	章	代码
☐ Linear（含 weight 布局为什么是 <code>(out, in)</code> ）	L1	23	<code>test_layers.py</code>
☐ Softmax（减 max 的数值稳定）	L1	23	<code>test_layers.py</code>
☐ LayerNorm	L1	23	<code>test_layers.py</code>
☐ RMSNorm	L1	23	<code>test_layers.py</code>
☐ BatchNorm1d（train/eval 双路径 + running stats）	L2	23	<code>test_layers.py</code>
☐ Dropout（inverted，eval 恒等）	L1	23	<code>test_layers.py</code>
☐ Embedding（backward 是 scatter-add）	L2	23	<code>test_layers.py</code>
☐ conv2d（im2col / unfold + GEMM）	L3	23	<code>test_layers.py</code>
☐ MaxPool2d（padding 补 <code>-inf</code> 不是 0）	L2	23	<code>test_layers.py</code>
☐ <code>autograd.Function</code> 写 ReLU（三件套模板）	L1	24	<code>test_custom_autograd.py</code>
☐ 手推 Softmax 的 backward（雅可比不用显式建）	L2	24	<code>test_custom_autograd.py</code>
☐ 手推 LayerNorm 的 backward	L3	24	<code>test_custom_autograd.py</code>
☐ STE（量化直通估计）	L2	24	<code>test_custom_autograd.py</code>
☐ GradientReversal（域自适应）	L2	24	<code>test_custom_autograd.py</code>
☐ 用 <code>gradcheck</code> / <code>gradgradcheck</code> 验证自己的实现	L1	24	<code>test_custom_autograd.py</code>

☐	MLP (最后一层不加激活这个送分点)	L1	25	test_classic_models.py
☐	ResNet BasicBlock (相加之后才 ReLU)	L2	25	test_classic_models.py
☐	LSTMCell (门顺序 i-f-g-o、双 bias)	L2	25	test_classic_models.py
☐	多层 LSTM (时间维循环 + 层间堆叠)	L3	25	test_classic_models.py
☐	GRUCell (reset gate 的作用位置)	L2	25	test_classic_models.py
☐	Bahdanau / Luong attention + mask	L2	25	test_classic_models.py
☐	scaled dot-product attention	L1	26	test_transformer.py
☐	MultiHeadAttention (reshape/transpose 顺序)	L1	26	test_transformer.py
☐	causal mask + padding mask 的组合	L1	26	test_transformer.py
☐	GroupedQueryAttention / MQA	L2	26	test_transformer.py
☐	RoPE (cache 构造 + apply)	L2	26	test_transformer.py
☐	KV cache (预分配 buffer + 游标)	L2	26	test_transformer.py
☐	TransformerBlock (pre-norm + RMSNorm + SwiGLU)	L2	26	test_transformer.py
☐	MiniGPT + generate (temperature / top-k / top-p)	L3	26	test_transformer.py
☐	ViT 的 PatchEmbed + cls token + pos embed	L2	27	test_advanced_models.py
☐	CLIP 对称 InfoNCE loss	L2	27	test_advanced_models.py
☐	MoE top-k gating + load-balancing aux loss	L2	27	test_advanced_models.py
☐	DDPM 的 q_{sample} 闭式解 + 噪声预测 loss	L3	27	test_advanced_models.py
☐	DDPM 反向采样循环	L3	27	test_advanced_models.py
☐	LoRALinear (B 零初始化 + merge)	L1	27	test_advanced_models.py
☐	label smoothing cross entropy	L2	28	test_training_tricks.py
☐	focal loss (gamma=0 退化为 CE)	L2	28	test_training_tricks.py
☐	mixup / cutmix	L1	28	test_training_tricks.py
☐	ModelEMA (buffer 是拷贝不是平均)	L2	28	test_training_tricks.py
☐	warmup + cosine scheduler	L1	28	test_training_tricks.py
☐	SGD with momentum / Nesterov	L2	28	test_training_tricks.py
☐	AdamW (含 bias correction)	L2	28	test_training_tricks.py
☐	clip_grad_norm_ (全局范数, 不放大)	L1	28	test_training_tricks.py

L1 一共 15 题。这 15 题必须全部能在白板上一次写对，这是硬门槛；L2 有 22 题，是拉开差距的部分；L3 的 6 题只要能讲清思路。

面试前一晚：一页速览

1. **view vs reshape** : `view` 只改元数据、要求兼容 stride ; `reshape` 能 `view` 就 `view` , 不能就 `copy`。transpose/切片之后 `view` 会失败。
2. **zero_grad(set_to_none=True)** (2.0+ 默认): 把 `.grad` 置 `None` 而不是填 0 , 省一次 `memset` 和一份显存 ; 副作用是没参与 forward 的参数梯度是 `None` 而不是 0。
3. **AMP** 三件套顺序 : `autocast` 里只放 `forward` → `scaler.scale(loss).backward()` → `scaler.unscale_(opt)` → `clip_grad_norm_` → `scaler.step(opt)` → `scaler.update()`。裁剪必须在 `unscale` 之后 , 否则裁的是放大过的梯度。bf16 不需要 `GradScaler`。
4. **.grad 是 None** : 非叶子 / `requires_grad=False` / 没参与 forward / 路径上有 `detach` / 在 `backward` 之前看的。

5. **retain_graph** : 报错让你加 , 但九成情况是你把带图的张量跨 step 累积了 (`total_loss += loss`)
6. 显存构成 (fp32 参数量 P + Adam): 参数 $4P$ + 梯度 $4P$ + 一阶二阶动量 $8P = 16P$, 激活另算且通常是大头。混合精度下另加 fp32 master weights。
7. **caching allocator** : `empty_cache()` 只还 reserved 不减 allocated , 对真 OOM 无用。reserved 远大于 allocated 才是碎片。
8. **kernel launch** 是异步的 : 计时必须 `torch.cuda.synchronize()` 或用 CUDA event。 `.item()` / `print(tensor)` / `.cpu()` 都是隐式同步点。
9. **DDP bucket** : 梯度按 bucket(默认 25 MB) 分组 , 一个 bucket 的梯度算完就发 AllReduce , 实现通信与 backward 计算 overlap。不分 bucket 就只能等全部 backward 结束才通信。
10. **ZeRO** 三级 : ZeRO-1 切优化器状态、ZeRO-2 加切梯度、ZeRO-3 加切参数。ZeRO-3 = FSDP , 通信量是 DDP 的 1.5 倍 (多一次参数 AllGather)
11. **torch.compile** 四层 : Dynamo 抓字节码出 FX graph (会 graph break) → AOTAutograd 出前反向联合图 → 后端 → Inductor 生成 Triton kernel。主要收益来自 fusion 省 HBM 往返和 kernel launch。
12. **attention** 缩放 : $\text{softmax}(Q K^T / \sqrt{d_k}) V$ 。不除 $\sqrt{d_k}$ 的话内积方差随 d_k 线性增长 , softmax 饱和、梯度消失。
13. **MHA** 的 **reshape**
顺序 : (B,S,H) -> view(B,S,nh,hd) -> transpose(1,2) -> (B,nh,S,hd)。先 transpose 再 view 是经典 bug , 不报错但算错。
14. **KV cache** : 解码时只有新 token 的 Q, K/V 从 cache 取 , 把每步的 $O(S^2)$ 降到 $O(S)$ 。显存代价是 $2 \cdot L \cdot S \cdot H \cdot \text{dtype 字节}$, GQA/MQA 就是来砍这一项的。
15. **AdamW vs Adam+L2** : Adam+L2 把 weight decay 加进梯度 , 会被自适应学习率的分母除掉 , 实际衰减强度和梯度大小耦合 ; AdamW 把它解耦 , 直接从权重里减 $\text{lr} \times \text{wd} \times w$ 。
16. **BatchNorm vs LayerNorm** : BN 沿 batch 维统计 , 依赖 batch size , train/eval 行为不同 , 分布式下要 SyncBN ; LN 沿特征维统计 , 与 batch 无关 , 所以 Transformer 用它。
17. **pre-norm vs post-norm** : pre-norm 的残差路径是纯恒等 , 深层训练稳定、通常不需要 warmup ; post-norm 效果上限略高但难训。现代 LLM 一律 pre-norm。
18. 梯度累积等价性 : loss 要除以累积步数才等价于大 batch。BatchNorm 是例外 (统计量只在 micro-batch 内算)。DDP 下前 $k-1$ 步要用 `no_sync()`。
19. **graph break** : `print`、`.item()`、依赖张量值的 `if` 都会切图。`fullgraph=True` 把它变成报错 , `TORCH_LOGS="graph_breaks"` 定位。
20. **TF32** : A100 上 fp32 matmul 可以走 TF32 tensor core(10 位尾数) , 快很多、精度略降。torch 2.10 上 `backends.cuda.matmul.allow_tf32` 默认 `False`、`backends.cudnn.allow_tf32` 默认 `True` ——两个开关默认值不一样 , 这是个爱考细节。

最后一条不写在纸上但比上面所有都重要 : 不会的题按“划边界 → 归约到会的 → 说怎么验证”三步答 , 别硬编。见前言最后一节。

附录 C：PyTorch 2.x 版本差异

“你用的什么版本？2.x 有什么变化？”——这题几乎每场面试都会问，而且是个陷阱：背版本号没用，面试官想听的是你知不知道 **PyTorch** 这几年往哪个方向走。答案只有两条主线：编译栈（`torch.compile` 那一整套）和 **DTensor**（分布式的统一底座）。其余都是这两条线上的枝节。

本书的环境是 `torch 2.10.0+cu128 / CUDA 12.8`。

Warning. 下面表格里的 minor 版本号，只有两条有硬证据：`torch.compile` 是 2.0 的发布主题，`torch.load` 的 `weights_only` 默认值变更由报错文本自己写明“In PyTorch 2.6, we changed the default value”。其余各行是按发布节奏归置的，可能差一个小版本。面试里报“2.x 中期”这种粒度就够，不要为了显得精确去报一个记不准的号——报错了反而扣分。

1.x → 2.0：唯一的分水岭

PyTorch 1.x 时代的图捕获方案是 **TorchScript**（`torch.jit.trace / torch.jit.script`），它的思路是让 Python 子集变成静态语言：`script` 直接解析 Python AST，遇到动态特性就报错；`trace` 只记录一次执行的算子序列，控制流和 `shape` 全被固化。结果是“能跑通的模型要重写一遍”，用户体验差到大部分人宁可不用。

2.0 换了思路：不去理解 **Python**，而是在字节码层面拦截。Dynamo 从 CPython 的 `frame evaluation` 钩子进去，把能捕获的部分抽成 FX graph，捕获不了的地方直接 **graph break**、回退 Python 执行。代价是图可能被切碎，好处是任何 **Python** 代码都能跑，只是收益不同。这是 `torch.compile` 成功而 TorchScript 失败的根本原因（第 12 章）。

同一时期 TorchScript 进入维护模式，到本书环境的 2.10，`torch.jit.script` 已经会打出显式弃用警告：

```
UserWarning: `torch.jit.script` is deprecated.
Please switch to `torch.compile` or `torch.export`.
```

逐版本要点

版本	关键特性	对你写代码的影响
2.0	<code>torch.compile</code> 发布（Dynamo + AOTAutograd + Inductor 三层）； <code>F.scaled_dot_product_attention</code> 进入 core；TorchScript 转维护模式	训练脚本加一行 <code>model = torch.compile(model)</code> 就能拿收益；手写 attention 一律改调 SDPA，它会自动选 FlashAttention / memory-efficient / math 后端
2.1	<code>torch.export</code> 预览；DTensor 原型；automatic dynamic shapes（第一次遇到 <code>shape</code> 变化就自动重编成动态版本）	导出路线开始从 TorchScript 迁到 export；变长输入的重编译问题被缓解，但仍需手动 <code>dynamic=True</code> 兜底
2.2	SDPA 加入 FlashAttention-2 后端； <code>DeviceMesh</code> 进入 beta；functional collectives（可被 compile 追踪的集合通信）	长序列 attention 明显变快且不用自己装 flash-attn 包；分布式代码可以开始用 <code>mesh</code> 描述拓扑，不再手工 <code>new_group</code>

2.3	Tensor Parallel 高层 API (<code>torch.distributed.ColwiseParallel</code> / <code>RowwiseParallel</code>); 用户自定义 Triton kernel 可被 <code>torch.compile</code> 追踪	TP 从“照抄 Megatron 的手工切分”变成“给每层挂一个 parallel style”; 自己写的 Triton kernel 不再是 graph break
2.4	FSDP2 (<code>fully_shard</code> , 基于 <code>DTensor</code> 重写); <code>torch.distributed.pipelining</code> ; TCPStore 换 libuv 后端	FSDP 的 <code>FlatParameter</code> 黑盒变成一等公民的 <code>DTensor</code> 参数, 能逐参数控制切分、和 TP 组合更干净 (第 19、21 章)
2.5	FlexAttention (用 Python 写 score modifier, 编译成融合 kernel); SDPA 的 cuDNN 后端; regional compilation (只编译重复的子模块以省编译时间)	ALiBi / sliding window / soft-cap 这类 attention 变体不用再手写 kernel; 大模型的编译时间可以从分钟级压下来
2.6	<code>torch.load</code> 的 <code>weights_only</code> 默认值从 <code>False</code> 改为 <code>True</code> ; <code>torch.compiler.set_stance</code> ; AOTInductor 打包能力	这是最容易踩的一次 break change, 见下一节。 <code>set_stance</code> 让你能在运行时切换“强制编译 / 强制 eager”, 调试和 A/B 很方便
2.7-2.8	Blackwell 与 CUDA 12.8 支持; 编译缓存与编译时间的持续优化; 量化生态 (<code>torchao</code>) 成熟	新卡上不用再自己编 torch; 分布式训练的编译缓存能跨 rank 复用, 冷启动变快
2.9-2.10 (本书环境)	<code>DTensor</code> 与 FSDP2 的公开路径稳定为 <code>torch.distributed.tensor</code> 和 <code>torch.distributed.fsdp.fully_shard</code> ; <code>torch.jit.script</code> 打出显式弃用警告; 引入统一的 <code>fp32_precision</code> 精度控制 API	新项目直接写公开路径, 不要再 <code>import _composable / _tensor</code> 这类私有模块; 精度开关从一堆 <code>allow_tf32</code> 布尔量往 <code>fp32_precision</code> 字符串收敛

Key insight. 把这张表压成一句话: **2.0-2.2** 是编译栈从能用到好用, **2.3-2.5** 是分布式从手工切分转到 **DTensor** 抽象, **2.6** 之后是工程收尾 (默认值、缓存、新硬件)。面试里这一句比背十个版本号有用。

那些“默认值变了”的坑

这类最容易踩, 因为代码没改、行为变了。

`torch.load` 的 `weights_only`

2.6 起默认从 `False` 变成 `True`。反序列化时不再执行任意 pickle 代码, 只允许张量 and 一小撮白名单类型。加载老 checkpoint (里面存了 `argparse.Namespace`、自定义 config 对象、`numpy` 标量) 会直接失败:

```
_pickle.UnpicklingError: Weights only load failed. ... (1) In PyTorch 2.6, we
changed the default value of the `weights_only` argument in `torch.load` from
`False` to `True`. ... WeightsUnpickler error: Unsupported global: GLOBAL
__main__.Foo was not an allowed global by default.
```

处理方式按信任程度分两档:


```
torch.load(path, weights_only=False) # 只对自己产出的文件
torch.serialization.add_safe_globals([MyConfig]) # 白名单指定类型
```

真正的根治是别往 **checkpoint** 里塞对象：`state_dict` 存张量，超参存 JSON。

zero_grad(set_to_none)

2.0 起默认 `True`（此前默认把梯度填 0）。省一次 `memset`、省一份显存，代价是行为差异：没参与 forward 的参数梯度是 `None` 而不是 0。自己遍历 `p.grad` 做统计的代码要加 `if p.grad is not None`。

checkpoint 的 use_reentrant

`torch.utils.checkpoint.checkpoint` 有两套实现。老的 `reentrant` 版本靠 `autograd.Function` 重入实现，限制多（不支持 `grad(inputs=)`、输出必须有 `requires_grad`、不能有多次 `backward`）；新的非重入版本用 `saved-tensor hook`，没这些限制。现在不传这个参数会警告：

```
UserWarning: torch.utils.checkpoint: the use_reentrant parameter should be
passed explicitly. Starting in PyTorch 2.9, calling checkpoint without
use_reentrant will raise an exception. use_reentrant=False is recommended ...
```

Note. 警告文本里说 2.9 起会抛异常，但在 2.10 上实测仍然只是警告——官方的弃用节奏比警告文本慢。不管哪个版本，一律显式传 `use_reentrant=False`。

TF32 开关

TF32 是 Ampere 引入的 19 位格式（8 位指数 + 10 位尾数），fp32 的 `matmul` 可以走 `tensor core`，快很多、精度降到约 fp16 的尾数水平。历史上这个默认值反复变过：早期 Ampere 上 `matmul` 默认开 TF32，后来因为“用户不知情地损失精度”改成默认关；`cuDNN` 那一路一直是开的。

torch 2.10 上实测的默认值：

```
torch.backends.cuda.matmul.allow_tf32 # False
torch.backends.cudnn.allow_tf32      # True
```

两个开关默认值不一样，这是个爱考的细节。想让 fp32 `matmul` 提速就得显式打开 `torch.backends.cuda.matmul.allow_tf32 = True`；反过来，追求数值可复现（比如比对两个实现是否等价）就得把 `cudnn` 那一路也关掉。较新版本引入了统一的 `fp32_precision` 字符串 API，同一台机器上 `torch.backends.cuda.matmul.fp32_precision` 是 `"none"`、`torch.backends.cudnn.conv.fp32_precision` 是 `"tf32"`，和上面两个布尔量是一致的两种表达。详见第 10 章。

怎么自己确认一个默认值

不要凭记忆，也不要问模型，当场查最快：

```
import inspect, torch
inspect.signature(torch.optim.SGD.zero_grad) # (self, set_to_none: bool =
True)
inspect.signature(torch.load).parameters["weights_only"]
torch.backends.cuda.matmul.allow_tf32 # 布尔量直接打印
```

签名里写死的默认值一目了然；签名是 `None` 的（比如 `torch.load` 的 `weights_only` 和 `checkpoint` 的 `use_reentrant`）说明真实默认值藏在函数体里，得看行为——最省事的办法是故意触发一次，读报错或警告文本，PyTorch 的这类消息通常会把版本和迁移方式一起写清楚。本章的几条硬证据就是这么来的。

已废弃 / 不推荐清单

东西	状态	用什么替代
<code>nn.DataParallel</code>	没有运行时警告，但文档明确推荐改用 DDP。单进程多线程，被 GIL 卡住、主卡显存不均	<code>DistributedDataParallel</code> + <code>torchrun</code> （第 18 章）
<code>torch.jit.script</code> / <code>torch.jit.trace</code>	2.10 上打出显式弃用警告，TorchScript 已是维护模式	训练加速用 <code>torch.compile</code> ，导出部署用 <code>torch.export</code> + <code>AOTInductor</code> （第 16 章）
<code>torch.autograd.Variable</code>	1.0 起就是 <code>Tensor</code> 的别名，纯历史遗留	直接用 <code>Tensor</code> 和 <code>requires_grad=</code>
<code>.data</code>	能用但绕过 <code>version counter</code> ，会让“原地修改”类 bug 变成静默错误	读值用 <code>.detach()</code> ，改值用 <code>with torch.no_grad():</code> （第 1、6 章）
<code>torch.cuda.amp.autocast</code> <code>torch.cuda.amp.GradScaler</code>	有明确的弃用警告： <code>torch.cuda.amp.autocast</code> is deprecated	<code>torch.autocast("cuda", dtype=...)</code> <code>torch.amp.GradScaler("cuda")</code> （第 5 章）
FSDP1（ <code>FullyShardedDataParallel</code> ）	仍可用，但新功能都在 FSDP2 上做	<code>torch.distributed.fsdp.fully_sharded_data_parallel</code> （第 19 章）
手工 <code>dist.new_group</code> 拼多维并行	不算废弃，但容易把 rank 算错	<code>DeviceMesh</code> + <code>DTensor placement</code> （第 21 章）

生态：各个 `torch*` 包是干什么的

面试官问“你了解 PyTorch 生态吗”，想听的是你能一句话说清每个包的定位：

- **torchao**：量化与稀疏。训练后量化、QAT、int8/fp8 训练、低比特优化器，是官方推的量化路线。
- **torchtitan**：大模型预训练的参考实现。用来看 FSDP2 + TP + PP + CP 怎么正确组合，而不是当框架用。
- **torchtune**：微调库。LoRA / QLoRA / 全参微调的配方式实现。
- **DTensor**：不是一个包，是 `torch.distributed` 里的核心抽象。FSDP2、TP、分布式 checkpoint 现在都建在它上面——这是趋势本身，其他几个包只是它的应用。

一句话概括方向：**PyTorch** 正在把“并行”从一堆互不相通的 **wrapper**（**DDP** / **FSDP** / **Megatron** 式 **TP**）收敛成“在 **DeviceMesh** 上给张量标 **placement**”这一个抽象。

面试怎么答版本问题

不要背版本号。三句话的模板：

1. 说环境：“我平时用 2.x，具体到项目上是 2.10 + CUDA 12.8。”
2. 说主线：“2.x 相比 1.x 的关键变化就两条：一条是编译栈，`torch.compile` 把 Dynamo 抓图 + Inductor codegen 这套做成了默认加速路径，TorchScript 退出历史舞台；另一条是分布式，从 DDP/FSDP1 这种整体 wrapper，转向 DeviceMesh + DTensor 这个统一抽象，FSDP2 和 TP 现在都建在上面。”
3. 展开你真用过的：“我实际用得比较深的是 `torch.compile` 这条线，踩过重编译超限静默 fallback 的坑，后来是靠 `TORCH_LOGS=recompiles` 定位到输入 shape 一直变，加 `drop_last` 和 `dynamic=True` 解决的。”

第三句是分水岭。前两句谁都能背，第三句有没有具体的坑和具体的定位手段，面试官一听就知道你是不是真用过。

Warning. 被追问“这个特性是哪个版本引入的”而你记不准时，说“记不清具体是 2.3 还是 2.4，我印象里是 TP 高层 API 那一批一起进来的”——这比报一个错误的号强得多。报错一个版本号，面试官会开始怀疑你前面说的其他数字。